

ClanTek Project Source Listing

Generated from clantek

Included files: 155

This document contains the source code and project files used for copyright submission.

Files included

.claude/launch.json
.github/workflows/deploy.yml
docs/api.md
drizzle.config.ts
drizzle/0000_cold_khan.sql
drizzle/0001_slim_lady_deathstrike.sql
drizzle/0002_melted_lilith.sql
drizzle/0003_odd_jasper_sitwell.sql
drizzle/0004_lazy_blur.sql
drizzle/0005_colossal_warbird.sql
drizzle/0006_youthful_power_pack.sql
drizzle/0007_dizzy_champions.sql
drizzle/0008_brown_silhouette.sql
drizzle/0009_groovy_tattoo.sql
drizzle/0010_faulty_puppet_master.sql
drizzle/0011_magical_iron_fist.sql
drizzle/0012_flawless_obadiah_stane.sql
drizzle/0013_nice_slapstick.sql
drizzle/0014_complex_titanium_man.sql
drizzle/0015_naive_giant_man.sql
drizzle/0016_volatile_red_hulk.sql
drizzle/0017_mute_dreaming_celestial.sql
drizzle/meta/_journal.json
drizzle/meta/0000_snapshot.json
drizzle/meta/0001_snapshot.json
drizzle/meta/0002_snapshot.json
drizzle/meta/0003_snapshot.json
drizzle/meta/0004_snapshot.json
drizzle/meta/0005_snapshot.json
drizzle/meta/0006_snapshot.json
drizzle/meta/0007_snapshot.json
drizzle/meta/0008_snapshot.json
drizzle/meta/0009_snapshot.json
drizzle/meta/0010_snapshot.json
drizzle/meta/0011_snapshot.json
drizzle/meta/0012_snapshot.json
drizzle/meta/0013_snapshot.json
drizzle/meta/0014_snapshot.json
drizzle/meta/0015_snapshot.json
drizzle/meta/0016_snapshot.json
drizzle/meta/0017_snapshot.json
package-lock.json
package.json
README.md
scripts/doctor.ts
scripts/generate-copyright-pdf-v2.js
scripts/generate-copyright-pdf.js
scripts/register-commands.ts
src/client/App.tsx

.claude/launch.json

```
{
  "version": "0.0.1",
  "configurations": [
    {
      "name": "clantek-dev",
      "runtimeExecutable": "npm",
      "runtimeArgs": ["run", "dev"],
      "port": 5173
    },
    {
      "name": "clantek-worker",
      "runtimeExecutable": "npx",
      "runtimeArgs": ["wrangler", "dev", "--port", "8787"],
      "port": 8787
    }
  ]
}
```

.github/workflows/deploy.yml

name: Deploy

```

on:
  push:
    branches: [main]
  workflow_dispatch:

jobs:
  # Always runs: proves the tree typechecks and builds on every push,
  # independent of whether deploy credentials are configured.
  build:
    runs-on: ubuntu-latest
    permissions:
      contents: read
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - name: Typecheck
        run: npm run typecheck
      - name: Build
        run: npm run build

  # Deploys only when the Cloudflare token is present. Until it is, this job
  # skips cleanly (a neutral result, not a failure) so ordinary pushes stay
  # green. Add CLOUDFLARE_API_TOKEN in repo Settings -> Secrets and variables ->
  # Actions to switch it on; CLOUDFLARE_ACCOUNT_ID is already set.
  deploy:
    needs: build
    runs-on: ubuntu-latest
    permissions:
      contents: read
    steps:
      - name: Check for deploy credentials
        id: creds
        env:
          CLOUDFLARE_API_TOKEN: ${ secrets.CLOUDFLARE_API_TOKEN }
        run: |
          if [ -n "$CLOUDFLARE_API_TOKEN" ]; then
            echo "ready=true" >> "$GITHUB_OUTPUT"
          else
            echo "ready=false" >> "$GITHUB_OUTPUT"
            echo "::notice::CLOUDFLARE_API_TOKEN not set ? skipping deploy. The site is deployed
manually with 'npm run deploy' until this is configured."
          fi

      - if: steps.creds.outputs.ready == 'true'
        uses: actions/checkout@v4
      - if: steps.creds.outputs.ready == 'true'
        uses: actions/setup-node@v4
        with:

```

.github/workflows/deploy.yml

```
  node-version: 22
  cache: npm
- if: steps.creds.outputs.ready == 'true'
  run: npm ci
- if: steps.creds.outputs.ready == 'true'
  run: npm run build

- name: Apply database migrations
  if: steps.creds.outputs.ready == 'true'
  uses: cloudflare/wrangler-action@v3
  with:
    apiToken: ${ secrets.CLOUDFLARE_API_TOKEN }
    accountId: ${ secrets.CLOUDFLARE_ACCOUNT_ID }
    command: d1 migrations apply clantek --remote

- name: Deploy Worker
  if: steps.creds.outputs.ready == 'true'
  uses: cloudflare/wrangler-action@v3
  with:
    apiToken: ${ secrets.CLOUDFLARE_API_TOKEN }
    accountId: ${ secrets.CLOUDFLARE_ACCOUNT_ID }
    command: deploy
```

docs/api.md

ClanTek API

The same JSON API that powers the web app also powers native/mobile clients. Every endpoint lives under `/api` on the site's origin (`https://mustr.gg`) and returns JSON.

- **Base URL:** `https://mustr.gg/api`
- **Content type:** `application/json`
- **Contract version:** `GET /api/version` -> `{ "apiVersion": 1, ... }`. `apiVersion` is bumped only on a breaking change to the shapes documented here.

Authentication

Two credential kinds resolve to the exact same identity and permissions:

Credential	How it's sent	Who uses it
Web session	<code>ct_session</code> httpOnly cookie	the browser app
Personal access token	<code>Authorization: Bearer clt_...</code>	native/mobile apps, scripts

A native app authenticates with a **personal access token**. The member creates one in the web app under **Account -> API access** (top-right menu -> Settings). The raw token (`clt_...`) is shown **once**, on creation ? store it securely; only its name and a short prefix are visible afterward. A token:

- carries exactly that member's permissions (it is not scoped down),
- expires one year after creation,
- can be revoked any time from the same screen (a lost device -> revoke).

```

```http
GET /api/me HTTP/1.1
Host: mustr.gg
Authorization: Bearer clt_ab12cd...your...token
```

```

There is no login-with-token endpoint by design: tokens are minted by a signed-in member in the browser, not exchanged from credentials. (A future release may add a Discord-OAuth deep-link handoff so the app can mint one without copy/paste.)

Errors

Failures return an HTTP status and `{ "error": "message" }`. Common cases:

- `401 { "error": "Authentication required" }` ? missing/invalid/expired token.
- `403 { "error": "Forbidden", "missing": "<permission>" }` ? authenticated but lacking the required permission.

Identity

`GET /api/me`

Who the token belongs to. `authKind` is `"token"` for bearer auth, `"web"` for a cookie, or `null` if unauthenticated.

docs/api.md

```
```json
{
 "viewer": {
 "id": 12,
 "discordId": "...",
 "username": "...",
 "displayName": "...",
 "avatar": "...",
 "isGod": false,
 "rank": { "id": 3, "name": "Sergeant", "sortOrder": 30 },
 "roles": [{ "id": 2, "name": "Member", "color": "#c0392b" }],
 "permissions": ["roster.view", "events.view"]
 },
 "siteName": "ClanTek",
 "authKind": "token"
}
```
```

The ``permissions`` array is the union across the member's roles; ``isGod: true`` bypasses all permission checks. Gate app UI on ``permissions``, exactly as the web app does.

Token management

All under ``/api/auth/tokens``. ****Minting and revoking require a web session**** (so a leaked token can't spawn or revoke siblings); listing works with either.

| Method | Path | Notes |
|-----------------------|-------------------------------------|---|
| --- | --- | --- |
| <code>`GET`</code> | <code>`/api/auth/tokens`</code> | The caller's tokens (never the secret). |
| <code>`POST`</code> | <code>`/api/auth/tokens`</code> | Body <code>`{ "label": "My iPhone" }`</code> . Returns the raw <code>`token`</code> **once** . Web session only. |
| <code>`DELETE`</code> | <code>`/api/auth/tokens/:id`</code> | Revoke one of your own. Web session only. |

Content endpoints

Reads are available to any authenticated member unless a permission is noted. Shapes below are the fields a client relies on (others may be present).

| Endpoint | Returns |
|--|---|
| --- | --- |
| <code>`GET /api/version`</code> | <code>`{ apiVersion, service, siteName }`</code> ? public. |
| <code>`GET /api/members?limit=&offset=`</code> | <code>`{ members: Member[], total }`</code> |
| <code>`GET /api/members/:id`</code> | a single member |
| <code>`GET /api/news`</code> | <code>`{ posts: NewsPost[] }`</code> |
| <code>`GET /api/news/:slug`</code> | a single post |
| <code>`GET /api/events`</code> | <code>`{ events: Event[] }`</code> (requires <code>`events.view`</code>) |
| <code>`GET /api/medals`</code> | <code>`{ medals: [...] }`</code> |
| <code>`GET /api/warrecords`</code> | <code>`{ warRecords: [...] }`</code> |
| <code>`GET /api/games`</code> | <code>`{ games: [...] }`</code> |
| <code>`GET /api/pages/nav`</code> | custom pages in the top nav: <code>`{ pages: [{ slug, title }] }`</code> |
| <code>`GET /api/pages/:slug`</code> | a page's portable layout (see below) |

Media referenced by these (``/media/...``) is served from the same origin.

docs/api.md

Pages ? portable layout JSON

`GET /api/pages/:slug` (e.g. `home`, or a custom page's slug) returns a layout a native client can render with the same data the web app uses:

```
```json
{
 "slug": "home",
 "title": "Home",
 "exists": true,
 "layout": {
 "version": 1,
 "rows": [
 {
 "id": "r-home",
 "columns": [
 {
 "id": "c-left",
 "span": 5,
 "modules": [
 { "id": "m-roster", "type": "roster", "config": { "title": "Roster", "limit": 12 } }
]
 }
]
 }
]
 }
}
```

- A row is a set of **columns** on a 12-unit grid (`span` = units, desktop); a native client collapses columns to a single stack on a narrow screen.
- Each column holds ordered **modules**. `type` is one of: `heading`, `text`, `html`, `image`, `button`, `embed`, `divider`, `news`, `roster`, `events`, `medals`, `warrecords`, `games`. Data modules (news/roster/...) fetch their own data from the endpoints above; content modules render from `config`.
- A module may carry `visibleToRole` (a role id) ? render it only if the viewer holds that role. The canonical type definitions live in `src/shared/layout.ts`.

### ## Versioning & stability

Treat `apiVersion` as the contract. Additive changes (new fields, new module types, new endpoints) will **not** bump it; a client should ignore unknown fields and module types it doesn't understand. A breaking change bumps `apiVersion` and is announced here.



**drizzle.config.ts**

```
import { defineConfig } from 'drizzle-kit';

export default defineConfig({
 schema: './src/db/schema.ts',
 out: './drizzle',
 dialect: 'sqlite',
 driver: 'd1-http',
 // `npm run db:generate` only needs the schema; applying migrations goes
 // through wrangler (see db:migrate:* scripts), which handles D1 auth.
 verbose: true,
 strict: true,
});
```

**drizzle/0000\_cold\_khan.sql**

```
CREATE TABLE `audit_log` (
 `id` integer PRIMARY KEY AUTOINCREMENT NOT NULL,
 `actor_id` integer,
 `action` text NOT NULL,
 `target_type` text,
 `target_id` text,
 `meta` text,
 `source` text DEFAULT 'web' NOT NULL,
 `ip` text,
 `created_at` integer DEFAULT (unixepoch()) NOT NULL,
 FOREIGN KEY (`actor_id`) REFERENCES `users`(`id`) ON UPDATE no action ON DELETE set null
);
--> statement-breakpoint
CREATE INDEX `audit_actor_idx` ON `audit_log` (`actor_id`);--> statement-breakpoint
CREATE INDEX `audit_created_idx` ON `audit_log` (`created_at`);--> statement-breakpoint
CREATE INDEX `audit_action_idx` ON `audit_log` (`action`);--> statement-breakpoint
CREATE TABLE `games` (
 `id` integer PRIMARY KEY AUTOINCREMENT NOT NULL,
 `name` text NOT NULL,
 `slug` text NOT NULL,
 `icon_url` text,
 `sort_order` integer DEFAULT 0 NOT NULL,
 `active` integer DEFAULT true NOT NULL,
 `created_at` integer DEFAULT (unixepoch()) NOT NULL
);
--> statement-breakpoint
CREATE UNIQUE INDEX `games_slug_unique` ON `games` (`slug`);--> statement-breakpoint
CREATE TABLE `match_participants` (
 `match_id` integer NOT NULL,
 `user_id` integer NOT NULL,
 `stats` text,
 PRIMARY KEY(`match_id`, `user_id`),
 FOREIGN KEY (`match_id`) REFERENCES `matches`(`id`) ON UPDATE no action ON DELETE cascade,
 FOREIGN KEY (`user_id`) REFERENCES `users`(`id`) ON UPDATE no action ON DELETE cascade
);
--> statement-breakpoint
CREATE INDEX `mp_user_idx` ON `match_participants` (`user_id`);--> statement-breakpoint
CREATE TABLE `matches` (
 `id` integer PRIMARY KEY AUTOINCREMENT NOT NULL,
 `game_id` integer NOT NULL,
 `opponent` text NOT NULL,
 `opponent_tag` text,
 `result` text NOT NULL,
 `score_us` integer,
 `score_them` integer,
 `map` text,
 `notes` text,
 `played_at` integer NOT NULL,
 `recorded_by` integer,
 `created_at` integer DEFAULT (unixepoch()) NOT NULL,
 FOREIGN KEY (`game_id`) REFERENCES `games`(`id`) ON UPDATE no action ON DELETE cascade,
 FOREIGN KEY (`recorded_by`) REFERENCES `users`(`id`) ON UPDATE no action ON DELETE set null
);
--> statement-breakpoint
```

**drizzle/0000\_cold\_khan.sql**

```

CREATE INDEX `matches_game_played_idx` ON `matches` (`game_id`,`played_at`);-->
statement-breakpoint
CREATE TABLE `medals` (
 `id` integer PRIMARY KEY AUTOINCREMENT NOT NULL,
 `name` text NOT NULL,
 `description` text,
 `image_url` text,
 `game_id` integer,
 `sort_order` integer DEFAULT 0 NOT NULL,
 `created_at` integer DEFAULT (unixepoch()) NOT NULL,
 FOREIGN KEY (`game_id`) REFERENCES `games`(`id`) ON UPDATE no action ON DELETE cascade
);
--> statement-breakpoint
CREATE INDEX `medals_game_idx` ON `medals` (`game_id`);--> statement-breakpoint
CREATE TABLE `member_medals` (
 `id` integer PRIMARY KEY AUTOINCREMENT NOT NULL,
 `user_id` integer NOT NULL,
 `medal_id` integer NOT NULL,
 `citation` text,
 `awarded_by` integer,
 `awarded_at` integer DEFAULT (unixepoch()) NOT NULL,
 FOREIGN KEY (`user_id`) REFERENCES `users`(`id`) ON UPDATE no action ON DELETE cascade,
 FOREIGN KEY (`medal_id`) REFERENCES `medals`(`id`) ON UPDATE no action ON DELETE cascade,
 FOREIGN KEY (`awarded_by`) REFERENCES `users`(`id`) ON UPDATE no action ON DELETE set null
);
--> statement-breakpoint
CREATE INDEX `member_medals_user_idx` ON `member_medals` (`user_id`);--> statement-breakpoint
CREATE INDEX `member_medals_medal_idx` ON `member_medals` (`medal_id`);--> statement-breakpoint
CREATE TABLE `news` (
 `id` integer PRIMARY KEY AUTOINCREMENT NOT NULL,
 `title` text NOT NULL,
 `slug` text NOT NULL,
 `excerpt` text,
 `body` text NOT NULL,
 `author_id` integer,
 `status` text DEFAULT 'draft' NOT NULL,
 `pinned` integer DEFAULT false NOT NULL,
 `published_at` integer,
 `created_at` integer DEFAULT (unixepoch()) NOT NULL,
 `updated_at` integer DEFAULT (unixepoch()) NOT NULL,
 FOREIGN KEY (`author_id`) REFERENCES `users`(`id`) ON UPDATE no action ON DELETE set null
);
--> statement-breakpoint
CREATE UNIQUE INDEX `news_slug_unique` ON `news` (`slug`);--> statement-breakpoint
CREATE INDEX `news_status_published_idx` ON `news` (`status`,`published_at`);-->
statement-breakpoint
CREATE TABLE `profiles` (
 `user_id` integer PRIMARY KEY NOT NULL,
 `bio` text,
 `location` text,
 `timezone` text,
 `gamertags` text,
 `updated_at` integer DEFAULT (unixepoch()) NOT NULL,
 FOREIGN KEY (`user_id`) REFERENCES `users`(`id`) ON UPDATE no action ON DELETE cascade

```

**drizzle/0000\_cold\_khan.sql**

```
);
--> statement-breakpoint
CREATE TABLE `ranks` (
 `id` integer PRIMARY KEY AUTOINCREMENT NOT NULL,
 `name` text NOT NULL,
 `abbreviation` text,
 `image_url` text,
 `sort_order` integer NOT NULL,
 `req_days` integer DEFAULT 0 NOT NULL,
 `req_wins` integer DEFAULT 0 NOT NULL,
 `is_default` integer DEFAULT false NOT NULL,
 `created_at` integer DEFAULT (unixepoch()) NOT NULL,
 `updated_at` integer DEFAULT (unixepoch()) NOT NULL
);
--> statement-breakpoint
CREATE UNIQUE INDEX `ranks_sort_order_idx` ON `ranks` (`sort_order`);--> statement-breakpoint
CREATE TABLE `role_permissions` (
 `role_id` integer NOT NULL,
 `permission` text NOT NULL,
 PRIMARY KEY(`role_id`, `permission`),
 FOREIGN KEY (`role_id`) REFERENCES `roles`(`id`) ON UPDATE no action ON DELETE cascade
);
--> statement-breakpoint
CREATE TABLE `roles` (
 `id` integer PRIMARY KEY AUTOINCREMENT NOT NULL,
 `name` text NOT NULL,
 `description` text,
 `color` text,
 `discord_role_id` text,
 `sort_order` integer DEFAULT 0 NOT NULL,
 `is_system` integer DEFAULT false NOT NULL,
 `created_at` integer DEFAULT (unixepoch()) NOT NULL,
 `updated_at` integer DEFAULT (unixepoch()) NOT NULL
);
--> statement-breakpoint
CREATE UNIQUE INDEX `roles_name_unique` ON `roles` (`name`);--> statement-breakpoint
CREATE UNIQUE INDEX `roles_discord_role_id_unique` ON `roles` (`discord_role_id`);-->
statement-breakpoint
CREATE TABLE `sessions` (
 `id` text PRIMARY KEY NOT NULL,
 `user_id` integer NOT NULL,
 `expires_at` integer NOT NULL,
 `user_agent` text,
 `ip` text,
 `created_at` integer DEFAULT (unixepoch()) NOT NULL,
 FOREIGN KEY (`user_id`) REFERENCES `users`(`id`) ON UPDATE no action ON DELETE cascade
);
--> statement-breakpoint
CREATE INDEX `sessions_user_idx` ON `sessions` (`user_id`);--> statement-breakpoint
CREATE INDEX `sessions_expires_idx` ON `sessions` (`expires_at`);--> statement-breakpoint
CREATE TABLE `settings` (
 `key` text PRIMARY KEY NOT NULL,
 `value` text NOT NULL,
 `updated_by` integer,
```

**drizzle/0000\_cold\_khan.sql**

```
`updated_at` integer DEFAULT (unixepoch()) NOT NULL,
FOREIGN KEY (`updated_by`) REFERENCES `users`(`id`) ON UPDATE no action ON DELETE set null
);
--> statement-breakpoint
CREATE TABLE `user_roles` (
 `user_id` integer NOT NULL,
 `role_id` integer NOT NULL,
 `granted_by` integer,
 `granted_at` integer DEFAULT (unixepoch()) NOT NULL,
 PRIMARY KEY(`user_id`, `role_id`),
 FOREIGN KEY (`user_id`) REFERENCES `users`(`id`) ON UPDATE no action ON DELETE cascade,
 FOREIGN KEY (`role_id`) REFERENCES `roles`(`id`) ON UPDATE no action ON DELETE cascade,
 FOREIGN KEY (`granted_by`) REFERENCES `users`(`id`) ON UPDATE no action ON DELETE set null
);
--> statement-breakpoint
CREATE INDEX `user_roles_role_idx` ON `user_roles` (`role_id`);--> statement-breakpoint
CREATE TABLE `users` (
 `id` integer PRIMARY KEY AUTOINCREMENT NOT NULL,
 `discord_id` text NOT NULL,
 `username` text NOT NULL,
 `global_name` text,
 `avatar` text,
 `email` text,
 `rank_id` integer,
 `is_god` integer DEFAULT false NOT NULL,
 `status` text DEFAULT 'active' NOT NULL,
 `recruiter_id` integer,
 `joined_at` integer DEFAULT (unixepoch()) NOT NULL,
 `last_seen_at` integer,
 `promoted_at` integer,
 `created_at` integer DEFAULT (unixepoch()) NOT NULL,
 `updated_at` integer DEFAULT (unixepoch()) NOT NULL,
 FOREIGN KEY (`rank_id`) REFERENCES `ranks`(`id`) ON UPDATE no action ON DELETE set null
);
--> statement-breakpoint
CREATE UNIQUE INDEX `users_discord_id_unique` ON `users` (`discord_id`);--> statement-breakpoint
CREATE INDEX `users_rank_idx` ON `users` (`rank_id`);--> statement-breakpoint
CREATE INDEX `users_status_idx` ON `users` (`status`);
```

**drizzle/0001\_slim\_lady\_deathstrike.sql**

```
CREATE TABLE `rank_roles` (
 `rank_id` integer NOT NULL,
 `role_id` integer NOT NULL,
 PRIMARY KEY(`rank_id`, `role_id`),
 FOREIGN KEY (`rank_id`) REFERENCES `ranks`(`id`) ON UPDATE no action ON DELETE cascade,
 FOREIGN KEY (`role_id`) REFERENCES `roles`(`id`) ON UPDATE no action ON DELETE cascade
);
--> statement-breakpoint
CREATE INDEX `rank_roles_role_idx` ON `rank_roles` (`role_id`);--> statement-breakpoint
ALTER TABLE `user_roles` ADD `source` text DEFAULT 'manual' NOT NULL;
```

**drizzle/0002\_melted\_lilith.sql**

```
ALTER TABLE `users` ADD `display_name` text;
```

**drizzle/0003\_odd\_jasper\_sitwell.sql**

```
ALTER TABLE `medals` ADD `auto_grant_months` integer;--> statement-breakpoint
ALTER TABLE `users` ADD `guild_joined_at` integer;
```



**drizzle/0004\_lazy\_blur.sql**

```
ALTER TABLE `users` ADD `profile_image_url` text;
```

**drizzle/0005\_colossal\_warbird.sql**

```
CREATE TABLE `member_war_records` (
 `id` integer PRIMARY KEY AUTOINCREMENT NOT NULL,
 `user_id` integer NOT NULL,
 `war_record_id` integer NOT NULL,
 `citation` text,
 `awarded_by` integer,
 `awarded_at` integer DEFAULT (unixepoch()) NOT NULL,
 FOREIGN KEY (`user_id`) REFERENCES `users`(`id`) ON UPDATE no action ON DELETE cascade,
 FOREIGN KEY (`war_record_id`) REFERENCES `war_records`(`id`) ON UPDATE no action ON DELETE
cascade,
 FOREIGN KEY (`awarded_by`) REFERENCES `users`(`id`) ON UPDATE no action ON DELETE set null
);
--> statement-breakpoint
CREATE INDEX `member_war_records_user_idx` ON `member_war_records` (`user_id`);-->
statement-breakpoint
CREATE INDEX `member_war_records_record_idx` ON `member_war_records` (`war_record_id`);-->
statement-breakpoint
CREATE TABLE `war_records` (
 `id` integer PRIMARY KEY AUTOINCREMENT NOT NULL,
 `name` text NOT NULL,
 `description` text,
 `image_url` text,
 `game_id` integer,
 `sort_order` integer DEFAULT 0 NOT NULL,
 `created_at` integer DEFAULT (unixepoch()) NOT NULL,
 FOREIGN KEY (`game_id`) REFERENCES `games`(`id`) ON UPDATE no action ON DELETE set null
);
--> statement-breakpoint
CREATE INDEX `war_records_game_idx` ON `war_records` (`game_id`);
```

**drizzle/0006\_youthful\_power\_pack.sql**

```
ALTER TABLE `users` ADD `last_reconciled_at` integer;
```

**drizzle/0007\_dizzy\_champions.sql**

```
CREATE TABLE `events` (
 `id` integer PRIMARY KEY AUTOINCREMENT NOT NULL,
 `title` text NOT NULL,
 `description` text,
 `starts_at` integer NOT NULL,
 `ends_at` integer NOT NULL,
 `location` text NOT NULL,
 `game_id` integer,
 `created_by` integer,
 `discord_event_id` text,
 `discord_message_id` text,
 `status` text DEFAULT 'scheduled' NOT NULL,
 `created_at` integer DEFAULT (unixepoch()) NOT NULL,
 `updated_at` integer DEFAULT (unixepoch()) NOT NULL,
 FOREIGN KEY (`game_id`) REFERENCES `games`(`id`) ON UPDATE no action ON DELETE set null,
 FOREIGN KEY (`created_by`) REFERENCES `users`(`id`) ON UPDATE no action ON DELETE set null
);
--> statement-breakpoint
CREATE INDEX `events_starts_idx` ON `events` (`starts_at`);
```

**drizzle/0008\_brown\_silhouette.sql**

```
ALTER TABLE `audit_log` ADD `reason` text;
```

**drizzle/0009\_groovy\_tattoo.sql**

```
CREATE TABLE `event_roles` (
 `id` integer PRIMARY KEY AUTOINCREMENT NOT NULL,
 `event_id` integer NOT NULL,
 `name` text NOT NULL,
 `emoji` text,
 `capacity` integer,
 `sort_order` integer DEFAULT 0 NOT NULL,
 FOREIGN KEY (`event_id`) REFERENCES `events`(`id`) ON UPDATE no action ON DELETE cascade
);
--> statement-breakpoint
CREATE INDEX `event_roles_event_idx` ON `event_roles` (`event_id`);--> statement-breakpoint
CREATE TABLE `event_signups` (
 `id` integer PRIMARY KEY AUTOINCREMENT NOT NULL,
 `event_id` integer NOT NULL,
 `user_id` integer NOT NULL,
 `event_role_id` integer,
 `created_at` integer DEFAULT (unixepoch()) NOT NULL,
 `updated_at` integer DEFAULT (unixepoch()) NOT NULL,
 FOREIGN KEY (`event_id`) REFERENCES `events`(`id`) ON UPDATE no action ON DELETE cascade,
 FOREIGN KEY (`user_id`) REFERENCES `users`(`id`) ON UPDATE no action ON DELETE cascade,
 FOREIGN KEY (`event_role_id`) REFERENCES `event_roles`(`id`) ON UPDATE no action ON DELETE set
 null
);
--> statement-breakpoint
CREATE UNIQUE INDEX `event_signups_event_user_idx` ON `event_signups` (`event_id`,`user_id`);-->
statement-breakpoint
CREATE INDEX `event_signups_event_idx` ON `event_signups` (`event_id`);--> statement-breakpoint
ALTER TABLE `events` ADD `image_url` text;
```

**drizzle/0010\_faulty\_puppet\_master.sql**

```
ALTER TABLE `events` ADD `recurrence` text DEFAULT 'none' NOT NULL;
```

**drizzle/0011\_magical\_iron\_fist.sql**

```
CREATE TABLE `page_layouts` (
 `slug` text PRIMARY KEY NOT NULL,
 `title` text,
 `layout` text NOT NULL,
 `updated_by` integer,
 `updated_at` integer DEFAULT (unixepoch()) NOT NULL,
 FOREIGN KEY (`updated_by`) REFERENCES `users`(`id`) ON UPDATE no action ON DELETE set null
);
```



**drizzle/0012\_flawless\_obadiah\_stane.sql**

```
ALTER TABLE `page_layouts` ADD `show_in_nav` integer DEFAULT false NOT NULL;-->
statement-breakpoint
ALTER TABLE `page_layouts` ADD `nav_order` integer DEFAULT 0 NOT NULL;
```

**drizzle/0013\_nice\_slapstick.sql**

```
CREATE TABLE `api_tokens` (
 `id` integer PRIMARY KEY AUTOINCREMENT NOT NULL,
 `user_id` integer NOT NULL,
 `token_hash` text NOT NULL,
 `label` text NOT NULL,
 `prefix` text NOT NULL,
 `expires_at` integer,
 `last_used_at` integer,
 `created_at` integer DEFAULT (unixepoch()) NOT NULL,
 FOREIGN KEY (`user_id`) REFERENCES `users`(`id`) ON UPDATE no action ON DELETE cascade
);
--> statement-breakpoint
CREATE UNIQUE INDEX `api_tokens_token_hash_unique` ON `api_tokens` (`token_hash`);-->
statement-breakpoint
CREATE INDEX `api_tokens_user_idx` ON `api_tokens` (`user_id`);
```

**drizzle/0014\_complex\_titanium\_man.sql**

```
CREATE TABLE `sc_hangars` (
 `user_id` integer PRIMARY KEY NOT NULL,
 `items` text NOT NULL,
 `item_count` integer DEFAULT 0 NOT NULL,
 `imported_at` integer DEFAULT (unixepoch()) NOT NULL,
 FOREIGN KEY (`user_id`) REFERENCES `users`(`id`) ON UPDATE no action ON DELETE cascade
);
```

**drizzle/0015\_naive\_giant\_man.sql**

```
ALTER TABLE `sc_hangars` ADD `is_public` integer DEFAULT false NOT NULL;
```

**drizzle/0016\_volatile\_red\_hulk.sql**

```
CREATE TABLE `sc_verifications` (
 `user_id` integer PRIMARY KEY NOT NULL,
 `pending_code` text,
 `pending_handle` text,
 `pending_at` integer,
 `rsi_handle` text,
 `verified_at` integer,
 `org_sid` text,
 `org_rank` text,
 `org_visible` integer,
 `in_org` integer,
 FOREIGN KEY (`user_id`) REFERENCES `users`(`id`) ON UPDATE no action ON DELETE cascade
);
--> statement-breakpoint
CREATE UNIQUE INDEX `sc_verifications_rsi_handle_unique` ON `sc_verifications` (`rsi_handle`);
```

**drizzle/0017\_mute\_dreaming\_celestial.sql**

```
ALTER TABLE `sc_verifications` ADD `last_check_at` integer;
```

**drizzle/meta/\_journal.json**

```
{
 "version": "7",
 "dialect": "sqlite",
 "entries": [
 {
 "idx": 0,
 "version": "6",
 "when": 1785780844121,
 "tag": "0000_cold_khan",
 "breakpoints": true
 },
 {
 "idx": 1,
 "version": "6",
 "when": 1785794157729,
 "tag": "0001_slim_lady_deathstrike",
 "breakpoints": true
 },
 {
 "idx": 2,
 "version": "6",
 "when": 1785796171473,
 "tag": "0002_melted_lilith",
 "breakpoints": true
 },
 {
 "idx": 3,
 "version": "6",
 "when": 1785799853409,
 "tag": "0003_odd_jasper_sitwell",
 "breakpoints": true
 },
 {
 "idx": 4,
 "version": "6",
 "when": 1785845996225,
 "tag": "0004_lazy_blur",
 "breakpoints": true
 },
 {
 "idx": 5,
 "version": "6",
 "when": 1785852151692,
 "tag": "0005_colossal_warbird",
 "breakpoints": true
 },
 {
 "idx": 6,
 "version": "6",
 "when": 1785854222965,
 "tag": "0006_youthful_power_pack",
 "breakpoints": true
 },
 {

```

**drizzle/meta/\_journal.json**

```
"idx": 7,
"version": "6",
"when": 1785856376012,
"tag": "0007_dizzy_champions",
"breakpoints": true
},
{
 "idx": 8,
 "version": "6",
 "when": 1785868209126,
 "tag": "0008_brown_silhouette",
 "breakpoints": true
},
{
 "idx": 9,
 "version": "6",
 "when": 1785869669330,
 "tag": "0009_groovy_tattoo",
 "breakpoints": true
},
{
 "idx": 10,
 "version": "6",
 "when": 1785873227977,
 "tag": "0010_faulty_puppet_master",
 "breakpoints": true
},
{
 "idx": 11,
 "version": "6",
 "when": 1785878796989,
 "tag": "0011_magical_iron_fist",
 "breakpoints": true
},
{
 "idx": 12,
 "version": "6",
 "when": 1785889196445,
 "tag": "0012_flawless_obadiah_stane",
 "breakpoints": true
},
{
 "idx": 13,
 "version": "6",
 "when": 1785936034850,
 "tag": "0013_nice_slapstick",
 "breakpoints": true
},
{
 "idx": 14,
 "version": "6",
 "when": 1786046626092,
 "tag": "0014_complex_titanium_man",
 "breakpoints": true
}
```



**drizzle/meta/\_journal.json**

```
{,
{
 "idx": 15,
 "version": "6",
 "when": 1786049672727,
 "tag": "0015_naive_giant_man",
 "breakpoints": true
},
{
 "idx": 16,
 "version": "6",
 "when": 1786110395616,
 "tag": "0016_volatile_red_hulk",
 "breakpoints": true
},
{
 "idx": 17,
 "version": "6",
 "when": 1786112455785,
 "tag": "0017_mute_dreaming_celestial",
 "breakpoints": true
}
]
```

**drizzle/meta/0000\_snapshot.json**

```
{
 "version": "6",
 "dialect": "sqlite",
 "id": "711e79f3-be5a-4edc-b4b4-02338854eb2c",
 "prevId": "000000000-0000-0000-0000-000000000000",
 "tables": {
 "audit_log": {
 "name": "audit_log",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "actor_id": {
 "name": "actor_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "action": {
 "name": "action",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "target_type": {
 "name": "target_type",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "target_id": {
 "name": "target_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "meta": {
 "name": "meta",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
```

**drizzle/meta/0000\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'web'"
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "audit_actor_idx": {
 "name": "audit_actor_idx",
 "columns": [
 "actor_id"
],
 "isUnique": false
 },
 "audit_created_idx": {
 "name": "audit_created_idx",
 "columns": [
 "created_at"
],
 "isUnique": false
 },
 "audit_action_idx": {
 "name": "audit_action_idx",
 "columns": [
 "action"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "audit_log_actor_id_users_id_fk": {
 "name": "audit_log_actor_id_users_id_fk",
 "tableFrom": "audit_log",
 "tableTo": "users",
 "columnsFrom": [
 "actor_id"
],
 "columnsTo": [
 "id"
]
 }
}
```

**drizzle/meta/0000\_snapshot.json**

```
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"games": {
 "name": "games",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "icon_url": {
 "name": "icon_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "active": {
 "name": "active",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
```

**drizzle/meta/0000\_snapshot.json**

```
 "default": true
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "games_slug_unique": {
 "name": "games_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"match_participants": {
 "name": "match_participants",
 "columns": {
 "match_id": {
 "name": "match_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "stats": {
 "name": "stats",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
},
"indexes": {
 "mp_user_idx": {
 "name": "mp_user_idx",
 "columns": [
```

**drizzle/meta/0000\_snapshot.json**

```
 "user_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "match_participants_match_id_matches_id_fk": {
 "name": "match_participants_match_id_matches_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "matches",
 "columnsFrom": [
 "match_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "match_participants_user_id_users_id_fk": {
 "name": "match_participants_user_id_users_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "match_participants_match_id_user_id_pk": {
 "columns": [
 "match_id",
 "user_id"
],
 "name": "match_participants_match_id_user_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"matches": {
 "name": "matches",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 }
 }
}
```

**drizzle/meta/0000\_snapshot.json**

```
,
"game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"opponent": {
 "name": "opponent",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"opponent_tag": {
 "name": "opponent_tag",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"result": {
 "name": "result",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"score_us": {
 "name": "score_us",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"score_them": {
 "name": "score_them",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"map": {
 "name": "map",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"notes": {
 "name": "notes",
 "type": "text",
 "primaryKey": false,
```

**drizzle/meta/0000\_snapshot.json**

```
 "notNull": false,
 "autoincrement": false
 },
 "played_at": {
 "name": "played_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "recorded_by": {
 "name": "recorded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "matches_game_played_idx": {
 "name": "matches_game_played_idx",
 "columns": [
 "game_id",
 "played_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "matches_game_id_games_id_fk": {
 "name": "matches_game_id_games_id_fk",
 "tableFrom": "matches",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "matches_recorded_by_users_id_fk": {
 "name": "matches_recorded_by_users_id_fk",
 "tableFrom": "matches",
 "tableTo": "users",
```



**drizzle/meta/0000\_snapshot.json**

```
 "columnsFrom": [
 "recorded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"medals": {
 "name": "medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
```

**drizzle/meta/0000\_snapshot.json**

```
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "medals_game_idx": {
 "name": "medals_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "medals_game_id_games_id_fk": {
 "name": "medals_game_id_games_id_fk",
 "tableFrom": "medals",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"member_medals": {
 "name": "member_medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
```

**drizzle/meta/0000\_snapshot.json**

```
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"medal_id": {
 "name": "medal_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
 "member_medals_user_idx": {
 "name": "member_medals_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "member_medals_medal_idx": {
 "name": "member_medals_medal_idx",
 "columns": [
 "medal_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "member_medals_user_id_users_id_fk": {
```

**drizzle/meta/0000\_snapshot.json**

```
 "name": "member_medals_user_id_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_medal_id_medals_id_fk": {
 "name": "member_medals_medal_id_medals_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "medals",
 "columnsFrom": [
 "medal_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_awarded_by_users_id_fk": {
 "name": "member_medals_awarded_by_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"news": {
 "name": "news",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
```

**drizzle/meta/0000\_snapshot.json**

```
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "excerpt": {
 "name": "excerpt",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "body": {
 "name": "body",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "author_id": {
 "name": "author_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'draft'"
 },
 "pinned": {
 "name": "pinned",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "published_at": {
 "name": "published_at",
 "type": "integer",
 "primaryKey": false,
```

**drizzle/meta/0000\_snapshot.json**

```
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "news_slug_unique": {
 "name": "news_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 },
 "news_status_published_idx": {
 "name": "news_status_published_idx",
 "columns": [
 "status",
 "published_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "news_author_id_users_id_fk": {
 "name": "news_author_id_users_id_fk",
 "tableFrom": "news",
 "tableTo": "users",
 "columnsFrom": [
 "author_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
```

**drizzle/meta/0000\_snapshot.json**

```
"checkConstraints": {},
},
"profiles": {
 "name": "profiles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "bio": {
 "name": "bio",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "timezone": {
 "name": "timezone",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "gamertags": {
 "name": "gamertags",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {},
 "foreignKeys": {
 "profiles_user_id_users_id_fk": {
 "name": "profiles_user_id_users_id_fk",
 "tableFrom": "profiles",
```

**drizzle/meta/0000\_snapshot.json**

```
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"ranks": {
 "name": "ranks",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "abbreviation": {
 "name": "abbreviation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "req_days": {
```



**drizzle/meta/0000\_snapshot.json**

```
 "name": "req_days",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "req_wins": {
 "name": "req_wins",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_default": {
 "name": "is_default",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "ranks_sort_order_idx": {
 "name": "ranks_sort_order_idx",
 "columns": [
 "sort_order"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
```

**drizzle/meta/0000\_snapshot.json**

```
"role_permissions": {
 "name": "role_permissions",
 "columns": {
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "permission": {
 "name": "permission",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "indexes": {},
 "foreignKeys": {
 "role_permissions_role_id_roles_id_fk": {
 "name": "role_permissions_role_id_roles_id_fk",
 "tableFrom": "role_permissions",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {
 "role_permissions_role_id_permission_pk": {
 "columns": [
 "role_id",
 "permission"
],
 "name": "role_permissions_role_id_permission_pk"
 }
 },
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"roles": {
 "name": "roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
```

**drizzle/meta/0000\_snapshot.json**

```
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "color": {
 "name": "color",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_role_id": {
 "name": "discord_role_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_system": {
 "name": "is_system",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
}
```

**drizzle/meta/0000\_snapshot.json**

```
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "roles_name_unique": {
 "name": "roles_name_unique",
 "columns": [
 "name"
],
 "isUnique": true
 },
 "roles_discord_role_id_unique": {
 "name": "roles_discord_role_id_unique",
 "columns": [
 "discord_role_id"
],
 "isUnique": true
 }
 },
 "foreignKeys": {},
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"sessions": {
 "name": "sessions",
 "columns": {
 "id": {
 "name": "id",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "expires_at": {
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 }
},
```

**drizzle/meta/0000\_snapshot.json**

```
"user_agent": {
 "name": "user_agent",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
 "sessions_user_idx": {
 "name": "sessions_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "sessions_expires_idx": {
 "name": "sessions_expires_idx",
 "columns": [
 "expires_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "sessions_user_id_users_id_fk": {
 "name": "sessions_user_id_users_id_fk",
 "tableFrom": "sessions",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
},
```

**drizzle/meta/0000\_snapshot.json**

```
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"settings": {
 "name": "settings",
 "columns": {
 "key": {
 "name": "key",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "value": {
 "name": "value",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {},
 "foreignKeys": {
 "settings_updated_by_users_id_fk": {
 "name": "settings_updated_by_users_id_fk",
 "tableFrom": "settings",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
```

**drizzle/meta/0000\_snapshot.json**

```
"uniqueConstraints": {},
"checkConstraints": {}
},
"user_roles": {
 "name": "user_roles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "granted_by": {
 "name": "granted_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "granted_at": {
 "name": "granted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "user_roles_role_idx": {
 "name": "user_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "user_roles_user_id_users_id_fk": {
 "name": "user_roles_user_id_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
```

**drizzle/meta/0000\_snapshot.json**

```

 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_role_id_roles_id_fk": {
 "name": "user_roles_role_id_roles_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_granted_by_users_id_fk": {
 "name": "user_roles_granted_by_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "granted_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "user_roles_user_id_role_id_pk": {
 "columns": [
 "user_id",
 "role_id"
],
 "name": "user_roles_user_id_role_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"users": {
 "name": "users",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 }
 },

```



**drizzle/meta/0000\_snapshot.json**

```
"discord_id": {
 "name": "discord_id",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"username": {
 "name": "username",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"global_name": {
 "name": "global_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"avatar": {
 "name": "avatar",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"email": {
 "name": "email",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"is_god": {
 "name": "is_god",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
},
"status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
```

**drizzle/meta/0000\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false,
 "default": "'active'"
 },
 "recruiter_id": {
 "name": "recruiter_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "joined_at": {
 "name": "joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "last_seen_at": {
 "name": "last_seen_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "promoted_at": {
 "name": "promoted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "users_discord_id_unique": {
 "name": "users_discord_id_unique",
 "columns": [
```

**drizzle/meta/0000\_snapshot.json**

```
 "discord_id"
],
 "isUnique": true
},
"users_rank_idx": {
 "name": "users_rank_idx",
 "columns": [
 "rank_id"
],
 "isUnique": false
},
"users_status_idx": {
 "name": "users_status_idx",
 "columns": [
 "status"
],
 "isUnique": false
}
},
"foreignKeys": {
 "users_rank_id_ranks_id_fk": {
 "name": "users_rank_id_ranks_id_fk",
 "tableFrom": "users",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
}
},
"views": {},
"enums": {},
"_meta": {
 "schemas": {},
 "tables": {},
 "columns": {}
},
"internal": {
 "indexes": {}
}
}
```

**drizzle/meta/0001\_snapshot.json**

```
{
 "version": "6",
 "dialect": "sqlite",
 "id": "2f10d190-dce5-46d6-b0bd-cc3bfb72950c",
 "prevId": "711e79f3-be5a-4edc-b4b4-02338854eb2c",
 "tables": {
 "audit_log": {
 "name": "audit_log",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "actor_id": {
 "name": "actor_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "action": {
 "name": "action",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "target_type": {
 "name": "target_type",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "target_id": {
 "name": "target_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "meta": {
 "name": "meta",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
```

**drizzle/meta/0001\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'web'"
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "audit_actor_idx": {
 "name": "audit_actor_idx",
 "columns": [
 "actor_id"
],
 "isUnique": false
 },
 "audit_created_idx": {
 "name": "audit_created_idx",
 "columns": [
 "created_at"
],
 "isUnique": false
 },
 "audit_action_idx": {
 "name": "audit_action_idx",
 "columns": [
 "action"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "audit_log_actor_id_users_id_fk": {
 "name": "audit_log_actor_id_users_id_fk",
 "tableFrom": "audit_log",
 "tableTo": "users",
 "columnsFrom": [
 "actor_id"
],
 "columnsTo": [
 "id"
]
 }
}
```

**drizzle/meta/0001\_snapshot.json**

```
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"games": {
 "name": "games",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "icon_url": {
 "name": "icon_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "active": {
 "name": "active",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
```

**drizzle/meta/0001\_snapshot.json**

```
 "default": true
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "games_slug_unique": {
 "name": "games_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"match_participants": {
 "name": "match_participants",
 "columns": {
 "match_id": {
 "name": "match_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "stats": {
 "name": "stats",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
},
"indexes": {
 "mp_user_idx": {
 "name": "mp_user_idx",
 "columns": [
```

**drizzle/meta/0001\_snapshot.json**

```

 "user_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "match_participants_match_id_matches_id_fk": {
 "name": "match_participants_match_id_matches_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "matches",
 "columnsFrom": [
 "match_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "match_participants_user_id_users_id_fk": {
 "name": "match_participants_user_id_users_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "match_participants_match_id_user_id_pk": {
 "columns": [
 "match_id",
 "user_id"
],
 "name": "match_participants_match_id_user_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"matches": {
 "name": "matches",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 }
 }
}

```



**drizzle/meta/0001\_snapshot.json**

```
,
"game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"opponent": {
 "name": "opponent",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"opponent_tag": {
 "name": "opponent_tag",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"result": {
 "name": "result",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"score_us": {
 "name": "score_us",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"score_them": {
 "name": "score_them",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"map": {
 "name": "map",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"notes": {
 "name": "notes",
 "type": "text",
 "primaryKey": false,
```

**drizzle/meta/0001\_snapshot.json**

```
 "notNull": false,
 "autoincrement": false
 },
 "played_at": {
 "name": "played_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "recorded_by": {
 "name": "recorded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "matches_game_played_idx": {
 "name": "matches_game_played_idx",
 "columns": [
 "game_id",
 "played_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "matches_game_id_games_id_fk": {
 "name": "matches_game_id_games_id_fk",
 "tableFrom": "matches",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "matches_recorded_by_users_id_fk": {
 "name": "matches_recorded_by_users_id_fk",
 "tableFrom": "matches",
 "tableTo": "users",
```

**drizzle/meta/0001\_snapshot.json**

```
 "columnsFrom": [
 "recorded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"medals": {
 "name": "medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
```

**drizzle/meta/0001\_snapshot.json**

```

 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "medals_game_idx": {
 "name": "medals_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "medals_game_id_games_id_fk": {
 "name": "medals_game_id_games_id_fk",
 "tableFrom": "medals",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"member_medals": {
 "name": "member_medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {

```

**drizzle/meta/0001\_snapshot.json**

```
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"medal_id": {
 "name": "medal_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
 "member_medals_user_idx": {
 "name": "member_medals_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "member_medals_medal_idx": {
 "name": "member_medals_medal_idx",
 "columns": [
 "medal_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "member_medals_user_id_users_id_fk": {
```

**drizzle/meta/0001\_snapshot.json**

```
 "name": "member_medals_user_id_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_medal_id_medals_id_fk": {
 "name": "member_medals_medal_id_medals_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "medals",
 "columnsFrom": [
 "medal_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_awarded_by_users_id_fk": {
 "name": "member_medals_awarded_by_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"news": {
 "name": "news",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
```

**drizzle/meta/0001\_snapshot.json**

```
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "excerpt": {
 "name": "excerpt",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "body": {
 "name": "body",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "author_id": {
 "name": "author_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'draft'"
 },
 "pinned": {
 "name": "pinned",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "published_at": {
 "name": "published_at",
 "type": "integer",
 "primaryKey": false,
```

**drizzle/meta/0001\_snapshot.json**

```
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "news_slug_unique": {
 "name": "news_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 },
 "news_status_published_idx": {
 "name": "news_status_published_idx",
 "columns": [
 "status",
 "published_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "news_author_id_users_id_fk": {
 "name": "news_author_id_users_id_fk",
 "tableFrom": "news",
 "tableTo": "users",
 "columnsFrom": [
 "author_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
```



**drizzle/meta/0001\_snapshot.json**

```
"checkConstraints": {},
},
"profiles": {
 "name": "profiles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "bio": {
 "name": "bio",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "timezone": {
 "name": "timezone",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "gamertags": {
 "name": "gamertags",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {},
 "foreignKeys": {
 "profiles_user_id_users_id_fk": {
 "name": "profiles_user_id_users_id_fk",
 "tableFrom": "profiles",
```

**drizzle/meta/0001\_snapshot.json**

```
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"rank_roles": {
 "name": "rank_roles",
 "columns": {
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "indexes": {
 "rank_roles_role_idx": {
 "name": "rank_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "rank_roles_rank_id_ranks_id_fk": {
 "name": "rank_roles_rank_id_ranks_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
```

**drizzle/meta/0001\_snapshot.json**

```
 "onUpdate": "no action"
 },
 "rank_roles_role_id_roles_id_fk": {
 "name": "rank_roles_role_id_roles_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "rank_roles_rank_id_role_id_pk": {
 "columns": [
 "rank_id",
 "role_id"
],
 "name": "rank_roles_rank_id_role_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"ranks": {
 "name": "ranks",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "abbreviation": {
 "name": "abbreviation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
```

**drizzle/meta/0001\_snapshot.json**

```
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "req_days": {
 "name": "req_days",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "req_wins": {
 "name": "req_wins",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_default": {
 "name": "is_default",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
```

**drizzle/meta/0001\_snapshot.json**

```
 "ranks_sort_order_idx": {
 "name": "ranks_sort_order_idx",
 "columns": [
 "sort_order"
],
 "isUnique": true
 }
 },
 "foreignKeys": {},
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"role_permissions": {
 "name": "role_permissions",
 "columns": {
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "permission": {
 "name": "permission",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "indexes": {},
 "foreignKeys": {
 "role_permissions_role_id_roles_id_fk": {
 "name": "role_permissions_role_id_roles_id_fk",
 "tableFrom": "role_permissions",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {
 "role_permissions_role_id_permission_pk": {
 "columns": [
 "role_id",
 "permission"
],
 "name": "role_permissions_role_id_permission_pk"
 }
 }
}
```

**drizzle/meta/0001\_snapshot.json**

```
 },
 "uniqueConstraints": {},
 "checkConstraints": {}
 },
 "roles": {
 "name": "roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "color": {
 "name": "color",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_role_id": {
 "name": "discord_role_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_system": {
 "name": "is_system",
 "type": "integer",
```

**drizzle/meta/0001\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "roles_name_unique": {
 "name": "roles_name_unique",
 "columns": [
 "name"
],
 "isUnique": true
 },
 "roles_discord_role_id_unique": {
 "name": "roles_discord_role_id_unique",
 "columns": [
 "discord_role_id"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"sessions": {
 "name": "sessions",
 "columns": {
 "id": {
 "name": "id",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
```

**drizzle/meta/0001\_snapshot.json**

```
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "expires_at": {
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_agent": {
 "name": "user_agent",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "sessions_user_idx": {
 "name": "sessions_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "sessions_expires_idx": {
 "name": "sessions_expires_idx",
 "columns": [
 "expires_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "sessions_user_id_users_id_fk": {
```



**drizzle/meta/0001\_snapshot.json**

```
 "name": "sessions_user_id_users_id_fk",
 "tableFrom": "sessions",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"settings": {
 "name": "settings",
 "columns": {
 "key": {
 "name": "key",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "value": {
 "name": "value",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {},
 "foreignKeys": {
 "settings_updated_by_users_id_fk": {
 "name": "settings_updated_by_users_id_fk",
```

**drizzle/meta/0001\_snapshot.json**

```
 "tableFrom": "settings",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"user_roles": {
 "name": "user_roles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'manual'"
 },
 "granted_by": {
 "name": "granted_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "granted_at": {
 "name": "granted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 }
 }
}
```

**drizzle/meta/0001\_snapshot.json**

```
 "default": "(unixepoch())"
 },
 "indexes": {
 "user_roles_role_idx": {
 "name": "user_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "user_roles_user_id_users_id_fk": {
 "name": "user_roles_user_id_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_role_id_roles_id_fk": {
 "name": "user_roles_role_id_roles_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_granted_by_users_id_fk": {
 "name": "user_roles_granted_by_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "granted_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {
```

**drizzle/meta/0001\_snapshot.json**

```
 "user_roles_user_id_role_id_pk": {
 "columns": [
 "user_id",
 "role_id"
],
 "name": "user_roles_user_id_role_id_pk"
 }
 },
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"users": {
 "name": "users",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "discord_id": {
 "name": "discord_id",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "username": {
 "name": "username",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "global_name": {
 "name": "global_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "avatar": {
 "name": "avatar",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "email": {
 "name": "email",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
```

**drizzle/meta/0001\_snapshot.json**

```
 "autoincrement": false
 },
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "is_god": {
 "name": "is_god",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'active'"
 },
 "recruiter_id": {
 "name": "recruiter_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "joined_at": {
 "name": "joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "last_seen_at": {
 "name": "last_seen_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "promoted_at": {
 "name": "promoted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
}
```

**drizzle/meta/0001\_snapshot.json**

```
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
},
"updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
 "users_discord_id_unique": {
 "name": "users_discord_id_unique",
 "columns": [
 "discord_id"
],
 "isUnique": true
 },
 "users_rank_idx": {
 "name": "users_rank_idx",
 "columns": [
 "rank_id"
],
 "isUnique": false
 },
 "users_status_idx": {
 "name": "users_status_idx",
 "columns": [
 "status"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "users_rank_id_ranks_id_fk": {
 "name": "users_rank_id_ranks_id_fk",
 "tableFrom": "users",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
}
```

**drizzle/meta/0001\_snapshot.json**

```
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
 }
},
"views": {},
"enums": {},
"_meta": {
 "schemas": {},
 "tables": {},
 "columns": {}
},
"internal": {
 "indexes": {}
}
}
```

**drizzle/meta/0002\_snapshot.json**

```
{
 "version": "6",
 "dialect": "sqlite",
 "id": "83457785-6abe-49ad-838c-487c55731618",
 "prevId": "2f10d190-dce5-46d6-b0bd-cc3bfb72950c",
 "tables": {
 "audit_log": {
 "name": "audit_log",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "actor_id": {
 "name": "actor_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "action": {
 "name": "action",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "target_type": {
 "name": "target_type",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "target_id": {
 "name": "target_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "meta": {
 "name": "meta",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
```



**drizzle/meta/0002\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'web'"
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "audit_actor_idx": {
 "name": "audit_actor_idx",
 "columns": [
 "actor_id"
],
 "isUnique": false
 },
 "audit_created_idx": {
 "name": "audit_created_idx",
 "columns": [
 "created_at"
],
 "isUnique": false
 },
 "audit_action_idx": {
 "name": "audit_action_idx",
 "columns": [
 "action"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "audit_log_actor_id_users_id_fk": {
 "name": "audit_log_actor_id_users_id_fk",
 "tableFrom": "audit_log",
 "tableTo": "users",
 "columnsFrom": [
 "actor_id"
],
 "columnsTo": [
 "id"
]
 }
}
```

**drizzle/meta/0002\_snapshot.json**

```
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"games": {
 "name": "games",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "icon_url": {
 "name": "icon_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "active": {
 "name": "active",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
```

**drizzle/meta/0002\_snapshot.json**

```
 "default": true
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "games_slug_unique": {
 "name": "games_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"match_participants": {
 "name": "match_participants",
 "columns": {
 "match_id": {
 "name": "match_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "stats": {
 "name": "stats",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
},
"indexes": {
 "mp_user_idx": {
 "name": "mp_user_idx",
 "columns": [
```

**drizzle/meta/0002\_snapshot.json**

```
 "user_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "match_participants_match_id_matches_id_fk": {
 "name": "match_participants_match_id_matches_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "matches",
 "columnsFrom": [
 "match_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "match_participants_user_id_users_id_fk": {
 "name": "match_participants_user_id_users_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "match_participants_match_id_user_id_pk": {
 "columns": [
 "match_id",
 "user_id"
],
 "name": "match_participants_match_id_user_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"matches": {
 "name": "matches",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 }
 }
}
```

**drizzle/meta/0002\_snapshot.json**

```
,
"game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"opponent": {
 "name": "opponent",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"opponent_tag": {
 "name": "opponent_tag",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"result": {
 "name": "result",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"score_us": {
 "name": "score_us",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"score_them": {
 "name": "score_them",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"map": {
 "name": "map",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"notes": {
 "name": "notes",
 "type": "text",
 "primaryKey": false,
```

**drizzle/meta/0002\_snapshot.json**

```
 "notNull": false,
 "autoincrement": false
 },
 "played_at": {
 "name": "played_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "recorded_by": {
 "name": "recorded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "matches_game_played_idx": {
 "name": "matches_game_played_idx",
 "columns": [
 "game_id",
 "played_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "matches_game_id_games_id_fk": {
 "name": "matches_game_id_games_id_fk",
 "tableFrom": "matches",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "matches_recorded_by_users_id_fk": {
 "name": "matches_recorded_by_users_id_fk",
 "tableFrom": "matches",
 "tableTo": "users",
```

**drizzle/meta/0002\_snapshot.json**

```
 "columnsFrom": [
 "recorded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"medals": {
 "name": "medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
```

**drizzle/meta/0002\_snapshot.json**

```
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "medals_game_idx": {
 "name": "medals_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "medals_game_id_games_id_fk": {
 "name": "medals_game_id_games_id_fk",
 "tableFrom": "medals",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"member_medals": {
 "name": "member_medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
```



**drizzle/meta/0002\_snapshot.json**

```
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "medal_id": {
 "name": "medal_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "member_medals_user_idx": {
 "name": "member_medals_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "member_medals_medal_idx": {
 "name": "member_medals_medal_idx",
 "columns": [
 "medal_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "member_medals_user_id_users_id_fk": {
```

**drizzle/meta/0002\_snapshot.json**

```

 "name": "member_medals_user_id_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_medal_id_medals_id_fk": {
 "name": "member_medals_medal_id_medals_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "medals",
 "columnsFrom": [
 "medal_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_awarded_by_users_id_fk": {
 "name": "member_medals_awarded_by_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"news": {
 "name": "news",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {

```

**drizzle/meta/0002\_snapshot.json**

```
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "excerpt": {
 "name": "excerpt",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "body": {
 "name": "body",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "author_id": {
 "name": "author_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'draft'"
 },
 "pinned": {
 "name": "pinned",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "published_at": {
 "name": "published_at",
 "type": "integer",
 "primaryKey": false,
```

**drizzle/meta/0002\_snapshot.json**

```
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "news_slug_unique": {
 "name": "news_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 },
 "news_status_published_idx": {
 "name": "news_status_published_idx",
 "columns": [
 "status",
 "published_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "news_author_id_users_id_fk": {
 "name": "news_author_id_users_id_fk",
 "tableFrom": "news",
 "tableTo": "users",
 "columnsFrom": [
 "author_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
```

**drizzle/meta/0002\_snapshot.json**

```
"checkConstraints": {},
},
"profiles": {
 "name": "profiles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "bio": {
 "name": "bio",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "timezone": {
 "name": "timezone",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "gamertags": {
 "name": "gamertags",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {},
 "foreignKeys": {
 "profiles_user_id_users_id_fk": {
 "name": "profiles_user_id_users_id_fk",
 "tableFrom": "profiles",
```

**drizzle/meta/0002\_snapshot.json**

```
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"rank_roles": {
 "name": "rank_roles",
 "columns": {
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "indexes": {
 "rank_roles_role_idx": {
 "name": "rank_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "rank_roles_rank_id_ranks_id_fk": {
 "name": "rank_roles_rank_id_ranks_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
```

**drizzle/meta/0002\_snapshot.json**

```
 "onUpdate": "no action"
 },
 "rank_roles_role_id_roles_id_fk": {
 "name": "rank_roles_role_id_roles_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {
 "rank_roles_rank_id_role_id_pk": {
 "columns": [
 "rank_id",
 "role_id"
],
 "name": "rank_roles_rank_id_role_id_pk"
 }
 },
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"ranks": {
 "name": "ranks",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "abbreviation": {
 "name": "abbreviation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
```

**drizzle/meta/0002\_snapshot.json**

```
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "req_days": {
 "name": "req_days",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "req_wins": {
 "name": "req_wins",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_default": {
 "name": "is_default",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
```



**drizzle/meta/0002\_snapshot.json**

```
"ranks_sort_order_idx": {
 "name": "ranks_sort_order_idx",
 "columns": [
 "sort_order"
],
 "isUnique": true
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"role_permissions": {
 "name": "role_permissions",
 "columns": {
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "permission": {
 "name": "permission",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "indexes": {},
 "foreignKeys": {
 "role_permissions_role_id_roles_id_fk": {
 "name": "role_permissions_role_id_roles_id_fk",
 "tableFrom": "role_permissions",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {
 "role_permissions_role_id_permission_pk": {
 "columns": [
 "role_id",
 "permission"
],
 "name": "role_permissions_role_id_permission_pk"
 }
 }
}
```

**drizzle/meta/0002\_snapshot.json**

```
 },
 "uniqueConstraints": {},
 "checkConstraints": {}
 },
 "roles": {
 "name": "roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "color": {
 "name": "color",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_role_id": {
 "name": "discord_role_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_system": {
 "name": "is_system",
 "type": "integer",
```

**drizzle/meta/0002\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "roles_name_unique": {
 "name": "roles_name_unique",
 "columns": [
 "name"
],
 "isUnique": true
 },
 "roles_discord_role_id_unique": {
 "name": "roles_discord_role_id_unique",
 "columns": [
 "discord_role_id"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"sessions": {
 "name": "sessions",
 "columns": {
 "id": {
 "name": "id",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
```

**drizzle/meta/0002\_snapshot.json**

```
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"expires_at": {
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"user_agent": {
 "name": "user_agent",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
 "sessions_user_idx": {
 "name": "sessions_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "sessions_expires_idx": {
 "name": "sessions_expires_idx",
 "columns": [
 "expires_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "sessions_user_id_users_id_fk": {
```

**drizzle/meta/0002\_snapshot.json**

```
 "name": "sessions_user_id_users_id_fk",
 "tableFrom": "sessions",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"settings": {
 "name": "settings",
 "columns": {
 "key": {
 "name": "key",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "value": {
 "name": "value",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {},
 "foreignKeys": {
 "settings_updated_by_users_id_fk": {
 "name": "settings_updated_by_users_id_fk",
```

**drizzle/meta/0002\_snapshot.json**

```
 "tableFrom": "settings",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"user_roles": {
 "name": "user_roles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'manual'"
 },
 "granted_by": {
 "name": "granted_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "granted_at": {
 "name": "granted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 }
 }
}
```

**drizzle/meta/0002\_snapshot.json**

```
 "default": "(unixepoch())"
 },
 "indexes": {
 "user_roles_role_idx": {
 "name": "user_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "user_roles_user_id_users_id_fk": {
 "name": "user_roles_user_id_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_role_id_roles_id_fk": {
 "name": "user_roles_role_id_roles_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_granted_by_users_id_fk": {
 "name": "user_roles_granted_by_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "granted_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {
```

**drizzle/meta/0002\_snapshot.json**

```
 "user_roles_user_id_role_id_pk": {
 "columns": [
 "user_id",
 "role_id"
],
 "name": "user_roles_user_id_role_id_pk"
 }
 },
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"users": {
 "name": "users",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "discord_id": {
 "name": "discord_id",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "username": {
 "name": "username",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "global_name": {
 "name": "global_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "display_name": {
 "name": "display_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "avatar": {
 "name": "avatar",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
```



**drizzle/meta/0002\_snapshot.json**

```
 "autoincrement": false
 },
 "email": {
 "name": "email",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "is_god": {
 "name": "is_god",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'active'"
 },
 "recruiter_id": {
 "name": "recruiter_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "joined_at": {
 "name": "joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "last_seen_at": {
 "name": "last_seen_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
}
```

**drizzle/meta/0002\_snapshot.json**

```
"promoted_at": {
 "name": "promoted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
},
"updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
 "users_discord_id_unique": {
 "name": "users_discord_id_unique",
 "columns": [
 "discord_id"
],
 "isUnique": true
 },
 "users_rank_idx": {
 "name": "users_rank_idx",
 "columns": [
 "rank_id"
],
 "isUnique": false
 },
 "users_status_idx": {
 "name": "users_status_idx",
 "columns": [
 "status"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "users_rank_id_ranks_id_fk": {
 "name": "users_rank_id_ranks_id_fk",
 "tableFrom": "users",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
]
 }
}
```

**drizzle/meta/0002\_snapshot.json**

```
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
}
},
"views": {},
"enums": {},
"_meta": {
 "schemas": {},
 "tables": {},
 "columns": {}
},
"internal": {
 "indexes": {}
}
}
```

**drizzle/meta/0003\_snapshot.json**

```
{
 "version": "6",
 "dialect": "sqlite",
 "id": "331a671a-17ad-402b-98ad-1da11008c69c",
 "prevId": "83457785-6abe-49ad-838c-487c55731618",
 "tables": {
 "audit_log": {
 "name": "audit_log",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "actor_id": {
 "name": "actor_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "action": {
 "name": "action",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "target_type": {
 "name": "target_type",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "target_id": {
 "name": "target_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "meta": {
 "name": "meta",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
```

**drizzle/meta/0003\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'web'"
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "audit_actor_idx": {
 "name": "audit_actor_idx",
 "columns": [
 "actor_id"
],
 "isUnique": false
 },
 "audit_created_idx": {
 "name": "audit_created_idx",
 "columns": [
 "created_at"
],
 "isUnique": false
 },
 "audit_action_idx": {
 "name": "audit_action_idx",
 "columns": [
 "action"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "audit_log_actor_id_users_id_fk": {
 "name": "audit_log_actor_id_users_id_fk",
 "tableFrom": "audit_log",
 "tableTo": "users",
 "columnsFrom": [
 "actor_id"
],
 "columnsTo": [
 "id"
]
 }
}
```

**drizzle/meta/0003\_snapshot.json**

```
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"games": {
 "name": "games",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "icon_url": {
 "name": "icon_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "active": {
 "name": "active",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
```

**drizzle/meta/0003\_snapshot.json**

```
 "default": true
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "games_slug_unique": {
 "name": "games_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"match_participants": {
 "name": "match_participants",
 "columns": {
 "match_id": {
 "name": "match_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "stats": {
 "name": "stats",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
},
"indexes": {
 "mp_user_idx": {
 "name": "mp_user_idx",
 "columns": [
```

**drizzle/meta/0003\_snapshot.json**

```
 "user_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "match_participants_match_id_matches_id_fk": {
 "name": "match_participants_match_id_matches_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "matches",
 "columnsFrom": [
 "match_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "match_participants_user_id_users_id_fk": {
 "name": "match_participants_user_id_users_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "match_participants_match_id_user_id_pk": {
 "columns": [
 "match_id",
 "user_id"
],
 "name": "match_participants_match_id_user_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"matches": {
 "name": "matches",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 }
 }
}
```



**drizzle/meta/0003\_snapshot.json**

```
,
"game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"opponent": {
 "name": "opponent",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"opponent_tag": {
 "name": "opponent_tag",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"result": {
 "name": "result",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"score_us": {
 "name": "score_us",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"score_them": {
 "name": "score_them",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"map": {
 "name": "map",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"notes": {
 "name": "notes",
 "type": "text",
 "primaryKey": false,
```

**drizzle/meta/0003\_snapshot.json**

```
 "notNull": false,
 "autoincrement": false
 },
 "played_at": {
 "name": "played_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "recorded_by": {
 "name": "recorded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "matches_game_played_idx": {
 "name": "matches_game_played_idx",
 "columns": [
 "game_id",
 "played_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "matches_game_id_games_id_fk": {
 "name": "matches_game_id_games_id_fk",
 "tableFrom": "matches",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "matches_recorded_by_users_id_fk": {
 "name": "matches_recorded_by_users_id_fk",
 "tableFrom": "matches",
 "tableTo": "users",
```

**drizzle/meta/0003\_snapshot.json**

```
 "columnsFrom": [
 "recorded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"medals": {
 "name": "medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "auto_grant_months": {
 "name": "auto_grant_months",
```

**drizzle/meta/0003\_snapshot.json**

```
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "medals_game_idx": {
 "name": "medals_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "medals_game_id_games_id_fk": {
 "name": "medals_game_id_games_id_fk",
 "tableFrom": "medals",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"member_medals": {
 "name": "member_medals",
 "columns": {
 "id": {
```

**drizzle/meta/0003\_snapshot.json**

```
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "medal_id": {
 "name": "medal_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "member_medals_user_idx": {
 "name": "member_medals_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "member_medals_medal_idx": {
 "name": "member_medals_medal_idx",
 "columns": [
```

**drizzle/meta/0003\_snapshot.json**

```
 "medal_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "member_medals_user_id_users_id_fk": {
 "name": "member_medals_user_id_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_medal_id_medals_id_fk": {
 "name": "member_medals_medal_id_medals_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "medals",
 "columnsFrom": [
 "medal_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_awarded_by_users_id_fk": {
 "name": "member_medals_awarded_by_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"news": {
 "name": "news",
 "columns": {
 "id": {
```

**drizzle/meta/0003\_snapshot.json**

```
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "excerpt": {
 "name": "excerpt",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "body": {
 "name": "body",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "author_id": {
 "name": "author_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'draft'"
 },
 "pinned": {
 "name": "pinned",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
```

**drizzle/meta/0003\_snapshot.json**

```
 "autoincrement": false,
 "default": false
 },
 "published_at": {
 "name": "published_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "news_slug_unique": {
 "name": "news_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 },
 "news_status_published_idx": {
 "name": "news_status_published_idx",
 "columns": [
 "status",
 "published_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "news_author_id_users_id_fk": {
 "name": "news_author_id_users_id_fk",
 "tableFrom": "news",
 "tableTo": "users",
 "columnsFrom": [
 "author_id"
],
 "columnsTo": [
 "id"
]
 }
}
```



**drizzle/meta/0003\_snapshot.json**

```
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"profiles": {
 "name": "profiles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "bio": {
 "name": "bio",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "timezone": {
 "name": "timezone",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "gamertags": {
 "name": "gamertags",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
```

**drizzle/meta/0003\_snapshot.json**

```
 }
 },
 "indexes": {},
 "foreignKeys": {
 "profiles_user_id_users_id_fk": {
 "name": "profiles_user_id_users_id_fk",
 "tableFrom": "profiles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"rank_roles": {
 "name": "rank_roles",
 "columns": {
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "indexes": {
 "rank_roles_role_idx": {
 "name": "rank_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "rank_roles_rank_id_ranks_id_fk": {
 "name": "rank_roles_rank_id_ranks_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "ranks",
```

**drizzle/meta/0003\_snapshot.json**

```

 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "rank_roles_role_id_roles_id_fk": {
 "name": "rank_roles_role_id_roles_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "rank_roles_rank_id_role_id_pk": {
 "columns": [
 "rank_id",
 "role_id"
],
 "name": "rank_roles_rank_id_role_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"ranks": {
 "name": "ranks",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "abbreviation": {
 "name": "abbreviation",

```

**drizzle/meta/0003\_snapshot.json**

```
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "req_days": {
 "name": "req_days",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "req_wins": {
 "name": "req_wins",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_default": {
 "name": "is_default",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
```

**drizzle/meta/0003\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "ranks_sort_order_idx": {
 "name": "ranks_sort_order_idx",
 "columns": [
 "sort_order"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"role_permissions": {
 "name": "role_permissions",
 "columns": {
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "permission": {
 "name": "permission",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 }
},
"indexes": {},
"foreignKeys": {
 "role_permissions_role_id_roles_id_fk": {
 "name": "role_permissions_role_id_roles_id_fk",
 "tableFrom": "role_permissions",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
}
```

**drizzle/meta/0003\_snapshot.json**

```
"compositePrimaryKeys": {
 "role_permissions_role_id_permission_pk": {
 "columns": [
 "role_id",
 "permission"
],
 "name": "role_permissions_role_id_permission_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"roles": {
 "name": "roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "color": {
 "name": "color",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_role_id": {
 "name": "discord_role_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
```

**drizzle/meta/0003\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_system": {
 "name": "is_system",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "roles_name_unique": {
 "name": "roles_name_unique",
 "columns": [
 "name"
],
 "isUnique": true
 },
 "roles_discord_role_id_unique": {
 "name": "roles_discord_role_id_unique",
 "columns": [
 "discord_role_id"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"sessions": {
 "name": "sessions",
 "columns": {
 "id": {
```

**drizzle/meta/0003\_snapshot.json**

```
 "name": "id",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "expires_at": {
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_agent": {
 "name": "user_agent",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "sessions_user_idx": {
 "name": "sessions_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "sessions_expires_idx": {
 "name": "sessions_expires_idx",
 "columns": [
```



**drizzle/meta/0003\_snapshot.json**

```
 "expires_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "sessions_user_id_users_id_fk": {
 "name": "sessions_user_id_users_id_fk",
 "tableFrom": "sessions",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"settings": {
 "name": "settings",
 "columns": {
 "key": {
 "name": "key",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "value": {
 "name": "value",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,

```

**drizzle/meta/0003\_snapshot.json**

```
 "default": "(unixepoch())"
 },
 "indexes": {},
 "foreignKeys": {
 "settings_updated_by_users_id_fk": {
 "name": "settings_updated_by_users_id_fk",
 "tableFrom": "settings",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"user_roles": {
 "name": "user_roles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'manual'"
 },
 "granted_by": {
 "name": "granted_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
}
```

**drizzle/meta/0003\_snapshot.json**

```
 },
 "granted_at": {
 "name": "granted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "user_roles_role_idx": {
 "name": "user_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "user_roles_user_id_users_id_fk": {
 "name": "user_roles_user_id_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_role_id_roles_id_fk": {
 "name": "user_roles_role_id_roles_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_granted_by_users_id_fk": {
 "name": "user_roles_granted_by_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "granted_by"
],
 "columnsTo": [
```

**drizzle/meta/0003\_snapshot.json**

```
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "user_roles_user_id_role_id_pk": {
 "columns": [
 "user_id",
 "role_id"
],
 "name": "user_roles_user_id_role_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"users": {
 "name": "users",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "discord_id": {
 "name": "discord_id",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "username": {
 "name": "username",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "global_name": {
 "name": "global_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "display_name": {
 "name": "display_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
```

**drizzle/meta/0003\_snapshot.json**

```
 "autoincrement": false
 },
 "avatar": {
 "name": "avatar",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "email": {
 "name": "email",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "is_god": {
 "name": "is_god",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "guild_joined_at": {
 "name": "guild_joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'active'"
 },
 "recruiter_id": {
 "name": "recruiter_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "joined_at": {
```

**drizzle/meta/0003\_snapshot.json**

```
 "name": "joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "last_seen_at": {
 "name": "last_seen_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "promoted_at": {
 "name": "promoted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "users_discord_id_unique": {
 "name": "users_discord_id_unique",
 "columns": [
 "discord_id"
],
 "isUnique": true
 },
 "users_rank_idx": {
 "name": "users_rank_idx",
 "columns": [
 "rank_id"
],
 "isUnique": false
 },
 "users_status_idx": {
```

**drizzle/meta/0003\_snapshot.json**

```
 "name": "users_status_idx",
 "columns": [
 "status"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "users_rank_id_ranks_id_fk": {
 "name": "users_rank_id_ranks_id_fk",
 "tableFrom": "users",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
}
},
"views": {},
"enums": {},
"_meta": {
 "schemas": {},
 "tables": {},
 "columns": {}
},
"internal": {
 "indexes": {}
}
}
```

**drizzle/meta/0004\_snapshot.json**

```
{
 "version": "6",
 "dialect": "sqlite",
 "id": "fb1e5032-1ba8-409b-8997-a52166290e29",
 "prevId": "331a671a-17ad-402b-98ad-1da11008c69c",
 "tables": {
 "audit_log": {
 "name": "audit_log",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "actor_id": {
 "name": "actor_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "action": {
 "name": "action",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "target_type": {
 "name": "target_type",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "target_id": {
 "name": "target_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "meta": {
 "name": "meta",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
```



**drizzle/meta/0004\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'web'"
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "audit_actor_idx": {
 "name": "audit_actor_idx",
 "columns": [
 "actor_id"
],
 "isUnique": false
 },
 "audit_created_idx": {
 "name": "audit_created_idx",
 "columns": [
 "created_at"
],
 "isUnique": false
 },
 "audit_action_idx": {
 "name": "audit_action_idx",
 "columns": [
 "action"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "audit_log_actor_id_users_id_fk": {
 "name": "audit_log_actor_id_users_id_fk",
 "tableFrom": "audit_log",
 "tableTo": "users",
 "columnsFrom": [
 "actor_id"
],
 "columnsTo": [
 "id"
]
 }
}
```

**drizzle/meta/0004\_snapshot.json**

```
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"games": {
 "name": "games",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "icon_url": {
 "name": "icon_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "active": {
 "name": "active",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
```

**drizzle/meta/0004\_snapshot.json**

```
 "default": true
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "games_slug_unique": {
 "name": "games_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"match_participants": {
 "name": "match_participants",
 "columns": {
 "match_id": {
 "name": "match_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "stats": {
 "name": "stats",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
},
"indexes": {
 "mp_user_idx": {
 "name": "mp_user_idx",
 "columns": [
```

**drizzle/meta/0004\_snapshot.json**

```
 "user_id"
],
 "isUnique": false
}
},
"foreignKeys": {
 "match_participants_match_id_matches_id_fk": {
 "name": "match_participants_match_id_matches_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "matches",
 "columnsFrom": [
 "match_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "match_participants_user_id_users_id_fk": {
 "name": "match_participants_user_id_users_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "match_participants_match_id_user_id_pk": {
 "columns": [
 "match_id",
 "user_id"
],
 "name": "match_participants_match_id_user_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"matches": {
 "name": "matches",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 }
 }
}
```

**drizzle/meta/0004\_snapshot.json**

```
,
"game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"opponent": {
 "name": "opponent",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"opponent_tag": {
 "name": "opponent_tag",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"result": {
 "name": "result",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"score_us": {
 "name": "score_us",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"score_them": {
 "name": "score_them",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"map": {
 "name": "map",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"notes": {
 "name": "notes",
 "type": "text",
 "primaryKey": false,
```

**drizzle/meta/0004\_snapshot.json**

```
 "notNull": false,
 "autoincrement": false
 },
 "played_at": {
 "name": "played_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "recorded_by": {
 "name": "recorded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "matches_game_played_idx": {
 "name": "matches_game_played_idx",
 "columns": [
 "game_id",
 "played_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "matches_game_id_games_id_fk": {
 "name": "matches_game_id_games_id_fk",
 "tableFrom": "matches",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "matches_recorded_by_users_id_fk": {
 "name": "matches_recorded_by_users_id_fk",
 "tableFrom": "matches",
 "tableTo": "users",
```

**drizzle/meta/0004\_snapshot.json**

```
 "columnsFrom": [
 "recorded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"medals": {
 "name": "medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "auto_grant_months": {
 "name": "auto_grant_months",
```

**drizzle/meta/0004\_snapshot.json**

```

 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "medals_game_idx": {
 "name": "medals_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "medals_game_id_games_id_fk": {
 "name": "medals_game_id_games_id_fk",
 "tableFrom": "medals",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"member_medals": {
 "name": "member_medals",
 "columns": {
 "id": {

```



**drizzle/meta/0004\_snapshot.json**

```
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "medal_id": {
 "name": "medal_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "member_medals_user_idx": {
 "name": "member_medals_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "member_medals_medal_idx": {
 "name": "member_medals_medal_idx",
 "columns": [
```

**drizzle/meta/0004\_snapshot.json**

```
 "medal_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "member_medals_user_id_users_id_fk": {
 "name": "member_medals_user_id_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_medal_id_medals_id_fk": {
 "name": "member_medals_medal_id_medals_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "medals",
 "columnsFrom": [
 "medal_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_awarded_by_users_id_fk": {
 "name": "member_medals_awarded_by_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"news": {
 "name": "news",
 "columns": {
 "id": {
```

**drizzle/meta/0004\_snapshot.json**

```
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "excerpt": {
 "name": "excerpt",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "body": {
 "name": "body",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "author_id": {
 "name": "author_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'draft'"
 },
 "pinned": {
 "name": "pinned",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
```

**drizzle/meta/0004\_snapshot.json**

```
 "autoincrement": false,
 "default": false
 },
 "published_at": {
 "name": "published_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "news_slug_unique": {
 "name": "news_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 },
 "news_status_published_idx": {
 "name": "news_status_published_idx",
 "columns": [
 "status",
 "published_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "news_author_id_users_id_fk": {
 "name": "news_author_id_users_id_fk",
 "tableFrom": "news",
 "tableTo": "users",
 "columnsFrom": [
 "author_id"
],
 "columnsTo": [
 "id"
]
 }
}
```

**drizzle/meta/0004\_snapshot.json**

```
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"profiles": {
 "name": "profiles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "bio": {
 "name": "bio",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "timezone": {
 "name": "timezone",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "gamertags": {
 "name": "gamertags",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
```

**drizzle/meta/0004\_snapshot.json**

```
 }
 },
 "indexes": {},
 "foreignKeys": {
 "profiles_user_id_users_id_fk": {
 "name": "profiles_user_id_users_id_fk",
 "tableFrom": "profiles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"rank_roles": {
 "name": "rank_roles",
 "columns": {
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "indexes": {
 "rank_roles_role_idx": {
 "name": "rank_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "rank_roles_rank_id_ranks_id_fk": {
 "name": "rank_roles_rank_id_ranks_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "ranks",
```

**drizzle/meta/0004\_snapshot.json**

```
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "rank_roles_role_id_roles_id_fk": {
 "name": "rank_roles_role_id_roles_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "rank_roles_rank_id_role_id_pk": {
 "columns": [
 "rank_id",
 "role_id"
],
 "name": "rank_roles_rank_id_role_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"ranks": {
 "name": "ranks",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "abbreviation": {
 "name": "abbreviation",
```

**drizzle/meta/0004\_snapshot.json**

```
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "req_days": {
 "name": "req_days",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "req_wins": {
 "name": "req_wins",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_default": {
 "name": "is_default",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
```



**drizzle/meta/0004\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "ranks_sort_order_idx": {
 "name": "ranks_sort_order_idx",
 "columns": [
 "sort_order"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"role_permissions": {
 "name": "role_permissions",
 "columns": {
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "permission": {
 "name": "permission",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 }
},
"indexes": {},
"foreignKeys": {
 "role_permissions_role_id_roles_id_fk": {
 "name": "role_permissions_role_id_roles_id_fk",
 "tableFrom": "role_permissions",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
}
```

**drizzle/meta/0004\_snapshot.json**

```
"compositePrimaryKeys": {
 "role_permissions_role_id_permission_pk": {
 "columns": [
 "role_id",
 "permission"
],
 "name": "role_permissions_role_id_permission_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"roles": {
 "name": "roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "color": {
 "name": "color",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_role_id": {
 "name": "discord_role_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
```

**drizzle/meta/0004\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_system": {
 "name": "is_system",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "roles_name_unique": {
 "name": "roles_name_unique",
 "columns": [
 "name"
],
 "isUnique": true
 },
 "roles_discord_role_id_unique": {
 "name": "roles_discord_role_id_unique",
 "columns": [
 "discord_role_id"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"sessions": {
 "name": "sessions",
 "columns": {
 "id": {
```

**drizzle/meta/0004\_snapshot.json**

```
 "name": "id",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "expires_at": {
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_agent": {
 "name": "user_agent",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "sessions_user_idx": {
 "name": "sessions_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "sessions_expires_idx": {
 "name": "sessions_expires_idx",
 "columns": [
```

**drizzle/meta/0004\_snapshot.json**

```
 "expires_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "sessions_user_id_users_id_fk": {
 "name": "sessions_user_id_users_id_fk",
 "tableFrom": "sessions",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"settings": {
 "name": "settings",
 "columns": {
 "key": {
 "name": "key",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "value": {
 "name": "value",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
```

**drizzle/meta/0004\_snapshot.json**

```
 "default": "(unixepoch())"
 },
 "indexes": {},
 "foreignKeys": {
 "settings_updated_by_users_id_fk": {
 "name": "settings_updated_by_users_id_fk",
 "tableFrom": "settings",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"user_roles": {
 "name": "user_roles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'manual'"
 },
 "granted_by": {
 "name": "granted_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
}
```

**drizzle/meta/0004\_snapshot.json**

```
 },
 "granted_at": {
 "name": "granted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "user_roles_role_idx": {
 "name": "user_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "user_roles_user_id_users_id_fk": {
 "name": "user_roles_user_id_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_role_id_roles_id_fk": {
 "name": "user_roles_role_id_roles_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_granted_by_users_id_fk": {
 "name": "user_roles_granted_by_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "granted_by"
],
 "columnsTo": [
```

**drizzle/meta/0004\_snapshot.json**

```
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
}
},
"compositePrimaryKeys": {
 "user_roles_user_id_role_id_pk": {
 "columns": [
 "user_id",
 "role_id"
],
 "name": "user_roles_user_id_role_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"users": {
 "name": "users",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "discord_id": {
 "name": "discord_id",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "username": {
 "name": "username",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "global_name": {
 "name": "global_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "display_name": {
 "name": "display_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
```



**drizzle/meta/0004\_snapshot.json**

```
 "autoincrement": false
 },
 "avatar": {
 "name": "avatar",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "profile_image_url": {
 "name": "profile_image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "email": {
 "name": "email",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "is_god": {
 "name": "is_god",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "guild_joined_at": {
 "name": "guild_joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'active'"
 },
 "recruiter_id": {
```

**drizzle/meta/0004\_snapshot.json**

```
 "name": "recruiter_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "joined_at": {
 "name": "joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "last_seen_at": {
 "name": "last_seen_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "promoted_at": {
 "name": "promoted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "users_discord_id_unique": {
 "name": "users_discord_id_unique",
 "columns": [
 "discord_id"
],
 "isUnique": true
 },
 "users_rank_idx": {
```

**drizzle/meta/0004\_snapshot.json**

```
 "name": "users_rank_idx",
 "columns": [
 "rank_id"
],
 "isUnique": false
 },
 "users_status_idx": {
 "name": "users_status_idx",
 "columns": [
 "status"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "users_rank_id_ranks_id_fk": {
 "name": "users_rank_id_ranks_id_fk",
 "tableFrom": "users",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
}
},
"views": {},
"enums": {},
"_meta": {
 "schemas": {},
 "tables": {},
 "columns": {}
},
"internal": {
 "indexes": {}
}
}
```

**drizzle/meta/0005\_snapshot.json**

```
{
 "version": "6",
 "dialect": "sqlite",
 "id": "20c4628c-bce9-4ab7-8633-609b9a365643",
 "prevId": "fb1e5032-1ba8-409b-8997-a52166290e29",
 "tables": {
 "audit_log": {
 "name": "audit_log",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "actor_id": {
 "name": "actor_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "action": {
 "name": "action",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "target_type": {
 "name": "target_type",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "target_id": {
 "name": "target_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "meta": {
 "name": "meta",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
```

**drizzle/meta/0005\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'web'"
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "audit_actor_idx": {
 "name": "audit_actor_idx",
 "columns": [
 "actor_id"
],
 "isUnique": false
 },
 "audit_created_idx": {
 "name": "audit_created_idx",
 "columns": [
 "created_at"
],
 "isUnique": false
 },
 "audit_action_idx": {
 "name": "audit_action_idx",
 "columns": [
 "action"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "audit_log_actor_id_users_id_fk": {
 "name": "audit_log_actor_id_users_id_fk",
 "tableFrom": "audit_log",
 "tableTo": "users",
 "columnsFrom": [
 "actor_id"
],
 "columnsTo": [
 "id"
]
 }
}
```

**drizzle/meta/0005\_snapshot.json**

```
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"games": {
 "name": "games",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "icon_url": {
 "name": "icon_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "active": {
 "name": "active",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
```

**drizzle/meta/0005\_snapshot.json**

```
 "default": true
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "games_slug_unique": {
 "name": "games_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"match_participants": {
 "name": "match_participants",
 "columns": {
 "match_id": {
 "name": "match_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "stats": {
 "name": "stats",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
},
"indexes": {
 "mp_user_idx": {
 "name": "mp_user_idx",
 "columns": [
```

**drizzle/meta/0005\_snapshot.json**

```
 "user_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "match_participants_match_id_matches_id_fk": {
 "name": "match_participants_match_id_matches_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "matches",
 "columnsFrom": [
 "match_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "match_participants_user_id_users_id_fk": {
 "name": "match_participants_user_id_users_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "match_participants_match_id_user_id_pk": {
 "columns": [
 "match_id",
 "user_id"
],
 "name": "match_participants_match_id_user_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"matches": {
 "name": "matches",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 }
 }
}
```



**drizzle/meta/0005\_snapshot.json**

```
,
"game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"opponent": {
 "name": "opponent",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"opponent_tag": {
 "name": "opponent_tag",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"result": {
 "name": "result",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"score_us": {
 "name": "score_us",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"score_them": {
 "name": "score_them",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"map": {
 "name": "map",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"notes": {
 "name": "notes",
 "type": "text",
 "primaryKey": false,
```

**drizzle/meta/0005\_snapshot.json**

```
 "notNull": false,
 "autoincrement": false
 },
 "played_at": {
 "name": "played_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "recorded_by": {
 "name": "recorded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "matches_game_played_idx": {
 "name": "matches_game_played_idx",
 "columns": [
 "game_id",
 "played_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "matches_game_id_games_id_fk": {
 "name": "matches_game_id_games_id_fk",
 "tableFrom": "matches",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "matches_recorded_by_users_id_fk": {
 "name": "matches_recorded_by_users_id_fk",
 "tableFrom": "matches",
 "tableTo": "users",
```

**drizzle/meta/0005\_snapshot.json**

```
 "columnsFrom": [
 "recorded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"medals": {
 "name": "medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "auto_grant_months": {
 "name": "auto_grant_months",
```

**drizzle/meta/0005\_snapshot.json**

```
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "medals_game_idx": {
 "name": "medals_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "medals_game_id_games_id_fk": {
 "name": "medals_game_id_games_id_fk",
 "tableFrom": "medals",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"member_medals": {
 "name": "member_medals",
 "columns": {
 "id": {
```

**drizzle/meta/0005\_snapshot.json**

```
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "medal_id": {
 "name": "medal_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "member_medals_user_idx": {
 "name": "member_medals_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "member_medals_medal_idx": {
 "name": "member_medals_medal_idx",
 "columns": [
```

**drizzle/meta/0005\_snapshot.json**

```
 "medal_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "member_medals_user_id_users_id_fk": {
 "name": "member_medals_user_id_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_medal_id_medals_id_fk": {
 "name": "member_medals_medal_id_medals_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "medals",
 "columnsFrom": [
 "medal_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_awarded_by_users_id_fk": {
 "name": "member_medals_awarded_by_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"member_war_records": {
 "name": "member_war_records",
 "columns": {
 "id": {
```

**drizzle/meta/0005\_snapshot.json**

```
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "war_record_id": {
 "name": "war_record_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "member_war_records_user_idx": {
 "name": "member_war_records_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "member_war_records_record_idx": {
 "name": "member_war_records_record_idx",
 "columns": [
```

**drizzle/meta/0005\_snapshot.json**

```
 "war_record_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "member_war_records_user_id_users_id_fk": {
 "name": "member_war_records_user_id_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_war_record_id_war_records_id_fk": {
 "name": "member_war_records_war_record_id_war_records_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "war_records",
 "columnsFrom": [
 "war_record_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_awarded_by_users_id_fk": {
 "name": "member_war_records_awarded_by_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"news": {
 "name": "news",
 "columns": {
 "id": {
```



**drizzle/meta/0005\_snapshot.json**

```
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "excerpt": {
 "name": "excerpt",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "body": {
 "name": "body",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "author_id": {
 "name": "author_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'draft'"
 },
 "pinned": {
 "name": "pinned",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
```

**drizzle/meta/0005\_snapshot.json**

```
 "autoincrement": false,
 "default": false
 },
 "published_at": {
 "name": "published_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "news_slug_unique": {
 "name": "news_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 },
 "news_status_published_idx": {
 "name": "news_status_published_idx",
 "columns": [
 "status",
 "published_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "news_author_id_users_id_fk": {
 "name": "news_author_id_users_id_fk",
 "tableFrom": "news",
 "tableTo": "users",
 "columnsFrom": [
 "author_id"
],
 "columnsTo": [
 "id"
]
 }
}
```

**drizzle/meta/0005\_snapshot.json**

```
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"profiles": {
 "name": "profiles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "bio": {
 "name": "bio",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "timezone": {
 "name": "timezone",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "gamertags": {
 "name": "gamertags",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
```

**drizzle/meta/0005\_snapshot.json**

```
 }
 },
 "indexes": {},
 "foreignKeys": {
 "profiles_user_id_users_id_fk": {
 "name": "profiles_user_id_users_id_fk",
 "tableFrom": "profiles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"rank_roles": {
 "name": "rank_roles",
 "columns": {
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "indexes": {
 "rank_roles_role_idx": {
 "name": "rank_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "rank_roles_rank_id_ranks_id_fk": {
 "name": "rank_roles_rank_id_ranks_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "ranks",
```

**drizzle/meta/0005\_snapshot.json**

```
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "rank_roles_role_id_roles_id_fk": {
 "name": "rank_roles_role_id_roles_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "rank_roles_rank_id_role_id_pk": {
 "columns": [
 "rank_id",
 "role_id"
],
 "name": "rank_roles_rank_id_role_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"ranks": {
 "name": "ranks",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "abbreviation": {
 "name": "abbreviation",
```

**drizzle/meta/0005\_snapshot.json**

```
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "req_days": {
 "name": "req_days",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "req_wins": {
 "name": "req_wins",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_default": {
 "name": "is_default",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
```

**drizzle/meta/0005\_snapshot.json**

```

 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "ranks_sort_order_idx": {
 "name": "ranks_sort_order_idx",
 "columns": [
 "sort_order"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"role_permissions": {
 "name": "role_permissions",
 "columns": {
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "permission": {
 "name": "permission",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 }
},
"indexes": {},
"foreignKeys": {
 "role_permissions_role_id_roles_id_fk": {
 "name": "role_permissions_role_id_roles_id_fk",
 "tableFrom": "role_permissions",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},

```

**drizzle/meta/0005\_snapshot.json**

```
"compositePrimaryKeys": {
 "role_permissions_role_id_permission_pk": {
 "columns": [
 "role_id",
 "permission"
],
 "name": "role_permissions_role_id_permission_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"roles": {
 "name": "roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "color": {
 "name": "color",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_role_id": {
 "name": "discord_role_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
```



**drizzle/meta/0005\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_system": {
 "name": "is_system",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "roles_name_unique": {
 "name": "roles_name_unique",
 "columns": [
 "name"
],
 "isUnique": true
 },
 "roles_discord_role_id_unique": {
 "name": "roles_discord_role_id_unique",
 "columns": [
 "discord_role_id"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"sessions": {
 "name": "sessions",
 "columns": {
 "id": {
```

**drizzle/meta/0005\_snapshot.json**

```
 "name": "id",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "expires_at": {
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_agent": {
 "name": "user_agent",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "sessions_user_idx": {
 "name": "sessions_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "sessions_expires_idx": {
 "name": "sessions_expires_idx",
 "columns": [
```

**drizzle/meta/0005\_snapshot.json**

```
 "expires_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "sessions_user_id_users_id_fk": {
 "name": "sessions_user_id_users_id_fk",
 "tableFrom": "sessions",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"settings": {
 "name": "settings",
 "columns": {
 "key": {
 "name": "key",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "value": {
 "name": "value",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
```

**drizzle/meta/0005\_snapshot.json**

```
 "default": "(unixepoch())"
 },
 "indexes": {},
 "foreignKeys": {
 "settings_updated_by_users_id_fk": {
 "name": "settings_updated_by_users_id_fk",
 "tableFrom": "settings",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"user_roles": {
 "name": "user_roles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'manual'"
 },
 "granted_by": {
 "name": "granted_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
}
```

**drizzle/meta/0005\_snapshot.json**

```
 },
 "granted_at": {
 "name": "granted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "user_roles_role_idx": {
 "name": "user_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "user_roles_user_id_users_id_fk": {
 "name": "user_roles_user_id_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_role_id_roles_id_fk": {
 "name": "user_roles_role_id_roles_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_granted_by_users_id_fk": {
 "name": "user_roles_granted_by_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "granted_by"
],
 "columnsTo": [
```

**drizzle/meta/0005\_snapshot.json**

```
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "user_roles_user_id_role_id_pk": {
 "columns": [
 "user_id",
 "role_id"
],
 "name": "user_roles_user_id_role_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"users": {
 "name": "users",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "discord_id": {
 "name": "discord_id",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "username": {
 "name": "username",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "global_name": {
 "name": "global_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "display_name": {
 "name": "display_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
```

**drizzle/meta/0005\_snapshot.json**

```
 "autoincrement": false
 },
 "avatar": {
 "name": "avatar",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "profile_image_url": {
 "name": "profile_image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "email": {
 "name": "email",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "is_god": {
 "name": "is_god",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "guild_joined_at": {
 "name": "guild_joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'active'"
 },
 "recruiter_id": {
```

**drizzle/meta/0005\_snapshot.json**

```
 "name": "recruiter_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "joined_at": {
 "name": "joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "last_seen_at": {
 "name": "last_seen_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "promoted_at": {
 "name": "promoted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "users_discord_id_unique": {
 "name": "users_discord_id_unique",
 "columns": [
 "discord_id"
],
 "isUnique": true
 },
 "users_rank_idx": {
```



**drizzle/meta/0005\_snapshot.json**

```
 "name": "users_rank_idx",
 "columns": [
 "rank_id"
],
 "isUnique": false
 },
 "users_status_idx": {
 "name": "users_status_idx",
 "columns": [
 "status"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "users_rank_id_ranks_id_fk": {
 "name": "users_rank_id_ranks_id_fk",
 "tableFrom": "users",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"war_records": {
 "name": "war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
```

**drizzle/meta/0005\_snapshot.json**

```
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "war_records_game_idx": {
 "name": "war_records_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "war_records_game_id_games_id_fk": {
 "name": "war_records_game_id_games_id_fk",
 "tableFrom": "war_records",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
]
 },
}
```

**drizzle/meta/0005\_snapshot.json**

```
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
}
},
"views": {},
"enums": {},
"_meta": {
 "schemas": {},
 "tables": {},
 "columns": {}
},
"internal": {
 "indexes": {}
}
}
```

**drizzle/meta/0006\_snapshot.json**

```
{
 "version": "6",
 "dialect": "sqlite",
 "id": "bde9b65e-52cf-44e2-84dc-98a310df7bd1",
 "prevId": "20c4628c-bce9-4ab7-8633-609b9a365643",
 "tables": {
 "audit_log": {
 "name": "audit_log",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "actor_id": {
 "name": "actor_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "action": {
 "name": "action",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "target_type": {
 "name": "target_type",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "target_id": {
 "name": "target_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "meta": {
 "name": "meta",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
```

**drizzle/meta/0006\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'web'"
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "audit_actor_idx": {
 "name": "audit_actor_idx",
 "columns": [
 "actor_id"
],
 "isUnique": false
 },
 "audit_created_idx": {
 "name": "audit_created_idx",
 "columns": [
 "created_at"
],
 "isUnique": false
 },
 "audit_action_idx": {
 "name": "audit_action_idx",
 "columns": [
 "action"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "audit_log_actor_id_users_id_fk": {
 "name": "audit_log_actor_id_users_id_fk",
 "tableFrom": "audit_log",
 "tableTo": "users",
 "columnsFrom": [
 "actor_id"
],
 "columnsTo": [
 "id"
]
 }
}
```

**drizzle/meta/0006\_snapshot.json**

```
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"games": {
 "name": "games",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "icon_url": {
 "name": "icon_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "active": {
 "name": "active",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
```

**drizzle/meta/0006\_snapshot.json**

```
 "default": true
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "games_slug_unique": {
 "name": "games_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"match_participants": {
 "name": "match_participants",
 "columns": {
 "match_id": {
 "name": "match_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "stats": {
 "name": "stats",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
},
"indexes": {
 "mp_user_idx": {
 "name": "mp_user_idx",
 "columns": [
```

**drizzle/meta/0006\_snapshot.json**

```
 "user_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "match_participants_match_id_matches_id_fk": {
 "name": "match_participants_match_id_matches_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "matches",
 "columnsFrom": [
 "match_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "match_participants_user_id_users_id_fk": {
 "name": "match_participants_user_id_users_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "match_participants_match_id_user_id_pk": {
 "columns": [
 "match_id",
 "user_id"
],
 "name": "match_participants_match_id_user_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"matches": {
 "name": "matches",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 }
 }
}
```



**drizzle/meta/0006\_snapshot.json**

```
,
"game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"opponent": {
 "name": "opponent",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"opponent_tag": {
 "name": "opponent_tag",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"result": {
 "name": "result",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"score_us": {
 "name": "score_us",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"score_them": {
 "name": "score_them",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"map": {
 "name": "map",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"notes": {
 "name": "notes",
 "type": "text",
 "primaryKey": false,
```

**drizzle/meta/0006\_snapshot.json**

```
 "notNull": false,
 "autoincrement": false
 },
 "played_at": {
 "name": "played_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "recorded_by": {
 "name": "recorded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "matches_game_played_idx": {
 "name": "matches_game_played_idx",
 "columns": [
 "game_id",
 "played_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "matches_game_id_games_id_fk": {
 "name": "matches_game_id_games_id_fk",
 "tableFrom": "matches",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "matches_recorded_by_users_id_fk": {
 "name": "matches_recorded_by_users_id_fk",
 "tableFrom": "matches",
 "tableTo": "users",
```

**drizzle/meta/0006\_snapshot.json**

```
 "columnsFrom": [
 "recorded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"medals": {
 "name": "medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "auto_grant_months": {
 "name": "auto_grant_months",
```

**drizzle/meta/0006\_snapshot.json**

```
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "medals_game_idx": {
 "name": "medals_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "medals_game_id_games_id_fk": {
 "name": "medals_game_id_games_id_fk",
 "tableFrom": "medals",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"member_medals": {
 "name": "member_medals",
 "columns": {
 "id": {
```

**drizzle/meta/0006\_snapshot.json**

```
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "medal_id": {
 "name": "medal_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "member_medals_user_idx": {
 "name": "member_medals_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "member_medals_medal_idx": {
 "name": "member_medals_medal_idx",
 "columns": [
```

**drizzle/meta/0006\_snapshot.json**

```
 "medal_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "member_medals_user_id_users_id_fk": {
 "name": "member_medals_user_id_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_medal_id_medals_id_fk": {
 "name": "member_medals_medal_id_medals_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "medals",
 "columnsFrom": [
 "medal_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_awarded_by_users_id_fk": {
 "name": "member_medals_awarded_by_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"member_war_records": {
 "name": "member_war_records",
 "columns": {
 "id": {
```

**drizzle/meta/0006\_snapshot.json**

```
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "war_record_id": {
 "name": "war_record_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "member_war_records_user_idx": {
 "name": "member_war_records_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "member_war_records_record_idx": {
 "name": "member_war_records_record_idx",
 "columns": [
```

**drizzle/meta/0006\_snapshot.json**

```

 "war_record_id"
],
 "isUnique": false
}
},
"foreignKeys": {
 "member_war_records_user_id_users_id_fk": {
 "name": "member_war_records_user_id_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_war_record_id_war_records_id_fk": {
 "name": "member_war_records_war_record_id_war_records_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "war_records",
 "columnsFrom": [
 "war_record_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_awarded_by_users_id_fk": {
 "name": "member_war_records_awarded_by_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"news": {
 "name": "news",
 "columns": {
 "id": {

```



**drizzle/meta/0006\_snapshot.json**

```
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "excerpt": {
 "name": "excerpt",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "body": {
 "name": "body",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "author_id": {
 "name": "author_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'draft'"
 },
 "pinned": {
 "name": "pinned",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
```

**drizzle/meta/0006\_snapshot.json**

```
 "autoincrement": false,
 "default": false
 },
 "published_at": {
 "name": "published_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "news_slug_unique": {
 "name": "news_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 },
 "news_status_published_idx": {
 "name": "news_status_published_idx",
 "columns": [
 "status",
 "published_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "news_author_id_users_id_fk": {
 "name": "news_author_id_users_id_fk",
 "tableFrom": "news",
 "tableTo": "users",
 "columnsFrom": [
 "author_id"
],
 "columnsTo": [
 "id"
]
 }
}
```

**drizzle/meta/0006\_snapshot.json**

```
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"profiles": {
 "name": "profiles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "bio": {
 "name": "bio",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "timezone": {
 "name": "timezone",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "gamertags": {
 "name": "gamertags",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
```

**drizzle/meta/0006\_snapshot.json**

```
 }
 },
 "indexes": {},
 "foreignKeys": {
 "profiles_user_id_users_id_fk": {
 "name": "profiles_user_id_users_id_fk",
 "tableFrom": "profiles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"rank_roles": {
 "name": "rank_roles",
 "columns": {
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "indexes": {
 "rank_roles_role_idx": {
 "name": "rank_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "rank_roles_rank_id_ranks_id_fk": {
 "name": "rank_roles_rank_id_ranks_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "ranks",
```

**drizzle/meta/0006\_snapshot.json**

```
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "rank_roles_role_id_roles_id_fk": {
 "name": "rank_roles_role_id_roles_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "rank_roles_rank_id_role_id_pk": {
 "columns": [
 "rank_id",
 "role_id"
],
 "name": "rank_roles_rank_id_role_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"ranks": {
 "name": "ranks",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "abbreviation": {
 "name": "abbreviation",
```

**drizzle/meta/0006\_snapshot.json**

```
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "req_days": {
 "name": "req_days",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "req_wins": {
 "name": "req_wins",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_default": {
 "name": "is_default",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
```

**drizzle/meta/0006\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "ranks_sort_order_idx": {
 "name": "ranks_sort_order_idx",
 "columns": [
 "sort_order"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"role_permissions": {
 "name": "role_permissions",
 "columns": {
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "permission": {
 "name": "permission",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 }
},
"indexes": {},
"foreignKeys": {
 "role_permissions_role_id_roles_id_fk": {
 "name": "role_permissions_role_id_roles_id_fk",
 "tableFrom": "role_permissions",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
}
```

**drizzle/meta/0006\_snapshot.json**

```
"compositePrimaryKeys": {
 "role_permissions_role_id_permission_pk": {
 "columns": [
 "role_id",
 "permission"
],
 "name": "role_permissions_role_id_permission_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"roles": {
 "name": "roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "color": {
 "name": "color",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_role_id": {
 "name": "discord_role_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
```



**drizzle/meta/0006\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_system": {
 "name": "is_system",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "roles_name_unique": {
 "name": "roles_name_unique",
 "columns": [
 "name"
],
 "isUnique": true
 },
 "roles_discord_role_id_unique": {
 "name": "roles_discord_role_id_unique",
 "columns": [
 "discord_role_id"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"sessions": {
 "name": "sessions",
 "columns": {
 "id": {
```

**drizzle/meta/0006\_snapshot.json**

```
 "name": "id",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "expires_at": {
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_agent": {
 "name": "user_agent",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "sessions_user_idx": {
 "name": "sessions_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "sessions_expires_idx": {
 "name": "sessions_expires_idx",
 "columns": [
```

**drizzle/meta/0006\_snapshot.json**

```
 "expires_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "sessions_user_id_users_id_fk": {
 "name": "sessions_user_id_users_id_fk",
 "tableFrom": "sessions",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"settings": {
 "name": "settings",
 "columns": {
 "key": {
 "name": "key",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "value": {
 "name": "value",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
```

**drizzle/meta/0006\_snapshot.json**

```
 "default": "(unixepoch())"
 },
 "indexes": {},
 "foreignKeys": {
 "settings_updated_by_users_id_fk": {
 "name": "settings_updated_by_users_id_fk",
 "tableFrom": "settings",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"user_roles": {
 "name": "user_roles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'manual'"
 },
 "granted_by": {
 "name": "granted_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
}
```

**drizzle/meta/0006\_snapshot.json**

```
 },
 "granted_at": {
 "name": "granted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "user_roles_role_idx": {
 "name": "user_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "user_roles_user_id_users_id_fk": {
 "name": "user_roles_user_id_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_role_id_roles_id_fk": {
 "name": "user_roles_role_id_roles_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_granted_by_users_id_fk": {
 "name": "user_roles_granted_by_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "granted_by"
],
 "columnsTo": [
```

**drizzle/meta/0006\_snapshot.json**

```
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "user_roles_user_id_role_id_pk": {
 "columns": [
 "user_id",
 "role_id"
],
 "name": "user_roles_user_id_role_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"users": {
 "name": "users",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "discord_id": {
 "name": "discord_id",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "username": {
 "name": "username",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "global_name": {
 "name": "global_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "display_name": {
 "name": "display_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
```

**drizzle/meta/0006\_snapshot.json**

```
 "autoincrement": false
 },
 "avatar": {
 "name": "avatar",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "profile_image_url": {
 "name": "profile_image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "email": {
 "name": "email",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "is_god": {
 "name": "is_god",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "guild_joined_at": {
 "name": "guild_joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'active'"
 },
 "recruiter_id": {
```

**drizzle/meta/0006\_snapshot.json**

```
 "name": "recruiter_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "joined_at": {
 "name": "joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "last_seen_at": {
 "name": "last_seen_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "last_reconciled_at": {
 "name": "last_reconciled_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "promoted_at": {
 "name": "promoted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "users_discord_id_unique": {
```



**drizzle/meta/0006\_snapshot.json**

```
 "name": "users_discord_id_unique",
 "columns": [
 "discord_id"
],
 "isUnique": true
 },
 "users_rank_idx": {
 "name": "users_rank_idx",
 "columns": [
 "rank_id"
],
 "isUnique": false
 },
 "users_status_idx": {
 "name": "users_status_idx",
 "columns": [
 "status"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "users_rank_id_ranks_id_fk": {
 "name": "users_rank_id_ranks_id_fk",
 "tableFrom": "users",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"war_records": {
 "name": "war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
```

**drizzle/meta/0006\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "war_records_game_idx": {
 "name": "war_records_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "war_records_game_id_games_id_fk": {
 "name": "war_records_game_id_games_id_fk",
 "tableFrom": "war_records",
```

**drizzle/meta/0006\_snapshot.json**

```
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
}
},
"views": {},
"enums": {},
"_meta": {
 "schemas": {},
 "tables": {},
 "columns": {}
},
"internal": {
 "indexes": {}
}
}
```

**drizzle/meta/0007\_snapshot.json**

```
{
 "version": "6",
 "dialect": "sqlite",
 "id": "a82efe73-f3f9-45e7-abbe-c6fefce68bae",
 "prevId": "bde9b65e-52cf-44e2-84dc-98a310df7bd1",
 "tables": {
 "audit_log": {
 "name": "audit_log",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "actor_id": {
 "name": "actor_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "action": {
 "name": "action",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "target_type": {
 "name": "target_type",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "target_id": {
 "name": "target_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "meta": {
 "name": "meta",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
```

**drizzle/meta/0007\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'web'"
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "audit_actor_idx": {
 "name": "audit_actor_idx",
 "columns": [
 "actor_id"
],
 "isUnique": false
 },
 "audit_created_idx": {
 "name": "audit_created_idx",
 "columns": [
 "created_at"
],
 "isUnique": false
 },
 "audit_action_idx": {
 "name": "audit_action_idx",
 "columns": [
 "action"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "audit_log_actor_id_users_id_fk": {
 "name": "audit_log_actor_id_users_id_fk",
 "tableFrom": "audit_log",
 "tableTo": "users",
 "columnsFrom": [
 "actor_id"
],
 "columnsTo": [
 "id"
]
 }
}
```

**drizzle/meta/0007\_snapshot.json**

```
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"events": {
 "name": "events",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "starts_at": {
 "name": "starts_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "ends_at": {
 "name": "ends_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 }
},
```

**drizzle/meta/0007\_snapshot.json**

```
"game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"created_by": {
 "name": "created_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"discord_event_id": {
 "name": "discord_event_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"discord_message_id": {
 "name": "discord_message_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'scheduled'"
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
},
"updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
```

**drizzle/meta/0007\_snapshot.json**

```
 "events_starts_idx": {
 "name": "events_starts_idx",
 "columns": [
 "starts_at"
],
 "isUnique": false
 },
},
"foreignKeys": {
 "events_game_id_games_id_fk": {
 "name": "events_game_id_games_id_fk",
 "tableFrom": "events",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 },
 "events_created_by_users_id_fk": {
 "name": "events_created_by_users_id_fk",
 "tableFrom": "events",
 "tableTo": "users",
 "columnsFrom": [
 "created_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"games": {
 "name": "games",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
```



**drizzle/meta/0007\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "icon_url": {
 "name": "icon_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "active": {
 "name": "active",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": true
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "games_slug_unique": {
 "name": "games_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
```

**drizzle/meta/0007\_snapshot.json**

```
"checkConstraints": {},
},
"match_participants": {
 "name": "match_participants",
 "columns": {
 "match_id": {
 "name": "match_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "stats": {
 "name": "stats",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 },
 "indexes": {
 "mp_user_idx": {
 "name": "mp_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "match_participants_match_id_matches_id_fk": {
 "name": "match_participants_match_id_matches_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "matches",
 "columnsFrom": [
 "match_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "match_participants_user_id_users_id_fk": {
 "name": "match_participants_user_id_users_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "users",
```

**drizzle/meta/0007\_snapshot.json**

```
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "match_participants_match_id_user_id_pk": {
 "columns": [
 "match_id",
 "user_id"
],
 "name": "match_participants_match_id_user_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"matches": {
 "name": "matches",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent": {
 "name": "opponent",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent_tag": {
 "name": "opponent_tag",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 },
 "result": {
```

**drizzle/meta/0007\_snapshot.json**

```
 "name": "result",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "score_us": {
 "name": "score_us",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "score_them": {
 "name": "score_them",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "map": {
 "name": "map",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "notes": {
 "name": "notes",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "played_at": {
 "name": "played_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "recorded_by": {
 "name": "recorded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
```

**drizzle/meta/0007\_snapshot.json**

```

 "default": "(unixepoch())"
 },
 "indexes": {
 "matches_game_played_idx": {
 "name": "matches_game_played_idx",
 "columns": [
 "game_id",
 "played_at"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "matches_game_id_games_id_fk": {
 "name": "matches_game_id_games_id_fk",
 "tableFrom": "matches",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "matches_recorded_by_users_id_fk": {
 "name": "matches_recorded_by_users_id_fk",
 "tableFrom": "matches",
 "tableTo": "users",
 "columnsFrom": [
 "recorded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"medals": {
 "name": "medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 }
 }
}

```

**drizzle/meta/0007\_snapshot.json**

```
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "auto_grant_months": {
 "name": "auto_grant_months",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
```

**drizzle/meta/0007\_snapshot.json**

```
 "medals_game_idx": {
 "name": "medals_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 },
 "foreignKeys": {
 "medals_game_id_games_id_fk": {
 "name": "medals_game_id_games_id_fk",
 "tableFrom": "medals",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"member_medals": {
 "name": "member_medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "medal_id": {
 "name": "medal_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
```

**drizzle/meta/0007\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "member_medals_user_idx": {
 "name": "member_medals_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "member_medals_medal_idx": {
 "name": "member_medals_medal_idx",
 "columns": [
 "medal_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "member_medals_user_id_users_id_fk": {
 "name": "member_medals_user_id_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_medal_id_medals_id_fk": {
 "name": "member_medals_medal_id_medals_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "medals",
```



**drizzle/meta/0007\_snapshot.json**

```
 "columnsFrom": [
 "medal_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_awarded_by_users_id_fk": {
 "name": "member_medals_awarded_by_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"member_war_records": {
 "name": "member_war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "war_record_id": {
 "name": "war_record_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
```

**drizzle/meta/0007\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "member_war_records_user_idx": {
 "name": "member_war_records_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "member_war_records_record_idx": {
 "name": "member_war_records_record_idx",
 "columns": [
 "war_record_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "member_war_records_user_id_users_id_fk": {
 "name": "member_war_records_user_id_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_war_record_id_war_records_id_fk": {
 "name": "member_war_records_war_record_id_war_records_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "war_records",
```

**drizzle/meta/0007\_snapshot.json**

```
 "columnsFrom": [
 "war_record_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_awarded_by_users_id_fk": {
 "name": "member_war_records_awarded_by_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"news": {
 "name": "news",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "excerpt": {
 "name": "excerpt",
 "type": "text",
```

**drizzle/meta/0007\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "body": {
 "name": "body",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "author_id": {
 "name": "author_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'draft'"
 },
 "pinned": {
 "name": "pinned",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "published_at": {
 "name": "published_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
```

**drizzle/meta/0007\_snapshot.json**

```
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "news_slug_unique": {
 "name": "news_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 },
 "news_status_published_idx": {
 "name": "news_status_published_idx",
 "columns": [
 "status",
 "published_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "news_author_id_users_id_fk": {
 "name": "news_author_id_users_id_fk",
 "tableFrom": "news",
 "tableTo": "users",
 "columnsFrom": [
 "author_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"profiles": {
 "name": "profiles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "bio": {
 "name": "bio",
 "type": "text",
 "primaryKey": false,
```

**drizzle/meta/0007\_snapshot.json**

```
 "notNull": false,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "timezone": {
 "name": "timezone",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "gamertags": {
 "name": "gamertags",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {},
"foreignKeys": {
 "profiles_user_id_users_id_fk": {
 "name": "profiles_user_id_users_id_fk",
 "tableFrom": "profiles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"rank_roles": {
```

**drizzle/meta/0007\_snapshot.json**

```
"name": "rank_roles",
"columns": {
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
},
"indexes": {
 "rank_roles_role_idx": {
 "name": "rank_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "rank_roles_rank_id_ranks_id_fk": {
 "name": "rank_roles_rank_id_ranks_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "rank_roles_role_id_roles_id_fk": {
 "name": "rank_roles_role_id_roles_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
},
```

**drizzle/meta/0007\_snapshot.json**

```
"compositePrimaryKeys": {
 "rank_roles_rank_id_role_id_pk": {
 "columns": [
 "rank_id",
 "role_id"
],
 "name": "rank_roles_rank_id_role_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"ranks": {
 "name": "ranks",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "abbreviation": {
 "name": "abbreviation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "req_days": {
 "name": "req_days",
 "type": "integer",
 "primaryKey": false,
```



**drizzle/meta/0007\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "req_wins": {
 "name": "req_wins",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_default": {
 "name": "is_default",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "ranks_sort_order_idx": {
 "name": "ranks_sort_order_idx",
 "columns": [
 "sort_order"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"role_permissions": {
 "name": "role_permissions",
 "columns": {
```

**drizzle/meta/0007\_snapshot.json**

```

 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "permission": {
 "name": "permission",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "indexes": {},
 "foreignKeys": {
 "role_permissions_role_id_roles_id_fk": {
 "name": "role_permissions_role_id_roles_id_fk",
 "tableFrom": "role_permissions",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {
 "role_permissions_role_id_permission_pk": {
 "columns": [
 "role_id",
 "permission"
],
 "name": "role_permissions_role_id_permission_pk"
 }
 },
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"roles": {
 "name": "roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {

```

**drizzle/meta/0007\_snapshot.json**

```
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "color": {
 "name": "color",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_role_id": {
 "name": "discord_role_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_system": {
 "name": "is_system",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
```

**drizzle/meta/0007\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "roles_name_unique": {
 "name": "roles_name_unique",
 "columns": [
 "name"
],
 "isUnique": true
 },
 "roles_discord_role_id_unique": {
 "name": "roles_discord_role_id_unique",
 "columns": [
 "discord_role_id"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"sessions": {
 "name": "sessions",
 "columns": {
 "id": {
 "name": "id",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "expires_at": {
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_agent": {
 "name": "user_agent",
 "type": "text",
```

**drizzle/meta/0007\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "sessions_user_idx": {
 "name": "sessions_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "sessions_expires_idx": {
 "name": "sessions_expires_idx",
 "columns": [
 "expires_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "sessions_user_id_users_id_fk": {
 "name": "sessions_user_id_users_id_fk",
 "tableFrom": "sessions",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
```

**drizzle/meta/0007\_snapshot.json**

```
{,
 "settings": {
 "name": "settings",
 "columns": {
 "key": {
 "name": "key",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "value": {
 "name": "value",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 }
 },
 "indexes": {},
 "foreignKeys": {
 "settings_updated_by_users_id_fk": {
 "name": "settings_updated_by_users_id_fk",
 "tableFrom": "settings",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
```

**drizzle/meta/0007\_snapshot.json**

```
"user_roles": {
 "name": "user_roles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'manual'"
 },
 "granted_by": {
 "name": "granted_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "granted_at": {
 "name": "granted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "user_roles_role_idx": {
 "name": "user_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "user_roles_user_id_users_id_fk": {
 "name": "user_roles_user_id_users_id_fk",
 "tableFrom": "user_roles",
```

**drizzle/meta/0007\_snapshot.json**

```

 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_role_id_roles_id_fk": {
 "name": "user_roles_role_id_roles_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_granted_by_users_id_fk": {
 "name": "user_roles_granted_by_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "granted_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "user_roles_user_id_role_id_pk": {
 "columns": [
 "user_id",
 "role_id"
],
 "name": "user_roles_user_id_role_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"users": {
 "name": "users",
 "columns": {
 "id": {
 "name": "id",

```



**drizzle/meta/0007\_snapshot.json**

```
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "discord_id": {
 "name": "discord_id",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "username": {
 "name": "username",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "global_name": {
 "name": "global_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "display_name": {
 "name": "display_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "avatar": {
 "name": "avatar",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "profile_image_url": {
 "name": "profile_image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "email": {
 "name": "email",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
}
```

**drizzle/meta/0007\_snapshot.json**

```
"rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"is_god": {
 "name": "is_god",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
},
"guild_joined_at": {
 "name": "guild_joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'active'"
},
"recruiter_id": {
 "name": "recruiter_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"joined_at": {
 "name": "joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
},
"last_seen_at": {
 "name": "last_seen_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"last_reconciled_at": {
 "name": "last_reconciled_at",
```

**drizzle/meta/0007\_snapshot.json**

```
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "promoted_at": {
 "name": "promoted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "users_discord_id_unique": {
 "name": "users_discord_id_unique",
 "columns": [
 "discord_id"
],
 "isUnique": true
 },
 "users_rank_idx": {
 "name": "users_rank_idx",
 "columns": [
 "rank_id"
],
 "isUnique": false
 },
 "users_status_idx": {
 "name": "users_status_idx",
 "columns": [
 "status"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "users_rank_id_ranks_id_fk": {
```

**drizzle/meta/0007\_snapshot.json**

```
 "name": "users_rank_id_ranks_id_fk",
 "tableFrom": "users",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"war_records": {
 "name": "war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
}
```

**drizzle/meta/0007\_snapshot.json**

```

 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "war_records_game_idx": {
 "name": "war_records_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "war_records_game_id_games_id_fk": {
 "name": "war_records_game_id_games_id_fk",
 "tableFrom": "war_records",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"views": {},
"enums": {},
"_meta": {
 "schemas": {},
 "tables": {},
 "columns": {}
},

```

**drizzle/meta/0007\_snapshot.json**

```
"internal": {
 "indexes": {}
}
}
```

**drizzle/meta/0008\_snapshot.json**

```
{
 "version": "6",
 "dialect": "sqlite",
 "id": "958552f2-2d3e-4dd0-90a7-290c5701ab70",
 "prevId": "a82efe73-f3f9-45e7-abbe-c6fefce68bae",
 "tables": {
 "audit_log": {
 "name": "audit_log",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "actor_id": {
 "name": "actor_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "action": {
 "name": "action",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "target_type": {
 "name": "target_type",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "target_id": {
 "name": "target_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "reason": {
 "name": "reason",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "meta": {
 "name": "meta",
 "type": "text",
```

**drizzle/meta/0008\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'web'"
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "audit_actor_idx": {
 "name": "audit_actor_idx",
 "columns": [
 "actor_id"
],
 "isUnique": false
 },
 "audit_created_idx": {
 "name": "audit_created_idx",
 "columns": [
 "created_at"
],
 "isUnique": false
 },
 "audit_action_idx": {
 "name": "audit_action_idx",
 "columns": [
 "action"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "audit_log_actor_id_users_id_fk": {
 "name": "audit_log_actor_id_users_id_fk",
```



**drizzle/meta/0008\_snapshot.json**

```
 "tableFrom": "audit_log",
 "tableTo": "users",
 "columnsFrom": [
 "actor_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"events": {
 "name": "events",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "starts_at": {
 "name": "starts_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "ends_at": {
 "name": "ends_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 }
},
```

**drizzle/meta/0008\_snapshot.json**

```
"location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"created_by": {
 "name": "created_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"discord_event_id": {
 "name": "discord_event_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"discord_message_id": {
 "name": "discord_message_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'scheduled'"
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
},
"updated_at": {
 "name": "updated_at",
 "type": "integer",
```

**drizzle/meta/0008\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "events_starts_idx": {
 "name": "events_starts_idx",
 "columns": [
 "starts_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "events_game_id_games_id_fk": {
 "name": "events_game_id_games_id_fk",
 "tableFrom": "events",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 },
 "events_created_by_users_id_fk": {
 "name": "events_created_by_users_id_fk",
 "tableFrom": "events",
 "tableTo": "users",
 "columnsFrom": [
 "created_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"games": {
 "name": "games",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
```

**drizzle/meta/0008\_snapshot.json**

```
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "icon_url": {
 "name": "icon_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "active": {
 "name": "active",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": true
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "games_slug_unique": {
 "name": "games_slug_unique",
 "columns": [
 "slug"
]
 }
}
```

**drizzle/meta/0008\_snapshot.json**

```
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"match_participants": {
 "name": "match_participants",
 "columns": {
 "match_id": {
 "name": "match_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "stats": {
 "name": "stats",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 },
 "indexes": {
 "mp_user_idx": {
 "name": "mp_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "match_participants_match_id_matches_id_fk": {
 "name": "match_participants_match_id_matches_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "matches",
 "columnsFrom": [
 "match_id"
],
 "columnsTo": [
 "id"
]
 }
 },
}
```

**drizzle/meta/0008\_snapshot.json**

```
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "match_participants_user_id_users_id_fk": {
 "name": "match_participants_user_id_users_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {
 "match_participants_match_id_user_id_pk": {
 "columns": [
 "match_id",
 "user_id"
],
 "name": "match_participants_match_id_user_id_pk"
 }
 },
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"matches": {
 "name": "matches",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent": {
 "name": "opponent",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent_tag": {
```

**drizzle/meta/0008\_snapshot.json**

```
 "name": "opponent_tag",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "result": {
 "name": "result",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "score_us": {
 "name": "score_us",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "score_them": {
 "name": "score_them",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "map": {
 "name": "map",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "notes": {
 "name": "notes",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "played_at": {
 "name": "played_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "recorded_by": {
 "name": "recorded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
```

**drizzle/meta/0008\_snapshot.json**

```
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "matches_game_played_idx": {
 "name": "matches_game_played_idx",
 "columns": [
 "game_id",
 "played_at"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "matches_game_id_games_id_fk": {
 "name": "matches_game_id_games_id_fk",
 "tableFrom": "matches",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "matches_recorded_by_users_id_fk": {
 "name": "matches_recorded_by_users_id_fk",
 "tableFrom": "matches",
 "tableTo": "users",
 "columnsFrom": [
 "recorded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"medals": {
 "name": "medals",
```



**drizzle/meta/0008\_snapshot.json**

```
"columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "auto_grant_months": {
 "name": "auto_grant_months",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
```

**drizzle/meta/0008\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "medals_game_idx": {
 "name": "medals_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "medals_game_id_games_id_fk": {
 "name": "medals_game_id_games_id_fk",
 "tableFrom": "medals",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"member_medals": {
 "name": "member_medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "medal_id": {
 "name": "medal_id",
 "type": "integer",
```

**drizzle/meta/0008\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "member_medals_user_idx": {
 "name": "member_medals_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "member_medals_medal_idx": {
 "name": "member_medals_medal_idx",
 "columns": [
 "medal_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "member_medals_user_id_users_id_fk": {
 "name": "member_medals_user_id_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 },
}
```

**drizzle/meta/0008\_snapshot.json**

```
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_medal_id_medals_id_fk": {
 "name": "member_medals_medal_id_medals_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "medals",
 "columnsFrom": [
 "medal_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_awarded_by_users_id_fk": {
 "name": "member_medals_awarded_by_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"member_war_records": {
 "name": "member_war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "war_record_id": {
 "name": "war_record_id",
 "type": "integer",
```

**drizzle/meta/0008\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "member_war_records_user_idx": {
 "name": "member_war_records_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "member_war_records_record_idx": {
 "name": "member_war_records_record_idx",
 "columns": [
 "war_record_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "member_war_records_user_id_users_id_fk": {
 "name": "member_war_records_user_id_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 },
}
```

**drizzle/meta/0008\_snapshot.json**

```
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_war_record_id_war_records_id_fk": {
 "name": "member_war_records_war_record_id_war_records_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "war_records",
 "columnsFrom": [
 "war_record_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_awarded_by_users_id_fk": {
 "name": "member_war_records_awarded_by_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"news": {
 "name": "news",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
```

**drizzle/meta/0008\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "excerpt": {
 "name": "excerpt",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "body": {
 "name": "body",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "author_id": {
 "name": "author_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'draft'"
 },
 "pinned": {
 "name": "pinned",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "published_at": {
 "name": "published_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
```

**drizzle/meta/0008\_snapshot.json**

```
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "news_slug_unique": {
 "name": "news_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 },
 "news_status_published_idx": {
 "name": "news_status_published_idx",
 "columns": [
 "status",
 "published_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "news_author_id_users_id_fk": {
 "name": "news_author_id_users_id_fk",
 "tableFrom": "news",
 "tableTo": "users",
 "columnsFrom": [
 "author_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"profiles": {
 "name": "profiles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
```



**drizzle/meta/0008\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false
 },
 "bio": {
 "name": "bio",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "timezone": {
 "name": "timezone",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "gamertags": {
 "name": "gamertags",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {},
"foreignKeys": {
 "profiles_user_id_users_id_fk": {
 "name": "profiles_user_id_users_id_fk",
 "tableFrom": "profiles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
}
```

**drizzle/meta/0008\_snapshot.json**

```
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
 },
 "rank_roles": {
 "name": "rank_roles",
 "columns": {
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 }
 },
 "indexes": {
 "rank_roles_role_idx": {
 "name": "rank_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "rank_roles_rank_id_ranks_id_fk": {
 "name": "rank_roles_rank_id_ranks_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "rank_roles_role_id_roles_id_fk": {
 "name": "rank_roles_role_id_roles_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 },
 },
}
```

**drizzle/meta/0008\_snapshot.json**

```
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "rank_roles_rank_id_role_id_pk": {
 "columns": [
 "rank_id",
 "role_id"
],
 "name": "rank_roles_rank_id_role_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"ranks": {
 "name": "ranks",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "abbreviation": {
 "name": "abbreviation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
```

**drizzle/meta/0008\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false
 },
 "req_days": {
 "name": "req_days",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "req_wins": {
 "name": "req_wins",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_default": {
 "name": "is_default",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "ranks_sort_order_idx": {
 "name": "ranks_sort_order_idx",
 "columns": [
 "sort_order"
],
 "isUnique": true
 }
},
"foreignKeys": {},
```

**drizzle/meta/0008\_snapshot.json**

```
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"role_permissions": {
 "name": "role_permissions",
 "columns": {
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "permission": {
 "name": "permission",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 }
},
"indexes": {},
"foreignKeys": {
 "role_permissions_role_id_roles_id_fk": {
 "name": "role_permissions_role_id_roles_id_fk",
 "tableFrom": "role_permissions",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "role_permissions_role_id_permission_pk": {
 "columns": [
 "role_id",
 "permission"
],
 "name": "role_permissions_role_id_permission_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"roles": {
 "name": "roles",
 "columns": {
 "id": {
```

**drizzle/meta/0008\_snapshot.json**

```
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "color": {
 "name": "color",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_role_id": {
 "name": "discord_role_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_system": {
 "name": "is_system",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
```

**drizzle/meta/0008\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "roles_name_unique": {
 "name": "roles_name_unique",
 "columns": [
 "name"
],
 "isUnique": true
 },
 "roles_discord_role_id_unique": {
 "name": "roles_discord_role_id_unique",
 "columns": [
 "discord_role_id"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"sessions": {
 "name": "sessions",
 "columns": {
 "id": {
 "name": "id",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "expires_at": {
 "name": "expires_at",
 "type": "integer",
```

**drizzle/meta/0008\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_agent": {
 "name": "user_agent",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "sessions_user_idx": {
 "name": "sessions_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "sessions_expires_idx": {
 "name": "sessions_expires_idx",
 "columns": [
 "expires_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "sessions_user_id_users_id_fk": {
 "name": "sessions_user_id_users_id_fk",
 "tableFrom": "sessions",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 },
}
```



**drizzle/meta/0008\_snapshot.json**

```
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"settings": {
 "name": "settings",
 "columns": {
 "key": {
 "name": "key",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "value": {
 "name": "value",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 }
},
"indexes": {},
"foreignKeys": {
 "settings_updated_by_users_id_fk": {
 "name": "settings_updated_by_users_id_fk",
 "tableFrom": "settings",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
```

**drizzle/meta/0008\_snapshot.json**

```
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"user_roles": {
 "name": "user_roles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'manual'"
 },
 "granted_by": {
 "name": "granted_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "granted_at": {
 "name": "granted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 }
},
"indexes": {
 "user_roles_role_idx": {
 "name": "user_roles_role_idx",
 "columns": [
 "role_id"
]
 },
```

**drizzle/meta/0008\_snapshot.json**

```
 "isUnique": false
 },
 "foreignKeys": {
 "user_roles_user_id_users_id_fk": {
 "name": "user_roles_user_id_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_role_id_roles_id_fk": {
 "name": "user_roles_role_id_roles_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_granted_by_users_id_fk": {
 "name": "user_roles_granted_by_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "granted_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {
 "user_roles_user_id_role_id_pk": {
 "columns": [
 "user_id",
 "role_id"
],
 "name": "user_roles_user_id_role_id_pk"
 }
 },
 "uniqueConstraints": {},
```

**drizzle/meta/0008\_snapshot.json**

```
"checkConstraints": {}
},
"users": {
 "name": "users",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "discord_id": {
 "name": "discord_id",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "username": {
 "name": "username",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "global_name": {
 "name": "global_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "display_name": {
 "name": "display_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "avatar": {
 "name": "avatar",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "profile_image_url": {
 "name": "profile_image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
},
```

**drizzle/meta/0008\_snapshot.json**

```
"email": {
 "name": "email",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"is_god": {
 "name": "is_god",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
},
"guild_joined_at": {
 "name": "guild_joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'active'"
},
"recruiter_id": {
 "name": "recruiter_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"joined_at": {
 "name": "joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
},
"last_seen_at": {
 "name": "last_seen_at",
```

**drizzle/meta/0008\_snapshot.json**

```
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "last_reconciled_at": {
 "name": "last_reconciled_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "promoted_at": {
 "name": "promoted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "users_discord_id_unique": {
 "name": "users_discord_id_unique",
 "columns": [
 "discord_id"
],
 "isUnique": true
 },
 "users_rank_idx": {
 "name": "users_rank_idx",
 "columns": [
 "rank_id"
],
 "isUnique": false
 },
 "users_status_idx": {
 "name": "users_status_idx",
 "columns": [
```

**drizzle/meta/0008\_snapshot.json**

```
 "status"
],
 "isUnique": false
}
},
"foreignKeys": {
 "users_rank_id_ranks_id_fk": {
 "name": "users_rank_id_ranks_id_fk",
 "tableFrom": "users",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"war_records": {
 "name": "war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
}
```

**drizzle/meta/0008\_snapshot.json**

```

 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "war_records_game_idx": {
 "name": "war_records_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "war_records_game_id_games_id_fk": {
 "name": "war_records_game_id_games_id_fk",
 "tableFrom": "war_records",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
}

```



**drizzle/meta/0008\_snapshot.json**

```
"views": {},
"enums": {},
"_meta": {
 "schemas": {},
 "tables": {},
 "columns": {}
},
"internal": {
 "indexes": {}
}
}
```

**drizzle/meta/0009\_snapshot.json**

```
{
 "version": "6",
 "dialect": "sqlite",
 "id": "64e807d1-0423-4bb7-9167-1e074d985790",
 "prevId": "958552f2-2d3e-4dd0-90a7-290c5701ab70",
 "tables": {
 "audit_log": {
 "name": "audit_log",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "actor_id": {
 "name": "actor_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "action": {
 "name": "action",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "target_type": {
 "name": "target_type",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "target_id": {
 "name": "target_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "reason": {
 "name": "reason",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "meta": {
 "name": "meta",
 "type": "text",
```

**drizzle/meta/0009\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'web'"
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "audit_actor_idx": {
 "name": "audit_actor_idx",
 "columns": [
 "actor_id"
],
 "isUnique": false
 },
 "audit_created_idx": {
 "name": "audit_created_idx",
 "columns": [
 "created_at"
],
 "isUnique": false
 },
 "audit_action_idx": {
 "name": "audit_action_idx",
 "columns": [
 "action"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "audit_log_actor_id_users_id_fk": {
 "name": "audit_log_actor_id_users_id_fk",
```

**drizzle/meta/0009\_snapshot.json**

```
 "tableFrom": "audit_log",
 "tableTo": "users",
 "columnsFrom": [
 "actor_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"event_roles": {
 "name": "event_roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "event_id": {
 "name": "event_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "emoji": {
 "name": "emoji",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "capacity": {
 "name": "capacity",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
},
```

**drizzle/meta/0009\_snapshot.json**

```
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 }
 },
 "indexes": {
 "event_roles_event_idx": {
 "name": "event_roles_event_idx",
 "columns": [
 "event_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "event_roles_event_id_events_id_fk": {
 "name": "event_roles_event_id_events_id_fk",
 "tableFrom": "event_roles",
 "tableTo": "events",
 "columnsFrom": [
 "event_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"event_signups": {
 "name": "event_signups",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "event_id": {
 "name": "event_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
}
```

**drizzle/meta/0009\_snapshot.json**

```
"user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"event_role_id": {
 "name": "event_role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
},
"updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
 "event_signups_event_user_idx": {
 "name": "event_signups_event_user_idx",
 "columns": [
 "event_id",
 "user_id"
],
 "isUnique": true
 },
 "event_signups_event_idx": {
 "name": "event_signups_event_idx",
 "columns": [
 "event_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "event_signups_event_id_events_id_fk": {
 "name": "event_signups_event_id_events_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "events",
 "columnsFrom": [
```

**drizzle/meta/0009\_snapshot.json**

```

 "event_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
},
"event_signups_user_id_users_id_fk": {
 "name": "event_signups_user_id_users_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
},
"event_signups_event_role_id_event_roles_id_fk": {
 "name": "event_signups_event_role_id_event_roles_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "event_roles",
 "columnsFrom": [
 "event_role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
}
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"events": {
 "name": "events",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,

```

**drizzle/meta/0009\_snapshot.json**

```
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "starts_at": {
 "name": "starts_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "ends_at": {
 "name": "ends_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_by": {
 "name": "created_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_event_id": {
 "name": "discord_event_id",
 "type": "text",
```



**drizzle/meta/0009\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_message_id": {
 "name": "discord_message_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'scheduled'"
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "events_starts_idx": {
 "name": "events_starts_idx",
 "columns": [
 "starts_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "events_game_id_games_id_fk": {
 "name": "events_game_id_games_id_fk",
 "tableFrom": "events",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
```

**drizzle/meta/0009\_snapshot.json**

```
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 },
 "events_created_by_users_id_fk": {
 "name": "events_created_by_users_id_fk",
 "tableFrom": "events",
 "tableTo": "users",
 "columnsFrom": [
 "created_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"games": {
 "name": "games",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "icon_url": {
 "name": "icon_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
},
```

**drizzle/meta/0009\_snapshot.json**

```
"sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
},
"active": {
 "name": "active",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": true
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
 "games_slug_unique": {
 "name": "games_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"match_participants": {
 "name": "match_participants",
 "columns": {
 "match_id": {
 "name": "match_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
```

**drizzle/meta/0009\_snapshot.json**

```
 "autoincrement": false
 },
 "stats": {
 "name": "stats",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
},
"indexes": {
 "mp_user_idx": {
 "name": "mp_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "match_participants_match_id_matches_id_fk": {
 "name": "match_participants_match_id_matches_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "matches",
 "columnsFrom": [
 "match_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "match_participants_user_id_users_id_fk": {
 "name": "match_participants_user_id_users_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "match_participants_match_id_user_id_pk": {
 "columns": [
 "match_id",
 "user_id"
],
 "name": "match_participants_match_id_user_id_pk"
 }
}
```

**drizzle/meta/0009\_snapshot.json**

```
 }
 },
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"matches": {
 "name": "matches",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent": {
 "name": "opponent",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent_tag": {
 "name": "opponent_tag",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "result": {
 "name": "result",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "score_us": {
 "name": "score_us",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "score_them": {
 "name": "score_them",
 "type": "integer",
 "primaryKey": false,
```

**drizzle/meta/0009\_snapshot.json**

```
 "notNull": false,
 "autoincrement": false
 },
 "map": {
 "name": "map",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "notes": {
 "name": "notes",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "played_at": {
 "name": "played_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "recorded_by": {
 "name": "recorded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "matches_game_played_idx": {
 "name": "matches_game_played_idx",
 "columns": [
 "game_id",
 "played_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "matches_game_id_games_id_fk": {
 "name": "matches_game_id_games_id_fk",
 "tableFrom": "matches",
```

**drizzle/meta/0009\_snapshot.json**

```
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "matches_recorded_by_users_id_fk": {
 "name": "matches_recorded_by_users_id_fk",
 "tableFrom": "matches",
 "tableTo": "users",
 "columnsFrom": [
 "recorded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"medals": {
 "name": "medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
```

**drizzle/meta/0009\_snapshot.json**

```
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "auto_grant_months": {
 "name": "auto_grant_months",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "medals_game_idx": {
 "name": "medals_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "medals_game_id_games_id_fk": {
 "name": "medals_game_id_games_id_fk",
 "tableFrom": "medals",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
```



**drizzle/meta/0009\_snapshot.json**

```
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"member_medals": {
 "name": "member_medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "medal_id": {
 "name": "medal_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 }
 }
}
```

**drizzle/meta/0009\_snapshot.json**

```
 "default": "(unixepoch())"
 },
 "indexes": {
 "member_medals_user_idx": {
 "name": "member_medals_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "member_medals_medal_idx": {
 "name": "member_medals_medal_idx",
 "columns": [
 "medal_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "member_medals_user_id_users_id_fk": {
 "name": "member_medals_user_id_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_medal_id_medals_id_fk": {
 "name": "member_medals_medal_id_medals_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "medals",
 "columnsFrom": [
 "medal_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_awarded_by_users_id_fk": {
 "name": "member_medals_awarded_by_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
```

**drizzle/meta/0009\_snapshot.json**

```
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"member_war_records": {
 "name": "member_war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "war_record_id": {
 "name": "war_record_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 }
 }
}
```

**drizzle/meta/0009\_snapshot.json**

```
 "default": "(unixepoch())"
 },
 "indexes": {
 "member_war_records_user_idx": {
 "name": "member_war_records_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "member_war_records_record_idx": {
 "name": "member_war_records_record_idx",
 "columns": [
 "war_record_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "member_war_records_user_id_users_id_fk": {
 "name": "member_war_records_user_id_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_war_record_id_war_records_id_fk": {
 "name": "member_war_records_war_record_id_war_records_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "war_records",
 "columnsFrom": [
 "war_record_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_awarded_by_users_id_fk": {
 "name": "member_war_records_awarded_by_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
```

**drizzle/meta/0009\_snapshot.json**

```
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"news": {
 "name": "news",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "excerpt": {
 "name": "excerpt",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "body": {
 "name": "body",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "author_id": {
 "name": "author_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
}
```

**drizzle/meta/0009\_snapshot.json**

```
{
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'draft'"
 },
 "pinned": {
 "name": "pinned",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "published_at": {
 "name": "published_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "news_slug_unique": {
 "name": "news_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 },
 "news_status_published_idx": {
 "name": "news_status_published_idx",
 "columns": [
 "status",
 "published_at"
]
 }
}
```

**drizzle/meta/0009\_snapshot.json**

```
],
 "isUnique": false
 }
},
"foreignKeys": {
 "news_author_id_users_id_fk": {
 "name": "news_author_id_users_id_fk",
 "tableFrom": "news",
 "tableTo": "users",
 "columnsFrom": [
 "author_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"profiles": {
 "name": "profiles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "bio": {
 "name": "bio",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "timezone": {
 "name": "timezone",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
},
```

**drizzle/meta/0009\_snapshot.json**

```
"gamertags": {
 "name": "gamertags",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {},
"foreignKeys": {
 "profiles_user_id_users_id_fk": {
 "name": "profiles_user_id_users_id_fk",
 "tableFrom": "profiles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"rank_roles": {
 "name": "rank_roles",
 "columns": {
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 }
},
},
```



**drizzle/meta/0009\_snapshot.json**

```
"indexes": {
 "rank_roles_role_idx": {
 "name": "rank_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "rank_roles_rank_id_ranks_id_fk": {
 "name": "rank_roles_rank_id_ranks_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "rank_roles_role_id_roles_id_fk": {
 "name": "rank_roles_role_id_roles_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "rank_roles_rank_id_role_id_pk": {
 "columns": [
 "rank_id",
 "role_id"
],
 "name": "rank_roles_rank_id_role_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"ranks": {
 "name": "ranks",
 "columns": {
 "id": {
 "name": "id",
```

**drizzle/meta/0009\_snapshot.json**

```
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "abbreviation": {
 "name": "abbreviation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "req_days": {
 "name": "req_days",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "req_wins": {
 "name": "req_wins",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_default": {
 "name": "is_default",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
```

**drizzle/meta/0009\_snapshot.json**

```
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "ranks_sort_order_idx": {
 "name": "ranks_sort_order_idx",
 "columns": [
 "sort_order"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"role_permissions": {
 "name": "role_permissions",
 "columns": {
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "permission": {
 "name": "permission",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 }
},
"indexes": {},
"foreignKeys": {
```

**drizzle/meta/0009\_snapshot.json**

```
 "role_permissions_role_id_roles_id_fk": {
 "name": "role_permissions_role_id_roles_id_fk",
 "tableFrom": "role_permissions",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
},
"compositePrimaryKeys": {
 "role_permissions_role_id_permission_pk": {
 "columns": [
 "role_id",
 "permission"
],
 "name": "role_permissions_role_id_permission_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"roles": {
 "name": "roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "color": {
 "name": "color",
 "type": "text",
 "primaryKey": false,
```

**drizzle/meta/0009\_snapshot.json**

```
 "notNull": false,
 "autoincrement": false
 },
 "discord_role_id": {
 "name": "discord_role_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_system": {
 "name": "is_system",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "roles_name_unique": {
 "name": "roles_name_unique",
 "columns": [
 "name"
],
 "isUnique": true
 },
 "roles_discord_role_id_unique": {
 "name": "roles_discord_role_id_unique",
 "columns": [
```

**drizzle/meta/0009\_snapshot.json**

```
 "discord_role_id"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"sessions": {
 "name": "sessions",
 "columns": {
 "id": {
 "name": "id",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "expires_at": {
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_agent": {
 "name": "user_agent",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 }
 }
}
```

**drizzle/meta/0009\_snapshot.json**

```
 "default": "(unixepoch())"
 },
 "indexes": {
 "sessions_user_idx": {
 "name": "sessions_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "sessions_expires_idx": {
 "name": "sessions_expires_idx",
 "columns": [
 "expires_at"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "sessions_user_id_users_id_fk": {
 "name": "sessions_user_id_users_id_fk",
 "tableFrom": "sessions",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"settings": {
 "name": "settings",
 "columns": {
 "key": {
 "name": "key",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "value": {
 "name": "value",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 }
}
```

**drizzle/meta/0009\_snapshot.json**

```
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {},
 "foreignKeys": {
 "settings_updated_by_users_id_fk": {
 "name": "settings_updated_by_users_id_fk",
 "tableFrom": "settings",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"user_roles": {
 "name": "user_roles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 }
},
```



**drizzle/meta/0009\_snapshot.json**

```
"source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'manual'"
},
"granted_by": {
 "name": "granted_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"granted_at": {
 "name": "granted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
 "user_roles_role_idx": {
 "name": "user_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "user_roles_user_id_users_id_fk": {
 "name": "user_roles_user_id_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_role_id_roles_id_fk": {
 "name": "user_roles_role_id_roles_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 },
}
```

**drizzle/meta/0009\_snapshot.json**

```
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_granted_by_users_id_fk": {
 "name": "user_roles_granted_by_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "granted_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "user_roles_user_id_role_id_pk": {
 "columns": [
 "user_id",
 "role_id"
],
 "name": "user_roles_user_id_role_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"users": {
 "name": "users",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "discord_id": {
 "name": "discord_id",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "username": {
 "name": "username",
 "type": "text",
 "primaryKey": false,
 "notNull": true,

```

**drizzle/meta/0009\_snapshot.json**

```
 "autoincrement": false
 },
 "global_name": {
 "name": "global_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "display_name": {
 "name": "display_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "avatar": {
 "name": "avatar",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "profile_image_url": {
 "name": "profile_image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "email": {
 "name": "email",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "is_god": {
 "name": "is_god",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "guild_joined_at": {
 "name": "guild_joined_at",
```

**drizzle/meta/0009\_snapshot.json**

```
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'active'"
 },
 "recruiter_id": {
 "name": "recruiter_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "joined_at": {
 "name": "joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "last_seen_at": {
 "name": "last_seen_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "last_reconciled_at": {
 "name": "last_reconciled_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "promoted_at": {
 "name": "promoted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
```

**drizzle/meta/0009\_snapshot.json**

```
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "users_discord_id_unique": {
 "name": "users_discord_id_unique",
 "columns": [
 "discord_id"
],
 "isUnique": true
 },
 "users_rank_idx": {
 "name": "users_rank_idx",
 "columns": [
 "rank_id"
],
 "isUnique": false
 },
 "users_status_idx": {
 "name": "users_status_idx",
 "columns": [
 "status"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "users_rank_id_ranks_id_fk": {
 "name": "users_rank_id_ranks_id_fk",
 "tableFrom": "users",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
```

**drizzle/meta/0009\_snapshot.json**

```
"war_records": {
 "name": "war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 }
}
```

**drizzle/meta/0009\_snapshot.json**

```
 },
 "indexes": {
 "war_records_game_idx": {
 "name": "war_records_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "war_records_game_id_games_id_fk": {
 "name": "war_records_game_id_games_id_fk",
 "tableFrom": "war_records",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"views": {},
"enums": {},
"_meta": {
 "schemas": {},
 "tables": {},
 "columns": {}
},
"internal": {
 "indexes": {}
}
}
```

**drizzle/meta/0010\_snapshot.json**

```
{
 "version": "6",
 "dialect": "sqlite",
 "id": "5bae322e-8ce7-4eb5-961b-1f5eae414f03",
 "prevId": "64e807d1-0423-4bb7-9167-1e074d985790",
 "tables": {
 "audit_log": {
 "name": "audit_log",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "actor_id": {
 "name": "actor_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "action": {
 "name": "action",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "target_type": {
 "name": "target_type",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "target_id": {
 "name": "target_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "reason": {
 "name": "reason",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "meta": {
 "name": "meta",
 "type": "text",
```



**drizzle/meta/0010\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'web'"
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "audit_actor_idx": {
 "name": "audit_actor_idx",
 "columns": [
 "actor_id"
],
 "isUnique": false
 },
 "audit_created_idx": {
 "name": "audit_created_idx",
 "columns": [
 "created_at"
],
 "isUnique": false
 },
 "audit_action_idx": {
 "name": "audit_action_idx",
 "columns": [
 "action"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "audit_log_actor_id_users_id_fk": {
 "name": "audit_log_actor_id_users_id_fk",
```

**drizzle/meta/0010\_snapshot.json**

```
 "tableFrom": "audit_log",
 "tableTo": "users",
 "columnsFrom": [
 "actor_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"event_roles": {
 "name": "event_roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "event_id": {
 "name": "event_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "emoji": {
 "name": "emoji",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "capacity": {
 "name": "capacity",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
},
```

**drizzle/meta/0010\_snapshot.json**

```
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 }
 },
 "indexes": {
 "event_roles_event_idx": {
 "name": "event_roles_event_idx",
 "columns": [
 "event_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "event_roles_event_id_events_id_fk": {
 "name": "event_roles_event_id_events_id_fk",
 "tableFrom": "event_roles",
 "tableTo": "events",
 "columnsFrom": [
 "event_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"event_signups": {
 "name": "event_signups",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "event_id": {
 "name": "event_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
}
```

**drizzle/meta/0010\_snapshot.json**

```
"user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"event_role_id": {
 "name": "event_role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
},
"updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
 "event_signups_event_user_idx": {
 "name": "event_signups_event_user_idx",
 "columns": [
 "event_id",
 "user_id"
],
 "isUnique": true
 },
 "event_signups_event_idx": {
 "name": "event_signups_event_idx",
 "columns": [
 "event_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "event_signups_event_id_events_id_fk": {
 "name": "event_signups_event_id_events_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "events",
 "columnsFrom": [
```

**drizzle/meta/0010\_snapshot.json**

```
 "event_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
},
"event_signups_user_id_users_id_fk": {
 "name": "event_signups_user_id_users_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
},
"event_signups_event_role_id_event_roles_id_fk": {
 "name": "event_signups_event_role_id_event_roles_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "event_roles",
 "columnsFrom": [
 "event_role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
}
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"events": {
 "name": "events",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
```

**drizzle/meta/0010\_snapshot.json**

```
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "starts_at": {
 "name": "starts_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "ends_at": {
 "name": "ends_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_by": {
 "name": "created_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_event_id": {
 "name": "discord_event_id",
 "type": "text",
```

**drizzle/meta/0010\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_message_id": {
 "name": "discord_message_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'scheduled'"
 },
 "recurrence": {
 "name": "recurrence",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'none'"
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "events_starts_idx": {
 "name": "events_starts_idx",
 "columns": [
 "starts_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
```

**drizzle/meta/0010\_snapshot.json**

```
"events_game_id_games_id_fk": {
 "name": "events_game_id_games_id_fk",
 "tableFrom": "events",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
},
"events_created_by_users_id_fk": {
 "name": "events_created_by_users_id_fk",
 "tableFrom": "events",
 "tableTo": "users",
 "columnsFrom": [
 "created_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
}
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"games": {
 "name": "games",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 }
}
```



**drizzle/meta/0010\_snapshot.json**

```
 },
 "icon_url": {
 "name": "icon_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "active": {
 "name": "active",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": true
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "games_slug_unique": {
 "name": "games_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 }
 },
 "foreignKeys": {},
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"match_participants": {
 "name": "match_participants",
 "columns": {
 "match_id": {
 "name": "match_id",
 "type": "integer",
 "primaryKey": false,
```

**drizzle/meta/0010\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "stats": {
 "name": "stats",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
},
"indexes": {
 "mp_user_idx": {
 "name": "mp_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "match_participants_match_id_matches_id_fk": {
 "name": "match_participants_match_id_matches_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "matches",
 "columnsFrom": [
 "match_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "match_participants_user_id_users_id_fk": {
 "name": "match_participants_user_id_users_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
}
```

**drizzle/meta/0010\_snapshot.json**

```
 },
 "compositePrimaryKeys": {
 "match_participants_match_id_user_id_pk": {
 "columns": [
 "match_id",
 "user_id"
],
 "name": "match_participants_match_id_user_id_pk"
 }
 },
 "uniqueConstraints": {},
 "checkConstraints": {}
 },
 "matches": {
 "name": "matches",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent": {
 "name": "opponent",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent_tag": {
 "name": "opponent_tag",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "result": {
 "name": "result",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "score_us": {
 "name": "score_us",
 "type": "integer",
```

**drizzle/meta/0010\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "score_them": {
 "name": "score_them",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "map": {
 "name": "map",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "notes": {
 "name": "notes",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "played_at": {
 "name": "played_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "recorded_by": {
 "name": "recorded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "matches_game_played_idx": {
 "name": "matches_game_played_idx",
 "columns": [
 "game_id",
 "played_at"
]
 }
}
```

**drizzle/meta/0010\_snapshot.json**

```

],
 "isUnique": false
 }
},
"foreignKeys": {
 "matches_game_id_games_id_fk": {
 "name": "matches_game_id_games_id_fk",
 "tableFrom": "matches",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "matches_recorded_by_users_id_fk": {
 "name": "matches_recorded_by_users_id_fk",
 "tableFrom": "matches",
 "tableTo": "users",
 "columnsFrom": [
 "recorded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"medals": {
 "name": "medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "description": {

```

**drizzle/meta/0010\_snapshot.json**

```
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "auto_grant_months": {
 "name": "auto_grant_months",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "medals_game_idx": {
 "name": "medals_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
```

**drizzle/meta/0010\_snapshot.json**

```
 "medals_game_id_games_id_fk": {
 "name": "medals_game_id_games_id_fk",
 "tableFrom": "medals",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"member_medals": {
 "name": "member_medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "medal_id": {
 "name": "medal_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
```

**drizzle/meta/0010\_snapshot.json**

```
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "member_medals_user_idx": {
 "name": "member_medals_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "member_medals_medal_idx": {
 "name": "member_medals_medal_idx",
 "columns": [
 "medal_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "member_medals_user_id_users_id_fk": {
 "name": "member_medals_user_id_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_medal_id_medals_id_fk": {
 "name": "member_medals_medal_id_medals_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "medals",
 "columnsFrom": [
 "medal_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
```



**drizzle/meta/0010\_snapshot.json**

```
 "member_medals_awarded_by_users_id_fk": {
 "name": "member_medals_awarded_by_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"member_war_records": {
 "name": "member_war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "war_record_id": {
 "name": "war_record_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
```

**drizzle/meta/0010\_snapshot.json**

```
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "member_war_records_user_idx": {
 "name": "member_war_records_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "member_war_records_record_idx": {
 "name": "member_war_records_record_idx",
 "columns": [
 "war_record_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "member_war_records_user_id_users_id_fk": {
 "name": "member_war_records_user_id_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_war_record_id_war_records_id_fk": {
 "name": "member_war_records_war_record_id_war_records_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "war_records",
 "columnsFrom": [
 "war_record_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
```

**drizzle/meta/0010\_snapshot.json**

```
"member_war_records_awarded_by_users_id_fk": {
 "name": "member_war_records_awarded_by_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
},
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"news": {
 "name": "news",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "excerpt": {
 "name": "excerpt",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "body": {
 "name": "body",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
```

**drizzle/meta/0010\_snapshot.json**

```
 "autoincrement": false
 },
 "author_id": {
 "name": "author_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'draft'"
 },
 "pinned": {
 "name": "pinned",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "published_at": {
 "name": "published_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "news_slug_unique": {
 "name": "news_slug_unique",
 "columns": [
 "slug"
]
 }
}
```

**drizzle/meta/0010\_snapshot.json**

```
],
 "isUnique": true
 },
 "news_status_published_idx": {
 "name": "news_status_published_idx",
 "columns": [
 "status",
 "published_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "news_author_id_users_id_fk": {
 "name": "news_author_id_users_id_fk",
 "tableFrom": "news",
 "tableTo": "users",
 "columnsFrom": [
 "author_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"profiles": {
 "name": "profiles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "bio": {
 "name": "bio",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
}
```

**drizzle/meta/0010\_snapshot.json**

```
 },
 "timezone": {
 "name": "timezone",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "gamertags": {
 "name": "gamertags",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {},
 "foreignKeys": {
 "profiles_user_id_users_id_fk": {
 "name": "profiles_user_id_users_id_fk",
 "tableFrom": "profiles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"rank_roles": {
 "name": "rank_roles",
 "columns": {
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 }
},
```

**drizzle/meta/0010\_snapshot.json**

```
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
},
"indexes": {
 "rank_roles_role_idx": {
 "name": "rank_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "rank_roles_rank_id_ranks_id_fk": {
 "name": "rank_roles_rank_id_ranks_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "rank_roles_role_id_roles_id_fk": {
 "name": "rank_roles_role_id_roles_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "rank_roles_rank_id_role_id_pk": {
 "columns": [
 "rank_id",
 "role_id"
],
 "name": "rank_roles_rank_id_role_id_pk"
 }
},
},
```

**drizzle/meta/0010\_snapshot.json**

```
"uniqueConstraints": {},
"checkConstraints": {}
},
"ranks": {
 "name": "ranks",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "abbreviation": {
 "name": "abbreviation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "req_days": {
 "name": "req_days",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "req_wins": {
 "name": "req_wins",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
```



**drizzle/meta/0010\_snapshot.json**

```
 "autoincrement": false,
 "default": 0
 },
 "is_default": {
 "name": "is_default",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "ranks_sort_order_idx": {
 "name": "ranks_sort_order_idx",
 "columns": [
 "sort_order"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"role_permissions": {
 "name": "role_permissions",
 "columns": {
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "permission": {
 "name": "permission",
```

**drizzle/meta/0010\_snapshot.json**

```
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
},
"indexes": {},
"foreignKeys": {
 "role_permissions_role_id_roles_id_fk": {
 "name": "role_permissions_role_id_roles_id_fk",
 "tableFrom": "role_permissions",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "role_permissions_role_id_permission_pk": {
 "columns": [
 "role_id",
 "permission"
],
 "name": "role_permissions_role_id_permission_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"roles": {
 "name": "roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
```

**drizzle/meta/0010\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "color": {
 "name": "color",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_role_id": {
 "name": "discord_role_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_system": {
 "name": "is_system",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "roles_name_unique": {
 "name": "roles_name_unique",
```

**drizzle/meta/0010\_snapshot.json**

```
 "columns": [
 {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 {
 "name": "discord_role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
],
 "indexes": [
 {
 "name": "roles_discord_role_id_unique",
 "columns": ["discord_role_id"],
 "unique": true
 }
],
 "foreignKeys": {},
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
{
 "name": "sessions",
 "columns": {
 "id": {
 "name": "id",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "expires_at": {
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_agent": {
 "name": "user_agent",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
}
```

**drizzle/meta/0010\_snapshot.json**

```
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "sessions_user_idx": {
 "name": "sessions_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "sessions_expires_idx": {
 "name": "sessions_expires_idx",
 "columns": [
 "expires_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "sessions_user_id_users_id_fk": {
 "name": "sessions_user_id_users_id_fk",
 "tableFrom": "sessions",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"settings": {
 "name": "settings",
 "columns": {
 "key": {
 "name": "key",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
```

**drizzle/meta/0010\_snapshot.json**

```
 "autoincrement": false
 },
 "value": {
 "name": "value",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {},
"foreignKeys": {
 "settings_updated_by_users_id_fk": {
 "name": "settings_updated_by_users_id_fk",
 "tableFrom": "settings",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"user_roles": {
 "name": "user_roles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
```

**drizzle/meta/0010\_snapshot.json**

```
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'manual'"
 },
 "granted_by": {
 "name": "granted_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "granted_at": {
 "name": "granted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "user_roles_role_idx": {
 "name": "user_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "user_roles_user_id_users_id_fk": {
 "name": "user_roles_user_id_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 }
}
```

**drizzle/meta/0010\_snapshot.json**

```

 },
 "user_roles_role_id_roles_id_fk": {
 "name": "user_roles_role_id_roles_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_granted_by_users_id_fk": {
 "name": "user_roles_granted_by_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "granted_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {
 "user_roles_user_id_role_id_pk": {
 "columns": [
 "user_id",
 "role_id"
],
 "name": "user_roles_user_id_role_id_pk"
 }
 },
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"users": {
 "name": "users",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "discord_id": {
 "name": "discord_id",
 "type": "text",
 "primaryKey": false,

```



**drizzle/meta/0010\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false
 },
 "username": {
 "name": "username",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "global_name": {
 "name": "global_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "display_name": {
 "name": "display_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "avatar": {
 "name": "avatar",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "profile_image_url": {
 "name": "profile_image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "email": {
 "name": "email",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "is_god": {
 "name": "is_god",
```

**drizzle/meta/0010\_snapshot.json**

```
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "guild_joined_at": {
 "name": "guild_joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'active'"
 },
 "recruiter_id": {
 "name": "recruiter_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "joined_at": {
 "name": "joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "last_seen_at": {
 "name": "last_seen_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "last_reconciled_at": {
 "name": "last_reconciled_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "promoted_at": {
 "name": "promoted_at",
 "type": "integer",
 "primaryKey": false,
```

**drizzle/meta/0010\_snapshot.json**

```
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "users_discord_id_unique": {
 "name": "users_discord_id_unique",
 "columns": [
 "discord_id"
],
 "isUnique": true
 },
 "users_rank_idx": {
 "name": "users_rank_idx",
 "columns": [
 "rank_id"
],
 "isUnique": false
 },
 "users_status_idx": {
 "name": "users_status_idx",
 "columns": [
 "status"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "users_rank_id_ranks_id_fk": {
 "name": "users_rank_id_ranks_id_fk",
 "tableFrom": "users",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 },
}
```

**drizzle/meta/0010\_snapshot.json**

```
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"war_records": {
 "name": "war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 }
 }
},
```

**drizzle/meta/0010\_snapshot.json**

```

 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "war_records_game_idx": {
 "name": "war_records_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "war_records_game_id_games_id_fk": {
 "name": "war_records_game_id_games_id_fk",
 "tableFrom": "war_records",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"views": {},
"enums": {},
"_meta": {
 "schemas": {},
 "tables": {},
 "columns": {}
},
"internal": {
 "indexes": {}
}
}

```

**drizzle/meta/0011\_snapshot.json**

```
{
 "version": "6",
 "dialect": "sqlite",
 "id": "d2eeffab-1ff8-42ac-a8bd-d19535055899",
 "prevId": "5bae322e-8ce7-4eb5-961b-1f5eae414f03",
 "tables": {
 "audit_log": {
 "name": "audit_log",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "actor_id": {
 "name": "actor_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "action": {
 "name": "action",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "target_type": {
 "name": "target_type",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "target_id": {
 "name": "target_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "reason": {
 "name": "reason",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "meta": {
 "name": "meta",
 "type": "text",
```

**drizzle/meta/0011\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'web'"
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "audit_actor_idx": {
 "name": "audit_actor_idx",
 "columns": [
 "actor_id"
],
 "isUnique": false
 },
 "audit_created_idx": {
 "name": "audit_created_idx",
 "columns": [
 "created_at"
],
 "isUnique": false
 },
 "audit_action_idx": {
 "name": "audit_action_idx",
 "columns": [
 "action"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "audit_log_actor_id_users_id_fk": {
 "name": "audit_log_actor_id_users_id_fk",
```

**drizzle/meta/0011\_snapshot.json**

```
 "tableFrom": "audit_log",
 "tableTo": "users",
 "columnsFrom": [
 "actor_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"event_roles": {
 "name": "event_roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "event_id": {
 "name": "event_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "emoji": {
 "name": "emoji",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "capacity": {
 "name": "capacity",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
},
```



**drizzle/meta/0011\_snapshot.json**

```
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 }
 },
 "indexes": {
 "event_roles_event_idx": {
 "name": "event_roles_event_idx",
 "columns": [
 "event_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "event_roles_event_id_events_id_fk": {
 "name": "event_roles_event_id_events_id_fk",
 "tableFrom": "event_roles",
 "tableTo": "events",
 "columnsFrom": [
 "event_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"event_signups": {
 "name": "event_signups",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "event_id": {
 "name": "event_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
}
```

**drizzle/meta/0011\_snapshot.json**

```
"user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"event_role_id": {
 "name": "event_role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
},
"updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
 "event_signups_event_user_idx": {
 "name": "event_signups_event_user_idx",
 "columns": [
 "event_id",
 "user_id"
],
 "isUnique": true
 },
 "event_signups_event_idx": {
 "name": "event_signups_event_idx",
 "columns": [
 "event_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "event_signups_event_id_events_id_fk": {
 "name": "event_signups_event_id_events_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "events",
 "columnsFrom": [
```

**drizzle/meta/0011\_snapshot.json**

```
 "event_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
},
"event_signups_user_id_users_id_fk": {
 "name": "event_signups_user_id_users_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
},
"event_signups_event_role_id_event_roles_id_fk": {
 "name": "event_signups_event_role_id_event_roles_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "event_roles",
 "columnsFrom": [
 "event_role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
}
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"events": {
 "name": "events",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
```

**drizzle/meta/0011\_snapshot.json**

```
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "starts_at": {
 "name": "starts_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "ends_at": {
 "name": "ends_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_by": {
 "name": "created_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_event_id": {
 "name": "discord_event_id",
 "type": "text",
```

**drizzle/meta/0011\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_message_id": {
 "name": "discord_message_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'scheduled'"
 },
 "recurrence": {
 "name": "recurrence",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'none'"
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "events_starts_idx": {
 "name": "events_starts_idx",
 "columns": [
 "starts_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
```

**drizzle/meta/0011\_snapshot.json**

```
"events_game_id_games_id_fk": {
 "name": "events_game_id_games_id_fk",
 "tableFrom": "events",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
},
"events_created_by_users_id_fk": {
 "name": "events_created_by_users_id_fk",
 "tableFrom": "events",
 "tableTo": "users",
 "columnsFrom": [
 "created_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
}
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"games": {
 "name": "games",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 }
}
```

**drizzle/meta/0011\_snapshot.json**

```
 },
 "icon_url": {
 "name": "icon_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "active": {
 "name": "active",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": true
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "games_slug_unique": {
 "name": "games_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 }
 },
 "foreignKeys": {},
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"match_participants": {
 "name": "match_participants",
 "columns": {
 "match_id": {
 "name": "match_id",
 "type": "integer",
 "primaryKey": false,
```

**drizzle/meta/0011\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "stats": {
 "name": "stats",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
},
"indexes": {
 "mp_user_idx": {
 "name": "mp_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "match_participants_match_id_matches_id_fk": {
 "name": "match_participants_match_id_matches_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "matches",
 "columnsFrom": [
 "match_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "match_participants_user_id_users_id_fk": {
 "name": "match_participants_user_id_users_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
}
```



**drizzle/meta/0011\_snapshot.json**

```
 },
 "compositePrimaryKeys": {
 "match_participants_match_id_user_id_pk": {
 "columns": [
 "match_id",
 "user_id"
],
 "name": "match_participants_match_id_user_id_pk"
 }
 },
 "uniqueConstraints": {},
 "checkConstraints": {}
 },
 "matches": {
 "name": "matches",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent": {
 "name": "opponent",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent_tag": {
 "name": "opponent_tag",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "result": {
 "name": "result",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "score_us": {
 "name": "score_us",
 "type": "integer",
```

**drizzle/meta/0011\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "score_them": {
 "name": "score_them",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "map": {
 "name": "map",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "notes": {
 "name": "notes",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "played_at": {
 "name": "played_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "recorded_by": {
 "name": "recorded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "matches_game_played_idx": {
 "name": "matches_game_played_idx",
 "columns": [
 "game_id",
 "played_at"
]
 }
}
```

**drizzle/meta/0011\_snapshot.json**

```
],
 "isUnique": false
 }
},
"foreignKeys": {
 "matches_game_id_games_id_fk": {
 "name": "matches_game_id_games_id_fk",
 "tableFrom": "matches",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "matches_recorded_by_users_id_fk": {
 "name": "matches_recorded_by_users_id_fk",
 "tableFrom": "matches",
 "tableTo": "users",
 "columnsFrom": [
 "recorded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"medals": {
 "name": "medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "description": {
```

**drizzle/meta/0011\_snapshot.json**

```
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "auto_grant_months": {
 "name": "auto_grant_months",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "medals_game_idx": {
 "name": "medals_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
```

**drizzle/meta/0011\_snapshot.json**

```
 "medals_game_id_games_id_fk": {
 "name": "medals_game_id_games_id_fk",
 "tableFrom": "medals",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"member_medals": {
 "name": "member_medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "medal_id": {
 "name": "medal_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
```

**drizzle/meta/0011\_snapshot.json**

```
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "member_medals_user_idx": {
 "name": "member_medals_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "member_medals_medal_idx": {
 "name": "member_medals_medal_idx",
 "columns": [
 "medal_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "member_medals_user_id_users_id_fk": {
 "name": "member_medals_user_id_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_medal_id_medals_id_fk": {
 "name": "member_medals_medal_id_medals_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "medals",
 "columnsFrom": [
 "medal_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
```

**drizzle/meta/0011\_snapshot.json**

```
 "member_medals_awarded_by_users_id_fk": {
 "name": "member_medals_awarded_by_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"member_war_records": {
 "name": "member_war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "war_record_id": {
 "name": "war_record_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
```

**drizzle/meta/0011\_snapshot.json**

```
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "member_war_records_user_idx": {
 "name": "member_war_records_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "member_war_records_record_idx": {
 "name": "member_war_records_record_idx",
 "columns": [
 "war_record_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "member_war_records_user_id_users_id_fk": {
 "name": "member_war_records_user_id_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_war_record_id_war_records_id_fk": {
 "name": "member_war_records_war_record_id_war_records_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "war_records",
 "columnsFrom": [
 "war_record_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
```



**drizzle/meta/0011\_snapshot.json**

```
"member_war_records_awarded_by_users_id_fk": {
 "name": "member_war_records_awarded_by_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
},
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"news": {
 "name": "news",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "excerpt": {
 "name": "excerpt",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "body": {
 "name": "body",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
```

**drizzle/meta/0011\_snapshot.json**

```
 "autoincrement": false
 },
 "author_id": {
 "name": "author_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'draft'"
 },
 "pinned": {
 "name": "pinned",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "published_at": {
 "name": "published_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "news_slug_unique": {
 "name": "news_slug_unique",
 "columns": [
 "slug"
]
 }
}
```

**drizzle/meta/0011\_snapshot.json**

```
],
 "isUnique": true
 },
 "news_status_published_idx": {
 "name": "news_status_published_idx",
 "columns": [
 "status",
 "published_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "news_author_id_users_id_fk": {
 "name": "news_author_id_users_id_fk",
 "tableFrom": "news",
 "tableTo": "users",
 "columnsFrom": [
 "author_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"page_layouts": {
 "name": "page_layouts",
 "columns": {
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "layout": {
 "name": "layout",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 }
}
```

**drizzle/meta/0011\_snapshot.json**

```
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {},
 "foreignKeys": {
 "page_layouts_updated_by_users_id_fk": {
 "name": "page_layouts_updated_by_users_id_fk",
 "tableFrom": "page_layouts",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"profiles": {
 "name": "profiles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "bio": {
 "name": "bio",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
},
```

**drizzle/meta/0011\_snapshot.json**

```
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "timezone": {
 "name": "timezone",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "gamertags": {
 "name": "gamertags",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {},
"foreignKeys": {
 "profiles_user_id_users_id_fk": {
 "name": "profiles_user_id_users_id_fk",
 "tableFrom": "profiles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"rank_roles": {
 "name": "rank_roles",
 "columns": {
 "rank_id": {
```

**drizzle/meta/0011\_snapshot.json**

```
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
},
"indexes": {
 "rank_roles_role_idx": {
 "name": "rank_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "rank_roles_rank_id_ranks_id_fk": {
 "name": "rank_roles_rank_id_ranks_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "rank_roles_role_id_roles_id_fk": {
 "name": "rank_roles_role_id_roles_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "rank_roles_rank_id_role_id_pk": {
 "columns": [
```

**drizzle/meta/0011\_snapshot.json**

```
 "rank_id",
 "role_id"
],
 "name": "rank_roles_rank_id_role_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"ranks": {
 "name": "ranks",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "abbreviation": {
 "name": "abbreviation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "req_days": {
 "name": "req_days",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 }
 }
}
```

**drizzle/meta/0011\_snapshot.json**

```
 },
 "req_wins": {
 "name": "req_wins",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_default": {
 "name": "is_default",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "ranks_sort_order_idx": {
 "name": "ranks_sort_order_idx",
 "columns": [
 "sort_order"
],
 "isUnique": true
 }
 },
 "foreignKeys": {},
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"role_permissions": {
 "name": "role_permissions",
 "columns": {
 "role_id": {
 "name": "role_id",
 "type": "integer",
```



**drizzle/meta/0011\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "permission": {
 "name": "permission",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
},
"indexes": {},
"foreignKeys": {
 "role_permissions_role_id_roles_id_fk": {
 "name": "role_permissions_role_id_roles_id_fk",
 "tableFrom": "role_permissions",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "role_permissions_role_id_permission_pk": {
 "columns": [
 "role_id",
 "permission"
],
 "name": "role_permissions_role_id_permission_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"roles": {
 "name": "roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
```

**drizzle/meta/0011\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "color": {
 "name": "color",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_role_id": {
 "name": "discord_role_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_system": {
 "name": "is_system",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
```

**drizzle/meta/0011\_snapshot.json**

```
 "default": "(unixepoch())"
 },
 "indexes": {
 "roles_name_unique": {
 "name": "roles_name_unique",
 "columns": [
 "name"
],
 "isUnique": true
 },
 "roles_discord_role_id_unique": {
 "name": "roles_discord_role_id_unique",
 "columns": [
 "discord_role_id"
],
 "isUnique": true
 }
 },
 "foreignKeys": {},
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"sessions": {
 "name": "sessions",
 "columns": {
 "id": {
 "name": "id",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "expires_at": {
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_agent": {
 "name": "user_agent",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
}
```

**drizzle/meta/0011\_snapshot.json**

```
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "sessions_user_idx": {
 "name": "sessions_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "sessions_expires_idx": {
 "name": "sessions_expires_idx",
 "columns": [
 "expires_at"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "sessions_user_id_users_id_fk": {
 "name": "sessions_user_id_users_id_fk",
 "tableFrom": "sessions",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"settings": {
 "name": "settings",
```

**drizzle/meta/0011\_snapshot.json**

```
"columns": {
 "key": {
 "name": "key",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "value": {
 "name": "value",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {},
"foreignKeys": {
 "settings_updated_by_users_id_fk": {
 "name": "settings_updated_by_users_id_fk",
 "tableFrom": "settings",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"user_roles": {
 "name": "user_roles",
 "columns": {
```

**drizzle/meta/0011\_snapshot.json**

```
"user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'manual'"
},
"granted_by": {
 "name": "granted_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"granted_at": {
 "name": "granted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
 "user_roles_role_idx": {
 "name": "user_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "user_roles_user_id_users_id_fk": {
 "name": "user_roles_user_id_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
```

**drizzle/meta/0011\_snapshot.json**

```
],
 "columnsTo": [
 "id"
],
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_role_id_roles_id_fk": {
 "name": "user_roles_role_id_roles_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
],
 "columnsTo": [
 "id"
],
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_granted_by_users_id_fk": {
 "name": "user_roles_granted_by_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "granted_by"
],
],
 "columnsTo": [
 "id"
],
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "user_roles_user_id_role_id_pk": {
 "columns": [
 "user_id",
 "role_id"
],
],
 "name": "user_roles_user_id_role_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"users": {
 "name": "users",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
```

**drizzle/meta/0011\_snapshot.json**

```
 "autoincrement": true
 },
 "discord_id": {
 "name": "discord_id",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "username": {
 "name": "username",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "global_name": {
 "name": "global_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "display_name": {
 "name": "display_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "avatar": {
 "name": "avatar",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "profile_image_url": {
 "name": "profile_image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "email": {
 "name": "email",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
```



**drizzle/meta/0011\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "is_god": {
 "name": "is_god",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "guild_joined_at": {
 "name": "guild_joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'active'"
 },
 "recruiter_id": {
 "name": "recruiter_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "joined_at": {
 "name": "joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "last_seen_at": {
 "name": "last_seen_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "last_reconciled_at": {
 "name": "last_reconciled_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
```

**drizzle/meta/0011\_snapshot.json**

```
 "autoincrement": false
 },
 "promoted_at": {
 "name": "promoted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "users_discord_id_unique": {
 "name": "users_discord_id_unique",
 "columns": [
 "discord_id"
],
 "isUnique": true
 },
 "users_rank_idx": {
 "name": "users_rank_idx",
 "columns": [
 "rank_id"
],
 "isUnique": false
 },
 "users_status_idx": {
 "name": "users_status_idx",
 "columns": [
 "status"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "users_rank_id_ranks_id_fk": {
 "name": "users_rank_id_ranks_id_fk",
 "tableFrom": "users",
 "tableTo": "ranks",
```

**drizzle/meta/0011\_snapshot.json**

```
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"war_records": {
 "name": "war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
```

**drizzle/meta/0011\_snapshot.json**

```

 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "war_records_game_idx": {
 "name": "war_records_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "war_records_game_id_games_id_fk": {
 "name": "war_records_game_id_games_id_fk",
 "tableFrom": "war_records",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
}
},
"views": {},
"enums": {},
"_meta": {
 "schemas": {},
 "tables": {},
 "columns": {}
},
"internal": {
 "indexes": {}
}

```

**drizzle/meta/0011\_snapshot.json**

}

**drizzle/meta/0012\_snapshot.json**

```
{
 "version": "6",
 "dialect": "sqlite",
 "id": "562a6309-74a4-45f7-8d53-52996dacddf9",
 "prevId": "d2eeffab-1ff8-42ac-a8bd-d19535055899",
 "tables": {
 "audit_log": {
 "name": "audit_log",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "actor_id": {
 "name": "actor_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "action": {
 "name": "action",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "target_type": {
 "name": "target_type",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "target_id": {
 "name": "target_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "reason": {
 "name": "reason",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "meta": {
 "name": "meta",
 "type": "text",
```

**drizzle/meta/0012\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'web'"
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "audit_actor_idx": {
 "name": "audit_actor_idx",
 "columns": [
 "actor_id"
],
 "isUnique": false
 },
 "audit_created_idx": {
 "name": "audit_created_idx",
 "columns": [
 "created_at"
],
 "isUnique": false
 },
 "audit_action_idx": {
 "name": "audit_action_idx",
 "columns": [
 "action"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "audit_log_actor_id_users_id_fk": {
 "name": "audit_log_actor_id_users_id_fk",
```

**drizzle/meta/0012\_snapshot.json**

```
 "tableFrom": "audit_log",
 "tableTo": "users",
 "columnsFrom": [
 "actor_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"event_roles": {
 "name": "event_roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "event_id": {
 "name": "event_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "emoji": {
 "name": "emoji",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "capacity": {
 "name": "capacity",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
},
```



**drizzle/meta/0012\_snapshot.json**

```
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 }
 },
 "indexes": {
 "event_roles_event_idx": {
 "name": "event_roles_event_idx",
 "columns": [
 "event_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "event_roles_event_id_events_id_fk": {
 "name": "event_roles_event_id_events_id_fk",
 "tableFrom": "event_roles",
 "tableTo": "events",
 "columnsFrom": [
 "event_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"event_signups": {
 "name": "event_signups",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "event_id": {
 "name": "event_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
}
```

**drizzle/meta/0012\_snapshot.json**

```
"user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"event_role_id": {
 "name": "event_role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
},
"updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
 "event_signups_event_user_idx": {
 "name": "event_signups_event_user_idx",
 "columns": [
 "event_id",
 "user_id"
],
 "isUnique": true
 },
 "event_signups_event_idx": {
 "name": "event_signups_event_idx",
 "columns": [
 "event_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "event_signups_event_id_events_id_fk": {
 "name": "event_signups_event_id_events_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "events",
 "columnsFrom": [
```

**drizzle/meta/0012\_snapshot.json**

```
 "event_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
},
"event_signups_user_id_users_id_fk": {
 "name": "event_signups_user_id_users_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
},
"event_signups_event_role_id_event_roles_id_fk": {
 "name": "event_signups_event_role_id_event_roles_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "event_roles",
 "columnsFrom": [
 "event_role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
}
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"events": {
 "name": "events",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
```

**drizzle/meta/0012\_snapshot.json**

```
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "starts_at": {
 "name": "starts_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "ends_at": {
 "name": "ends_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_by": {
 "name": "created_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_event_id": {
 "name": "discord_event_id",
 "type": "text",
```

**drizzle/meta/0012\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_message_id": {
 "name": "discord_message_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'scheduled'"
 },
 "recurrence": {
 "name": "recurrence",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'none'"
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "events_starts_idx": {
 "name": "events_starts_idx",
 "columns": [
 "starts_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
```

**drizzle/meta/0012\_snapshot.json**

```
"events_game_id_games_id_fk": {
 "name": "events_game_id_games_id_fk",
 "tableFrom": "events",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
},
"events_created_by_users_id_fk": {
 "name": "events_created_by_users_id_fk",
 "tableFrom": "events",
 "tableTo": "users",
 "columnsFrom": [
 "created_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
}
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"games": {
 "name": "games",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 }
}
```

**drizzle/meta/0012\_snapshot.json**

```
 },
 "icon_url": {
 "name": "icon_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "active": {
 "name": "active",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": true
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "games_slug_unique": {
 "name": "games_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 }
 },
 "foreignKeys": {},
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"match_participants": {
 "name": "match_participants",
 "columns": {
 "match_id": {
 "name": "match_id",
 "type": "integer",
 "primaryKey": false,
```

**drizzle/meta/0012\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "stats": {
 "name": "stats",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
},
"indexes": {
 "mp_user_idx": {
 "name": "mp_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "match_participants_match_id_matches_id_fk": {
 "name": "match_participants_match_id_matches_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "matches",
 "columnsFrom": [
 "match_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "match_participants_user_id_users_id_fk": {
 "name": "match_participants_user_id_users_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
}
```



**drizzle/meta/0012\_snapshot.json**

```
 },
 "compositePrimaryKeys": {
 "match_participants_match_id_user_id_pk": {
 "columns": [
 "match_id",
 "user_id"
],
 "name": "match_participants_match_id_user_id_pk"
 }
 },
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"matches": {
 "name": "matches",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent": {
 "name": "opponent",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent_tag": {
 "name": "opponent_tag",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "result": {
 "name": "result",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "score_us": {
 "name": "score_us",
 "type": "integer",
```

**drizzle/meta/0012\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "score_them": {
 "name": "score_them",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "map": {
 "name": "map",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "notes": {
 "name": "notes",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "played_at": {
 "name": "played_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "recorded_by": {
 "name": "recorded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "matches_game_played_idx": {
 "name": "matches_game_played_idx",
 "columns": [
 "game_id",
 "played_at"
]
 }
}
```

**drizzle/meta/0012\_snapshot.json**

```
],
 "isUnique": false
 }
},
"foreignKeys": {
 "matches_game_id_games_id_fk": {
 "name": "matches_game_id_games_id_fk",
 "tableFrom": "matches",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "matches_recorded_by_users_id_fk": {
 "name": "matches_recorded_by_users_id_fk",
 "tableFrom": "matches",
 "tableTo": "users",
 "columnsFrom": [
 "recorded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"medals": {
 "name": "medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "description": {
```

**drizzle/meta/0012\_snapshot.json**

```
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "auto_grant_months": {
 "name": "auto_grant_months",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "medals_game_idx": {
 "name": "medals_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
```

**drizzle/meta/0012\_snapshot.json**

```
 "medals_game_id_games_id_fk": {
 "name": "medals_game_id_games_id_fk",
 "tableFrom": "medals",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"member_medals": {
 "name": "member_medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "medal_id": {
 "name": "medal_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
```

**drizzle/meta/0012\_snapshot.json**

```
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "member_medals_user_idx": {
 "name": "member_medals_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "member_medals_medal_idx": {
 "name": "member_medals_medal_idx",
 "columns": [
 "medal_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "member_medals_user_id_users_id_fk": {
 "name": "member_medals_user_id_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_medal_id_medals_id_fk": {
 "name": "member_medals_medal_id_medals_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "medals",
 "columnsFrom": [
 "medal_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
```

**drizzle/meta/0012\_snapshot.json**

```
 "member_medals_awarded_by_users_id_fk": {
 "name": "member_medals_awarded_by_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"member_war_records": {
 "name": "member_war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "war_record_id": {
 "name": "war_record_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
```

**drizzle/meta/0012\_snapshot.json**

```
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "member_war_records_user_idx": {
 "name": "member_war_records_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "member_war_records_record_idx": {
 "name": "member_war_records_record_idx",
 "columns": [
 "war_record_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "member_war_records_user_id_users_id_fk": {
 "name": "member_war_records_user_id_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_war_record_id_war_records_id_fk": {
 "name": "member_war_records_war_record_id_war_records_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "war_records",
 "columnsFrom": [
 "war_record_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
```



**drizzle/meta/0012\_snapshot.json**

```
"member_war_records_awarded_by_users_id_fk": {
 "name": "member_war_records_awarded_by_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
},
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"news": {
 "name": "news",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "excerpt": {
 "name": "excerpt",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "body": {
 "name": "body",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
```

**drizzle/meta/0012\_snapshot.json**

```
 "autoincrement": false
 },
 "author_id": {
 "name": "author_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'draft'"
 },
 "pinned": {
 "name": "pinned",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "published_at": {
 "name": "published_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "news_slug_unique": {
 "name": "news_slug_unique",
 "columns": [
 "slug"
]
 }
}
```

**drizzle/meta/0012\_snapshot.json**

```
],
 "isUnique": true
 },
 "news_status_published_idx": {
 "name": "news_status_published_idx",
 "columns": [
 "status",
 "published_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "news_author_id_users_id_fk": {
 "name": "news_author_id_users_id_fk",
 "tableFrom": "news",
 "tableTo": "users",
 "columnsFrom": [
 "author_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"page_layouts": {
 "name": "page_layouts",
 "columns": {
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "layout": {
 "name": "layout",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 }
}
```

**drizzle/meta/0012\_snapshot.json**

```
 },
 "show_in_nav": {
 "name": "show_in_nav",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "nav_order": {
 "name": "nav_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {},
 "foreignKeys": {
 "page_layouts_updated_by_users_id_fk": {
 "name": "page_layouts_updated_by_users_id_fk",
 "tableFrom": "page_layouts",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"profiles": {
```

**drizzle/meta/0012\_snapshot.json**

```
"name": "profiles",
"columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "bio": {
 "name": "bio",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "timezone": {
 "name": "timezone",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "gamertags": {
 "name": "gamertags",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {},
"foreignKeys": {
 "profiles_user_id_users_id_fk": {
 "name": "profiles_user_id_users_id_fk",
 "tableFrom": "profiles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
]
 }
}
```

**drizzle/meta/0012\_snapshot.json**

```
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"rank_roles": {
 "name": "rank_roles",
 "columns": {
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "indexes": {
 "rank_roles_role_idx": {
 "name": "rank_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "rank_roles_rank_id_ranks_id_fk": {
 "name": "rank_roles_rank_id_ranks_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "rank_roles_role_id_roles_id_fk": {
```

**drizzle/meta/0012\_snapshot.json**

```
 "name": "rank_roles_role_id_roles_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "rank_roles_rank_id_role_id_pk": {
 "columns": [
 "rank_id",
 "role_id"
],
 "name": "rank_roles_rank_id_role_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"ranks": {
 "name": "ranks",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "abbreviation": {
 "name": "abbreviation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
```

**drizzle/meta/0012\_snapshot.json**

```
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "req_days": {
 "name": "req_days",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "req_wins": {
 "name": "req_wins",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_default": {
 "name": "is_default",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "ranks_sort_order_idx": {
 "name": "ranks_sort_order_idx",
 "columns": [
```



**drizzle/meta/0012\_snapshot.json**

```
 "sort_order"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"role_permissions": {
 "name": "role_permissions",
 "columns": {
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "permission": {
 "name": "permission",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "indexes": {},
 "foreignKeys": {
 "role_permissions_role_id_roles_id_fk": {
 "name": "role_permissions_role_id_roles_id_fk",
 "tableFrom": "role_permissions",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {
 "role_permissions_role_id_permission_pk": {
 "columns": [
 "role_id",
 "permission"
],
 "name": "role_permissions_role_id_permission_pk"
 }
 },
 "uniqueConstraints": {},
```

**drizzle/meta/0012\_snapshot.json**

```
"checkConstraints": {},
},
"roles": {
 "name": "roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "color": {
 "name": "color",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_role_id": {
 "name": "discord_role_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_system": {
 "name": "is_system",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
```

**drizzle/meta/0012\_snapshot.json**

```
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "roles_name_unique": {
 "name": "roles_name_unique",
 "columns": [
 "name"
],
 "isUnique": true
 },
 "roles_discord_role_id_unique": {
 "name": "roles_discord_role_id_unique",
 "columns": [
 "discord_role_id"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"sessions": {
 "name": "sessions",
 "columns": {
 "id": {
 "name": "id",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
```

**drizzle/meta/0012\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false
 },
 "expires_at": {
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_agent": {
 "name": "user_agent",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "sessions_user_idx": {
 "name": "sessions_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "sessions_expires_idx": {
 "name": "sessions_expires_idx",
 "columns": [
 "expires_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "sessions_user_id_users_id_fk": {
 "name": "sessions_user_id_users_id_fk",
 "tableFrom": "sessions",
 "tableTo": "users",
```

**drizzle/meta/0012\_snapshot.json**

```
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"settings": {
 "name": "settings",
 "columns": {
 "key": {
 "name": "key",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "value": {
 "name": "value",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 }
},
"indexes": {},
"foreignKeys": {
 "settings_updated_by_users_id_fk": {
 "name": "settings_updated_by_users_id_fk",
 "tableFrom": "settings",
 "tableTo": "users",
 "columnsFrom": [
```

**drizzle/meta/0012\_snapshot.json**

```
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"user_roles": {
 "name": "user_roles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'manual'"
 },
 "granted_by": {
 "name": "granted_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "granted_at": {
 "name": "granted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 }
},
```

**drizzle/meta/0012\_snapshot.json**

```
"indexes": {
 "user_roles_role_idx": {
 "name": "user_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "user_roles_user_id_users_id_fk": {
 "name": "user_roles_user_id_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_role_id_roles_id_fk": {
 "name": "user_roles_role_id_roles_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_granted_by_users_id_fk": {
 "name": "user_roles_granted_by_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "granted_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "user_roles_user_id_role_id_pk": {
 "columns": [
 "user_id",
```

**drizzle/meta/0012\_snapshot.json**

```
 "role_id"
],
 "name": "user_roles_user_id_role_id_pk"
}
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"users": {
 "name": "users",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "discord_id": {
 "name": "discord_id",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "username": {
 "name": "username",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "global_name": {
 "name": "global_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "display_name": {
 "name": "display_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "avatar": {
 "name": "avatar",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "profile_image_url": {
```



**drizzle/meta/0012\_snapshot.json**

```
 "name": "profile_image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "email": {
 "name": "email",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "is_god": {
 "name": "is_god",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "guild_joined_at": {
 "name": "guild_joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'active'"
 },
 "recruiter_id": {
 "name": "recruiter_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "joined_at": {
 "name": "joined_at",
 "type": "integer",
 "primaryKey": false,
```

**drizzle/meta/0012\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "last_seen_at": {
 "name": "last_seen_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "last_reconciled_at": {
 "name": "last_reconciled_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "promoted_at": {
 "name": "promoted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "users_discord_id_unique": {
 "name": "users_discord_id_unique",
 "columns": [
 "discord_id"
],
 "isUnique": true
 },
 "users_rank_idx": {
 "name": "users_rank_idx",
 "columns": [
 "rank_id"
]
 }
}
```

**drizzle/meta/0012\_snapshot.json**

```
],
 "isUnique": false
 },
 "users_status_idx": {
 "name": "users_status_idx",
 "columns": [
 "status"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "users_rank_id_ranks_id_fk": {
 "name": "users_rank_id_ranks_id_fk",
 "tableFrom": "users",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"war_records": {
 "name": "war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
},
```

**drizzle/meta/0012\_snapshot.json**

```
"image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
 "war_records_game_idx": {
 "name": "war_records_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "war_records_game_id_games_id_fk": {
 "name": "war_records_game_id_games_id_fk",
 "tableFrom": "war_records",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
}
```

**drizzle/meta/0012\_snapshot.json**

```
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
 }
},
"views": {},
"enums": {},
"_meta": {
 "schemas": {},
 "tables": {},
 "columns": {}
},
"internal": {
 "indexes": {}
}
}
```

**drizzle/meta/0013\_snapshot.json**

```
{
 "version": "6",
 "dialect": "sqlite",
 "id": "092e1d03-fdbf-4143-bcc2-600144c6b8f2",
 "prevId": "562a6309-74a4-45f7-8d53-52996dacddf9",
 "tables": {
 "api_tokens": {
 "name": "api_tokens",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "token_hash": {
 "name": "token_hash",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "label": {
 "name": "label",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "prefix": {
 "name": "prefix",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "expires_at": {
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "last_used_at": {
 "name": "last_used_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
 }
 }
}
```

**drizzle/meta/0013\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "api_tokens_token_hash_unique": {
 "name": "api_tokens_token_hash_unique",
 "columns": [
 "token_hash"
],
 "isUnique": true
 },
 "api_tokens_user_idx": {
 "name": "api_tokens_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "api_tokens_user_id_users_id_fk": {
 "name": "api_tokens_user_id_users_id_fk",
 "tableFrom": "api_tokens",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"audit_log": {
 "name": "audit_log",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
```

**drizzle/meta/0013\_snapshot.json**

```
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "actor_id": {
 "name": "actor_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "action": {
 "name": "action",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "target_type": {
 "name": "target_type",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "target_id": {
 "name": "target_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "reason": {
 "name": "reason",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "meta": {
 "name": "meta",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'web'"
 },
}
```



**drizzle/meta/0013\_snapshot.json**

```
"ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
 "audit_actor_idx": {
 "name": "audit_actor_idx",
 "columns": [
 "actor_id"
],
 "isUnique": false
 },
 "audit_created_idx": {
 "name": "audit_created_idx",
 "columns": [
 "created_at"
],
 "isUnique": false
 },
 "audit_action_idx": {
 "name": "audit_action_idx",
 "columns": [
 "action"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "audit_log_actor_id_users_id_fk": {
 "name": "audit_log_actor_id_users_id_fk",
 "tableFrom": "audit_log",
 "tableTo": "users",
 "columnsFrom": [
 "actor_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
},
```

**drizzle/meta/0013\_snapshot.json**

```
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"event_roles": {
 "name": "event_roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "event_id": {
 "name": "event_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "emoji": {
 "name": "emoji",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "capacity": {
 "name": "capacity",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 }
 }
},
"indexes": {
 "event_roles_event_idx": {
 "name": "event_roles_event_idx",
```

**drizzle/meta/0013\_snapshot.json**

```
 "columns": [
 "event_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "event_roles_event_id_events_id_fk": {
 "name": "event_roles_event_id_events_id_fk",
 "tableFrom": "event_roles",
 "tableTo": "events",
 "columnsFrom": [
 "event_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"event_signups": {
 "name": "event_signups",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "event_id": {
 "name": "event_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "event_role_id": {
 "name": "event_role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
```

**drizzle/meta/0013\_snapshot.json**

```
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "event_signups_event_user_idx": {
 "name": "event_signups_event_user_idx",
 "columns": [
 "event_id",
 "user_id"
],
 "isUnique": true
 },
 "event_signups_event_idx": {
 "name": "event_signups_event_idx",
 "columns": [
 "event_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "event_signups_event_id_events_id_fk": {
 "name": "event_signups_event_id_events_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "events",
 "columnsFrom": [
 "event_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "event_signups_user_id_users_id_fk": {
 "name": "event_signups_user_id_users_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "users",
```

**drizzle/meta/0013\_snapshot.json**

```
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "event_signups_event_role_id_event_roles_id_fk": {
 "name": "event_signups_event_role_id_event_roles_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "event_roles",
 "columnsFrom": [
 "event_role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"events": {
 "name": "events",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
```

**drizzle/meta/0013\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "starts_at": {
 "name": "starts_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "ends_at": {
 "name": "ends_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_by": {
 "name": "created_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_event_id": {
 "name": "discord_event_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_message_id": {
 "name": "discord_message_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
```

**drizzle/meta/0013\_snapshot.json**

```
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'scheduled'"
 },
 "recurrence": {
 "name": "recurrence",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'none'"
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "events_starts_idx": {
 "name": "events_starts_idx",
 "columns": [
 "starts_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "events_game_id_games_id_fk": {
 "name": "events_game_id_games_id_fk",
 "tableFrom": "events",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
}
```

**drizzle/meta/0013\_snapshot.json**

```
 },
 "events_created_by_users_id_fk": {
 "name": "events_created_by_users_id_fk",
 "tableFrom": "events",
 "tableTo": "users",
 "columnsFrom": [
 "created_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"games": {
 "name": "games",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "icon_url": {
 "name": "icon_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
```



**drizzle/meta/0013\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "active": {
 "name": "active",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": true
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "games_slug_unique": {
 "name": "games_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"match_participants": {
 "name": "match_participants",
 "columns": {
 "match_id": {
 "name": "match_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "stats": {
 "name": "stats",
```

**drizzle/meta/0013\_snapshot.json**

```
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
},
"indexes": {
 "mp_user_idx": {
 "name": "mp_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "match_participants_match_id_matches_id_fk": {
 "name": "match_participants_match_id_matches_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "matches",
 "columnsFrom": [
 "match_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "match_participants_user_id_users_id_fk": {
 "name": "match_participants_user_id_users_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "match_participants_match_id_user_id_pk": {
 "columns": [
 "match_id",
 "user_id"
],
 "name": "match_participants_match_id_user_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
```

**drizzle/meta/0013\_snapshot.json**

```
{,
"matches": {
 "name": "matches",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent": {
 "name": "opponent",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent_tag": {
 "name": "opponent_tag",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "result": {
 "name": "result",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "score_us": {
 "name": "score_us",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "score_them": {
 "name": "score_them",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 },
"map": {
```

**drizzle/meta/0013\_snapshot.json**

```
 "name": "map",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "notes": {
 "name": "notes",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "played_at": {
 "name": "played_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "recorded_by": {
 "name": "recorded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "matches_game_played_idx": {
 "name": "matches_game_played_idx",
 "columns": [
 "game_id",
 "played_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "matches_game_id_games_id_fk": {
 "name": "matches_game_id_games_id_fk",
 "tableFrom": "matches",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
```

**drizzle/meta/0013\_snapshot.json**

```
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "matches_recorded_by_users_id_fk": {
 "name": "matches_recorded_by_users_id_fk",
 "tableFrom": "matches",
 "tableTo": "users",
 "columnsFrom": [
 "recorded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"medals": {
 "name": "medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
}
```

**drizzle/meta/0013\_snapshot.json**

```
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "auto_grant_months": {
 "name": "auto_grant_months",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "medals_game_idx": {
 "name": "medals_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "medals_game_id_games_id_fk": {
 "name": "medals_game_id_games_id_fk",
 "tableFrom": "medals",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 }
}
```

**drizzle/meta/0013\_snapshot.json**

```
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"member_medals": {
 "name": "member_medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "medal_id": {
 "name": "medal_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 }
},
"indexes": {
```

**drizzle/meta/0013\_snapshot.json**

```
"member_medals_user_idx": {
 "name": "member_medals_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
},
"member_medals_medal_idx": {
 "name": "member_medals_medal_idx",
 "columns": [
 "medal_id"
],
 "isUnique": false
}
},
"foreignKeys": {
 "member_medals_user_id_users_id_fk": {
 "name": "member_medals_user_id_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_medal_id_medals_id_fk": {
 "name": "member_medals_medal_id_medals_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "medals",
 "columnsFrom": [
 "medal_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_awarded_by_users_id_fk": {
 "name": "member_medals_awarded_by_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
}
```



**drizzle/meta/0013\_snapshot.json**

```
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
 },
 "member_war_records": {
 "name": "member_war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "war_record_id": {
 "name": "war_record_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 }
 },
 "indexes": {
```

**drizzle/meta/0013\_snapshot.json**

```
"member_war_records_user_idx": {
 "name": "member_war_records_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
},
"member_war_records_record_idx": {
 "name": "member_war_records_record_idx",
 "columns": [
 "war_record_id"
],
 "isUnique": false
}
},
"foreignKeys": {
 "member_war_records_user_id_users_id_fk": {
 "name": "member_war_records_user_id_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_war_record_id_war_records_id_fk": {
 "name": "member_war_records_war_record_id_war_records_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "war_records",
 "columnsFrom": [
 "war_record_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_awarded_by_users_id_fk": {
 "name": "member_war_records_awarded_by_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
}
```

**drizzle/meta/0013\_snapshot.json**

```
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
 },
 "news": {
 "name": "news",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "excerpt": {
 "name": "excerpt",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "body": {
 "name": "body",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "author_id": {
 "name": "author_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
```

**drizzle/meta/0013\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'draft'"
 },
 "pinned": {
 "name": "pinned",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "published_at": {
 "name": "published_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "news_slug_unique": {
 "name": "news_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 },
 "news_status_published_idx": {
 "name": "news_status_published_idx",
 "columns": [
 "status",
 "published_at"
],
 "isUnique": false
 }
},
},
```

**drizzle/meta/0013\_snapshot.json**

```
"foreignKeys": {
 "news_author_id_users_id_fk": {
 "name": "news_author_id_users_id_fk",
 "tableFrom": "news",
 "tableTo": "users",
 "columnsFrom": [
 "author_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"page_layouts": {
 "name": "page_layouts",
 "columns": {
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "layout": {
 "name": "layout",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "show_in_nav": {
 "name": "show_in_nav",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "nav_order": {
 "name": "nav_order",
 "type": "integer",
```

**drizzle/meta/0013\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {},
"foreignKeys": {
 "page_layouts_updated_by_users_id_fk": {
 "name": "page_layouts_updated_by_users_id_fk",
 "tableFrom": "page_layouts",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"profiles": {
 "name": "profiles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "bio": {
 "name": "bio",
 "type": "text",
```

**drizzle/meta/0013\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "timezone": {
 "name": "timezone",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "gamertags": {
 "name": "gamertags",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {},
"foreignKeys": {
 "profiles_user_id_users_id_fk": {
 "name": "profiles_user_id_users_id_fk",
 "tableFrom": "profiles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
```

**drizzle/meta/0013\_snapshot.json**

```
"rank_roles": {
 "name": "rank_roles",
 "columns": {
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "indexes": {
 "rank_roles_role_idx": {
 "name": "rank_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "rank_roles_rank_id_ranks_id_fk": {
 "name": "rank_roles_rank_id_ranks_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "rank_roles_role_id_roles_id_fk": {
 "name": "rank_roles_role_id_roles_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 }
}
```



**drizzle/meta/0013\_snapshot.json**

```
 },
 "compositePrimaryKeys": {
 "rank_roles_rank_id_role_id_pk": {
 "columns": [
 "rank_id",
 "role_id"
],
 "name": "rank_roles_rank_id_role_id_pk"
 }
 },
 "uniqueConstraints": {},
 "checkConstraints": {}
 },
 "ranks": {
 "name": "ranks",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "abbreviation": {
 "name": "abbreviation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "req_days": {
 "name": "req_days",
 "type": "integer",
```

**drizzle/meta/0013\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "req_wins": {
 "name": "req_wins",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_default": {
 "name": "is_default",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "ranks_sort_order_idx": {
 "name": "ranks_sort_order_idx",
 "columns": [
 "sort_order"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"role_permissions": {
 "name": "role_permissions",
```

**drizzle/meta/0013\_snapshot.json**

```
"columns": {
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "permission": {
 "name": "permission",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
},
"indexes": {},
"foreignKeys": {
 "role_permissions_role_id_roles_id_fk": {
 "name": "role_permissions_role_id_roles_id_fk",
 "tableFrom": "role_permissions",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "role_permissions_role_id_permission_pk": {
 "columns": [
 "role_id",
 "permission"
],
 "name": "role_permissions_role_id_permission_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"roles": {
 "name": "roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 }
 },
}
```

**drizzle/meta/0013\_snapshot.json**

```
"name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"color": {
 "name": "color",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"discord_role_id": {
 "name": "discord_role_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
},
"is_system": {
 "name": "is_system",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
},
"updated_at": {
 "name": "updated_at",
```

**drizzle/meta/0013\_snapshot.json**

```
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "roles_name_unique": {
 "name": "roles_name_unique",
 "columns": [
 "name"
],
 "isUnique": true
 },
 "roles_discord_role_id_unique": {
 "name": "roles_discord_role_id_unique",
 "columns": [
 "discord_role_id"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"sessions": {
 "name": "sessions",
 "columns": {
 "id": {
 "name": "id",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "expires_at": {
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_agent": {
 "name": "user_agent",
```

**drizzle/meta/0013\_snapshot.json**

```
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "sessions_user_idx": {
 "name": "sessions_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "sessions_expires_idx": {
 "name": "sessions_expires_idx",
 "columns": [
 "expires_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "sessions_user_id_users_id_fk": {
 "name": "sessions_user_id_users_id_fk",
 "tableFrom": "sessions",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
```

**drizzle/meta/0013\_snapshot.json**

```
"checkConstraints": {},
},
"settings": {
 "name": "settings",
 "columns": {
 "key": {
 "name": "key",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "value": {
 "name": "value",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {},
 "foreignKeys": {
 "settings_updated_by_users_id_fk": {
 "name": "settings_updated_by_users_id_fk",
 "tableFrom": "settings",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
}
```

**drizzle/meta/0013\_snapshot.json**

```
 },
 "user_roles": {
 "name": "user_roles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'manual'"
 },
 "granted_by": {
 "name": "granted_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "granted_at": {
 "name": "granted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 }
 },
 "indexes": {
 "user_roles_role_idx": {
 "name": "user_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "user_roles_user_id_users_id_fk": {
 "name": "user_roles_user_id_users_id_fk",
```



**drizzle/meta/0013\_snapshot.json**

```

 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_role_id_roles_id_fk": {
 "name": "user_roles_role_id_roles_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_granted_by_users_id_fk": {
 "name": "user_roles_granted_by_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "granted_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "user_roles_user_id_role_id_pk": {
 "columns": [
 "user_id",
 "role_id"
],
 "name": "user_roles_user_id_role_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"users": {
 "name": "users",
 "columns": {
 "id": {

```

**drizzle/meta/0013\_snapshot.json**

```
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "discord_id": {
 "name": "discord_id",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "username": {
 "name": "username",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "global_name": {
 "name": "global_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "display_name": {
 "name": "display_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "avatar": {
 "name": "avatar",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "profile_image_url": {
 "name": "profile_image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "email": {
 "name": "email",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
```

**drizzle/meta/0013\_snapshot.json**

```
,
"rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"is_god": {
 "name": "is_god",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
},
"guild_joined_at": {
 "name": "guild_joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'active'"
},
"recruiter_id": {
 "name": "recruiter_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"joined_at": {
 "name": "joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
},
"last_seen_at": {
 "name": "last_seen_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"last_reconciled_at": {
```

**drizzle/meta/0013\_snapshot.json**

```
 "name": "last_reconciled_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "promoted_at": {
 "name": "promoted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "users_discord_id_unique": {
 "name": "users_discord_id_unique",
 "columns": [
 "discord_id"
],
 "isUnique": true
 },
 "users_rank_idx": {
 "name": "users_rank_idx",
 "columns": [
 "rank_id"
],
 "isUnique": false
 },
 "users_status_idx": {
 "name": "users_status_idx",
 "columns": [
 "status"
],
 "isUnique": false
 }
},
"foreignKeys": {
```

**drizzle/meta/0013\_snapshot.json**

```
 "users_rank_id_ranks_id_fk": {
 "name": "users_rank_id_ranks_id_fk",
 "tableFrom": "users",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"war_records": {
 "name": "war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
```

**drizzle/meta/0013\_snapshot.json**

```

 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "war_records_game_idx": {
 "name": "war_records_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "war_records_game_id_games_id_fk": {
 "name": "war_records_game_id_games_id_fk",
 "tableFrom": "war_records",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
}
},
"views": {},
"enums": {},
"_meta": {
 "schemas": {},
 "tables": {},
 "columns": {}
}

```

**drizzle/meta/0013\_snapshot.json**

```
 },
 "internal": {
 "indexes": {}
 }
}
```

**drizzle/meta/0014\_snapshot.json**

```
{
 "version": "6",
 "dialect": "sqlite",
 "id": "c20c92db-aa96-44f2-b48d-b364ced4a77e",
 "prevId": "092e1d03-fdbf-4143-bcc2-600144c6b8f2",
 "tables": {
 "api_tokens": {
 "name": "api_tokens",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "token_hash": {
 "name": "token_hash",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "label": {
 "name": "label",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "prefix": {
 "name": "prefix",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "expires_at": {
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "last_used_at": {
 "name": "last_used_at",
 "type": "integer",
```



**drizzle/meta/0014\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "api_tokens_token_hash_unique": {
 "name": "api_tokens_token_hash_unique",
 "columns": [
 "token_hash"
],
 "isUnique": true
 },
 "api_tokens_user_idx": {
 "name": "api_tokens_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "api_tokens_user_id_users_id_fk": {
 "name": "api_tokens_user_id_users_id_fk",
 "tableFrom": "api_tokens",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"audit_log": {
 "name": "audit_log",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
```

**drizzle/meta/0014\_snapshot.json**

```
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "actor_id": {
 "name": "actor_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "action": {
 "name": "action",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "target_type": {
 "name": "target_type",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "target_id": {
 "name": "target_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "reason": {
 "name": "reason",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "meta": {
 "name": "meta",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'web'"
 },
}
```

**drizzle/meta/0014\_snapshot.json**

```
"ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
 "audit_actor_idx": {
 "name": "audit_actor_idx",
 "columns": [
 "actor_id"
],
 "isUnique": false
 },
 "audit_created_idx": {
 "name": "audit_created_idx",
 "columns": [
 "created_at"
],
 "isUnique": false
 },
 "audit_action_idx": {
 "name": "audit_action_idx",
 "columns": [
 "action"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "audit_log_actor_id_users_id_fk": {
 "name": "audit_log_actor_id_users_id_fk",
 "tableFrom": "audit_log",
 "tableTo": "users",
 "columnsFrom": [
 "actor_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
},
```

**drizzle/meta/0014\_snapshot.json**

```
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"event_roles": {
 "name": "event_roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "event_id": {
 "name": "event_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "emoji": {
 "name": "emoji",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "capacity": {
 "name": "capacity",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 }
 }
},
"indexes": {
 "event_roles_event_idx": {
 "name": "event_roles_event_idx",
```

**drizzle/meta/0014\_snapshot.json**

```
 "columns": [
 "event_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "event_roles_event_id_events_id_fk": {
 "name": "event_roles_event_id_events_id_fk",
 "tableFrom": "event_roles",
 "tableTo": "events",
 "columnsFrom": [
 "event_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"event_signups": {
 "name": "event_signups",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "event_id": {
 "name": "event_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "event_role_id": {
 "name": "event_role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
```

**drizzle/meta/0014\_snapshot.json**

```
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "event_signups_event_user_idx": {
 "name": "event_signups_event_user_idx",
 "columns": [
 "event_id",
 "user_id"
],
 "isUnique": true
 },
 "event_signups_event_idx": {
 "name": "event_signups_event_idx",
 "columns": [
 "event_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "event_signups_event_id_events_id_fk": {
 "name": "event_signups_event_id_events_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "events",
 "columnsFrom": [
 "event_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "event_signups_user_id_users_id_fk": {
 "name": "event_signups_user_id_users_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "users",
```

**drizzle/meta/0014\_snapshot.json**

```
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "event_signups_event_role_id_event_roles_id_fk": {
 "name": "event_signups_event_role_id_event_roles_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "event_roles",
 "columnsFrom": [
 "event_role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"events": {
 "name": "events",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
```

**drizzle/meta/0014\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "starts_at": {
 "name": "starts_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "ends_at": {
 "name": "ends_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_by": {
 "name": "created_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_event_id": {
 "name": "discord_event_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_message_id": {
 "name": "discord_message_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
```



**drizzle/meta/0014\_snapshot.json**

```
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'scheduled'"
 },
 "recurrence": {
 "name": "recurrence",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'none'"
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "events_starts_idx": {
 "name": "events_starts_idx",
 "columns": [
 "starts_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "events_game_id_games_id_fk": {
 "name": "events_game_id_games_id_fk",
 "tableFrom": "events",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
}
```

**drizzle/meta/0014\_snapshot.json**

```
 },
 "events_created_by_users_id_fk": {
 "name": "events_created_by_users_id_fk",
 "tableFrom": "events",
 "tableTo": "users",
 "columnsFrom": [
 "created_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"games": {
 "name": "games",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "icon_url": {
 "name": "icon_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
```

**drizzle/meta/0014\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "active": {
 "name": "active",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": true
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "games_slug_unique": {
 "name": "games_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"match_participants": {
 "name": "match_participants",
 "columns": {
 "match_id": {
 "name": "match_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "stats": {
 "name": "stats",
```

**drizzle/meta/0014\_snapshot.json**

```
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
},
"indexes": {
 "mp_user_idx": {
 "name": "mp_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "match_participants_match_id_matches_id_fk": {
 "name": "match_participants_match_id_matches_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "matches",
 "columnsFrom": [
 "match_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "match_participants_user_id_users_id_fk": {
 "name": "match_participants_user_id_users_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "match_participants_match_id_user_id_pk": {
 "columns": [
 "match_id",
 "user_id"
],
 "name": "match_participants_match_id_user_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
```

**drizzle/meta/0014\_snapshot.json**

```
,
"matches": {
 "name": "matches",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent": {
 "name": "opponent",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent_tag": {
 "name": "opponent_tag",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "result": {
 "name": "result",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "score_us": {
 "name": "score_us",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "score_them": {
 "name": "score_them",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 },
 "map": {
```

**drizzle/meta/0014\_snapshot.json**

```
 "name": "map",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "notes": {
 "name": "notes",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "played_at": {
 "name": "played_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "recorded_by": {
 "name": "recorded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "matches_game_played_idx": {
 "name": "matches_game_played_idx",
 "columns": [
 "game_id",
 "played_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "matches_game_id_games_id_fk": {
 "name": "matches_game_id_games_id_fk",
 "tableFrom": "matches",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
```

**drizzle/meta/0014\_snapshot.json**

```
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "matches_recorded_by_users_id_fk": {
 "name": "matches_recorded_by_users_id_fk",
 "tableFrom": "matches",
 "tableTo": "users",
 "columnsFrom": [
 "recorded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"medals": {
 "name": "medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
}
```

**drizzle/meta/0014\_snapshot.json**

```
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "auto_grant_months": {
 "name": "auto_grant_months",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "medals_game_idx": {
 "name": "medals_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "medals_game_id_games_id_fk": {
 "name": "medals_game_id_games_id_fk",
 "tableFrom": "medals",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 }
}
```



**drizzle/meta/0014\_snapshot.json**

```
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"member_medals": {
 "name": "member_medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "medal_id": {
 "name": "medal_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 }
},
"indexes": {
```

**drizzle/meta/0014\_snapshot.json**

```
"member_medals_user_idx": {
 "name": "member_medals_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
},
"member_medals_medal_idx": {
 "name": "member_medals_medal_idx",
 "columns": [
 "medal_id"
],
 "isUnique": false
}
},
"foreignKeys": {
 "member_medals_user_id_users_id_fk": {
 "name": "member_medals_user_id_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_medal_id_medals_id_fk": {
 "name": "member_medals_medal_id_medals_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "medals",
 "columnsFrom": [
 "medal_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_awarded_by_users_id_fk": {
 "name": "member_medals_awarded_by_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
}
```

**drizzle/meta/0014\_snapshot.json**

```
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
 },
 "member_war_records": {
 "name": "member_war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "war_record_id": {
 "name": "war_record_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 }
 },
 "indexes": {
```

**drizzle/meta/0014\_snapshot.json**

```
"member_war_records_user_idx": {
 "name": "member_war_records_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
},
"member_war_records_record_idx": {
 "name": "member_war_records_record_idx",
 "columns": [
 "war_record_id"
],
 "isUnique": false
}
},
"foreignKeys": {
 "member_war_records_user_id_users_id_fk": {
 "name": "member_war_records_user_id_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_war_record_id_war_records_id_fk": {
 "name": "member_war_records_war_record_id_war_records_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "war_records",
 "columnsFrom": [
 "war_record_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_awarded_by_users_id_fk": {
 "name": "member_war_records_awarded_by_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
}
```

**drizzle/meta/0014\_snapshot.json**

```
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
 },
 "news": {
 "name": "news",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "excerpt": {
 "name": "excerpt",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "body": {
 "name": "body",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "author_id": {
 "name": "author_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
```

**drizzle/meta/0014\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'draft'"
 },
 "pinned": {
 "name": "pinned",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "published_at": {
 "name": "published_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "news_slug_unique": {
 "name": "news_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 },
 "news_status_published_idx": {
 "name": "news_status_published_idx",
 "columns": [
 "status",
 "published_at"
],
 "isUnique": false
 }
},
}
```

**drizzle/meta/0014\_snapshot.json**

```
"foreignKeys": {
 "news_author_id_users_id_fk": {
 "name": "news_author_id_users_id_fk",
 "tableFrom": "news",
 "tableTo": "users",
 "columnsFrom": [
 "author_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"page_layouts": {
 "name": "page_layouts",
 "columns": {
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "layout": {
 "name": "layout",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "show_in_nav": {
 "name": "show_in_nav",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "nav_order": {
 "name": "nav_order",
 "type": "integer",
```

**drizzle/meta/0014\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {},
"foreignKeys": {
 "page_layouts_updated_by_users_id_fk": {
 "name": "page_layouts_updated_by_users_id_fk",
 "tableFrom": "page_layouts",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"profiles": {
 "name": "profiles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "bio": {
 "name": "bio",
 "type": "text",
```



**drizzle/meta/0014\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "timezone": {
 "name": "timezone",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "gamertags": {
 "name": "gamertags",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {},
"foreignKeys": {
 "profiles_user_id_users_id_fk": {
 "name": "profiles_user_id_users_id_fk",
 "tableFrom": "profiles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
```

**drizzle/meta/0014\_snapshot.json**

```
"rank_roles": {
 "name": "rank_roles",
 "columns": {
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "indexes": {
 "rank_roles_role_idx": {
 "name": "rank_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "rank_roles_rank_id_ranks_id_fk": {
 "name": "rank_roles_rank_id_ranks_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "rank_roles_role_id_roles_id_fk": {
 "name": "rank_roles_role_id_roles_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 }
}
```

**drizzle/meta/0014\_snapshot.json**

```
 },
 "compositePrimaryKeys": {
 "rank_roles_rank_id_role_id_pk": {
 "columns": [
 "rank_id",
 "role_id"
],
 "name": "rank_roles_rank_id_role_id_pk"
 }
 },
 "uniqueConstraints": {},
 "checkConstraints": {}
 },
 "ranks": {
 "name": "ranks",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "abbreviation": {
 "name": "abbreviation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "req_days": {
 "name": "req_days",
 "type": "integer",
```

**drizzle/meta/0014\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "req_wins": {
 "name": "req_wins",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_default": {
 "name": "is_default",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "ranks_sort_order_idx": {
 "name": "ranks_sort_order_idx",
 "columns": [
 "sort_order"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"role_permissions": {
 "name": "role_permissions",
```

**drizzle/meta/0014\_snapshot.json**

```
"columns": {
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "permission": {
 "name": "permission",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
},
"indexes": {},
"foreignKeys": {
 "role_permissions_role_id_roles_id_fk": {
 "name": "role_permissions_role_id_roles_id_fk",
 "tableFrom": "role_permissions",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "role_permissions_role_id_permission_pk": {
 "columns": [
 "role_id",
 "permission"
],
 "name": "role_permissions_role_id_permission_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"roles": {
 "name": "roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 }
 },
}
```

**drizzle/meta/0014\_snapshot.json**

```
"name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"color": {
 "name": "color",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"discord_role_id": {
 "name": "discord_role_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
},
"is_system": {
 "name": "is_system",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
},
"updated_at": {
 "name": "updated_at",
```

**drizzle/meta/0014\_snapshot.json**

```
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "roles_name_unique": {
 "name": "roles_name_unique",
 "columns": [
 "name"
],
 "isUnique": true
 },
 "roles_discord_role_id_unique": {
 "name": "roles_discord_role_id_unique",
 "columns": [
 "discord_role_id"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"sc_hangars": {
 "name": "sc_hangars",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "items": {
 "name": "items",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "item_count": {
 "name": "item_count",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 }
 },
 "imported_at": {
```

**drizzle/meta/0014\_snapshot.json**

```
 "name": "imported_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {},
"foreignKeys": {
 "sc_hangars_user_id_users_id_fk": {
 "name": "sc_hangars_user_id_users_id_fk",
 "tableFrom": "sc_hangars",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"sessions": {
 "name": "sessions",
 "columns": {
 "id": {
 "name": "id",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "expires_at": {
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_agent": {
 "name": "user_agent",
```



**drizzle/meta/0014\_snapshot.json**

```
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "sessions_user_idx": {
 "name": "sessions_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "sessions_expires_idx": {
 "name": "sessions_expires_idx",
 "columns": [
 "expires_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "sessions_user_id_users_id_fk": {
 "name": "sessions_user_id_users_id_fk",
 "tableFrom": "sessions",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
```

**drizzle/meta/0014\_snapshot.json**

```
"checkConstraints": {},
},
"settings": {
 "name": "settings",
 "columns": {
 "key": {
 "name": "key",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "value": {
 "name": "value",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
},
"indexes": {},
"foreignKeys": {
 "settings_updated_by_users_id_fk": {
 "name": "settings_updated_by_users_id_fk",
 "tableFrom": "settings",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
```

**drizzle/meta/0014\_snapshot.json**

```
 },
 "user_roles": {
 "name": "user_roles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'manual'"
 },
 "granted_by": {
 "name": "granted_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "granted_at": {
 "name": "granted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 }
 },
 "indexes": {
 "user_roles_role_idx": {
 "name": "user_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "user_roles_user_id_users_id_fk": {
 "name": "user_roles_user_id_users_id_fk",
```

**drizzle/meta/0014\_snapshot.json**

```
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_role_id_roles_id_fk": {
 "name": "user_roles_role_id_roles_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_granted_by_users_id_fk": {
 "name": "user_roles_granted_by_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "granted_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "user_roles_user_id_role_id_pk": {
 "columns": [
 "user_id",
 "role_id"
],
 "name": "user_roles_user_id_role_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"users": {
 "name": "users",
 "columns": {
 "id": {
```

**drizzle/meta/0014\_snapshot.json**

```
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "discord_id": {
 "name": "discord_id",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "username": {
 "name": "username",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "global_name": {
 "name": "global_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "display_name": {
 "name": "display_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "avatar": {
 "name": "avatar",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "profile_image_url": {
 "name": "profile_image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "email": {
 "name": "email",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
```

**drizzle/meta/0014\_snapshot.json**

```
{,
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "is_god": {
 "name": "is_god",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "guild_joined_at": {
 "name": "guild_joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'active'"
 },
 "recruiter_id": {
 "name": "recruiter_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "joined_at": {
 "name": "joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "last_seen_at": {
 "name": "last_seen_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "last_reconciled_at": {
```

**drizzle/meta/0014\_snapshot.json**

```
 "name": "last_reconciled_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "promoted_at": {
 "name": "promoted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "users_discord_id_unique": {
 "name": "users_discord_id_unique",
 "columns": [
 "discord_id"
],
 "isUnique": true
 },
 "users_rank_idx": {
 "name": "users_rank_idx",
 "columns": [
 "rank_id"
],
 "isUnique": false
 },
 "users_status_idx": {
 "name": "users_status_idx",
 "columns": [
 "status"
],
 "isUnique": false
 }
},
"foreignKeys": {
```

**drizzle/meta/0014\_snapshot.json**

```
 "users_rank_id_ranks_id_fk": {
 "name": "users_rank_id_ranks_id_fk",
 "tableFrom": "users",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"war_records": {
 "name": "war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
```



**drizzle/meta/0014\_snapshot.json**

```
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "war_records_game_idx": {
 "name": "war_records_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "war_records_game_id_games_id_fk": {
 "name": "war_records_game_id_games_id_fk",
 "tableFrom": "war_records",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
}
},
"views": {},
"enums": {},
"_meta": {
 "schemas": {},
 "tables": {},
 "columns": {}
}
```

**drizzle/meta/0014\_snapshot.json**

```
 },
 "internal": {
 "indexes": {}
 }
}
```

**drizzle/meta/0015\_snapshot.json**

```
{
 "version": "6",
 "dialect": "sqlite",
 "id": "6748edb1-ef34-49a9-9645-5b550972216b",
 "prevId": "c20c92db-aa96-44f2-b48d-b364ced4a77e",
 "tables": {
 "api_tokens": {
 "name": "api_tokens",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "token_hash": {
 "name": "token_hash",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "label": {
 "name": "label",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "prefix": {
 "name": "prefix",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "expires_at": {
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "last_used_at": {
 "name": "last_used_at",
 "type": "integer",

```

**drizzle/meta/0015\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "api_tokens_token_hash_unique": {
 "name": "api_tokens_token_hash_unique",
 "columns": [
 "token_hash"
],
 "isUnique": true
 },
 "api_tokens_user_idx": {
 "name": "api_tokens_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "api_tokens_user_id_users_id_fk": {
 "name": "api_tokens_user_id_users_id_fk",
 "tableFrom": "api_tokens",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"audit_log": {
 "name": "audit_log",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
```

**drizzle/meta/0015\_snapshot.json**

```
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "actor_id": {
 "name": "actor_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "action": {
 "name": "action",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "target_type": {
 "name": "target_type",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "target_id": {
 "name": "target_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "reason": {
 "name": "reason",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "meta": {
 "name": "meta",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'web'"
 },
}
```

**drizzle/meta/0015\_snapshot.json**

```
"ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
 "audit_actor_idx": {
 "name": "audit_actor_idx",
 "columns": [
 "actor_id"
],
 "isUnique": false
 },
 "audit_created_idx": {
 "name": "audit_created_idx",
 "columns": [
 "created_at"
],
 "isUnique": false
 },
 "audit_action_idx": {
 "name": "audit_action_idx",
 "columns": [
 "action"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "audit_log_actor_id_users_id_fk": {
 "name": "audit_log_actor_id_users_id_fk",
 "tableFrom": "audit_log",
 "tableTo": "users",
 "columnsFrom": [
 "actor_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
},
```

**drizzle/meta/0015\_snapshot.json**

```
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"event_roles": {
 "name": "event_roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "event_id": {
 "name": "event_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "emoji": {
 "name": "emoji",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "capacity": {
 "name": "capacity",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 }
 }
},
"indexes": {
 "event_roles_event_idx": {
 "name": "event_roles_event_idx",
```

**drizzle/meta/0015\_snapshot.json**

```
 "columns": [
 "event_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "event_roles_event_id_events_id_fk": {
 "name": "event_roles_event_id_events_id_fk",
 "tableFrom": "event_roles",
 "tableTo": "events",
 "columnsFrom": [
 "event_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"event_signups": {
 "name": "event_signups",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "event_id": {
 "name": "event_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "event_role_id": {
 "name": "event_role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
```



**drizzle/meta/0015\_snapshot.json**

```
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "event_signups_event_user_idx": {
 "name": "event_signups_event_user_idx",
 "columns": [
 "event_id",
 "user_id"
],
 "isUnique": true
 },
 "event_signups_event_idx": {
 "name": "event_signups_event_idx",
 "columns": [
 "event_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "event_signups_event_id_events_id_fk": {
 "name": "event_signups_event_id_events_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "events",
 "columnsFrom": [
 "event_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "event_signups_user_id_users_id_fk": {
 "name": "event_signups_user_id_users_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "users",
```

**drizzle/meta/0015\_snapshot.json**

```
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "event_signups_event_role_id_event_roles_id_fk": {
 "name": "event_signups_event_role_id_event_roles_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "event_roles",
 "columnsFrom": [
 "event_role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"events": {
 "name": "events",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
```

**drizzle/meta/0015\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "starts_at": {
 "name": "starts_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "ends_at": {
 "name": "ends_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_by": {
 "name": "created_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_event_id": {
 "name": "discord_event_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_message_id": {
 "name": "discord_message_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
```

**drizzle/meta/0015\_snapshot.json**

```
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'scheduled'"
 },
 "recurrence": {
 "name": "recurrence",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'none'"
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "events_starts_idx": {
 "name": "events_starts_idx",
 "columns": [
 "starts_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "events_game_id_games_id_fk": {
 "name": "events_game_id_games_id_fk",
 "tableFrom": "events",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
}
```

**drizzle/meta/0015\_snapshot.json**

```
 },
 "events_created_by_users_id_fk": {
 "name": "events_created_by_users_id_fk",
 "tableFrom": "events",
 "tableTo": "users",
 "columnsFrom": [
 "created_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"games": {
 "name": "games",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "icon_url": {
 "name": "icon_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
```

**drizzle/meta/0015\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "active": {
 "name": "active",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": true
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "games_slug_unique": {
 "name": "games_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"match_participants": {
 "name": "match_participants",
 "columns": {
 "match_id": {
 "name": "match_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "stats": {
 "name": "stats",
```

**drizzle/meta/0015\_snapshot.json**

```
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
},
"indexes": {
 "mp_user_idx": {
 "name": "mp_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "match_participants_match_id_matches_id_fk": {
 "name": "match_participants_match_id_matches_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "matches",
 "columnsFrom": [
 "match_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "match_participants_user_id_users_id_fk": {
 "name": "match_participants_user_id_users_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "match_participants_match_id_user_id_pk": {
 "columns": [
 "match_id",
 "user_id"
],
 "name": "match_participants_match_id_user_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
```

**drizzle/meta/0015\_snapshot.json**

```
 },
 "matches": {
 "name": "matches",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent": {
 "name": "opponent",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent_tag": {
 "name": "opponent_tag",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "result": {
 "name": "result",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "score_us": {
 "name": "score_us",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "score_them": {
 "name": "score_them",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
 },
 "map": {
```



**drizzle/meta/0015\_snapshot.json**

```
 "name": "map",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "notes": {
 "name": "notes",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "played_at": {
 "name": "played_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "recorded_by": {
 "name": "recorded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "matches_game_played_idx": {
 "name": "matches_game_played_idx",
 "columns": [
 "game_id",
 "played_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "matches_game_id_games_id_fk": {
 "name": "matches_game_id_games_id_fk",
 "tableFrom": "matches",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
```

**drizzle/meta/0015\_snapshot.json**

```
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "matches_recorded_by_users_id_fk": {
 "name": "matches_recorded_by_users_id_fk",
 "tableFrom": "matches",
 "tableTo": "users",
 "columnsFrom": [
 "recorded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"medals": {
 "name": "medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
}
```

**drizzle/meta/0015\_snapshot.json**

```
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "auto_grant_months": {
 "name": "auto_grant_months",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "medals_game_idx": {
 "name": "medals_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "medals_game_id_games_id_fk": {
 "name": "medals_game_id_games_id_fk",
 "tableFrom": "medals",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 }
}
```

**drizzle/meta/0015\_snapshot.json**

```
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"member_medals": {
 "name": "member_medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "medal_id": {
 "name": "medal_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 }
},
"indexes": {
```

**drizzle/meta/0015\_snapshot.json**

```
"member_medals_user_idx": {
 "name": "member_medals_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
},
"member_medals_medal_idx": {
 "name": "member_medals_medal_idx",
 "columns": [
 "medal_id"
],
 "isUnique": false
}
},
"foreignKeys": {
 "member_medals_user_id_users_id_fk": {
 "name": "member_medals_user_id_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_medal_id_medals_id_fk": {
 "name": "member_medals_medal_id_medals_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "medals",
 "columnsFrom": [
 "medal_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_awarded_by_users_id_fk": {
 "name": "member_medals_awarded_by_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
}
```

**drizzle/meta/0015\_snapshot.json**

```
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
 },
 "member_war_records": {
 "name": "member_war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "war_record_id": {
 "name": "war_record_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 }
 },
 "indexes": {
```

**drizzle/meta/0015\_snapshot.json**

```
"member_war_records_user_idx": {
 "name": "member_war_records_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
},
"member_war_records_record_idx": {
 "name": "member_war_records_record_idx",
 "columns": [
 "war_record_id"
],
 "isUnique": false
}
},
"foreignKeys": {
 "member_war_records_user_id_users_id_fk": {
 "name": "member_war_records_user_id_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_war_record_id_war_records_id_fk": {
 "name": "member_war_records_war_record_id_war_records_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "war_records",
 "columnsFrom": [
 "war_record_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_awarded_by_users_id_fk": {
 "name": "member_war_records_awarded_by_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
}
```

**drizzle/meta/0015\_snapshot.json**

```
 },
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
 },
 "news": {
 "name": "news",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "excerpt": {
 "name": "excerpt",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "body": {
 "name": "body",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "author_id": {
 "name": "author_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
```



**drizzle/meta/0015\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'draft'"
 },
 "pinned": {
 "name": "pinned",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "published_at": {
 "name": "published_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "news_slug_unique": {
 "name": "news_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 },
 "news_status_published_idx": {
 "name": "news_status_published_idx",
 "columns": [
 "status",
 "published_at"
],
 "isUnique": false
 }
},
}
```

**drizzle/meta/0015\_snapshot.json**

```
"foreignKeys": {
 "news_author_id_users_id_fk": {
 "name": "news_author_id_users_id_fk",
 "tableFrom": "news",
 "tableTo": "users",
 "columnsFrom": [
 "author_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"page_layouts": {
 "name": "page_layouts",
 "columns": {
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "layout": {
 "name": "layout",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "show_in_nav": {
 "name": "show_in_nav",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "nav_order": {
 "name": "nav_order",
 "type": "integer",
```

**drizzle/meta/0015\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {},
"foreignKeys": {
 "page_layouts_updated_by_users_id_fk": {
 "name": "page_layouts_updated_by_users_id_fk",
 "tableFrom": "page_layouts",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"profiles": {
 "name": "profiles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "bio": {
 "name": "bio",
 "type": "text",
```

**drizzle/meta/0015\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "timezone": {
 "name": "timezone",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "gamertags": {
 "name": "gamertags",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {},
"foreignKeys": {
 "profiles_user_id_users_id_fk": {
 "name": "profiles_user_id_users_id_fk",
 "tableFrom": "profiles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
```

**drizzle/meta/0015\_snapshot.json**

```
"rank_roles": {
 "name": "rank_roles",
 "columns": {
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "indexes": {
 "rank_roles_role_idx": {
 "name": "rank_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "rank_roles_rank_id_ranks_id_fk": {
 "name": "rank_roles_rank_id_ranks_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "rank_roles_role_id_roles_id_fk": {
 "name": "rank_roles_role_id_roles_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 }
}
```

**drizzle/meta/0015\_snapshot.json**

```
 },
 "compositePrimaryKeys": {
 "rank_roles_rank_id_role_id_pk": {
 "columns": [
 "rank_id",
 "role_id"
],
 "name": "rank_roles_rank_id_role_id_pk"
 }
 },
 "uniqueConstraints": {},
 "checkConstraints": {}
 },
 "ranks": {
 "name": "ranks",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "abbreviation": {
 "name": "abbreviation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "req_days": {
 "name": "req_days",
 "type": "integer",
```

**drizzle/meta/0015\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "req_wins": {
 "name": "req_wins",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_default": {
 "name": "is_default",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "ranks_sort_order_idx": {
 "name": "ranks_sort_order_idx",
 "columns": [
 "sort_order"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"role_permissions": {
 "name": "role_permissions",
```

**drizzle/meta/0015\_snapshot.json**

```
"columns": {
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "permission": {
 "name": "permission",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
},
"indexes": {},
"foreignKeys": {
 "role_permissions_role_id_roles_id_fk": {
 "name": "role_permissions_role_id_roles_id_fk",
 "tableFrom": "role_permissions",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "role_permissions_role_id_permission_pk": {
 "columns": [
 "role_id",
 "permission"
],
 "name": "role_permissions_role_id_permission_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"roles": {
 "name": "roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 }
 },
}
```



**drizzle/meta/0015\_snapshot.json**

```
"name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"color": {
 "name": "color",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"discord_role_id": {
 "name": "discord_role_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
},
"is_system": {
 "name": "is_system",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
},
"updated_at": {
 "name": "updated_at",
```

**drizzle/meta/0015\_snapshot.json**

```
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "roles_name_unique": {
 "name": "roles_name_unique",
 "columns": [
 "name"
],
 "isUnique": true
 },
 "roles_discord_role_id_unique": {
 "name": "roles_discord_role_id_unique",
 "columns": [
 "discord_role_id"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"sc_hangars": {
 "name": "sc_hangars",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "items": {
 "name": "items",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "item_count": {
 "name": "item_count",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 }
 },
 "is_public": {
```

**drizzle/meta/0015\_snapshot.json**

```
 "name": "is_public",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "imported_at": {
 "name": "imported_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {},
"foreignKeys": {
 "sc_hangars_user_id_users_id_fk": {
 "name": "sc_hangars_user_id_users_id_fk",
 "tableFrom": "sc_hangars",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"sessions": {
 "name": "sessions",
 "columns": {
 "id": {
 "name": "id",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "expires_at": {
```

**drizzle/meta/0015\_snapshot.json**

```
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_agent": {
 "name": "user_agent",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "sessions_user_idx": {
 "name": "sessions_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "sessions_expires_idx": {
 "name": "sessions_expires_idx",
 "columns": [
 "expires_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "sessions_user_id_users_id_fk": {
 "name": "sessions_user_id_users_id_fk",
 "tableFrom": "sessions",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
```

**drizzle/meta/0015\_snapshot.json**

```
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
}
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"settings": {
 "name": "settings",
 "columns": {
 "key": {
 "name": "key",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "value": {
 "name": "value",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 }
},
"indexes": {},
"foreignKeys": {
 "settings_updated_by_users_id_fk": {
 "name": "settings_updated_by_users_id_fk",
 "tableFrom": "settings",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
]
 }
}
```

**drizzle/meta/0015\_snapshot.json**

```
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"user_roles": {
 "name": "user_roles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'manual'"
 },
 "granted_by": {
 "name": "granted_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "granted_at": {
 "name": "granted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 }
},
"indexes": {
 "user_roles_role_idx": {
 "name": "user_roles_role_idx",
 "columns": [
```

**drizzle/meta/0015\_snapshot.json**

```
 "role_id"
],
 "isUnique": false
}
},
"foreignKeys": {
 "user_roles_user_id_users_id_fk": {
 "name": "user_roles_user_id_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_role_id_roles_id_fk": {
 "name": "user_roles_role_id_roles_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_granted_by_users_id_fk": {
 "name": "user_roles_granted_by_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "granted_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "user_roles_user_id_role_id_pk": {
 "columns": [
 "user_id",
 "role_id"
],
 "name": "user_roles_user_id_role_id_pk"
 }
}
```

**drizzle/meta/0015\_snapshot.json**

```
 },
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"users": {
 "name": "users",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "discord_id": {
 "name": "discord_id",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "username": {
 "name": "username",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "global_name": {
 "name": "global_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "display_name": {
 "name": "display_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "avatar": {
 "name": "avatar",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "profile_image_url": {
 "name": "profile_image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
```



**drizzle/meta/0015\_snapshot.json**

```
 "autoincrement": false
 },
 "email": {
 "name": "email",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "is_god": {
 "name": "is_god",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "guild_joined_at": {
 "name": "guild_joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'active'"
 },
 "recruiter_id": {
 "name": "recruiter_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "joined_at": {
 "name": "joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
}
```

**drizzle/meta/0015\_snapshot.json**

```
"last_seen_at": {
 "name": "last_seen_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"last_reconciled_at": {
 "name": "last_reconciled_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"promoted_at": {
 "name": "promoted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
},
"updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
 "users_discord_id_unique": {
 "name": "users_discord_id_unique",
 "columns": [
 "discord_id"
],
 "isUnique": true
 },
 "users_rank_idx": {
 "name": "users_rank_idx",
 "columns": [
 "rank_id"
],
 "isUnique": false
 },
 "users_status_idx": {
```

**drizzle/meta/0015\_snapshot.json**

```
 "name": "users_status_idx",
 "columns": [
 "status"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "users_rank_id_ranks_id_fk": {
 "name": "users_rank_id_ranks_id_fk",
 "tableFrom": "users",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"war_records": {
 "name": "war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
```

**drizzle/meta/0015\_snapshot.json**

```
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "war_records_game_idx": {
 "name": "war_records_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "war_records_game_id_games_id_fk": {
 "name": "war_records_game_id_games_id_fk",
 "tableFrom": "war_records",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
```

**drizzle/meta/0015\_snapshot.json**

```
 }
 },
 "views": {},
 "enums": {},
 "_meta": {
 "schemas": {},
 "tables": {},
 "columns": {}
 },
 "internal": {
 "indexes": {}
 }
}
```

**drizzle/meta/0016\_snapshot.json**

```
{
 "version": "6",
 "dialect": "sqlite",
 "id": "cf8f5204-e028-4493-a798-ccfb607fb9e2",
 "prevId": "6748edb1-ef34-49a9-9645-5b550972216b",
 "tables": {
 "api_tokens": {
 "name": "api_tokens",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "token_hash": {
 "name": "token_hash",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "label": {
 "name": "label",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "prefix": {
 "name": "prefix",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "expires_at": {
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "last_used_at": {
 "name": "last_used_at",
 "type": "integer",
```

**drizzle/meta/0016\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "api_tokens_token_hash_unique": {
 "name": "api_tokens_token_hash_unique",
 "columns": [
 "token_hash"
],
 "isUnique": true
 },
 "api_tokens_user_idx": {
 "name": "api_tokens_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "api_tokens_user_id_users_id_fk": {
 "name": "api_tokens_user_id_users_id_fk",
 "tableFrom": "api_tokens",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"audit_log": {
 "name": "audit_log",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
```

**drizzle/meta/0016\_snapshot.json**

```
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "actor_id": {
 "name": "actor_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "action": {
 "name": "action",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "target_type": {
 "name": "target_type",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "target_id": {
 "name": "target_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "reason": {
 "name": "reason",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "meta": {
 "name": "meta",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'web'"
 },
}
```



**drizzle/meta/0016\_snapshot.json**

```
"ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
 "audit_actor_idx": {
 "name": "audit_actor_idx",
 "columns": [
 "actor_id"
],
 "isUnique": false
 },
 "audit_created_idx": {
 "name": "audit_created_idx",
 "columns": [
 "created_at"
],
 "isUnique": false
 },
 "audit_action_idx": {
 "name": "audit_action_idx",
 "columns": [
 "action"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "audit_log_actor_id_users_id_fk": {
 "name": "audit_log_actor_id_users_id_fk",
 "tableFrom": "audit_log",
 "tableTo": "users",
 "columnsFrom": [
 "actor_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
},
```

**drizzle/meta/0016\_snapshot.json**

```
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"event_roles": {
 "name": "event_roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "event_id": {
 "name": "event_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "emoji": {
 "name": "emoji",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "capacity": {
 "name": "capacity",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 }
 }
},
"indexes": {
 "event_roles_event_idx": {
 "name": "event_roles_event_idx",
```

**drizzle/meta/0016\_snapshot.json**

```
 "columns": [
 "event_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "event_roles_event_id_events_id_fk": {
 "name": "event_roles_event_id_events_id_fk",
 "tableFrom": "event_roles",
 "tableTo": "events",
 "columnsFrom": [
 "event_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"event_signups": {
 "name": "event_signups",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "event_id": {
 "name": "event_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "event_role_id": {
 "name": "event_role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
```

**drizzle/meta/0016\_snapshot.json**

```
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "event_signups_event_user_idx": {
 "name": "event_signups_event_user_idx",
 "columns": [
 "event_id",
 "user_id"
],
 "isUnique": true
 },
 "event_signups_event_idx": {
 "name": "event_signups_event_idx",
 "columns": [
 "event_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "event_signups_event_id_events_id_fk": {
 "name": "event_signups_event_id_events_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "events",
 "columnsFrom": [
 "event_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "event_signups_user_id_users_id_fk": {
 "name": "event_signups_user_id_users_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "users",
```

**drizzle/meta/0016\_snapshot.json**

```
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "event_signups_event_role_id_event_roles_id_fk": {
 "name": "event_signups_event_role_id_event_roles_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "event_roles",
 "columnsFrom": [
 "event_role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"events": {
 "name": "events",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
```

**drizzle/meta/0016\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "starts_at": {
 "name": "starts_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "ends_at": {
 "name": "ends_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_by": {
 "name": "created_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_event_id": {
 "name": "discord_event_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_message_id": {
 "name": "discord_message_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
```

**drizzle/meta/0016\_snapshot.json**

```
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'scheduled'"
 },
 "recurrence": {
 "name": "recurrence",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'none'"
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "events_starts_idx": {
 "name": "events_starts_idx",
 "columns": [
 "starts_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "events_game_id_games_id_fk": {
 "name": "events_game_id_games_id_fk",
 "tableFrom": "events",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
}
```

**drizzle/meta/0016\_snapshot.json**

```
 },
 "events_created_by_users_id_fk": {
 "name": "events_created_by_users_id_fk",
 "tableFrom": "events",
 "tableTo": "users",
 "columnsFrom": [
 "created_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"games": {
 "name": "games",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "icon_url": {
 "name": "icon_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
```



**drizzle/meta/0016\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "active": {
 "name": "active",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": true
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "games_slug_unique": {
 "name": "games_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"match_participants": {
 "name": "match_participants",
 "columns": {
 "match_id": {
 "name": "match_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "stats": {
 "name": "stats",
```

**drizzle/meta/0016\_snapshot.json**

```
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
},
"indexes": {
 "mp_user_idx": {
 "name": "mp_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "match_participants_match_id_matches_id_fk": {
 "name": "match_participants_match_id_matches_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "matches",
 "columnsFrom": [
 "match_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "match_participants_user_id_users_id_fk": {
 "name": "match_participants_user_id_users_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "match_participants_match_id_user_id_pk": {
 "columns": [
 "match_id",
 "user_id"
],
 "name": "match_participants_match_id_user_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
```

**drizzle/meta/0016\_snapshot.json**

```
,
"matches": {
 "name": "matches",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent": {
 "name": "opponent",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent_tag": {
 "name": "opponent_tag",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "result": {
 "name": "result",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "score_us": {
 "name": "score_us",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "score_them": {
 "name": "score_them",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 },
 "map": {
```

**drizzle/meta/0016\_snapshot.json**

```
 "name": "map",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "notes": {
 "name": "notes",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "played_at": {
 "name": "played_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "recorded_by": {
 "name": "recorded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "matches_game_played_idx": {
 "name": "matches_game_played_idx",
 "columns": [
 "game_id",
 "played_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "matches_game_id_games_id_fk": {
 "name": "matches_game_id_games_id_fk",
 "tableFrom": "matches",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
```

**drizzle/meta/0016\_snapshot.json**

```
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "matches_recorded_by_users_id_fk": {
 "name": "matches_recorded_by_users_id_fk",
 "tableFrom": "matches",
 "tableTo": "users",
 "columnsFrom": [
 "recorded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"medals": {
 "name": "medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
}
```

**drizzle/meta/0016\_snapshot.json**

```
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "auto_grant_months": {
 "name": "auto_grant_months",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "medals_game_idx": {
 "name": "medals_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "medals_game_id_games_id_fk": {
 "name": "medals_game_id_games_id_fk",
 "tableFrom": "medals",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 }
}
```

**drizzle/meta/0016\_snapshot.json**

```
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"member_medals": {
 "name": "member_medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "medal_id": {
 "name": "medal_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 }
},
"indexes": {
```

**drizzle/meta/0016\_snapshot.json**

```
"member_medals_user_idx": {
 "name": "member_medals_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
},
"member_medals_medal_idx": {
 "name": "member_medals_medal_idx",
 "columns": [
 "medal_id"
],
 "isUnique": false
}
},
"foreignKeys": {
 "member_medals_user_id_users_id_fk": {
 "name": "member_medals_user_id_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_medal_id_medals_id_fk": {
 "name": "member_medals_medal_id_medals_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "medals",
 "columnsFrom": [
 "medal_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_awarded_by_users_id_fk": {
 "name": "member_medals_awarded_by_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
}
```



**drizzle/meta/0016\_snapshot.json**

```
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"member_war_records": {
 "name": "member_war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "war_record_id": {
 "name": "war_record_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 }
},
"indexes": {
```

**drizzle/meta/0016\_snapshot.json**

```
"member_war_records_user_idx": {
 "name": "member_war_records_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
},
"member_war_records_record_idx": {
 "name": "member_war_records_record_idx",
 "columns": [
 "war_record_id"
],
 "isUnique": false
}
},
"foreignKeys": {
 "member_war_records_user_id_users_id_fk": {
 "name": "member_war_records_user_id_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_war_record_id_war_records_id_fk": {
 "name": "member_war_records_war_record_id_war_records_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "war_records",
 "columnsFrom": [
 "war_record_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_awarded_by_users_id_fk": {
 "name": "member_war_records_awarded_by_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
}
```

**drizzle/meta/0016\_snapshot.json**

```
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
 },
 "news": {
 "name": "news",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "excerpt": {
 "name": "excerpt",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "body": {
 "name": "body",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "author_id": {
 "name": "author_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
```

**drizzle/meta/0016\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'draft'"
 },
 "pinned": {
 "name": "pinned",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "published_at": {
 "name": "published_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "news_slug_unique": {
 "name": "news_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 },
 "news_status_published_idx": {
 "name": "news_status_published_idx",
 "columns": [
 "status",
 "published_at"
],
 "isUnique": false
 }
},
```

**drizzle/meta/0016\_snapshot.json**

```
"foreignKeys": {
 "news_author_id_users_id_fk": {
 "name": "news_author_id_users_id_fk",
 "tableFrom": "news",
 "tableTo": "users",
 "columnsFrom": [
 "author_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"page_layouts": {
 "name": "page_layouts",
 "columns": {
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "layout": {
 "name": "layout",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "show_in_nav": {
 "name": "show_in_nav",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "nav_order": {
 "name": "nav_order",
 "type": "integer",
```

**drizzle/meta/0016\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {},
"foreignKeys": {
 "page_layouts_updated_by_users_id_fk": {
 "name": "page_layouts_updated_by_users_id_fk",
 "tableFrom": "page_layouts",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"profiles": {
 "name": "profiles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "bio": {
 "name": "bio",
 "type": "text",
```

**drizzle/meta/0016\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "timezone": {
 "name": "timezone",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "gamertags": {
 "name": "gamertags",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {},
"foreignKeys": {
 "profiles_user_id_users_id_fk": {
 "name": "profiles_user_id_users_id_fk",
 "tableFrom": "profiles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
```

**drizzle/meta/0016\_snapshot.json**

```
"rank_roles": {
 "name": "rank_roles",
 "columns": {
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "indexes": {
 "rank_roles_role_idx": {
 "name": "rank_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "rank_roles_rank_id_ranks_id_fk": {
 "name": "rank_roles_rank_id_ranks_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "rank_roles_role_id_roles_id_fk": {
 "name": "rank_roles_role_id_roles_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 }
}
```



**drizzle/meta/0016\_snapshot.json**

```
 },
 "compositePrimaryKeys": {
 "rank_roles_rank_id_role_id_pk": {
 "columns": [
 "rank_id",
 "role_id"
],
 "name": "rank_roles_rank_id_role_id_pk"
 }
 },
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"ranks": {
 "name": "ranks",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "abbreviation": {
 "name": "abbreviation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "req_days": {
 "name": "req_days",
 "type": "integer",
```

**drizzle/meta/0016\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "req_wins": {
 "name": "req_wins",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_default": {
 "name": "is_default",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "ranks_sort_order_idx": {
 "name": "ranks_sort_order_idx",
 "columns": [
 "sort_order"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"role_permissions": {
 "name": "role_permissions",
```

**drizzle/meta/0016\_snapshot.json**

```
"columns": {
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "permission": {
 "name": "permission",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
},
"indexes": {},
"foreignKeys": {
 "role_permissions_role_id_roles_id_fk": {
 "name": "role_permissions_role_id_roles_id_fk",
 "tableFrom": "role_permissions",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "role_permissions_role_id_permission_pk": {
 "columns": [
 "role_id",
 "permission"
],
 "name": "role_permissions_role_id_permission_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"roles": {
 "name": "roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 }
 },
}
```

**drizzle/meta/0016\_snapshot.json**

```
"name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"color": {
 "name": "color",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"discord_role_id": {
 "name": "discord_role_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
},
"is_system": {
 "name": "is_system",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
},
"updated_at": {
 "name": "updated_at",
```

**drizzle/meta/0016\_snapshot.json**

```
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "roles_name_unique": {
 "name": "roles_name_unique",
 "columns": [
 "name"
],
 "isUnique": true
 },
 "roles_discord_role_id_unique": {
 "name": "roles_discord_role_id_unique",
 "columns": [
 "discord_role_id"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"sc_hangars": {
 "name": "sc_hangars",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "items": {
 "name": "items",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "item_count": {
 "name": "item_count",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 }
 },
 "is_public": {
```

**drizzle/meta/0016\_snapshot.json**

```
 "name": "is_public",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "imported_at": {
 "name": "imported_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {},
"foreignKeys": {
 "sc_hangars_user_id_users_id_fk": {
 "name": "sc_hangars_user_id_users_id_fk",
 "tableFrom": "sc_hangars",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"sc_verifications": {
 "name": "sc_verifications",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "pending_code": {
 "name": "pending_code",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "pending_handle": {
```

**drizzle/meta/0016\_snapshot.json**

```
 "name": "pending_handle",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "pending_at": {
 "name": "pending_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "rsi_handle": {
 "name": "rsi_handle",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "verified_at": {
 "name": "verified_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "org_sid": {
 "name": "org_sid",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "org_rank": {
 "name": "org_rank",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "org_visible": {
 "name": "org_visible",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "in_org": {
 "name": "in_org",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
```

**drizzle/meta/0016\_snapshot.json**

```
 }
 },
 "indexes": {
 "sc_verifications_rsi_handle_unique": {
 "name": "sc_verifications_rsi_handle_unique",
 "columns": [
 "rsi_handle"
],
 "isUnique": true
 }
 },
 "foreignKeys": {
 "sc_verifications_user_id_users_id_fk": {
 "name": "sc_verifications_user_id_users_id_fk",
 "tableFrom": "sc_verifications",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"sessions": {
 "name": "sessions",
 "columns": {
 "id": {
 "name": "id",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "expires_at": {
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 }
},
```



**drizzle/meta/0016\_snapshot.json**

```
 "user_agent": {
 "name": "user_agent",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "sessions_user_idx": {
 "name": "sessions_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "sessions_expires_idx": {
 "name": "sessions_expires_idx",
 "columns": [
 "expires_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "sessions_user_id_users_id_fk": {
 "name": "sessions_user_id_users_id_fk",
 "tableFrom": "sessions",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
```

**drizzle/meta/0016\_snapshot.json**

```
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"settings": {
 "name": "settings",
 "columns": {
 "key": {
 "name": "key",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "value": {
 "name": "value",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {},
 "foreignKeys": {
 "settings_updated_by_users_id_fk": {
 "name": "settings_updated_by_users_id_fk",
 "tableFrom": "settings",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
```

**drizzle/meta/0016\_snapshot.json**

```
"uniqueConstraints": {},
"checkConstraints": {}
},
"user_roles": {
 "name": "user_roles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'manual'"
 },
 "granted_by": {
 "name": "granted_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "granted_at": {
 "name": "granted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "user_roles_role_idx": {
 "name": "user_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
```

**drizzle/meta/0016\_snapshot.json**

```
 "user_roles_user_id_users_id_fk": {
 "name": "user_roles_user_id_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_role_id_roles_id_fk": {
 "name": "user_roles_role_id_roles_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_granted_by_users_id_fk": {
 "name": "user_roles_granted_by_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "granted_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "user_roles_user_id_role_id_pk": {
 "columns": [
 "user_id",
 "role_id"
],
 "name": "user_roles_user_id_role_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"users": {
 "name": "users",
```

**drizzle/meta/0016\_snapshot.json**

```
"columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "discord_id": {
 "name": "discord_id",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "username": {
 "name": "username",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "global_name": {
 "name": "global_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "display_name": {
 "name": "display_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "avatar": {
 "name": "avatar",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "profile_image_url": {
 "name": "profile_image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "email": {
 "name": "email",
 "type": "text",
 "primaryKey": false,
```

**drizzle/meta/0016\_snapshot.json**

```
 "notNull": false,
 "autoincrement": false
 },
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "is_god": {
 "name": "is_god",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "guild_joined_at": {
 "name": "guild_joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'active'"
 },
 "recruiter_id": {
 "name": "recruiter_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "joined_at": {
 "name": "joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "last_seen_at": {
 "name": "last_seen_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
```

**drizzle/meta/0016\_snapshot.json**

```
 },
 "last_reconciled_at": {
 "name": "last_reconciled_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "promoted_at": {
 "name": "promoted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "users_discord_id_unique": {
 "name": "users_discord_id_unique",
 "columns": [
 "discord_id"
],
 "isUnique": true
 },
 "users_rank_idx": {
 "name": "users_rank_idx",
 "columns": [
 "rank_id"
],
 "isUnique": false
 },
 "users_status_idx": {
 "name": "users_status_idx",
 "columns": [
 "status"
],
 "isUnique": false
 }
 }
}
```

**drizzle/meta/0016\_snapshot.json**

```
 },
 "foreignKeys": {
 "users_rank_id_ranks_id_fk": {
 "name": "users_rank_id_ranks_id_fk",
 "tableFrom": "users",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"war_records": {
 "name": "war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
```



**drizzle/meta/0016\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "war_records_game_idx": {
 "name": "war_records_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "war_records_game_id_games_id_fk": {
 "name": "war_records_game_id_games_id_fk",
 "tableFrom": "war_records",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
}
},
"views": {},
"enums": {},
"_meta": {
 "schemas": {},
```

**drizzle/meta/0016\_snapshot.json**

```
 "tables": {},
 "columns": {}
},
"internal": {
 "indexes": {}
}
}
```

**drizzle/meta/0017\_snapshot.json**

```
{
 "version": "6",
 "dialect": "sqlite",
 "id": "7286a6d4-7862-49ab-9e0f-98a3ce231b86",
 "prevId": "cf8f5204-e028-4493-a798-ccfb607fb9e2",
 "tables": {
 "api_tokens": {
 "name": "api_tokens",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "token_hash": {
 "name": "token_hash",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "label": {
 "name": "label",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "prefix": {
 "name": "prefix",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "expires_at": {
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "last_used_at": {
 "name": "last_used_at",
 "type": "integer",
```

**drizzle/meta/0017\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "api_tokens_token_hash_unique": {
 "name": "api_tokens_token_hash_unique",
 "columns": [
 "token_hash"
],
 "isUnique": true
 },
 "api_tokens_user_idx": {
 "name": "api_tokens_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "api_tokens_user_id_users_id_fk": {
 "name": "api_tokens_user_id_users_id_fk",
 "tableFrom": "api_tokens",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"audit_log": {
 "name": "audit_log",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
```

**drizzle/meta/0017\_snapshot.json**

```
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "actor_id": {
 "name": "actor_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "action": {
 "name": "action",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "target_type": {
 "name": "target_type",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "target_id": {
 "name": "target_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "reason": {
 "name": "reason",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "meta": {
 "name": "meta",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'web'"
 },
}
```

**drizzle/meta/0017\_snapshot.json**

```
"ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
 "audit_actor_idx": {
 "name": "audit_actor_idx",
 "columns": [
 "actor_id"
],
 "isUnique": false
 },
 "audit_created_idx": {
 "name": "audit_created_idx",
 "columns": [
 "created_at"
],
 "isUnique": false
 },
 "audit_action_idx": {
 "name": "audit_action_idx",
 "columns": [
 "action"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "audit_log_actor_id_users_id_fk": {
 "name": "audit_log_actor_id_users_id_fk",
 "tableFrom": "audit_log",
 "tableTo": "users",
 "columnsFrom": [
 "actor_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
},
```

**drizzle/meta/0017\_snapshot.json**

```
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"event_roles": {
 "name": "event_roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "event_id": {
 "name": "event_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "emoji": {
 "name": "emoji",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "capacity": {
 "name": "capacity",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 }
 }
},
"indexes": {
 "event_roles_event_idx": {
 "name": "event_roles_event_idx",
```

**drizzle/meta/0017\_snapshot.json**

```
 "columns": [
 "event_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "event_roles_event_id_events_id_fk": {
 "name": "event_roles_event_id_events_id_fk",
 "tableFrom": "event_roles",
 "tableTo": "events",
 "columnsFrom": [
 "event_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"event_signups": {
 "name": "event_signups",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "event_id": {
 "name": "event_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "event_role_id": {
 "name": "event_role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
```



**drizzle/meta/0017\_snapshot.json**

```
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "event_signups_event_user_idx": {
 "name": "event_signups_event_user_idx",
 "columns": [
 "event_id",
 "user_id"
],
 "isUnique": true
 },
 "event_signups_event_idx": {
 "name": "event_signups_event_idx",
 "columns": [
 "event_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "event_signups_event_id_events_id_fk": {
 "name": "event_signups_event_id_events_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "events",
 "columnsFrom": [
 "event_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "event_signups_user_id_users_id_fk": {
 "name": "event_signups_user_id_users_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "users",
```

**drizzle/meta/0017\_snapshot.json**

```
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "event_signups_event_role_id_event_roles_id_fk": {
 "name": "event_signups_event_role_id_event_roles_id_fk",
 "tableFrom": "event_signups",
 "tableTo": "event_roles",
 "columnsFrom": [
 "event_role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"events": {
 "name": "events",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
```

**drizzle/meta/0017\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "starts_at": {
 "name": "starts_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "ends_at": {
 "name": "ends_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_by": {
 "name": "created_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_event_id": {
 "name": "discord_event_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "discord_message_id": {
 "name": "discord_message_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
```

**drizzle/meta/0017\_snapshot.json**

```
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'scheduled'"
 },
 "recurrence": {
 "name": "recurrence",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'none'"
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "events_starts_idx": {
 "name": "events_starts_idx",
 "columns": [
 "starts_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "events_game_id_games_id_fk": {
 "name": "events_game_id_games_id_fk",
 "tableFrom": "events",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
}
```

**drizzle/meta/0017\_snapshot.json**

```
 },
 "events_created_by_users_id_fk": {
 "name": "events_created_by_users_id_fk",
 "tableFrom": "events",
 "tableTo": "users",
 "columnsFrom": [
 "created_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"games": {
 "name": "games",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "icon_url": {
 "name": "icon_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
```

**drizzle/meta/0017\_snapshot.json**

```
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "active": {
 "name": "active",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": true
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "games_slug_unique": {
 "name": "games_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"match_participants": {
 "name": "match_participants",
 "columns": {
 "match_id": {
 "name": "match_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "stats": {
 "name": "stats",
```

**drizzle/meta/0017\_snapshot.json**

```
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
},
"indexes": {
 "mp_user_idx": {
 "name": "mp_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "match_participants_match_id_matches_id_fk": {
 "name": "match_participants_match_id_matches_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "matches",
 "columnsFrom": [
 "match_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "match_participants_user_id_users_id_fk": {
 "name": "match_participants_user_id_users_id_fk",
 "tableFrom": "match_participants",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "match_participants_match_id_user_id_pk": {
 "columns": [
 "match_id",
 "user_id"
],
 "name": "match_participants_match_id_user_id_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
```

**drizzle/meta/0017\_snapshot.json**

```
,
"matches": {
 "name": "matches",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent": {
 "name": "opponent",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "opponent_tag": {
 "name": "opponent_tag",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "result": {
 "name": "result",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "score_us": {
 "name": "score_us",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "score_them": {
 "name": "score_them",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 },
 "map": {
```



**drizzle/meta/0017\_snapshot.json**

```
 "name": "map",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "notes": {
 "name": "notes",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "played_at": {
 "name": "played_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "recorded_by": {
 "name": "recorded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "matches_game_played_idx": {
 "name": "matches_game_played_idx",
 "columns": [
 "game_id",
 "played_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "matches_game_id_games_id_fk": {
 "name": "matches_game_id_games_id_fk",
 "tableFrom": "matches",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
```

**drizzle/meta/0017\_snapshot.json**

```
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "matches_recorded_by_users_id_fk": {
 "name": "matches_recorded_by_users_id_fk",
 "tableFrom": "matches",
 "tableTo": "users",
 "columnsFrom": [
 "recorded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"medals": {
 "name": "medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 }
}
```

**drizzle/meta/0017\_snapshot.json**

```
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "auto_grant_months": {
 "name": "auto_grant_months",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {
 "medals_game_idx": {
 "name": "medals_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "medals_game_id_games_id_fk": {
 "name": "medals_game_id_games_id_fk",
 "tableFrom": "medals",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 }
}
```

**drizzle/meta/0017\_snapshot.json**

```
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
 },
 "member_medals": {
 "name": "member_medals",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "medal_id": {
 "name": "medal_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 }
 },
 "indexes": {
```

**drizzle/meta/0017\_snapshot.json**

```
"member_medals_user_idx": {
 "name": "member_medals_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
},
"member_medals_medal_idx": {
 "name": "member_medals_medal_idx",
 "columns": [
 "medal_id"
],
 "isUnique": false
}
},
"foreignKeys": {
 "member_medals_user_id_users_id_fk": {
 "name": "member_medals_user_id_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_medal_id_medals_id_fk": {
 "name": "member_medals_medal_id_medals_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "medals",
 "columnsFrom": [
 "medal_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_medals_awarded_by_users_id_fk": {
 "name": "member_medals_awarded_by_users_id_fk",
 "tableFrom": "member_medals",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
}
```

**drizzle/meta/0017\_snapshot.json**

```
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
 },
 "member_war_records": {
 "name": "member_war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "war_record_id": {
 "name": "war_record_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "citation": {
 "name": "citation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_by": {
 "name": "awarded_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "awarded_at": {
 "name": "awarded_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 }
 },
 "indexes": {
```

**drizzle/meta/0017\_snapshot.json**

```
"member_war_records_user_idx": {
 "name": "member_war_records_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
},
"member_war_records_record_idx": {
 "name": "member_war_records_record_idx",
 "columns": [
 "war_record_id"
],
 "isUnique": false
}
},
"foreignKeys": {
 "member_war_records_user_id_users_id_fk": {
 "name": "member_war_records_user_id_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_war_record_id_war_records_id_fk": {
 "name": "member_war_records_war_record_id_war_records_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "war_records",
 "columnsFrom": [
 "war_record_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "member_war_records_awarded_by_users_id_fk": {
 "name": "member_war_records_awarded_by_users_id_fk",
 "tableFrom": "member_war_records",
 "tableTo": "users",
 "columnsFrom": [
 "awarded_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
}
```

**drizzle/meta/0017\_snapshot.json**

```
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
 },
 "news": {
 "name": "news",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "excerpt": {
 "name": "excerpt",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "body": {
 "name": "body",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "author_id": {
 "name": "author_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
```



**drizzle/meta/0017\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'draft'"
 },
 "pinned": {
 "name": "pinned",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "published_at": {
 "name": "published_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "news_slug_unique": {
 "name": "news_slug_unique",
 "columns": [
 "slug"
],
 "isUnique": true
 },
 "news_status_published_idx": {
 "name": "news_status_published_idx",
 "columns": [
 "status",
 "published_at"
],
 "isUnique": false
 }
},
```

**drizzle/meta/0017\_snapshot.json**

```
"foreignKeys": {
 "news_author_id_users_id_fk": {
 "name": "news_author_id_users_id_fk",
 "tableFrom": "news",
 "tableTo": "users",
 "columnsFrom": [
 "author_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"page_layouts": {
 "name": "page_layouts",
 "columns": {
 "slug": {
 "name": "slug",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "title": {
 "name": "title",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "layout": {
 "name": "layout",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "show_in_nav": {
 "name": "show_in_nav",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "nav_order": {
 "name": "nav_order",
 "type": "integer",
```

**drizzle/meta/0017\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {},
"foreignKeys": {
 "page_layouts_updated_by_users_id_fk": {
 "name": "page_layouts_updated_by_users_id_fk",
 "tableFrom": "page_layouts",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"profiles": {
 "name": "profiles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "bio": {
 "name": "bio",
 "type": "text",
```

**drizzle/meta/0017\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "location": {
 "name": "location",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "timezone": {
 "name": "timezone",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "gamertags": {
 "name": "gamertags",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {},
"foreignKeys": {
 "profiles_user_id_users_id_fk": {
 "name": "profiles_user_id_users_id_fk",
 "tableFrom": "profiles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
```

**drizzle/meta/0017\_snapshot.json**

```
"rank_roles": {
 "name": "rank_roles",
 "columns": {
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
 "indexes": {
 "rank_roles_role_idx": {
 "name": "rank_roles_role_idx",
 "columns": [
 "role_id"
],
 "isUnique": false
 }
 },
 "foreignKeys": {
 "rank_roles_rank_id_ranks_id_fk": {
 "name": "rank_roles_rank_id_ranks_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "rank_roles_role_id_roles_id_fk": {
 "name": "rank_roles_role_id_roles_id_fk",
 "tableFrom": "rank_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 }
}
```

**drizzle/meta/0017\_snapshot.json**

```
 },
 "compositePrimaryKeys": {
 "rank_roles_rank_id_role_id_pk": {
 "columns": [
 "rank_id",
 "role_id"
],
 "name": "rank_roles_rank_id_role_id_pk"
 }
 },
 "uniqueConstraints": {},
 "checkConstraints": {}
 },
 "ranks": {
 "name": "ranks",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "abbreviation": {
 "name": "abbreviation",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "req_days": {
 "name": "req_days",
 "type": "integer",
```

**drizzle/meta/0017\_snapshot.json**

```
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "req_wins": {
 "name": "req_wins",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "is_default": {
 "name": "is_default",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "ranks_sort_order_idx": {
 "name": "ranks_sort_order_idx",
 "columns": [
 "sort_order"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"role_permissions": {
 "name": "role_permissions",
```

**drizzle/meta/0017\_snapshot.json**

```
"columns": {
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "permission": {
 "name": "permission",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
},
"indexes": {},
"foreignKeys": {
 "role_permissions_role_id_roles_id_fk": {
 "name": "role_permissions_role_id_roles_id_fk",
 "tableFrom": "role_permissions",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "role_permissions_role_id_permission_pk": {
 "columns": [
 "role_id",
 "permission"
],
 "name": "role_permissions_role_id_permission_pk"
 }
},
"uniqueConstraints": {},
"checkConstraints": {}
},
"roles": {
 "name": "roles",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 }
 },

```



**drizzle/meta/0017\_snapshot.json**

```
"name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"color": {
 "name": "color",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"discord_role_id": {
 "name": "discord_role_id",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
},
"is_system": {
 "name": "is_system",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
},
"updated_at": {
 "name": "updated_at",
```

**drizzle/meta/0017\_snapshot.json**

```
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "roles_name_unique": {
 "name": "roles_name_unique",
 "columns": [
 "name"
],
 "isUnique": true
 },
 "roles_discord_role_id_unique": {
 "name": "roles_discord_role_id_unique",
 "columns": [
 "discord_role_id"
],
 "isUnique": true
 }
},
"foreignKeys": {},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"sc_hangars": {
 "name": "sc_hangars",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "items": {
 "name": "items",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "item_count": {
 "name": "item_count",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 }
 },
 "is_public": {
```

**drizzle/meta/0017\_snapshot.json**

```
 "name": "is_public",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "imported_at": {
 "name": "imported_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {},
"foreignKeys": {
 "sc_hangars_user_id_users_id_fk": {
 "name": "sc_hangars_user_id_users_id_fk",
 "tableFrom": "sc_hangars",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"sc_verifications": {
 "name": "sc_verifications",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "pending_code": {
 "name": "pending_code",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "pending_handle": {
```

**drizzle/meta/0017\_snapshot.json**

```
 "name": "pending_handle",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "pending_at": {
 "name": "pending_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "last_check_at": {
 "name": "last_check_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "rsi_handle": {
 "name": "rsi_handle",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "verified_at": {
 "name": "verified_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "org_sid": {
 "name": "org_sid",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "org_rank": {
 "name": "org_rank",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "org_visible": {
 "name": "org_visible",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
```

**drizzle/meta/0017\_snapshot.json**

```
 },
 "in_org": {
 "name": "in_org",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 }
 },
 "indexes": {
 "sc_verifications_rsi_handle_unique": {
 "name": "sc_verifications_rsi_handle_unique",
 "columns": [
 "rsi_handle"
],
 "isUnique": true
 }
 },
 "foreignKeys": {
 "sc_verifications_user_id_users_id_fk": {
 "name": "sc_verifications_user_id_users_id_fk",
 "tableFrom": "sc_verifications",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
 },
 "compositePrimaryKeys": {},
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"sessions": {
 "name": "sessions",
 "columns": {
 "id": {
 "name": "id",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 }
 },
}
```

**drizzle/meta/0017\_snapshot.json**

```
"expires_at": {
 "name": "expires_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
},
"user_agent": {
 "name": "user_agent",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"ip": {
 "name": "ip",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
},
"created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
}
},
"indexes": {
 "sessions_user_idx": {
 "name": "sessions_user_idx",
 "columns": [
 "user_id"
],
 "isUnique": false
 },
 "sessions_expires_idx": {
 "name": "sessions_expires_idx",
 "columns": [
 "expires_at"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "sessions_user_id_users_id_fk": {
 "name": "sessions_user_id_users_id_fk",
 "tableFrom": "sessions",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
```

**drizzle/meta/0017\_snapshot.json**

```
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"settings": {
 "name": "settings",
 "columns": {
 "key": {
 "name": "key",
 "type": "text",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": false
 },
 "value": {
 "name": "value",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "updated_by": {
 "name": "updated_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 },
 "indexes": {},
 "foreignKeys": {
 "settings_updated_by_users_id_fk": {
 "name": "settings_updated_by_users_id_fk",
 "tableFrom": "settings",
 "tableTo": "users",
 "columnsFrom": [
 "updated_by"
],
 "columnsTo": [
```

**drizzle/meta/0017\_snapshot.json**

```
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"user_roles": {
 "name": "user_roles",
 "columns": {
 "user_id": {
 "name": "user_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "role_id": {
 "name": "role_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "source": {
 "name": "source",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'manual'"
 },
 "granted_by": {
 "name": "granted_by",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "granted_at": {
 "name": "granted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
 }
},
"indexes": {
 "user_roles_role_idx": {
 "name": "user_roles_role_idx",
```



**drizzle/meta/0017\_snapshot.json**

```
 "columns": [
 "role_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "user_roles_user_id_users_id_fk": {
 "name": "user_roles_user_id_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "user_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_role_id_roles_id_fk": {
 "name": "user_roles_role_id_roles_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "roles",
 "columnsFrom": [
 "role_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "cascade",
 "onUpdate": "no action"
 },
 "user_roles_granted_by_users_id_fk": {
 "name": "user_roles_granted_by_users_id_fk",
 "tableFrom": "user_roles",
 "tableTo": "users",
 "columnsFrom": [
 "granted_by"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {
 "user_roles_user_id_role_id_pk": {
 "columns": [
 "user_id",
 "role_id"
],
 "name": "user_roles_user_id_role_id_pk"
 }
}
```

**drizzle/meta/0017\_snapshot.json**

```
 }
 },
 "uniqueConstraints": {},
 "checkConstraints": {}
},
"users": {
 "name": "users",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "discord_id": {
 "name": "discord_id",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "username": {
 "name": "username",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "global_name": {
 "name": "global_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "display_name": {
 "name": "display_name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "avatar": {
 "name": "avatar",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "profile_image_url": {
 "name": "profile_image_url",
 "type": "text",
 "primaryKey": false,
```

**drizzle/meta/0017\_snapshot.json**

```
 "notNull": false,
 "autoincrement": false
 },
 "email": {
 "name": "email",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "rank_id": {
 "name": "rank_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "is_god": {
 "name": "is_god",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": false
 },
 "guild_joined_at": {
 "name": "guild_joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "status": {
 "name": "status",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'active'"
 },
 "recruiter_id": {
 "name": "recruiter_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "joined_at": {
 "name": "joined_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
```

**drizzle/meta/0017\_snapshot.json**

```
,
 "last_seen_at": {
 "name": "last_seen_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "last_reconciled_at": {
 "name": "last_reconciled_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "promoted_at": {
 "name": "promoted_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 },
 "updated_at": {
 "name": "updated_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "users_discord_id_unique": {
 "name": "users_discord_id_unique",
 "columns": [
 "discord_id"
],
 "isUnique": true
 },
 "users_rank_idx": {
 "name": "users_rank_idx",
 "columns": [
 "rank_id"
],
 "isUnique": false
 }
},
```

**drizzle/meta/0017\_snapshot.json**

```
 "users_status_idx": {
 "name": "users_status_idx",
 "columns": [
 "status"
],
 "isUnique": false
 },
},
"foreignKeys": {
 "users_rank_id_ranks_id_fk": {
 "name": "users_rank_id_ranks_id_fk",
 "tableFrom": "users",
 "tableTo": "ranks",
 "columnsFrom": [
 "rank_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
"checkConstraints": {}
},
"war_records": {
 "name": "war_records",
 "columns": {
 "id": {
 "name": "id",
 "type": "integer",
 "primaryKey": true,
 "notNull": true,
 "autoincrement": true
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "description": {
 "name": "description",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "image_url": {
 "name": "image_url",
 "type": "text",
```

**drizzle/meta/0017\_snapshot.json**

```
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "game_id": {
 "name": "game_id",
 "type": "integer",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "sort_order": {
 "name": "sort_order",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": 0
 },
 "created_at": {
 "name": "created_at",
 "type": "integer",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(unixepoch())"
 }
},
"indexes": {
 "war_records_game_idx": {
 "name": "war_records_game_idx",
 "columns": [
 "game_id"
],
 "isUnique": false
 }
},
"foreignKeys": {
 "war_records_game_id_games_id_fk": {
 "name": "war_records_game_id_games_id_fk",
 "tableFrom": "war_records",
 "tableTo": "games",
 "columnsFrom": [
 "game_id"
],
 "columnsTo": [
 "id"
],
 "onDelete": "set null",
 "onUpdate": "no action"
 }
},
"compositePrimaryKeys": {},
"uniqueConstraints": {},
```

**drizzle/meta/0017\_snapshot.json**

```
 "checkConstraints": {}
},
"views": {},
"enums": {},
"_meta": {
 "schemas": {},
 "tables": {},
 "columns": {}
},
"internal": {
 "indexes": {}
}
}
```

**package-lock.json**

```

{
 "name": "clantek",
 "version": "3.0.0",
 "lockfileVersion": 3,
 "requires": true,
 "packages": {
 "": {
 "name": "clantek",
 "version": "3.0.0",
 "dependencies": {
 "@tiptap/pm": "^3.29.2",
 "@tiptap/react": "^3.29.2",
 "@tiptap/starter-kit": "^3.29.2",
 "dompurify": "^3.4.13",
 "drizzle-orm": "^0.45.2",
 "hono": "^4.12.34",
 "lucide": "^1.28.0",
 "morphicons": "^1.4.0",
 "pdf-lib": "^1.17.1",
 "react": "^19.0.0",
 "react-dom": "^19.0.0",
 "react-router-dom": "^7.1.0"
 },
 "devDependencies": {
 "@cloudflare/workers-types": "^5.20260801.1",
 "@types/react": "^19.0.0",
 "@types/react-dom": "^19.0.0",
 "@vitejs/plugin-react": "^5.0.0",
 "concurrently": "^9.1.0",
 "drizzle-kit": "^0.31.10",
 "tsx": "^4.19.0",
 "typescript": "^5.7.0",
 "vite": "^7.0.0",
 "wrangler": "^4.118.0"
 }
 },
 "node_modules/@babel/code-frame": {
 "version": "7.29.7",
 "resolved": "https://registry.npmjs.org/@babel/code-frame/-/code-frame-7.29.7.tgz",
 "integrity": "sha512-A2aG0GooEzFtM1K10UW0ZuX03D11eWt8W351K03z3JFqMTGzj8zr14/Seu3YQbNtJ3sH36kPtqg5A==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@babel/helper-validator-identifier": "^7.29.7",
 "js-tokens": "^4.0.0",
 "picocolors": "^1.1.1"
 }
 },
 "node_modules/@babel/compat-data": {
 "version": "7.29.7",

```



**package-lock.json**

```
"resolved": "https://registry.npmjs.org/@babel/compat-data/-/compat-data-7.29.7.tgz",
"integrity":
"sha512-locTkQyKvWIEgBzVrn8693ebc97F2U8ZHjbXwDXJ5Fn2TCpNwTlKcaKLkdHop5c/ic0FE7qt7Q9JC5hnKNa6Gg==",
"dev": true,
"license": "MIT",
"engines": {
 "node": ">=6.9.0"
}
},
"node_modules/@babel/core": {
 "version": "7.29.7",
 "resolved": "https://registry.npmjs.org/@babel/core/-/core-7.29.7.tgz",
 "integrity":
"sha512-RgHBCvtjb0K2gXSNBNIkNoEc9qoVEtau3hj8gEqKQuL3HZAibKarWFEI3Lfm6EYKkLa10h8eSrj9b+ch9H/VBA==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@babel/code-frame": "^7.29.7",
 "@babel/generator": "^7.29.7",
 "@babel/helper-compilation-targets": "^7.29.7",
 "@babel/helper-module-transforms": "^7.29.7",
 "@babel/helpers": "^7.29.7",
 "@babel/parser": "^7.29.7",
 "@babel/template": "^7.29.7",
 "@babel/traverse": "^7.29.7",
 "@babel/types": "^7.29.7",
 "@jridgewell/remapping": "^2.3.5",
 "convert-source-map": "^2.0.0",
 "debug": "^4.1.0",
 "gensync": "^1.0.0-beta.2",
 "json5": "^2.2.3",
 "semver": "^6.3.1"
 },
 "engines": {
 "node": ">=6.9.0"
 },
 "funding": {
 "type": "opencollective",
 "url": "https://opencollective.com/babel"
 }
},
"node_modules/@babel/generator": {
 "version": "7.29.8",
 "resolved": "https://registry.npmjs.org/@babel/generator/-/generator-7.29.8.tgz",
 "integrity":
"sha512-gZbepsdh3WDtgZKWL+vTPH71LSBrm/Y4/QDZBVCCyfmeTEEUo0YwLSy+G1StfJg+/Zy550u/3TATbm7qDbbMtg==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@babel/parser": "^7.29.8",
 "@babel/types": "^7.29.8",
 "@jridgewell/gen-mapping": "^0.3.12",
 "@jridgewell/trace-mapping": "^0.3.28",
 "jsesc": "^3.0.2"
 }
}
```

**package-lock.json**

```
 },
 "engines": {
 "node": ">=6.9.0"
 }
 },
 "node_modules/@babel/helper-compilation-targets": {
 "version": "7.29.7",
 "resolved":
"https://registry.npmjs.org/@babel/helper-compilation-targets/-/helper-compilation-targets-7.29.7.tgz",
 "integrity":
"sha512-wem6WabJ4NaVYVdNhLPPVacES6ZJ+KBBfSkTMD3YZxbP3rm3Di85tJU5ljaUNhaOynt+Ajt0xruhYuzQBt8n71g==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@babel/compat-data": "^7.29.7",
 "@babel/helper-validator-option": "^7.29.7",
 "browserslist": "^4.24.0",
 "lru-cache": "^5.1.1",
 "semver": "^6.3.1"
 },
 "engines": {
 "node": ">=6.9.0"
 }
 },
 "node_modules/@babel/helper-globals": {
 "version": "7.29.7",
 "resolved":
"https://registry.npmjs.org/@babel/helper-globals/-/helper-globals-7.29.7.tgz",
 "integrity":
"sha512-3nQUAtvkkH9zahfWgw96Jc/uF0mjACE1kQz82E2lqwmHBgjzbNlsC22nuQTFahmWeQtTq5nQ/4Nnd2A1wj4zA==",
 "dev": true,
 "license": "MIT",
 "engines": {
 "node": ">=6.9.0"
 }
 },
 "node_modules/@babel/helper-module-imports": {
 "version": "7.29.7",
 "resolved":
"https://registry.npmjs.org/@babel/helper-module-imports/-/helper-module-imports-7.29.7.tgz",
 "integrity":
"sha512-ejHwrQQYcm9xnTivShn2ID0lIzInN34AXskvq9QicvCtEzq1Vzclu/tKF8Jq1Cg8JG2GL6/EmjgsCT7lXepE3g==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@babel/traverse": "^7.29.7",
 "@babel/types": "^7.29.7"
 },
 "engines": {
 "node": ">=6.9.0"
 }
 },
 "node_modules/@babel/helper-module-transforms": {
 "version": "7.29.7",
 "resolved":
```

**package-lock.json**

```
"https://registry.npmjs.org/@babel/helper-module-transforms/-/helper-module-transforms-7.29.7.tgz",
 "integrity":
"sha512-UPUVSyXb0h627KiCIGQSGwWzGeBKLkaJ9PJEdrngIwMSzxLR4jS4+f1f1jb7VzBbg8nFLaYotvVPFCTqdrmTAg==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@babel/helper-module-imports": "^7.29.7",
 "@babel/helper-validator-identifier": "^7.29.7",
 "@babel/traverse": "^7.29.7"
 },
 "engines": {
 "node": ">=6.9.0"
 },
 "peerDependencies": {
 "@babel/core": "^7.0.0"
 }
},
"node_modules/@babel/helper-plugin-utils": {
 "version": "7.29.7",
 "resolved":
"https://registry.npmjs.org/@babel/helper-plugin-utils/-/helper-plugin-utils-7.29.7.tgz",
 "integrity":
"sha512-G7sHYigPY17o05SYWnFD/0MTBwVR781S/JI643e/JhUYgVgWE/61SoW3NH9KWUKyKq5LVh3npif99Wkt6j86Jw==",
 "dev": true,
 "license": "MIT",
 "engines": {
 "node": ">=6.9.0"
 }
},
"node_modules/@babel/helper-string-parser": {
 "version": "7.29.7",
 "resolved":
"https://registry.npmjs.org/@babel/helper-string-parser/-/helper-string-parser-7.29.7.tgz",
 "integrity":
"sha512-Pb51jPrZ89GDH8223L4UP8i6QApWxs04RbPQJTeWDV0/keR2E36MeKnYr6LYmUUvqRRI+Iv87SuF1W6ErINzYw==",
 "dev": true,
 "license": "MIT",
 "engines": {
 "node": ">=6.9.0"
 }
},
"node_modules/@babel/helper-validator-identifier": {
 "version": "7.29.7",
 "resolved":
"https://registry.npmjs.org/@babel/helper-validator-identifier/-/helper-validator-identifier-7.29.7.tgz",
 "integrity":
"sha512-qehxGkRj55h/ff8EMaJ+cYhyaKlHIXqYDn682wQD7RNP9Uuj0QsHog2uS0r2vzr4pW+sXf90NeeayjcNaX3fFg==",
 "dev": true,
 "license": "MIT",
 "engines": {
 "node": ">=6.9.0"
 }
},
"node_modules/@babel/helper-validator-option": {
```

**package-lock.json**

```
 "version": "7.29.7",
 "resolved":
"https://registry.npmjs.org/@babel/helper-validator-option/-/helper-validator-option-7.29.7.tgz",
 "integrity":
"sha512-N9ZErrD+yW5geCDtBqnOoxmR8+tNKiGuxKlDpuJxfsqpa2dFcexaziGAE/qoHLiDDrevNMupxGmSoNlyvsA3gw==",
 "dev": true,
 "license": "MIT",
 "engines": {
 "node": ">=6.9.0"
 }
},
"node_modules/@babel/helpers": {
 "version": "7.29.7",
 "resolved": "https://registry.npmjs.org/@babel/helpers/-/helpers-7.29.7.tgz",
 "integrity":
"sha512-1k2LAGRMfHTcwuNYcCNUmaUffmQv8KWMfh2iJUUErLwlwH4FdNG7mfPI10NPfLHJFThE4Tyr4mv7kTNZ0iPuBg==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@babel/template": "^7.29.7",
 "@babel/types": "^7.29.7"
 },
 "engines": {
 "node": ">=6.9.0"
 }
},
"node_modules/@babel/parser": {
 "version": "7.29.8",
 "resolved": "https://registry.npmjs.org/@babel/parser/-/parser-7.29.8.tgz",
 "integrity":
"sha512-E8lTAYNB1KW+FH+VGJuZM1ioAx2E6oVlvQFRrf5P8ZZmsiJXYAD9VTFV7yyEURNzgh1dFqMZu06tUwcARbqFCA==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@babel/types": "^7.29.8"
 },
 "bin": {
 "parser": "bin/babel-parser.js"
 },
 "engines": {
 "node": ">=6.0.0"
 }
},
"node_modules/@babel/plugin-transform-react-jsx-self": {
 "version": "7.29.7",
 "resolved":
"https://registry.npmjs.org/@babel/plugin-transform-react-jsx-self/-/plugin-transform-react-jsx-self-7.29.7.tgz",
 "integrity":
"sha512-TL0HMc9xzy86VD31nUiwzd5otRACyEPcsegCxol00PvcXuH1v0kECe/UIznYFihpkvU5wg/jk4v0TTEFfm53fw==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@babel/helper-plugin-utils": "^7.29.7"
 },
 "engines": {
 "node": ">=6.9.0"
 }
}
```

**package-lock.json**

```
 "engines": {
 "node": ">=6.9.0"
 },
 "peerDependencies": {
 "@babel/core": "^7.0.0-0"
 }
},
"node_modules/@babel/plugin-transform-react-jsx-source": {
 "version": "7.29.7",
 "resolved": "https://registry.npmjs.org/@babel/plugin-transform-react-jsx-source/-/plugin-transform-react-jsx-source-7.29.7.tgz",
 "integrity": "sha512-6Xf0Qya65R9/2A7P2E0KJF3I0UR4FRqkGwSvU10pGzwZy736z0sWj/25YU1ZnHCX4AsfEDLjUVKVfjH7qA==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@babel/helper-plugin-utils": "^7.29.7"
 },
 "engines": {
 "node": ">=6.9.0"
 },
 "peerDependencies": {
 "@babel/core": "^7.0.0-0"
 }
},
"node_modules/@babel/template": {
 "version": "7.29.7",
 "resolved": "https://registry.npmjs.org/@babel/template/-/template-7.29.7.tgz",
 "integrity": "sha512-4MLR6sYk0E9YI7Lp2TqoEa207F4V506D7peWIZeUqrvKY0zUqY2p40xtcWZpVMAU93j6Zzd9WzIzFtlv5A==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@babel/code-frame": "^7.29.7",
 "@babel/parser": "^7.29.7",
 "@babel/types": "^7.29.7"
 },
 "engines": {
 "node": ">=6.9.0"
 }
},
"node_modules/@babel/traverse": {
 "version": "7.29.8",
 "resolved": "https://registry.npmjs.org/@babel/traverse/-/traverse-7.29.8.tgz",
 "integrity": "sha512-zWPhGwO34QBPwDZiWc9Us7Pz94lR7Y/8Gw11Q8S1P84aqO3bQV0JXW8Mih10iYHFnTDRnH2U11/PyUwMGw==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@babel/code-frame": "^7.29.7",
 "@babel/generator": "^7.29.8",
 "@babel/helper-globals": "^7.29.7",
 "@babel/parser": "^7.29.8",
 "@babel/template": "^7.29.7",
 "@babel/types": "^7.29.8"
 },
 "engines": {
 "node": ">=6.9.0"
 }
}
```

**package-lock.json**

```
 "@babel/types": "^7.29.8",
 "debug": "^4.3.1"
 },
 "engines": {
 "node": ">=6.9.0"
 }
},
"node_modules/@babel/types": {
 "version": "7.29.8",
 "resolved": "https://registry.npmjs.org/@babel/types/-/types-7.29.8.tgz",
 "integrity":
"sha512-Vj1jF3cPfXg70AfoI7QnVKLoILlm2JF9pnVHrX8qx7AHMiYWT+NDAA7jChlNgRS4WTLc/fD1lXLmPixluJ+3Gg==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@babel/helper-string-parser": "^7.29.7",
 "@babel/helper-validator-identifier": "^7.29.7"
 },
 "engines": {
 "node": ">=6.9.0"
 }
},
"node_modules/@cloudflare/kv-asset-handler": {
 "version": "0.5.0",
 "resolved":
"https://registry.npmjs.org/@cloudflare/kv-asset-handler/-/kv-asset-handler-0.5.0.tgz",
 "integrity":
"sha512-jxYQkj8dSIzc0cD6cMMNd0c1UVjqSqu8BZdor5s8cGjW2I8Bj0Dt/kWPVdY+u9zj3ms75Q5qaZgnxUad83+eAg==",
 "dev": true,
 "license": "MIT OR Apache-2.0",
 "engines": {
 "node": ">=22.0.0"
 }
},
"node_modules/@cloudflare/unenv-preset": {
 "version": "2.16.1",
 "resolved": "https://registry.npmjs.org/@cloudflare/unenv-preset/-/unenv-preset-2.16.1.tgz",
 "integrity":
"sha512-ECxObrMfyTl5bhQf/lZCXwo5G6xX9IAUo+nDMKK4SZ8m4Jvvxp52vilxyySSWh2YTz8+HQ07qGH/2rEom1vDw==",
 "dev": true,
 "license": "MIT OR Apache-2.0",
 "peerDependencies": {
 "unenv": "2.0.0-rc.24",
 "workerd": ">1.20260305.0 <2.0.0-0"
 },
 "peerDependenciesMeta": {
 "workerd": {
 "optional": true
 }
 }
},
"node_modules/@cloudflare/workerd-darwin-64": {
 "version": "1.20260730.1",
 "resolved":
```

**package-lock.json**

```
"https://registry.npmjs.org/@cloudflare/workerd-darwin-64/-/workerd-darwin-64-1.20260730.1.tgz",
 "integrity":
 "sha512-+MBHmPaiTe2KajryW0T24rZvWfxb41hD3d8anNzQqHzft6vSEb18+sp0znSwxgij7ApPhSM1+vhkNg4f3YMguA==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "Apache-2.0",
 "optional": true,
 "os": [
 "darwin"
],
 "engines": {
 "node": ">=16"
 }
},
"node_modules/@cloudflare/workerd-darwin-arm64": {
 "version": "1.20260730.1",
 "resolved":
 "https://registry.npmjs.org/@cloudflare/workerd-darwin-arm64/-/workerd-darwin-arm64-1.20260730.1.tgz",
 "integrity":
 "sha512-SBHKntPkKvNPgaCrTe99xC1CA18ygJDzlyfK0LbuJ1muKadIw35Wnh00wu894fKBtllsVQdNzDLee+cm0ppLSQ==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "Apache-2.0",
 "optional": true,
 "os": [
 "darwin"
],
 "engines": {
 "node": ">=16"
 }
},
"node_modules/@cloudflare/workerd-linux-64": {
 "version": "1.20260730.1",
 "resolved":
 "https://registry.npmjs.org/@cloudflare/workerd-linux-64/-/workerd-linux-64-1.20260730.1.tgz",
 "integrity":
 "sha512-ouyPOSMbiKPeSwUJUvxtMcxGAXs2J4aPE4T5ABIYX5ClcQx5j5bbHTmnq0QEY8sAuLTPjH7dY+iB6UI5ISlwwA==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "Apache-2.0",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=16"
 }
},
```

**package-lock.json**

```
"node_modules/@cloudflare/workerd-linux-arm64": {
 "version": "1.20260730.1",
 "resolved":
"https://registry.npmjs.org/@cloudflare/workerd-linux-arm64/-/workerd-linux-arm64-1.20260730.1.tgz",
 "integrity":
"sha512-YQ+Mi78U3TPdgBPtwq+Sm6rJU+Ihl2y0pjYtuuKkdmUbYzL7oLR6Xqq9wljhasnuCFICssDJaqhMep5WizYoEQ==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "Apache-2.0",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=16"
 }
},
"node_modules/@cloudflare/workerd-windows-64": {
 "version": "1.20260730.1",
 "resolved":
"https://registry.npmjs.org/@cloudflare/workerd-windows-64/-/workerd-windows-64-1.20260730.1.tgz",
 "integrity":
"sha512-27fAN+vUECW1oYVc1K0cHYpkL8COM2Uxtxql7TL595kxbjoqS5yckw7NLz7bTf2pALFCZwjqXDjZGJ/xbG4ZKQ==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "Apache-2.0",
 "optional": true,
 "os": [
 "win32"
],
 "engines": {
 "node": ">=16"
 }
},
"node_modules/@cloudflare/workers-types": {
 "version": "5.20260801.1",
 "resolved":
"https://registry.npmjs.org/@cloudflare/workers-types/-/workers-types-5.20260801.1.tgz",
 "integrity":
"sha512-XCv5xWi47WQ0K0LpLa6997Mrpz8Ct+nZmp/M5Xp8Z4BFsarf7nYjkznG0c0oYK5m1GfbMFEEuQ20IZnbIw9A==",
 "devOptional": true,
 "license": "MIT OR Apache-2.0"
},
"node_modules/@cspotcode/source-map-support": {
 "version": "0.8.1",
 "resolved":
"https://registry.npmjs.org/@cspotcode/source-map-support/-/source-map-support-0.8.1.tgz",
 "integrity":
"sha512-IchNF6dN4tHoMFIIn/70E8LWZ19Y6q/67Bmf6vnGREv8RSbBVb9LPJxEcnwrcwX6ixSvaiGoomAUvu4YSxXrVgw==",
 "dev": true,
```



**package-lock.json**

```
"license": "MIT",
"dependencies": {
 "@jridgewell/trace-mapping": "0.3.9"
},
"engines": {
 "node": ">=12"
}
},
"node_modules/@cspotcode/source-map-support/node_modules/@jridgewell/trace-mapping": {
 "version": "0.3.9",
 "resolved":
"https://registry.npmjs.org/@jridgewell/trace-mapping/-/trace-mapping-0.3.9.tgz",
 "integrity":
"sha512-3Belt6tdc8bPgAtbcmtdtNJlirVoTmEb5e2gC94PnkW9jI6CAHUeoG85tjWP5WquqfavoMtMwiG4P926ZKKuQ==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@jridgewell/resolve-uri": "^3.0.3",
 "@jridgewell/sourcemap-codec": "^1.4.10"
 }
},
"node_modules/@drizzle-team/brocli": {
 "version": "0.10.2",
 "resolved": "https://registry.npmjs.org/@drizzle-team/brocli/-/brocli-0.10.2.tgz",
 "integrity":
"sha512-z33Il7l5dKjUgGULTqBsQBQwckHh5AbIuxhdsIxDDiZAzB0rZ06q9ogcWC65kU382AfynTfgNumVcNIjuIua6w==",
 "dev": true,
 "license": "Apache-2.0"
},
"node_modules/@emnapi/runtime": {
 "version": "1.11.3",
 "resolved": "https://registry.npmjs.org/@emnapi/runtime/-/runtime-1.11.3.tgz",
 "integrity":
"sha512-Xz4Tpyki7XyrpbUK1jR1AhdAdaXyhhY4lZ3neLodmhpUwfy2PAQN5B46sAiU4liOXGLkHypn/qU+jvFWSCYYLA==",
 "dev": true,
 "license": "MIT",
 "optional": true,
 "dependencies": {
 "tslib": "^2.4.0"
 }
},
"node_modules/@esbuild-kit/core-utils": {
 "version": "3.3.2",
 "resolved": "https://registry.npmjs.org/@esbuild-kit/core-utils/-/core-utils-3.3.2.tgz",
 "integrity":
"sha512-sPRAnw9CdSsRmEtnsl2WXWdyquogVpB3yZ3dgwJfe8zr0zTsV7cJvwmrKVva+0ma5BoiGJ+BoqkMvawbayKUSqQ==",
 "deprecated": "Merged into tsx: https://tsx.hirok.io",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "esbuild": "~0.18.20",
 "source-map-support": "^0.5.21"
 }
},
```

**package-lock.json**

```

"node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/android-arm": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/@esbuild/android-arm/-/android-arm-0.18.20.tgz",
 "integrity":
"sha512-fyi7TDI/ijKKNZTUJAQqiG5T7YjJXgnzkURqmGj13C6dCqckZBLdL4h7bkhHt/t0WP+z09/zwroDvANaOq05Sw==",
 "cpu": [
 "arm"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "android"
],
 "engines": {
 "node": ">=12"
 }
},
"node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/android-arm64": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/@esbuild/android-arm64/-/android-arm64-0.18.20.tgz",
 "integrity":
"sha512-Nz4rJcchGDtENV0eMKUNa6L12zz2zBDXuhj/Vjh18zGqB44Bi7MBMSXjgunJgjRhCmK0jnPuZp4Mb60KqtMHLQ==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "android"
],
 "engines": {
 "node": ">=12"
 }
},
"node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/android-x64": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/@esbuild/android-x64/-/android-x64-0.18.20.tgz",
 "integrity":
"sha512-8GDdlPJA8D6zLZYJV/jnrRAi6rOiNaCC/JclCXPB+KIuvfBN4owLtgzY2bsxnx666XjJx2kDPumnTtR8qKQUg==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "android"
],
 "engines": {
 "node": ">=12"
 }
},

```

**package-lock.json**

```
"node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/darwin-arm64": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/@esbuild/darwin-arm64/-/darwin-arm64-0.18.20.tgz",
 "integrity":
"sha512-bxRHW5kHU38zS2lPTP0yuyTm+S+eobPUntNkdJEfAddYgEc1l4xkT8DB9d2008DtTb17uJag2HuE5NZAZgnNEA==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "darwin"
],
 "engines": {
 "node": ">=12"
 }
},
"node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/darwin-x64": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/@esbuild/darwin-x64/-/darwin-x64-0.18.20.tgz",
 "integrity":
"sha512-pc5gx1MDxzm513qPGbCbDukOdsGtKhfxD1zJKXjCCcU7ju5007MeAZ8c4krSJc0IJGFR+qx21yMMVYwiQvyTyQ==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "darwin"
],
 "engines": {
 "node": ">=12"
 }
},
"node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/freebsd-arm64": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/@esbuild/freebsd-arm64/-/freebsd-arm64-0.18.20.tgz",
 "integrity":
"sha512-yqDQHy4QHevpMAaxhhIwYPMv1NECw0vIpGCZkECn8w2WFHXjEwrBn3CeNIYsibZ/iZEUemj++M26W3cNR5h+Tw==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "freebsd"
],
 "engines": {
 "node": ">=12"
 }
},
```

**package-lock.json**

```
"node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/freebsd-x64": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/@esbuild/freebsd-x64/-/freebsd-x64-0.18.20.tgz",
 "integrity":
"sha512-tgWRPPuQsd3RmBZwarGVHZQvtzfEB0reNuxEMKFcd5DaDn2PbBxfwLcj4+aenoh7ctXcbXm0QIn8HI6mCSw5MQ==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "freebsd"
],
 "engines": {
 "node": ">=12"
 }
},
"node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/linux-arm": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-arm/-/linux-arm-0.18.20.tgz",
 "integrity":
"sha512-/5bHkMwnq1EgKr1V+Ybz3s1hWXok7mDFUMQ4cG10AfW3wL02PSZi5kFpYKrptDsgb2WAJIVRcDm+qIvXf/apvg==",
 "cpu": [
 "arm"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=12"
 }
},
"node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/linux-arm64": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-arm64/-/linux-arm64-0.18.20.tgz",
 "integrity":
"sha512-2YbscF+UL7SQAIVpnWvYwM+3LskyDmPhe31pE7/aoTMFKKzIc9lLbyGUpmmb8a8Aix0L61sQ/mFh3jEjHYFvdA==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=12"
 }
},
}
```

**package-lock.json**

```
"node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/linux-ia32": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-ia32/-/linux-ia32-0.18.20.tgz",
 "integrity":
"sha512-P4etWwq6IsReT0E1KHU40bOnzMHoH73aXp96Fs8TIT6z9Hu8G6+0SHSw9i2isWrD2nbx2qo5yUqACgdfVGx7TA==",
 "cpu": [
 "ia32"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=12"
 }
},
"node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/linux-loong64": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-loong64/-/linux-loong64-0.18.20.tgz",
 "integrity":
"sha512-nXW8nqBTrOpDLPgPY9uV+/1DjxoQ7DoB2N8eocyq8I9XuqJ7BiAMDMf9n1xZM9TgW0J8zrquIb/A7s3BJv7rjg==",
 "cpu": [
 "loong64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=12"
 }
},
"node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/linux-mips64el": {
 "version": "0.18.20",
 "resolved":
"https://registry.npmjs.org/@esbuild/linux-mips64el/-/linux-mips64el-0.18.20.tgz",
 "integrity":
"sha512-d5NeaXZcHp8PzYy5VnXV3VSd2D328Zb+9dEq5HE6bw6+N86JVPExA60680PwobntbNJ0pzCpUFZTo3w0GyetQ==",
 "cpu": [
 "mips64el"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=12"
 }
}
```

**package-lock.json**

```
 },
 "node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/linux-ppc64": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-ppc64/-/linux-ppc64-0.18.20.tgz",
 "integrity":
 "sha512-WHPyeScRNcmANNLQkq6AfyXRFr5D6N2sKgkFo2FqguP44Nw2eyDlbTdZwd9GYk98DZG9QItIiTLFLHJHjxP3FA==",
 "cpu": [
 "ppc64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=12"
 }
 },
 "node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/linux-riscv64": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-riscv64/-/linux-riscv64-0.18.20.tgz",
 "integrity":
 "sha512-WSxo6h5ecI5XH34KC7w5veNnKkju3zBRLEQNY7mv5mtBmrP/MjNBCAIsM2u5hDBLS3NGcTQpoBvRzqBcRtpq1A==",
 "cpu": [
 "riscv64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=12"
 }
 },
 "node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/linux-s390x": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-s390x/-/linux-s390x-0.18.20.tgz",
 "integrity":
 "sha512-+8231GMS3mAEth6Ja1iK0a1sQ3ohfcpzpRLH8uuc5/KVDFneH6jtAJLFGafpzMRO6DzJ6AvXKze9LFFMrIHVQ==",
 "cpu": [
 "s390x"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=12"
 }
 }
```

**package-lock.json**

```
 },
 "node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/linux-x64": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-x64/-/linux-x64-0.18.20.tgz",
 "integrity":
 "sha512-UYqiqemphJcNsFEskc73jQ7B9jgwjWrSayxawS6UVFZGWrAAtkzjxSqnoclCXxWtfwLdzU+vTpcNYhpn43uP1w==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=12"
 }
 },
 "node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/netbsd-x64": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/@esbuild/netbsd-x64/-/netbsd-x64-0.18.20.tgz",
 "integrity":
 "sha512-i01c++VP6xUBUmltHZoMtCUDPlnPGdBom6Ir04gyKPFFVBKioIImVooR5I83nTew5U0Yrk3gIJhbZh8X44y06A==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "netbsd"
],
 "engines": {
 "node": ">=12"
 }
 },
 "node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/openbsd-x64": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/@esbuild/openbsd-x64/-/openbsd-x64-0.18.20.tgz",
 "integrity":
 "sha512-e5e4YSsuQfX4cxcygw/UCPIEP6wbIL+se3sxpDciMbFLBWu0eiZ0J7WoD+ptCLrmjZBK1Wk7I6D/I3NglUGOxg==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "openbsd"
],
 "engines": {
 "node": ">=12"
 }
 }
```

**package-lock.json**

```
 },
 "node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/sunos-x64": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/@esbuild/sunos-x64/-/sunos-x64-0.18.20.tgz",
 "integrity":
 "sha512-kDbFRFP0YpTQVVrqUd5FTYmWo45zGaXe0X8E1G/LKFC0v8x0vWrh0WSLITcCn63lmZIXfOMXtCfti/RxN/0wnQ==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "sunos"
],
 "engines": {
 "node": ">=12"
 }
 },
 "node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/win32-arm64": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/@esbuild/win32-arm64/-/win32-arm64-0.18.20.tgz",
 "integrity":
 "sha512-ddYFR6ItYgoaq4v4JmQQaAI5s7npztfV4Ag6Nrhiaw0Rrn0XqBkgwZLofVTlq1daVTQNhtI5oieTvkrPZrePg==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "win32"
],
 "engines": {
 "node": ">=12"
 }
 },
 "node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/win32-ia32": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/@esbuild/win32-ia32/-/win32-ia32-0.18.20.tgz",
 "integrity":
 "sha512-Wv7QB3ID/rROT08SABTS7eV4hX26sVduqD0Te1MvGMjNd3Ej0z4b7zeexIR62GTIEKrfJXKL9LFxTYgkyeu7g==",
 "cpu": [
 "ia32"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "win32"
],
 "engines": {
 "node": ">=12"
 }
 }
```



**package-lock.json**

```
 },
 "node_modules/@esbuild-kit/core-utils/node_modules/@esbuild/win32-x64": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/@esbuild/win32-x64/-/win32-x64-0.18.20.tgz",
 "integrity":
 "sha512-kTdfRcSiDfQca/y9QIkng02avJ+NCAQvrMejlsB3RRv5sE9rRoeBPISaZpKxHELzRxZyLvNts1P27W3wV+8geQ==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "win32"
],
 "engines": {
 "node": ">=12"
 }
 },
 "node_modules/@esbuild-kit/core-utils/node_modules/esbuild": {
 "version": "0.18.20",
 "resolved": "https://registry.npmjs.org/esbuild/-/esbuild-0.18.20.tgz",
 "integrity":
 "sha512-ceqxoedUrcayh7Y7ZX6NdbbDzGR0iyVBgC4PriJThBKSVWnnFHZAkfi1lJT8QFkOwH4qOS2SJkS4wvpGl8BpA==",
 "dev": true,
 "hasInstallScript": true,
 "license": "MIT",
 "bin": {
 "esbuild": "bin/esbuild"
 },
 "engines": {
 "node": ">=12"
 }
 },
 "optionalDependencies": {
 "@esbuild/android-arm": "0.18.20",
 "@esbuild/android-arm64": "0.18.20",
 "@esbuild/android-x64": "0.18.20",
 "@esbuild/darwin-arm64": "0.18.20",
 "@esbuild/darwin-x64": "0.18.20",
 "@esbuild/freebsd-arm64": "0.18.20",
 "@esbuild/freebsd-x64": "0.18.20",
 "@esbuild/linux-arm": "0.18.20",
 "@esbuild/linux-arm64": "0.18.20",
 "@esbuild/linux-ia32": "0.18.20",
 "@esbuild/linux-loong64": "0.18.20",
 "@esbuild/linux-mips64el": "0.18.20",
 "@esbuild/linux-ppc64": "0.18.20",
 "@esbuild/linux-riscv64": "0.18.20",
 "@esbuild/linux-s390x": "0.18.20",
 "@esbuild/linux-x64": "0.18.20",
 "@esbuild/netbsd-x64": "0.18.20",
 "@esbuild/openbsd-x64": "0.18.20",
 "@esbuild/sunos-x64": "0.18.20",
 "@esbuild/win32-arm64": "0.18.20",
 }
```

**package-lock.json**

```

 "@esbuild/win32-ia32": "0.18.20",
 "@esbuild/win32-x64": "0.18.20"
 },
 },
 "node_modules/@esbuild-kit/esm-loader": {
 "version": "2.6.5",
 "resolved": "https://registry.npmjs.org/@esbuild-kit/esm-loader/-/esm-loader-2.6.5.tgz",
 "integrity": "sha512-FxEMiJKnodyA10aCUoEvbYRkoZLLZ4d/eXFu9Fh8CbBBgP5EmZxrfTRyN0qpXZ4v0vqnE5YdRdcrmUUXuU+dA==",
 "deprecated": "Merged into tsx: https://tsx.hirok.io",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@esbuild-kit/core-utils": "^3.3.2",
 "get-tsconfig": "^4.7.0"
 }
 },
 },
 "node_modules/@esbuild/aix-ppc64": {
 "version": "0.25.12",
 "resolved": "https://registry.npmjs.org/@esbuild/aix-ppc64/-/aix-ppc64-0.25.12.tgz",
 "integrity": "sha512-Hhmwd6CInZ3dwpUGTF8fJG6yoWmsToE+vYgD4nytZVxcu1ulHpUQRAB1UJ8+N1Am3Mz4+x0ByoQoSzf4D+CpkA==",
 "cpu": [
 "ppc64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "aix"
],
 "engines": {
 "node": ">=18"
 }
 },
 },
 "node_modules/@esbuild/android-arm": {
 "version": "0.25.12",
 "resolved": "https://registry.npmjs.org/@esbuild/android-arm/-/android-arm-0.25.12.tgz",
 "integrity": "sha512-VJ+sKvNA/GE7Ccacc9Cha7bpS8nyzVv0jdVgwNDaR4gDMC/2TTRc33Ip8qrNYUcpk0HUT50Z0bUcNNVZQ9RLlg==",
 "cpu": [
 "arm"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "android"
],
 "engines": {
 "node": ">=18"
 }
 },
 },
 "node_modules/@esbuild/android-arm64": {

```

**package-lock.json**

```
"version": "0.25.12",
"resolved": "https://registry.npmjs.org/@esbuild/android-arm64/-/android-arm64-0.25.12.tgz",
"integrity":
"sha512-6AAmLG7zwd1Z159jCKPvAxZd4y/VT00VkprYy+3N2FtJ8+BQWFXU+OxARIwA46c5tdD9SsKGZ/1ocqBS/gAKHg==",
"cpu": [
 "arm64"
],
"dev": true,
"license": "MIT",
"optional": true,
"os": [
 "android"
],
"engines": {
 "node": ">=18"
}
},
"node_modules/@esbuild/android-x64": {
 "version": "0.25.12",
 "resolved": "https://registry.npmjs.org/@esbuild/android-x64/-/android-x64-0.25.12.tgz",
 "integrity":
"sha512-5jbb+2hhDHx5phYR2By8GTWEzn6I9UqR11Kwf22iKbNpYrsmRB18aX/9ivc5cabCuiAT/wM+YIZ6SG9Q06a8kg==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "android"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/@esbuild/darwin-arm64": {
 "version": "0.25.12",
 "resolved": "https://registry.npmjs.org/@esbuild/darwin-arm64/-/darwin-arm64-0.25.12.tgz",
 "integrity":
"sha512-N3zl+lxHCifgIlcMUP5016ESkeQjLj/959RxxNYIthIg+CQHInujFuXewbWMgnTo4cp5XVHqFPmpyu9J65C1Yg==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "darwin"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/@esbuild/darwin-x64": {
```

**package-lock.json**

```
"version": "0.25.12",
"resolved": "https://registry.npmjs.org/@esbuild/darwin-x64/-/darwin-x64-0.25.12.tgz",
"integrity":
"sha512-HQ9Ka4Kx21qHXwtlTUVbKJOAnmG1ipXhdWTmNXiPzPfWKpXqASVcWdnf2bnL73wgjNrFXAa3yYvBSd9pzfEIpA==",
"cpu": [
 "x64"
],
"dev": true,
"license": "MIT",
"optional": true,
"os": [
 "darwin"
],
"engines": {
 "node": ">=18"
}
},
"node_modules/@esbuild/freebsd-arm64": {
 "version": "0.25.12",
 "resolved": "https://registry.npmjs.org/@esbuild/freebsd-arm64/-/freebsd-arm64-0.25.12.tgz",
 "integrity":
"sha512-gA0Bx759+7Jve03K1S0vk0u5Lg/85dou3Ese0GUes8fLV0GxbhDDh/iZaoek11Y8mtYKPGF3vP8XhmkDEAmzeg==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "freebsd"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/@esbuild/freebsd-x64": {
 "version": "0.25.12",
 "resolved": "https://registry.npmjs.org/@esbuild/freebsd-x64/-/freebsd-x64-0.25.12.tgz",
 "integrity":
"sha512-TGb026Yw2xsHzxtbVFGEXBFH0FRAP7gtcPE7P5yP7wGy7cXK2o07Ry0hL5NLiqTlBh47XhmIUXuGciXEqYfBQ==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "freebsd"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/@esbuild/linux-arm": {
```

**package-lock.json**

```
"version": "0.25.12",
"resolved": "https://registry.npmjs.org/@esbuild/linux-arm/-/linux-arm-0.25.12.tgz",
"integrity":
"sha512-lPDGyC1JPDou8kGcywY0YILzWlhnnRjdof3UlcqYmS9El818LLfJJc3PXXgZHRHCAKs/Z2SeZtDJr5Mrkxt0w==",
"cpu": [
 "arm"
],
"dev": true,
"license": "MIT",
"optional": true,
"os": [
 "linux"
],
"engines": {
 "node": ">=18"
}
},
"node_modules/@esbuild/linux-arm64": {
 "version": "0.25.12",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-arm64/-/linux-arm64-0.25.12.tgz",
 "integrity":
"sha512-8bwX7a8FgHIgrupcxb4aUmYDLp8pX06rGh5HqDT7bB+8Rdells6mHvrFHHW2JAOPZUbnjUpKTLg6ECyzvas2AQ==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/@esbuild/linux-ia32": {
 "version": "0.25.12",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-ia32/-/linux-ia32-0.25.12.tgz",
 "integrity":
"sha512-0y9KrdVnbMM2/vG8KfU0byhUN+EFCny9+8g202gYqSSVMonbsCfLjU0+rCci7pM0WBETz+oK/PIwHkzxkyharA==",
 "cpu": [
 "ia32"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/@esbuild/linux-loong64": {
```

**package-lock.json**

```
"version": "0.25.12",
"resolved": "https://registry.npmjs.org/@esbuild/linux-loong64/-/linux-loong64-0.25.12.tgz",
"integrity":
"sha512-h///Lr5a9rib/v1GGqXVGzjL4TMvVTv+s1DPoxQdz7l/AYv6LDSxdIwzxkrPW438oUXiDtwM10o9Pmws/6Z0Ng==",
"cpu": [
 "loong64"
],
"dev": true,
"license": "MIT",
"optional": true,
"os": [
 "linux"
],
"engines": {
 "node": ">=18"
}
},
"node_modules/@esbuild/linux-mips64el": {
 "version": "0.25.12",
 "resolved":
"https://registry.npmjs.org/@esbuild/linux-mips64el/-/linux-mips64el-0.25.12.tgz",
 "integrity":
"sha512-iyRrM1Pzy9GFMdLsXn1iHUUm18nhKnNMwscjmp4+hpafcZjrr2WbT//d20xaGljXDBYHqRcl8HnxbX6uaA/eGVw==",
 "cpu": [
 "mips64el"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/@esbuild/linux-ppc64": {
 "version": "0.25.12",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-ppc64/-/linux-ppc64-0.25.12.tgz",
 "integrity":
"sha512-9meM/lRXxMi5PSUqEXRctVjEZBGwB7P/D4yT8UG/mwIdze2aV4Vo6U5gD3+RsoHXKkHCfSxZKzmDssVlRj1QQA==",
 "cpu": [
 "ppc64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
},
```

**package-lock.json**

```
"node_modules/@esbuild/linux-riscv64": {
 "version": "0.25.12",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-riscv64/-/linux-riscv64-0.25.12.tgz",
 "integrity":
"sha512-Zr7KR4hgKUpWAwb1f3o5ygT04MzqVrGEGXGlnj15YQDJErYu/BGg+wmFlID0dJp0PmB0lLvxFIOXZgFRrdjR0w==",
 "cpu": [
 "riscv64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/@esbuild/linux-s390x": {
 "version": "0.25.12",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-s390x/-/linux-s390x-0.25.12.tgz",
 "integrity":
"sha512-MsKnc0cgTNvdtiISc/jZs/Zf8d0cl/t3gYWx8J9ubBnV0wlk65UIEEvgB0RTiljloIWnBzLs4qhzPkJcitIzIg==",
 "cpu": [
 "s390x"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/@esbuild/linux-x64": {
 "version": "0.25.12",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-x64/-/linux-x64-0.25.12.tgz",
 "integrity":
"sha512-uqZMTLr/zR/ed4jIGnwSLkaHmPj0jJvnm6TVVitAa08SLS9Z0VM8wIRx7gWbJB5/J54YuIMInDquWyYvQLZkgw==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
```

**package-lock.json**

```
"node_modules/@esbuild/netbsd-arm64": {
 "version": "0.25.12",
 "resolved": "https://registry.npmjs.org/@esbuild/netbsd-arm64/-/netbsd-arm64-0.25.12.tgz",
 "integrity":
"sha512-xXwcTq4GhRM7J9A8Gv5boanHhRa/Q9KLVmcyXHCTaM4wKfIpWkdXiMog/KsnxzJ0A1+nD+zoecuzqPmCRyBGjg==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "netbsd"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/@esbuild/netbsd-x64": {
 "version": "0.25.12",
 "resolved": "https://registry.npmjs.org/@esbuild/netbsd-x64/-/netbsd-x64-0.25.12.tgz",
 "integrity":
"sha512-Ld5pTlZPy3YwGec40uHh1aCVCRv0Xdh8DgRjfdy/oumVovmuSzWfnSJg+VtakB9Cm0gxN09BzWkj6mt01FMXkQ==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "netbsd"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/@esbuild/openbsd-arm64": {
 "version": "0.25.12",
 "resolved": "https://registry.npmjs.org/@esbuild/openbsd-arm64/-/openbsd-arm64-0.25.12.tgz",
 "integrity":
"sha512-ff96T6KsBo/pkQI950FARU9apGNTSLZGsv1jZBAlcLL1MLjLNIWPBkj5NlSz8aAzYKg+eNqknrUJ24QBybeR5A==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "openbsd"
],
 "engines": {
 "node": ">=18"
 }
},
```



**package-lock.json**

```
"node_modules/@esbuild/openbsd-x64": {
 "version": "0.25.12",
 "resolved": "https://registry.npmjs.org/@esbuild/openbsd-x64/-/openbsd-x64-0.25.12.tgz",
 "integrity":
"sha512-MZyXUkZHjQxUvzK7rN8DJ3SRmrVrke8ZyRusHlP+kuwqTcfWLyqMOE3sScPPyeIXN/mDJIfGXvcMqCgYKekoQw==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "openbsd"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/@esbuild/openharmony-arm64": {
 "version": "0.25.12",
 "resolved":
"https://registry.npmjs.org/@esbuild/openharmony-arm64/-/openharmony-arm64-0.25.12.tgz",
 "integrity":
"sha512-rm0YWsQUSRrjncSXGA7Zv78Nbnw4XL6/dzr20cyrQf7ZmRcsovpCRBdhD43Nuk3y7XIow20xMVvwuRvk9XdASg==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "openharmony"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/@esbuild/sunos-x64": {
 "version": "0.25.12",
 "resolved": "https://registry.npmjs.org/@esbuild/sunos-x64/-/sunos-x64-0.25.12.tgz",
 "integrity":
"sha512-3wGSCDyuTHQUzt0nV7bocDy72r2lI33QL3gkDNGkod22EsYl04sMf0qLb8luNKT0mgF/eDEDP5BFNwoBKH441w==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "sunos"
],
 "engines": {
 "node": ">=18"
 }
}
```

**package-lock.json**

```
 },
 "node_modules/@esbuild/win32-arm64": {
 "version": "0.25.12",
 "resolved": "https://registry.npmjs.org/@esbuild/win32-arm64/-/win32-arm64-0.25.12.tgz",
 "integrity":
"sha512-rMmLrur64A7+DKlnSuwqUdRKyd3UE7oPJZmnljqEptesKM8wx9J8gx5u0+9Pq0fQQW8vqeKebwNXdf0yP+8Bsg==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "win32"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/@esbuild/win32-ia32": {
 "version": "0.25.12",
 "resolved": "https://registry.npmjs.org/@esbuild/win32-ia32/-/win32-ia32-0.25.12.tgz",
 "integrity":
"sha512-HkqnmMBoCbCwxUKKNPBixiWDGcpQGVsrQfJoVGYLPT41XWF8lHuE5N6WhVia2n4o5QK5M4tYr21827fNhi4byQ==",
 "cpu": [
 "ia32"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "win32"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/@esbuild/win32-x64": {
 "version": "0.25.12",
 "resolved": "https://registry.npmjs.org/@esbuild/win32-x64/-/win32-x64-0.25.12.tgz",
 "integrity":
"sha512-alJC0uCZpTFrSL0CCDjcgLBXPnCrEAhTBILpeAp7M/OFgoqtAetfBzX0xM00MUSVVPpVj lPuMbREqnZCXaTnA==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "win32"
],
 "engines": {
 "node": ">=18"
 }
 }
```

**package-lock.json**

```
{
 "node_modules/@floating-ui/core": {
 "version": "1.8.0",
 "resolved": "https://registry.npmjs.org/@floating-ui/core/-/core-1.8.0.tgz",
 "integrity":
"sha512-0CI25itps/8x7BG8dEIhs53BvCUH2PCoogtakwRTut+Arm58sJooJ0AuZhLw2HJYIR5cMLNPBSS728sPho2khQ==",
 "license": "MIT",
 "optional": true,
 "dependencies": {
 "@floating-ui/utils": "^0.2.12"
 }
 },
 "node_modules/@floating-ui/dom": {
 "version": "1.8.0",
 "resolved": "https://registry.npmjs.org/@floating-ui/dom/-/dom-1.8.0.tgz",
 "integrity":
"sha512-yXSrZeHZBTZadL0lfyhCkJHNeLJnHRnRInwdZ40L7ZiaAtrBwoYlsDrX3v5zB1Utk7CLfzc0VnVWwoXEky7Ceg==",
 "license": "MIT",
 "optional": true,
 "dependencies": {
 "@floating-ui/core": "^1.8.0",
 "@floating-ui/utils": "^0.2.12"
 }
 },
 "node_modules/@floating-ui/utils": {
 "version": "0.2.12",
 "resolved": "https://registry.npmjs.org/@floating-ui/utils/-/utils-0.2.12.tgz",
 "integrity":
"sha512-HpCo8tmWzLVad5s2d19EhAz5zqrrQ6s69qd6moPMQvk0uSwDT1YgRfWSVuc4ennqrgv30Hppi0GMQ7oC13yIww==",
 "license": "MIT",
 "optional": true
 },
 "node_modules/@img/colour": {
 "version": "1.1.0",
 "resolved": "https://registry.npmjs.org/@img/colour/-/colour-1.1.0.tgz",
 "integrity":
"sha512-Td76q7j570/tLVdgS746cYARfSyxk8iEfRxewL9h40MzYhbW4TAcpl0mT4eyqXddh6L/jwoM75mo7ixa/pCeQ==",
 "dev": true,
 "license": "MIT",
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/@img/sharp-darwin-arm64": {
 "version": "0.35.2",
 "resolved":
"https://registry.npmjs.org/@img/sharp-darwin-arm64/-/sharp-darwin-arm64-0.35.2.tgz",
 "integrity":
"sha512-eEieHsMksAW4Ii05NzauESRl2D2qz3J/kwUxUrSfV06A93eEaRfMpHXyUb1mAqrR7i8U9A0GRqE9pjn6u1Jjpg==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "Apache-2.0",
 }
}
```

**package-lock.json**

```
"optional": true,
"os": [
 "darwin"
],
"engines": {
 "node": ">=20.9.0"
},
"funding": {
 "url": "https://opencollective.com/libvips"
},
"optionalDependencies": {
 "@img/sharp-libvips-darwin-arm64": "1.3.1"
}
},
"node_modules/@img/sharp-darwin-x64": {
 "version": "0.35.2",
 "resolved":
"https://registry.npmjs.org/@img/sharp-darwin-x64/-/sharp-darwin-x64-0.35.2.tgz",
 "integrity":
"sha512-BaktuGPCeHJMARpodR8jK4uKiZrPAy9WrfQW0sdI37clracq8Bp01AYS3SZgi5FS/y5twa9t4+LIuuxQjqRrWw==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "Apache-2.0",
 "optional": true,
 "os": [
 "darwin"
],
 "engines": {
 "node": ">=20.9.0"
 },
 "funding": {
 "url": "https://opencollective.com/libvips"
 },
 "optionalDependencies": {
 "@img/sharp-libvips-darwin-x64": "1.3.1"
 }
},
"node_modules/@img/sharp-freebsd-wasm32": {
 "version": "0.35.2",
 "resolved":
"https://registry.npmjs.org/@img/sharp-freebsd-wasm32/-/sharp-freebsd-wasm32-0.35.2.tgz",
 "integrity":
"sha512-YoAxdnd8hPUkvLHd3bWY+YA8nw3xM/RyRopYucNsWHVSan8NLVM3X2volSfoRDcXdUJPg6tXahSd7HXPk7lRnw==",
 "dev": true,
 "license": "Apache-2.0",
 "optional": true,
 "os": [
 "freebsd"
],
 "dependencies": {
 "@img/sharp-wasm32": "0.35.2"
 }
},
```

**package-lock.json**

```

 "engines": {
 "node": ">=20.9.0"
 },
 "funding": {
 "url": "https://opencollective.com/libvips"
 }
 },
 "node_modules/@img/sharp-libvips-darwin-arm64": {
 "version": "1.3.1",
 "resolved":
 "https://registry.npmjs.org/@img/sharp-libvips-darwin-arm64/-/sharp-libvips-darwin-arm64-1.3.1.tgz",
 "integrity":
 "sha512-4V/M3rORTYjiwZY9IOVQOE80yeCxFakYmyZDrZl51uOKjibm3oeEJ4WAmLxutAfzFbC9jqUiPs2gbnGflH+7g==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "LGPL-3.0-or-later",
 "optional": true,
 "os": [
 "darwin"
],
 "funding": {
 "url": "https://opencollective.com/libvips"
 }
 },
 "node_modules/@img/sharp-libvips-darwin-x64": {
 "version": "1.3.1",
 "resolved":
 "https://registry.npmjs.org/@img/sharp-libvips-darwin-x64/-/sharp-libvips-darwin-x64-1.3.1.tgz",
 "integrity":
 "sha512-c0/DxItpJv2+dGhgycJBBgotdgruGYDvA79drdh0MD1dFpy7JzJ/PlXwi1H4rFf0eTy8tgbI91aHDnZIceY3jQ==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "LGPL-3.0-or-later",
 "optional": true,
 "os": [
 "darwin"
],
 "funding": {
 "url": "https://opencollective.com/libvips"
 }
 },
 "node_modules/@img/sharp-libvips-linux-arm": {
 "version": "1.3.1",
 "resolved":
 "https://registry.npmjs.org/@img/sharp-libvips-linux-arm/-/sharp-libvips-linux-arm-1.3.1.tgz",
 "integrity":
 "sha512-aGGy9awzXgHBG7HNYQPworZthlp7+x6fDRoPAQbG03ThcttuTyKIX3NuSHb6zb4gBNq6/yNn9f1cy9nFKS/Vmg==",
 "cpu": [
 "arm"
],

```

**package-lock.json**

```
 "dev": true,
 "libc": [
 "glibc"
],
 "license": "LGPL-3.0-or-later",
 "optional": true,
 "os": [
 "linux"
],
 "funding": {
 "url": "https://opencollective.com/libvips"
 }
},
"node_modules/@img/sharp-libvips-linux-arm64": {
 "version": "1.3.1",
 "resolved":
"https://registry.npmjs.org/@img/sharp-libvips-linux-arm64/-/sharp-libvips-linux-arm64-1.3.1.tgz",
 "integrity":
"sha512-Jznefmck9j1JKPz8AkQDh89kjojubyf0asWBPKfzMIhPwsgDy9evpE/naJTXXXmghS1iFwR8u/kTwh/I2/+GCw==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "libc": [
 "glibc"
],
 "license": "LGPL-3.0-or-later",
 "optional": true,
 "os": [
 "linux"
],
 "funding": {
 "url": "https://opencollective.com/libvips"
 }
},
"node_modules/@img/sharp-libvips-linux-ppc64": {
 "version": "1.3.1",
 "resolved":
"https://registry.npmjs.org/@img/sharp-libvips-linux-ppc64/-/sharp-libvips-linux-ppc64-1.3.1.tgz",
 "integrity":
"sha512-1EkWGNcZk6iWNCMwqrvdJ+r1j0PT1zIz60CNPhYnJlK/zyewqlsPZie+ocBVqPF8k/Ssee/NCK+tE9Ryrko6ng==",
 "cpu": [
 "ppc64"
],
 "dev": true,
 "libc": [
 "glibc"
],
 "license": "LGPL-3.0-or-later",
 "optional": true,
 "os": [
 "linux"
],
 "funding": {
```

**package-lock.json**

```
 "url": "https://opencollective.com/libvips"
 },
 "node_modules/@img/sharp-libvips-linux-riscv64": {
 "version": "1.3.1",
 "resolved":
"https://registry.npmjs.org/@img/sharp-libvips-linux-riscv64/-/sharp-libvips-linux-riscv64-1.3.1.tgz",
 "integrity":
"sha512-Ilays+w2bXdnxxtQdmXR62u8o8GYa3eL4+Gr+1KiE4xperMZUslRaVPJwwPkzLHEjGfXAfRVAA/7CYCtSqsBw==",
 "cpu": [
 "riscv64"
],
 "dev": true,
 "libc": [
 "glibc"
],
 "license": "LGPL-3.0-or-later",
 "optional": true,
 "os": [
 "linux"
],
 "funding": {
 "url": "https://opencollective.com/libvips"
 }
 },
 "node_modules/@img/sharp-libvips-linux-s390x": {
 "version": "1.3.1",
 "resolved":
"https://registry.npmjs.org/@img/sharp-libvips-linux-s390x/-/sharp-libvips-linux-s390x-1.3.1.tgz",
 "integrity":
"sha512-VfBwVHQtbRoj4XlpA/KLZ7ltgMpz+4WSejFzQ+GnoImjo1PtEJ59QB2qR1xQEeRPYIkNrPIIm2L4cICMvz4C2ew==",
 "cpu": [
 "s390x"
],
 "dev": true,
 "libc": [
 "glibc"
],
 "license": "LGPL-3.0-or-later",
 "optional": true,
 "os": [
 "linux"
],
 "funding": {
 "url": "https://opencollective.com/libvips"
 }
 },
 "node_modules/@img/sharp-libvips-linux-x64": {
 "version": "1.3.1",
 "resolved":
"https://registry.npmjs.org/@img/sharp-libvips-linux-x64/-/sharp-libvips-linux-x64-1.3.1.tgz",
 "integrity":
"sha512-+c8UkgwU62DS54nCAjw7keOfHUKmr0B5QHEdc0qRnodF/MNXJbVI8Eopoj4B/0H8Asr65I+A4Amrn7a85/md6A==",
 "cpu": [
```

**package-lock.json**

```
 "x64"
],
 "dev": true,
 "libc": [
 "glibc"
],
 "license": "LGPL-3.0-or-later",
 "optional": true,
 "os": [
 "linux"
],
 "funding": {
 "url": "https://opencollective.com/libvips"
 }
},
"node_modules/@img/sharp-libvips-linuxmusl-arm64": {
 "version": "1.3.1",
 "resolved":
"https://registry.npmjs.org/@img/sharp-libvips-linuxmusl-arm64/-/sharp-libvips-linuxmusl-arm64-1.3.1.tgz",
 "integrity":
"sha512-qlKb/pwbkAi1wMsJrYHk7CuDrD12s27U2QnRhFYUoJNrRCmkosMTttuRfat/DDB3IlDm5qE1TJgZ4JDnHX8Ldw==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "libc": [
 "musl"
],
 "license": "LGPL-3.0-or-later",
 "optional": true,
 "os": [
 "linux"
],
 "funding": {
 "url": "https://opencollective.com/libvips"
 }
},
"node_modules/@img/sharp-libvips-linuxmusl-x64": {
 "version": "1.3.1",
 "resolved":
"https://registry.npmjs.org/@img/sharp-libvips-linuxmusl-x64/-/sharp-libvips-linuxmusl-x64-1.3.1.tgz",
 "integrity":
"sha512-y021HwoUVLN8Qa+/SBjQLMYwBWAJVjeGPNe+hc00UeMeifEtJqu5a1c4HayE1nNpDih9y3/KkoItfkDodmKAlg==",
 "cpu": [
 "x64"
],
 "dev": true,
 "libc": [
 "musl"
],
 "license": "LGPL-3.0-or-later",
 "optional": true,
 "os": [
 "linux"
]
}
```



**package-lock.json**

```
],
 "funding": {
 "url": "https://opencollective.com/libvips"
 }
},
"node_modules/@img/sharp-linux-arm": {
 "version": "0.35.2",
 "resolved": "https://registry.npmjs.org/@img/sharp-linux-arm/-/sharp-linux-arm-0.35.2.tgz",
 "integrity":
"sha512-SE4kzF2mepn6z+6E7L6lsV8FzuLL6IPQdyX8ZiwR0AG/G8td+hP/m7FsFPwidtrF19gvajuC9l6TxAVcsA4S7A==",
 "cpu": [
 "arm"
],
 "dev": true,
 "libc": [
 "glibc"
],
 "license": "Apache-2.0",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=20.9.0"
 },
 "funding": {
 "url": "https://opencollective.com/libvips"
 },
 "optionalDependencies": {
 "@img/sharp-libvips-linux-arm": "1.3.1"
 }
},
"node_modules/@img/sharp-linux-arm64": {
 "version": "0.35.2",
 "resolved":
"https://registry.npmjs.org/@img/sharp-linux-arm64/-/sharp-linux-arm64-0.35.2.tgz",
 "integrity":
"sha512-af12Pnd0ZGu2HfP8NayB0kk6eC/lrfbQE6HlR4jD+34wdJ1Vw9TF6TMn6ZvffT+WgqVs10hRbmNvz2u/23VmwA==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "libc": [
 "glibc"
],
 "license": "Apache-2.0",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=20.9.0"
 },
 "funding": {
```

**package-lock.json**

```
 "url": "https://opencollective.com/libvips"
 },
 "optionalDependencies": {
 "@img/sharp-libvips-linux-arm64": "1.3.1"
 }
},
"node_modules/@img/sharp-linux-ppc64": {
 "version": "0.35.2",
 "resolved":
"https://registry.npmjs.org/@img/sharp-linux-ppc64/-/sharp-linux-ppc64-0.35.2.tgz",
 "integrity":
"sha512-hYSBm7zcNtDCoZCxQHYZJiu63b/bXsgRZu0xCIBZsStMM9Vap47iFHdbX4kCvQsb1PB/k+c1hELpdQJHQLSHvg==",
 "cpu": [
 "ppc64"
],
 "dev": true,
 "libc": [
 "glibc"
],
 "license": "Apache-2.0",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=20.9.0"
 },
 "funding": {
 "url": "https://opencollective.com/libvips"
 },
 "optionalDependencies": {
 "@img/sharp-libvips-linux-ppc64": "1.3.1"
 }
},
"node_modules/@img/sharp-linux-riscv64": {
 "version": "0.35.2",
 "resolved":
"https://registry.npmjs.org/@img/sharp-linux-riscv64/-/sharp-linux-riscv64-0.35.2.tgz",
 "integrity":
"sha512-qQt0Kc13+Hoan/Awq/qMSQw3L+RI1NCRPgD5cUJ/1WSSmIoysL0c72j1RM3E00HN9Yr313jgeQ2T+zW+F03QFA==",
 "cpu": [
 "riscv64"
],
 "dev": true,
 "libc": [
 "glibc"
],
 "license": "Apache-2.0",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=20.9.0"
```

**package-lock.json**

```

 },
 "funding": {
 "url": "https://opencollective.com/libvips"
 },
 "optionalDependencies": {
 "@img/sharp-libvips-linux-riscv64": "1.3.1"
 }
 },
 "node_modules/@img/sharp-linux-s390x": {
 "version": "0.35.2",
 "resolved": "https://registry.npmjs.org/@img/sharp-linux-s390x/-/sharp-linux-s390x-0.35.2.tgz",
 "integrity": "sha512-E4fLLfRPzDLlEeDaTzI980FLcv++WL5ChLLMwPoVd0CIoZQqubBSNb0isPL5am9XsbQ9T84+iiMpUvbFtkunbA==",
 "cpu": [
 "s390x"
],
 "dev": true,
 "libc": [
 "glibc"
],
 "license": "Apache-2.0",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=20.9.0"
 },
 "funding": {
 "url": "https://opencollective.com/libvips"
 },
 "optionalDependencies": {
 "@img/sharp-libvips-linux-s390x": "1.3.1"
 }
 },
 "node_modules/@img/sharp-linux-x64": {
 "version": "0.35.2",
 "resolved": "https://registry.npmjs.org/@img/sharp-linux-x64/-/sharp-linux-x64-0.35.2.tgz",
 "integrity": "sha512-gi0zFJJRLswfCZmHtJdikXP0c5u7qamSOS3NHedLqLd4W8Q0NqjdBr6TTIRgsfFjqfTsHFgdfvJ9LwqSgcHiAA==",
 "cpu": [
 "x64"
],
 "dev": true,
 "libc": [
 "glibc"
],
 "license": "Apache-2.0",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {

```

**package-lock.json**

```
 "node": ">=20.9.0"
 },
 "funding": {
 "url": "https://opencollective.com/libvips"
 },
 "optionalDependencies": {
 "@img/sharp-libvips-linux-x64": "1.3.1"
 }
},
"node_modules/@img/sharp-linuxmusl-arm64": {
 "version": "0.35.2",
 "resolved":
"https://registry.npmjs.org/@img/sharp-linuxmusl-arm64/-/sharp-linuxmusl-arm64-0.35.2.tgz",
 "integrity":
"sha512-siWb0W1u6HFnlrP0waKyW7VEf7jYvcDWdrXEfa8AkdAQgEvuu5Fz8/Y70w9EeqAdwDtfU012BhEHhAdqvQNzg==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "libc": [
 "musl"
],
 "license": "Apache-2.0",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=20.9.0"
 },
 "funding": {
 "url": "https://opencollective.com/libvips"
 },
 "optionalDependencies": {
 "@img/sharp-libvips-linuxmusl-arm64": "1.3.1"
 }
},
"node_modules/@img/sharp-linuxmusl-x64": {
 "version": "0.35.2",
 "resolved":
"https://registry.npmjs.org/@img/sharp-linuxmusl-x64/-/sharp-linuxmusl-x64-0.35.2.tgz",
 "integrity":
"sha512-YBqMMcjDi4QGYiSn4vNOYBhmLC4z5AXqkOUUqI2e0AFA4urNv4ESgOgwNl3K+4etQhha0twXlzeF20bbULm9Yg==",
 "cpu": [
 "x64"
],
 "dev": true,
 "libc": [
 "musl"
],
 "license": "Apache-2.0",
 "optional": true,
 "os": [
 "linux"
]
}
```

**package-lock.json**

```
],
 "engines": {
 "node": ">=20.9.0"
 },
 "funding": {
 "url": "https://opencollective.com/libvips"
 },
 "optionalDependencies": {
 "@img/sharp-libvips-linuxmusl-x64": "1.3.1"
 }
 },
 "node_modules/@img/sharp-wasm32": {
 "version": "0.35.2",
 "resolved": "https://registry.npmjs.org/@img/sharp-wasm32/-/sharp-wasm32-0.35.2.tgz",
 "integrity":
"sha512-Mrv4JQNYVQ94xH+jzZ9r+gowleN8mv2FTgKT+PI6bx5C0G8TdNYndu161pg2i7uoBwxy2ImPMHrJOM2LZef7Bw==",
 "dev": true,
 "license": "Apache-2.0 AND LGPL-3.0-or-later AND MIT",
 "optional": true,
 "dependencies": {
 "@emnapi/runtime": "^1.11.1"
 },
 "engines": {
 "node": ">=20.9.0"
 },
 "funding": {
 "url": "https://opencollective.com/libvips"
 }
 },
 "node_modules/@img/sharp-webcontainers-wasm32": {
 "version": "0.35.2",
 "resolved":
"https://registry.npmjs.org/@img/sharp-webcontainers-wasm32/-/sharp-webcontainers-wasm32-0.35.2.tgz",
 "integrity":
"sha512-QNV27pxs9wpApEiCfvHM1RDoP1w1+2KrUWWDPEhEwg+latv0rfuhWrHWZKwdSFwU6jh3myjw/yOCRsUIuOft3g==",
 "cpu": [
 "wasm32"
],
 "dev": true,
 "license": "Apache-2.0",
 "optional": true,
 "dependencies": {
 "@img/sharp-wasm32": "0.35.2"
 },
 "engines": {
 "node": ">=20.9.0"
 },
 "funding": {
 "url": "https://opencollective.com/libvips"
 }
 },
 "node_modules/@img/sharp-win32-arm64": {
 "version": "0.35.2",
 "resolved":
```

**package-lock.json**

```
"https://registry.npmjs.org/@img/sharp-win32-arm64/-/sharp-win32-arm64-0.35.2.tgz",
 "integrity":
"sha512-BiVRYc/t6/Vl3e1hBx0hugG4oN9Pydf4fgMSpxTQJmwGUg/YoXTWHiFeRymHfCZzifxu4F4rpk/I67D0LQ20wQ==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "Apache-2.0 AND LGPL-3.0-or-later",
 "optional": true,
 "os": [
 "win32"
],
 "engines": {
 "node": ">=20.9.0"
 },
 "funding": {
 "url": "https://opencollective.com/libvips"
 }
},
"node_modules/@img/sharp-win32-ia32": {
 "version": "0.35.2",
 "resolved":
"https://registry.npmjs.org/@img/sharp-win32-ia32/-/sharp-win32-ia32-0.35.2.tgz",
 "integrity":
"sha512-YYEhx9PIImCC7T0tI8JDMi4DB9LwLCXCU50WNYEXAxb5Q1ShKkyC6byxzoBJ3gEFDnH2lQckWuDe70G7mB2XJog==",
 "cpu": [
 "ia32"
],
 "dev": true,
 "license": "Apache-2.0 AND LGPL-3.0-or-later",
 "optional": true,
 "os": [
 "win32"
],
 "engines": {
 "node": "^20.9.0"
 },
 "funding": {
 "url": "https://opencollective.com/libvips"
 }
},
"node_modules/@img/sharp-win32-x64": {
 "version": "0.35.2",
 "resolved": "https://registry.npmjs.org/@img/sharp-win32-x64/-/sharp-win32-x64-0.35.2.tgz",
 "integrity":
"sha512-imoOyBcoM/iiUr4J6VPpCNjPnjvP/Gks95898yB8YqoGGYmHYbOyCuNv9FMhFgtaiHFGbHW8bxKqRV6VjtXThQ==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "Apache-2.0 AND LGPL-3.0-or-later",
 "optional": true,
 "os": [
 "win32"
]
}
```

**package-lock.json**

```
],
 "engines": {
 "node": ">=20.9.0"
 },
 "funding": {
 "url": "https://opencollective.com/libvips"
 }
},
"node_modules/@jridgewell/gen-mapping": {
 "version": "0.3.13",
 "resolved": "https://registry.npmjs.org/@jridgewell/gen-mapping/-/gen-mapping-0.3.13.tgz",
 "integrity":
"sha512-2kkt/7niJ6MgEPxF0bYdQ6etZaA+fQvDcLKckhy1yIQ0zaoKjBBjSj63/aLVjYE3qhRt5dvM+uUyfCg6UKCBbA==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@jridgewell/sourcemap-codec": "^1.5.0",
 "@jridgewell/trace-mapping": "^0.3.24"
 }
},
"node_modules/@jridgewell/remapping": {
 "version": "2.3.5",
 "resolved": "https://registry.npmjs.org/@jridgewell/remapping/-/remapping-2.3.5.tgz",
 "integrity":
"sha512-LI9u/+laYG4Ds1TDKSJW2YPrIlcVY0wi2fUC6xB43lueCjgxV4lffOCZCtYFiH6TN0X+tQKXx97T4IKHbhyHEQ==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@jridgewell/gen-mapping": "^0.3.5",
 "@jridgewell/trace-mapping": "^0.3.24"
 }
},
"node_modules/@jridgewell/resolve-uri": {
 "version": "3.1.2",
 "resolved": "https://registry.npmjs.org/@jridgewell/resolve-uri/-/resolve-uri-3.1.2.tgz",
 "integrity":
"sha512-bRISgCIjP20/tbWSPWMEi54QVPRZExkuD9lJL+UIxUKtwVJA8wW1Trb1jMs1RFXo1CBTNZ/5hpC9QvmKWdopKw==",
 "dev": true,
 "license": "MIT",
 "engines": {
 "node": ">=6.0.0"
 }
},
"node_modules/@jridgewell/sourcemap-codec": {
 "version": "1.5.5",
 "resolved":
"https://registry.npmjs.org/@jridgewell/sourcemap-codec/-/sourcemap-codec-1.5.5.tgz",
 "integrity":
"sha512-cYQ9310grqxueWbl+WuIUIaiUaDcj7W0q5fVhEljNVgRf0UhyY9fy2zTvfoqWsnebh8Sl70VScFbICvJnLKB00g==",
 "dev": true,
 "license": "MIT"
},
"node_modules/@jridgewell/trace-mapping": {
 "version": "0.3.31",
```

**package-lock.json**

```
"resolved":
"https://registry.npmjs.org/@jridgewell/trace-mapping/-/trace-mapping-0.3.31.tgz",
"integrity":
"sha512-zzNR+SdQSDJzc8joaeP8QQoCQr8NuYx2dIIytl1QeBEZHJ9uW6hebsrYgbz8hJwUQao3TWCmtmfV8Nu1twOLAw==",
"dev": true,
"license": "MIT",
"dependencies": {
 "@jridgewell/resolve-uri": "^3.1.0",
 "@jridgewell/sourcemap-codec": "^1.4.14"
}
},
"node_modules/@napi-rs/lzma-linux-x64-gnu": {
 "version": "1.5.1",
 "resolved":
"https://registry.npmjs.org/@napi-rs/lzma-linux-x64-gnu/-/lzma-linux-x64-gnu-1.5.1.tgz",
 "integrity":
"sha512-oTXEIha4SsuXdTA4Iyskj0kpdx2yVXdhd75c2v3xGrHFfVMSbhTPZU/nMPL4sWko4pBhm3aucLaqGlF696dTqQ==",
 "cpu": [
 "x64"
],
 "dev": true,
 "libc": [
 "glibc"
],
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": "^22.20 || ^24.12 || >=25"
 }
},
"node_modules/@pdf-lib/standard-fonts": {
 "version": "1.0.0",
 "resolved": "https://registry.npmjs.org/@pdf-lib/standard-fonts/-/standard-fonts-1.0.0.tgz",
 "integrity":
"sha512-hU30BK9IUN/su0Mn9VdlVKsWBS6GyhVfqjw1lFjZN4TxP6cCw0jP2w7V3Hf5uX7M0AZJ16vey9yE0ny7Sa59ZA==",
 "license": "MIT",
 "dependencies": {
 "pako": "^1.0.6"
 }
},
"node_modules/@pdf-lib/upng": {
 "version": "1.0.1",
 "resolved": "https://registry.npmjs.org/@pdf-lib/upng/-/upng-1.0.1.tgz",
 "integrity":
"sha512-dQK2FUMQtowVP0mtIksrLZhdfXQZPC+taih1q4CvPZ5vqdxR/LKBaFg0oAfzd1GLHZXXSPdQfzQnt+ViGvEIQ==",
 "license": "MIT",
 "dependencies": {
 "pako": "^1.0.10"
 }
},
"node_modules/@poppinss/colors": {
```



**package-lock.json**

```
"version": "4.1.6",
"resolved": "https://registry.npmjs.org/@poppinss/colors/-/colors-4.1.6.tgz",
"integrity":
"sha512-H9xkIdFswbS8n1d6vmRd8+c10t2Qe+rZITbbDHHkQixH5+2x1FDGmi/0K+WgWiqQFKPSLIYB7jLH6Kpfn6Fleg==",
"dev": true,
"license": "MIT",
"dependencies": {
 "kleur": "^4.1.5"
}
},
"node_modules/@poppinss/dumper": {
"version": "0.6.5",
"resolved": "https://registry.npmjs.org/@poppinss/dumper/-/dumper-0.6.5.tgz",
"integrity":
"sha512-NBdYIb90J7Lf0I32d0ewKI1r7wnkiH6m920puQ3qHUeZkxNkQiFnXVWoE6YtFSv6Q0iPPf7ys6i+HWWecDz7sw==",
"dev": true,
"license": "MIT",
"dependencies": {
"@poppinss/colors": "^4.1.5",
"@sindresorhus/is": "^7.0.2",
"supports-color": "^10.0.0"
}
},
"node_modules/@poppinss/dumper/node_modules/supports-color": {
"version": "10.2.2",
"resolved": "https://registry.npmjs.org/supports-color/-/supports-color-10.2.2.tgz",
"integrity":
"sha512-SS+jx45GF1QjgEXQx4NJZV9Imqm02NPz5FNsIHrsDjh2YsHnawpan7SNQ1o8NuhrbHZy9AZhIoCUiCeaW/C80g==",
"dev": true,
"license": "MIT",
"engines": {
 "node": ">=18"
}
},
"funding": {
"url": "https://github.com/chalk/supports-color?sponsor=1"
}
},
"node_modules/@poppinss/exception": {
"version": "1.2.3",
"resolved": "https://registry.npmjs.org/@poppinss/exception/-/exception-1.2.3.tgz",
"integrity":
"sha512-dCED+QRChTVatE9ibtoaxc+Wkdz0SjYTKi/+uachHWIsfodVfpsueo3+DKpgU5Px8qXjgmXkSvhXvSCz3fnP9lw==",
"dev": true,
"license": "MIT"
},
"node_modules/@rolldown/pluginutils": {
"version": "1.0.0-rc.3",
"resolved": "https://registry.npmjs.org/@rolldown/pluginutils/-/pluginutils-1.0.0-rc.3.tgz",
"integrity":
"sha512-eybk3TjzzzV97Dlj5c+XrBFW57eTNhzod66y9HrBlzJ6NsCrWCp/2kaPS3K9wJmurBC0TdW4yPjXKZqlznim3Q==",
"dev": true,
"license": "MIT"
},
"node_modules/@rollup/rollup-android-arm-eabi": {
```

**package-lock.json**

```
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-android-arm-eabi/-/rollup-android-arm-eabi-4.62.4.tgz",
 "integrity":
"sha512-RrPokAb7dmbxFoe03TloqHy0jgye8RkBhSqmp4aJMIex4c9r46ZstPnleDQ0q1t46V0VjwIuwNogIqbodV1Vvg==",
 "cpu": [
 "arm"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "android"
]
},
"node_modules/@rollup/rollup-android-arm64": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-android-arm64/-/rollup-android-arm64-4.62.4.tgz",
 "integrity":
"sha512-JKuJc+pnps2pjy7L/N3v/cAkZxYlnmuZoD840ldbMI5KDbC4i09NKwPKYdjYFCMAIILBzYSFHxIJVYZRo2/8A==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "android"
]
},
"node_modules/@rollup/rollup-darwin-arm64": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-darwin-arm64/-/rollup-darwin-arm64-4.62.4.tgz",
 "integrity":
"sha512-krw5uS2STmvJ02x0uTXHbqQNuz+9eZ1iw+qXk9dmW2gvV4jV702hEo0nuhFrp0Pie1mBFtqbxyZZtC46hXL0w==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "darwin"
]
},
"node_modules/@rollup/rollup-darwin-x64": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-darwin-x64/-/rollup-darwin-x64-4.62.4.tgz",
 "integrity":
"sha512-wsTtxtgApb4PrOsNJIm0FZ1h3WvCC+k9uxLJ4ad75hgoS4NiRes2SoJF1DAyMwiUY8IssDqGcHbXuN0sx1tfF1A==",
 "cpu": [
```

**package-lock.json**

```
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "darwin"
]
},
"node_modules/@rollup/rollup-freebsd-arm64": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-freebsd-arm64/-/rollup-freebsd-arm64-4.62.4.tgz",
 "integrity":
"sha512-GUOnQlyZe3yAXhW0t0Msn5Qkrv5E5mZXa0thbARWi5Ei2szlVXJFQhddZ4HbAzh8q92w5twp+CQvs/eFanz9YQ==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "freebsd"
]
},
"node_modules/@rollup/rollup-freebsd-x64": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-freebsd-x64/-/rollup-freebsd-x64-4.62.4.tgz",
 "integrity":
"sha512-/Y7f3QuxjzPKsJA/rfEDa3+0vXqyjmJ50Ln8dPpCmWkKTrUoWHG1cWhTqaAMLob2m2nESWuC7yGrREz019Ztqg==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "freebsd"
]
},
"node_modules/@rollup/rollup-linux-arm-gnueabihf": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-linux-arm-gnueabihf/-/rollup-linux-arm-gnueabihf-4.62.4.tgz",
 "integrity":
"sha512-81wiiX3v7aqy+T+bT61TJ78yJjRquqFFTTbApt08imfQQzkPIW8t6aJbkTagtCCrXMNc9D66+geq1K7ydLPNqA==",
 "cpu": [
 "arm"
],
 "dev": true,
 "libc": [
 "glibc"
],
 "dev": true,
```

**package-lock.json**

```
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
]
},
"node_modules/@rollup/rollup-linux-arm-musleabihf": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-linux-arm-musleabihf/-/rollup-linux-arm-musleabihf-4.62.4.tgz",
 "integrity":
"sha512-9kmDIvNZqdoHOBZgNtpTBeLWY0/LVipM3H/j62P8848/l/VPEQL6N3uxU9pvP1oZAsXyC2MEnFP3ovRjo7WYNQ==",
 "cpu": [
 "arm"
],
 "dev": true,
 "libc": [
 "musl"
],
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
]
},
"node_modules/@rollup/rollup-linux-arm64-gnu": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-linux-arm64-gnu/-/rollup-linux-arm64-gnu-4.62.4.tgz",
 "integrity":
"sha512-CcnXHwnXg69g+DX5VWL3FHts3qMRN2uVEHX+BZvGLdd07/gXkn3ePjYt01LDJvxkGKVHMcLKBra1QUTH+6toYQ==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "libc": [
 "glibc"
],
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
]
},
"node_modules/@rollup/rollup-linux-arm64-musl": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-linux-arm64-musl/-/rollup-linux-arm64-musl-4.62.4.tgz",
 "integrity":
"sha512-iF0IbiHnTRuhrWLlRs0QFdZJJia7S80wkneJr4ocALP16u5yk6lWLINFwhHaEqBFMSkDUZofLkGos7+CPzGB3g==",
 "cpu": [
 "arm64"
],
 "dev": true,
```

**package-lock.json**

```
 "libc": [
 "musl"
],
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
]
},
"node_modules/@rollup/rollup-linux-loong64-gnu": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-linux-loong64-gnu/-/rollup-linux-loong64-gnu-4.62.4.tgz",
 "integrity":
"sha512-XnWYMI7euHlb5a871xPja+Gm7DRCFU+FGRrtS2sMq9N8FvqtpagUy6gD4YOemC5MRk9xbh8+jYMEJbigFQwsgA==",
 "cpu": [
 "loong64"
],
 "dev": true,
 "libc": [
 "glibc"
],
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
]
},
"node_modules/@rollup/rollup-linux-loong64-musl": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-linux-loong64-musl/-/rollup-linux-loong64-musl-4.62.4.tgz",
 "integrity":
"sha512-qGDA100U8xedCcsdRm9oaoQY8DAx/QT7uIXJWhCdx0ceIWX783UC9QSYkdpzAe29wNiVfp24+bZdQmn49o45SQ==",
 "cpu": [
 "loong64"
],
 "dev": true,
 "libc": [
 "musl"
],
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
]
},
"node_modules/@rollup/rollup-linux-ppc64-gnu": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-linux-ppc64-gnu/-/rollup-linux-ppc64-gnu-4.62.4.tgz",
 "integrity":
"sha512-ru4H6ezD7ysA5EiEK6qkkaEb4modH8CTej6kUy/gQi20u3kB3G7Zn8snXXkeJSC0FKG/rbPPtM/+9Wgas1961w==",
 "cpu": [
```

**package-lock.json**

```
 "ppc64"
],
 "dev": true,
 "libc": [
 "glibc"
],
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
]
},
"node_modules/@rollup/rollup-linux-ppc64-musl": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-linux-ppc64-musl/-/rollup-linux-ppc64-musl-4.62.4.tgz",
 "integrity":
"sha512-2W4M05WQVJnbJaZdvDb9rhBDuFU1nKIEpPFpJUBsTh2k1YY2g+0DViaWuy0AjQ5c0P7NvrvLzt3wvH0oiAvc7w==",
 "cpu": [
 "ppc64"
],
 "dev": true,
 "libc": [
 "musl"
],
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
]
},
"node_modules/@rollup/rollup-linux-riscv64-gnu": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-linux-riscv64-gnu/-/rollup-linux-riscv64-gnu-4.62.4.tgz",
 "integrity":
"sha512-+fxjfuoAmVMCYV5QyjoIpu0cp5D0i0TeqYFk1AVaxGr+/ravWLX89XfQmptsoWcaVy/TGf2hexzbU0rCQIL1CQ==",
 "cpu": [
 "riscv64"
],
 "dev": true,
 "libc": [
 "glibc"
],
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
]
},
"node_modules/@rollup/rollup-linux-riscv64-musl": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-linux-riscv64-musl/-/rollup-linux-riscv64-musl-4.62.4.tgz",
```

**package-lock.json**

```
"integrity":
"sha512-jTn8JfHGL4djjFxFuM06LmNUJDsst2jeVlsd90mIH6zc5sC9K6rIu04YajXatLUpBmBKL6b35ro1QZocLi+tcA==",
 "cpu": [
 "riscv64"
],
 "dev": true,
 "libc": [
 "musl"
],
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
]
},
"node_modules/@rollup/rollup-linux-s390x-gnu": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-linux-s390x-gnu/-/rollup-linux-s390x-gnu-4.62.4.tgz",
 "integrity":
"sha512-oCJCJL4pXsoDcP2QZ+JVLPTIRc6266zsIaeJJswImmF7H00W8nb6HuSgZLMWxJwaPf8ehbSw8yo0EUw925hKsA==",
 "cpu": [
 "s390x"
],
 "dev": true,
 "libc": [
 "glibc"
],
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
]
},
"node_modules/@rollup/rollup-linux-x64-gnu": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-linux-x64-gnu/-/rollup-linux-x64-gnu-4.62.4.tgz",
 "integrity":
"sha512-W69hukhZ3KKNRcAMIEzKvcFye42hh0FE1+YoYaf5+Ikacuftoco6y0/xouz0hc5d5W/s3yBro5jRiuEE/Q5vUw==",
 "cpu": [
 "x64"
],
 "dev": true,
 "libc": [
 "glibc"
],
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
]
},
"node_modules/@rollup/rollup-linux-x64-musl": {
```

**package-lock.json**

```
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-linux-x64-musl/-/rollup-linux-x64-musl-4.62.4.tgz",
 "integrity":
"sha512-qixbGG2jkjXhzXpsFZSR2Xpb8DN/UaxYsbb/STbuR/6fpaDgRmmaq1B/LmtF2wQF0F0SsK2jdE0RZ3a0zHn4QA==",
 "cpu": [
 "x64"
],
 "dev": true,
 "libc": [
 "musl"
],
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
]
},
"node_modules/@rollup/rollup-openbsd-x64": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-openbsd-x64/-/rollup-openbsd-x64-4.62.4.tgz",
 "integrity":
"sha512-nwEM//hxxv8mIo6jD7Hu4o48DvmV9pbV6gsKawU+4NFyqHoPKwrkRiZGLKUh0Bk8qNmDmpwFtPKg80Bo/Tn4xiQ==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "openbsd"
]
},
"node_modules/@rollup/rollup-openharmony-arm64": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-openharmony-arm64/-/rollup-openharmony-arm64-4.62.4.tgz",
 "integrity":
"sha512-s62SQ/vgsRSvMwDk0EFtqfASf0f26ZNaQuTA6Aok5lrikf89yI2W0gFHVZb2Jpgc6N8Jn0KZgCK2ici03CsxQ==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "openharmony"
]
},
"node_modules/@rollup/rollup-win32-arm64-msvc": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-win32-arm64-msvc/-/rollup-win32-arm64-msvc-4.62.4.tgz",
```



**package-lock.json**

```
"integrity":
"sha512-J6wGf8TVGbXJq+HH+ttTvrCfNKPbuZecV6KT1B8I18BC5IURUh5kl4Yl50EP5eFIUoI5BWxCsyYMhFsDx8kek==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "win32"
]
},
"node_modules/@rollup/rollup-win32-ia32-msvc": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-win32-ia32-msvc/-/rollup-win32-ia32-msvc-4.62.4.tgz",
 "integrity":
"sha512-zmfrQd/0wu6oJs8Vq8KwY/YtsKSsLtKe/HwAP4Wqy8LhwjeT55fHRAk0hYQ12wI3ayS4Tt12d5CDRD7N96SAYQ==",
 "cpu": [
 "ia32"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "win32"
]
},
"node_modules/@rollup/rollup-win32-x64-gnu": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-win32-x64-gnu/-/rollup-win32-x64-gnu-4.62.4.tgz",
 "integrity":
"sha512-qPzHqdj9rfUD+w79dtE07zi/kFwKyCJqplp5K5ygeLTp7jLpAoc160AH39HSmRC9UpozaecslEi8uAdEj6v2yw==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "win32"
]
},
"node_modules/@rollup/rollup-win32-x64-msvc": {
 "version": "4.62.4",
 "resolved":
"https://registry.npmjs.org/@rollup/rollup-win32-x64-msvc/-/rollup-win32-x64-msvc-4.62.4.tgz",
 "integrity":
"sha512-zD6NdewEBYGE9QF9vCrLj5YQB4oq9q91kPZS37Jwj5h0kvR1lTBSpSxKhKDw4IJtbQ35LS1HD9DZYGKIshU1Q==",
 "cpu": [
 "x64"
],
 "dev": true,
```

**package-lock.json**

```
 "license": "MIT",
 "optional": true,
 "os": [
 "win32"
]
},
"node_modules/@sindresorhus/is": {
 "version": "7.2.0",
 "resolved": "https://registry.npmjs.org/@sindresorhus/is/-/is-7.2.0.tgz",
 "integrity":
"sha512-P1Cz1dWwFfR4IR+U13mqqiGsLFFf1KbayybWwdd2vfctdV6hDpUkgCY0nKOLLTMSorD/jJNjtbqzf13K8DCCXQw==",
 "dev": true,
 "license": "MIT",
 "engines": {
 "node": ">=18"
 },
 "funding": {
 "url": "https://github.com/sindresorhus/is?sponsor=1"
 }
},
"node_modules/@speed-highlight/core": {
 "version": "1.2.23",
 "resolved": "https://registry.npmjs.org/@speed-highlight/core/-/core-1.2.23.tgz",
 "integrity":
"sha512-iRoq6i6JDJP6Mt2A5JaPvzw0pgYHH6k92ij+yXiTrB7T2y9N789aWE3EHwj/5zt1JBokcCBja3iYLVdu5wgnkg==",
 "dev": true,
 "license": "CC0-1.0"
},
"node_modules/@tiptap/core": {
 "version": "3.29.2",
 "resolved": "https://registry.npmjs.org/@tiptap/core/-/core-3.29.2.tgz",
 "integrity":
"sha512-oKUKiPUB7noilVYxI9lNzUD4rX17sHub+PYjMfHMWHG9A3nvIy+FdePIViiThKWF7ijhr3eIqHY51Bn+GAftw==",
 "license": "MIT",
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "peerDependencies": {
 "@tiptap/pm": "3.29.2"
 }
},
"node_modules/@tiptap/extension-blockquote": {
 "version": "3.29.2",
 "resolved":
"https://registry.npmjs.org/@tiptap/extension-blockquote/-/extension-blockquote-3.29.2.tgz",
 "integrity":
"sha512-ca4OzKDh0yaxg2+Z56bC2QnWsNsFp2YMRfVig1PDXYMVFMNJpLcnhxgq/9btn+xYAlYrj8RymOeCTYREOR6Zjg==",
 "license": "MIT",
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "peerDependencies": {

```

**package-lock.json**

```
 "@tiptap/core": "3.29.2",
 "@tiptap/pm": "3.29.2"
 },
 {
 "node_modules/@tiptap/extension-bold": {
 "version": "3.29.2",
 "resolved": "https://registry.npmjs.org/@tiptap/extension-bold/-/extension-bold-3.29.2.tgz",
 "integrity":
"sha512-elYbGxJsYnBb4leqrcjdIJuiG380BcOgN+UUzv0v+qEjFGVzHodFOMBl3qnmD6urYHNu5/qQK2S0qSSRXKCLNQ==",
 "license": "MIT",
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "peerDependencies": {
 "@tiptap/core": "3.29.2"
 }
 },
 "node_modules/@tiptap/extension-bubble-menu": {
 "version": "3.29.2",
 "resolved":
"https://registry.npmjs.org/@tiptap/extension-bubble-menu/-/extension-bubble-menu-3.29.2.tgz",
 "integrity":
"sha512-kzcWarcpr031rZu+R3Hzut5uxb3Qj/xxMDEYZIQ3uiq1yb5vEMXj8/SIyWautk6Nu8Rr1leV3rGdR4NzhzyFAA==",
 "license": "MIT",
 "optional": true,
 "dependencies": {
 "@floating-ui/dom": "^1.0.0"
 },
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "peerDependencies": {
 "@tiptap/core": "3.29.2",
 "@tiptap/pm": "3.29.2"
 }
 },
 "node_modules/@tiptap/extension-bullet-list": {
 "version": "3.29.2",
 "resolved":
"https://registry.npmjs.org/@tiptap/extension-bullet-list/-/extension-bullet-list-3.29.2.tgz",
 "integrity":
"sha512-3bWcCUPbCHv0XttlMdnAtXLNYwx2pblByMgxmGsaP9FU0QnslGXty6A6gHCqI33ygRg1vrA6U5Wtpwbi5aKu5g==",
 "license": "MIT",
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "peerDependencies": {
 "@tiptap/extension-list": "3.29.2"
 }
 },
 "node_modules/@tiptap/extension-code": {
```

**package-lock.json**

```
"version": "3.29.2",
"resolved": "https://registry.npmjs.org/@tiptap/extension-code/-/extension-code-3.29.2.tgz",
"integrity":
"sha512-c6W5UGuB7WNLpYocsgRzp0200TI4QjaI9jjHRMuty9z+s9DtaYM/HrRLNwVh6MopkHb+i/89Wkv8gCS34fftig==",
"license": "MIT",
"funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
},
"peerDependencies": {
 "@tiptap/core": "3.29.2"
}
},
"node_modules/@tiptap/extension-code-block": {
 "version": "3.29.2",
 "resolved":
"https://registry.npmjs.org/@tiptap/extension-code-block/-/extension-code-block-3.29.2.tgz",
 "integrity":
"sha512-w153ct8g6dLiPTdXQ6S0IMxX4SEo5Q50AmjdEEEcJ7ZcYUcde/ScSskLHF0Ymyt5ZFAiyEwr121+pux+p3/oAQ==",
 "license": "MIT",
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "peerDependencies": {
 "@tiptap/core": "3.29.2",
 "@tiptap/pm": "3.29.2"
 }
},
"node_modules/@tiptap/extension-document": {
 "version": "3.29.2",
 "resolved":
"https://registry.npmjs.org/@tiptap/extension-document/-/extension-document-3.29.2.tgz",
 "integrity":
"sha512-YUamvefLnsqu6124GavVTI7nqcFlQJ12ROB0oSwG69eSBZYNjg1tIs05LFrBxkwf4Xgqd6YzfJ9+FeG428RvzQ==",
 "license": "MIT",
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "peerDependencies": {
 "@tiptap/core": "3.29.2"
 }
},
"node_modules/@tiptap/extension-dropcursor": {
 "version": "3.29.2",
 "resolved":
"https://registry.npmjs.org/@tiptap/extension-dropcursor/-/extension-dropcursor-3.29.2.tgz",
 "integrity":
"sha512-KKno7cU9r1HdR48CRrsDu69/1UjZdoslq/UcE+Kx+tdhAv/aljXMkRSNzGMrBNOBDMHRgS1+58zm21WQWdQzwA==",
 "license": "MIT",
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 }
}
```

**package-lock.json**

```
 },
 "peerDependencies": {
 "@tiptap/extensions": "3.29.2"
 }
 },
 "node_modules/@tiptap/extension-floating-menu": {
 "version": "3.29.2",
 "resolved":
 "https://registry.npmjs.org/@tiptap/extension-floating-menu/-/extension-floating-menu-3.29.2.tgz",
 "integrity":
 "sha512-CBTq4Xs5aGWr65uFCIkKBms7960hmirWf/ax//iJ4fl9IfwqjwFuc2vQs9PmuwpKn9ctDHo2sMTtvyeCvIt5LQ==",
 "license": "MIT",
 "optional": true,
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "peerDependencies": {
 "@floating-ui/dom": "^1.0.0",
 "@tiptap/core": "3.29.2",
 "@tiptap/pm": "3.29.2"
 }
 },
 "node_modules/@tiptap/extension-gapcursor": {
 "version": "3.29.2",
 "resolved":
 "https://registry.npmjs.org/@tiptap/extension-gapcursor/-/extension-gapcursor-3.29.2.tgz",
 "integrity":
 "sha512-8Q39UR4/Tit759Iew9xZIE3NMwN11GsuA3FLheDyyGn7RrW02HD3HhUDlaze54Ki4HoosjFmChPlN4Ik2ubdRQ==",
 "license": "MIT",
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "peerDependencies": {
 "@tiptap/extensions": "3.29.2"
 }
 },
 "node_modules/@tiptap/extension-hard-break": {
 "version": "3.29.2",
 "resolved":
 "https://registry.npmjs.org/@tiptap/extension-hard-break/-/extension-hard-break-3.29.2.tgz",
 "integrity":
 "sha512-eUw3LN3fq8rXnjEUeI3D2Q0NYdLsU3yYQm4jxlErs2h4cfwrFjgf19VSUFVmm6LrFbbQ00nDVPeVLL6i0wDw2w==",
 "license": "MIT",
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "peerDependencies": {
 "@tiptap/core": "3.29.2"
 }
 },
 "node_modules/@tiptap/extension-heading": {
```

**package-lock.json**

```
 "version": "3.29.2",
 "resolved":
"https://registry.npmjs.org/@tiptap/extension-heading/-/extension-heading-3.29.2.tgz",
 "integrity":
"sha512-6W4aIy70Mh7BN1bG9zZ5FBLhJhU2UUEzgZJ/jwYSCcB30o8McLxJSEjhtoHiX8R78Ah2/JzBGvIe5oLZlbeE4A==",
 "license": "MIT",
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "peerDependencies": {
 "@tiptap/core": "3.29.2"
 }
},
"node_modules/@tiptap/extension-horizontal-rule": {
 "version": "3.29.2",
 "resolved":
"https://registry.npmjs.org/@tiptap/extension-horizontal-rule/-/extension-horizontal-rule-3.29.2.tgz",
 "integrity":
"sha512-8/ZPzbB9X85Mc9/7xVLZupQKBr2UVcQTGr512xtqMW+XkCQRHCph46tRo828YE13IMWI5fWn/FaNcQXG9cULSw==",
 "license": "MIT",
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "peerDependencies": {
 "@tiptap/core": "3.29.2",
 "@tiptap/pm": "3.29.2"
 }
},
"node_modules/@tiptap/extension-italic": {
 "version": "3.29.2",
 "resolved":
"https://registry.npmjs.org/@tiptap/extension-italic/-/extension-italic-3.29.2.tgz",
 "integrity":
"sha512-iH63V/5wsaMnY4Jz0+meaAGhaec4Ai0zOduzl6ZZr5IyGhZ1kthyW84ELt0dyLI3hNceAUhaNwc+I7+vX0aoXA==",
 "license": "MIT",
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "peerDependencies": {
 "@tiptap/core": "3.29.2"
 }
},
"node_modules/@tiptap/extension-link": {
 "version": "3.29.2",
 "resolved": "https://registry.npmjs.org/@tiptap/extension-link/-/extension-link-3.29.2.tgz",
 "integrity":
"sha512-DcVer5SqrexKCEP6Ip1UPxJUMvcRCCItSv0wxoGytanrimBh2smvcg6X0Dwnjlsi5H0updhyl+atYCmmQXUIXA==",
 "license": "MIT",
 "dependencies": {
 "linkifyjs": "^4.3.3"
 }
},
```

**package-lock.json**

```
"funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
},
"peerDependencies": {
 "@tiptap/core": "3.29.2",
 "@tiptap/pm": "3.29.2"
}
},
"node_modules/@tiptap/extension-list": {
 "version": "3.29.2",
 "resolved": "https://registry.npmjs.org/@tiptap/extension-list/-/extension-list-3.29.2.tgz",
 "integrity":
"sha512-WPZ9BHAPT6QeIm1vdVkuo0Wvy9a8/EZeJwV2VhU8LXyTAttvzyj4rsbbHyJWvYwLUSTt/QF2AZ2zhKo7u1w3/A==",
 "license": "MIT",
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "peerDependencies": {
 "@tiptap/core": "3.29.2",
 "@tiptap/pm": "3.29.2"
 }
},
"node_modules/@tiptap/extension-list-item": {
 "version": "3.29.2",
 "resolved":
"https://registry.npmjs.org/@tiptap/extension-list-item/-/extension-list-item-3.29.2.tgz",
 "integrity":
"sha512-s8vBVHFT0Qpu7CzAZ7S1kYmSiVaDvUNvMNCzUWnxj6VPfiwmX0eXd9FsjePrRCMoM5tFmPnhFTHDxY3D/eZeQ==",
 "license": "MIT",
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "peerDependencies": {
 "@tiptap/extension-list": "3.29.2"
 }
},
"node_modules/@tiptap/extension-list-keymap": {
 "version": "3.29.2",
 "resolved":
"https://registry.npmjs.org/@tiptap/extension-list-keymap/-/extension-list-keymap-3.29.2.tgz",
 "integrity":
"sha512-R+3k80LnxDC7Xy9ie0wUt5m2Je74u8mikothGmsYV02Zyq48fIbmZ+X6RBPCu7DB0I2FIUhHEFbKQeDWvDNmA==",
 "license": "MIT",
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "peerDependencies": {
 "@tiptap/extension-list": "3.29.2"
 }
},
}
```

**package-lock.json**

```
"node_modules/@tiptap/extension-ordered-list": {
 "version": "3.29.2",
 "resolved":
 "https://registry.npmjs.org/@tiptap/extension-ordered-list/-/extension-ordered-list-3.29.2.tgz",
 "integrity":
 "sha512-ndCunC+UsY0pkOtL7vGnDz21UNa45WUlc09wMT1fbuYow2QnRhsuMlCWENXI52YPPARWuQ0RDgN7q6TaxPERBg==",
 "license": "MIT",
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "peerDependencies": {
 "@tiptap/extension-list": "3.29.2"
 }
},
"node_modules/@tiptap/extension-paragraph": {
 "version": "3.29.2",
 "resolved":
 "https://registry.npmjs.org/@tiptap/extension-paragraph/-/extension-paragraph-3.29.2.tgz",
 "integrity":
 "sha512-7qJj5YTr11vvjNgjDN1yp0fwTovc0Q0CYcit/rskeuVgnmQZ0ZQzC/BbyKLLG7UGnpRLemU/mEGbW9pAqjAXkQ==",
 "license": "MIT",
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "peerDependencies": {
 "@tiptap/core": "3.29.2"
 }
},
"node_modules/@tiptap/extension-strike": {
 "version": "3.29.2",
 "resolved":
 "https://registry.npmjs.org/@tiptap/extension-strike/-/extension-strike-3.29.2.tgz",
 "integrity":
 "sha512-aEvLAbddUQZ+FukCrev3q4G2HfNI+odE7E9U+wbq6XsSWKyo8/pDu1muz+TFKNre4bLSMOQ3JQmw5UeHDKy+fg==",
 "license": "MIT",
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "peerDependencies": {
 "@tiptap/core": "3.29.2"
 }
},
"node_modules/@tiptap/extension-text": {
 "version": "3.29.2",
 "resolved": "https://registry.npmjs.org/@tiptap/extension-text/-/extension-text-3.29.2.tgz",
 "integrity":
 "sha512-Ubko45JWwHe8glBt2PiGNF8hcbys/JNalFhiR7Y1X4i00tAxAKJJxh3+eq+//NTlGuBPdWgP7zw8EEUp7anjKA==",
 "license": "MIT",
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 }
}
```



**package-lock.json**

```

 },
 "peerDependencies": {
 "@tiptap/core": "3.29.2"
 }
 },
 "node_modules/@tiptap/extension-underline": {
 "version": "3.29.2",
 "resolved": "https://registry.npmjs.org/@tiptap/extension-underline/-/extension-underline-3.29.2.tgz",
 "integrity": "sha512-K7XwH/xS/5AIREWQ00VTEf/W5U0olp7j6wwit7cdd/8nHv6h6AGr1+iEApHKoLXWQZLfGQKzJlT9W61LAl+fHA==",
 "license": "MIT",
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "peerDependencies": {
 "@tiptap/core": "3.29.2"
 }
 },
 "node_modules/@tiptap/extensions": {
 "version": "3.29.2",
 "resolved": "https://registry.npmjs.org/@tiptap/extensions/-/extensions-3.29.2.tgz",
 "integrity": "sha512-BCz+FCACHSYtUe4BFj97HE0+nSK+J7GxbJgZG4Hg7DT/gI+hRyeNndU8efiQAx3WGdzsFi3UxRpcF1tTQM7iMQ==",
 "license": "MIT",
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "peerDependencies": {
 "@tiptap/core": "3.29.2",
 "@tiptap/pm": "3.29.2"
 }
 },
 "node_modules/@tiptap/pm": {
 "version": "3.29.2",
 "resolved": "https://registry.npmjs.org/@tiptap/pm/-/pm-3.29.2.tgz",
 "integrity": "sha512-GC0me7xHaS+DSoaA4CDcAD3l6JyBlvZhvCyfsy2Vp6j8tEoBkZWio7soYVosmlyn7zq8/64VeFZP5s47yfG7fQ==",
 "license": "MIT",
 "dependencies": {
 "prosemirror-changeset": "^2.4.1",
 "prosemirror-commands": "^1.7.1",
 "prosemirror-dropcursor": "^1.8.2",
 "prosemirror-gapcursor": "^1.4.1",
 "prosemirror-history": "^1.5.0",
 "prosemirror-inputrules": "^1.5.1",
 "prosemirror-keymap": "^1.2.3",
 "prosemirror-model": "^1.25.11",
 "prosemirror-schema-list": "^1.5.1",
 "prosemirror-state": "^1.4.4",
 "prosemirror-tables": "^1.8.5",
 "prosemirror-transform": "^1.12.0",

```

**package-lock.json**

```
 "prosemirror-view": "^1.41.9"
 },
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 }
},
"node_modules/@tiptap/react": {
 "version": "3.29.2",
 "resolved": "https://registry.npmjs.org/@tiptap/react/-/react-3.29.2.tgz",
 "integrity": "sha512-ZMKgteYmZXnq7C22U9fbCTC6jAF8XlQM5n2K2FLWCdYAlP4FNV3XeQ9hY2a13KSQ0Q3yltWXZlcWBpt5ClxScg==",
 "license": "MIT",
 "dependencies": {
 "@types/use-sync-external-store": "^0.0.6",
 "fast-equals": "^5.3.3",
 "use-sync-external-store": "^1.4.0"
 },
 "funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
 },
 "optionalDependencies": {
 "@tiptap/extension-bubble-menu": "^3.29.2",
 "@tiptap/extension-floating-menu": "^3.29.2"
 },
 "peerDependencies": {
 "@tiptap/core": "3.29.2",
 "@tiptap/pm": "3.29.2",
 "@types/react": "^17.0.0 || ^18.0.0 || ^19.0.0",
 "@types/react-dom": "^17.0.0 || ^18.0.0 || ^19.0.0",
 "react": "^17.0.0 || ^18.0.0 || ^19.0.0",
 "react-dom": "^17.0.0 || ^18.0.0 || ^19.0.0"
 }
},
"node_modules/@tiptap/starter-kit": {
 "version": "3.29.2",
 "resolved": "https://registry.npmjs.org/@tiptap/starter-kit/-/starter-kit-3.29.2.tgz",
 "integrity": "sha512-oTu0tysiqlk4zgJEtxRHjAQgxUKaAevZwueOWwSWubHdokqp7SpCbE5n9USJv89HKuTUDm3GjnQH6q8HNN/2DsA==",
 "license": "MIT",
 "dependencies": {
 "@tiptap/core": "^3.29.2",
 "@tiptap/extension-blockquote": "^3.29.2",
 "@tiptap/extension-bold": "^3.29.2",
 "@tiptap/extension-bullet-list": "^3.29.2",
 "@tiptap/extension-code": "^3.29.2",
 "@tiptap/extension-code-block": "^3.29.2",
 "@tiptap/extension-document": "^3.29.2",
 "@tiptap/extension-dropcursor": "^3.29.2",
 "@tiptap/extension-gapcursor": "^3.29.2",
 "@tiptap/extension-hard-break": "^3.29.2",
 "@tiptap/extension-heading": "^3.29.2",
 "@tiptap/extension-horizontal-rule": "^3.29.2",

```

**package-lock.json**

```
"@tiptap/extension-italic": "^3.29.2",
"@tiptap/extension-link": "^3.29.2",
"@tiptap/extension-list": "^3.29.2",
"@tiptap/extension-list-item": "^3.29.2",
"@tiptap/extension-list-keymap": "^3.29.2",
"@tiptap/extension-ordered-list": "^3.29.2",
"@tiptap/extension-paragraph": "^3.29.2",
"@tiptap/extension-strike": "^3.29.2",
"@tiptap/extension-text": "^3.29.2",
"@tiptap/extension-underline": "^3.29.2",
"@tiptap/extensions": "^3.29.2",
"@tiptap/pm": "^3.29.2"
},
"funding": {
 "type": "github",
 "url": "https://github.com/sponsors/ueberdosis"
}
},
"node_modules/@types/babel__core": {
 "version": "7.20.5",
 "resolved": "https://registry.npmjs.org/@types/babel__core/-/babel__core-7.20.5.tgz",
 "integrity":
"sha512-qoQprZvz5wQFJwMDqeserXWv3rqMvhgpbXFfVYWhbx9X47P0IA6i/+dXefEmZKoAg0aTdaIgNSMqMIU61yRyzA==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@babel/parser": "^7.20.7",
 "@babel/types": "^7.20.7",
 "@types/babel__generator": "*",
 "@types/babel__template": "*",
 "@types/babel__traverse": "*"
 }
},
"node_modules/@types/babel__generator": {
 "version": "7.27.0",
 "resolved":
"https://registry.npmjs.org/@types/babel__generator/-/babel__generator-7.27.0.tgz",
 "integrity":
"sha512-ufFd2Xi920AVPYsy+P4n7/U7e68fex0+Ee8gSG9KX7eo084CWiQ4sdxktvdl0b0PupXtVJPY19zk6EwwqUQ8lg==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@babel/types": "^7.0.0"
 }
},
"node_modules/@types/babel__template": {
 "version": "7.4.4",
 "resolved": "https://registry.npmjs.org/@types/babel__template/-/babel__template-7.4.4.tgz",
 "integrity":
"sha512-h/NUaSyG5EyxBIp8YRxo4RMe2/qQgvyowRwVMzhYhBCONbw8PUsg4lkFMrhgZhUe5z3L3MiLDuvyJ/CaPa2A8A==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@babel/parser": "^7.1.0",
```



**package-lock.json**

```
"https://registry.npmjs.org/@types/use-sync-external-store/-/use-sync-external-store-0.0.6.tgz",
 "integrity":
"sha512-zFDAD+tlpf2r4asuHEj0XH6pY6i0g5NeAHPn+15wk3BV6JA69eERFXC1gyGThDkVa1zCyKr5jox1+2LbV/AMLg==",
 "license": "MIT"
},
"node_modules/@vitejs/plugin-react": {
 "version": "5.2.0",
 "resolved": "https://registry.npmjs.org/@vitejs/plugin-react/-/plugin-react-5.2.0.tgz",
 "integrity":
"sha512-YmKkfH0Ai3wsB1PhJq5Scj3GXMn3WvtQ/JC0xoopuHoXSdmtStOpFrYaT1kie2YgFBcIe64R0zMYRjCrY0dYw==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@babel/core": "^7.29.0",
 "@babel/plugin-transform-react-jsx-self": "^7.27.1",
 "@babel/plugin-transform-react-jsx-source": "^7.27.1",
 "@rollup/pluginutils": "1.0.0-rc.3",
 "@types/babel__core": "^7.20.5",
 "react-refresh": "^0.18.0"
 },
 "engines": {
 "node": "^20.19.0 || >=22.12.0"
 },
 "peerDependencies": {
 "vite": "^4.2.0 || ^5.0.0 || ^6.0.0 || ^7.0.0 || ^8.0.0"
 }
},
"node_modules/ansi-regex": {
 "version": "5.0.1",
 "resolved": "https://registry.npmjs.org/ansi-regex/-/ansi-regex-5.0.1.tgz",
 "integrity":
"sha512-quJQlTUSUGL2LH9SUXo8VwsY4soanhgo6LNSm84E1LBCe8s300wpdiRzyR9z/ZZJMLMwv37q00b9pdJLMUEKFQ==",
 "dev": true,
 "license": "MIT",
 "engines": {
 "node": ">=8"
 }
},
"node_modules/ansi-styles": {
 "version": "4.3.0",
 "resolved": "https://registry.npmjs.org/ansi-styles/-/ansi-styles-4.3.0.tgz",
 "integrity":
"sha512-zbB9rCJAT1rbjiVDb2hqKFHNYLxgtk8NURxZ3IZwD3F6NtxbXZQCnnSi1Lkx+IDohdPlFp222wVALIheZJQSEg==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "color-convert": "^2.0.1"
 },
 "engines": {
 "node": ">=8"
 },
 "funding": {
 "url": "https://github.com/chalk/ansi-styles?sponsor=1"
 }
}
```

**package-lock.json**

```
 },
 "node_modules/baseline-browser-mapping": {
 "version": "2.11.12",
 "resolved": "https://registry.npmjs.org/baseline-browser-mapping/-/baseline-browser-mapping-2.11.12.tgz",
 "integrity": "sha512-r7WnVImvVCeFpf2D0Xfy41aPWzeNg3H/A2X4dKmy1QL0MSyyk/e7z8ihJ3N6Nn2PsdhkVlqnEfnUE4a05P2aTA==",
 "dev": true,
 "license": "Apache-2.0",
 "bin": {
 "baseline-browser-mapping": "dist/cli.cjs"
 },
 "engines": {
 "node": ">=6.0.0"
 }
 },
 "node_modules/blake3-wasm": {
 "version": "2.1.5",
 "resolved": "https://registry.npmjs.org/blake3-wasm/-/blake3-wasm-2.1.5.tgz",
 "integrity": "sha512-F1+K8EbF0ZE49dtoPtmxUQrpXaBIl3ICvasLh+nJta0xkz+9kF/7uet9fLnwKqhDrmj6g+6K3Tw9yQPUg2ka5g==",
 "dev": true,
 "license": "MIT"
 },
 "node_modules/browserslist": {
 "version": "4.28.7",
 "resolved": "https://registry.npmjs.org/browserslist/-/browserslist-4.28.7.tgz",
 "integrity": "sha512-Jxv13hNrFxfqjOc8alRbq9dK1MM79NEXYpma2B2J4wAtpWS5zIEIKqWPGCl7N4o7Uc7B7itylh7SuDujATRyyTw==",
 "dev": true,
 "funding": [
 {
 "type": "opencollective",
 "url": "https://opencollective.com/browserslist"
 },
 {
 "type": "tidelift",
 "url": "https://tidelift.com/funding/github/npm/browserslist"
 },
 {
 "type": "github",
 "url": "https://github.com/sponsors/ai"
 }
],
 "license": "MIT",
 "dependencies": {
 "baseline-browser-mapping": "^2.10.44",
 "caniuse-lite": "^1.0.30001806",
 "electron-to-chromium": "^1.5.393",
 "node-releases": "^2.0.51",
 "update-browserslist-db": "^1.2.3"
 },
 "bin": {
 "browserslist": "cli.js"
 }
 }
}
```

**package-lock.json**

```
 },
 "engines": {
 "node": "^6 || ^7 || ^8 || ^9 || ^10 || ^11 || ^12 || >=13.7"
 }
 },
 "node_modules/buffer-from": {
 "version": "1.1.2",
 "resolved": "https://registry.npmjs.org/buffer-from/-/buffer-from-1.1.2.tgz",
 "integrity": "sha512-E+XQCRwSbaaiChtv6k6DwgC+bx+Bs6vuKJHHL5kox/BaKbhiXzqQ0wK4c022yElGp20CmjlwVhT3HmxgyPGnJfQ==",
 "dev": true,
 "license": "MIT"
 },
 "node_modules/caniuse-lite": {
 "version": "1.0.30001806",
 "resolved": "https://registry.npmjs.org/caniuse-lite/-/caniuse-lite-1.0.30001806.tgz",
 "integrity": "sha512-72Cuvd95zbSYPKq6Fhg8eDJRlZgWdf7/mtoZv6Qe/DYNCEBdNxoA3+rZAU2ZhGCPzlns3EssFavaZomckT5Uuw==",
 "dev": true,
 "funding": [
 {
 "type": "opencollective",
 "url": "https://opencollective.com/browserslist"
 },
 {
 "type": "tidelift",
 "url": "https://tidelift.com/funding/github/npm/caniuse-lite"
 },
 {
 "type": "github",
 "url": "https://github.com/sponsors/ai"
 }
],
 "license": "CC-BY-4.0"
 },
 "node_modules/chalk": {
 "version": "4.1.2",
 "resolved": "https://registry.npmjs.org/chalk/-/chalk-4.1.2.tgz",
 "integrity": "sha512-oKnbhFyRlXpUuez8iBMmyEa4nbj4IOQyuhc/wy9kY7/WVPcwIO9VA668Pu8Rk07+0G76SLR0eyw9CpQ061i4mA==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "ansi-styles": "^4.1.0",
 "supports-color": "^7.1.0"
 },
 "engines": {
 "node": ">=10"
 },
 "funding": {
 "url": "https://github.com/chalk/chalk?sponsor=1"
 }
 },
 "node_modules/chalk/node_modules/supports-color": {
```

**package-lock.json**

```
 "version": "7.2.0",
 "resolved": "https://registry.npmjs.org/supports-color/-/supports-color-7.2.0.tgz",
 "integrity":
"sha512-qpCAvRl9stu0HveKsn7HncJRvv501qIacKzQl0/+Lwxc9+0q2wLyv4Dfvt80/DPn2pq0BsJdDiogXGR9+0vwRw==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "has-flag": "^4.0.0"
 },
 "engines": {
 "node": ">=8"
 }
},
"node_modules/cliui": {
 "version": "8.0.1",
 "resolved": "https://registry.npmjs.org/cliui/-/cliui-8.0.1.tgz",
 "integrity":
"sha512-BSeNnyus75C4//NQ9gQt1/cSTXyo/8Sb+afLakzAptFuMsod9HFokGNudZpi/oQV73hnVK+sR+5PVRMd+Dr7YQ==",
 "dev": true,
 "license": "ISC",
 "dependencies": {
 "string-width": "^4.2.0",
 "strip-ansi": "^6.0.1",
 "wrap-ansi": "^7.0.0"
 },
 "engines": {
 "node": ">=12"
 }
},
"node_modules/color-convert": {
 "version": "2.0.1",
 "resolved": "https://registry.npmjs.org/color-convert/-/color-convert-2.0.1.tgz",
 "integrity":
"sha512-RRECPsj7iu/xb5oKYcsFHSppFNnsj/520VTRKb4zP5onXwVF3zVmmToNcOfGC+CRDpfK/U584fMg38ZHCaElKQ==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "color-name": "~1.1.4"
 },
 "engines": {
 "node": ">=7.0.0"
 }
},
"node_modules/color-name": {
 "version": "1.1.4",
 "resolved": "https://registry.npmjs.org/color-name/-/color-name-1.1.4.tgz",
 "integrity":
"sha512-d0y+3AuW3a2wNbZHIuMZpTcgjGuLU/uBL/ubcZF9OXbDo8ff408yVp5Bf0efS8uEoYo5q4Fx7dY90gQGxgAsQA==",
 "dev": true,
 "license": "MIT"
},
"node_modules/concurrently": {
 "version": "9.2.4",
 "resolved": "https://registry.npmjs.org/concurrently/-/concurrently-9.2.4.tgz",
```



**package-lock.json**

```
"integrity":
"sha512-TZ0CEhyzvFjgtAvHTusDMgj7wNdiCh7LLLrzdUOXIhdlnL2JBBGA9eJxR24rtqgmdjh30A3hrN1rCHj6HM8qA==",
"dev": true,
"license": "MIT",
"dependencies": {
 "chalk": "4.1.2",
 "rxjs": "7.8.2",
 "shell-quote": "1.9.0",
 "supports-color": "8.1.1",
 "tree-kill": "1.2.2",
 "yargs": "17.7.2"
},
"bin": {
 "conc": "dist/bin/concurrently.js",
 "concurrently": "dist/bin/concurrently.js"
},
"engines": {
 "node": ">=18"
},
"funding": {
 "url": "https://github.com/open-cli-tools/concurrently?sponsor=1"
}
},
"node_modules/convert-source-map": {
 "version": "2.0.0",
 "resolved": "https://registry.npmjs.org/convert-source-map/-/convert-source-map-2.0.0.tgz",
 "integrity":
"sha512-Kvp459HrV2FEJ1CAasi1Ku+MY3kasH19TFykTz2xWmMeq6bk2NU3XXvfJ+Q61m0xktWwt+1HSYf3JZsTms3aRJg==",
 "dev": true,
 "license": "MIT"
},
"node_modules/cookie": {
 "version": "1.1.1",
 "resolved": "https://registry.npmjs.org/cookie/-/cookie-1.1.1.tgz",
 "integrity":
"sha512-ei8Aos7ja0WeRpFzJnEA9UHJ/7XQmqglbRwnf2ATjcB9Wq874VKH9kfjjirM6UhU2/E5fFYadylyhFldcqSidQ==",
 "license": "MIT",
 "engines": {
 "node": ">=18"
 },
 "funding": {
 "type": "opencollective",
 "url": "https://opencollective.com/express"
 }
},
"node_modules/csstype": {
 "version": "3.2.3",
 "resolved": "https://registry.npmjs.org/csstype/-/csstype-3.2.3.tgz",
 "integrity":
"sha512-z1HGKcYy2xA8AGQfwrn0PAy+PB7X/GSj3UVJW9qKyn43xWa+gl5nXmU4qqLMRzWVLF8KusUX8T/0kCi0YpAIQ==",
 "license": "MIT"
},
"node_modules/debug": {
 "version": "4.4.3",
```

**package-lock.json**

```
"resolved": "https://registry.npmjs.org/debug/-/debug-4.4.3.tgz",
"integrity":
"sha512-RGwwWnwQvkVfavKVt22FGLw+xYSdzARwm0ru6DhTVA3umU5hZc28V3k04stgYryrTlLpuvgI9GiiJltAjNbcqA==",
"dev": true,
"license": "MIT",
"dependencies": {
 "ms": "^2.1.3"
},
"engines": {
 "node": ">=6.0"
},
"peerDependenciesMeta": {
 "supports-color": {
 "optional": true
 }
}
},
"node_modules/detect-libc": {
 "version": "2.1.2",
 "resolved": "https://registry.npmjs.org/detect-libc/-/detect-libc-2.1.2.tgz",
 "integrity":
"sha512-Btj2B0008303WyH59e8MgXsxEQVcarkUOpEYrubB0urwnN10yQ364rsiByU11nZlqWYZm05i/of7io4mzihBtQ==",
 "dev": true,
 "license": "Apache-2.0",
 "engines": {
 "node": ">=8"
 }
},
"node_modules/dompurify": {
 "version": "3.4.13",
 "resolved": "https://registry.npmjs.org/dompurify/-/dompurify-3.4.13.tgz",
 "integrity":
"sha512-2vmYIoqjze2d+kakP8S/nS5shfsl587kzwEjcgLTdiksUVgFHNFCsLYDVj/JNqJV0QZGSYBTmuycv0PodwmnMQ==",
 "license": "(MPL-2.0 OR Apache-2.0)",
 "optionalDependencies": {
 "@types/trusted-types": "^2.0.7"
 }
},
"node_modules/drizzle-kit": {
 "version": "0.31.10",
 "resolved": "https://registry.npmjs.org/drizzle-kit/-/drizzle-kit-0.31.10.tgz",
 "integrity":
"sha512-70ZcmQUrdGI+DUNNSkBN1aw8qSoKuTH7d0mYgSP8bAzdFzKoovxEFnoGQp2dVs82E0JeYycqRtciopszwUf8bw==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@drizzle-team/brocli": "^0.10.2",
 "@esbuild-kit/esm-loader": "^2.5.5",
 "esbuild": "^0.25.4",
 "tsx": "^4.21.0"
 },
 "bin": {
 "drizzle-kit": "bin.cjs"
 }
}
```

**package-lock.json**

```
 },
 "node_modules/drizzle-orm": {
 "version": "0.45.2",
 "resolved": "https://registry.npmjs.org/drizzle-orm/-/drizzle-orm-0.45.2.tgz",
 "integrity":
"sha512-kY0BSaTNYWnoDMVoyY8uxmyHjpJW1geOmBmdSSicKo9CIIWkSxMIj2rkeSR51b8KAPB7m+qysjuHme5nKP+E5Q==",
 "license": "Apache-2.0",
 "peerDependencies": {
 "@aws-sdk/client-rds-data": ">=3",
 "@cloudflare/workers-types": ">=4",
 "@electric-sql/pglite": ">=0.2.0",
 "@libsql/client": ">=0.10.0",
 "@libsql/client-wasm": ">=0.10.0",
 "@neondatabase/serverless": ">=0.10.0",
 "@op-engineering/op-sqlite": ">=2",
 "@opentelemetry/api": "^1.4.1",
 "@planetscale/database": ">=1.13",
 "@prisma/client": "*",
 "@tidbcloud/serverless": "*",
 "@types/better-sqlite3": "*",
 "@types/pg": "*",
 "@types/sql.js": "*",
 "@upstash/redis": ">=1.34.7",
 "@vercel/postgres": ">=0.8.0",
 "@xata.io/client": "*",
 "better-sqlite3": ">=7",
 "bun-types": "*",
 "expo-sqlite": ">=14.0.0",
 "gel": ">=2",
 "knex": "*",
 "kysely": "*",
 "mysql2": ">=2",
 "pg": ">=8",
 "postgres": ">=3",
 "sql.js": ">=1",
 "sqlite3": ">=5"
 },
 },
 "peerDependenciesMeta": {
 "@aws-sdk/client-rds-data": {
 "optional": true
 },
 "@cloudflare/workers-types": {
 "optional": true
 },
 "@electric-sql/pglite": {
 "optional": true
 },
 "@libsql/client": {
 "optional": true
 },
 "@libsql/client-wasm": {
 "optional": true
 },
 "@neondatabase/serverless": {
```

**package-lock.json**

```
 "optional": true
 },
 "@op-engineering/op-sqlite": {
 "optional": true
 },
 "@opentelemetry/api": {
 "optional": true
 },
 "@planetscale/database": {
 "optional": true
 },
 "@prisma/client": {
 "optional": true
 },
 "@tidbcloud/serverless": {
 "optional": true
 },
 "@types/better-sqlite3": {
 "optional": true
 },
 "@types/pg": {
 "optional": true
 },
 "@types/sql.js": {
 "optional": true
 },
 "@upstash/redis": {
 "optional": true
 },
 "@vercel/postgres": {
 "optional": true
 },
 "@xata.io/client": {
 "optional": true
 },
 "better-sqlite3": {
 "optional": true
 },
 "bun-types": {
 "optional": true
 },
 "expo-sqlite": {
 "optional": true
 },
 "gel": {
 "optional": true
 },
 "knex": {
 "optional": true
 },
 "kysely": {
 "optional": true
 },
 "mysql2": {
```

**package-lock.json**

```
 "optional": true
 },
 "pg": {
 "optional": true
 },
 "postgres": {
 "optional": true
 },
 "prisma": {
 "optional": true
 },
 "sql.js": {
 "optional": true
 },
 "sqlite3": {
 "optional": true
 }
 }
},
"node_modules/electron-to-chromium": {
 "version": "1.5.399",
 "resolved":
"https://registry.npmjs.org/electron-to-chromium/-/electron-to-chromium-1.5.399.tgz",
 "integrity":
"sha512-lEcqhErbHjXRvd41rnWLPzbyU/IXfIYo7QwaFwmXGeLiLyY2TBCdHnWY88vB+p3ubnihRypDm66panXl7TyLLA==",
 "dev": true,
 "license": "ISC"
},
"node_modules/emoji-regex": {
 "version": "8.0.0",
 "resolved": "https://registry.npmjs.org/emoji-regex/-/emoji-regex-8.0.0.tgz",
 "integrity":
"sha512-MSjYzcwNOA0ewAhpz0MxpYFvwg6yjy1NG3xteoqz644VCo/RPgnr1/GGt+ic3iJTzQ8Eu3TdM14SawnVUmGE6A==",
 "dev": true,
 "license": "MIT"
},
"node_modules/error-stack-parser-es": {
 "version": "1.0.5",
 "resolved":
"https://registry.npmjs.org/error-stack-parser-es/-/error-stack-parser-es-1.0.5.tgz",
 "integrity":
"sha512-5qucVt2XcuGMcEGgWI7i+yZpmpByQ8J1lHhCL7PwqCwu9FPP3VUXzT4ltHe5i2z9dePwEHcDV0AfSnHs0lCXRA==",
 "dev": true,
 "license": "MIT",
 "funding": {
 "url": "https://github.com/sponsors/antfu"
 }
},
"node_modules/esbuild": {
 "version": "0.25.12",
 "resolved": "https://registry.npmjs.org/esbuild/-/esbuild-0.25.12.tgz",
 "integrity":
"sha512-bbPBYrtZbkt60s6FiTLCTFvxq4tt3JKall1vRwshA3fdVztsLAatFaZobhkBC8/BrPetoa0oksYoKXoG4ryJg==",
 "dev": true,
```

**package-lock.json**

```
"hasInstallScript": true,
"license": "MIT",
"bin": {
 "esbuild": "bin/esbuild"
},
"engines": {
 "node": ">=18"
},
"optionalDependencies": {
 "@esbuild/aix-ppc64": "0.25.12",
 "@esbuild/android-arm": "0.25.12",
 "@esbuild/android-arm64": "0.25.12",
 "@esbuild/android-x64": "0.25.12",
 "@esbuild/darwin-arm64": "0.25.12",
 "@esbuild/darwin-x64": "0.25.12",
 "@esbuild/freebsd-arm64": "0.25.12",
 "@esbuild/freebsd-x64": "0.25.12",
 "@esbuild/linux-arm": "0.25.12",
 "@esbuild/linux-arm64": "0.25.12",
 "@esbuild/linux-ia32": "0.25.12",
 "@esbuild/linux-loong64": "0.25.12",
 "@esbuild/linux-mips64el": "0.25.12",
 "@esbuild/linux-ppc64": "0.25.12",
 "@esbuild/linux-riscv64": "0.25.12",
 "@esbuild/linux-s390x": "0.25.12",
 "@esbuild/linux-x64": "0.25.12",
 "@esbuild/netbsd-arm64": "0.25.12",
 "@esbuild/netbsd-x64": "0.25.12",
 "@esbuild/openbsd-arm64": "0.25.12",
 "@esbuild/openbsd-x64": "0.25.12",
 "@esbuild/openharmony-arm64": "0.25.12",
 "@esbuild/sunos-x64": "0.25.12",
 "@esbuild/win32-arm64": "0.25.12",
 "@esbuild/win32-ia32": "0.25.12",
 "@esbuild/win32-x64": "0.25.12"
}
},
"node_modules/escalade": {
 "version": "3.2.0",
 "resolved": "https://registry.npmjs.org/escalade/-/escalade-3.2.0.tgz",
 "integrity":
"sha512-WUj2qlxaQt04g6Pq5c29GTcWGdYd8itL8zTlipgECz3JesAii0Kotd8JU6otB3PACgG6xkJUyVhboMS+bje/jA==",
 "dev": true,
 "license": "MIT",
 "engines": {
 "node": ">=6"
 }
},
"node_modules/fast-equals": {
 "version": "5.4.1",
 "resolved": "https://registry.npmjs.org/fast-equals/-/fast-equals-5.4.1.tgz",
 "integrity":
"sha512-DjlFSM5Pk9cGcL0q5QXl66eGzx0N6szNgaswwc5ZphlBohjTVJSnGgI+rJV0g0i65qUoQnDZN4nDqi33udtydQ==",
 "license": "MIT",
```

**package-lock.json**

```
 "engines": {
 "node": ">=6.0.0"
 },
 "node_modules/fdir": {
 "version": "6.5.0",
 "resolved": "https://registry.npmjs.org/fdir/-/fdir-6.5.0.tgz",
 "integrity":
"sha512-tIbYtZbuc0s0BRGqPJkshJUyDL+SDH7dVM8gjy+ERp3WAUjLEFJE+02kanyHtwjW0nwrKYBiwAmM0p4kLJAnXg==",
 "dev": true,
 "license": "MIT",
 "engines": {
 "node": ">=12.0.0"
 },
 "peerDependencies": {
 "picomatch": "^3 || ^4"
 },
 "peerDependenciesMeta": {
 "picomatch": {
 "optional": true
 }
 }
 },
 "node_modules/fsevents": {
 "version": "2.3.3",
 "resolved": "https://registry.npmjs.org/fsevents/-/fsevents-2.3.3.tgz",
 "integrity":
"sha512-5x0DfX+flL7faATnagmWPpbFtwh/R77WmMMqqHGS65C3vvB0YHrgF+B1YmZ3441tMj5n63k0212XNoJwzlhffQw==",
 "dev": true,
 "hasInstallScript": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "darwin"
],
 "engines": {
 "node": "^8.16.0 || ^10.6.0 || >=11.0.0"
 }
 },
 "node_modules/gensync": {
 "version": "1.0.0-beta.2",
 "resolved": "https://registry.npmjs.org/gensync/-/gensync-1.0.0-beta.2.tgz",
 "integrity":
"sha512-3hN7NaskYvMDLQY55gnW3NQ+mesEAepTqlg+VEbj7zzqEMBVNhzcGYReqFo/Tlyz6eQiFcp1HcsCZ0+nGgS8zg==",
 "dev": true,
 "license": "MIT",
 "engines": {
 "node": ">=6.9.0"
 }
 },
 "node_modules/get-caller-file": {
 "version": "2.0.5",
 "resolved": "https://registry.npmjs.org/get-caller-file/-/get-caller-file-2.0.5.tgz",
 "integrity":
```

**package-lock.json**

```
"sha512-DyFP3BM/3YHTQ0CUL/w00ZHR0lpKeGrxotcHwcqNEdnltqFwXVfhEBQ94eIo34AfQpo0rGki4cyIiftY06h2Fg==",
 "dev": true,
 "license": "ISC",
 "engines": {
 "node": "6.* || 8.* || >= 10.*"
 }
},
"node_modules/get-tsconfig": {
 "version": "4.14.1",
 "resolved": "https://registry.npmjs.org/get-tsconfig/-/get-tsconfig-4.14.1.tgz",
 "integrity":
"sha512-Dz/6HxkrxgNehhxLVeyv8sad9UzF2xBVeaKBQNDfJ5XiSXmp2gTR0e00RwiT2NCKS5aGP9jjkOMggTN90qU50A==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "resolve-pkg-maps": "^1.0.0"
 },
 "funding": {
 "url": "https://github.com/privatenumber/get-tsconfig?sponsor=1"
 }
},
"node_modules/has-flag": {
 "version": "4.0.0",
 "resolved": "https://registry.npmjs.org/has-flag/-/has-flag-4.0.0.tgz",
 "integrity":
"sha512-EykJT/Q1KjTwtcpgIAgfsS00tKVuZUjhgMr17kqTumMl6Afv3EISleU7qZUzoXDFTAHTDC4N0oG/ZxU3EvLMPQ==",
 "dev": true,
 "license": "MIT",
 "engines": {
 "node": ">=8"
 }
},
"node_modules/hono": {
 "version": "4.12.34",
 "resolved": "https://registry.npmjs.org/hono/-/hono-4.12.34.tgz",
 "integrity":
"sha512-GqXJqY/xJkJmuloTrnV1ZEXG3fqte+VjkUqoRNZXcrUidiU0P4fMSIHYY4tsqZBK++kVyWmt/AAfSUuy57/eSA==",
 "license": "MIT",
 "engines": {
 "node": ">=16.9.0"
 }
},
"node_modules/is-fullwidth-code-point": {
 "version": "3.0.0",
 "resolved":
"https://registry.npmjs.org/is-fullwidth-code-point/-/is-fullwidth-code-point-3.0.0.tgz",
 "integrity":
"sha512-ozNi+pMw20ouOegDhrEkuh8Gq38v2cf2BEG68Udy17LXqzRkz7ePvDh/16GAYzWRaSH5nLp2W4z8NOr/bW4="
},
"sha512-zymm5+u+sCsSwyD9qNaejV3DFvhCKclKdizYaJUuHA83RLjb7nSuGnddCHGv0hk+KY7BMAlsWeK4Ueg6EV6XQg==",
 "dev": true,
 "license": "MIT",
 "engines": {
 "node": ">=8"
 }
},
```



**package-lock.json**

```
"node_modules/js-tokens": {
 "version": "4.0.0",
 "resolved": "https://registry.npmjs.org/js-tokens/-/js-tokens-4.0.0.tgz",
 "integrity":
"sha512-RdJUflcE3cUzKiMqQgsCu06FPu9UdIJ00beYbPhHN4k6apgJtifcoCtT9bcx0pYBtpD2kCM6Sbzg4CausW/PKQ==",
 "dev": true,
 "license": "MIT"
},
"node_modules/jstest": {
 "version": "3.1.0",
 "resolved": "https://registry.npmjs.org/jstest/-/jstest-3.1.0.tgz",
 "integrity":
"sha512-sM3d02F0zXjKQhJuo0Q173wf2K0o8t4I8vHy6lF9poUp7bKT0/NHE8fPX23PwfhnkyfqnC2xRx0nVw5XuGIaA==",
 "dev": true,
 "license": "MIT",
 "bin": {
 "jstest": "bin/jstest"
 },
 "engines": {
 "node": ">=6"
 }
},
"node_modules/json5": {
 "version": "2.2.3",
 "resolved": "https://registry.npmjs.org/json5/-/json5-2.2.3.tgz",
 "integrity":
"sha512-XmOWe7eyHYH14cLdVPoyg+G0H3rYX++KpzrylJwSW98t3Nk+U8X0l8FWK0gwtzdb8lXGf6zYwDUzeHMFxasyg==",
 "dev": true,
 "license": "MIT",
 "bin": {
 "json5": "lib/cli.js"
 },
 "engines": {
 "node": ">=6"
 }
},
"node_modules/kleur": {
 "version": "4.1.5",
 "resolved": "https://registry.npmjs.org/kleur/-/kleur-4.1.5.tgz",
 "integrity":
"sha512-o+NO+8WrRiQEE4/7nwrJhN1HwpVmJm511pBHUXPLtp0BUIszlBplORYSmTclCnJvQq2tKu/sgl3xVpkc7ZWuQQ==",
 "dev": true,
 "license": "MIT",
 "engines": {
 "node": ">=6"
 }
},
"node_modules/linkifyjs": {
 "version": "4.3.3",
 "resolved": "https://registry.npmjs.org/linkifyjs/-/linkifyjs-4.3.3.tgz",
 "integrity":
"sha512-P8aEP5U/D1/ILTY20eYsErdwh9bGuLE30NcXtKEjgdHcahveQoQwM2yZnsioQHsWFz0P7KKudisbrzCgR0sDHg==",
 "license": "MIT"
},
```

**package-lock.json**

```
"node_modules/lru-cache": {
 "version": "5.1.1",
 "resolved": "https://registry.npmjs.org/lru-cache/-/lru-cache-5.1.1.tgz",
 "integrity":
"sha512-KpNARQA3Iwv+jTA0utUVVbrh+Jlrr1Fv0e56GGzAF0XN7dk/FviaDW8LHmK52Dlch4WP2n6gI8vN1aesBFgo9w==",
 "dev": true,
 "license": "ISC",
 "dependencies": {
 "yallist": "^3.0.2"
 }
},
"node_modules/lucide": {
 "version": "1.28.0",
 "resolved": "https://registry.npmjs.org/lucide/-/lucide-1.28.0.tgz",
 "integrity":
"sha512-sxVxWEXfq7rRNzmffDBRohYB0Fjfo0hr07jGGCLNNRoY2fPjeuqi2boXR2UAR0EyQUm+xe13DG0QNtnmwv6Y3w==",
 "license": "ISC"
},
"node_modules/miniflare": {
 "version": "5.20260730.0-alpha",
 "resolved": "https://registry.npmjs.org/miniflare/-/miniflare-5.20260730.0-alpha.tgz",
 "integrity":
"sha512-8/dspSXDshP6nSkCpjK07BYc2qZoYSXm7iM+QxY7qJyJpAB3onnQSaiu0cvKJlfuMGwULl55hG69FJCcCMXU1Q==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@cspotcode/source-map-support": "0.8.1",
 "sharp": "0.35.2",
 "undici": "7.28.0",
 "workerd": "1.20260730.1",
 "ws": "8.21.0",
 "youch": "4.1.0-beta.10"
 },
 "engines": {
 "node": ">=22.0.0"
 }
},
"node_modules/morphicons": {
 "version": "1.4.0",
 "resolved": "https://registry.npmjs.org/morphicons/-/morphicons-1.4.0.tgz",
 "integrity":
"sha512-Yerf8QnogXcyaW3rtm6IEnrDgeh5gFmk/md5dNdyoqjDMbCmgI9UTW00lMccKcjwAHZFNULhZCRolWqB01BLzw==",
 "license": "MIT",
 "peerDependencies": {
 "react": ">=18",
 "react-native": ">=0.71",
 "react-native-svg": ">=14",
 "svelte": ">=5",
 "vue": ">=3.3"
 },
 "peerDependenciesMeta": {
 "react": {
 "optional": true
 }
 }
},
```

**package-lock.json**

```
 "react-native": {
 "optional": true
 },
 "react-native-svg": {
 "optional": true
 },
 "svelte": {
 "optional": true
 },
 "vue": {
 "optional": true
 }
 },
 "node_modules/ms": {
 "version": "2.1.3",
 "resolved": "https://registry.npmjs.org/ms/-/ms-2.1.3.tgz",
 "integrity": "sha512-6Fb1g93bA9E0b9o2vR0gHGO42XWz+VWQ5eRwblp569fYrZvQ0ZlRtWJgvcPb6pOZ3jQysYYzVj0kHuL9A==",
 "dev": true,
 "license": "MIT"
 },
 "node_modules/nanoid": {
 "version": "3.3.17",
 "resolved": "https://registry.npmjs.org/nanoid/-/nanoid-3.3.17.tgz",
 "integrity": "sha512-6p4bY1dHVx1I1AcpVvZ/1SgC6MteHfQ5sKYdXVWf8D1pZi1h3c3obVh3K/2xt83U8hhI6/pUa66V+cxBD==",
 "dev": true,
 "funding": [
 {
 "type": "github",
 "url": "https://github.com/sponsors/ai"
 }
],
 "license": "MIT",
 "bin": {
 "nanoid": "bin/nanoid.cjs"
 },
 "engines": {
 "node": "^10 || ^12 || ^13.7 || ^14 || >=15.0.1"
 }
 },
 "node_modules/node-releases": {
 "version": "2.0.51",
 "resolved": "https://registry.npmjs.org/node-releases/-/node-releases-2.0.51.tgz",
 "integrity": "sha512-cuLcYUK4A298l3uN4VzQ1L1VhbW063uN1uZmZvF0k2/q3aZb+5ZGzI2JwKwHhztmPzVwRyUwW1Tg8oE==",
 "dev": true,
 "license": "MIT",
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/orderedmap": {
```

## package-lock.json

```

"version": "2.1.1",
"resolved": "https://registry.npmjs.org/orderedmap/-/orderedmap-2.1.1.tgz",
"integrity":
"sha512-TvAWxi0nDe1j/rtMcWcIj94+Ffe6n7zhow33h40SKxmsmozs6dz/e+EajymfoFcHd7sxNn8yHM8839uixMOV6g==",
"license": "MIT"
},
"node_modules/pako": {
"version": "1.0.11",
"resolved": "https://registry.npmjs.org/pako/-/pako-1.0.11.tgz",
"integrity":
"sha512-4hLB8Py4zZce5s4yd9XzopqwVv/yGNhV1Bl8NTmCq1763HeK2+EwVTv+leGeL13Dnh2wfbqowVPXCI00z4taYw==",
"license": "(MIT AND Zlib)"
},
"node_modules/path-to-regexp": {
"version": "6.3.0",
"resolved": "https://registry.npmjs.org/path-to-regexp/-/path-to-regexp-6.3.0.tgz",
"integrity":
"sha512-YQ05YbAPGmI013tjw7GTru6wYx/bo51w8GjPvDzPkwf88js0NDEI5pG2u8wQBpcknE8R0PwSvK2L1azFw==",
"dev": true,
"license": "MIT"
},
"node_modules/pathe": {
"version": "2.0.3",
"resolved": "https://registry.npmjs.org/pathe/-/pathe-2.0.3.tgz",
"integrity":
"sha512-WUjGcAqP1gQacoQe+0BJSFA7Ld4DyXuUIjZ5cc75cLHVJ7dtNsTugphxIADwspS+AraAUePCKrSVtPLFj/F88w==",
"dev": true,
"license": "MIT"
},
"node_modules/pdf-lib": {
"version": "1.17.1",
"resolved": "https://registry.npmjs.org/pdf-lib/-/pdf-lib-1.17.1.tgz",
"integrity":
"sha512-V/mpyJAoTsN4cnP31vc0wfNA1+p20evqqnap0KLoRUN0Yk/p3wN52D0EsL4oBFcLdb76hlpKPtzJIgo67j/XLw==",
"license": "MIT",
"dependencies": {
"@pdf-lib/standard-fonts": "^1.0.0",
"@pdf-lib/upng": "^1.0.1",
"pako": "^1.0.11",
"tslib": "^1.11.1"
}
},
"node_modules/pdf-lib/node_modules/tslib": {
"version": "1.14.1",
"resolved": "https://registry.npmjs.org/tslib/-/tslib-1.14.1.tgz",
"integrity":
"sha512-Xni35NKzjgMrwevysHTCArtLDpPvyee8zV/0E4EyYn43P7/7qvQwPh9BGkHewbMulVntbigmCT7rdX3BNo9wRJg==",
"license": "0BSD"
},
"node_modules/picocolors": {
"version": "1.1.1",
"resolved": "https://registry.npmjs.org/picocolors/-/picocolors-1.1.1.tgz",
"integrity":
"sha512-pfT14X0luMc69KsXGO9WdrfmqjWIKTIjckNYTt7IrfobqeDFbQD8A7FuoWz1iybAzmNxfH1r9Bg8YgWwwA==",

```

**package-lock.json**

```

 "dev": true,
 "license": "ISC"
 },
 "node_modules/picomatch": {
 "version": "4.0.5",
 "resolved": "https://registry.npmjs.org/picomatch/-/picomatch-4.0.5.tgz",
 "integrity":
"sha512-RvNWcruNjI1ncT5xRaKeyS9Lf8lcItv34KD+aif+VH9kduAyfYBipGh12274xtenIPZ119/R9BdTba8gAwSh0A==",
 "dev": true,
 "license": "MIT",
 "engines": {
 "node": ">=12"
 },
 "funding": {
 "url": "https://github.com/sponsors/jonschlinkert"
 }
 },
 "node_modules/postcss": {
 "version": "8.5.25",
 "resolved": "https://registry.npmjs.org/postcss/-/postcss-8.5.25.tgz",
 "integrity":
"sha512-DTPx3RWSSnWyzLxQnlH0rJP+Ew5ekl16ZU4/psbIhA0e53kJfdgaN5vKM+xP7yJtXVu+nfdVFmLgFDEKAe4Pyw==",
 "dev": true,
 "funding": [
 {
 "type": "opencollective",
 "url": "https://opencollective.com/postcss/"
 },
 {
 "type": "tidelift",
 "url": "https://tidelift.com/funding/github/npm/postcss"
 },
 {
 "type": "github",
 "url": "https://github.com/sponsors/ai"
 }
],
 "license": "MIT",
 "dependencies": {
 "nanoid": "^3.3.16",
 "picocolors": "^1.1.1",
 "source-map-js": "^1.2.1"
 },
 "engines": {
 "node": ">=10 || ^12 || >=14"
 }
 },
 "node_modules/prosemirror-changeset": {
 "version": "2.4.1",
 "resolved":
"https://registry.npmjs.org/prosemirror-changeset/-/prosemirror-changeset-2.4.1.tgz",
 "integrity":
"sha512-96WBLh0aYhJ+kPhLg3uW359Tz6I/MfcrQfL4EGv4SrcqKEMC1gmoGrXHecPE8e0wTVcJ4IwgfzM8fFad25wNfw==",
 "license": "MIT",

```

**package-lock.json**

```
 "dependencies": {
 "prosemirror-transform": "^1.0.0"
 },
 "node_modules/prosemirror-commands": {
 "version": "1.7.1",
 "resolved":
"https://registry.npmjs.org/prosemirror-commands/-/prosemirror-commands-1.7.1.tgz",
 "integrity":
"sha512-rT7qZnQt5c0/y/KlYaGvtG411S97UaL6gdp6RIZ23DLHanMYLyfGBV5DtSnZdthQql7W+lEVbpSfwT08T+L2w==",
 "license": "MIT",
 "dependencies": {
 "prosemirror-model": "^1.0.0",
 "prosemirror-state": "^1.0.0",
 "prosemirror-transform": "^1.10.2"
 }
 },
 "node_modules/prosemirror-dropcursor": {
 "version": "1.8.3",
 "resolved":
"https://registry.npmjs.org/prosemirror-dropcursor/-/prosemirror-dropcursor-1.8.3.tgz",
 "integrity":
"sha512-FoYbsJR8gK+DGLqhNoE29Loa38eIZPzQRIb1VMaDNBoo40LP6vVof/jR8qFY/6XvUd6Dhug8MDCHl2a/h8RTfQ==",
 "license": "MIT",
 "dependencies": {
 "prosemirror-state": "^1.0.0",
 "prosemirror-transform": "^1.1.0",
 "prosemirror-view": "^1.1.0"
 }
 },
 "node_modules/prosemirror-gapcursor": {
 "version": "1.4.1",
 "resolved":
"https://registry.npmjs.org/prosemirror-gapcursor/-/prosemirror-gapcursor-1.4.1.tgz",
 "integrity":
"sha512-pMDYaeEnjNMSwl11yjEGtgTmLkR08m/Vl+Jj443167p9eB3HVQKhYCC4gmHVDsLP0DfZfjr/MmirsdyZziXbQKw==",
 "license": "MIT",
 "dependencies": {
 "prosemirror-keymap": "^1.0.0",
 "prosemirror-model": "^1.0.0",
 "prosemirror-state": "^1.0.0",
 "prosemirror-view": "^1.0.0"
 }
 },
 "node_modules/prosemirror-history": {
 "version": "1.5.0",
 "resolved":
"https://registry.npmjs.org/prosemirror-history/-/prosemirror-history-1.5.0.tgz",
 "integrity":
"sha512-zlzTiH01eKA55UAF1MEjtssJeHnGx00j4K4Dpx+gnmX9n+SHNlDqI2o01Kv1iPN5B1dm5fsljCfqKF9nFL6HRg==",
 "license": "MIT",
 "dependencies": {
 "prosemirror-state": "^1.2.2",
 "prosemirror-transform": "^1.0.0",
```

**package-lock.json**

```
 "prosemirror-view": "^1.31.0",
 "rope-sequence": "^1.3.0"
 },
 },
 "node_modules/prosemirror-inputrules": {
 "version": "1.5.1",
 "resolved":
"https://registry.npmjs.org/prosemirror-inputrules/-/prosemirror-inputrules-1.5.1.tgz",
 "integrity":
"sha512-7wj4uMjKaXWAQ1CDgxNzNtR9AlsuzwHfdFH1ygEHA2KHF2D0EaXl1CJfNPAKCG9qNEh4rum975QLaCiQPyY6Fw==",
 "license": "MIT",
 "dependencies": {
 "prosemirror-state": "^1.0.0",
 "prosemirror-transform": "^1.0.0"
 }
 },
 },
 "node_modules/prosemirror-keymap": {
 "version": "1.2.3",
 "resolved": "https://registry.npmjs.org/prosemirror-keymap/-/prosemirror-keymap-1.2.3.tgz",
 "integrity":
"sha512-4HucRlpiLd1IPQQXNqeo81BGtkY8Ai5smHhKW9jjPKRc2wQIxksG7Hl1tTI2IfT2B/LgX6bfYvXxEPJl7aKYKw==",
 "license": "MIT",
 "dependencies": {
 "prosemirror-state": "^1.0.0",
 "w3c-keyname": "^2.2.0"
 }
 },
 },
 "node_modules/prosemirror-model": {
 "version": "1.25.11",
 "resolved": "https://registry.npmjs.org/prosemirror-model/-/prosemirror-model-1.25.11.tgz",
 "integrity":
"sha512-QWg9RhnpllogAmp3p96uEFrE5txQpFynd4vhBAELkwgOCwQs/X0yCzB3/hrHqiPwf91RG5KyWq6553zs9JqIQ==",
 "license": "MIT",
 "dependencies": {
 "orderedmap": "^2.0.0"
 }
 },
 },
 "node_modules/prosemirror-schema-list": {
 "version": "1.5.1",
 "resolved":
"https://registry.npmjs.org/prosemirror-schema-list/-/prosemirror-schema-list-1.5.1.tgz",
 "integrity":
"sha512-927lFx/uwyQaGwJxLWCZRkjXG0p48KpMj6ueoYiu4JX05GGuGcgzAy62dfiV8eFZftgyBUvLx76RsMe20fJl+Q==",
 "license": "MIT",
 "dependencies": {
 "prosemirror-model": "^1.0.0",
 "prosemirror-state": "^1.0.0",
 "prosemirror-transform": "^1.7.3"
 }
 },
 },
 "node_modules/prosemirror-state": {
 "version": "1.4.4",
 "resolved": "https://registry.npmjs.org/prosemirror-state/-/prosemirror-state-1.4.4.tgz",
 "integrity":
```

**package-lock.json**

```
"sha512-6jiYHH2CIGbCfnxdHbXZ12gySFY/fz/ulZE333G6bPqIZ4F+TXo9ifiR86nAHpWnfoNj0b3o5ESi7J8Uz1jXHw==",
 "license": "MIT",
 "dependencies": {
 "prosemirror-model": "^1.0.0",
 "prosemirror-transform": "^1.0.0",
 "prosemirror-view": "^1.27.0"
 }
},
"node_modules/prosemirror-tables": {
 "version": "1.8.5",
 "resolved": "https://registry.npmjs.org/prosemirror-tables/-/prosemirror-tables-1.8.5.tgz",
 "integrity":
"sha512-V/0cDCsHKHe/tfWkeCmthNUcEp1IV03p6vwN8XtwE9PZQLAZJigbw3QoraAdfJPir4NKJtNV0B8oYGKRL+t0Dw==",
 "license": "MIT",
 "dependencies": {
 "prosemirror-keymap": "^1.2.3",
 "prosemirror-model": "^1.25.4",
 "prosemirror-state": "^1.4.4",
 "prosemirror-transform": "^1.10.5",
 "prosemirror-view": "^1.41.4"
 }
},
"node_modules/prosemirror-transform": {
 "version": "1.12.0",
 "resolved":
"https://registry.npmjs.org/prosemirror-transform/-/prosemirror-transform-1.12.0.tgz",
 "integrity":
"sha512-GxboyN4AMIsoHNtz5uf2r2Ru551i5hweCMD6E2Ib4Eogqoub0NflniaBPVQ4MrGE5yZ8JV9tUHg9qcZTTrcN4w==",
 "license": "MIT",
 "dependencies": {
 "prosemirror-model": "^1.21.0"
 }
},
"node_modules/prosemirror-view": {
 "version": "1.42.2",
 "resolved": "https://registry.npmjs.org/prosemirror-view/-/prosemirror-view-1.42.2.tgz",
 "integrity":
"sha512-Pdg0l5kXm8aLDquFANQFTCITg0q44sLqBlHlpsVLD9segd0ao8T0fQdAhCrCXYVgPSRr6UDDR00IWA3bIrN9YQ==",
 "license": "MIT",
 "dependencies": {
 "prosemirror-model": "^1.25.8",
 "prosemirror-state": "^1.0.0",
 "prosemirror-transform": "^1.1.0"
 }
},
"node_modules/react": {
 "version": "19.2.8",
 "resolved": "https://registry.npmjs.org/react/-/react-19.2.8.tgz",
 "integrity":
"sha512-PWaya1L/q9u2u7xYQi+Y3L3Yfnie7XyLeaJICV1MGD6LprsBxcAqGjYyr0eY3p+QdsA+x/Irkt4Qif8D63+Sbw==",
 "license": "MIT",
 "engines": {
 "node": ">=0.10.0"
 }
}
```



**package-lock.json**

```
 },
 "node_modules/react-dom": {
 "version": "19.2.8",
 "resolved": "https://registry.npmjs.org/react-dom/-/react-dom-19.2.8.tgz",
 "integrity":
"sha512-rVprimfGBG3DR+Tq0IQG2DT5PxKth1WIGDmj5yPmlzr4YBe7uyE+Du4oVqTDXZSHGGGXRTTJEGSSePyQCMBglQ==",
 "license": "MIT",
 "dependencies": {
 "scheduler": "^0.27.0"
 },
 "peerDependencies": {
 "react": "^19.2.8"
 }
 },
 "node_modules/react-refresh": {
 "version": "0.18.0",
 "resolved": "https://registry.npmjs.org/react-refresh/-/react-refresh-0.18.0.tgz",
 "integrity":
"sha512-QgtT5//D3jffjJb6Gsjsxv0Slpj23ip+HtOpnNgnb2S5zU3CB26G/IDPGoy4RJB42wzFE46DRsstbW6tKHoKbhAxw==",
 "dev": true,
 "license": "MIT",
 "engines": {
 "node": ">=0.10.0"
 }
 },
 "node_modules/react-router": {
 "version": "7.18.2",
 "resolved": "https://registry.npmjs.org/react-router/-/react-router-7.18.2.tgz",
 "integrity":
"sha512-aUVMjFm3GAPTTZL7oYr5E7ETiqfQCHRLH+B+5afnICvf0r7kkK4eR6SMuwbSTJw/7t+12khT/Kahij49fqOCIg==",
 "license": "MIT",
 "dependencies": {
 "cookie": "^1.0.1",
 "set-cookie-parser": "^2.6.0"
 },
 "engines": {
 "node": ">=20.0.0"
 },
 "peerDependencies": {
 "react": ">=18",
 "react-dom": ">=18"
 },
 "peerDependenciesMeta": {
 "react-dom": {
 "optional": true
 }
 }
 },
 "node_modules/react-router-dom": {
 "version": "7.18.2",
 "resolved": "https://registry.npmjs.org/react-router-dom/-/react-router-dom-7.18.2.tgz",
 "integrity":
"sha512-AIKJ/jgGlFb3EbfcXk5Gzshiwt+l3mqbCrNjmEWMmjQxNJ3svBa6bgzFyCC2Sw3RA0VWF1kg3uQf20Fhxb8hw==",
 "license": "MIT",

```

**package-lock.json**

```
 "dependencies": {
 "react-router": "7.18.2"
 },
 "engines": {
 "node": ">=20.0.0"
 },
 "peerDependencies": {
 "react": ">=18",
 "react-dom": ">=18"
 }
},
"node_modules/require-directory": {
 "version": "2.1.1",
 "resolved": "https://registry.npmjs.org/require-directory/-/require-directory-2.1.1.tgz",
 "integrity":
"sha512-fGxIEI7+wsG9xrvdjsrImL22OMTTiHRwAMroiEeMgq8gzolC/PQr7RsRDSTLUg/bZAZtF+TVIkHc6/4RIKrui+Q==",
 "dev": true,
 "license": "MIT",
 "engines": {
 "node": ">=0.10.0"
 }
},
"node_modules/resolve-pkg-maps": {
 "version": "1.0.0",
 "resolved": "https://registry.npmjs.org/resolve-pkg-maps/-/resolve-pkg-maps-1.0.0.tgz",
 "integrity":
"sha512-seS2Tj26TBVOC2NIc2r0e2y2Z07efxITtLZcGS0nHHNQ7CkiUBfw0Iw2ck6xkIhPwLhKNLS8B0+hEpngQlqzw==",
 "dev": true,
 "license": "MIT",
 "funding": {
 "url": "https://github.com/privatenumber/resolve-pkg-maps?sponsor=1"
 }
},
"node_modules/rollup": {
 "version": "4.62.4",
 "resolved": "https://registry.npmjs.org/rollup/-/rollup-4.62.4.tgz",
 "integrity":
"sha512-RX0QwaPsBGjMNMma4sQjDjHieHEZDFoj/Rdr46l2MU5DfEs16wHJPC2RPTPHwNl+M3aI472LLqFkFKut4Sbl0g==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@types/estree": "1.0.9"
 },
 "bin": {
 "rollup": "dist/bin/rollup"
 },
 "engines": {
 "node": ">=18.0.0",
 "npm": ">=8.0.0"
 }
},
"optionalDependencies": {
 "@napi-rs/lzma-linux-x64-gnu": "1.5.1",
 "@rollup/rollup-android-arm-eabi": "4.62.4",
 "@rollup/rollup-android-arm64": "4.62.4",
```

**package-lock.json**

```

 "@rollup/rollup-darwin-arm64": "4.62.4",
 "@rollup/rollup-darwin-x64": "4.62.4",
 "@rollup/rollup-freebsd-arm64": "4.62.4",
 "@rollup/rollup-freebsd-x64": "4.62.4",
 "@rollup/rollup-linux-arm-gnueabihf": "4.62.4",
 "@rollup/rollup-linux-arm-musleabihf": "4.62.4",
 "@rollup/rollup-linux-arm64-gnu": "4.62.4",
 "@rollup/rollup-linux-arm64-musl": "4.62.4",
 "@rollup/rollup-linux-loong64-gnu": "4.62.4",
 "@rollup/rollup-linux-loong64-musl": "4.62.4",
 "@rollup/rollup-linux-ppc64-gnu": "4.62.4",
 "@rollup/rollup-linux-ppc64-musl": "4.62.4",
 "@rollup/rollup-linux-riscv64-gnu": "4.62.4",
 "@rollup/rollup-linux-riscv64-musl": "4.62.4",
 "@rollup/rollup-linux-s390x-gnu": "4.62.4",
 "@rollup/rollup-linux-x64-gnu": "4.62.4",
 "@rollup/rollup-linux-x64-musl": "4.62.4",
 "@rollup/rollup-openbsd-x64": "4.62.4",
 "@rollup/rollup-openharmony-arm64": "4.62.4",
 "@rollup/rollup-win32-arm64-msvc": "4.62.4",
 "@rollup/rollup-win32-ia32-msvc": "4.62.4",
 "@rollup/rollup-win32-x64-gnu": "4.62.4",
 "@rollup/rollup-win32-x64-msvc": "4.62.4",
 "fsevents": "~2.3.2"
 }
},
"node_modules/rope-sequence": {
 "version": "1.3.4",
 "resolved": "https://registry.npmjs.org/rope-sequence/-/rope-sequence-1.3.4.tgz",
 "integrity":
"sha512-UT5EDe2cu2E/604igUr5PSFs23nvvukicWx6Gn0PLHAiiYbzNuCRQCuiUdHJQcQKaLLKlrYJnjY0ySGsXNQXQ==",
 "license": "MIT"
},
"node_modules/rxjs": {
 "version": "7.8.2",
 "resolved": "https://registry.npmjs.org/rxjs/-/rxjs-7.8.2.tgz",
 "integrity":
"sha512-dhkF903U/PQZY6boNNTAGdWbG85WAbJT/1xYoZIC7FAY0yWap0BQVsVrDl58W86//e1VpMNBtRV4MaXfdMySFA==",
 "dev": true,
 "license": "Apache-2.0",
 "dependencies": {
 "tslib": "^2.1.0"
 }
},
"node_modules/scheduler": {
 "version": "0.27.0",
 "resolved": "https://registry.npmjs.org/scheduler/-/scheduler-0.27.0.tgz",
 "integrity":
"sha512-eNv+wrVbKu1f3vbYJT/xtiF5syA5HPIMtf9IgY/nKg0sWqzAUEvqY/xm70cZc/qaFLx/i09Fg0meSAp4v5ti/Q==",
 "license": "MIT"
},
"node_modules/semver": {
 "version": "6.3.1",
 "resolved": "https://registry.npmjs.org/semver/-/semver-6.3.1.tgz",

```

**package-lock.json**

```

 "integrity":
"sha512-BR7VvDCVH0+q2xBEWskxS6DJE1qRnb7DxzUrogb71CWoSficBxYsiAGd+Kl0mmq/MprG9yArRkyrQxT06XjMzA==",
 "dev": true,
 "license": "ISC",
 "bin": {
 "semver": "bin/semver.js"
 }
 },
 "node_modules/set-cookie-parser": {
 "version": "2.7.2",
 "resolved": "https://registry.npmjs.org/set-cookie-parser/-/set-cookie-parser-2.7.2.tgz",
 "integrity":
"sha512-oeM1lpU/UvhTxw+g3cIfxXHjJRc/uidd3yK1P242gzHds0udQBYzs3y8j4gCCW+ZJ7ad0yctld8RY0+bdurLvw==",
 "license": "MIT"
 },
 "node_modules/sharp": {
 "version": "0.35.2",
 "resolved": "https://registry.npmjs.org/sharp/-/sharp-0.35.2.tgz",
 "integrity":
"sha512-FVtFjtBCMiJS6yB5CX7Sop45WFMpeGw6oRKuJnXYgf/f1ms/D7LE/ZUSNxnW7rZ/dbslQWYkoqFHGPADBtaK4w==",
 "dev": true,
 "license": "Apache-2.0",
 "dependencies": {
 "@img/colour": "^1.1.0",
 "detect-libc": "^2.1.2",
 "semver": "^7.8.4"
 },
 "engines": {
 "node": ">=20.9.0"
 },
 "funding": {
 "url": "https://opencollective.com/libvips"
 },
 "optionalDependencies": {
 "@img/sharp-darwin-arm64": "0.35.2",
 "@img/sharp-darwin-x64": "0.35.2",
 "@img/sharp-freebsd-wasm32": "0.35.2",
 "@img/sharp-libvips-darwin-arm64": "1.3.1",
 "@img/sharp-libvips-darwin-x64": "1.3.1",
 "@img/sharp-libvips-linux-arm": "1.3.1",
 "@img/sharp-libvips-linux-arm64": "1.3.1",
 "@img/sharp-libvips-linux-ppc64": "1.3.1",
 "@img/sharp-libvips-linux-riscv64": "1.3.1",
 "@img/sharp-libvips-linux-s390x": "1.3.1",
 "@img/sharp-libvips-linux-x64": "1.3.1",
 "@img/sharp-libvips-linuxmusl-arm64": "1.3.1",
 "@img/sharp-libvips-linuxmusl-x64": "1.3.1",
 "@img/sharp-linux-arm": "0.35.2",
 "@img/sharp-linux-arm64": "0.35.2",
 "@img/sharp-linux-ppc64": "0.35.2",
 "@img/sharp-linux-riscv64": "0.35.2",
 "@img/sharp-linux-s390x": "0.35.2",
 "@img/sharp-linux-x64": "0.35.2",
 "@img/sharp-linuxmusl-arm64": "0.35.2",

```

**package-lock.json**

```
 "@img/sharp-linuxmusl-x64": "0.35.2",
 "@img/sharp-webcontainers-wasm32": "0.35.2",
 "@img/sharp-win32-arm64": "0.35.2",
 "@img/sharp-win32-ia32": "0.35.2",
 "@img/sharp-win32-x64": "0.35.2"
 }
},
"node_modules/sharp/node_modules/semver": {
 "version": "7.8.5",
 "resolved": "https://registry.npmjs.org/semver/-/semver-7.8.5.tgz",
 "integrity":
"sha512-Y7/KDsb8LjooZpwaqGyulO6DQlksgCncchHGk+sZIY4SBvUocMBEFH5Ur1fI4dV+Jvl0w6cjvucaIi40puRioA==",
 "dev": true,
 "license": "ISC",
 "bin": {
 "semver": "bin/semver.js"
 },
 "engines": {
 "node": ">=10"
 }
},
"node_modules/shell-quote": {
 "version": "1.9.0",
 "resolved": "https://registry.npmjs.org/shell-quote/-/shell-quote-1.9.0.tgz",
 "integrity":
"sha512-Iov+JwFv/2HcTpcwNMkd8+IWNb8tboQJNQTKAY/LLVK7gGH9jy+LGkVqPxfekHl+yMmiqXszdGWXgkfml7hjqA==",
 "dev": true,
 "license": "MIT",
 "engines": {
 "node": ">= 0.4"
 },
 "funding": {
 "url": "https://github.com/sponsors/ljharb"
 }
},
"node_modules/source-map": {
 "version": "0.6.1",
 "resolved": "https://registry.npmjs.org/source-map/-/source-map-0.6.1.tgz",
 "integrity":
"sha512-UjgapumWlbMhkBgzt7Ykc5YXUT46F0iKu8SGXq0bcwP5dz/h0Plj6enJqjz1Zbq2l5WaqYnrVbwW0WMyF3F47g==",
 "dev": true,
 "license": "BSD-3-Clause",
 "engines": {
 "node": ">=0.10.0"
 }
},
"node_modules/source-map-js": {
 "version": "1.2.1",
 "resolved": "https://registry.npmjs.org/source-map-js/-/source-map-js-1.2.1.tgz",
 "integrity":
"sha512-UjgapumWlbMhkBgzt7Ykc5YXUT46F0iKu8SGXq0bcwP5dz/h0Plj6enJqjz1Zbq2l5WaqYnrVbwW0WMyF3F47g==",
 "dev": true,
 "license": "BSD-3-Clause",
 "engines": {
```

**package-lock.json**

```
 "node": ">=0.10.0"
 },
 "node_modules/source-map-support": {
 "version": "0.5.21",
 "resolved": "https://registry.npmjs.org/source-map-support/-/source-map-support-0.5.21.tgz",
 "integrity": "sha512-uBHU3L3czsIyYXKX88fdrGovxdSCoTGDRZ6SYXtSRxLZUzHg5P/66Ht6uoUlHu9EZod+inXhKo3qQgwXUT/y1w==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "buffer-from": "^1.0.0",
 "source-map": "^0.6.0"
 }
 },
 "node_modules/string-width": {
 "version": "4.2.3",
 "resolved": "https://registry.npmjs.org/string-width/-/string-width-4.2.3.tgz",
 "integrity": "sha512-wKyQRPjJ0sIp62ErSzdGsjMJWsap5oRNihHhu6G7JV0/9jIB6UyevL+txu0qrng8j/cxKTWYUwvStriiZz/g==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "emoji-regex": "^8.0.0",
 "is-fullwidth-code-point": "^3.0.0",
 "strip-ansi": "^6.0.1"
 },
 "engines": {
 "node": ">=8"
 }
 },
 "node_modules/strip-ansi": {
 "version": "6.0.1",
 "resolved": "https://registry.npmjs.org/strip-ansi/-/strip-ansi-6.0.1.tgz",
 "integrity": "sha512-Y38VPSHcqkFrCpFnQ9vuSXmquuv5oX0KpGeT6aGrr3o3Gc9AlVa6JBfUSOCnbxGGZF+/0ooI7KrPuUSztUdU5A==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "ansi-regex": "^5.0.1"
 },
 "engines": {
 "node": ">=8"
 }
 },
 "node_modules/supports-color": {
 "version": "8.1.1",
 "resolved": "https://registry.npmjs.org/supports-color/-/supports-color-8.1.1.tgz",
 "integrity": "sha512-iU7gBNKl984mXVutFwi1w0MwVt36P91XV5T38pM17aeag3J6e+GCiHs8GVd/lVpCUUx8ufh0p29DMg==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "has-flag": "^4.0.0"
 }
 }
}
```

**package-lock.json**

```
 },
 "engines": {
 "node": ">=10"
 },
 "funding": {
 "url": "https://github.com/chalk/supports-color?sponsor=1"
 }
 },
 "node_modules/tinyglobby": {
 "version": "0.2.17",
 "resolved": "https://registry.npmjs.org/tinyglobby/-/tinyglobby-0.2.17.tgz",
 "integrity": "sha512-wXRdYpcqKmfWpEdZjiKJOWCNFndD0DMnrW/cYjVGttEkBfVgcLFHoNrlj47mj0Vic9yyNu65alsgF4NQyTa2g==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "fdir": "^6.5.0",
 "picomatch": "^4.0.4"
 },
 "engines": {
 "node": ">=12.0.0"
 },
 "funding": {
 "url": "https://github.com/sponsors/SuperchupuDev"
 }
 },
 "node_modules/tree-kill": {
 "version": "1.2.2",
 "resolved": "https://registry.npmjs.org/tree-kill/-/tree-kill-1.2.2.tgz",
 "integrity": "sha512-L00Rpi8qGpRG//Nd+H90vFB+3iHnue1zSSGmN00Ch1GLJ7rUKVwV2HvijphGQS2UmhUZewS9VgvxYIdgr+fG1A==",
 "dev": true,
 "license": "MIT",
 "bin": {
 "tree-kill": "cli.js"
 }
 },
 "node_modules/tslib": {
 "version": "2.8.1",
 "resolved": "https://registry.npmjs.org/tslib/-/tslib-2.8.1.tgz",
 "integrity": "sha512-9bYqoXj28WYKJ9N0pW9j06Nq66H7S1q2sU4Qw9rkeG0UWzQeIDM0BVC4lV1Q41DzKq/m3LWz0bU1Wt0B==",
 "dev": true,
 "license": "0BSD"
 },
 "node_modules/tsx": {
 "version": "4.23.5",
 "resolved": "https://registry.npmjs.org/tsx/-/tsx-4.23.5.tgz",
 "integrity": "sha512-rw55FUAq0oI7RvLQwLbh04nSDApnQ4/CykPuiQ/EPvtrX3WA9Ig55jIt9VvbBJbzJuj12ueRu4PMZ2SxPVbihg==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "esbuild": "~0.28.0"
 }
 }
}
```

**package-lock.json**

```
 },
 "bin": {
 "tsx": "dist/cli.mjs"
 },
 "engines": {
 "node": ">=18.0.0"
 },
 "optionalDependencies": {
 "fsevents": "~2.3.3"
 }
 },
 "node_modules/tsx/node_modules/@esbuild/aix-ppc64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/aix-ppc64/-/aix-ppc64-0.28.1.tgz",
 "integrity":
"sha512-Svl7tq8k/08+p6CXPpRjQ1fKX+1odH/BQbb48fV6fj3CWHsoIOoY87w1oHXm0qEpKIK3ZfVgp0hed3XBXzXMq==",
 "cpu": [
 "ppc64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "aix"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/tsx/node_modules/@esbuild/android-arm": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/android-arm/-/android-arm-0.28.1.tgz",
 "integrity":
"sha512-0k2F129Xdio1TdJfzJ8sy1Q47vUD2NnwdhiAf7drUN1EBTfPf4hsFCtmMgu/6m8JSzsBr lmvJudMBQq0fG8usQ==",
 "cpu": [
 "arm"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "android"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/tsx/node_modules/@esbuild/android-arm64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/android-arm64/-/android-arm64-0.28.1.tgz",
 "integrity":
"sha512-34EGEbCIAgosYz6goLcopX6Mo7NyGv9tfwEM2/7Ce2VcVRk568iSvniGwCUXIy7wEDR1wzolcxcriFVrWYcwBg==",
 "cpu": [
 "arm64"
]
 }
}
```



**package-lock.json**

```
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "android"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/tsx/node_modules/@esbuild/android-x64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/android-x64/-/android-x64-0.28.1.tgz",
 "integrity":
"sha512-dbwY7ltSMDwsRatcRpCnES4F+im880CUgGZjy52shC7GqHRE/cYlXNbB4Z4UpJswpcc4Qxd2oE/ufM0p61IKng==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "android"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/tsx/node_modules/@esbuild/darwin-arm64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/darwin-arm64/-/darwin-arm64-0.28.1.tgz",
 "integrity":
"sha512-TZbWkQY7kvTAXbXUT7uVACR5cMHsDiSz9z7ZKAX/RTq/WJEk3QyRr0wZpNhBDX+/0CtdqUIJl0iodQcta6tY3Q==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "darwin"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/tsx/node_modules/@esbuild/darwin-x64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/darwin-x64/-/darwin-x64-0.28.1.tgz",
 "integrity":
"sha512-zfdzgK9ACBNZLI/CyHT0x81SyNbM6YXn7rxSgX97VjyiPl9W1i4Ka4fgKECEoFCKGpvBj5qArWIGgQjOwkgsKQ==",
 "cpu": [
 "x64"
],
```

**package-lock.json**

```
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "darwin"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/tsx/node_modules/@esbuild/freebsd-arm64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/freebsd-arm64/-/freebsd-arm64-0.28.1.tgz",
 "integrity": "sha512-wG2EA8ENdEI0qhKSZMjfqrDY+ziCYCPMmtZjjIw0mXFjmyzEHn+UUXk5of+SYsjtfs3VpnLC7QLzSI5hY/r0Aw==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "freebsd"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/tsx/node_modules/@esbuild/freebsd-x64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/freebsd-x64/-/freebsd-x64-0.28.1.tgz",
 "integrity": "sha512-i7dZ9vQgnvSCzi/rYCXNgtF/U+eKZNBzu3eTQbRgHnM7tNSizLOkRfAl3qzVc/Op/u5YkHhA4pf/3D0YHthLQ==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "freebsd"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/tsx/node_modules/@esbuild/linux-arm": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-arm/-/linux-arm-0.28.1.tgz",
 "integrity": "sha512-qVXB0HQs+d5Y722GwJzJUtoLlX7km3Cra0aGormF1pDtPd2C/l1SHRPgjLunLGe51Sh5YYWKMFDyV4SxgMQYTQ==",
 "cpu": [
 "arm"
]
 }
}
```

**package-lock.json**

```
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/tsx/node_modules/@esbuild/linux-arm64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-arm64/-/linux-arm64-0.28.1.tgz",
 "integrity":
"sha512-yHs+0uc8+nVEAfAfXrWQKK5peSNzBc4PegcM00EJ2hT71uA7vB8Ihg2e77R2P7SG5uYjPbHlLLmve4LLLRCf0g==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/tsx/node_modules/@esbuild/linux-ia32": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-ia32/-/linux-ia32-0.28.1.tgz",
 "integrity":
"sha512-d1z4ZuP0ajrfz/FhGT4vv278rX8KnPPJx8i5+AtK7TYbx9Le9F1hyzurZpkEyjkGa9dUGhQow4C1NmeGvqxN2w==",
 "cpu": [
 "ia32"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/tsx/node_modules/@esbuild/linux-loong64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-loong64/-/linux-loong64-0.28.1.tgz",
 "integrity":
"sha512-M5sRjUVZrkm10APR3dl0YzNmN+loZKGV11VUQGrwuqLcbR6qeAz+famMhjASeH3YVKvZz+zT1jLh/keC3Rj/Lg==",
 "cpu": [
 "loong64"
]
 }
}
```

**package-lock.json**

```
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/tsx/node_modules/@esbuild/linux-mips64el": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-mips64el/-/linux-mips64el-0.28.1.tgz",
 "integrity": "sha512-mRObBZeHh20xcBFPWE/FjylkRgZdYuiTR3vaTozquCG0H14iP9oN4x4Ge81CoIDYQrXmIxpFumJBu5MtZpnQJQ==",
 "cpu": [
 "mips64el"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/tsx/node_modules/@esbuild/linux-ppc64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-ppc64/-/linux-ppc64-0.28.1.tgz",
 "integrity": "sha512-slScBsMAB3GFDcdrCgLwZtPYRoH2H/youv10QiZyRjmsP48fznoveWytSgCI/R0ZcUgpc0ZhIUEx6LHts8yrfQ==",
 "cpu": [
 "ppc64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/tsx/node_modules/@esbuild/linux-riscv64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-riscv64/-/linux-riscv64-0.28.1.tgz",
 "integrity": "sha512-kw0owk1o0GFETUJyW0jc0G4Yzs0BHZn0JDZ8JRT088vjJYX777BAs1fDGxAC+q831q0s2DTC96mNsG2opdfyyQ==",
 "cpu": [
```

**package-lock.json**

```
 "riscv64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/tsx/node_modules/@esbuild/linux-s390x": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-s390x/-/linux-s390x-0.28.1.tgz",
 "integrity":
"sha512-/lAIjX8aYFRByhh6L5rYtPEDRqa9de/4V/ju0Xcta5frjvzX04/sqEtyytse0g3zZFuwu5cDN0MkLz2qRDD2Ag==",
 "cpu": [
 "s390x"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/tsx/node_modules/@esbuild/linux-x64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-x64/-/linux-x64-0.28.1.tgz",
 "integrity":
"sha512-u/anNYF2mmVOEDwLtnQ1wOr3EZ9sTNGLWrSYGYwHWzGA3Si84IOkHXlbWTD1NB+9/1lcuweYK054uhxZydNzfA==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/tsx/node_modules/@esbuild/netbsd-arm64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/netbsd-arm64/-/netbsd-arm64-0.28.1.tgz",
 "integrity":
"sha512-oks0DYbLwWmmaakTsCb+zL4E+aHRVLom9IJZ0AthMQEPiQmydXHkziYEsGYRx0uNV/IjEKGAV941JzH02pflqw==",
 "cpu": [
```

**package-lock.json**

```
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "netbsd"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/tsx/node_modules/@esbuild/netbsd-x64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/netbsd-x64/-/netbsd-x64-0.28.1.tgz",
 "integrity":
"sha512-aEL6lAN89Hz43Mlh1G8ARasbuoYvSITDEx0tHh5b7jJnHcssqgjy9Yx430GDpmCa60yrKoS0aNRjKundRizGg==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "netbsd"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/tsx/node_modules/@esbuild/openbsd-arm64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/openbsd-arm64/-/openbsd-arm64-0.28.1.tgz",
 "integrity":
"sha512-MEFJe5C3R8pwXdZ5Y21oo6m7ePiS0d9pwucn990/wvyJZChoIQKrQDxKrGew8F5+T0okTHesAmDeiHDTIq0V/Q==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "openbsd"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/tsx/node_modules/@esbuild/openbsd-x64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/openbsd-x64/-/openbsd-x64-0.28.1.tgz",
 "integrity":
"sha512-i/ZLI0afE0Z8cI/XANJAixoJL/uRAoS2x0A3rb0xN+KK0K177cMASQYkzHtBrMXAKuAc7HGgcWiZ/sRC1Nxgw==",
 "cpu": [
```

**package-lock.json**

```
"x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "openbsd"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/tsx/node_modules/@esbuild/openharmony-arm64": {
 "version": "0.28.1",
 "resolved":
"https://registry.npmjs.org/@esbuild/openharmony-arm64/-/openharmony-arm64-0.28.1.tgz",
 "integrity":
"sha512-ge+Z7EXFnt2B01oAMsVpiQ8EwndV9i1xXerAeTIK7AtPs3bKFXQM7n1RxDSIUIMeueR1CNXxqztLzdNeReKBJg==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "openharmony"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/tsx/node_modules/@esbuild/sunos-x64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/sunos-x64/-/sunos-x64-0.28.1.tgz",
 "integrity":
"sha512-BEjgtECKL3vY+SaSQ6nzVfiALUeFxpawyp8Jmf5PtYhf1Ug40N1h/hx1hts+f1FvSvarEigdxS3BlSMI2PJLcQ==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "sunos"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/tsx/node_modules/@esbuild/win32-arm64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/win32-arm64/-/win32-arm64-0.28.1.tgz",
 "integrity":
"sha512-lCv9eK/H6ZJWbE7bh2nw54CZ9M2nupBxJcTsdk/QQnWkdSjKGuxmmH8/GwrLT1eMmZfn4dGcCjRte397WqfQXA==",
```

**package-lock.json**

```
"cpu": [
 "arm64"
],
"dev": true,
"license": "MIT",
"optional": true,
"os": [
 "win32"
],
"engines": {
 "node": ">=18"
}
},
"node_modules/tsx/node_modules/@esbuild/win32-ia32": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/win32-ia32/-/win32-ia32-0.28.1.tgz",
 "integrity":
"sha512-zvb/mB2bSCoJOpocBgYKKpX6YM6mJBlBUVUtVj41DlZJVEB6/0CKlRYxP5wWl1C1ILiCoAU5wZZ4q1P3qeS6Eg==",
 "cpu": [
 "ia32"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "win32"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/tsx/node_modules/@esbuild/win32-x64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/win32-x64/-/win32-x64-0.28.1.tgz",
 "integrity":
"sha512-bm4Mowrv+GXMlpWX++EcXw/iLyd1o3+bJkC2DkWXyVvgZCqD/bSj9ctZeAMC3cIxgjRVR2Dufaiu4YPxr5gW1A==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "win32"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/tsx/node_modules/esbuild": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/esbuild/-/esbuild-0.28.1.tgz",
 "integrity":
"sha512-HrJrvZv5ayxBzPfwpH0oNzkz0IILifzk0KJrGK2c8R4+LKpMtpYLQeUdjnwjWv/LZlKH2laZk+4w78pi99D4Vw==",
```



**package-lock.json**

```
"dev": true,
"hasInstallScript": true,
"license": "MIT",
"bin": {
 "esbuild": "bin/esbuild"
},
"engines": {
 "node": ">=18"
},
"optionalDependencies": {
 "@esbuild/aix-ppc64": "0.28.1",
 "@esbuild/android-arm": "0.28.1",
 "@esbuild/android-arm64": "0.28.1",
 "@esbuild/android-x64": "0.28.1",
 "@esbuild/darwin-arm64": "0.28.1",
 "@esbuild/darwin-x64": "0.28.1",
 "@esbuild/freebsd-arm64": "0.28.1",
 "@esbuild/freebsd-x64": "0.28.1",
 "@esbuild/linux-arm": "0.28.1",
 "@esbuild/linux-arm64": "0.28.1",
 "@esbuild/linux-ia32": "0.28.1",
 "@esbuild/linux-loong64": "0.28.1",
 "@esbuild/linux-mips64el": "0.28.1",
 "@esbuild/linux-ppc64": "0.28.1",
 "@esbuild/linux-riscv64": "0.28.1",
 "@esbuild/linux-s390x": "0.28.1",
 "@esbuild/linux-x64": "0.28.1",
 "@esbuild/netbsd-arm64": "0.28.1",
 "@esbuild/netbsd-x64": "0.28.1",
 "@esbuild/openbsd-arm64": "0.28.1",
 "@esbuild/openbsd-x64": "0.28.1",
 "@esbuild/openharmony-arm64": "0.28.1",
 "@esbuild/sunos-x64": "0.28.1",
 "@esbuild/win32-arm64": "0.28.1",
 "@esbuild/win32-ia32": "0.28.1",
 "@esbuild/win32-x64": "0.28.1"
}
},
"node_modules/typescript": {
 "version": "5.9.3",
 "resolved": "https://registry.npmjs.org/typescript/-/typescript-5.9.3.tgz",
 "integrity":
"sha512-jl1vZzPDinLr9eUt3J/t7V6FgNEw9QjvBPdysz9KfQDD41fQrC2Y4vKQdiaUpFT4bXlb1RHhLpp8wtm6M5TgSw==",
 "dev": true,
 "license": "Apache-2.0",
 "bin": {
 "tsc": "bin/tsc",
 "tsserver": "bin/tsserver"
 },
 "engines": {
 "node": ">=14.17"
 }
},
"node_modules/undici": {
```

**package-lock.json**

```
"version": "7.28.0",
"resolved": "https://registry.npmjs.org/undici/-/undici-7.28.0.tgz",
"integrity":
"sha512-cRZYrTDWznlnRiPjggAGxZXanty6M8RV1ff8Wm4LWXBp7/IG8v5Dn0m74DtUBp900NpK75YlPnIjQqX0dBDtA==",
"dev": true,
"license": "MIT",
"engines": {
 "node": ">=20.18.1"
}
},
"node_modules/unenv": {
 "version": "2.0.0-rc.24",
 "resolved": "https://registry.npmjs.org/unenv/-/unenv-2.0.0-rc.24.tgz",
 "integrity":
"sha512-i7qRCmY42zmCwnYlh9H2SvLEypEFGye5iRmEMKjcGi7zk9UquigRjFtTLz0TYqr0ZGLZhaMHL/foy1bZR+CwLw==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "pathe": "^2.0.3"
 }
},
"node_modules/update-browserslist-db": {
 "version": "1.2.3",
 "resolved":
"https://registry.npmjs.org/update-browserslist-db/-/update-browserslist-db-1.2.3.tgz",
 "integrity":
"sha512-Js0m9cx+q0gDxo0eMiFGEueWztz+d4+M3rGlmKPT+T4IS/jP4ylw3Nwpu6cpTTP8R1MAC1kF4VbdLt3ARf209w==",
 "dev": true,
 "funding": [
 {
 "type": "opencollective",
 "url": "https://opencollective.com/browserslist"
 },
 {
 "type": "tidelift",
 "url": "https://tidelift.com/funding/github/npm/browserslist"
 },
 {
 "type": "github",
 "url": "https://github.com/sponsors/ai"
 }
],
 "license": "MIT",
 "dependencies": {
 "escalade": "^3.2.0",
 "picocolors": "^1.1.1"
 },
 "bin": {
 "update-browserslist-db": "cli.js"
 },
 "peerDependencies": {
 "browserslist": ">= 4.21.0"
 }
},
```

**package-lock.json**

```
"node_modules/use-sync-external-store": {
 "version": "1.6.0",
 "resolved":
 "https://registry.npmjs.org/use-sync-external-store/-/use-sync-external-store-1.6.0.tgz",
 "integrity":
 "sha512-Pp6GSWGP/NrPIrxVFAIk0Qeyw8lFen0HijQWkUTrDvrF4ALqylP2C/KCkeS9dpUM3KvYRQhna5vt7IL95+ZQ9w==",
 "license": "MIT",
 "peerDependencies": {
 "react": "^16.8.0 || ^17.0.0 || ^18.0.0 || ^19.0.0"
 }
},
"node_modules/vite": {
 "version": "7.3.6",
 "resolved": "https://registry.npmjs.org/vite/-/vite-7.3.6.tgz",
 "integrity":
 "sha512-4XP60spRGjSZFf1qYH+dJIkk2znL3zQfL9Kk0V9MkkRR/3Dls0dxaBsQPTloEc5BLXWPL9vs0xopxyKoMmDueg==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "esbuild": "^0.27.0 || ^0.28.0",
 "fdir": "^6.5.0",
 "picomatch": "^4.0.3",
 "postcss": "^8.5.6",
 "rollup": "^4.43.0",
 "tinyglobby": "^0.2.15"
 },
 "bin": {
 "vite": "bin/vite.js"
 },
 "engines": {
 "node": "^20.19.0 || >=22.12.0"
 },
 "funding": {
 "url": "https://github.com/vitejs/vite?sponsor=1"
 },
 "optionalDependencies": {
 "fsevents": "~2.3.3"
 },
 "peerDependencies": {
 "@types/node": "^20.19.0 || >=22.12.0",
 "jiti": ">=1.21.0",
 "less": "^4.0.0",
 "lightningcss": "^1.21.0",
 "sass": "^1.70.0",
 "sass-embedded": "^1.70.0",
 "stylus": ">=0.54.8",
 "sugarss": "^5.0.0",
 "terser": "^5.16.0",
 "tsx": "^4.8.1",
 "yaml": "^2.4.2"
 },
 "peerDependenciesMeta": {
 "@types/node": {
 "optional": true
 }
 }
}
```

**package-lock.json**

```
 },
 "jiti": {
 "optional": true
 },
 "less": {
 "optional": true
 },
 "lightningcss": {
 "optional": true
 },
 "sass": {
 "optional": true
 },
 "sass-embedded": {
 "optional": true
 },
 "stylus": {
 "optional": true
 },
 "sugarss": {
 "optional": true
 },
 "terser": {
 "optional": true
 },
 "tsx": {
 "optional": true
 },
 "yaml": {
 "optional": true
 }
 }
},
"node_modules/vite/node_modules/@esbuild/aix-ppc64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/aix-ppc64/-/aix-ppc64-0.28.1.tgz",
 "integrity":
"sha512-Svl7tq8k/08+p6CXPpRjQ1fKX+1odH/BQbb48fV6fj3CWHsoIOoY87w1oHXm0qEpKIK3ZfVgp0hed3XBzXMq==",
 "cpu": [
 "ppc64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "aix"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/vite/node_modules/@esbuild/android-arm": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/android-arm/-/android-arm-0.28.1.tgz",
```

**package-lock.json**

```
"integrity":
"sha512-0k2F129Xdio1TdJfzJ8sy1Q47vUD2NnwdhiAf7drUN1EBTfPf4hsFCtmMgu/6m8JSzsBr lmvjudMBQqOfG8usQ==",
 "cpu": [
 "arm"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "android"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/vite/node_modules/@esbuild/android-arm64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/android-arm64/-/android-arm64-0.28.1.tgz",
 "integrity":
"sha512-34EEbCIAgosYz6goLcopX6Mo7NyGv9tfwEM2/7Ce2VcVRk568iSvniGwcUXIy7wEDR1wzolcxcrlFVrWYcwBg==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "android"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/vite/node_modules/@esbuild/android-x64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/android-x64/-/android-x64-0.28.1.tgz",
 "integrity":
"sha512-dbwY7ltSMDwsRatcRpCnES4F+im880CUgGZjy52shC7GqHRE/cYlxNbB4Z4UpJswpcc4Qxd2oE/ufM0p61IKng==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "android"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/vite/node_modules/@esbuild/darwin-arm64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/darwin-arm64/-/darwin-arm64-0.28.1.tgz",
```

**package-lock.json**

```
"integrity":
"sha512-TZbWkQY7kvTAXbXUT7uVACR5cMHsDiSz9z7ZKAX/RTq/WJEk3QyRr0wZpNhBDX+/0CtdqUIJl0iodQcta6tY3Q==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "darwin"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/vite/node_modules/@esbuild/darwin-x64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/darwin-x64/-/darwin-x64-0.28.1.tgz",
 "integrity":
"sha512-zfdzgK9ACBNZLI/CyHT0x81SyNbM6YXn7rxSgX97VjyiPl9W1i4Ka4fgKECEoFCKGpvBj5qArWIGgQjOwkgsKQ==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "darwin"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/vite/node_modules/@esbuild/freebsd-arm64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/freebsd-arm64/-/freebsd-arm64-0.28.1.tgz",
 "integrity":
"sha512-wG2EA8ENdEI0qhKSZMjfqrDY+ziCYCPMmtZjjIw0mXFjmyzEHn+UUXk5of+SYsjtfs3VpnlC7QLzSI5hY/r0Aw==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "freebsd"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/vite/node_modules/@esbuild/freebsd-x64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/freebsd-x64/-/freebsd-x64-0.28.1.tgz",
```

**package-lock.json**

```
"integrity":
"sha512-i7dZ9vQgnvSCzi/rYCXNgtF/U+eKZNBzu3eTQbRgHnM7tNSizLOkRFAl3qzVc/Op/u5YkHhA4pf/3D0YHthLQ==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "freebsd"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/vite/node_modules/@esbuild/linux-arm": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-arm/-/linux-arm-0.28.1.tgz",
 "integrity":
"sha512-qVXB0HQs+d5Y722GwJzJUtoLlX7km3Cra0aGormF1pDtPd2C/l1SHRPgjLunLGe51Sh5YYWKMFDyV4SxgMQYTQ==",
 "cpu": [
 "arm"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/vite/node_modules/@esbuild/linux-arm64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-arm64/-/linux-arm64-0.28.1.tgz",
 "integrity":
"sha512-yHs+0uc8+nVEAfAfxrWQKK5peSNzBc4PegcM00EJ2ht71uA7vB8Ihg2e77R2P7SG5uYjPbHlLLmve4LLLRCf0g==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/vite/node_modules/@esbuild/linux-ia32": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-ia32/-/linux-ia32-0.28.1.tgz",
```

**package-lock.json**

```
"integrity":
"sha512-d1z4ZuP0ajrfz/FhGT4vv278rX8KnPPJx8i5+AtK7TYbx9Le9F1hyzurZpkEyjkGa9dUGhQow4C1NmeGvqxN2w==",
 "cpu": [
 "ia32"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/vite/node_modules/@esbuild/linux-loong64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-loong64/-/linux-loong64-0.28.1.tgz",
 "integrity":
"sha512-M5sRjUVZrkm10APR3dl0YzNmN+loZKGV11VUQGrwuqLcbR6qeAz+famMhjASeH3YVKvZz+zT1jLh/keC3Rj/lg==",
 "cpu": [
 "loong64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/vite/node_modules/@esbuild/linux-mips64el": {
 "version": "0.28.1",
 "resolved":
"https://registry.npmjs.org/@esbuild/linux-mips64el/-/linux-mips64el-0.28.1.tgz",
 "integrity":
"sha512-mRObBZeHh20xcBFPWE/FjylkRgZdYuiTR3vaTozquCGOH14iP9oN4x4Ge81CoIDYQrXmIxpFumJBu5MtZpnQJQ==",
 "cpu": [
 "mips64el"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/vite/node_modules/@esbuild/linux-ppc64": {
 "version": "0.28.1",
```



**package-lock.json**

```
"resolved": "https://registry.npmjs.org/@esbuild/linux-ppc64/-/linux-ppc64-0.28.1.tgz",
"integrity":
"sha512-slScBsMAB3GFDcdrCgLwZtPYRoH2H/youv10QiZyRjmsP48fznoveWytSgCI/R0ZcUgpc0ZhIUEx6LHts8yrfQ==",
"cpu": [
 "ppc64"
],
"dev": true,
"license": "MIT",
"optional": true,
"os": [
 "linux"
],
"engines": {
 "node": ">=18"
}
},
"node_modules/vite/node_modules/@esbuild/linux-riscv64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-riscv64/-/linux-riscv64-0.28.1.tgz",
 "integrity":
"sha512-kw0owk1o0GFETUJyW0jc0G4Yzs0BHZn0JDZ8JRT088vjJYX777BAS1fDGxAC+q831q0s2DTC96mNsG2opdfyyQ==",
 "cpu": [
 "riscv64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/vite/node_modules/@esbuild/linux-s390x": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-s390x/-/linux-s390x-0.28.1.tgz",
 "integrity":
"sha512-/lAIjX8aYFRByhh6L5rYtPEDRqa9de/4V/ju0Xcta5frjvzX04/sqEtyytse0g3zZFuwu5cDN0MkLz2qRDD2Ag==",
 "cpu": [
 "s390x"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/vite/node_modules/@esbuild/linux-x64": {
 "version": "0.28.1",
```

**package-lock.json**

```
"resolved": "https://registry.npmjs.org/@esbuild/linux-x64/-/linux-x64-0.28.1.tgz",
"integrity":
"sha512-u/anNYF2mmVOEDwLtnQ1wOr3EZ9sTNGLWrsYGYWHWzGA3Si84IOkHXlbWTD1NB+9/1lcwYK054uhxZydNzfA==",
"cpu": [
 "x64"
],
"dev": true,
"license": "MIT",
"optional": true,
"os": [
 "linux"
],
"engines": {
 "node": ">=18"
}
},
"node_modules/vite/node_modules/@esbuild/netbsd-arm64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/netbsd-arm64/-/netbsd-arm64-0.28.1.tgz",
 "integrity":
"sha512-oks0DYbLWMMaakTsCb+zL4E+aHRVLom9IJZ0AthMQEPiQmydXHkziYEsGYRx0uNV/IjEKGAV941JzH02pflqw==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "netbsd"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/vite/node_modules/@esbuild/netbsd-x64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/netbsd-x64/-/netbsd-x64-0.28.1.tgz",
 "integrity":
"sha512-aEL6lAN89Hz43Mlh1G8ARasbuoYvSITDEX0tHh5b7jJnHcssqgjy9Yx430GDpmCa60yrKoS0aNRjKundRizGg==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "netbsd"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/vite/node_modules/@esbuild/openbsd-arm64": {
 "version": "0.28.1",
```

**package-lock.json**

```
"resolved": "https://registry.npmjs.org/@esbuild/openbsd-arm64/-/openbsd-arm64-0.28.1.tgz",
"integrity":
"sha512-MEFJe5C3R8pWXdZ5Y21oo6m7ePiS0d9pWucn990/wvyJZChoIQKrQDxKrGeW8F5+T0okTHesAmDeiHDTIq0V/Q==",
"cpu": [
 "arm64"
],
"dev": true,
"license": "MIT",
"optional": true,
"os": [
 "openbsd"
],
"engines": {
 "node": ">=18"
}
},
"node_modules/vite/node_modules/@esbuild/openbsd-x64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/openbsd-x64/-/openbsd-x64-0.28.1.tgz",
 "integrity":
"sha512-i/ZLI0afE0Z8cI/XANJAiXoJL/uRAoS2x0A3rb0xN+KK0K177cMASQYkzHtBrMXAKuAc7HGgcWiZ/sRC1Nxgw==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "openbsd"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/vite/node_modules/@esbuild/openharmony-arm64": {
 "version": "0.28.1",
 "resolved":
"https://registry.npmjs.org/@esbuild/openharmony-arm64/-/openharmony-arm64-0.28.1.tgz",
 "integrity":
"sha512-ge+Z7EXFnt2B01oAMsVpiQ8EwndV9i1xXerAeTIK7AtPs3bKFXQM7n1RxDSIUIMeueR1CNXxqztLzdNeReKBJg==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "openharmony"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/vite/node_modules/@esbuild/sunos-x64": {
```

**package-lock.json**

```
"version": "0.28.1",
"resolved": "https://registry.npmjs.org/@esbuild/sunos-x64/-/sunos-x64-0.28.1.tgz",
"integrity":
"sha512-BEjgtEckL3vY+SaSQ6nzVfiALUeFxpawyp8Jmf5PtYhf1Ug40N1h/hx1hts+f1FvSvarEigdxS3BlSMI2PJLcQ==",
"cpu": [
 "x64"
],
"dev": true,
"license": "MIT",
"optional": true,
"os": [
 "sunos"
],
"engines": {
 "node": ">=18"
}
},
"node_modules/vite/node_modules/@esbuild/win32-arm64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/win32-arm64/-/win32-arm64-0.28.1.tgz",
 "integrity":
"sha512-lCv9eK/H6ZJWbE7bh2nw54CZ9M2nupBxJcTsdk/QQnWkdSjKGuxmmH8/GWrLT1eMmZfn4dGcCjRte397WqfQXA==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "win32"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/vite/node_modules/@esbuild/win32-ia32": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/win32-ia32/-/win32-ia32-0.28.1.tgz",
 "integrity":
"sha512-zvb/mB2bSCoJOpoCBgYKKpX6YM6mJB1BUVUtVj41DlZJVEB6/0CKlRYxP5wWl1C1ILiCoAU5wZZ4q1P3qeS6Eg==",
 "cpu": [
 "ia32"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "win32"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/vite/node_modules/@esbuild/win32-x64": {
```

**package-lock.json**

```
"version": "0.28.1",
"resolved": "https://registry.npmjs.org/@esbuild/win32-x64/-/win32-x64-0.28.1.tgz",
"integrity":
"sha512-bm4Mowrv+GXMlpwX++EcXw/iLyd1o3+bJKC2DkWXVvgZCqD/bSj9ctZeAMC3cIxcgRVR2Dufaiu4YPxr5gW1A==",
"cpu": [
 "x64"
],
"dev": true,
"license": "MIT",
"optional": true,
"os": [
 "win32"
],
"engines": {
 "node": ">=18"
}
},
"node_modules/vite/node_modules/esbuild": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/esbuild/-/esbuild-0.28.1.tgz",
 "integrity":
"sha512-HrJrvZv5ayxBzPfwph0oNzKz0Iilifzk0KJrGK2c8R4+LKpMtpYLQeUdjnwjWv/LZlkH2laZk+4w78pi99D4Vw==",
 "dev": true,
 "hasInstallScript": true,
 "license": "MIT",
 "bin": {
 "esbuild": "bin/esbuild"
 },
 "engines": {
 "node": ">=18"
 },
 "optionalDependencies": {
 "@esbuild/aix-ppc64": "0.28.1",
 "@esbuild/android-arm": "0.28.1",
 "@esbuild/android-arm64": "0.28.1",
 "@esbuild/android-x64": "0.28.1",
 "@esbuild/darwin-arm64": "0.28.1",
 "@esbuild/darwin-x64": "0.28.1",
 "@esbuild/freebsd-arm64": "0.28.1",
 "@esbuild/freebsd-x64": "0.28.1",
 "@esbuild/linux-arm": "0.28.1",
 "@esbuild/linux-arm64": "0.28.1",
 "@esbuild/linux-ia32": "0.28.1",
 "@esbuild/linux-loong64": "0.28.1",
 "@esbuild/linux-mips64el": "0.28.1",
 "@esbuild/linux-ppc64": "0.28.1",
 "@esbuild/linux-riscv64": "0.28.1",
 "@esbuild/linux-s390x": "0.28.1",
 "@esbuild/linux-x64": "0.28.1",
 "@esbuild/netbsd-arm64": "0.28.1",
 "@esbuild/netbsd-x64": "0.28.1",
 "@esbuild/openbsd-arm64": "0.28.1",
 "@esbuild/openbsd-x64": "0.28.1",
 "@esbuild/openharmony-arm64": "0.28.1",
```

**package-lock.json**

```
 "@esbuild/sunos-x64": "0.28.1",
 "@esbuild/win32-arm64": "0.28.1",
 "@esbuild/win32-ia32": "0.28.1",
 "@esbuild/win32-x64": "0.28.1"
 }
},
"node_modules/w3c-keyname": {
 "version": "2.2.8",
 "resolved": "https://registry.npmjs.org/w3c-keyname/-/w3c-keyname-2.2.8.tgz",
 "integrity":
"sha512-dpojBhNsCNN7T82Tm7k26A6G9ML3NkhDsnw9n/eoxSRlVBB4CEtIQ/KTCLi2Fwf3ataSXRhYFkQi3SlnFwPvPQ==",
 "license": "MIT"
},
"node_modules/workerd": {
 "version": "1.20260730.1",
 "resolved": "https://registry.npmjs.org/workerd/-/workerd-1.20260730.1.tgz",
 "integrity":
"sha512-zmfNIjwYSWFY5chGB0jWtH3xAE7p97FTC6vR4Ep98290ho6AeAR/NVcBD274YCLEUYzqm8yxdtZlxMybU8a3jA==",
 "dev": true,
 "hasInstallScript": true,
 "license": "Apache-2.0",
 "bin": {
 "workerd": "bin/workerd"
 },
 "engines": {
 "node": ">=16"
 },
 "optionalDependencies": {
 "@cloudflare/workerd-darwin-64": "1.20260730.1",
 "@cloudflare/workerd-darwin-arm64": "1.20260730.1",
 "@cloudflare/workerd-linux-64": "1.20260730.1",
 "@cloudflare/workerd-linux-arm64": "1.20260730.1",
 "@cloudflare/workerd-windows-64": "1.20260730.1"
 }
},
"node_modules/wrangler": {
 "version": "4.118.0",
 "resolved": "https://registry.npmjs.org/wrangler/-/wrangler-4.118.0.tgz",
 "integrity":
"sha512-9pkBw/b8zWqGx2S+oLhgHMR1M/4V0E8SynUFABnGWiSFGlc0Q4xiI/B71Xf66RYP2xzngU37IQFptUruij3lYw==",
 "dev": true,
 "license": "MIT OR Apache-2.0",
 "dependencies": {
 "@cloudflare/kv-asset-handler": "0.5.0",
 "@cloudflare/unenv-preset": "2.16.1",
 "blake3-wasm": "2.1.5",
 "esbuild": "0.28.1",
 "miniflare": "5.20260730.0-alpha",
 "path-to-regexp": "6.3.0",
 "unenv": "2.0.0-rc.24",
 "workerd": "1.20260730.1"
 },
 "bin": {
 "cf-wrangler": "bin/cf-wrangler.js",

```

**package-lock.json**

```

 "wrangler": "bin/wrangler.js",
 "wrangler2": "bin/wrangler.js"
 },
 "engines": {
 "node": ">=22.0.0"
 },
 "optionalDependencies": {
 "fsevents": "2.3.3"
 },
 "peerDependencies": {
 "@cloudflare/workers-types": "^5.20260730.1"
 },
 "peerDependenciesMeta": {
 "@cloudflare/workers-types": {
 "optional": true
 }
 }
},
"node_modules/wrangler/node_modules/@esbuild/aix-ppc64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/aix-ppc64/-/aix-ppc64-0.28.1.tgz",
 "integrity":
"sha512-Svl7tq8k/08+p6CXPpRjQ1fKX+1odH/BQbb48fV6fj3CWHhsoIOoY87w1oHXm0qEpkIK3ZfVgp0hed3XBXzXMQ==",
 "cpu": [
 "ppc64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "aix"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/wrangler/node_modules/@esbuild/android-arm": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/android-arm/-/android-arm-0.28.1.tgz",
 "integrity":
"sha512-0k2F129Xdio1TdJfzJ8sy1Q47vUD2NnwdhiAf7drUN1EBTfPf4hsFCtmMgu/6m8JSzsBr l mVjudMBQq0fG8usQ==",
 "cpu": [
 "arm"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "android"
],
 "engines": {
 "node": ">=18"
 }
},

```

**package-lock.json**

```
"node_modules/wrangler/node_modules/@esbuild/android-arm64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/android-arm64/-/android-arm64-0.28.1.tgz",
 "integrity":
"sha512-34EGEbCIAgosYz6goLcopX6Mo7NyGv9tfwEM2/7Ce2VcVRk568iSvniGwcUXIy7wEDR1wzolcxcricFVrWYcwBg==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "android"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/wrangler/node_modules/@esbuild/android-x64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/android-x64/-/android-x64-0.28.1.tgz",
 "integrity":
"sha512-dbwY7ltSMDWsRatcRpCnES4F+im880CUgGZjy52shC7GqHRE/cYlxNbB4Z4UpJswpcc4Qxd2oE/ufM0p61IKng==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "android"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/wrangler/node_modules/@esbuild/darwin-arm64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/darwin-arm64/-/darwin-arm64-0.28.1.tgz",
 "integrity":
"sha512-TZbwkQY7kvTAXbXUT7uVACR5cMHsDiSz9z7ZKAX/RTq/WJEk3QyRr0wZpNhBDX+/0CtdqUIJl0iodQcta6tY3Q==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "darwin"
],
 "engines": {
 "node": ">=18"
 }
},
```



**package-lock.json**

```
"node_modules/wrangler/node_modules/@esbuild/darwin-x64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/darwin-x64/-/darwin-x64-0.28.1.tgz",
 "integrity":
"sha512-zfdzgK9ACBNZLI/CyHT0x81SyNbM6YXn7rxSgX97VjyiPl9W1i4Ka4fgKECEoFCKGpvBj5qArWIGgQjOwkgsKQ==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "darwin"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/wrangler/node_modules/@esbuild/freebsd-arm64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/freebsd-arm64/-/freebsd-arm64-0.28.1.tgz",
 "integrity":
"sha512-wG2EA8ENdEI0qhKsZMjfqrDY+ziCYCPMmtZjjIw0mXFjmyzEHn+UUXk5of+SYsjtfs3VpnlC7QLzSI5hY/r0Aw==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "freebsd"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/wrangler/node_modules/@esbuild/freebsd-x64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/freebsd-x64/-/freebsd-x64-0.28.1.tgz",
 "integrity":
"sha512-i7dZ9vQgnvSCzi/rYCXNgtF/U+eKZNJBzu3eTQbRgHnM7tNSizL0kRFAl3qzVc/Op/u5YkHhA4pf/3D0YHthLQ==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "freebsd"
],
 "engines": {
 "node": ">=18"
 }
},
}
```

**package-lock.json**

```
"node_modules/wrangler/node_modules/@esbuild/linux-arm": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-arm/-/linux-arm-0.28.1.tgz",
 "integrity":
"sha512-qVXB0HQs+d5Y722GwJzJUtoLlX7km3Cra0aGormF1pDtPd2C/l1SHRPgjLunLGe51Sh5YYWKMFDyV4SxgMQYTQ==",
 "cpu": [
 "arm"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/wrangler/node_modules/@esbuild/linux-arm64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-arm64/-/linux-arm64-0.28.1.tgz",
 "integrity":
"sha512-yHs+0uc8+nvEAfAfxrWQKK5peSNzBc4PegcM00EJ2ht71uA7vB8Ihg2e77R2P7SG5uYjPbHlLLmve4LLLRCf0g==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/wrangler/node_modules/@esbuild/linux-ia32": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-ia32/-/linux-ia32-0.28.1.tgz",
 "integrity":
"sha512-d1z4ZuP0ajrfz/FhGT4vv278rX8KnPPJx8i5+AtK7TYbx9Le9F1hyzurZpkEyjkGa9dUGhQow4C1NmeGvqxN2w==",
 "cpu": [
 "ia32"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
```

**package-lock.json**

```
"node_modules/wrangler/node_modules/@esbuild/linux-loong64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-loong64/-/linux-loong64-0.28.1.tgz",
 "integrity":
"sha512-M5sRjUVZrkm10APR3dl0YzNmN+loZKGV11VUQGrwuqLcbR6qeAz+famMhjASeH3YVKvZz+zT1j1h/keC3Rj/lg==",
 "cpu": [
 "loong64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/wrangler/node_modules/@esbuild/linux-mips64el": {
 "version": "0.28.1",
 "resolved":
"https://registry.npmjs.org/@esbuild/linux-mips64el/-/linux-mips64el-0.28.1.tgz",
 "integrity":
"sha512-mRObBZeHh20xcBFPWE/FjylkRgZdYuiTR3vaTozquCGOH14iP9oN4x4Ge81CoIDYQrXmIxpFumJBu5MtZpnQJQ==",
 "cpu": [
 "mips64el"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/wrangler/node_modules/@esbuild/linux-ppc64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-ppc64/-/linux-ppc64-0.28.1.tgz",
 "integrity":
"sha512-slScBsMAb3GFDcdrCgLwZtPYRoH2H/youv10QiZyRjmsP48fznoveWytSgCI/R0ZcUgpc0ZhIUEx6LHts8yrfQ==",
 "cpu": [
 "ppc64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
}
```

**package-lock.json**

```
 },
 "node_modules/wrangler/node_modules/@esbuild/linux-riscv64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-riscv64/-/linux-riscv64-0.28.1.tgz",
 "integrity": "sha512-kw0owk1o0GFETUJyW0jc0G4Yzs0BHZn0JDZ8JRT088vjJYX777BAs1fDGxAC+q831q0s2DTC96mNsG2opdfyyQ==",
 "cpu": [
 "riscv64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/wrangler/node_modules/@esbuild/linux-s390x": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-s390x/-/linux-s390x-0.28.1.tgz",
 "integrity": "sha512-/lAIjX8aYFRByhh6L5rYtPEDRqa9de/4V/ju0Xcta5frjvzX04/sqEtyytse0g3zZFuwu5cDN0MkLz2qRDD2Ag==",
 "cpu": [
 "s390x"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/wrangler/node_modules/@esbuild/linux-x64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/linux-x64/-/linux-x64-0.28.1.tgz",
 "integrity": "sha512-u/anNYF2mmVOEDwLtnQ1wOr3EZ9sTNGLWrsYGYwHWzGA3Si84IOkHXlbWTD1NB+9/1lcwweYK054uhxZydNzfA==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "linux"
],
 "engines": {
 "node": ">=18"
 }
 }
```

**package-lock.json**

```
 },
 "node_modules/wrangler/node_modules/@esbuild/netbsd-arm64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/netbsd-arm64/-/netbsd-arm64-0.28.1.tgz",
 "integrity":
 "sha512-oks0DYbLWMMaakTsCb+zL4E+aHRVLom9IJZ0AthMQEPiQmydXHkziYEsGYRx0uNV/IjEKGAV941JzH02pflqw==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "netbsd"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/wrangler/node_modules/@esbuild/netbsd-x64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/netbsd-x64/-/netbsd-x64-0.28.1.tgz",
 "integrity":
 "sha512-aEL6lAN89Hz43Mlh1G8ARasbuoYvSITDEx0tHh5b7jJnHcssqgjy9Yx430GDpmCa60yrKoS0aNRjKundRizGg==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "netbsd"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/wrangler/node_modules/@esbuild/openbsd-arm64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/openbsd-arm64/-/openbsd-arm64-0.28.1.tgz",
 "integrity":
 "sha512-MEFJe5C3R8pwXdZ5Y21oo6m7ePiS0d9pwucn990/wvyJZChoIQKrQDxKrGew8F5+T0okTHesAmDeiHDTIq0V/Q==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "openbsd"
],
 "engines": {
 "node": ">=18"
 }
 }
```

**package-lock.json**

```
 },
 "node_modules/wrangler/node_modules/@esbuild/openbsd-x64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/openbsd-x64/-/openbsd-x64-0.28.1.tgz",
 "integrity":
"sha512-i/ZLI0afE0Z8cI/XANJAix0JL/uRAoS2x0A3rb0xN+KK0K177cMAsQYkzHtBrMXAKuAc7HGgcWiZ/sRC1Nxgw==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "openbsd"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/wrangler/node_modules/@esbuild/openharmony-arm64": {
 "version": "0.28.1",
 "resolved":
"https://registry.npmjs.org/@esbuild/openharmony-arm64/-/openharmony-arm64-0.28.1.tgz",
 "integrity":
"sha512-ge+Z7EXFnt2B01oAMsVpiQ8EwndV9i1xXerAeTIK7AtPs3bKFXQM7n1RxDSIUIMeueR1CNXxqztLzdNeReKBjg==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "openharmony"
],
 "engines": {
 "node": ">=18"
 }
 },
 "node_modules/wrangler/node_modules/@esbuild/sunos-x64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/sunos-x64/-/sunos-x64-0.28.1.tgz",
 "integrity":
"sha512-BEjgtECkL3vY+SaSQ6nzVfiALUeFxpawyp8Jmf5PtYhf1Ug40N1h/hx1hts+f1FvSvarEigdxS3BlSMI2PJLcQ==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "sunos"
],
 "engines": {
 "node": ">=18"
 }
 }
```

**package-lock.json**

```
}
},
"node_modules/wrangler/node_modules/@esbuild/win32-arm64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/win32-arm64/-/win32-arm64-0.28.1.tgz",
 "integrity":
"sha512-lCv9eK/H6ZJWbE7bh2nw54CZ9M2nupBxJcTsdk/QQnWkdSjKGuxmmH8/GWrLT1eMmZfn4dGcCjRte397WqfQXA==",
 "cpu": [
 "arm64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "win32"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/wrangler/node_modules/@esbuild/win32-ia32": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/win32-ia32/-/win32-ia32-0.28.1.tgz",
 "integrity":
"sha512-zvb/mB2bSCoJOpCBgYKKpX6YM6mJB1BUVUtVj41dLZJVEB6/0CKlRYxP5wWl1C1ILiCoAU5wZZ4q1P3qeS6Eg==",
 "cpu": [
 "ia32"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "win32"
],
 "engines": {
 "node": ">=18"
 }
},
"node_modules/wrangler/node_modules/@esbuild/win32-x64": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/@esbuild/win32-x64/-/win32-x64-0.28.1.tgz",
 "integrity":
"sha512-bm4Mowrv+GXMlpWX++EcXw/iLyd1o3+bJKC2DkWXyVvgZCqD/bSj9ctZeAMC3cIxgjRVR2Dufaiu4YPxr5gW1A==",
 "cpu": [
 "x64"
],
 "dev": true,
 "license": "MIT",
 "optional": true,
 "os": [
 "win32"
],
 "engines": {
 "node": ">=18"
 }
}
```

**package-lock.json**

```
}
},
"node_modules/wrangler/node_modules/esbuild": {
 "version": "0.28.1",
 "resolved": "https://registry.npmjs.org/esbuild/-/esbuild-0.28.1.tgz",
 "integrity":
"sha512-HrJrvZv5ayxBzPfwph0oNzkz0Iilifzk0KJrGK2c8R4+LKpMtpYLQeUdjnwjWv/LZlkH2laZk+4w78pi99D4Vw==",
 "dev": true,
 "hasInstallScript": true,
 "license": "MIT",
 "bin": {
 "esbuild": "bin/esbuild"
 },
 "engines": {
 "node": ">=18"
 },
 "optionalDependencies": {
 "@esbuild/aix-ppc64": "0.28.1",
 "@esbuild/android-arm": "0.28.1",
 "@esbuild/android-arm64": "0.28.1",
 "@esbuild/android-x64": "0.28.1",
 "@esbuild/darwin-arm64": "0.28.1",
 "@esbuild/darwin-x64": "0.28.1",
 "@esbuild/freebsd-arm64": "0.28.1",
 "@esbuild/freebsd-x64": "0.28.1",
 "@esbuild/linux-arm": "0.28.1",
 "@esbuild/linux-arm64": "0.28.1",
 "@esbuild/linux-ia32": "0.28.1",
 "@esbuild/linux-loong64": "0.28.1",
 "@esbuild/linux-mips64el": "0.28.1",
 "@esbuild/linux-ppc64": "0.28.1",
 "@esbuild/linux-riscv64": "0.28.1",
 "@esbuild/linux-s390x": "0.28.1",
 "@esbuild/linux-x64": "0.28.1",
 "@esbuild/netbsd-arm64": "0.28.1",
 "@esbuild/netbsd-x64": "0.28.1",
 "@esbuild/openbsd-arm64": "0.28.1",
 "@esbuild/openbsd-x64": "0.28.1",
 "@esbuild/openharmony-arm64": "0.28.1",
 "@esbuild/sunos-x64": "0.28.1",
 "@esbuild/win32-arm64": "0.28.1",
 "@esbuild/win32-ia32": "0.28.1",
 "@esbuild/win32-x64": "0.28.1"
 }
},
"node_modules/wrap-ansi": {
 "version": "7.0.0",
 "resolved": "https://registry.npmjs.org/wrap-ansi/-/wrap-ansi-7.0.0.tgz",
 "integrity":
"sha512-YVGIj2kamLSTxw6NsZj0BxfSwsn0ycdesmc4p+Q21c5zPuZ1pl+NfxVdxPtdHvmNV0Q6XSYG4AUtyt/Fi7D16Q==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "ansi-styles": "^4.0.0",

```



**package-lock.json**

```
 "string-width": "^4.1.0",
 "strip-ansi": "^6.0.0"
 },
 "engines": {
 "node": ">=10"
 },
 "funding": {
 "url": "https://github.com/chalk/wrap-ansi?sponsor=1"
 }
},
"node_modules/ws": {
 "version": "8.21.0",
 "resolved": "https://registry.npmjs.org/ws/-/ws-8.21.0.tgz",
 "integrity":
"sha512-Vsp28b7DRcimFQvrqu2Wek3z1iYxDCWqHYB8Qsnk/S4RfaCQzPGPyBNUVjJV3cd6UiKtUtp6sNM77gWvzcCH+g==",
 "dev": true,
 "license": "MIT",
 "engines": {
 "node": ">=10.0.0"
 },
 "peerDependencies": {
 "bufferutil": "^4.0.1",
 "utf-8-validate": ">=5.0.2"
 },
 "peerDependenciesMeta": {
 "bufferutil": {
 "optional": true
 },
 "utf-8-validate": {
 "optional": true
 }
 }
},
"node_modules/y18n": {
 "version": "5.0.8",
 "resolved": "https://registry.npmjs.org/y18n/-/y18n-5.0.8.tgz",
 "integrity":
"sha512-0pPfFzeyeDWHJHIAmTLRP2DwHjdF5s7jo9tuztdQxAhINCdVS+3nGINqPd00AphqJR/0LhANUS6/+7SCb98Y0fA==",
 "dev": true,
 "license": "ISC",
 "engines": {
 "node": ">=10"
 }
},
"node_modules/yallist": {
 "version": "3.1.1",
 "resolved": "https://registry.npmjs.org/yallist/-/yallist-3.1.1.tgz",
 "integrity":
"sha512-a4UGQawPH59mOXUYnAG2ewncQS4i4F43Tv3JoAM+s2VDAMs9NsK8GpDMLrCHPkFT7h3K6T0oUNn2pb7RoXx4g==",
 "dev": true,
 "license": "ISC"
},
"node_modules/yargs": {
 "version": "17.7.2",
```

**package-lock.json**

```
"resolved": "https://registry.npmjs.org/yargs/-/yargs-17.7.2.tgz",
"integrity":
"sha512-7dSzzRQ++CKnNI/krKnYRV7JKKPUMeh61soaHKG9mrWEhZFWhFnXPxGl+69cD10u63C13NUPCnmIcrvqCuM6w==",
"dev": true,
"license": "MIT",
"dependencies": {
 "cliui": "^8.0.1",
 "escalade": "^3.1.1",
 "get-caller-file": "^2.0.5",
 "require-directory": "^2.1.1",
 "string-width": "^4.2.3",
 "y18n": "^5.0.5",
 "yargs-parser": "^21.1.1"
},
"engines": {
 "node": ">=12"
}
},
"node_modules/yargs-parser": {
 "version": "21.1.1",
 "resolved": "https://registry.npmjs.org/yargs-parser/-/yargs-parser-21.1.1.tgz",
 "integrity":
"sha512-tVPsJW7DdjecAiFpbIB1e3qxIQsE6NoPc5/eTdrbbIC4h0LVsWhnoa3g+m2HclBIujHzsxZ4VJVA+GUuc2/LBw==",
 "dev": true,
 "license": "ISC",
 "engines": {
 "node": ">=12"
 }
},
"node_modules/youch": {
 "version": "4.1.0-beta.10",
 "resolved": "https://registry.npmjs.org/youch/-/youch-4.1.0-beta.10.tgz",
 "integrity":
"sha512-rLfVLB4FgQneDr0dv1oddCVZmKjcJ6yX6mS4pU82Mq/Dt9a3cLZQ62pDBL4AU0+uVrCvtWz3ZFUL2HFAFJ/BXQ==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@poppinss/colors": "^4.1.5",
 "@poppinss/dumper": "^0.6.4",
 "@speed-highlight/core": "^1.2.7",
 "cookie": "^1.0.2",
 "youch-core": "^0.3.3"
 }
},
"node_modules/youch-core": {
 "version": "0.3.3",
 "resolved": "https://registry.npmjs.org/youch-core/-/youch-core-0.3.3.tgz",
 "integrity":
"sha512-ho7XuGjLaJ2hWHoK8yFnsUGy2Y5uDpqSTq1FkHLK4/oqKtyUU1AFb00xY4IpC9f0FTLjwYbslUz0Po5BpD1wrA==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
 "@poppinss/exception": "^1.2.2",
 "error-stack-parser-es": "^1.0.5"
 }
}
```

**package-lock.json**

```
}
 }
}
}
```

**package.json**

```
{
 "name": "clantek",
 "version": "3.0.0",
 "private": true,
 "type": "module",
 "description": "Clan management system ? Cloudflare Workers + D1 + React. A rewrite of the 2003 PHP original.",
 "scripts": {
 "dev": "concurrently -n vite,worker -c cyan,magenta \"vite\" \"wrangler dev\"",
 "build": "tsc --noEmit && vite build",
 "deploy": "npm run build && wrangler deploy",
 "typecheck": "tsc --noEmit",
 "lint": "eslint .",
 "db:generate": "drizzle-kit generate",
 "db:migrate:local": "wrangler d1 migrations apply clantek --local",
 "db:migrate:remote": "wrangler d1 migrations apply clantek --remote",
 "db:seed:local": "wrangler d1 execute clantek --local --file=./src/db/seed.sql",
 "db:seed:remote": "wrangler d1 execute clantek --remote --file=./src/db/seed.sql",
 "db:studio": "drizzle-kit studio",
 "discord:register": "tsx scripts/register-commands.ts",
 "doctor": "tsx scripts/doctor.ts",
 "cf-typegen": "wrangler types"
 },
 "dependencies": {
 "@tiptap/pm": "^3.29.2",
 "@tiptap/react": "^3.29.2",
 "@tiptap/starter-kit": "^3.29.2",
 "dompurify": "^3.4.13",
 "drizzle-orm": "^0.45.2",
 "hono": "^4.12.34",
 "lucide": "^1.28.0",
 "morphicons": "^1.4.0",
 "pdf-lib": "^1.17.1",
 "react": "^19.0.0",
 "react-dom": "^19.0.0",
 "react-router-dom": "^7.1.0"
 },
 "devDependencies": {
 "@cloudflare/workers-types": "^5.20260801.1",
 "@types/react": "^19.0.0",
 "@types/react-dom": "^19.0.0",
 "@vitejs/plugin-react": "^5.0.0",
 "concurrently": "^9.1.0",
 "drizzle-kit": "^0.31.10",
 "tsx": "^4.19.0",
 "typescript": "^5.7.0",
 "vite": "^7.0.0",
 "wrangler": "^4.118.0"
 },
 "allowScripts": {
 "esbuild@0.18.20": true,
 "esbuild@0.25.12": true,
 "esbuild@0.28.1": true,
 "workerd@1.20260730.1": true
 }
}
```

**package.json**

```
}
}
```

## README.md

# ClanTek

Clan management for gaming communities ? rosters, ranks, medals, match records, and news, with Discord as both the login and the control surface.

A ground-up rewrite of the original 2003 PHP/MySQL ClanTek. No MySQL server, no license checks, no HTML pasted into a textarea.

### ## Stack

Layer	Technology
---	---
Runtime	Cloudflare Workers
Database	Cloudflare D1 (SQLite) via Drizzle ORM
Files	Cloudflare R2
API	Hono
Admin UI	React 19 + Vite
Identity	Discord OAuth2 ? no passwords

Runs within Cloudflare's free tier for a clan-sized site.

### ## How ranks and roles differ

The original gated every action on ``member.rank >= auth.<action>`` ? one linear ladder, so "trusted member who isn't an officer" was impossible to express. Those are two concepts here:

- **\*\*Rank\*\*** ? ladder position (Recruit -> General). One per member, ordered, fully admin-defined. Add, rename, reorder, and delete at will.
- **\*\*Role\*\*** ? a bundle of permissions, optionally mirrored to a Discord role. Many per member. Granting one here can grant the Discord role too, which is how website roles end up gating Discord channels.

**\*\*God status\*\*** (``users.is_god``) bypasses every permission check. It is seeded directly in ``src/db/seed.sql`` and is deliberately not assignable through the UI ? it's the recovery hatch that prevents anyone from locking themselves out.

### ## Setup

#### ### 1. Install

```

```bash
npm install
```

```

#### ### 2. Create the Discord application

At <https://discord.com/developers/applications> -> **\*\*New Application\*\***.

- **\*\*OAuth2\*\*** -> add redirect URI ``http://localhost:8787/api/auth/callback`` for local dev, and ``https://mustr.gg/api/auth/callback`` for production.
- **\*\*Bot\*\*** -> add a bot, copy the token.
- **\*\*General Information\*\*** -> copy the Public Key.

## README.md

Invite the bot to your server with the `bot` and `applications.commands` scopes and the **Manage Roles** permission.

```
> The one that bites everyone: in Server Settings -> Roles, drag the bot's
> role above every role it needs to manage. Discord returns `50013 Missing
> Permissions` otherwise, even for an administrator bot.
```

### ### 3. Create the Cloudflare resources

```
```bash  
npx wrangler d1 create clantek  
```
```

Copy the printed `database\_id` into `wrangler.jsonc`, then:

```
```bash  
npx wrangler r2 bucket create clantek-media  
```
```

### ### 4. Configure

```
```bash  
cp .dev.vars.example .dev.vars  
```
```

Fill in `.dev.vars` (gitignored). `DISCORD\_CLIENT\_ID` and `DISCORD\_PUBLIC\_KEY` are public values and already live in the `vars` block of `wrangler.jsonc`; set `DISCORD\_GUILD\_ID` there too (Developer Mode on -> right-click your server -> Copy Server ID).

Generate a session secret with:

```
```bash  
openssl rand -base64 32  
```
```

### ### 5. Initialize the database

```
```bash  
npm run db:generate  
npm run db:migrate:local  
npm run db:seed:local  
```
```

The seed creates ten ranks, five roles, default theme tokens, and the founder account. Edit the Discord ID in `src/db/seed.sql` before seeding if you are not the original owner.

### ### 6. Register slash commands

```
```bash  
npm run discord:register  
```
```

**README.md****### 7. Run**

```
```bash
npm run dev
```
```

Vite serves the UI on `:5173` and proxies `/api` to `wrangler dev` on `:8787`.

**## Deploying**

```
```bash
npx wrangler secret put DISCORD_CLIENT_SECRET
npx wrangler secret put DISCORD_BOT_TOKEN
npx wrangler secret put SESSION_SECRET
npm run db:migrate:remote
npm run db:seed:remote
npm run deploy
```
```

Then set the **Interactions Endpoint URL** on your Discord application to `https://mustr.gg/api/discord/interactions`. Discord probes it with a deliberately invalid signature on save ? rejecting that is what proves the endpoint is genuine.

For CI deploys, add `CLOUDFLARE\_API\_TOKEN` and `CLOUDFLARE\_ACCOUNT\_ID` as repository secrets; `.github/workflows/deploy.yml` handles the rest.

**## Discord commands**

| Command           | Requires                                        |
|-------------------|-------------------------------------------------|
| /roster           | any member                                      |
| /whois <member>   | any member                                      |
| /promote <member> | roster.promote, and you must outrank the target |

Commands run the same permission checks as the web portal ? one identity, one permission model, two surfaces.

**### A limitation worth knowing up front**

Workers serve Discord over **HTTP interactions**, not a gateway WebSocket. That covers slash commands and button clicks, but Workers cannot passively observe Discord events (someone joining, or a role changed by hand in Discord's UI). `reconcileMember()` in `src/server/discord/sync.ts` pulls Discord back into line on demand or on a cron, which covers most of the gap without a second host.

**## Layout**

```
...
```

```
src/
 db/schema.ts Drizzle schema ? the whole data model
 db/seed.sql Ranks, roles, permissions, founder account, theme
 shared/permissions.ts Permission vocabulary + can() / outranks()
 server/
```



**README.md**

```
index.ts Worker entry, OAuth, interactions endpoint
auth/ Discord OAuth2 and session handling
discord/ REST client, slash commands, role sync
routes/ API routers
client/ React admin (Vite)
scripts/
 register-commands.ts Pushes slash commands to Discord
...
```

**## About the 2003 original**

The original PHP source is archived separately, outside this repo, at `E:\Google Drive\Work\Development\ClanTek Unlocked`. It is deliberately not copied here: `dump.php` contains live MySQL credentials and `ctbd.php` contains a hardcoded password, and neither should gain a second copy on disk. If you do pull it in for reference, `.gitignore` already blocks it.

What carried forward: the rank ladder with time and win requirements, medals awarded per game, match records, and the audit log. What did not: the `license\_check` phone-home, Zend Encoder obfuscation, and the `templates` / `header` tables of `` attributes and IE scrollbar colors ? those are CSS custom properties now, edited with a live preview.

**## License**

MIT

**scripts/doctor.ts**

```

/**
 * Checks the Discord side of a ClanTek install and says what is wrong.
 *
 * npm run doctor
 *
 * Runs entirely against your local .dev.vars ? the bot token is never printed
 * and never leaves your machine.
 */

import { readFileSync } from 'node:fs';

const API = 'https://discord.com/api/v10';

function loadDevVars(): Record<string, string> {
 try {
 const out: Record<string, string> = {};
 for (const line of readFileSync('.dev.vars', 'utf8').split('\n')) {
 const match = line.match(/^s*([A-Z_]+)\s*=\s*(.*)$/);
 if (match) out[match[1]!] = match[2]!.trim();
 }
 return out;
 } catch {
 return {};
 }
}

function wranglerVar(name: string): string | undefined {
 try {
 const source = readFileSync('wrangler.jsonc', 'utf8');
 return source.match(new RegExp(`"${name}"\s*:\s*"([^\"]*)"`))?.[1] || undefined;
 } catch {
 return undefined;
 }
}

const secrets = { ...loadDevVars(), ...process.env };
const clientId = process.env.DISCORD_CLIENT_ID || wranglerVar('DISCORD_CLIENT_ID');
const guildId = process.env.DISCORD_GUILD_ID || wranglerVar('DISCORD_GUILD_ID');
const botToken = secrets.DISCORD_BOT_TOKEN;

const ok = (m: string) => console.log(` ok    ${m}`);
const bad = (m: string) => console.log(` FAIL  ${m}`);
const note = (m: string) => console.log(`      ${m}`);

async function call(path: string) {
 const res = await fetch(`${API}${path}`, { headers: { Authorization: `Bot ${botToken}` } });
 return { res, body: await res.text() };
}

async function main(): Promise<number> {
 let failed = 0;

 console.log('\nConfiguration');
 if (clientId) ok(`application id ${clientId}`);

```

**scripts/doctor.ts**

```

 else {
 bad('DISCORD_CLIENT_ID missing from wrangler.jsonc');
 failed++;
 }
 if (guildId) ok(`target guild    ${guildId}`);
 else {
 bad('DISCORD_GUILD_ID missing from wrangler.jsonc');
 failed++;
 }
 if (botToken) ok('bot token present in .dev.vars');
 else {
 bad('DISCORD_BOT_TOKEN missing from .dev.vars');
 return 1;
 }
 if (!clientId || !guildId) return 1;

 console.log('\nBot identity');
 const me = await call('/users/@me');
 if (!me.res.ok) {
 bad(`token rejected (${me.res.status}) ? reset it on the Bot tab and update .dev.vars`);
 return 1;
 }
 const bot = JSON.parse(me.body) as { id: string; username: string };
 ok(`authenticated as ${bot.username} (${bot.id})`);
 if (bot.id !== clientId) {
 bad(`this token belongs to application ${bot.id}, but wrangler.jsonc says ${clientId}`);
 note('The bot token and the client id are from different Discord applications.');
```

failed++;

```

 }

 console.log('\nGuild membership');
 const guilds = await call('/users/@me/guilds');
 if (!guilds.res.ok) {
 bad(`could not list guilds (${guilds.res.status}): ${guilds.body}`);
 return 1;
 }
 const list = JSON.parse(guilds.body) as { id: string; name: string }[];

 if (list.length === 0) {
 bad('the bot is not in ANY server');
 note('The invite was never completed. Open the URL below, pick your server,');
 note('and make sure you click through to the end:');
 note('');

 note(`https://discord.com/oauth2/authorize?client_id=${clientId}&permissions=268435456&scope=bot+applicati
 return 1;
 }

 console.log(`        bot is in ${list.length} server(s):`);
 for (const g of list) {
 console.log(`        ${g.id}    ${g.name}${g.id === guildId ? ' '    <-- target' : ''}`);
 }

 if (!list.some((g) => g.id === guildId)) {

```

**scripts/doctor.ts**

```

 bad(`the bot is NOT in the configured guild ${guildId}`);
 note('Either it was invited to a different server, or DISCORD_GUILD_ID is wrong.');
```

note('If one of the servers listed above is the right one, update');

note('DISCORD\_GUILD\_ID in wrangler.jsonc to match and redeploy.');

return 1;

}

ok('bot is in the configured guild');

console.log(`\nRole hierarchy`);

const roles = await call(`/guilds/\${guildId}/roles`);

const member = await call(`/guilds/\${guildId}/members/\${bot.id}`);

if (!roles.res.ok || !member.res.ok) {

bad('could not read roles ? the bot may lack View Server permissions');

return 1;

}

const all = JSON.parse(roles.body) as {

id: string;

name: string;

position: number;

permissions: string;

}[];

const held = (JSON.parse(member.body) as { roles: string[] }).roles;

// Effective guild permissions = OR of every held role's bitfield.

const ADMIN = 1n << 3n;

const MANAGE\_ROLES = 1n << 28n;

const MANAGE\_NICKNAMES = 1n << 27n;

let bits = 0n;

for (const r of all) if (held.includes(r.id) || r.name === '@everyone') bits |=

BigInt(r.permissions);

const isAdmin = (bits & ADMIN) === ADMIN;

const hasPerm = (bit: bigint) => isAdmin || (bits & bit) === bit;

console.log(`\nPermissions`);

if (isAdmin) ok('bot has Administrator (covers everything)');

const permChecks: [string, bigint, string][] = [

['Manage Roles', MANAGE\_ROLES, 'role sync (grant/revoke, rename, recolour)'],

['Manage Nicknames', MANAGE\_NICKNAMES, 'display-name -> Discord nickname sync'],

];

for (const [name, bit, why] of permChecks) {

if (hasPerm(bit)) ok(`\${name}`);

else {

bad(`missing \${name} ? needed for \${why}`);

note(`Server Settings -> Roles -> the bot's role -> enable \${name}.`);

failed++;

}

}

const botTop = Math.max(...all.filter((r) => held.includes(r.id)).map((r) => r.position), 0);

const above = all.filter((r) => r.position > botTop && r.name !== '@everyone');

ok(`bot's highest role sits at position \${botTop}`);

if (above.length) {

bad(`\${above.length} role(s) sit ABOVE the bot and cannot be managed:`);

**scripts/doctor.ts**

```
 for (const r of above) note(` ${r.name} (position ${r.position})`);
 note('');
 note('Server Settings -> Roles, drag the bot role above these.');
```

note('Discord returns 50013 otherwise, even for an administrator bot.');

failed++;

```
 } else {
 ok('bot outranks every other role');
 }

 console.log(
 failed === 0 ? '\nAll checks passed.\n' : '\n${failed} problem(s) found ? see above.\n',
);
 return failed === 0 ? 0 : 1;
}

process.exitCode = await main();
```

**scripts/generate-copyright-pdf-v2.js**

```

import fs from 'node:fs';
import path from 'node:path';
import { PDFDocument, StandardFonts, rgb } from 'pdf-lib';

const root = process.cwd();
const outPath = path.resolve(root, 'exports/copyright-submission.pdf');

const skipDirs = new Set(['.git', 'node_modules', 'dist', 'build', '.next', '.turbo', 'coverage',
 '.wrangler', '.yarn', '.vscode']);
const includeExts = new Set(['.ts', '.tsx', '.js', '.jsx', '.css', '.html', '.json', '.md',
 '.sql', '.cjs', '.mjs', '.yaml', '.yml', '.toml', '.txt', '.svg', '.sh', '.ps1', '.env']);

function walk(dir) {
 const entries = fs.readdirSync(dir, { withFileTypes: true });
 const files = [];
 for (const entry of entries) {
 const full = path.join(dir, entry.name);
 if (entry.isDirectory()) {
 if (!skipDirs.has(entry.name)) files.push(...walk(full));
 } else if (entry.isFile()) {
 const ext = path.extname(entry.name).toLowerCase();
 if (includeExts.has(ext) || entry.name === 'README' || entry.name === 'LICENSE') {
 files.push(full);
 }
 }
 }
 return files;
}

function normalize(text) {
 return text.replace(/\r\n/g, '\n').replace(/\r/g, '\n');
}

function sanitize(text) {
 const map = {
 '->': '->',
 '<': '<',
 '*': '*',
 '': ' ',
 '': ' ',
 '': ' ',
 '': ' ',
 '': ' ',
 '...': '...',
 };
 let out = '';
 for (const ch of normalize(text)) {
 if (ch in map) {
 out += map[ch];
 continue;
 }
 const code = ch.charCodeAt(0);
 if (code === 9 || code === 10 || code === 13 || (code >= 32 && code <= 126)) {
 out += ch;
 } else {
 out += '?';
 }
 }
}

```

**scripts/generate-copyright-pdf-v2.js**

```

 }
 }
 return out;
}

function wrapText(text, font, size, maxWidth) {
 const lines = [];
 for (const raw of normalize(text).split('\n')) {
 const line = raw.replace(/\t/g, ' ');
 if (font.widthOfTextAtSize(line, size) <= maxWidth) {
 lines.push(line);
 continue;
 }
 const words = line.split(/(\s+)/);
 let current = '';
 for (const word of words) {
 if (!word) continue;
 const candidate = current ? current + word : word;
 if (font.widthOfTextAtSize(candidate, size) <= maxWidth) {
 current = candidate;
 } else {
 if (current) lines.push(current);
 current = word;
 }
 }
 if (current) lines.push(current);
 }
 return lines;
}

function addHeader(page, font, title, pageno, totalPages) {
 const { width, height } = page.getSize();
 page.drawText(title, { x: 40, y: height - 40, size: 10, font, color: rgb(0.2, 0.2, 0.2) });
 page.drawText(`Page ${pageno} of ${totalPages}`, { x: width - 120, y: height - 40, size: 10,
font, color: rgb(0.2, 0.2, 0.2) });
 page.drawLine({ start: { x: 40, y: height - 52 }, end: { x: width - 40, y: height - 52 },
thickness: 0.5, color: rgb(0.7, 0.7, 0.7) });
}

async function main() {
 fs.mkdirSync(path.dirname(outPath), { recursive: true });
 const files = walk(root).filter((f) => !f.includes('exports')).sort((a, b) =>
a.localeCompare(b));
 const pdfDoc = await PDFDocument.create();
 const courier = await pdfDoc.embedFont(StandardFonts.Courier);
 const bold = await pdfDoc.embedFont(StandardFonts.HelveticaBold);
 const pageWidth = 612;
 const pageHeight = 792;
 const margin = 40;
 const maxWidth = pageWidth - margin * 2;
 const lineHeight = 12;
 const linesPerPage = 54;

 const cover = pdfDoc.addPage([pageWidth, pageHeight]);

```

**scripts/generate-copyright-pdf-v2.js**

```

 addHeader(cover, courier, 'Copyright Submission Bundle', 1, 2 + files.length);
 cover.drawText('ClanTek Project Source Listing', { x: 40, y: 650, size: 24, font: bold, color:
 rgb(0.08, 0.08, 0.08) });
 cover.drawText(`Generated from ${path.basename(root)}`, { x: 40, y: 610, size: 12, font:
 courier, color: rgb(0.2, 0.2, 0.2) });
 cover.drawText(`Included files: ${files.length}`, { x: 40, y: 575, size: 12, font: courier,
 color: rgb(0.2, 0.2, 0.2) });
 cover.drawText('This document contains the source code and project files used for copyright
 submission.', { x: 40, y: 525, size: 11, font: courier, color: rgb(0.2, 0.2, 0.2) });

 const toc = pdfDoc.addPage([pageWidth, pageHeight]);
 addHeader(toc, courier, 'Table of Contents', 2, 2 + files.length);
 toc.drawText('Files included', { x: 40, y: 700, size: 18, font: bold, color: rgb(0.08, 0.08,
 0.08) });
 let y = 660;
 for (const file of files.slice(0, 80)) {
 const rel = path.relative(root, file).replace(/\\/g, '/');
 if (y < 80) break;
 toc.drawText(rel, { x: 50, y, size: 8, font: courier, color: rgb(0.2, 0.2, 0.2) });
 y -= 12;
 }

 let pageNumber = 3;
 for (const file of files) {
 const relPath = path.relative(root, file).replace(/\\/g, '/');
 const text = fs.readFileSync(file, 'utf8');
 const lines = wrapText(sanitize(text), courier, 9, maxWidth);

 let currentPage = null;
 let currentY = 0;

 const newPage = () => {
 currentPage = pdfDoc.addPage([pageWidth, pageHeight]);
 addHeader(currentPage, courier, relPath, pageNumber, 2 + files.length +
 Math.ceil(lines.length / linesPerPage));
 pageNumber += 1;
 currentY = pageHeight - 90;
 currentPage.drawText(relPath, { x: 40, y: pageHeight - 78, size: 10, font: bold, color:
 rgb(0.08, 0.08, 0.08) });
 return currentPage;
 };

 if (!currentPage) newPage();
 for (const line of lines) {
 if (currentY < 60) newPage();
 currentPage.drawText(line, { x: 40, y: currentY, size: 9, font: courier, color: rgb(0.1,
 0.1, 0.1) });
 currentY -= lineHeight;
 }
 }

 const bytes = await pdfDoc.save();
 fs.writeFileSync(outPath, bytes);
 const saved = await PDFDocument.load(outPath);

```



**scripts/generate-copyright-pdf-v2.js**

```
 console.log(`Created ${path.relative(root, outPath)} with ${saved.getPageCount()} pages`);
 }

main().catch((err) => {
 console.error(err);
 process.exit(1);
});
```

**scripts/generate-copyright-pdf.js**

```
import fs from 'node:fs';
import path from 'node:path';
import { PDFDocument, StandardFonts, rgb } from 'pdf-lib';

const root = process.cwd();
const outputPath = path.resolve(root, 'exports/copyright-submission.pdf');

const excludedDirs = new Set([
 '.git',
 'node_modules',
 'dist',
 'build',
 '.next',
 '.turbo',
 'coverage',
 '.wrangler',
 '.yarn',
 '.vscode',
]);

const includedExtensions = new Set([
 '.ts',
 '.tsx',
 '.js',
 '.jsx',
 '.css',
 '.html',
 '.json',
 '.md',
 '.sql',
 '.cjs',
 '.mjs',
 '.yaml',
 '.yml',
 '.toml',
 '.txt',
 '.svg',
 '.sh',
 '.ps1',
 '.env',
]);

function walk(dir) {
 const entries = fs.readdirSync(dir, { withFileTypes: true });
 const files = [];
 for (const entry of entries) {
 if (entry.name.startsWith('.')) {
 if (entry.name === '.git' || entry.name === '.vscode') continue;
 }
 const fullPath = path.join(dir, entry.name);
 if (entry.isDirectory()) {
 if (!excludedDirs.has(entry.name)) {
 files.push(...walk(fullPath));
 }
 }
 }
}
```

**scripts/generate-copyright-pdf.js**

```

 } else if (entry.isFile()) {
 const ext = path.extname(entry.name).toLowerCase();
 if (includedExtensions.has(ext) || entry.name === 'README' || entry.name === 'LICENSE') {
 files.push(fullPath);
 }
 }
 }
 return files;
}

function normalizeLineEndings(text) {
 return text.replace(/\r\n/g, '\n').replace(/\r/g, '\n');
}

function sanitizeText(text) {
 return normalizeLineEndings(text)
 .replace(/\t/g, ' ')
 .replace(/->/g, '->')
 .replace(/<- /g, '<- ')
 .replace(/*/g, '*')
 .replace(/'/g, "'")
 .replace(/"/g, '"')
 .replace(/"/g, '"')
 .replace(/.../g, '...')
 .replace(/\u0000/g, '')
 .replace(/[^\
?0-?]/g, '?');
}

function wrapText(text, font, size, maxWidth) {
 const lines = [];
 for (const rawLine of normalizeLineEndings(text).split('\n')) {
 const line = rawLine.replace(/\t/g, ' ');
 if (font.widthOfTextAtSize(line, size) <= maxWidth) {
 lines.push(line);
 continue;
 }

 const words = line.split(/(\s+)/);
 let current = '';
 for (const word of words) {
 if (!word) continue;
 const candidate = current ? current + word : word;
 if (font.widthOfTextAtSize(candidate, size) <= maxWidth) {
 current = candidate;
 } else {
 if (current) {
 lines.push(current);
 }
 current = word;
 }
 }
 if (current) lines.push(current);
 }
}

```

**scripts/generate-copyright-pdf.js**

```
 }
 return lines;
}

function addPageWithHeader(doc, page, font, fontSize, title, pageNumber, totalPages) {
 const { width, height } = page.getSize();
 page.drawText(title, {
 x: 40,
 y: height - 40,
 size: 10,
 font,
 color: rgb(0.2, 0.2, 0.2),
 });
 page.drawText(`Page ${pageNumber} of ${totalPages}`, {
 x: width - 120,
 y: height - 40,
 size: 10,
 font,
 color: rgb(0.2, 0.2, 0.2),
 });
 page.drawLine({
 start: { x: 40, y: height - 52 },
 end: { x: width - 40, y: height - 52 },
 thickness: 0.5,
 color: rgb(0.7, 0.7, 0.7),
 });
 page.drawText('Generated for copyright submission', {
 x: 40,
 y: height - 60,
 size: 8,
 font,
 color: rgb(0.45, 0.45, 0.45),
 });
}

async function main() {
 fs.mkdirSync(path.dirname(outputPath), { recursive: true });
 const files = walk(root)
 .filter((file) => !file.includes('\\exports\\'))
 .sort((a, b) => a.localeCompare(b));

 const pdfDoc = await PDFDocument.create();
 const font = await pdfDoc.embedFont(StandardFonts.Courier);
 const titleFont = await pdfDoc.embedFont(StandardFonts.HelveticaBold);
 const pageHeight = 792;
 const pageWidth = 612;
 const margin = 40;
 const maxWidth = pageWidth - margin * 2;
 const lineHeight = 12;
 const maxLines = 54;

 const coverPage = pdfDoc.addPage([pageWidth, pageHeight]);
 addPageWithHeader(pdfDoc, coverPage, font, 10, 'Copyright Submission Bundle', 1, 1);
 coverPage.drawText('ClanTek Project Source Listing', {
```

**scripts/generate-copyright-pdf.js**

```
x: 40,
y: 650,
size: 24,
font: titleFont,
color: rgb(0.08, 0.08, 0.08),
});
coverPage.drawText(`Generated from ${path.basename(root)}`, {
 x: 40,
 y: 610,
 size: 12,
 font,
 color: rgb(0.2, 0.2, 0.2),
});
coverPage.drawText(`Included files: ${files.length}`, {
 x: 40,
 y: 575,
 size: 12,
 font,
 color: rgb(0.2, 0.2, 0.2),
});
coverPage.drawText('This document contains the source code and project files used for copyright
submission.', {
 x: 40,
 y: 525,
 size: 11,
 font,
 color: rgb(0.2, 0.2, 0.2),
});
coverPage.drawText('Generated automatically by scripts/generate-copyright-pdf.js', {
 x: 40,
 y: 480,
 size: 10,
 font,
 color: rgb(0.45, 0.45, 0.45),
});

const tocPage = pdfDoc.addPage([pageWidth, pageHeight]);
addPageWithHeader(pdfDoc, tocPage, font, 10, 'Table of Contents', 2, 2 + files.length);
tocPage.drawText('Files included', {
 x: 40,
 y: 700,
 size: 18,
 font: titleFont,
 color: rgb(0.08, 0.08, 0.08),
});
let y = 660;
for (const file of files.slice(0, 40)) {
 const rel = path.relative(root, file).replace(/\\/g, '/');
 const line = `${rel}`;
 if (y < 80) break;
 tocPage.drawText(line, {
 x: 50,
 y,
 size: 8,
```

**scripts/generate-copyright-pdf.js**

```

 font,
 color: rgb(0.2, 0.2, 0.2),
 });
 y -= 12;
}

let pageNumber = 3;
for (const file of files) {
 const relPath = path.relative(root, file).replace(/\\/g, '/');
 const contents = fs.readFileSync(file, 'utf8');

 const lines = wrapText(sanitizeText(contents), font, 9, maxWidth);
 let currentPage = null;
 let currentY = 0;
 const addNewPage = () => {
 const page = pdfDoc.addPage([pageWidth, pageHeight]);
 addPageWithHeader(pdfDoc, page, font, 10, relPath, pageNumber, 2 + files.length +
Math.ceil(lines.length / maxLines));
 pageNumber += 1;
 currentPage = page;
 currentY = pageHeight - 90;
 currentPage.drawText(relPath, {
 x: 40,
 y: pageHeight - 78,
 size: 10,
 font: titleFont,
 color: rgb(0.08, 0.08, 0.08),
 });
 return page;
 };

 if (!currentPage) addNewPage();

 for (const line of lines) {
 if (currentY < 60) {
 addNewPage();
 }
 currentPage.drawText(line, {
 x: 40,
 y: currentY,
 size: 9,
 font,
 color: rgb(0.1, 0.1, 0.1),
 });
 currentY -= lineHeight;
 }
}

const bytes = await pdfDoc.save();
fs.writeFileSync(outputPath, bytes);
const savedPdf = await PDFDocument.load(outputPath);
console.log(`Created ${path.relative(root, outputPath)} with ${savedPdf.getPageCount()} pages`);
}

```

**scripts/generate-copyright-pdf.js**

```
main().catch((error) => {
 console.error(error);
 process.exit(1);
});
```

**scripts/register-commands.ts**

```

/**
 * Registers slash commands with Discord.
 *
 * Run after adding or changing a command: npm run discord:register
 *
 * Guild commands (what this uses) appear instantly. Global commands can take
 * up to an hour to propagate, which makes them painful to iterate on.
 *
 * Reads DISCORD_CLIENT_ID / DISCORD_GUILD_ID / DISCORD_BOT_TOKEN from the
 * environment, falling back to .dev.vars so local setup needs no extra steps.
 */

import { readFileSync } from 'node:fs';

const OptionType = { STRING: 3, INTEGER: 4, USER: 6 } as const;

const COMMANDS = [
 {
 name: 'whois',
 description: 'Look up a member's rank, roles, and medals',
 options: [
 {
 name: 'member',
 description: 'The member to look up',
 type: OptionType.USER,
 required: true,
 },
],
 },
 {
 name: 'roster',
 description: 'Show the clan roster broken down by rank',
 },
 {
 name: 'promote',
 description: 'Promote a member one step up the rank ladder',
 options: [
 {
 name: 'member',
 description: 'The member to promote',
 type: OptionType.USER,
 required: true,
 },
],
 },
];

/** Secrets only. Public config lives in wrangler.jsonc. */
function loadDevVars(): Record<string, string> {
 try {
 const out: Record<string, string> = {};
 for (const line of readFileSync('.dev.vars', 'utf8').split('\n')) {
 const match = line.match(/^\\s*([A-Z_]+)\\s*=\\s*(.*)$/);
 if (match) out[match[1]!] = match[2]!.trim();
 }
 }
}

```



**scripts/register-commands.ts**

```

 }
 return out;
 } catch {
 return {};
 }
}

/**
 * Reads a var straight out of wrangler.jsonc. Matching the key directly avoids
 * having to strip JSONC comments, which JSON.parse would choke on.
 */
function wranglerVar(name: string): string | undefined {
 try {
 const source = readFileSync('wrangler.jsonc', 'utf8');
 return source.match(new RegExp(`"${name}"\\s*:\\s*"([^"]*)"`))?.[1] || undefined;
 } catch {
 return undefined;
 }
}

/** Turns Discord's terser error codes into something actionable. */
function explain(status: number, body: string): string {
 if (body.includes('50001')) {
 return [
 'The bot is not authorized in this server yet.',
 '',
 'Guild command registration needs the applications.commands scope in the',
 'target guild. Invite the bot first, then re-run this:',
 '',
 `https://discord.com/oauth2/authorize?client_id=${clientId}&permissions=268435456&scope=bot+applications.commands`
].join('\n');
 }
 if (status === 401) {
 return 'Bot token rejected. Check DISCORD_BOT_TOKEN in .dev.vars ? reset it in the\nDeveloper Portal (Bot tab) if you are unsure it is current.';
 }
 if (body.includes('50035')) {
 return 'Discord rejected a command definition. Check names are lowercase and\ndescriptions are non-empty and under 100 characters.';
 }
 return '';
}

const secrets = { ...loadDevVars(), ...process.env };

const clientId = process.env.DISCORD_CLIENT_ID || wranglerVar('DISCORD_CLIENT_ID');
const guildId = process.env.DISCORD_GUILD_ID || wranglerVar('DISCORD_GUILD_ID');
const botToken = secrets.DISCORD_BOT_TOKEN;

// Setting exitCode and returning lets Node drain stdio and exit cleanly.
// process.exit() here trips a libuv assertion on Windows mid-flush.
async function main(): Promise<number> {
 if (!clientId || !guildId) {

```

**scripts/register-commands.ts**

```

 console.error(
 'Missing DISCORD_CLIENT_ID or DISCORD_GUILD_ID.\n' +
 'Both live in the "vars" block of wrangler.jsonc. Run this from the project root.',
);
 return 1;
 }

 if (!botToken) {
 console.error(
 'Missing DISCORD_BOT_TOKEN.\n' +
 'Copy .dev.vars.example to .dev.vars and paste your bot token into it.\n' +
 '(.dev.vars is gitignored.)',
);
 return 1;
 }

 const res = await fetch(
 `https://discord.com/api/v10/applications/${clientId}/guilds/${guildId}/commands`,
 {
 method: 'PUT', // wholesale replace ? removes commands deleted from COMMANDS
 headers: {
 Authorization: `Bot ${botToken}`,
 'Content-Type': 'application/json',
 },
 body: JSON.stringify(COMMANDS),
 },
);

 if (!res.ok) {
 const body = await res.text();
 console.error(`Registration failed (${res.status}): ${body}`);
 const hint = explain(res.status, body);
 if (hint) console.error(`\n${hint}`);
 return 1;
 }

 const registered = (await res.json()) as { name: string }[];
 console.log(
 `Registered ${registered.length} command(s): ${registered.map((c) => `/${c.name}`).join(', ')}`,
);
 return 0;
}

process.exitCode = await main();

```

**src/client/App.tsx**

```

import { lazy, Suspense, useEffect, useState, type CSSProperties, type ReactNode } from 'react';
import { Link, Navigate, Route, Routes } from 'react-router-dom';
import { MorphIcon } from 'morphicons/react';
import { Menu, X } from 'lucide';
import { useSession } from '../lib/session';
import { useBranding } from '../lib/branding';
import type { Permission } from '../shared/permissions';
import AccountMenu from '../components/AccountMenu';
import SiteNav from '../components/SiteNav';
import type { NavItem } from '../shared/nav';
import Login from '../pages/Login';
import MemberDetail from '../pages/MemberDetail';
import AccountSettings from '../pages/AccountSettings';
import Roster from '../pages/Roster';
import Home from '../pages/Home';
import CustomPage from '../pages/CustomPage';
import News from '../pages/News';
import NewsPost from '../pages/NewsPost';
import Events from '../pages/Events';
import { api } from '../lib/api';
import { sanitizeHtml } from '../lib/richtext';
import type { FooterConfig } from '../shared/footer';

```

```

// The admin panel pulls in the WYSIWYG editor (TipTap/ProseMirror), which is
// large and admin-only ? load it on demand so the feed and roster stay light.
const Admin = lazy(() => import('../pages/Admin'));

```

```

function Protected({ permission, children }: { permission?: Permission; children: ReactNode }) {
 const { viewer, loading, can } = useSession();
 if (loading) return <div className="loading">Loading...</div>;
 if (!viewer) return <Navigate to="/login" replace />;
 if (permission && !can(permission)) {
 return <div className="empty">You don't have access to this area.</div>;
 }
 return <>{children}</>;
}

```

```

export default function App() {
 const { viewer, siteName, loading } = useSession();
 const { branding } = useBranding();
 const [menuOpen, setMenuOpen] = useState(false);
 const [navItems, setNavItems] = useState<NavItem[]>([]);
 const [footer, setFooter] = useState<FooterConfig | null>(null);
 const [scrolled, setScrolled] = useState(false);
 // Logo aspect ratio (width/height), measured on load, so the header can
 // reserve exactly the collapsed logo's width and the nav never jumps.
 const [logoAspect, setLogoAspect] = useState(1);

 // Keep the browser-tab title in sync with the configured site name (the served
 // HTML title is set server-side; this covers client-side navigation + updates).
 useEffect(() => {
 if (siteName) document.title = siteName;
 }, [siteName]);

```

**src/client/App.tsx**

```

// The site footer is public (shows on the login page too) and rarely changes,
// so load it once on mount regardless of auth.
useEffect(() => {
 api
 .get<{ footer: FooterConfig }>('/settings/footer')
 .then((d) => setFooter(d.footer))
 .catch(() => setFooter(null));
}, []);

// Shrink the header (and its logo) once the page scrolls past the top.
useEffect(() => {
 const onScroll = () => setScrolled(window.scrollY > 12);
 onScroll();
 window.addEventListener('scroll', onScroll, { passive: true });
 return () => window.removeEventListener('scroll', onScroll);
}, []);

// The admin-arranged menu tree. Loaded once the viewer is known, and refreshed
// live when the Navigation builder saves (`ct-nav-changed`) or a page is
// created/deleted (`ct-pages-changed` ? the server prunes deleted pages).
useEffect(() => {
 if (!viewer) {
 setNavItems([]);
 return;
 }
 const loadNav = () =>
 api
 .get<{ nav: { items: NavItem[] } }>('/nav')
 .then((d) => setNavItems(d.nav.items))
 .catch(() => setNavItems([]));
 void loadNav();
 window.addEventListener('ct-nav-changed', loadNav);
 window.addEventListener('ct-pages-changed', loadNav);
 return () => {
 window.removeEventListener('ct-nav-changed', loadNav);
 window.removeEventListener('ct-pages-changed', loadNav);
 };
}, [viewer]);

if (loading) return <div className="loading">Loading...</div>;

const hasLogo = !!branding.logoUrl;
const collapsed = 38; // logo height (px) once docked inside the bar
const cap = 460; // matches the logo's CSS max-width, so the reserved slot agrees
const topbarStyle = {
 '--logo-expanded': `${branding.logoSize}px`,
 '--logo-collapsed': `${collapsed}px`,
 // Reserve the logo's full (expanded) footprint so it never covers the menu;
 // --logo-fp-min is the docked footprint used on mobile.
 '--logo-fp': `${Math.min(Math.round(branding.logoSize * logoAspect), cap)}px`,
 '--logo-fp-min': `${Math.round(collapsed * logoAspect)}px`,
} as CSSProperties;

return (

```

**src/client/App.tsx**

```

<div className="app">
 <header
 className={`topbar${scrolled ? ' scrolled' : ''}${hasLogo ? ' has-logo' : ''}`}
 style={hasLogo ? topbarStyle : undefined}
 >
 {viewer && (
 <button
 className="nav-toggle"
 aria-label="Menu"
 aria-expanded={menuOpen}
 onClick={() => setMenuOpen((o) => !o)}
 >
 <MorphIcon icon={menuOpen ? X : Menu} size={22} spring="snappy" aria-hidden />
 </button>
)}
 <Link to="/" className="brand" aria-label={siteName}>
 {hasLogo ? (
 <img
 className="brand-logo"
 src={branding.logoUrl}
 alt={siteName}
 onLoad={(e) => {
 const img = e.currentTarget;
 if (img.naturalHeight > 0) setLogoAspect(img.naturalWidth / img.naturalHeight);
 }}
 />
) : (
 {siteName}
)}
 </Link>

 {viewer && (
 <nav className={menuOpen ? 'nav open' : 'nav'}>
 <SiteNav items={navItems} onNavigate={() => setMenuOpen(false)} />
 </nav>
)}

 <div className="account">
 {viewer ? (
 <AccountMenu />
) : (

 Sign in with Discord

)}
 </div>
 </header>

 <main className="content">
 <Suspense fallback={<div className="loading">Loading...</div>}>
 <Routes>
 <Route path="/login" element={<Login />} />
 <Route
 path="/"

```

**src/client/App.tsx**

```
 element={
 <Protected>
 <Home />
 </Protected>
 }
 />
 <Route
 path="/news"
 element={
 <Protected>
 <News />
 </Protected>
 }
 />
 <Route
 path="/news/:slug"
 element={
 <Protected>
 <NewsPost />
 </Protected>
 }
 />
 { /* Leadership folded into the roster; keep the old path working. */ }
 <Route path="/leadership" element={ <Navigate to="/roster" replace /> } />
 <Route
 path="/roster"
 element={
 <Protected>
 <Roster />
 </Protected>
 }
 />
 <Route
 path="/p/:slug"
 element={
 <Protected>
 <CustomPage />
 </Protected>
 }
 />
 <Route
 path="/events"
 element={
 <Protected permission="events.view">
 <Events />
 </Protected>
 }
 />
 <Route
 path="/members/:id"
 element={
 <Protected>
 <MemberDetail />
 </Protected>
 }
 />
```

**src/client/App.tsx**

```

 }
 />
 <Route
 path="/account"
 element={
 <Protected>
 <AccountSettings />
 </Protected>
 }
 />
 {/* One shell for every admin tool; the panel gates each item/tab.
 /admin/:item is a sidebar entry; /admin/:item/:tab picks a tool
 within a multi-tool item (e.g. Ranks & Roles). */}
 <Route
 path="/admin"
 element={
 <Protected>
 <Admin />
 </Protected>
 }
 />
 <Route
 path="/admin/:item"
 element={
 <Protected>
 <Admin />
 </Protected>
 }
 />
 <Route
 path="/admin/:item/:tab"
 element={
 <Protected>
 <Admin />
 </Protected>
 }
 />
 <Route path="" element={<div className="empty">Not found.</div>} />
</Routes>
</Suspense>
</main>

{footer && (footer.text || footer.links.length > 0 || footer.copyright) && (
 <footer className="site-footer">
 {footer.text && (
 <div
 className="site-footer-text"
 dangerouslySetInnerHTML={{ __html: sanitizeHtml(footer.text) }}
 />
)}
 {footer.links.length > 0 && (
 <nav className="site-footer-links" aria-label="Footer">
 {/* Plain anchors (full navigation): footer links may point at
 server routes like /legal that aren't SPA routes, so a client

```

**src/client/App.tsx**

```
 <Link> would 404. External links open in a new tab. */}
 {footer.links.map((l, i) =>
 /^https?:\/\//i.test(l.href) ? (

 {l.label}

) : (

 {l.label}

),
)}
 </nav>
)}
<div className="site-footer-copy">
 {footer.copyright || `? ${new Date().getFullYear()} ${siteName}`}
</div>
</footer>
)}
</div>
);
}
```



**src/client/components/AccountMenu.tsx**

```

/**
 * The top-right account menu.
 *
 * Replaces the bare "Sign out" button with the viewer's avatar + name, opening
 * a dropdown of account actions. "Admin" only appears when the viewer can reach
 * at least one admin section (derived from the same ADMIN_SECTIONS registry the
 * panel uses, so the two never drift apart).
 */

import { useEffect, useRef, useState } from 'react';
import { Link } from 'react-router-dom';
import { MorphIcon } from 'morphicons/react';
import { ChevronDown, ChevronUp } from 'lucide';
import { useSession } from '../lib/session';
import { memberName } from '../shared/names';
import { memberAvatar } from '../shared/avatar';
import { canAccessAdmin } from '../lib/adminSections';

export default function AccountMenu() {
 const { viewer, can, logout } = useSession();
 const [open, setOpen] = useState(false);
 const ref = useRef<HTMLDivElement>(null);

 // Close on outside click or Escape.
 useEffect(() => {
 if (!open) return;
 const onDown = (e: MouseEvent) => {
 if (ref.current && !ref.current.contains(e.target as Node)) setOpen(false);
 };
 const onKey = (e: KeyboardEvent) => {
 if (e.key === 'Escape') setOpen(false);
 };
 document.addEventListener('mousedown', onDown);
 document.addEventListener('keydown', onKey);
 return () => {
 document.removeEventListener('mousedown', onDown);
 document.removeEventListener('keydown', onKey);
 };
 }, [open]);

 if (!viewer) return null;

 const showAdmin = canAccessAdmin(can);

 return (
 <div className="account-menu" ref={ref}>
 <button
 className="account-trigger"
 onClick={() => setOpen((o) => !o)}
 aria-haspopup="menu"
 aria-expanded={open}
 >

 {memberName(viewer)}
 </div>
 </div>
);
}

```

**src/client/components/AccountMenu.tsx**

```

 {viewer.rank && {viewer.rank.name}}
 {viewer.isGod && (

 ?

)}
 <MorphIcon className="caret" icon={open ? ChevronUp : ChevronDown} size={16} aria-hidden
 />
</button>

{open && (
 <div className="account-dropdown" role="menu">
 <Link
 className="menu-item"
 role="menuitem"
 to={`~/members/${viewer.id}`}
 onClick={() => setOpen(false)}
 >
 Edit Profile
 </Link>
 <Link className="menu-item" role="menuitem" to="/account" onClick={() =>
setOpen(false)}>
 Settings
 </Link>
 {showAdmin && (
 <Link className="menu-item" role="menuitem" to="/admin" onClick={() =>
setOpen(false)}>
 Admin
 </Link>
)}
 <div className="menu-sep" />
 <button
 className="menu-item"
 role="menuitem"
 onClick={() => {
 setOpen(false);
 void logout();
 }}
 >
 Sign Out
 </button>
 </div>
)}
</div>
);
}

```

**src/client/components/HangarImport.tsx**

```

/**
 * Import/update your own Star Citizen hangar. Shown only on your own profile.
 *
 * The RSI hangar is login-private with no API, so the member runs a bookmarklet
 * on their hangar (it copies the scraped items to the clipboard) and pastes the
 * result here. A collapsible "how to set up" section carries the bookmarklet and
 * install steps; once installed it's one click + paste per update.
 */

import { useEffect, useState } from 'react';
import { api } from '../lib/api';
import { SC_HANGAR_BOOKMARKLET } from '../../shared/hangar';

export default function HangarImport({ userId, onImported }: { userId: number; onImported: () => void }) {
 const [json, setJson] = useState('');
 const [busy, setBusy] = useState(false);
 const [msg, setMsg] = useState<{ text: string; ok: boolean } | null>(null);
 const [setupOpen, setSetupOpen] = useState(false);
 // Own sharing state: whether a hangar exists and whether it's shared.
 const [hasHangar, setHasHangar] = useState(false);
 const [isPublic, setIsPublic] = useState(false);
 const [sharing, setSharing] = useState(false);

 const loadVisibility = () =>
 api
 .get<{ hasHangar: boolean; isPublic: boolean }>('/hangar/me/visibility')
 .then((v) => {
 setHasHangar(v.hasHangar);
 setIsPublic(v.isPublic);
 })
 .catch(() => {});

 useEffect(() => {
 void loadVisibility();
 // Re-check when the parent signals an import/clear happened.
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [userId]);

 const togglePublic = async (next: boolean) => {
 setSharing(true);
 setIsPublic(next); // optimistic
 try {
 await api.patch<{ isPublic: boolean }>('/hangar/visibility', { public: next });
 } catch {
 setIsPublic(!next); // revert on failure
 } finally {
 setSharing(false);
 }
 };

 const load = async () => {
 setBusy(true);
 setMsg(null);

```

**src/client/components/HangarImport.tsx**

```

 let parsed: unknown;
 try {
 parsed = JSON.parse(json);
 } catch {
 setMsg({ text: 'That doesn't look like the copied data ? re-run the bookmarklet and paste
again.', ok: false });
 setBusy(false);
 return;
 }
 try {
 const { itemCount } = await api.put<{ itemCount: number }>('/hangar', { items: parsed });
 setJson('');
 setMsg({ text: `Imported ${itemCount} items ? your hangar is updated below.`, ok: true });
 void loadVisibility();
 onImported();
 } catch (e) {
 setMsg({ text: e instanceof Error ? e.message : 'Import failed.', ok: false });
 } finally {
 setBusy(false);
 }
 };

 const clear = async () => {
 if (!window.confirm('Remove your imported hangar?')) return;
 setBusy(true);
 try {
 await api.del('/hangar');
 setMsg({ text: 'Hangar cleared.', ok: true });
 setHasHangar(false);
 setIsPublic(false);
 onImported();
 } finally {
 setBusy(false);
 }
 };

 return (
 <div className="hangar-import">
 <details className="hangar-setup" open={setupOpen} onToggle={(e) => setSetupOpen((e.target
as HTMLDetailsElement).open)}>
 <summary>How to import your hangar</summary>
 <ol className="hangar-setup-steps">

 One-time setup: create a bookmark named Grab SC hangar, and
paste this as its URL
 (right-click your bookmarks bar -> Add page -> paste in the URL field):
 <textarea className="hangar-bookmarklet" readOnly rows={3}
value={SC_HANGAR_BOOKMARKLET} onClick={(e) => e.currentTarget.select()} />
 <button type="button" className="ghost small" onClick={() => void
navigator.clipboard.writeText(SC_HANGAR_BOOKMARKLET).catch(() => {})}>
 Copy bookmarklet
 </button>


```

**src/client/components/HangarImport.tsx**

```

 Open your <a className="ext-link"
href="https://robertsspaceindustries.com/en/account/pledges" target="_blank" rel="noopener
noreferrer">RSI hangar,
 let it finish loading, and click the bookmark. It copies your hangar
to the clipboard (nothing is uploaded from your browser to anyone but this site).

 Paste it in the box below and press Import.

</details>

<div className="hangar-import-row">
 <textarea
 className="hangar-json"
 rows={2}
 placeholder="Paste your copied hangar here..."
 value={json}
 onChange={(e) => setJson(e.target.value)}
 disabled={busy}
 />
 <div className="hangar-import-actions">
 <button className="primary" disabled={busy || !json.trim()} onClick={() => void load()}>
 {busy ? 'Importing...' : 'Import'}
 </button>
 <button className="ghost small" disabled={busy} onClick={() => void clear()}
title="Remove your imported hangar">
 Clear
 </button>
 </div>
</div>

{msg && <p className={msg.ok ? 'muted small' : 'small warn'}>{msg.text}</p>}

{hasHangar && (
 <label className="hangar-share">
 <input
 type="checkbox"
 checked={isPublic}
 disabled={sharing}
 onChange={(e) => void togglePublic(e.target.checked)}
 />

 Show my hangar to other members

 {isPublic
 ? 'Members with permission to view hangars can see yours.'
 : 'Only you can see your hangar. Turn this on to share it with authorized
members.'}

 </label>
)}
</div>
);
}

```

**src/client/components/HangarImport.tsx**

**src/client/components/HangarView.tsx**

```

/**
 * A member's Star Citizen hangar ? the same categorised, searchable, image grid
 * we prototyped, reading the member's stored items. Shown on the member profile
 * when the SC module is enabled. Read-only; importing lives in HangarImport.
 */

import { useEffect, useMemo, useState } from 'react';
import { api } from '../lib/api';
import { useSession } from '../lib/session';
import {
 HANGAR_CATEGORIES,
 hangarImageUrl,
 hangarValueNum,
 visibleHangarItems,
 type HangarItem,
} from '../../shared/hangar';

interface HangarResponse {
 hangar: { items: HangarItem[]; importedAt: number } | null;
 self: boolean;
 canView: boolean;
 isPublic: boolean;
}

export default function HangarView({ userId, refreshKey = 0 }: { userId: number; refreshKey?: number }) {
 const { can } = useSession();
 const [items, setItems] = useState<HangarItem[] | null>(null);
 const [importedAt, setImportedAt] = useState<number | null>(null);
 const [meta, setMeta] = useState<{ self: boolean; canView: boolean; isPublic: boolean }>({
 self: false,
 canView: true,
 isPublic: false,
 });
 const [loading, setLoading] = useState(true);
 const [cat, setCat] = useState('all');
 const [q, setQ] = useState('');
 const [sort, setSort] = useState<{ k: string; dir: 1 | -1 }>({ k: 'name', dir: 1 });

 useEffect(() => {
 setLoading(true);
 api
 .get<HangarResponse>(`/hangar/${userId}`)
 .then((res) => {
 // Only display-worthy types (drops equipment/loot/coupons/etc.).
 setItems(res.hangar ? visibleHangarItems(res.hangar.items) : null);
 setImportedAt(res.hangar?.importedAt ?? null);
 setMeta({ self: res.self, canView: res.canView, isPublic: res.isPublic });
 })
 .catch(() => setItems(null))
 .finally(() => setLoading(false));
 }, [userId, refreshKey]);

 const rows = useMemo(() => {

```

**src/client/components/HangarView.tsx**

```

 if (!items) return [];
 const catTest = HANGAR_CATEGORIES.find((c) => c.key === cat)!.test;
 const needle = q.toLowerCase();
 const list = items
 .filter(catTest)
 .filter((i) => !needle || i.name.toLowerCase().includes(needle) ||
i.type.toLowerCase().includes(needle));
 return [...list].sort((a, b) => {
 let x: string | number = sort.k === 'valueNum' ? hangarValueNum(a) : (a[sort.k as keyof
HangarItem] ?? '');
 let y: string | number = sort.k === 'valueNum' ? hangarValueNum(b) : (b[sort.k as keyof
HangarItem] ?? '');
 if (typeof x === 'string') x = x.toLowerCase();
 if (typeof y === 'string') y = y.toLowerCase();
 return (x > y ? 1 : x < y ? -1 : 0) * sort.dir;
 });
 }, [items, cat, q, sort]);

 if (loading) return <div className="loading">Loading hangar...</div>;
 if (!items) {
 // A permission-holder looking at a member who hasn't shared their hangar.
 if (!meta.self && meta.canView === false) {
 return <p className="muted">This member keeps their hangar private.</p>;
 }
 return <p className="muted">No hangar imported yet.</p>;
 }

 const sortBy = (k: string) => setSort((s) => (s.k === k ? { k, dir: s.dir === 1 ? -1 : 1 } : {
k, dir: 1 }));
 const arrow = (k: string) => (sort.k === k ? (sort.dir === 1 ? ' ?' : ' ?') : '');

 // At-a-glance totals over the displayed items: how many ships, and the summed
 // pledge value (rough ? packages list their own price alongside their ships).
 const shipCount = items.filter((i) => i.type === 'ship').length;
 const totalValue = items.reduce((sum, i) => sum + Math.max(0, hangarValueNum(i)), 0);

 // Monetary value is the main incentive to inflate a self-reported hangar, so it
 // is hidden from the roster at large: owners always see their own; everyone else
 // needs the `hangar.value` permission (officer-only by default).
 const showValue = meta.self || can('hangar.value');

 return (
 <div className="hangar-view">
 <div className="hangar-toolbar">
 <div className="hangar-filters">
 {HANGAR_CATEGORIES.map((c) => (
 <button
 key={c.key}
 type="button"
 className={c.key === cat ? 'hangar-chip active' : 'hangar-chip'}
 onClick={() => setCat(c.key)}
 >
 {c.label}
 {items.filter(c.test).length}

```



**src/client/components/HangarView.tsx**

```

 </button>
)}}
 </div>
 <input
 className="hangar-search"
 placeholder="Search name or type..."
 value={q}
 onChange={(e) => setQ(e.target.value)}
 />
 </div>

 <div className="hangar-summary">

 {items.length} items

 {shipCount} ships

 {showValue && totalValue > 0 && (

 ${totalValue.toLocaleString(undefined, { maximumFractionDigits: 0 })}
value

)}
 </div>

 <div className="hangar-count muted small">
 Showing {rows.length} of {items.length}
 {importedAt ? ` ? imported ${new Date(importedAt * 1000).toLocaleDateString()}` : ''}
 </div>

 <p className="hangar-selfreport muted small" title="Imported by the member from their RSI
hangar; must not verify its contents.">
 ? Self-reported ? imported by the member, not verified.
 </p>

 <div className="hangar-table-wrap">
 <table className="hangar-table">
 <thead>
 <tr>
 <th>Preview</th>
 <th onClick={() => sortBy('name')}>Name{arrow('name')}</th>
 <th onClick={() => sortBy('type')}>Type{arrow('type')}</th>
 {showValue && <th onClick={() => sortBy('valueNum')}>Value{arrow('valueNum')}</th>}
 <th onClick={() => sortBy('availability')}>Availability{arrow('availability')}</th>
 </tr>
 </thead>
 <tbody>
 {rows.map((i, idx) => {
 const src = hangarImageUrl(i);
 return (
 <tr key={i.id || idx}>
 <td>{src && <img className="hangar-thumb" loading="lazy" src={src} alt=""
/>}</td>

```

**src/client/components/HangarView.tsx**

```
 <td className="hangar-name">{i.name}</td>
 <td>
 {i.type}
 </td>
 {showValue && <td>{i.value}</td>}
 <td className="muted small">{i.availability}</td>
 </tr>
);
 })}
</tbody>
</table>
{rows.length === 0 && <p className="muted small hangar-empty">No items match this
filter.</p>}
</div>
</div>
);
}
```

**src/client/components/modules.tsx**

```

/**
 * The module registry ? one renderer per module type from shared/layout.ts.
 * Each module is self-contained: it fetches its own data and renders a compact
 * card suitable for sitting in a layout column. A module that can't load (no
 * permission, empty data) fails quietly rather than breaking the page.
 *
 * These are the *display* forms shown on a laid-out page (e.g. the home page);
 * the full-page routes (/roster, /events, ...) remain the place to manage things.
 */

import { useEffect, useState, type ReactNode } from 'react';
import { Link } from 'react-router-dom';
import { api } from '../lib/api';
import { sanitizeHtml, sanitizePageHtml, excerptFromHtml } from '../lib/richtext';
import { memberName } from '../../../shared/names';
import { memberAvatar } from '../../../shared/avatar';
import { isAllowedEmbedSrc } from '../../../shared/embeds';
import type { ModuleType } from '../../../shared/layout';

type Config = Record<string, unknown>;

const str = (c: Config, k: string, d = ''): string => (typeof c[k] === 'string' ? (c[k] as string) : d);
const num = (c: Config, k: string, d: number): number => {
 const n = Number(c[k]);
 return Number.isFinite(n) ? n : d;
};

/** A link that stays inside the SPA for internal paths and opens externally otherwise. */
function SmartLink({ href, className, children }: { href: string; className?: string; children: ReactNode }) {
 if (href.startsWith('/')) {
 return (
 <Link to={href} className={className}>
 {children}
 </Link>
);
 }
 return (

 {children}

);
}

/** A small image gallery shared by the medals / war-records / games modules. */
function GalleryModule({
 title,
 endpoint,
 pick,
 limit,
 fallbackIcon,
}: {
 title: string;

```

**src/client/components/modules.tsx**

```

 endpoint: string;
 pick: (data: unknown) => { key: string | number; name: string; image: string | null; note?:
string }[];
 limit: number;
 fallbackIcon: string;
 }) {
 const [items, setItems] = useState<ReturnType<typeof pick> | null>(null);

 useEffect(() => {
 api
 .get<unknown>(endpoint)
 .then((data) => setItems(pick(data)))
 .catch(() => setItems([]));
 // pick/endpoint are stable per module instance.
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [endpoint]);

 if (items === null) return <ModuleCard title={title}><p
className="muted">Loading...</p></ModuleCard>;

 return (
 <ModuleCard title={title}>
 {items.length === 0 ? (
 <p className="muted">Nothing here yet.</p>
) : (
 <ul className="module-gallery">
 {items.slice(0, limit).map((it) => (
 <li key={it.key} className="module-gallery-item" title={it.note ? `${it.name} ?
${it.note}` : it.name}>
 {it.image ? (

) : (

 {fallbackIcon}

)}
 {it.name}

))}

)}
 </ModuleCard>
);
 }

 /** Shared card chrome so every module reads as part of one system. */
 function ModuleCard({
 title,
 action,
 children,
 }: {
 title?: string;
 action?: ReactNode;
 children: ReactNode;
 }) {

```

**src/client/components/modules.tsx**

```

})) {
 return (
 <section className="panel module">
 {(title || action) && (
 <header className="panel-head">
 {title ? <h2>{title}</h2> : }
 {action}
 </header>
)}
 {children}
 </section>
);
}

/* ----- *
 * Heading + rich text ? static content, no fetching.
 * ----- */

function HeadingModule({ config }: { config: Config }) {
 const text = str(config, 'text', 'Heading');
 const level = num(config, 'level', 2);
 const Tag = (level === 1 ? 'h1' : level === 3 ? 'h3' : 'h2') as 'h1' | 'h2' | 'h3';
 return <Tag className="module-heading">{text}</Tag>;
}

function TextModule({ config }: { config: Config }) {
 const html = sanitizeHtml(str(config, 'html'));
 return (
 <section className="panel module module-text">
 <div className="news-body" dangerouslySetInnerHTML={{ __html: html }} />
 </section>
);
}

/**
 * Hand-written HTML from a page author. Sanitized here at render time with the
 * wider page allow-list ? this render-time pass is the authoritative backstop
 * (the stored html is never trusted), so even a hand-crafted API write can't
 * execute. See sanitizePageHtml for exactly what survives.
 */
function HtmlModule({ config }: { config: Config }) {
 const html = sanitizePageHtml(str(config, 'html'));
 if (!html.trim()) return null;
 return (
 <section className="module module-html">
 <div className="news-body page-html" dangerouslySetInnerHTML={{ __html: html }} />
 </section>
);
}

/**
 * A video embed. `config.src` is produced by resolveEmbed inside sanitizeLayout,
 * but we re-verify it here against the origin allow-list before emitting an
 * iframe ? belt and suspenders. The frame is sandboxed and cross-origin, so the

```

**src/client/components/modules.tsx**

```

* provider's player runs boxed off from the page.
*/
function EmbedModule({ config }: { config: Config }) {
 const src = str(config, 'src');
 const title = str(config, 'title') || 'Embedded video';
 const ratio = str(config, 'ratio', '16:9') === '4:3' ? '4:3' : '16:9';

 if (!isAllowedEmbedSrc(src)) {
 return (
 <section className="panel module module-embed-empty">
 <p className="muted small">
 No video yet ? add a YouTube, Twitch, Vimeo, or Streamable link in the editor.
 </p>
 </section>
);
 }

 return (
 <div className="module module-embed" data-ratio={ratio}>
 <iframe
 src={src}
 title={title}
 loading="lazy"
 allowFullScreen
 sandbox="allow-scripts allow-same-origin allow-presentation allow-popups"
 allow="autoplay; fullscreen; picture-in-picture; encrypted-media; clipboard-write"
 referrerPolicy="strict-origin-when-cross-origin"
 />
 </div>
);
}

/* ----- */
* Data modules
* ----- */

interface NewsPost {
 id: number;
 slug: string;
 title: string;
 excerpt: string | null;
 body: string;
 pinned: boolean;
 publishedAt: number | null;
 author: string | null;
}

function NewsModule({ config }: { config: Config }) {
 const title = str(config, 'title', 'Latest News');
 const limit = num(config, 'limit', 5);
 const [posts, setPosts] = useState<NewsPost[] | null>(null);

 useEffect(() => {
 api

```

**src/client/components/modules.tsx**

```

 .get<{ posts: NewsPost[] }>('/news')
 .then(({ posts }) => setPosts(posts))
 .catch(() => setPosts([]));
 }, []);

 if (posts === null) return <ModuleCard title={title}><p
className="muted">Loading...</p></ModuleCard>;

 return (
 <ModuleCard
 title={title}
 action={
 <Link className="btn-link" to="/news">
 All news
 </Link>
 }
 >
 {posts.length === 0 ? (
 <p className="muted">Nothing has been posted yet.</p>
) : (
 <ul className="news-feed">
 {posts.slice(0, limit).map((p) => (
 <li key={p.id} className="news-item">
 <Link to={` /news/${p.slug}`} className="news-item-link">
 <div className="news-item-head">
 {p.pinned && ? Pinned}
 <h3>{p.title}</h3>
 </div>
 <p className="news-excerpt">{p.excerpt || excerptFromHtml(p.body)}</p>
 <div className="muted small news-meta">
 {p.author ?? 'Unknown'}
 {p.publishedAt && <? {new Date(p.publishedAt * 1000).toLocaleDateString()}</?>}
 </div>
 </Link>

))}

)}
 </ModuleCard>
);
}

interface RosterMember {
 id: number;
 discordId: string;
 username: string;
 globalName: string | null;
 displayName: string | null;
 avatar: string | null;
 profileImageUrl: string | null;
 rankName: string | null;
}

function RosterModule({ config }: { config: Config }) {

```

**src/client/components/modules.tsx**

```

const title = str(config, 'title', 'Roster');
const limit = num(config, 'limit', 12);
const [state, setState] = useState<{ members: RosterMember[]; total: number } | null>(null);
const [denied, setDenied] = useState(false);

useEffect(() => {
 api
 .get<{ members: RosterMember[]; total: number }>(`/members?limit=${limit}&offset=0`)
 .then(setState)
 .catch(() => {
 // Most commonly a 403 for a viewer without roster.view ? hide quietly.
 setDenied(true);
 setState({ members: [], total: 0 });
 });
}, [limit]);

if (denied) return null;
if (state === null) return <ModuleCard title={title}><p
className="muted">Loading...</p></ModuleCard>;

return (
 <ModuleCard
 title={title}
 action={
 <Link className="btn-link" to="/roster">
 {state.total > state.members.length ? `All ${state.total}` : 'Full roster'}
 </Link>
 }
 >
 {state.members.length === 0 ? (
 <p className="muted">No members yet.</p>
) : (
 <ul className="roster-mini">
 {state.members.map((m) => (
 <li key={m.id}>
 <Link to={`/${members}/${m.id}`} className="roster-mini-item">

 {memberName(m)}
 {m.rankName && {m.rankName}}
 </Link>

))}

)}
 </ModuleCard>
);
}

interface EventItem {
 id: number;
 title: string;
 startsAt: number;
 endsAt: number;
 location: string;
}

```



**src/client/components/modules.tsx**

```

}

function EventsModule({ config }: { config: Config }) {
 const title = str(config, 'title', 'Upcoming Events');
 const limit = num(config, 'limit', 5);
 const [events, setEvents] = useState<EventItem[] | null>(null);
 const [denied, setDenied] = useState(false);

 useEffect(() => {
 api
 .get<{ events: EventItem[] }>('/events')
 .then(({ events }) => setEvents(events))
 .catch(() => {
 // Most commonly a 403 for a viewer without events.view ? hide quietly.
 setDenied(true);
 setEvents([]);
 });
 }, []);

 if (denied) return null;
 if (events === null) return <ModuleCard title={title}><p
className="muted">Loading...</p></ModuleCard>;

 const now = Math.floor(Date.now() / 1000);
 const upcoming = events.filter((e) => e.endsAt >= now).slice(0, limit);

 return (
 <ModuleCard
 title={title}
 action={
 <Link className="btn-link" to="/events">
 All events
 </Link>
 }
 >
 {upcoming.length === 0 ? (
 <p className="muted">Nothing scheduled.</p>
) : (
 <ul className="events-mini">
 {upcoming.map((e) => (
 <li key={e.id} className="events-mini-item">
 <div className="events-mini-date">

 {new Date(e.startsAt * 1000).toLocaleString(undefined, { month: 'short' })}

 {new Date(e.startsAt * 1000).getDate()}
 </div>
 <div className="events-mini-body">
 <div className="events-mini-title">{e.title}</div>
 <div className="muted small">
 {new Date(e.startsAt * 1000).toLocaleString(undefined, {
 hour: 'numeric',
 minute: '2-digit',
 })}
 </div>
 </div>

))}

)}
)
}

```

**src/client/components/modules.tsx**

```

 {e.location ? ` ? ${e.location}` : ''}
 </div>
 </div>

)}}

)}
</ModuleCard>
);
}

function ImageModule({ config }: { config: Config }) {
 const url = str(config, 'url');
 if (!url) return null;
 const href = str(config, 'href');
 const alt = str(config, 'alt');
 const caption = str(config, 'caption');
 const img = ;
 return (
 <figure className="module module-image">
 {href ? <SmartLink href={href}></SmartLink> : img}
 {caption && <figcaption className="muted small module-image-caption">{caption}</figcaption>}
 </figure>
);
}

function ButtonModule({ config }: { config: Config }) {
 const href = str(config, 'href');
 if (!href) return null;
 const label = str(config, 'label', 'Learn more');
 const primary = str(config, 'style', 'primary') === 'primary';
 return (
 <div className="module module-button">
 <SmartLink href={href} className={primary ? 'btn-cta primary' : 'btn-cta'}>
 {label}
 </SmartLink>
 </div>
);
}

function DividerModule() {
 return <hr className="module module-divider" />;
}

/**
 * A hero CTA link. Unlike SmartLink, a root-relative path that targets a *server*
 * route (/api/..., /media/...) must be a real navigation, not an SPA <Link> ? the
 * router would otherwise try to match it as a page and 404. External links open
 * in a new tab; everything else internal stays in the SPA.
 */
function HeroCta({ href, label, primary }: { href: string; label: string; primary: boolean }) {
 const cls = primary ? 'hero-btn hero-btn-primary' : 'hero-btn hero-btn-secondary';
 const external = /^https?:\/\//i.test(href);
 const serverRoute = href.startsWith('/api/') || href.startsWith('/media/');

```

**src/client/components/modules.tsx**

```

 if (external) {
 return (

 {label}

);
 }
 if (serverRoute) {
 return (

 {label}

);
 }
 return (
 <Link className={cls} to={href}>
 {label}
 </Link>
);
 }

 interface HeroCard {
 icon: string;
 title: string;
 tag: string;
 body: string;
 }

 function HeroModule({ config }: { config: Config }) {
 const eyebrow = str(config, 'eyebrow');
 const headline = str(config, 'headline');
 const subhead = str(config, 'subhead');
 const primaryLabel = str(config, 'primaryLabel');
 const primaryHref = str(config, 'primaryHref');
 const secondaryLabel = str(config, 'secondaryLabel');
 const secondaryHref = str(config, 'secondaryHref');
 const chips = (Array.isArray(config.chips) ? config.chips : []).filter(
 (c): c is string => typeof c === 'string' && c.length > 0,
);
 const cards = (Array.isArray(config.cards) ? config.cards : [])
 .map((c) => {
 const o = (c && typeof c === 'object' ? c : {}) as Record<string, unknown>;
 return {
 icon: typeof o.icon === 'string' ? o.icon : '',
 title: typeof o.title === 'string' ? o.title : '',
 tag: typeof o.tag === 'string' ? o.tag : '',
 body: typeof o.body === 'string' ? o.body : '',
 } as HeroCard;
 })
 .filter((c) => c.title || c.body);

 return (
 <div className="module module-hero">
 <section className="hero-panel">

```

**src/client/components/modules.tsx**

```

 {eyebrow && {eyebrow}}
 {headline && <h1 className="hero-headline">{headline}</h1>}
 {subhead && <p className="hero-subhead">{subhead}</p>}
 {(primaryLabel && primaryHref) || (secondaryLabel && secondaryHref) ? (
 <div className="hero-cta">
 {primaryLabel && primaryHref && (
 <HeroCta href={primaryHref} label={primaryLabel} primary />
)}
 {secondaryLabel && secondaryHref && (
 <HeroCta href={secondaryHref} label={secondaryLabel} primary={false} />
)}
 </div>
) : null}
 {chips.length > 0 && (
 <div className="hero-chips">
 {chips.map((c, i) => (

 {c}

))}
 </div>
)}
 </section>

 {cards.length > 0 && (
 <div className="hero-grid">
 {cards.map((c, i) => (
 <div className="hero-card" key={i}>
 {c.icon && (

 {c.icon}

)}
 {c.title && <h3 className="hero-card-title">{c.title}</h3>}
 {c.tag && <p className="hero-card-tag">{c.tag}</p>}
 {c.body && <p className="hero-card-body">{c.body}</p>}
 </div>
))}
 </div>
)}
</div>
);
}

function MedalsModule({ config }: { config: Config }) {
 return (
 <GalleryModule
 title={str(config, 'title', 'Medals')}
 endpoint="/medals"
 limit={num(config, 'limit', 12)}
 fallbackIcon="???"
 pick={(d) =>
 ((d as { medals?: { id: number; name: string; imageUrl: string | null; awardCount?: number }[] }).medals ?? []).map(

```

**src/client/components/modules.tsx**

```

 (m) => ({ key: m.id, name: m.name, image: m.imageUrl ?? null, note: m.awardCount ?
`${m.awardCount} awarded` : undefined })),
)
 }
/>
);
}

function WarRecordsModule({ config }: { config: Config }) {
 return (
 <GalleryModule
 title={str(config, 'title', 'War Records')}
 endpoint="/warrecords"
 limit={num(config, 'limit', 12)}
 fallbackIcon="?"
 pick={(d) =>
 ((d as { warRecords?: { id: number; name: string; imageUrl: string | null; gameName?:
string | null }[] })).warRecords ?? []).map(
 (w) => ({ key: w.id, name: w.name, image: w.imageUrl ?? null, note: w.gameName ??
undefined })),
)
 }
 />
);
}

function GamesModule({ config }: { config: Config }) {
 return (
 <GalleryModule
 title={str(config, 'title', 'Games We Play')}
 endpoint="/games"
 limit={num(config, 'limit', 12)}
 fallbackIcon="?"
 pick={(d) =>
 ((d as { games?: { id: number; name: string; iconUrl: string | null; active?: boolean }[]
}).games ?? []).
 .filter((g) => g.active !== false)
 .map((g) => ({ key: g.id, name: g.name, image: g.iconUrl ?? null })))
 }
 />
);
}

/* -----
 * Registry ? the single map the renderer and editor read.
 * ----- */

export const MODULE_RENDERERS: Record<ModuleType, (props: { config: Config }) => ReactNode> = {
 heading: HeadingModule,
 text: TextModule,
 html: HtmlModule,
 hero: HeroModule,
 image: ImageModule,
 button: ButtonModule,

```

**src/client/components/modules.tsx**

```
 embed: EmbedModule,
 divider: DividerModule,
 news: NewsModule,
 roster: RosterModule,
 events: EventsModule,
 medals: MedalsModule,
 warrecords: WarRecordsModule,
 games: GamesModule,
};
```

**src/client/components/OrgChartView.tsx**

```

/**
 * Read-only leadership chart: absolutely-positioned member boxes with SVG
 * connectors drawn manager -> report. The designer reuses the geometry constants
 * and edge-path helper so the editor and the public view line up exactly.
 */

import type { CSSProperties } from 'react';
import { Link } from 'react-router-dom';
import { memberName } from '../../../shared/names';
import { memberAvatar } from '../../../shared/avatar';
import type { OrgChart } from '../../../shared/orgchart';

export interface ChartMember {
 id: number;
 discordId: string;
 username: string;
 globalName: string | null;
 displayName: string | null;
 avatar: string | null;
 profileImageUrl: string | null;
 rankName: string | null;
 rankSortOrder: number | null;
}

export const BOX_W = 190;
export const BOX_H = 62;
const PAD = 48;

/** A smooth vertical S-curve from a manager's bottom to a report's top. */
export function edgePath(x1: number, y1: number, x2: number, y2: number): string {
 const my = (y1 + y2) / 2;
 return `M ${x1} ${y1} C ${x1} ${my}, ${x2} ${my}, ${x2} ${y2}`;
}

export function chartSize(nodes: { x: number; y: number }[]): { width: number; height: number } {
 return {
 width: Math.max(400, ...nodes.map((n) => n.x + BOX_W)) + PAD,
 height: Math.max(260, ...nodes.map((n) => n.y + BOX_H)) + PAD,
 };
}

export default function OrgChartView({
 chart,
 members,
 linkMembers = false,
}: {
 chart: OrgChart;
 members: ChartMember[];
 /** Wrap boxes in a link to the member's profile (only where the viewer can open it). */
 linkMembers?: boolean;
}) {
 const byId = new Map(members.map((m) => [m.id, m]));
 const nodes = chart.nodes.filter((n) => byId.has(n.memberId));

```

**src/client/components/OrgChartView.tsx**

```

 if (nodes.length === 0) {
 return <p className="empty">The leadership chart hasn't been set up yet.</p>;
 }

 const { width, height } = chartSize(nodes);
 const nodeById = new Map(nodes.map((n) => [n.memberId, n]));

 return (
 <div className="org-scroll">
 <div className="org-canvas" style={{ width, height }}>
 <svg className="org-edges" width={width} height={height} aria-hidden>
 {chart.edges.map((e, i) => {
 const a = nodeById.get(e.from);
 const b = nodeById.get(e.to);
 if (!a || !b) return null;
 return (
 <path
 key={i}
 className="org-edge"
 d={edgePath(a.x + BOX_W / 2, a.y + BOX_H, b.x + BOX_W / 2, b.y)}
 />
);
 })}
 </svg>

 {nodes.map((n) => {
 const m = byId.get(n.memberId)!;
 const style: CSSProperties = { left: n.x, top: n.y, width: BOX_W, height: BOX_H };
 const inner = (
 <>

 <div className="org-box-text">
 {memberName(m)}
 {m.rankName && {m.rankName}}
 </div>
 </>
);
 return linkMembers ? (
 <Link key={n.memberId} to={`/${members}/${n.memberId}`} className="org-box"
style={style}>
 {inner}
 </Link>
) : (
 <div key={n.memberId} className="org-box" style={style}>
 {inner}
 </div>
);
 })}
 </div>
 </div>
);
 }
}

```



**src/client/components/PageRenderer.tsx**

```

/**
 * Renders a stored PageLayout: a stack of rows, each a responsive 12-unit grid
 * of columns, each column an ordered list of modules. Columns collapse to a
 * single readable stack on narrow screens (see .page-grid in styles.css). The
 * same component renders the live page and the editor preview, so what an admin
 * arranges is exactly what members see.
 *
 * A module may carry a `visibleToRole` gate; on the live page it is skipped for
 * viewers who don't hold that role (god bypasses). In the editor preview
 * (`showHidden`) those modules still render, tagged with a badge, so the admin
 * can see and manage them ? pass `roles` there so the badge can name the role.
 */

import type { CSSProperties } from 'react';
import type { PageLayout } from '../../shared/layout';
import { useSession } from '../../lib/session';
import { MODULE_RENDERERS } from './modules';

export default function PageRenderer({
 layout,
 showHidden = false,
 roles,
}: {
 layout: PageLayout;
 showHidden?: boolean;
 /** All roles, by id -> name; only needed to label gated modules in the preview. */
 roles?: { id: number; name: string }[];
}) {
 const { viewer } = useSession();

 const canSeeRole = (roleId: number): boolean =>
 !!viewer && (viewer.isGod || viewer.roles.some((r) => r.id === roleId));
 const roleName = (roleId: number): string =>
 roles?.find((r) => r.id === roleId)?.name ?? 'Restricted';

 if (!layout.rows.length) {
 return <p className="empty">This page has no content yet.</p>;
 }

 return (
 <div className="page-rows">
 {layout.rows.map((row) => (
 <div className="page-grid" key={row.id}>
 {row.columns.map((col) => (
 <div
 className="page-col"
 key={col.id}
 style={{ '--span': col.span } as CSSProperties}
 >
 {col.modules.map((m) => {
 const Renderer = MODULE_RENDERERS[m.type];
 if (!Renderer) return null;

 const gated = m.visibleToRole !== null ? !canSeeRole(m.visibleToRole) : false;

```

**src/client/components/PageRenderer.tsx**

```
 if (gated && !showHidden) return null;

 const node = <Renderer config={m.config} />;
 if (gated && showHidden) {
 const label = roleName(m.visibleToRole!);
 return (
 <div className="module-gated" key={m.id}>

 ? {label}

 {node}
 </div>
);
 }
 return <div key={m.id}>{node}</div>;
 }}}
</div>
)}}
</div>
)}}
</div>
);
}
```

**src/client/components/ReassignDialog.tsx**

```

/**
 * Inline confirm panel for deleting a rank or role that members still hold.
 *
 * Deleting one out from under its members is a destructive, negative action, so
 * this makes the admin decide ? explicitly ? where those members go (another
 * rank/role, or the "none" fallback) and record a reason, which lands in the
 * audit log alongside demotions and revocations. Shown only when there are
 * members to move; a member-less rank/role deletes with a plain confirm.
 */

import { useState } from 'react';

export interface ReassignOption {
 value: number;
 label: string;
}

export default function ReassignDialog({
 title,
 count,
 options,
 defaultValue,
 noneLabel,
 busy,
 onConfirm,
 onCancel,
}: {
 title: string;
 /** How many members currently hold the thing being deleted. */
 count: number;
 options: ReassignOption[];
 /** Pre-selected target (e.g. the next-lower rank), or null for the "none" fallback. */
 defaultValue: number | null;
 /** Label for the empty option ? "No rank (unranked)" / "Remove from everyone". */
 noneLabel: string;
 busy: boolean;
 onConfirm: (target: number | null, reason: string) => void;
 onCancel: () => void;
}) {
 const [target, setTarget] = useState<string>(defaultValue != null ? String(defaultValue) : '');
 const [reason, setReason] = useState('');

 return (
 <div className="reassign-dialog panel">
 <h3>{title}</h3>
 <p className="muted small">
 {count} member{count === 1 ? '' : 's'} currently {count === 1 ? 'holds' : 'hold'} it.
 Choose
 where {count === 1 ? 'it' : 'they'} should go, then confirm.
 </p>

 <label className="reassign-field">
 Move members to
 <select value={target} onChange={(e) => setTarget(e.target.value)} disabled={busy}>

```

**src/client/components/ReassignDialog.tsx**

```

 <option value="">{noneLabel}</option>
 {options.map((o) => (
 <option key={o.value} value={o.value}>
 {o.label}
 </option>
))}
 </select>
</label>

<label className="reassign-field">
 Reason
 <input
 value={reason}
 placeholder="Why it's being deleted (recorded in the log)"
 onChange={(e) => setReason(e.target.value)}
 disabled={busy}
 autoFocus
 />
</label>

<div className="reassign-actions">
 <button onClick={onCancel} disabled={busy}>
 Cancel
 </button>
 <button
 className="danger"
 disabled={busy || !reason.trim()}
 onClick={() => onConfirm(target === '' ? null : Number(target), reason.trim())}
 >
 Delete & move
 </button>
</div>
</div>
);
}

```

**src/client/components/RichTextEditor.tsx**

```
/**
 * The WYSIWYG editor for news posts (TipTap).
 *
 * Emits sanitized HTML on every change (see richtext.ts) so the value the
 * parent holds ? and later saves ? is already clean; it's sanitized again on
 * render. Uncontrolled after mount: remount it with a `key` to load different
 * content (e.g. when switching which post is being edited).
 */

import { useEditor, EditorContent, type Editor } from '@tiptap/react';
import StarterKit from '@tiptap/starter-kit';
import { sanitizeHtml } from '../../lib/richtext';

function ToolButton({
 editor,
 onClick,
 active,
 disabled,
 label,
 title,
}: {
 editor: Editor;
 onClick: () => void;
 active?: boolean;
 disabled?: boolean;
 label: string;
 title: string;
}) {
 return (
 <button
 type="button"
 className={active ? 'rte-tool active' : 'rte-tool'}
 title={title}
 // Keep focus in the document so the command applies to the selection.
 onMouseDown={(e) => e.preventDefault()}
 onClick={() => {
 onClick();
 editor.commands.focus();
 }}
 disabled={disabled}
 >
 {label}
 </button>
);
}

export default function RichTextEditor({
 value,
 onChange,
 disabled,
}: {
 value: string;
 onChange: (html: string) => void;
 disabled?: boolean;
```

**src/client/components/RichTextEditor.tsx**

```

})) {
 const editor = useEditor({
 extensions: [
 StarterKit.configure({
 link: {
 openOnClick: false,
 HTMLAttributes: { rel: 'noopener noreferrer nofollow', target: '_blank' },
 },
 }),
],
 content: value,
 editable: !disabled,
 onUpdate: ({ editor }) => onChange(sanitizeHtml(editor.getHTML())),
 });

 if (!editor) return null;

 const setLink = () => {
 const prev = editor.getAttributes('link').href as string | undefined;
 const url = window.prompt('Link URL (leave blank to remove):', prev ?? 'https://');
 if (url === null) return; // cancelled
 if (url.trim() === '') {
 editor.chain().focus().extendMarkRange('link').unsetLink().run();
 return;
 }
 editor.chain().focus().extendMarkRange('link').setLink({ href: url.trim() }).run();
 };

 return (
 <div className="rte">
 <div className="rte-toolbar">
 <ToolButton editor={editor} label="B" title="Bold" active={editor.isActive('bold')}
disabled={disabled} onClick={() => editor.chain().focus().toggleBold().run()} />
 <ToolButton editor={editor} label="I" title="Italic" active={editor.isActive('italic')}
disabled={disabled} onClick={() => editor.chain().focus().toggleItalic().run()} />
 <ToolButton editor={editor} label="S" title="Strikethrough"
active={editor.isActive('strike')} disabled={disabled} onClick={() =>
editor.chain().focus().toggleStrike().run()} />

 <ToolButton editor={editor} label="H1" title="Heading 1"
active={editor.isActive('heading', { level: 1 })} disabled={disabled} onClick={() =>
editor.chain().focus().toggleHeading({ level: 1 }).run()} />
 <ToolButton editor={editor} label="H2" title="Heading 2"
active={editor.isActive('heading', { level: 2 })} disabled={disabled} onClick={() =>
editor.chain().focus().toggleHeading({ level: 2 }).run()} />
 <ToolButton editor={editor} label="H3" title="Heading 3"
active={editor.isActive('heading', { level: 3 })} disabled={disabled} onClick={() =>
editor.chain().focus().toggleHeading({ level: 3 }).run()} />

 <ToolButton editor={editor} label="• List" title="Bullet list"
active={editor.isActive('bulletList')} disabled={disabled} onClick={() =>
editor.chain().focus().toggleBulletList().run()} />
 <ToolButton editor={editor} label="1. List" title="Numbered list"
active={editor.isActive('orderedList')} disabled={disabled} onClick={() =>

```

**src/client/components/RichTextEditor.tsx**

```
editor.chain().focus().toggleOrderedList().run()) />
 <ToolButton editor={editor} label="?" title="Quote" active={editor.isActive('blockquote')}
 disabled={disabled} onClick={() => editor.chain().focus().toggleBlockquote().run()} />
 <ToolButton editor={editor} label="{ }" title="Code block"
active={editor.isActive('codeBlock')} disabled={disabled} onClick={() =>
editor.chain().focus().toggleCodeBlock().run()} />

 <ToolButton editor={editor} label="?" title="Link" active={editor.isActive('link')}
disabled={disabled} onClick={setLink} />
 </div>
 <EditorContent editor={editor} className="rte-content" />
</div>
);
}
```

**src/client/components/ScVerify.tsx**

```

/**
 * Star Citizen account verification ? badge + (on your own profile) the flow.
 *
 * Proves a member controls an RSI account by having them paste a one-time code
 * into their public RSI Short Bio; the server fetches the profile, confirms the
 * code, and records org membership. Verification is informational ? it gates
 * nothing. Everyone viewing the SC section sees the badge; only the owner sees
 * the claim/confirm controls.
 */

import { useCallback, useEffect, useState } from 'react';
import { api } from '../lib/api';

interface VerifyStatus {
 verified: boolean;
 rsiHandle: string | null;
 orgSid: string | null;
 orgRank: string | null;
 inOrg: boolean;
 orgVisible: boolean;
 verifiedAt: number | null;
 pending: { code: string; handle: string } | null;
}

const RSI_PROFILE_SETTINGS = 'https://robertsspaceindustries.com/account/profile';

export default function ScVerify({ userId, isSelf }: { userId: number; isSelf: boolean }) {
 const [status, setStatus] = useState<VerifyStatus | null>(null);
 const [handle, setHandle] = useState('');
 // When already verified, "Re-verify" flips this on to show the handle entry
 // form again (rather than silently reusing a possibly-empty handle).
 const [reverifying, setReverifying] = useState(false);
 const [busy, setBusy] = useState(false);
 const [msg, setMsg] = useState<{ text: string; ok: boolean } | null>(null);

 const load = useCallback(() => {
 api
 .get<VerifyStatus>(`/sc/verify/${userId}`)
 .then((v) => {
 setStatus(v);
 // Prefill the handle from the pending challenge or the verified handle so
 // the entry field is never blank when re-verifying.
 if (v.pending?.handle) setHandle(v.pending.handle);
 else if (v.rsiHandle) setHandle(v.rsiHandle);
 })
 .catch(() => setStatus(null));
 }, [userId]);

 useEffect(() => load(), [load]);

 const start = async () => {
 setBusy(true);
 setMsg(null);
 try {

```



**src/client/components/ScVerify.tsx**

```

 await api.post('/sc/verify/start', { handle: handle.trim() });
 load();
 } catch (e) {
 setMsg({ text: e instanceof Error ? e.message : 'Could not start verification.', ok: false
});
 } finally {
 setBusy(false);
 }
};

const confirm = async () => {
 setBusy(true);
 setMsg(null);
 try {
 const res = await api.post<{ ok: boolean; error?: string }>('/sc/verify/confirm', {});
 if (!res.ok) {
 setMsg({ text: res.error ?? 'Verification failed.', ok: false });
 } else {
 setMsg({ text: 'Verified! Your RSI account is confirmed.', ok: true });
 setReverifying(false);
 load();
 }
 } catch (e) {
 setMsg({ text: e instanceof Error ? e.message : 'Verification failed.', ok: false });
 } finally {
 setBusy(false);
 }
};

const remove = async () => {
 if (!window.confirm('Remove your RSI verification?')) return;
 setBusy(true);
 setMsg(null);
 try {
 await api.del('/sc/verify');
 setHandle('');
 setReverifying(false);
 load();
 } finally {
 setBusy(false);
 }
};

// Abandon an in-flight challenge. Keeps an existing verification (clears only
// the pending code); if there's no prior verification, drops the empty row.
const cancelPending = async () => {
 setBusy(true);
 setMsg(null);
 try {
 if (status?.verified) await api.post('/sc/verify/cancel', {});
 else await api.del('/sc/verify');
 setReverifying(false);
 load();
 } finally {

```

**src/client/components/ScVerify.tsx**

```

 setBusy(false);
 }
};

if (!status) return null;

// The badge (shown to everyone viewing the section).
const badge = status.verified ? (
 <div className="sc-verify-badge verified" title={`RSI account verified${status.verifiedAt ? `
on ${new Date(status.verifiedAt * 1000).toLocaleDateString()}` : ''}`}>
 <span className="sc-verify-check" aria-hidden=?
 Verified RSI: {status.rsiHandle}
 {status.inOrg && status.orgSid && (
 <span className="sc-verify-org"? {status.orgSid}{status.orgRank ? ` (${status.orgRank})`
: ''}
)}
 {!status.inOrg && status.orgVisible && <span className="sc-verify-org muted"? not in this
org}
 </div>
) : (
 <div className="sc-verify-badge unverified">Unverified RSI account</div>
);

// Non-owners just see the badge.
if (!isSelf) return <div className="sc-verify">{badge}</div>;

return (
 <div className="sc-verify">
 {badge}

 {status.pending ? (
 <div className="sc-verify-flow">
 <p className="muted small">
 Add this code to your{' '}
 <a className="ext-link" href={RSI_PROFILE_SETTINGS} target="_blank" rel="noopener
noreferrer">
 RSI Short Bio
 {' '}
 (Account -> Profile), Save, then confirm:
 </p>
 <code className="sc-verify-code">{status.pending.code}</code>
 <div className="sc-verify-actions">
 <button type="button" className="primary small" disabled={busy} onClick={() => void
confirm()}>
 {busy ? 'Checking...' : 'I've added it ? Verify'}
 </button>
 <button type="button" className="ghost small" disabled={busy} onClick={() => void
cancelPending()}>
 Cancel
 </button>
 </div>
 <p className="muted small">
 Verifying {status.pending.handle}. You can remove the code from your
bio once verified.

```

**src/client/components/ScVerify.tsx**

```

 </p>
 </div>
) : status.verified && !reverifying ? (
 <div className="sc-verify-actions">
 <button
 type="button"
 className="ghost small"
 disabled={busy}
 onClick={() => {
 setReverifying(true);
 setMsg(null);
 }}
 >
 Re-verify
 </button>
 <button type="button" className="ghost small danger" disabled={busy} onClick={() => void
remove()}>
 Remove
 </button>
 </div>
) : (
 <div className="sc-verify-flow">
 <p className="muted small">
 {reverifying ? 'Re-verify your RSI account.' : 'Prove you own your RSI account to earn
a verified badge.'}
 </p>
 <div className="sc-verify-start">
 <input
 type="text"
 placeholder="Your RSI handle"
 value={handle}
 disabled={busy}
 onChange={(e) => setHandle(e.target.value)}
 />
 <button type="button" className="primary small" disabled={busy || !handle.trim()}
onClick={() => void start()}>
 Get code
 </button>
 {reverifying && (
 <button
 type="button"
 className="ghost small"
 disabled={busy}
 onClick={() => {
 setReverifying(false);
 setHandle(status.rsiHandle ?? '');
 setMsg(null);
 }}
 >
 Cancel
 </button>
)}
 </div>
 </div>
)

```

**src/client/components/ScVerify.tsx**

```
)}

 {msg && <p className={msg.ok ? 'muted small' : 'small warn'}>{msg.text}</p>}
 </div>
);
}
```

**src/client/components/SiteNav.tsx**

```

/**
 * Renders the admin-arranged top menu (see shared/nav.ts).
 *
 * Two gates decide what shows, both applied here so the stored tree can stay
 * complete: a built-in's own permission (Events/Admin), which a menu entry can
 * never bypass, and the optional per-entry role the admin set. Categories become
 * click-to-open dropdowns on desktop and expand inline inside the mobile
 * hamburger. Empty categories (nothing the viewer may see) hide themselves.
 */

import { useEffect, useRef, useState } from 'react';
import { NavLink } from 'react-router-dom';
import { useSession } from '../lib/session';
import { canAccessAdmin } from '../lib/adminSections';
import { BUILTIN_TARGETS, navItemHref, navItemLabel, type NavItem } from '../../shared/nav';

/** Close the mobile menu after a real navigation (not a category toggle). */
type NavProps = { items: NavItem[]; onNavigate: () => void };

function useVisibility() {
 const { viewer, can } = useSession();

 const roleOk = (roleId?: number): boolean =>
 roleId == null || (!!viewer && (viewer.isGod || viewer.roles.some((r) => r.id === roleId)));

 const targetOk = (item: NavItem): boolean => {
 if (item.kind === 'builtin' && item.target) {
 const b = BUILTIN_TARGETS[item.target];
 if (!b) return false;
 if (b.admin) return canAccessAdmin(can);
 if (b.permission) return can(b.permission);
 }
 return true;
 };

 const linkVisible = (item: NavItem): boolean => targetOk(item) && roleOk(item.visibleToRole);
 return { roleOk, linkVisible };
}

function Leaf({ item, onNavigate }: { item: NavItem; onNavigate: () => void }) {
 const href = navItemHref(item);
 if (!href) return null;
 const label = navItemLabel(item);
 const external = item.kind === 'url' && /^https?:\/\//i.test(href);

 if (external) {
 return (

 {label}

);
 }
 return (
 <NavLink to={href} end={href === '/'} onClick={onNavigate} className={item.kind === 'builtin'

```

**src/client/components/SiteNav.tsx**

```

&& item.target === 'admin' ? 'nav-admin' : undefined}>
 {label}
</NavLink>
);
}

function Category({
 group,
 onNavigate,
}: {
 group: NavItem;
 onNavigate: () => void;
}) {
 const { linkVisible } = useVisibility();
 const [open, setOpen] = useState(false);
 const ref = useRef<HTMLDivElement>(null);

 useEffect(() => {
 if (!open) return;
 const onDown = (e: MouseEvent) => {
 if (ref.current && !ref.current.contains(e.target as Node)) setOpen(false);
 };
 const onKey = (e: KeyboardEvent) => e.key === 'Escape' && setOpen(false);
 document.addEventListener('mousedown', onDown);
 document.addEventListener('keydown', onKey);
 return () => {
 document.removeEventListener('mousedown', onDown);
 document.removeEventListener('keydown', onKey);
 };
 }, [open]);

 const kids = (group.children ?? []).filter(linkVisible);
 if (kids.length === 0) return null;

 const navigateAndClose = () => {
 setOpen(false);
 onNavigate();
 };

 return (
 <div className={open ? 'nav-cat open' : 'nav-cat'} ref={ref}>
 <button
 type="button"
 className="nav-cat-trigger"
 aria-haspopup="menu"
 aria-expanded={open}
 // Don't let the click bubble to the mobile <nav>, which would close the
 // whole hamburger before the submenu can open.
 onClick={(e) => {
 e.stopPropagation();
 setOpen(o => !o);
 }}
 >
 {navItemLabel(group)}
 </button>
 </div>
);
}

```

**src/client/components/SiteNav.tsx**

```

 ?

 </button>
 <div className="nav-cat-menu" role="menu">
 {kids.map((k) => (
 <Leaf key={k.id} item={k} onNavigate={navigateAndClose} />
))}
 </div>
</div>
);
}

export default function SiteNav({ items, onNavigate }: NavProps) {
 const { linkVisible, roleOk } = useVisibility();

 return (
 <>
 {items.map((item) => {
 if (item.type === 'group') {
 if (!roleOk(item.visibleToRole)) return null;
 return <Category key={item.id} group={item} onNavigate={onNavigate} />;
 }
 if (!linkVisible(item)) return null;
 return <Leaf key={item.id} item={item} onNavigate={onNavigate} />;
 })}
 </>
);
}
```

**src/client/components/UsageBar.tsx**

```

/**
 * Free-tier usage gauges, shown across the top of the admin panel.
 *
 * Storage always renders; the daily request/query rates render once a
 * read-only Cloudflare Analytics token is configured (otherwise a short setup
 * hint takes their place). Each bar shifts from green toward red as it nears
 * its limit. Non-critical: if the endpoint fails, the bar just hides itself.
 */

import { useEffect, useState } from 'react';
import { MorphIcon } from 'morphicons/react';
import { ChevronDown, ChevronRight } from 'lucide';
import { api } from '../lib/api';

interface Metric {
 used: number;
 limit: number;
}

interface Usage {
 storage: {
 r2: { bytes: number; objects: number; truncated: boolean; limitBytes: number } | null;
 d1: { bytes: number; limitBytes: number };
 };
 rates: {
 windowHours: number;
 workersRequests: Metric;
 d1RowsRead: Metric;
 d1RowsWritten: Metric;
 } | null;
 ratesConfigured: boolean;
 ratesError: string | null;
}

function formatBytes(n: number): string {
 if (n < 1024) return `${n} B`;
 const units = ['KB', 'MB', 'GB', 'TB'];
 let v = n;
 let i = -1;
 do {
 v /= 1024;
 i += 1;
 } while (v >= 1024 && i < units.length - 1);
 return `${v.toFixed(v < 10 ? 1 : 0)} ${units[i]}`;
}

const compact = new Intl.NumberFormat('en', { notation: 'compact', maximumFractionDigits: 1 });
const formatNum = (n: number) => compact.format(n);

/** 0% -> green, 100% -> red, interpolated through amber. */
function gaugeColor(pct: number): string {
 return `hsl(${Math.max(0, 1 - pct) * 140}, 68%, 46%)`;
}

function Gauge({

```



**src/client/components/UsageBar.tsx**

```

 label,
 sub,
 used,
 limit,
 format,
 }: {
 label: string;
 sub?: string;
 used: number;
 limit: number;
 format: (n: number) => string;
 }) {
 const pct = limit > 0 ? Math.min(1, used / limit) : 0;
 const color = gaugeColor(pct);
 const shown = pct * 100;
 return (
 <div className="usage-gauge">
 <div className="usage-gauge-head">

 {label}
 {sub && ? {sub}}

 {format(used)} / {format(limit)}

 </div>
 <div className="usage-track">
 <div className="usage-fill" style={{ width: `${Math.max(shown, 1.5)}%`, background: color
 </div>
 <div className="usage-pct" style={{ color }}>
 {shown < 1 ? shown.toFixed(1) : Math.round(shown)}%
 </div>
 </div>
);
 }

export default function UsageBar() {
 const [data, setData] = useState<Usage | null>(null);
 const [failed, setFailed] = useState(false);
 const [connectOpen, setConnectOpen] = useState(false);

 useEffect(() => {
 api
 .get<Usage>('/usage')
 .then(setData)
 .catch(() => setFailed(true));
 }, []);

 if (failed) return null;
 if (!data) return <div className="usage-bar usage-loading">Loading usage...</div>;

 return (
 <div className="usage-bar">

```

**src/client/components/UsageBar.tsx**

```

 <div className="usage-gauges">
 {data.storage.r2 && (
 <Gauge label="R2 storage" used={data.storage.r2.bytes}
limit={data.storage.r2.limitBytes} format={formatBytes} />
)}
 <Gauge label="D1 storage" used={data.storage.d1.bytes} limit={data.storage.d1.limitBytes}
format={formatBytes} />

 {data.rates ? (
 <>
 <Gauge label="Requests" sub="24h" used={data.rates.workersRequests.used}
limit={data.rates.workersRequests.limit} format={formatNum} />
 <Gauge label="D1 rows read" sub="24h" used={data.rates.d1RowsRead.used}
limit={data.rates.d1RowsRead.limit} format={formatNum} />
 <Gauge label="D1 rows written" sub="24h" used={data.rates.d1RowsWritten.used}
limit={data.rates.d1RowsWritten.limit} format={formatNum} />
 </>
) : (
 <details
 className="usage-connect"
 onToggle={(e) => setConnectOpen((e.target as HTMLDetailsElement).open)}
 >
 <summary>
 <MorphIcon
 className="disclosure-caret"
 icon={connectOpen ? ChevronDown : ChevronRight}
 size={14}
 aria-hidden
 />
 Connect Analytics for live request & query rates
 </summary>
 <ol className="small muted">
 Create a Cloudflare API token with Account Analytics:
Read.

 Paste it, with your Account ID, under{' '}
 Settings -> Analytics ? no redeploy needed.

 </details>
)}
 </div>

 {data.ratesError && (
 <div className="usage-error small">Live rates unavailable: {data.ratesError}</div>
)}
 </div>
);
}

```

**src/client/index.html**

```
<!doctype html>
<html lang="en">
 <head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
 <title>ClanTek</title>
 <link rel="manifest" href="/manifest.webmanifest" />
 <link rel="icon" type="image/svg+xml" href="/icon.svg" />
 <link rel="apple-touch-icon" href="/icon.svg" />
 <meta name="theme-color" content="#0f1115" />
 <meta name="apple-mobile-web-app-capable" content="yes" />
 <meta name="apple-mobile-web-app-title" content="ClanTek" />
 </head>
 <body>
 <div id="root"></div>
 <script type="module" src="/main.tsx"></script>
 </body>
</html>
```

**src/client/lib/action.tsx**

```

/**
 * Shared handling for a mutating action and its three outcomes:
 * - throws -> red error alert
 * - returns a string -> grey success notice
 * - returns {warning} -> amber warning (saved here, but a downstream push,
 * usually Discord, was refused)
 *
 * Keeps the editors from each re-implementing busy/error/notice/warning state.
 */

import { useState, type ReactNode } from 'react';
import { ApiError } from './api';

export type ActionResult = string | { warning: string } | void | null;

export function useAction(reload?: () => Promise<unknown>) {
 const [busy, setBusy] = useState(false);
 const [error, setError] = useState<string | null>(null);
 const [notice, setNotice] = useState<string | null>(null);
 const [warning, setWarning] = useState<string | null>(null);

 async function run(fn: () => Promise<ActionResult>) {
 setBusy(true);
 setError(null);
 setNotice(null);
 setWarning(null);
 try {
 const result = await fn();
 if (reload) await reload();
 if (typeof result === 'string') setNotice(result);
 else if (result && typeof result === 'object' && 'warning' in result) {
 setWarning(result.warning);
 }
 } catch (err) {
 setError(err instanceof ApiError ? err.message : 'Something went wrong.');
```

**src/client/lib/action.tsx**

```
 {error && <div className="alert">{error}</div>}
 {warning && <div className="warn-alert">{warning}</div>}
 {notice && <div className="notice">{notice}</div>}
 </>
);
}
```

**src/client/lib/adminSections.ts**

```

/**
 * The admin menu tree ? metadata only (keys, labels, permissions), with no
 * component imports, so lightweight consumers (the account menu, the primary
 * nav) can gate on "can this viewer reach any admin tool" without pulling the
 * heavy admin pages ? and TipTap ? into the initial bundle. Admin.tsx maps tab
 * keys to components; this file never imports a page.
 *
 * Structure: GROUPS (Content / People / Settings) -> ITEMS (a sidebar entry, a
 * route at /admin/:item) -> TABS (one or more tools shown as tabs inside the
 * item, each gated on its own permission). An item with a single tab shows no
 * tab bar. A tab is visible if the viewer holds its permission; an item is
 * visible if any tab is; a group is visible if any item is.
 */

import type { Permission } from '../../shared/permissions';

export interface AdminTab {
 /** Maps to a component in Admin.tsx's renderer map, and to /admin/:item/:tab. */
 key: string;
 label: string;
 permission: Permission;
}

export interface AdminItem {
 /** Route segment: /admin/:key */
 key: string;
 label: string;
 tabs: AdminTab[];
}

export interface AdminGroup {
 key: string;
 label: string;
 items: AdminItem[];
}

export const ADMIN_GROUPS: AdminGroup[] = [
 {
 key: 'content',
 label: 'Content',
 items: [
 { key: 'pages', label: 'Pages', tabs: [{ key: 'pages', label: 'Pages', permission:
'pages.manage' }] },
 { key: 'navigation', label: 'Navigation', tabs: [{ key: 'navigation', label: 'Navigation',
permission: 'pages.manage' }] },
 { key: 'news', label: 'News', tabs: [{ key: 'news', label: 'News', permission: 'news.create'
}] },
],
 },
 {
 key: 'people',
 label: 'People',
 items: [
 {

```

**src/client/lib/adminSections.ts**

```

 key: 'ranks-roles',
 label: 'Ranks & Roles',
 tabs: [
 { key: 'ranks', label: 'Ranks', permission: 'ranks.manage' },
 { key: 'roles', label: 'Roles', permission: 'roles.manage' },
],
 },
 {
 key: 'medals-records',
 label: 'Medals & Records',
 tabs: [
 { key: 'medals', label: 'Medals', permission: 'medals.manage' },
 { key: 'warrecords', label: 'War Records', permission: 'warrecords.manage' },
 { key: 'games', label: 'Games', permission: 'games.manage' },
],
 },
 { key: 'org-chart', label: 'Org Chart', tabs: [{ key: 'orgchart', label: 'Org Chart',
permission: 'ranks.manage' }] },
],
},
{
 key: 'settings',
 label: 'Settings',
 items: [
 { key: 'identity', label: 'Identity & Discord', tabs: [{ key: 'identity', label: 'Identity &
Discord', permission: 'settings.manage' }] },
 { key: 'bot', label: 'Bot Settings', tabs: [{ key: 'announcements', label: 'Bot Settings',
permission: 'settings.manage' }] },
 { key: 'analytics', label: 'Analytics', tabs: [{ key: 'analytics', label: 'Analytics',
permission: 'settings.manage' }] },
 { key: 'modules', label: 'Modules', tabs: [{ key: 'modules', label: 'Modules', permission:
'settings.manage' }] },
],
 {
 key: 'appearance',
 label: 'Theme & Branding',
 tabs: [
 { key: 'theme', label: 'Theme', permission: 'theme.manage' },
 { key: 'branding', label: 'Branding', permission: 'settings.manage' },
 { key: 'seo', label: 'SEO & Sharing', permission: 'settings.manage' },
 { key: 'footer', label: 'Footer', permission: 'settings.manage' },
],
 },
 { key: 'logs', label: 'Logs', tabs: [{ key: 'audit', label: 'Logs', permission: 'audit.view'
}] },
],
},
];

```

```
type Can = (permission: Permission) => boolean;
```

```

/** The tree pruned to what a viewer may see: tabs -> items -> groups they can reach. */
export function visibleAdminGroups(can: Can): AdminGroup[] {
 return ADMIN_GROUPS.map((g) => ({
 ...g,

```

**src/client/lib/adminSections.ts**

```
 items: g.items
 .map((i) => ({ ...i, tabs: i.tabs.filter((t) => can(t.permission)) }))
 .filter((i) => i.tabs.length > 0),
 })).filter((g) => g.items.length > 0);
}

/** Which group an item belongs to (for tagging recently-viewed records). */
export function groupKeyForItem(itemKey: string): string | undefined {
 return ADMIN_GROUPS.find((g) => g.items.some((i) => i.key === itemKey)).key;
}

export function canAccessAdmin(can: Can): boolean {
 return visibleAdminGroups(can).length > 0;
}
```



**src/client/lib/api.ts**

```

export class ApiError extends Error {
 constructor(
 readonly status: number,
 message: string,
 readonly body?: unknown,
) {
 super(message);
 this.name = 'ApiError';
 }
}

async function request<T>(path: string, init?: RequestInit): Promise<T> {
 const res = await fetch(`/api${path}`, {
 ...init,
 headers: {
 'Content-Type': 'application/json',
 ...init?.headers,
 },
 credentials: 'same-origin',
 });

 const text = await res.text();
 const body = text ? JSON.parse(text) : null;

 if (!res.ok) {
 throw new ApiError(res.status, body?.error ?? `Request failed (${res.status})`, body);
 }
 return body as T;
}

/**
 * Multipart upload ? kept separate from request() because the browser must set
 * its own multipart Content-Type (with boundary); forcing application/json here
 * would break the upload.
 */
async function upload<T>(path: string, file: File): Promise<T> {
 const form = new FormData();
 form.append('file', file);
 const res = await fetch(`/api${path}`, { method: 'POST', body: form, credentials: 'same-origin'
});
 const text = await res.text();
 const body = text ? JSON.parse(text) : null;
 if (!res.ok) {
 throw new ApiError(res.status, body?.error ?? `Upload failed (${res.status})`, body);
 }
 return body as T;
}

export const api = {
 get: <T,>(path: string) => request<T>(path),
 post: <T,>(path: string, data?: unknown) =>
 request<T>(path, { method: 'POST', body: data ? JSON.stringify(data) : undefined }),
 patch: <T,>(path: string, data: unknown) =>
 request<T>(path, { method: 'PATCH', body: JSON.stringify(data) }),

```

**src/client/lib/api.ts**

```
put: <T,>(path: string, data: unknown) =>
 request<T>(path, { method: 'PUT', body: JSON.stringify(data) }),
del: <T,>(path: string, data?: unknown) =>
 request<T>(path, { method: 'DELETE', body: data ? JSON.stringify(data) : undefined }),
upload,
};
```

**src/client/lib/branding.tsx**

```

/**
 * Site branding ? the uploaded header logo and how big it sits, stored under the
 * public `site` settings key. Mirrors ThemeProvider: it loads once on boot (so
 * the logo is present before anyone signs in, on the login screen too), exposes
 * a live `preview` for the admin's size slider, and a `save` that persists.
 *
 * `logoSize` is the logo's *expanded* height in px (its size at the top of the
 * page, overhanging the bar). When the page is scrolled the header shrinks it to
 * a fixed collapsed height that fits inside the bar ? that part is pure CSS.
 */

import { createContext, useCallback, useContext, useEffect, useState, type ReactNode } from
'react';
import { api } from './api';

export interface Branding {
 logoUrl: string;
 logoSize: number;
 /** Uploaded browser-tab icon (favicon). Blank falls back to the default icon. */
 faviconUrl: string;
}

export const DEFAULT_BRANDING: Branding = { logoUrl: '', logoSize: 88, faviconUrl: '' };

interface BrandingValue {
 branding: Branding;
 /** Applies immediately (live header preview) without persisting. */
 preview: (next: Branding) => void;
 save: (next: Branding) => Promise<void>;
}

const BrandingContext = createContext<BrandingValue | null>(null);

function coerce(raw: Partial<Branding> | undefined): Branding {
 const size = Math.round(Number(raw?.logoSize));
 return {
 logoUrl: typeof raw?.logoUrl === 'string' ? raw.logoUrl : '',
 logoSize: Number.isFinite(size) ? Math.min(200, Math.max(40, size)) :
DEFAULT_BRANDING.logoSize,
 faviconUrl: typeof raw?.faviconUrl === 'string' ? raw.faviconUrl : '',
 };
}

export function BrandingProvider({ children }: { children: ReactNode }) {
 const [branding, setBranding] = useState<Branding>(DEFAULT_BRANDING);

 useEffect(() => {
 api
 .get<{ site: Partial<Branding> }>('/settings/site')
 .then(({ site }) => setBranding(coerce(site)))
 .catch(() => {
 /* Not configured yet ? defaults (no logo) are fine. */
 });
 }, []);
}

```

**src/client/lib/branding.tsx**

```

// Keep the live tab icon in sync with the current (previewed or saved) favicon,
// so an admin sees the change at once and SPA navigation keeps the right icon.
// A blank value restores the default /icon.svg baked into index.html.
useEffect(() => {
 const href = branding.faviconUrl || '/icon.svg';
 for (const rel of ['icon', 'apple-touch-icon']) {
 let link = document.head.querySelector<HTMLLinkElement>(`link[rel="${rel}"]`);
 if (!link) {
 link = document.createElement('link');
 link.rel = rel;
 document.head.appendChild(link);
 }
 link.href = href;
 // Let the browser sniff the format of an uploaded icon.
 if (branding.faviconUrl) link.removeAttribute('type');
 }
}, [branding.faviconUrl]);

const preview = useCallback((next: Branding) => setBranding(coerce(next)), []);

const save = useCallback(async (next: Branding) => {
 const clean = coerce(next);
 const { site } = await api.put<{ site: Branding }>('/settings/site', { site: clean });
 setBranding(coerce(site ?? clean));
}, []);

return (
 <BrandingContext.Provider value={{ branding, preview, save
}}>{children}</BrandingContext.Provider>
);
}

export function useBranding(): BrandingValue {
 const ctx = useContext(BrandingContext);
 if (!ctx) throw new Error('useBranding must be used inside <BrandingProvider>');
 return ctx;
}

```

**src/client/lib/modules.ts**

```

/**
 * Which optional game modules are enabled, so UI (e.g. the hangar on a profile)
 * can show/hide itself. Fetched once and cached process-wide ? the flags rarely
 * change and every profile page would otherwise re-request them.
 */

import { useEffect, useState } from 'react';
import { api } from './api';

export interface ModuleFlags {
 starcitizen: boolean;
}

const DEFAULT: ModuleFlags = { starcitizen: false };

let cache: ModuleFlags | null = null;
let inflight: Promise<ModuleFlags> | null = null;

function fetchModules(): Promise<ModuleFlags> {
 if (cache) return Promise.resolve(cache);
 if (!inflight) {
 inflight = api
 .get<{ modules: ModuleFlags }>('/settings/modules')
 .then((r) => (cache = r.modules))
 .catch(() => DEFAULT);
 }
 return inflight;
}

export function useModules(): ModuleFlags {
 const [flags, setFlags] = useState<ModuleFlags>(cache ?? DEFAULT);
 useEffect(() => {
 let live = true;
 void fetchModules().then((v) => live && setFlags(v));
 return () => {
 live = false;
 };
 }, []);
 return flags;
}

/** Drop the cache after an admin toggles modules, so the next read re-fetches. */
export function clearModulesCache() {
 cache = null;
 inflight = null;
}

/* ----- *
 * Star Citizen module config ? org SID + the per-feature kill switches. Cached
 * the same way; profiles read it to hide a killed feature without a reload.
 * ----- */

export interface ScConfig {
 orgSid: string;
}

```

**src/client/lib/modules.ts**

```
 hangarEnabled: boolean;
 verifyEnabled: boolean;
 }

// Defaults are ON so the UI shows while loading; the server is the authority
// (killed routes 404 regardless of what the client briefly renders).
const SC_DEFAULT: ScConfig = { orgSid: '', hangarEnabled: true, verifyEnabled: true };

let scCache: ScConfig | null = null;
let scInflight: Promise<ScConfig> | null = null;

function fetchScConfig(): Promise<ScConfig> {
 if (scCache) return Promise.resolve(scCache);
 if (!scInflight) {
 scInflight = api
 .get<{ sc: ScConfig }>('/settings/sc')
 .then((r) => (scCache = r.sc))
 .catch(() => SC_DEFAULT);
 }
 return scInflight;
}

export function useScConfig(): ScConfig {
 const [cfg, setCfg] = useState<ScConfig>(scCache ?? SC_DEFAULT);
 useEffect(() => {
 let live = true;
 void fetchScConfig().then((v) => live && setCfg(v));
 return () => {
 live = false;
 };
 }, []);
 return cfg;
}

/** Drop the SC-config cache after an admin edits it (e.g. flips a kill switch). */
export function clearScConfigCache() {
 scCache = null;
 scInflight = null;
}
```

**src/client/lib/recent.ts**

```

/**
 * "Recently viewed" records, per browser. Pages call recordRecent() when a
 * specific record is opened (a news post, a page, a member, ...); the admin
 * sidebar shows the latest few under the matching group header so you can jump
 * straight back. Stored in localStorage ? personal, device-local, no server.
 */

import { useEffect } from 'react';

export interface RecentEntry {
 /** Admin group key this record belongs under: 'content' | 'people' | 'settings'. */
 group: string;
 label: string;
 /** Where clicking it goes (any in-app path). */
 to: string;
 at: number;
}

const KEY = 'ct_recent_v1';
const MAX = 30;

function read(): RecentEntry[] {
 try {
 const raw = localStorage.getItem(KEY);
 const list = raw ? (JSON.parse(raw) as RecentEntry[]) : [];
 return Array.isArray(list) ? list : [];
 } catch {
 return [];
 }
}

function write(list: RecentEntry[]): void {
 try {
 localStorage.setItem(KEY, JSON.stringify(list.slice(0, MAX)));
 // Let other mounted components (the sidebar) refresh.
 window.dispatchEvent(new Event('ct-recent'));
 } catch {
 /* storage full or unavailable ? non-fatal */
 }
}

export function recordRecent(entry: Omit<RecentEntry, 'at'>): void {
 if (!entry.label || !entry.to) return;
 const list = read().filter((e) => e.to !== entry.to);
 list.unshift({ ...entry, at: Date.now() });
 write(list);
}

export function getRecent(group?: string, limit = 4): RecentEntry[] {
 const list = read();
 return (group ? list.filter((e) => e.group === group) : list).slice(0, limit);
}

/**

```

**src/client/lib/recent.ts**

```
* Record a record view once it's known (label truthy). Safe to call every render;
* it only writes when `to`/`label` change.
*/
export function useRecordRecent(entry: { group: string; label: string; to: string } | null): void
{
 const key = entry ? `${entry.to}|${entry.label}` : '';
 useEffect(() => {
 if (entry && entry.label) recordRecent(entry);
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [key]);
}
```



**src/client/lib/richtext.ts**

```

/**
 * Sanitization for news post HTML.
 *
 * Posts are authored in a WYSIWYG editor (TipTap) that emits HTML, and that
 * HTML is stored in news.body. Stored markup is NEVER trusted: it's sanitized
 * here both before it's saved and again every time it's rendered, so a payload
 * that somehow reached the database (e.g. a hand-crafted API call) still can't
 * execute when displayed.
 *
 * The allow-list matches what the editor can actually produce ? nothing more.
 */

import DOMPurify from 'dompurify';

const ALLOWED_TAGS = [
 'p', 'br', 'hr',
 'h1', 'h2', 'h3',
 'strong', 'b', 'em', 'i', 's', 'u', 'code', 'pre',
 'blockquote',
 'ul', 'ol', 'li',
 'a',
];

const ALLOWED_ATTR = ['href', 'target', 'rel'];

export function sanitizeHtml(html: string): string {
 return DOMPurify.sanitize(html, {
 ALLOWED_TAGS,
 ALLOWED_ATTR,
 // Only http(s) and mailto links; blocks javascript: and data: URIs.
 ALLOWED_URI_REGEXP: /^(?:https?:|mailto:)/i,
 });
}

/**
 * The wider allow-list for the "HTML block" module. A page author with
 * pages.manage may hand-write markup, so this permits layout/table/media tags a
 * news post never needs ? but the security floor is identical to news: DOMPurify
 * still strips <script>, every on* handler, <iframe>/<object>/<embed>, <form>,
 * <style>, and (via FORBID_ATTR) inline `style=` attributes. Videos are NOT done
 * through raw <iframe> here ? they go through the separate, origin-allowlisted
 * Video-embed module. URLs are limited to http(s), mailto, and root-relative
 * ("/media/...") paths, so no javascript:/data: URI can ride in on href/src.
 */
const PAGE_ALLOWED_TAGS = [
 ...ALLOWED_TAGS,
 'h4', 'h5', 'h6',
 'div', 'span', 'section', 'article', 'header', 'footer', 'figure', 'figcaption',
 'img',
 'table', 'thead', 'tbody', 'tfoot', 'tr', 'th', 'td', 'caption', 'colgroup', 'col',
 'dl', 'dt', 'dd',
 'sub', 'sup', 'small', 'mark', 'ins', 'del', 'abbr', 'kbd', 'samp', 'var', 'time',
];

```

**src/client/lib/richtext.ts**

```

const PAGE_ALLOWED_ATTR = [
 'href', 'target', 'rel',
 'src', 'alt', 'title', 'width', 'height', 'loading', 'decoding',
 'class', 'colspan', 'rowspan', 'span', 'datetime',
];

export function sanitizePageHtml(html: string): string {
 return DOMPurify.sanitize(html, {
 ALLOWED_TAGS: PAGE_ALLOWED_TAGS,
 ALLOWED_ATTR: PAGE_ALLOWED_ATTR,
 // http(s), mailto, and root-relative ("/..."), but never protocol-relative ("//").
 ALLOWED_URI_REGEXP: /^(?:https?:|mailto:|\/(?:!\\/))/i,
 // Inline CSS can exfiltrate (background:url()) and enables clickjacking overlays.
 FORBID_ATTR: ['style'],
 FORBID_TAGS: ['style'],
 });
}

// One-time hardening applied to every sanitize pass: force safe rel on any link
// that opens a new tab (reverse-tabnabbing), and make author images lazy/low-risk.
// Runs in the browser only (DOMPurify needs a DOM); harmless for news markup.
let hooksInstalled = false;
if (!hooksInstalled && typeof DOMPurify.addHook === 'function') {
 hooksInstalled = true;
 DOMPurify.addHook('afterSanitizeAttributes', (node) => {
 const el = node as unknown as Element;
 if (el.tagName === 'A' && el.getAttribute('target') === '_blank') {
 el.setAttribute('rel', 'noopener noreferrer');
 }
 if (el.tagName === 'IMG') {
 el.setAttribute('loading', 'lazy');
 el.setAttribute('decoding', 'async');
 el.setAttribute('referrerpolicy', 'no-referrer');
 }
 });
}

/**
 * A plain-text snippet for feed cards when a post has no explicit excerpt.
 * Strips all markup and collapses whitespace, then truncates on a word boundary.
 */
export function excerptFromHtml(html: string, max = 180): string {
 const text = DOMPurify.sanitize(html, { ALLOWED_TAGS: [], ALLOWED_ATTR: [] })
 .replace(/\s+/g, ' ')
 .trim();
 if (text.length <= max) return text;
 return text.slice(0, text.lastIndexOf(' ', max)).trimEnd() + '...';
}

```

**src/client/lib/session.tsx**

```

import { createContext, useContext, useEffect, useState, type ReactNode } from 'react';
import { api } from './api';
import { can, type Permission, type Viewer } from '../../shared/permissions';

interface SessionValue {
 viewer: Viewer | null;
 siteName: string;
 loading: boolean;
 /** Same check the server enforces ? used to hide UI, never to secure it. */
 can: (permission: Permission) => boolean;
 refresh: () => Promise<void>;
 logout: () => Promise<void>;
}

const SessionContext = createContext<SessionValue | null>(null);

export function SessionProvider({ children }: { children: ReactNode }) {
 const [viewer, setViewer] = useState<Viewer | null>(null);
 const [siteName, setSiteName] = useState('ClanTek');
 const [loading, setLoading] = useState(true);

 async function refresh() {
 try {
 const data = await api.get<{ viewer: Viewer | null; siteName: string }>('/me');
 setViewer(data.viewer);
 setSiteName(data.siteName);
 } catch {
 setViewer(null);
 } finally {
 setLoading(false);
 }
 }

 useEffect(() => {
 void refresh();
 }, []);

 async function logout() {
 await api.post('/auth/logout');
 setViewer(null);
 }

 return (
 <SessionContext.Provider
 value={{
 viewer,
 siteName,
 loading,
 can: (permission) => can(viewer, permission),
 refresh,
 logout,
 }}
 >
 {children}
 </SessionContext.Provider>
);
}

```

**src/client/lib/session.tsx**

```
 </SessionContext.Provider>
);
}

export function useSession(): SessionValue {
 const ctx = useContext(SessionContext);
 if (!ctx) throw new Error('useSession must be used inside <SessionProvider>');
 return ctx;
}
```

**src/client/lib/theme.tsx**

```

/**
 * The modern replacement for the original `templates` and `header` tables.
 *
 * 2003: font face/size/color rows were concatenated into `` tags and
 * echoed inline on every page, alongside IE-only scrollbar colors.
 * Now: the same idea, as CSS custom properties applied to :root. The admin
 * edits tokens, every component reacts, and nothing renders raw HTML.
 */

import { createContext, useCallback, useContext, useEffect, useState, type ReactNode } from
'react';
import { api } from './api';

export type ThemeTokens = Record<string, string>;

export const DEFAULT_THEME: ThemeTokens = {
 '--color-bg': '#0f1115',
 '--color-surface': '#171a21',
 '--color-border': '#262b36',
 '--color-text': '#e6e8ec',
 '--color-muted': '#9aa3b2',
 '--color-accent': '#c0392b',
 '--color-accent-text': '#ffffff',
 '--font-body': 'system-ui, sans-serif',
 '--font-display': 'system-ui, sans-serif',
 '--radius': '8px',
 // Header menu alignment: flex-start (left) | center | flex-end (right).
 '--nav-justify': 'flex-start',
};

interface ThemeValue {
 tokens: ThemeTokens;
 /** Applies immediately for live preview without persisting. */
 preview: (tokens: ThemeTokens) => void;
 save: (tokens: ThemeTokens) => Promise<void>;
 reset: () => void;
}

const ThemeContext = createContext<ThemeValue | null>(null);

function apply(tokens: ThemeTokens) {
 const root = document.documentElement;
 for (const [key, value] of Object.entries(tokens)) {
 // Only set custom properties; a stray key can't inject arbitrary CSS.
 if (key.startsWith('--')) root.style.setProperty(key, value);
 }
}

export function ThemeProvider({ children }: { children: ReactNode }) {
 const [tokens, setTokens] = useState<ThemeTokens>(DEFAULT_THEME);

 useEffect(() => {
 apply(DEFAULT_THEME);
 api

```

**src/client/lib/theme.tsx**

```

 .get<{ theme: ThemeTokens }>('/settings/theme')
 .then(({ theme }) => {
 const merged = { ...DEFAULT_THEME, ...theme };
 setTokens(merged);
 apply(merged);
 })
 .catch(() => {
 /* Not configured yet ? defaults are already applied. */
 });
 }, []);

 const preview = useCallback((next: ThemeTokens) => {
 setTokens(next);
 apply(next);
 }, []);

 const save = useCallback(async (next: ThemeTokens) => {
 await api.put('/settings/theme', { theme: next });
 setTokens(next);
 apply(next);
 }, []);

 const reset = useCallback(() => {
 setTokens(DEFAULT_THEME);
 apply(DEFAULT_THEME);
 }, []);

 return (
 <ThemeContext.Provider value={{ tokens, preview, save, reset }}>
 {children}
 </ThemeContext.Provider>
);
}

export function useTheme(): ThemeValue {
 const ctx = useContext(ThemeContext);
 if (!ctx) throw new Error('useTheme must be used inside <ThemeProvider>');
 return ctx;
}

```

**src/client/main.tsx**

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App';
import { SessionProvider } from './lib/session';
import { ThemeProvider } from './lib/theme';
import { BrandingProvider } from './lib/branding';
import './styles.css';

createRoot(document.getElementById('root')).render(
 <StrictMode>
 <BrowserRouter>
 <ThemeProvider>
 <BrandingProvider>
 <SessionProvider>
 <App />
 </SessionProvider>
 </BrandingProvider>
 </ThemeProvider>
 </BrowserRouter>
 </StrictMode>,
);

// Register the PWA service worker (offline shell + installable). Best-effort:
// on localhost dev and unsupported browsers this simply no-ops. Skipped for the
// Vite dev server, whose module URLs the SW's asset caching shouldn't touch.
if ('serviceWorker' in navigator && !import.meta.env.DEV) {
 window.addEventListener('load', () => {
 navigator.serviceWorker.register('/sw.js').catch(() => {});
 });
}
```

**src/client/pages/AccountSettings.tsx**

```
/**
 * Personal account settings. Today this is "API access": a member mints and
 * revokes personal access tokens for the mobile/native app (or scripts). The
 * raw token is shown exactly once, right after it's created ? after that only
 * its label and short prefix are ever visible.
 */

import { useEffect, useState } from 'react';
import { api } from '../lib/api';
import { useSession } from '../lib/session';

interface ApiToken {
 id: number;
 label: string;
 prefix: string;
 createdAt: number;
 lastUsedAt: number | null;
 expiresAt: number | null;
}

interface CreatedToken extends ApiToken {
 token: string;
}

function when(ts: number | null): string {
 if (!ts) return '?';
 return new Date(ts * 1000).toLocaleDateString(undefined, { year: 'numeric', month: 'short', day: 'numeric' });
}

export default function AccountSettings() {
 const { viewer } = useSession();
 const [tokens, setTokens] = useState<ApiToken[] | null>(null);
 const [label, setLabel] = useState('');
 const [busy, setBusy] = useState(false);
 const [error, setError] = useState<string | null>(null);
 // The one-time full token value, shown until the member dismisses it.
 const [fresh, setFresh] = useState<CreatedToken | null>(null);
 const [copied, setCopied] = useState(false);

 const load = () =>
 api
 .get<{ tokens: ApiToken[] }>('/auth/tokens')
 .then(({ tokens }) => setTokens(tokens))
 .catch(() => setTokens([]));

 useEffect(() => {
 void load();
 }, []);

 async function create() {
 setBusy(true);
 setError(null);
 try {
```



**src/client/pages/AccountSettings.tsx**

```

 const created = await api.post<CreatedToken>('/auth/tokens', { label: label.trim() ||
'Mobile app' });
 setFresh(created);
 setCopied(false);
 setLabel('');
 await load();
 } catch (e) {
 setError(e instanceof Error ? e.message : 'Could not create the token.');
```

} finally {
 setBusy(false);
 }
 }
}

async function revoke(id: number, name: string) {
 if (!window.confirm(`Revoke "\${name}"? Any app or script using it will stop working
immediately.`)) return;
 try {
 await api.del(`/auth/tokens/\${id}`);
 if (fresh?.id === id) setFresh(null);
 await load();
 } catch {
 setError('Could not revoke the token.');

}
 }
}

async function copyFresh() {
 if (!fresh) return;
 try {
 await navigator.clipboard.writeText(fresh.token);
 setCopied(true);
 } catch {
 /\* Clipboard blocked ? the value is selectable in the box regardless. \*/
 }
}

return (
 <section className="panel account-settings">
 <header className="panel-head">
 <div>
 <h2>Account</h2>
 <p className="muted">Signed in as {viewer ? viewer.username : ''}. Manage access for
apps and scripts below.</p>
 </div>
 </header>

 <h3 className="account-subhead">API access</h3>
 <p className="muted">
 A personal access token lets the mobile app (or a script) sign in as you. It carries
exactly your
 permissions. Treat it like a password ? anyone with it can act as you until you revoke it.
 </p>

 {error && <div className="notice error">{error}</div>}

**src/client/pages/AccountSettings.tsx**

```

 {fresh && (
 <div className="token-reveal">
 Copy your new token now ? it won't be shown again.
 <div className="token-reveal-row">
 <code className="token-value">{fresh.token}</code>
 <button type="button" className="primary" onClick={copyFresh}>
 {copied ? 'Copied ?' : 'Copy'}
 </button>
 </div>
 <button type="button" className="ghost" onClick={() => setFresh(null)}>
 Done
 </button>
 </div>
)}

 <div className="token-create">
 <input
 type="text"
 value={label}
 maxLength={60}
 placeholder="Name this token (e.g. "My iPhone")"
 onChange={(e) => setLabel(e.target.value)}
 disabled={busy}
 />
 <button type="button" className="primary" onClick={() => void create()} disabled={busy}>
 {busy ? 'Creating...' : 'Create token'}
 </button>
 </div>

 {tokens === null ? (
 <p className="muted">Loading...</p>
) : tokens.length === 0 ? (
 <p className="muted">No tokens yet.</p>
) : (
 <table className="token-table">
 <thead>
 <tr>
 <th>Name</th>
 <th>Token</th>
 <th>Created</th>
 <th>Last used</th>
 <th>Expires</th>
 <tr>
 <th />
 <tr>
 </thead>
 <tbody>
 {tokens.map((t) => (
 <tr key={t.id}>
 <td>{t.label}</td>
 <td><code>{t.prefix}...</code></td>
 <td>{when(t.createdAt)}</td>
 <td>{when(t.lastUsedAt)}</td>
 <td>{when(t.expiresAt)}</td>
 <td>

```

**src/client/pages/AccountSettings.tsx**

```
 <button type="button" className="mini danger" onClick={() => void revoke(t.id,
t.label)}}>
 Revoke
 </button>
 </td>
</tr>
</tbody>
</table>
)}
</section>
);
}
```

**src/client/pages/Admin.tsx**

```
/**
 * The admin panel shell.
 *
 * A grouped left sidebar (Content / People / Settings) of items; each item opens
 * in the content area and may host several tools as tabs (e.g. Ranks & Roles).
 * Everything is gated by permission via the tree in lib/adminSections.ts, so the
 * sidebar only shows what the viewer can touch. Under each group we also surface
 * a few recently-viewed records for quick return.
 *
 * To keep it from feeling like a separate app, the panel wears the same top bar
 * as the rest of the site, adds a breadcrumb so you always know where you are,
 * and the section rail is sticky + icon-led and can be collapsed (a drawer on
 * mobile, so the tool shows first instead of being pushed below the whole menu).
 */

import { useEffect, useState, type ReactNode } from 'react';
import { NavLink, Navigate, Link, useParams } from 'react-router-dom';
import { MorphIcon } from 'morphicons/react';
import {
 PanelLeft,
 FileText,
 Navigation,
 Newspaper,
 Users,
 Award,
 Network,
 Fingerprint,
 Bot,
 Gauge,
 Palette,
 ScrollText,
} from 'lucide';
import { useSession } from '../lib/session';
import { visibleAdminGroups } from '../lib/adminSections';
import { getRecent } from '../lib/recent';
import UsageBar from '../components/UsageBar';
import NewsAdmin from './NewsAdmin';
import Ranks from './Ranks';
import Roles from './Roles';
import Medals from './Medals';
import Games from './Games';
import WarRecords from './WarRecords';
import Announcements from './Announcements';
import IdentityAdmin from './IdentityAdmin';
import AnalyticsAdmin from './AnalyticsAdmin';
import ModulesAdmin from './ModulesAdmin';
import Theme from './Theme';
import BrandingAdmin from './BrandingAdmin';
import SeoAdmin from './SeoAdmin';
import FooterAdmin from './FooterAdmin';
import PagesAdmin from './PagesAdmin';
import NavAdmin from './NavAdmin';
import OrgChartDesigner from './OrgChartDesigner';
import AuditLog from './AuditLog';
```

**src/client/pages/Admin.tsx**

```

/** Sidebar item key -> an icon, so the rail reads like a real nav rail. */
const ITEM_ICONS: Record<string, typeof FileText> = {
 pages: FileText,
 navigation: Navigation,
 news: Newspaper,
 'ranks-roles': Users,
 'medals-records': Award,
 'org-chart': Network,
 identity: Fingerprint,
 bot: Bot,
 analytics: Gauge,
 appearance: Palette,
 logs: ScrollText,
};

/** Tab key -> its component. Every tab key in adminSections.ts needs an entry. */
const TAB_RENDERERS: Record<string, () => ReactNode> = {
 pages: () => <PagesAdmin />,
 navigation: () => <NavAdmin />,
 news: () => <NewsAdmin />,
 ranks: () => <Ranks />,
 roles: () => <Roles />,
 medals: () => <Medals />,
 warrecords: () => <WarRecords />,
 games: () => <Games />,
 orgchart: () => <OrgChartDesigner />,
 announcements: () => <Announcements />,
 identity: () => <IdentityAdmin />,
 analytics: () => <AnalyticsAdmin />,
 modules: () => <ModulesAdmin />,
 theme: () => <Theme />,
 branding: () => <BrandingAdmin />,
 seo: () => <SeoAdmin />,
 footer: () => <FooterAdmin />,
 audit: () => <AuditLog />,
};

export default function Admin() {
 const { item: itemParam, tab: tabParam } = useParams();
 const { can } = useSession();

 // Section rail open/closed. Persisted so a chosen state sticks; defaults open
 // on desktop and closed on mobile (there the rail is a drawer over the tool).
 const [navOpen, setNavOpen] = useState<boolean>(() => {
 if (typeof window === 'undefined') return true;
 const stored = localStorage.getItem('ct-admin-nav');
 if (stored === '0') return false;
 if (stored === '1') return true;
 return window.innerWidth > 720;
 });
 useEffect(() => {
 try {
 localStorage.setItem('ct-admin-nav', navOpen ? '1' : '0');
 }
 }

```

**src/client/pages/Admin.tsx**

```

 } catch {
 /* private mode ? the toggle still works for the session */
 }
 }, [navOpen]);
 // On a phone the rail overlays the tool, so pick-a-section should close it.
 const closeOnMobile = () => {
 if (typeof window !== 'undefined' && window.innerWidth <= 720) setNavOpen(false);
 };

 // Re-read recently-viewed when it changes (a tool recorded one, another tab wrote it).
 const [, setTick] = useState(0);
 useEffect(() => {
 const bump = () => setTick((t) => t + 1);
 window.addEventListener('ct-recent', bump);
 window.addEventListener('storage', bump);
 return () => {
 window.removeEventListener('ct-recent', bump);
 window.removeEventListener('storage', bump);
 };
 }, []);

 const groups = visibleAdminGroups(can);
 if (groups.length === 0) {
 return <div className="empty">You don't have access to any admin tools.</div>;
 }

 const items = groups.flatMap((g) => g.items);
 const activeItem = items.find((i) => i.key === itemParam);

 // No item, or one they can't reach -> land on the first available item.
 if (!activeItem) return <Navigate to={`/admin/${items[0]!.key}`} replace />;

 const tabs = activeItem.tabs; // already filtered to allowed
 const activeTab = tabs.find((t) => t.key === tabParam) ?? tabs[0]!;

 // Recently-viewed records for the group the active item belongs to, shown as a
 // horizontal strip above the content (not crammed into the sidebar).
 const activeGroup = groups.find((g) => g.items.some((i) => i.key === activeItem.key));
 const recent = activeGroup ? getRecent(activeGroup.key) : [];

 return (
 <>
 <UsageBar />

 {/* Header: the section toggle + a breadcrumb, so entering admin is an
 oriented landing rather than an abrupt layout swap. */}
 <div className="admin-header">
 <button
 type="button"
 className="admin-nav-toggle"
 aria-expanded={navOpen}
 aria-controls="admin-sections"
 onClick={() => setNavOpen((o) => !o)}
 >

```

**src/client/pages/Admin.tsx**

```

 <MorphIcon icon={PanelLeft} size={16} aria-hidden />
 Sections
 </button>
 <nav className="admin-crumbs" aria-label="Breadcrumb">
 <Link to="/admin" className="admin-crumb">
 Admin
 </Link>
 {activeGroup && (
 <>

 ?

 {activeGroup.label}
 </>
)}

 ?

 <MorphIcon
 className="admin-crumb-icon"
 icon={ITEM_ICONS[activeItem.key] ?? FileText}
 size={15}
 spring="snappy"
 aria-hidden
 />
 {activeItem.label}

 </nav>
</div>

<div className={navOpen ? 'admin-shell nav-open' : 'admin-shell'}>
 <nav id="admin-sections" className="admin-nav" aria-label="Admin sections">
 {groups.map((g) => (
 <div className="admin-nav-group" key={g.key}>
 <div className="admin-nav-title">{g.label}</div>
 {g.items.map((i) => (
 <NavLink
 key={i.key}
 to={` /admin/${i.key}`}
 onClick={closeOnMobile}
 className={i.key === activeItem.key ? 'admin-nav-link active' :
'admin-nav-link'}
 >
 <MorphIcon className="admin-nav-icon" icon={ITEM_ICONS[i.key] ?? FileText}
size={16} aria-hidden />
 {i.label}
 </NavLink>
))}
 </div>
))}
 </nav>

 <div className="admin-content">

```

**src/client/pages/Admin.tsx**

```

 {recent.length > 0 && (
 <div className="admin-recentbar">
 Recently viewed
 {recent.map((r) => (
 <Link key={r.to} to={r.to} className="admin-recentbar-link" title={r.label}>
 {r.label}
 </Link>
))}
 </div>
)}

 {tabs.length > 1 && (
 <div className="admin-tabs" role="tablist" aria-label={activeItem.label}>
 {tabs.map((t) => (
 <NavLink
 key={t.key}
 to={` /admin/${activeItem.key}/${t.key}`}
 className={t.key === activeTab.key ? 'admin-tab active' : 'admin-tab'}
 role="tab"
 aria-selected={t.key === activeTab.key}
 >
 {t.label}
 </NavLink>
))}
 </div>
)}
 {TAB_RENDERERS[activeTab.key]?.()}
 </div>
</div>
</>
);
}

```



**src/client/pages/AnalyticsAdmin.tsx**

```

/**
 * Analytics admin ? wire up (or fix) the live usage dashboard's request &
 * query-rate gauges without editing wrangler.jsonc or redeploying.
 *
 * The storage gauges (R2 bytes, D1 size) always work ? they're measured inside
 * the Worker. The daily *rate* gauges need Cloudflare's GraphQL Analytics API,
 * which requires a read-only token plus the account/worker/database this install
 * runs on. Those four values are edited here; saved values override the deploy
 * defaults. The API token is write-only ? the form only knows whether one is set.
 */

import { useEffect, useState } from 'react';
import { api } from '../lib/api';
import { useAction, Alerts } from '../lib/action';

interface Analytics {
 accountId: string;
 scriptName: string;
 d1DatabaseId: string;
 apiTokenSet: boolean;
}

/** A plain external link that opens safely in a new tab. */
function Ext({ href, children }: { href: string; children: string }) {
 return (

 {children}

);
}

export default function AnalyticsAdmin() {
 const [loading, setLoading] = useState(true);
 const [saved, setSaved] = useState<Analytics | null>(null);

 const [accountId, setAccountId] = useState('');
 const [scriptName, setScriptName] = useState('');
 const [d1DatabaseId, setD1DatabaseId] = useState('');
 // Write-only: empty means "leave unchanged".
 const [apiToken, setApiToken] = useState('');

 const { run, busy, error, notice, warning } = useAction();

 const apply = (a: Analytics) => {
 setSaved(a);
 setAccountId(a.accountId);
 setScriptName(a.scriptName);
 setD1DatabaseId(a.d1DatabaseId);
 setApiToken('');
 };

 useEffect(() => {
 api
 .get<{ analytics: Analytics }>('/settings/analytics')

```

**src/client/pages/AnalyticsAdmin.tsx**

```

 .then(({ analytics }) => apply(analytics))
 .finally(() => setLoading(false));
 }, []);

 const dirty =
 !!saved &&
 (accountId !== saved.accountId ||
 scriptName !== saved.scriptName ||
 d1DatabaseId !== saved.d1DatabaseId ||
 apiToken !== '');

 const save = () =>
 run(async () => {
 const { analytics } = await api.put<{ analytics: Analytics }>('/settings/analytics', {
 analytics: { accountId, scriptName, d1DatabaseId, apiToken },
 });
 apply(analytics);
 return 'Saved. The usage dashboard now uses these settings.';
 });

 const test = () =>
 run(async () => {
 const res = await api.post<{ ok: boolean; requests?: number; error?: string }>(
 '/settings/analytics/test',
);
 if (!res.ok) return { warning: res.error ?? 'The Analytics API could not be reached.' };
 return `Connected ? Cloudflare reports ${res.requests?.toLocaleString() ?? 0} requests in
the last 24h.`;
 });

 if (loading) return <div className="loading">Loading...</div>;

 return (
 <section className="panel analytics-admin">
 <header className="panel-head">
 <h2>Analytics</h2>
 <p className="muted">
 Storage gauges always work. The daily request & query-rate gauges need a read-only
 Cloudflare Analytics token and the IDs for this install. Saved values override the
 defaults; leave a field blank to fall back to the default.
 </p>
 </header>

 <Alerts error={error} warning={warning} notice={notice} />

 <fieldset>
 <legend>Cloudflare Analytics</legend>
 <p className="muted small">
 These come from your{' '}
 <Ext href="https://dash.cloudflare.com/">Cloudflare dashboard</Ext>. If you deployed
 this
 site yourself, the Worker name and D1 ID are already in your
 <code>wrangler.jsonc</code>.

```

**src/client/pages/AnalyticsAdmin.tsx**

```

 </p>
 <label>
 Account ID
 <input
 value={accountId}
 onChange={(e) => setAccountId(e.target.value)}
 disabled={busy}
 placeholder="32 hex characters"
 />

 Not secret. Dashboard -> Workers & Pages -> the Account ID is in
the
 right-hand sidebar (and in your dashboard URL).{' '}
 <Ext href="https://dash.cloudflare.com/?to=:account/workers-and-pages">Open ?</Ext>

 </label>
 <label>
 API Token
 <input
 type="password"
 value={apiToken}
 placeholder={saved?.apiTokenSet ? '***** (set ? leave blank to keep)' : 'not set'}
 onChange={(e) => setApiToken(e.target.value)}
 disabled={busy}
 autoComplete="new-password"
 />

 <Ext href="https://dash.cloudflare.com/profile/api-tokens">Create a token ?</Ext> ->
 "Create Custom Token" -> add the Account ? Account Analytics ? Read{'
 '}
 permission. Stored in this site's database ? see the note below.

 </label>
 <label>
 Worker name
 <input
 value={scriptName}
 onChange={(e) => setScriptName(e.target.value)}
 disabled={busy}
 placeholder="e.g. clantek"
 />

 The <code>name</code> in your <code>wrangler.jsonc</code> ? its requests are counted.
Also
 shown under Dashboard -> Workers & Pages.

 </label>
 <label>
 D1 Database ID
 <input
 value={d1DatabaseId}
 onChange={(e) => setD1DatabaseId(e.target.value)}
 disabled={busy}
 placeholder="36-character UUID"
 />
 </label>
 </div>
}

```

**src/client/pages/AnalyticsAdmin.tsx**

```

 />

 The <code>database_id</code> in your <code>wrangler.jsonc</code> ? its rows
read/written
 are counted. Also under Dashboard -> Storage & Databases -> D1 ->
your
 database.

 </label>
</fieldset>

 <p className="muted small warn">
 Note: the API token is stored in this site's database (not an encrypted Cloudflare
secret),
 so anyone with database access or god-admin rights can read it. Use a read-only token
scoped
 to analytics only.
 </p>

 <div className="news-editor-actions">
 <button className="primary" disabled={busy || !dirty} onClick={() => void save()}>
 Save
 </button>
 <button
 disabled={busy}
 onClick={() => void test()}
 title="Query the Analytics API with the saved token to confirm it works"
 >
 Test connection
 </button>
 </div>
</section>
);
}

```

**src/client/pages/Announcements.tsx**

```

/**
 * Discord announcements admin.
 *
 * Choose the channel the bot posts to and toggle which events announce. "Send
 * test" confirms the bot can actually reach the channel before you rely on it.
 */

import { useEffect, useState } from 'react';
import { api } from '../lib/api';
import { useAction, Alerts } from '../lib/action';

interface Config {
 channelId: string | null;
 events: { medalAward: boolean; warRecordAward: boolean; promotion: boolean };
}

interface Channel {
 id: string;
 name: string;
 position: number;
}

const EVENT_LABELS: [key: keyof Config['events'], label: string, hint: string][] = [
 ['medalAward', 'Medal awarded', 'Posts when a member is awarded a medal.'],
 ['warRecordAward', 'War record awarded', 'Posts when a member earns a war record.'],
 ['promotion', 'Promotion', 'Posts when a member is promoted (website or /promote).'],
];

export default function Announcements() {
 const [saved, setSaved] = useState<Config | null>(null);
 const [channels, setChannels] = useState<Channel[]>([]);
 const [channelWarning, setChannelWarning] = useState<string | null>(null);
 const [loading, setLoading] = useState(true);

 const [channelId, setChannelId] = useState('');
 const [events, setEvents] = useState<Config['events']>({
 medalAward: false,
 warRecordAward: false,
 promotion: false,
 });

 const { run, busy, error, notice, warning } = useAction();

 useEffect(() => {
 Promise.all([
 api.get<{ announcements: Config }>('/settings/announcements').then(({ announcements }) => {
 setSaved(announcements);
 setChannelId(announcements.channelId ?? '');
 setEvents(announcements.events);
 }),
 api
 .get<{ channels: Channel[]; warning?: string }>('/settings/discord-channels')
 .then(({ channels, warning }) => {
 setChannels(channels);
 setChannelWarning(warning ?? null);
 })
])
 }, []);

```

**src/client/pages/Announcements.tsx**

```

 })
 .catch(() => setChannelWarning('Could not reach Discord to load channels.')),
]).finally(() => setLoading(false));
}, []);

const dirty =
 !!saved &&
 (channelId !== (saved.channelId ?? '') ||
 (Object.keys(events) as (keyof Config['events'])[]).some((k) => events[k] !==
saved.events[k]));

const save = () =>
 run(async () => {
 const { announcements } = await api.put<{ announcements: Config
}>('/settings/announcements', {
 channelId: channelId || null,
 events,
 });
 setSaved(announcements);
 return 'Saved.';
 });

const test = () =>
 run(async () => {
 if (!channelId) return { warning: 'Choose a channel first.' };
 await api.post('/settings/announcements/test', { channelId });
 return 'Test message sent ? check the channel.';
 });

const toggle = (key: keyof Config['events']) =>
 setEvents((prev) => ({ ...prev, [key]: !prev[key] }));

if (loading) return <div className="loading">Loading...</div>;

return (
 <section className="panel">
 <header className="panel-head">
 <h2>Announcements</h2>
 <p className="muted">Have the bot post to Discord when things happen on the site.</p>
 </header>

 <Alerts error={error} warning={warning} notice={notice} />

 <label>
 Channel
 {channels.length > 0 ? (
 <select value={channelId} onChange={(e) => setChannelId(e.target.value)}
disabled={busy}>
 <option value="">? None (announcements off) ?</option>
 {channels.map((ch) => (
 <option key={ch.id} value={ch.id}>
 #{ch.name}
 </option>
))}
 </select>
) : null}
 </label>
 </section>
);

```

**src/client/pages/Announcements.tsx**

```

 </select>
) : (
 <input
 value={channelId}
 placeholder="Discord channel ID"
 onChange={(e) => setChannelId(e.target.value)}
 disabled={busy}
 />
)}
</label>
{channelWarning && <p className="muted small warn">{channelWarning}</p>}

<fieldset className="tenure">
 <legend>Announce these events</legend>
 {EVENT_LABELS.map(([key, label, hint]) => (
 <label key={key} className="check announce-event">
 <input type="checkbox" checked={events[key]} onChange={() => toggle(key)}
disabled={busy} />

 {label}
 ? {hint}

 </label>
))}
 <p className="muted small">
 Nothing posts unless a channel is chosen above and the event is ticked.
 </p>
</fieldset>

<div className="news-editor-actions">
 <button className="primary" disabled={busy || !dirty} onClick={() => void save()}>
 Save
 </button>
 <button disabled={busy || !channelId} onClick={() => void test()} title="Post a test
message to the chosen channel">
 Send test
 </button>
</div>
</section>
);
}

```

**src/client/pages/AuditLog.tsx**

```

/**
 * The activity log ? a read-only feed of who did what, gated on audit.view.
 *
 * Every mutation elsewhere on the site writes an entry; this surfaces them,
 * newest first, with the actor, the member affected, the reason (mandatory on
 * negative actions), and where it happened (web or a Discord slash command).
 * Category chips narrow to the action group; "Load more" pages through.
 */

import { useCallback, useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import { api } from '../lib/api';
import { Alerts, useAction } from '../lib/action';
import { memberAvatar } from '../../shared/avatar';

interface Actor {
 id: number;
 name: string;
 discordId: string;
 avatar: string | null;
 profileImageUrl: string | null;
}

interface AuditEntry {
 id: number;
 action: string;
 reason: string | null;
 meta: Record<string, unknown> | null;
 source: 'web' | 'discord' | 'system';
 createdAt: number;
 targetType: string | null;
 targetId: string | null;
 actor: Actor | null;
 target: { id: number; name: string } | null;
}

const PAGE = 50;

// Category chips -> the `action` prefix they filter on. Empty = everything.
const CATEGORIES: { key: string; label: string; prefix: string }[] = [
 { key: 'all', label: 'All', prefix: '' },
 { key: 'members', label: 'Members', prefix: 'member' },
 { key: 'medals', label: 'Medals', prefix: 'medal' },
 { key: 'warrecords', label: 'War records', prefix: 'warrecord' },
 { key: 'roles', label: 'Roles', prefix: 'role' },
];

// Human labels for the known actions; anything else falls back to a title-case
// of the raw string so a new action type still reads sensibly.
const ACTION_LABELS: Record<string, string> = {
 'member.rank_change': 'Rank change',
 'member.promote': 'Promoted',
 'member.status': 'Status change',
 'member.display_name': 'Display name changed',
 'member.join': 'Joined',

```



**src/client/pages/AuditLog.tsx**

```

 'medal.award': 'Medal awarded',
 'medal.revoke': 'Medal revoked',
 'warrecord.award': 'War record awarded',
 'warrecord.revoke': 'War record revoked',
 'role.grant': 'Role granted',
 'role.revoke': 'Role revoked',
 };

 // Negative actions are called out with a red chip.
 const NEGATIVE = new Set([
 'medal.revoke',
 'warrecord.revoke',
 'role.revoke',
]);

 function actionLabel(entry: AuditEntry): string {
 if (entry.action === 'member.rank_change' && entry.meta?.demotion) return 'Demoted';
 return ACTION_LABELS[entry.action] ?? titleCase(entry.action);
 }

 function isNegative(entry: AuditEntry): boolean {
 if (NEGATIVE.has(entry.action)) return true;
 if (entry.action === 'member.rank_change' && entry.meta?.demotion) return true;
 if (entry.action === 'member.status') {
 const to = entry.meta?.to;
 return to === 'banned' || to === 'retired';
 }
 return false;
 }

 function titleCase(action: string): string {
 return action
 .replace(/[_]/g, ' ')
 .replace(/\b\w/g, (m) => m.toUpperCase());
 }

 /** A short human detail pulled from the entry's meta (the "what", specifically). */
 function detail(entry: AuditEntry): string | null {
 const m = entry.meta ?? {};
 const str = (v: unknown) => (typeof v === 'string' ? v : null);
 switch (entry.action) {
 case 'member.rank_change':
 case 'member.promote':
 return `${str(m.from) ?? 'Unranked'} -> ${str(m.to) ?? 'Unranked'}`;
 case 'member.status':
 return `${str(m.from) ?? '?'} -> ${str(m.to) ?? '?'}`;
 case 'medal.award':
 case 'medal.revoke':
 return str(m.medalName);
 case 'warrecord.award':
 case 'warrecord.revoke':
 return str(m.name);
 case 'role.grant':
 case 'role.revoke':

```

**src/client/pages/AuditLog.tsx**

```

 return str(m.roleName);
 default:
 return null;
 }
}

function whenLabel(unixSec: number): string {
 const then = unixSec * 1000;
 const diff = Date.now() - then;
 const min = Math.round(diff / 60000);
 if (min < 1) return 'just now';
 if (min < 60) return `${min}m ago`;
 const hr = Math.round(min / 60);
 if (hr < 24) return `${hr}h ago`;
 const day = Math.round(hr / 24);
 if (day < 7) return `${day}d ago`;
 return new Date(then).toLocaleDateString();
}

export default function AuditLog() {
 const [entries, setEntries] = useState<AuditEntry[]>([]);
 const [category, setCategory] = useState('all');
 const [offset, setOffset] = useState(0);
 const [hasMore, setHasMore] = useState(false);
 const [loading, setLoading] = useState(true);

 const prefix = CATEGORIES.find((ct) => ct.key === category)?.prefix ?? '';

 const fetchPage = useCallback(
 async (nextOffset: number, replace: boolean) => {
 const params = new URLSearchParams({ limit: String(PAGE), offset: String(nextOffset) });
 if (prefix) params.set('action', prefix);
 const { entries: page, hasMore } = await api.get<{ entries: AuditEntry[]; hasMore: boolean }>(
 `/audit?${params.toString()}`,
);
 setEntries((prev) => (replace ? page : [...prev, ...page]));
 setOffset(nextOffset + page.length);
 setHasMore(hasMore);
 },
 [prefix],
);

 const { run, busy, error, notice, warning } = useAction();

 useEffect(() => {
 setLoading(true);
 fetchPage(0, true).finally(() => setLoading(false));
 }, [fetchPage]);

 const loadMore = () => run(() => fetchPage(offset, false));

 return (
 <section className="panel">

```



**src/client/pages/AuditLog.tsx**

```


 {det && {det}}
 {e.target && (
 <>
 ?
 <Link to={` /members/${e.target.id}`}>{e.target.name}</Link>
 </>
)}
 {e.reason && <div className="audit-reason">{e.reason}</div>}
 </div>

 <div className="audit-meta muted small">
 {e.source === 'discord' && discord}

 {whenLabel(e.createdAt)}

 </div>

);
}}

{hasMore && (
 <div className="center">
 <button onClick={() => void loadMore()} disabled={busy}>
 {busy ? 'Loading...' : 'Load more'}
 </button>
 </div>
)}
</>
)}
</section>
);
}

```

**src/client/pages/BrandingAdmin.tsx**

```
/**
 * Branding admin ? upload a header logo and set how big it sits at the top of
 * the page. The logo replaces the "ClanTek" wordmark in the header: oversized and
 * overhanging the bar at the top of a page, shrinking to fit inside the bar once
 * the visitor scrolls. Saving updates the live header immediately (shared
 * BrandingProvider), and clearing it falls back to the site-name text.
 */

import { useEffect, useState } from 'react';
import { api } from '../lib/api';
import { useBranding, type Branding } from '../lib/branding';

export default function BrandingAdmin() {
 const { branding, preview, save } = useBranding();
 const [draft, setDraft] = useState<Branding>(branding);
 const [uploading, setUploading] = useState(false);
 const [saving, setSaving] = useState(false);
 const [message, setMessage] = useState<string | null>(null);
 const [err, setErr] = useState<string | null>(null);

 // Seed the draft from the loaded branding once it arrives.
 useEffect(() => {
 setDraft(branding);
 // Only when the persisted value changes (load/save), not on every preview.
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [branding.logoUrl]);

 /** Update the draft and push a live header preview so changes are visible at once. */
 function edit(patch: Partial<Branding>) {
 const next = { ...draft, ...patch };
 setDraft(next);
 preview(next);
 setMessage(null);
 }

 async function onSave() {
 setSaving(true);
 setMessage(null);
 try {
 await save(draft);
 setMessage('Saved. The header logo is live.');
```

**src/client/pages/BrandingAdmin.tsx**

Upload a logo for the header. It shows in place of the site name ? large and overhanging the bar at the top of the page, shrinking into the bar as visitors scroll.

```

 </p>
 </div>
 <button type="button" className="primary" onClick={onSave} disabled={saving || uploading}>
 {saving ? 'Saving...' : 'Save'}
 </button>
</header>

{message && <div className="notice">{message}</div>}

<div className="branding-grid">
 <div className="branding-controls">
 {draft.logoUrl && (
 <div className="branding-preview" style={{ ['--h' as string]: `${draft.logoSize}px`
 >

 </div>
)}

 <div className="avatar-controls">
 <label className="upload-btn">
 {uploading ? 'Uploading...' : draft.logoUrl ? 'Change logo' : 'Upload logo'}
 <input
 type="file"
 accept="image/png,image/jpeg,image/gif,image/webp"
 hidden
 disabled={uploading}
 onChange={async (e) => {
 const file = e.target.files?.[0];
 e.target.value = '';
 if (!file) return;
 setErr(null);
 setUploading(true);
 try {
 const res = await api.upload<{ url: string }>('/media/branding', file);
 edit({ logoUrl: res.url });
 } catch (e2) {
 setErr(e2 instanceof Error ? e2.message : 'Upload failed.');
```

**src/client/pages/BrandingAdmin.tsx**

```

 </button>
)}
</div>
{err && <p className="muted small branding-err">{err}</p>}

<label className="branding-size">
 Logo size at the top of the page
 <input
 type="range"
 min={48}
 max={160}
 step={2}
 value={draft.logoSize}
 disabled={!draft.logoUrl}
 onChange={(e) => edit({ logoSize: Number(e.target.value) })}
 />
 {draft.logoSize}px tall (shrinks to 38px on
scroll)
</label>

 {!draft.logoUrl && (
 <p className="muted small">With no logo uploaded, the header shows the site name as
text.</p>
)}

 <div className="branding-favicon">
 <div className="branding-favicon-head">
 {draft.faviconUrl ? (
 <img className="branding-favicon-preview" src={draft.faviconUrl} alt="Favicon
preview" />
) : (

)}
 <div>
 Browser tab icon
 <p className="muted small">
 The little icon shown in the browser tab and bookmarks (the favicon). A square
 PNG works best ? 512?512 with a transparent background.
 </p>
 </div>
 </div>
 <div className="avatar-controls">
 <label className="upload-btn">
 {uploading ? 'Uploading...' : draft.faviconUrl ? 'Change icon' : 'Upload icon'}
 <input
 type="file"
 accept="image/png,image/jpeg,image/gif,image/webp"
 hidden
 disabled={uploading}
 onChange={async (e) => {
 const file = e.target.files?.[0];
 e.target.value = '';
 if (!file) return;
 setErr(null);
 }}
 </input>
 </div>

```

**src/client/pages/BrandingAdmin.tsx**

```

 setUploading(true);
 try {
 const res = await api.upload<{ url: string }>('/media/branding', file);
 edit({ faviconUrl: res.url });
 } catch (e2) {
 setErr(e2 instanceof Error ? e2.message : 'Upload failed.');
```

```
 } finally {
 setUploading(false);
 }
 }
}
```

```
 />
```

```
</label>
```

```
{draft.faviconUrl && (
```

```
 <button
```

```
 type="button"
```

```
 className="ghost"
```

```
 disabled={uploading}
```

```
 onClick={() => edit({ faviconUrl: '' })}
```

```
 >
```

```
 Remove icon
```

```
 </button>
```

```
)}
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div className="branding-hint muted small">
```

```
 <p>Tips</p>
```

```

```

```
 A transparent PNG looks best ? the header background shows through.
```

```
 A wide wordmark or a compact emblem both work; the size slider sets its
```

```
height.
```

```
 Scroll the page after saving to see it dock into the bar.
```

```

```

```
</div>
```

```
</div>
```

```
</section>
```

```
);
```

```
}
```



**src/client/pages/CustomPage.tsx**

```

/**
 * Renders an admin-created custom page at /p/:slug through the same module
 * system the home page uses. A slug with no stored page shows "not found".
 */

import { useEffect, useState } from 'react';
import { useParams } from 'react-router-dom';
import { api } from '../lib/api';
import PageRenderer from '../components/PageRenderer';
import { useRecordRecent } from '../lib/recent';
import { sanitizeLayout, type PageLayout } from '../shared/layout';

export default function CustomPage() {
 const { slug } = useParams();
 const [state, setState] = useState<{ layout: PageLayout; exists: boolean; title: string } | null>(null);

 useEffect(() => {
 setState(null);
 api
 .get<{ layout: unknown; exists: boolean; title: string }>(`/pages/${slug}`)
 .then((d) => setState({ layout: sanitizeLayout(d.layout), exists: d.exists, title: d.title
}))
 .catch(() => setState({ layout: { version: 1, rows: [] }, exists: false, title: '' }));
 }, [slug]);

 useRecordRecent(state?.exists ? { group: 'content', label: state.title || slug || 'Page', to:
`/p/${slug}` } : null);

 if (!state) return <div className="loading">Loading...</div>;
 if (!state.exists) return <div className="empty">Page not found.</div>;

 return (
 <>
 {state.title && <h1 className="page-title">{state.title}</h1>}
 <PageRenderer layout={state.layout} />
 </>
);
}

```

**src/client/pages/Events.tsx**

```

/**
 * Events ? upcoming clan happenings. Everyone with events.view sees the list
 * and can sign themselves up (optionally picking a role like Tank/Healer/DPS).
 * events.manage adds create/edit/cancel; events.attendees reveals the full
 * "who's coming" roster. Each event mirrors to a Discord scheduled event and a
 * sign-up message whose buttons write the same rows ? so the two stay in sync.
 */

import { useEffect, useState, type ChangeEvent } from 'react';
import { api } from '../lib/api';
import { useAction, Alerts } from '../lib/action';
import { useSession } from '../lib/session';

interface RoleState {
 id: number;
 name: string;
 emoji: string | null;
 capacity: number | null;
 count: number;
}

interface EventItem {
 id: number;
 title: string;
 description: string | null;
 imageUrl: string | null;
 startsAt: number;
 endsAt: number;
 location: string;
 gameId: number | null;
 gameName: string | null;
 createdBy: number | null;
 recurrence: Recurrence;
 roles: RoleState[];
 signupCount: number;
 mySignup: { roleId: number | null } | null;
}

interface GameOption {
 id: number;
 name: string;
}

type Recurrence = 'none' | 'daily' | 'weekly' | 'biweekly' | 'monthly';
const RECURRENCE_OPTIONS: { value: Recurrence; label: string }[] = [
 { value: 'none', label: 'Doesn't repeat' },
 { value: 'daily', label: 'Daily' },
 { value: 'weekly', label: 'Weekly' },
 { value: 'biweekly', label: 'Every 2 weeks' },
 { value: 'monthly', label: 'Monthly' },
];

const RECURRENCE_BADGE: Record<Recurrence, string> = {
 none: '',
 daily: 'Daily',
 weekly: 'Weekly',
 biweekly: 'Every 2 weeks',

```

**src/client/pages/Events.tsx**

```

 monthly: 'Monthly',
 };

const pad = (n: number) => String(n).padStart(2, '0');

// Time is picked as an hour (12-hour labels) plus a 15-minute slot, rather than
// a raw datetime field, so the choices are unambiguous.
const HOUR_OPTIONS = Array.from({ length: 24 }, (_, h) => ({
 value: h,
 label: `${((h + 11) % 12) + 1} ${h < 12 ? 'AM' : 'PM'}`,
}));
const MINUTE_OPTIONS = [0, 15, 30, 45];
const roundTo15 = (m: number) => MINUTE_OPTIONS.reduce((a, b) => (Math.abs(b - m) < Math.abs(a - m) ? b : a), 0);

// A curated set of icons for the Discord sign-up buttons. Optional ? "?" leaves
// the button text-only.
const ROLE_EMOJIS = ['??', '??', '?', '?', '?', '?', '?', '?', '?', '?', '?', '?', '??', '??', '?'];
// Capacity choices for a role (1?30, or no limit).
const CAP_OPTIONS = Array.from({ length: 30 }, (_, i) => i + 1);

function whenLabel(startsAt: number, endsAt: number): string {
 const start = new Date(startsAt * 1000);
 const end = new Date(endsAt * 1000);
 const sameDay = start.toDateString() === end.toDateString();
 const date = start.toLocaleDateString(undefined, { weekday: 'short', month: 'short', day: 'numeric' });
 const t = (d: Date) => d.toLocaleTimeString(undefined, { hour: 'numeric', minute: '2-digit' });
 return sameDay ? `${date} ? ${t(start)} ? ${t(end)}` : `${start.toLocaleString()} -> ${end.toLocaleString()}`;
}

export default function Events() {
 const { can } = useSession();
 const canManage = can('events.manage');
 const canSeeAttendees = can('events.attendees');
 const [events, setEvents] = useState<EventItem[]>([]);
 const [games, setGames] = useState<GameOption[]>([]);
 const [loading, setLoading] = useState(true);
 const [creating, setCreating] = useState(false);
 const [editingId, setEditingId] = useState<number | null>(null);
 const [confirmDelete, setConfirmDelete] = useState<number | null>(null);

 async function load() {
 const { events } = await api.get<{ events: EventItem[] }>('/events');
 setEvents(events);
 }

 const { run, busy, error, notice, warning } = useAction(load);

 useEffect(() => {
 Promise.all([
 load(),
]),
 }

```

**src/client/pages/Events.tsx**

```

 canManage
 ? api.get<{ games: GameOption[] }>('/games').then(({ games }) => setGames(games)).catch(()
=> setGames([]))
 : Promise.resolve(),
]).finally(() => setLoading(false));
 }, [canManage]);

const create = (payload: EventPayload) =>
 run(async () => {
 await api.post('/events', payload);
 setCreating(false);
 return 'Event created ? posting to Discord.';
 });

const saveEdit = (id: number, payload: EventPayload) =>
 run(async () => {
 await api.patch(`/events/${id}`, payload);
 setEditingId(null);
 return 'Event updated.';
 });

const remove = (id: number) =>
 run(async () => {
 await api.del(`/events/${id}`);
 setConfirmDelete(null);
 return 'Event cancelled and removed from Discord.';
 });

const signup = (eventId: number, roleId: number | null) =>
 run(async () => {
 await api.post(`/events/${eventId}/signup`, { roleId });
 return roleId === null ? 'You're attending.' : 'Signed up.';
 });

const withdraw = (eventId: number) =>
 run(async () => {
 await api.del(`/events/${eventId}/signup`);
 return 'Withdrawn from the event.';
 });

if (loading) return <div className="loading">Loading events...</div>;

return (
 <section className="panel">
 <header className="panel-head news-head">
 <div>
 <h2>Events</h2>
 <p className="muted">
 {events.length === 0 ? 'Nothing scheduled.' : `${events.length} upcoming`}
 </p>
 </div>
 {canManage && !creating && (
 <button onClick={() => { setCreating(true); setEditingId(null); }}>+ New event</button>
)}
 </header>
 </section>
)

```

**src/client/pages/Events.tsx**

```

</header>

<Alerts error={error} warning={warning} notice={notice} />

{creating && (
 <EventForm games={games} busy={busy} onSave={create} onCancel={() => setCreating(false)}
/>
)}

{events.length === 0 && !creating ? (
 <p className="muted">No upcoming events.</p>
) : (
 <ul className="event-list">
 {events.map((ev) =>
 editingId === ev.id ? (
 <li key={ev.id}>
 <EventForm
 initial={ev}
 games={games}
 busy={busy}
 onSave={(p) => saveEdit(ev.id, p)}
 onCancel={() => setEditingId(null)}
 />

) : (
 <li key={ev.id} className="event-card">
 {ev.imageUrl && }
 <div className="event-when">{whenLabel(ev.startsAt, ev.endsAt)}</div>
 <h3>{ev.title}</h3>
 <div className="muted small event-meta">
 ? {ev.location}
 {ev.gameName && <> ? ? {ev.gameName}</>}
 {ev.recurrence !== 'none' && <> ? ? {RECURRENT_BADGE[ev.recurrence]}</>}
 {ev.signupCount > 0 && <> ? ? {ev.signupCount} signed up</>}
 </div>
 {ev.description && <p className="event-desc">{ev.description}</p>}

 <SignupControls
 event={ev}
 busy={busy}
 onSignup={(roleId) => void signup(ev.id, roleId)}
 onWithdraw={() => void withdraw(ev.id)}
 />

 {canSeeAttendees && <AttendeeRoster eventId={ev.id} count={ev.signupCount} />}

 {canManage && (
 <div className="event-actions">
 <button className="mini" disabled={busy} onClick={() => { setEditingId(ev.id);
setCreating(false); }}>
 Edit
 </button>
 {confirmDelete === ev.id ? (


```

**src/client/pages/Events.tsx**

```

 Cancel this event?
 <button className="mini danger" disabled={busy} onClick={() => void
remove(ev.id)}>
 Confirm
 </button>
 <button className="mini" disabled={busy} onClick={() =>
setConfirmDelete(null)}>
 Keep
 </button>

) : (
 <button className="mini danger" disabled={busy} onClick={() =>
setConfirmDelete(ev.id)}>
 Cancel event
 </button>
)}
</div>
)}

),
)}

)}
</section>
);
}

```

/\*\* The member's own RSVP: role buttons (highlighting their pick) + attend/withdraw. \*/

```

function SignupControls({
 event,
 busy,
 onSignup,
 onWithdraw,
}): {
 event: EventItem;
 busy: boolean;
 onSignup: (roleId: number | null) => void;
 onWithdraw: () => void;
}) {
 const my = event.mySignup;
 const myRole = my?.roleId ?? null;
 const attending = my != null;

 return (
 <div className="signup-controls">
 {attending ? "You're in:" : 'Sign up:'}
 {event.roles.map((r) => {
 const selected = attending && myRole === r.id;
 const full = r.capacity != null && r.count >= r.capacity && !selected;
 return (
 <button
 key={r.id}
 className={selected ? 'chip active' : 'chip'}
 disabled={busy || full}

```

**src/client/pages/Events.tsx**

```

 title={full ? `${r.name} is full` : `Sign up as ${r.name}`}
 onClick={() => onSignup(r.id)}
 >
 {r.emoji ? `${r.emoji} ` : ''}
 {r.name}

 {r.count}
 {r.capacity !== null ? `/${r.capacity}` : ''}

 </button>
);
 }}
<button
 className={attending && myRole === null ? 'chip active' : 'chip'}
 disabled={busy}
 onClick={() => onSignup(null)}
 title="Attend without a specific role"
>
 Attending
</button>
{attending && (
 <button className="chip withdraw" disabled={busy} onClick={onWithdraw} title="Withdraw">
 Withdraw
 </button>
)}
</div>
);
}

interface SignupEntry {
 userId: number;
 name: string;
 avatarUrl: string;
 roleId: number | null;
 roleName: string | null;
}

/** Collapsible "who's coming" roster, grouped by role. Fetched on first expand. */
function AttendeeRoster({ eventId, count }: { eventId: number; count: number }) {
 const [open, setOpen] = useState(false);
 const [signups, setSignups] = useState<SignupEntry[] | null>(null);
 const [loading, setLoading] = useState(false);

 async function toggle() {
 const next = !open;
 setOpen(next);
 if (next && signups === null) {
 setLoading(true);
 try {
 const { signups } = await api.get<{ signups: SignupEntry[]
 }>(`/events/${eventId}/signups`);
 setSignups(signups);
 } catch {
 setSignups([]);
 }
 }
 }
}

```

**src/client/pages/Events.tsx**

```

 } finally {
 setLoading(false);
 }
 }
}

// Group by role name for display; no-role attendees fall under "Attending".
const groups = new Map<string, SignupEntry[]>();
for (const su of signups ?? []) {
 const key = su.roleName ?? 'Attending';
 const list = groups.get(key) ?? [];
 list.push(su);
 groups.set(key, list);
}

return (
 <div className="attendees">
 <button className="mini link-btn" onClick={() => void toggle()}>
 {open ? '?' : '?'} Who's coming{count > 0 ? ` (${count})` : ''}
 </button>
 {open &&
 (loading ? (
 <p className="muted small">Loading...</p>
) : (signups?.length ?? 0) === 0 ? (
 <p className="muted small">No one has signed up yet.</p>
) : (
 <div className="attendee-groups">
 {[...groups.entries()].map(([role, members]) => (
 <div key={role} className="attendee-group">
 <div className="attendee-role">
 {role} ({members.length})
 </div>
 <ul className="attendee-members">
 {members.map((m) => (
 <li key={m.userId}>
 <img className="avatar sm" src={m.avatarUrl} alt="" width={24} height={24}
 {m.name}

))}

 </div>
))}
 </div>
)}
)}
 </div>
);
}

interface RolePayload {
 id?: number;
 name: string;
 emoji: string | null;
 capacity: number | null;
}

```



**src/client/pages/Events.tsx**

```

}
interface EventPayload {
 title: string;
 description: string;
 imageUrl: string | null;
 startsAt: number;
 endsAt: number;
 location: string;
 gameId: number | null;
 recurrence: Recurrence;
 roles: RolePayload[];
}

interface RoleDraft {
 id?: number;
 name: string;
 emoji: string;
 capacity: string;
}

/**
 * Date + time picker: a native date field plus an hour dropdown (12-hour
 * labels) and a 15-minute slot dropdown. Emits the combined moment as unix
 * seconds, or null until a date is chosen.
 */
function DateTimeField({
 label,
 value,
 onChange,
 disabled,
}: {
 label: string;
 value: number | null;
 onChange: (v: number | null) => void;
 disabled?: boolean;
}) {
 const init = value !== null ? new Date(value * 1000) : null;
 const [date, setDate] = useState(
 init ? `${init.getFullYear()}-${pad(init.getMonth() + 1)}-${pad(init.getDate())}` : '',
);
 const [hour, setHour] = useState(init ? init.getHours() : 19);
 const [minute, setMinute] = useState(init ? roundTo15(init.getMinutes()) : 0);

 useEffect(() => {
 if (!date) {
 onChange(null);
 return;
 }
 const dt = new Date(`${date}T00:00:00`);
 dt.setHours(hour, minute, 0, 0);
 onChange(Math.floor(dt.getTime() / 1000));
 // onChange identity isn't a dependency ? only the picked values matter.
 }, [date, hour, minute]); // eslint-disable-line react-hooks/exhaustive-deps

```

**src/client/pages/Events.tsx**

```

 return (
 <div className="datetime-field">
 {label}
 <div className="datetime-inputs">
 <input type="date" value={date} onChange={(e) => setDate(e.target.value)}
disabled={disabled} />
 <select value={hour} onChange={(e) => setHour(Number(e.target.value))} disabled={disabled}
aria-label={` ${label} hour`} >
 {HOURL_OPTIONS.map((o) => (
 <option key={o.value} value={o.value}>
 {o.label}
 </option>
))}
 </select>
 <select value={minute} onChange={(e) => setMinute(Number(e.target.value))}
disabled={disabled} aria-label={` ${label} minute`} >
 {MINUTE_OPTIONS.map((m) => (
 <option key={m} value={m}>
 :{pad(m)}
 </option>
))}
 </select>
 </div>
 </div>
);
 }

function EventForm({
 initial,
 games,
 busy,
 onSave,
 onCancel,
}: {
 initial?: EventItem;
 games: GameOption[];
 busy: boolean;
 onSave: (payload: EventPayload) => void;
 onCancel: () => void;
}) {
 const [title, setTitle] = useState(initial?.title ?? '');
 const [description, setDescription] = useState(initial?.description ?? '');
 const [imageUrl, setImageUrl] = useState<string | null>(initial?.imageUrl ?? null);
 const [startsAt, setStartsAt] = useState<number | null>(initial?.startsAt ?? null);
 const [endsAt, setEndsAt] = useState<number | null>(initial?.endsAt ?? null);
 const [location, setLocation] = useState(initial?.location ?? '');
 const [gameId, setGameId] = useState<number | null>(initial?.gameId ?? null);
 const [recurrence, setRecurrence] = useState<Recurrence>(initial?.recurrence ?? 'none');
 const [roles, setRoles] = useState<RoleDraft[]>(
 initial?.roles.map((r) => ({
 id: r.id,
 name: r.name,
 emoji: r.emoji ?? '',
 capacity: r.capacity != null ? String(r.capacity) : '',
 }

```

**src/client/pages/Events.tsx**

```

))) ?? [],
);

 const timesValid = startsAt !== null && endsAt !== null && endsAt > startsAt;
 const valid = title.trim() && location.trim() && timesValid;

 const addRole = () => setRoles((rs) => [...rs, { name: '', emoji: '', capacity: '' }]);
 const updateRole = (i: number, patch: Partial<RoleDraft>) =>
 setRoles((rs) => rs.map((r, idx) => (idx === i ? { ...r, ...patch } : r)));
 const removeRole = (i: number) => setRoles((rs) => rs.filter((_, idx) => idx !== i));

 const submit = () =>
 onSave({
 title: title.trim(),
 description: description.trim(),
 imageUrl,
 startsAt: startsAt!,
 endsAt: endsAt!,
 location: location.trim(),
 gameId,
 recurrence,
 roles: roles
 .filter((r) => r.name.trim())
 .map((r) => ({
 id: r.id,
 name: r.name.trim(),
 emoji: r.emoji.trim() || null,
 capacity: r.capacity.trim() ? Math.max(1, Number(r.capacity)) : null,
 })),
 });

 return (
 <div className="role-editor event-form">
 <label>
 Title
 <input value={title} onChange={(e) => setTitle(e.target.value)} disabled={busy}
placeholder="Clan war vs. ..." />
 </label>

 <EventImageField imageUrl={imageUrl} busy={busy} onSet={setImageUrl} />

 <div className="field-row">
 <DateTimeField label="Starts" value={startsAt} onChange={setStartsAt} disabled={busy} />
 <DateTimeField label="Ends" value={endsAt} onChange={setEndsAt} disabled={busy} />
 </div>
 {!timesValid && (startsAt !== null || endsAt !== null) && (
 <p className="small warn">The end must be after the start.</p>
)}

 <label>
 Location
 <input
 value={location}
 onChange={(e) => setLocation(e.target.value)}

```

**src/client/pages/Events.tsx**

```

 disabled={busy}
 placeholder="In-game lobby, Discord voice, ..."
 />
</label>

<div className="field-row">
 <label>
 Game (optional)
 <select value={gameId ?? ''} onChange={(e) => setGameId(e.target.value ?
Number(e.target.value) : null)} disabled={busy}>
 <option value="">? None ?</option>
 {games.map((g) => (
 <option key={g.id} value={g.id}>
 {g.name}
 </option>
))}
 </select>
 </label>
 <label>
 Repeats
 <select value={recurrence} onChange={(e) => setRecurrence(e.target.value as Recurrence)}
disabled={busy}>
 {RECURRENT_OPTIONS.map((o) => (
 <option key={o.value} value={o.value}>
 {o.label}
 </option>
))}
 </select>
 </label>
</div>

<label>
 Description (optional)
 <textarea value={description} rows={3} onChange={(e) => setDescription(e.target.value)}
disabled={busy} />
</label>

<div className="event-roles-editor">
 <div className="field-label">
 Sign-up roles (optional ? e.g. Tank, Healer, DPS)
 </div>
 {roles.length > 0 && (
 <div className="role-row role-row-head">
 Icon
 Role name
 Max players

 </div>
)}
 {roles.map((r, i) => (
 <div key={i} className="role-row">
 <select
 className="role-col-icon"
 value={r.emoji}

```

**src/client/pages/Events.tsx**

```

 onChange={(e) => updateRole(i, { emoji: e.target.value })}
 disabled={busy}
 aria-label="Role icon"
 >
 <option value="">?</option>
 {ROLE_EMOJIS.map((em) => (
 <option key={em} value={em}>
 {em}
 </option>
))}
 </select>
 <input
 className="role-col-name"
 value={r.name}
 maxLength={40}
 placeholder="e.g. Tank"
 onChange={(e) => updateRole(i, { name: e.target.value })}
 disabled={busy}
 />
 <select
 className="role-col-cap"
 value={r.capacity}
 onChange={(e) => updateRole(i, { capacity: e.target.value })}
 disabled={busy}
 aria-label="Max players"
 >
 <option value="">No limit</option>
 {CAP_OPTIONS.map((n) => (
 <option key={n} value={String(n)}>
 {n}
 </option>
))}
 </select>
 <button
 className="mini danger role-col-rm"
 onClick={() => removeRole(i)}
 disabled={busy}
 title="Remove role"
 >
 ?
 </button>
 </div>
)}}
```

```

<button className="mini" onClick={addRole} disabled={busy || roles.length >= 20}>
 + Add role
</button>
</div>

<div className="news-editor-actions">
 <button className="primary" disabled={busy || !valid} onClick={submit}>
 {initial ? 'Save changes' : 'Create event'}
 </button>
 <button disabled={busy} onClick={onCancel}>
 Cancel

```

**src/client/pages/Events.tsx**

```

 </button>
 </div>
 </div>
);
}

/** Banner-image upload for an event, mirroring the medal/game icon uploaders. */
function EventImageField({
 imageUrl,
 busy,
 onSet,
}): {
 imageUrl: string | null;
 busy: boolean;
 onSet: (url: string | null) => void;
}) {
 const [uploading, setUploading] = useState(false);
 const [uploadError, setUploadError] = useState<string | null>(null);

 async function pickFile(e: ChangeEvent<HTMLInputElement>) {
 const file = e.target.files?.[0];
 e.target.value = '';
 if (!file) return;
 setUploadError(null);
 setUploading(true);
 try {
 const { url } = await api.upload<{ url: string }>('/media/events', file);
 onSet(url);
 } catch (err) {
 setUploadError(err instanceof Error ? err.message : 'Upload failed.');
```

} finally {
 setUploading(false);
 }
 }

 return (
 <div className="event-image-field">
 <div className="field-label">
 Banner image <span className="muted small">(optional)</span>
 </div>
 {imageUrl && <img className="event-banner preview" src={imageUrl} alt="" />}
 <div className="avatar-controls">
 <label className="upload-btn mini">
 {uploading ? 'Uploading...' : imageUrl ? 'Change' : 'Upload'}
 <input
 type="file"
 accept="image/png,image/jpeg,image/gif,image/webp"
 onChange={(e) => void pickFile(e)}
 disabled={busy || uploading}
 hidden
 />
 </label>
 {imageUrl && (
 <button className="mini" disabled={busy || uploading} onClick={() => onSet(null)}

**src/client/pages/Events.tsx**

```
title="Remove image">
 Remove
 </button>
)}
 {uploadError && {uploadError}}
</div>
</div>
);
}
```

**src/client/pages/FooterAdmin.tsx**

```

/**
 * Settings -> Theme & Branding -> Footer.
 *
 * A lean, per-install site footer: a short blurb (basic HTML allowed, sanitised
 * on render), a few links, and a copyright line. Shows on every page. The
 * default ships with the trademark / non-affiliation notice + a Legal link.
 */

import { useEffect, useState } from 'react';
import { api } from '../lib/api';
import { useAction, Alerts } from '../lib/action';
import type { FooterConfig, FooterLink } from '../shared/footer';

export default function FooterAdmin() {
 const [loading, setLoading] = useState(true);
 const [text, setText] = useState('');
 const [links, setLinks] = useState<FooterLink[]>([]);
 const [copyright, setCopyright] = useState('');
 const { run, busy, error, notice, warning } = useAction();

 const apply = (f: FooterConfig) => {
 setText(f.text);
 setLinks(f.links);
 setCopyright(f.copyright);
 };

 useEffect(() => {
 api
 .get<{ footer: FooterConfig }>('/settings/footer')
 .then(({ footer }) => apply(footer))
 .finally(() => setLoading(false));
 }, []);

 const save = () =>
 run(async () => {
 const { footer } = await api.put<{ footer: FooterConfig }>('/settings/footer', {
 footer: { text, links, copyright },
 });
 apply(footer);
 return 'Saved. The footer is live on every page.';
 });

 const setLink = (i: number, patch: Partial<FooterLink>) =>
 setLinks((ls) => ls.map((l, li) => (li === i ? { ...l, ...patch } : l)));
 const addLink = () => setLinks((ls) => [...ls, { label: '', href: '' }]);
 const removeLink = (i: number) => setLinks((ls) => ls.filter((_, li) => li !== i));

 if (loading) return <div className="loading">Loading...</div>;

 return (
 <section className="panel footer-admin">
 <header className="panel-head">
 <h2>Footer</h2>
 <p className="muted">

```



**src/client/pages/FooterAdmin.tsx**

Shown at the bottom of every page. Basic formatting is allowed in the blurb; scripts and styles are stripped when it renders.

```

 </p>
 </header>

 <Alerts error={error} warning={warning} notice={notice} />

 <label>
 Blurb
 <textarea
 rows={3}
 value={text}
 disabled={busy}
 placeholder="A short line ? e.g. a disclaimer or tagline. Basic HTML (links, bold)
allowed."
 onChange={(e) => setText(e.target.value)}
 />
 </label>

 <fieldset>
 <legend>Links</legend>
 {links.length === 0 && <p className="muted small">No links yet.</p>}
 {links.map((l, i) => (
 <div className="footer-link-row" key={i}>
 <input
 type="text"
 value={l.label}
 placeholder="Label"
 maxLength={60}
 disabled={busy}
 onChange={(e) => setLink(i, { label: e.target.value })}
 />
 <input
 type="text"
 value={l.href}
 placeholder="/p/legal or https://..."
 disabled={busy}
 onChange={(e) => setLink(i, { href: e.target.value })}
 />
 <button type="button" className="mini danger" title="Remove link" disabled={busy}
onClick={() => removeLink(i)}>
 ?
 </button>
 </div>
))}
 <button type="button" className="mini" disabled={busy} onClick={addLink}>
 + Add link
 </button>
 </fieldset>

 <label>
 Copyright line
 <input
 type="text"

```

**src/client/pages/FooterAdmin.tsx**

```
 value={copyright}
 maxLength={200}
 disabled={busy}
 placeholder="Leave blank to auto-show "? {year} {site name}""
 onChange={(e) => setCopyright(e.target.value)}
 />
 </label>

 <div className="news-editor-actions">
 <button className="primary" disabled={busy} onClick={() => void save()}>
 Save
 </button>
 </div>
 </section>
);
}
```

**src/client/pages/Games.tsx**

```

/**
 * Games catalog administration ? add, rename, toggle active, set an icon, or
 * delete the titles the clan plays. War records can be tied to a game.
 */

import { useEffect, useState, type ChangeEvent } from 'react';
import { api } from '../lib/api';
import { useAction, Alerts } from '../lib/action';

interface Game {
 id: number;
 name: string;
 slug: string;
 iconUrl: string | null;
 active: boolean;
 sortOrder: number;
 recordCount: number;
}

export default function Games() {
 const [games, setGames] = useState<Game[]>([]);
 const [newName, setNewName] = useState('');
 const [loading, setLoading] = useState(true);

 async function load() {
 const { games } = await api.get<{ games: Game[] }>('/games');
 setGames(games);
 }

 const { run, busy, error, notice, warning } = useAction(load);

 useEffect(() => {
 load().finally(() => setLoading(false));
 }, []);

 const addGame = () =>
 run(async () => {
 if (!newName.trim()) return;
 await api.post('/games', { name: newName.trim() });
 setNewName('');
 return `Added "${newName.trim()}"`;
 });

 const rename = (id: number, name: string) => run(() => api.patch(`/games/${id}`, { name }));
 const setActive = (id: number, active: boolean) => run(() => api.patch(`/games/${id}`, { active }));
 const setIcon = (id: number, iconUrl: string | null) => run(() => api.patch(`/games/${id}`, { iconUrl }));
 const remove = (id: number) => run(() => api.del(`/games/${id}`));

 if (loading) return <div className="loading">Loading games...</div>;

 return (
 <section className="panel">

```

**src/client/pages/Games.tsx**

```

<header className="panel-head">
 <h2>Games</h2>
 <p className="muted">
 {games.length} game{games.length === 1 ? '' : 's'}. Inactive games stay for history but
 drop off pickers.
 </p>
</header>

<Alerts error={error} warning={warning} notice={notice} />

<div className="add-row">
 <input
 value={newName}
 placeholder="New game name"
 onChange={(e) => setNewName(e.target.value)}
 onKeyDown={(e) => e.key === 'Enter' && void addGame()}
 disabled={busy}
 />
 <button onClick={() => void addGame()} disabled={busy || !newName.trim()}>
 Add game
 </button>
</div>

{games.length === 0 ? (
 <p className="muted">No games yet. Add the titles your clan plays.</p>
) : (
 <table className="grid">
 <thead>
 <tr>
 <th>Icon</th>
 <th>Name</th>
 <th>Active</th>
 <th>War records</th>
 </tr>
 </thead>
 <tbody>
 {games.map((game) => (
 <tr key={game.id}>
 <td>
 <GameIconCell game={game} busy={busy} onSet={(url) => setIcon(game.id, url)} />
 </td>
 <td>
 <input
 defaultValue={game.name}
 onBlur={(e) => e.target.value !== game.name && void rename(game.id,
e.target.value)}
 disabled={busy}
 />
 </td>
 <td>
 <input
 type="checkbox"
 checked={game.active}

```

**src/client/pages/Games.tsx**

```

 onChange={(e) => void setActive(game.id, e.target.checked)}
 disabled={busy}
 title={game.active ? 'Active' : 'Inactive'}
 />
 </td>
 <td>{game.recordCount}</td>
 <td>
 <button
 className="danger"
 onClick={() => void remove(game.id)}
 disabled={busy}
 title="Delete this game (war records tied to it become clan-wide)"
 >
 Delete
 </button>
 </td>
 </tr>
))}
</tbody>
</table>
)}
</section>
);
}

/** A game's icon, with upload/remove ? mirrors the rank insignia cell. */
function GameIconCell({
 game,
 busy,
 onSet,
}): {
 game: Game;
 busy: boolean;
 onSet: (url: string | null) => void;
}) {
 const [uploading, setUploading] = useState(false);

 async function pickFile(e: ChangeEvent<HTMLInputElement>) {
 const file = e.target.files?.[0];
 e.target.value = '';
 if (!file) return;
 setUploading(true);
 try {
 const { url } = await api.upload<{ url: string }>('/media/games', file);
 onSet(url);
 } finally {
 setUploading(false);
 }
 }

 return (
 <div className="rank-image-cell">

 {game.iconUrl ? (

```

**src/client/pages/Games.tsx**

```


) : (
 ?
)}

<label className="upload-btn mini">
 {uploading ? '...' : game.iconUrl ? 'Change' : 'Upload'}
 <input
 type="file"
 accept="image/png,image/jpeg,image/gif,image/webp"
 onChange={(e) => void pickFile(e)}
 disabled={busy || uploading}
 hidden
 />
</label>
{game.iconUrl && (
 <button className="mini" disabled={busy || uploading} onClick={() => onSet(null)}
title="Remove icon">
 ?
 </button>
)}
</div>
);
}

```

**src/client/pages/Home.tsx**

```
/**
 * The front page. Rather than hard-coding a layout, it loads the admin-arranged
 * 'home' layout (falling back to the built-in default) and renders it through
 * the module system, so what appears here is whatever modules an admin has
 * arranged in the Pages editor.
 */

import { useEffect, useState } from 'react';
import { api } from '../lib/api';
import PageRenderer from '../components/PageRenderer';
import { defaultLayout, sanitizeLayout, type PageLayout } from '../shared/layout';

export default function Home() {
 const [layout, setLayout] = useState<PageLayout | null>(null);

 useEffect(() => {
 api
 .get<{ layout: unknown }>('/pages/home')
 .then(({ layout }) => setLayout(sanitizeLayout(layout)))
 .catch(() => setLayout(defaultLayout('home')));
 }, []);

 if (!layout) return <div className="loading">Loading...</div>;
 return <PageRenderer layout={layout} />;
}
```

**src/client/pages/IdentityAdmin.tsx**

```

/**
 * Identity & Discord admin ? the setup/config surface for a self-hosted install.
 *
 * Everything Discord-login and site-identity related that used to live only in
 * wrangler.jsonc is editable here: point the site at your own Discord app and
 * domain without redeploying. Saved values override the deploy defaults; blank
 * fields fall back to them. Secret values (client secret, bot token) are never
 * sent back from the server ? the form only knows whether one is set ? so a
 * blank secret field means "keep the current one".
 */

import { useEffect, useState } from 'react';
import { api } from '../lib/api';
import { useAction, Alerts } from '../lib/action';

interface Identity {
 siteName: string;
 siteUrl: string;
 discord: {
 clientId: string;
 guildId: string;
 publicKey: string;
 clientSecretSet: boolean;
 botTokenSet: boolean;
 };
}

interface Urls {
 redirectUri: string;
 interactionsUrl: string;
}

/** A plain external link that opens safely in a new tab. */
function Ext({ href, children }: { href: string; children: string }) {
 return (

 {children}

);
}

function CopyField({ label, value }: { label: string; value: string }) {
 const [copied, setCopied] = useState(false);
 const copy = async () => {
 try {
 await navigator.clipboard.writeText(value);
 setCopied(true);
 setTimeout(() => setCopied(false), 1500);
 } catch {
 /* clipboard blocked ? the value is still visible to select manually */
 }
 };
 return (
 <div className="identity-url">

```



**src/client/pages/IdentityAdmin.tsx**

```

 {label}
 <div className="identity-url-row">
 <code>{value}</code>
 <button type="button" className="ghost small" onClick={() => void copy()}>
 {copied ? 'Copied' : 'Copy'}
 </button>
 </div>
 </div>
</div>
);
}

```

```

export default function IdentityAdmin() {
 const [loading, setLoading] = useState(true);
 const [saved, setSaved] = useState<Identity | null>(null);
 const [urls, setUrls] = useState<Urls | null>(null);

 const [siteName, setSiteName] = useState('');
 const [siteUrl, setSiteUrl] = useState('');
 const [clientId, setClientId] = useState('');
 const [guildId, setGuildId] = useState('');
 const [publicKey, setPublicKey] = useState('');
 // Secrets are write-only: empty means "leave unchanged".
 const [clientSecret, setClientSecret] = useState('');
 const [botToken, setBotToken] = useState('');

 const { run, busy, error, notice, warning } = useAction();

 const apply = (id: Identity) => {
 setSaved(id);
 setSiteName(id.siteName);
 setSiteUrl(id.siteUrl);
 setClientId(id.discord.clientId);
 setGuildId(id.discord.guildId);
 setPublicKey(id.discord.publicKey);
 setClientSecret('');
 setBotToken('');
 };

 useEffect(() => {
 api
 .get<{ identity: Identity; urls: Urls }>('/settings/identity')
 .then(({ identity, urls }) => {
 apply(identity);
 setUrls(urls);
 })
 .finally(() => setLoading(false));
 }, []);

 const dirty =
 !!saved &&
 (siteName !== saved.siteName ||
 siteUrl !== saved.siteUrl ||
 clientId !== saved.discord.clientId ||
 guildId !== saved.discord.guildId ||

```

**src/client/pages/IdentityAdmin.tsx**

```

 publicKey !== saved.discord.publicKey ||
 clientSecret !== '' ||
 botToken !== '');

const save = () =>
 run(async () => {
 const { identity } = await api.put<{ identity: Identity }>('/settings/identity', {
 identity: {
 siteName,
 siteUrl,
 discord: { clientId, guildId, publicKey, clientSecret, botToken },
 },
 });
 apply(identity);
 return 'Saved. Discord login now uses these settings.';
 });

const test = () =>
 run(async () => {
 const res = await api.post<{ ok: boolean; channelCount?: number; error?: string }>(
 '/settings/identity/test',
);
 if (!res.ok) return { warning: res.error ?? 'The bot could not connect.' };
 return `Bot connected ? it can see ${res.channelCount} text channel${res.channelCount === 1
 ? '' : 's'}.`;
 });

if (loading) return <div className="loading">Loading...</div>;

// Deep-link straight to this app's pages once the Client ID is known; otherwise
// send them to the applications list to pick (or create) the app.
const appBase = clientId
 ? `https://discord.com/developers/applications/${clientId}`
 : 'https://discord.com/developers/applications';

return (
 <section className="panel identity-admin">
 <header className="panel-head">
 <h2>Identity & Discord</h2>
 <p className="muted">
 Point this site at your own Discord application and domain. Saved values override the
 deploy defaults; leave a field blank to fall back to the default.
 </p>
 </header>

 <Alerts error={error} warning={warning} notice={notice} />

 <fieldset>
 <legend>Site</legend>
 <label>
 Site name
 <input value={siteName} onChange={(e) => setSiteName(e.target.value)} disabled={busy}
 maxLength={80} />
 </label>

```

**src/client/pages/IdentityAdmin.tsx**

```

 <label>
 Canonical site URL
 <input
 value={siteUrl}
 placeholder="https://mustr.gg"
 onChange={(e) => setSiteUrl(e.target.value)}
 disabled={busy}
 />

 Used to build absolute links in Discord embeds. Blank = use the deploy default.

 </label>
</fieldset>

<fieldset>
 <legend>Discord application</legend>
 <p className="muted small">
 All four values come from{' '}
 <Ext href="https://discord.com/developers/applications">the Discord Developer
Portal</Ext>.
 Create an application there (or open your existing one), then copy each value from the
page
 noted below. {clientId} && <>Quick links open your app.</>
 </p>
 <label>
 Client ID
 <input value={clientId} onChange={(e) => setClientId(e.target.value)} disabled={busy}
inputMode="numeric" />

 Your app -> General Information -> "Application ID".{' '}
 <Ext href={clientId ? `${appBase}/information` : appBase}>Open ?</Ext>

 </label>
 <label>
 Server (Guild) ID
 <input value={guildId} onChange={(e) => setGuildId(e.target.value)} disabled={busy}
inputMode="numeric" />

 Login is gated on membership in this Discord server. In Discord, enable{' '}
 Settings -> Advanced -> Developer Mode, then right-click your server
icon ->
 "Copy Server ID".{' '}
 <Ext href="https://support.discord.com/hc/en-us/articles/206346498">How ?</Ext>

 </label>
 <label>
 Public Key
 <input value={publicKey} onChange={(e) => setPublicKey(e.target.value)} disabled={busy}
/>

 64 hex characters. Your app -> General Information -> "Public Key".{'
 '
 <Ext href={clientId ? `${appBase}/information` : appBase}>Open ?</Ext>


```

**src/client/pages/IdentityAdmin.tsx**

```

 </label>
 <label>
 Client Secret
 <input
 type="password"
 value={clientSecret}
 placeholder={saved?.discord.clientSecretSet ? '***** (set ? leave blank to keep)' :
'not set'}
 onChange={(e) => setClientSecret(e.target.value)}
 disabled={busy}
 autoComplete="new-password"
 />

 Your app -> OAuth2 -> "Reset Secret" (shown once). Powers Discord
login.{ ' ' }
 <Ext href={clientId ? `${appBase}/oauth2` : appBase}>Open ?</Ext>

 </label>
 <label>
 Bot Token
 <input
 type="password"
 value={botToken}
 placeholder={saved?.discord.botTokenSet ? '***** (set ? leave blank to keep)' :
'not set'}
 onChange={(e) => setBotToken(e.target.value)}
 disabled={busy}
 autoComplete="new-password"
 />

 Your app -> Bot -> "Reset Token" (shown once). Powers announcements
and role
 sync. Stored in this site's database ? see the note below.{ ' ' }
 <Ext href={clientId ? `${appBase}/bot` : appBase}>Open ?</Ext>

 </label>
 </fieldset>

 {urls && (
 <fieldset>
 <legend>Paste these into Discord</legend>
 <p className="muted small">
 In the Discord Developer Portal for your app, add the redirect URL under{' ' }
 OAuth2 -> Redirects, and set the{' ' }
 Interactions Endpoint URL under General Information. They must match
 this site's address exactly.
 </p>
 <CopyField label="OAuth2 redirect URL" value={urls.redirectUri} />
 <CopyField label="Interactions Endpoint URL" value={urls.interactionsUrl} />
 </fieldset>
)}

 <p className="muted small warn">
 Note: the client secret and bot token are stored in this site's database (not encrypted

```

**src/client/pages/IdentityAdmin.tsx**

```
Cloudflare secrets), so anyone with database access or god-admin rights can read them. Use
a
 Discord app dedicated to this site.
</p>

<div className="news-editor-actions">
 <button className="primary" disabled={busy || !dirty} onClick={() => void save()}>
 Save
 </button>
 <button
 disabled={busy}
 onClick={() => void test()}
 title="Check the saved bot token and Server ID actually reach Discord"
 >
 Test bot connection
 </button>
</div>
</section>
);
}
```

**src/client/pages/Login.tsx**

```
import { Navigate, useSearchParams } from 'react-router-dom';
import { useSession } from '../lib/session';

const ERRORS: Record<string, string> = {
 state: 'That sign-in attempt expired or didn't match. Please try again.',
 not_in_guild: 'You need to be a member of the clan's Discord server to sign in.',
 banned: 'This account has been banned.',
 discord: 'Discord declined the sign-in.',
};

export default function Login() {
 const { viewer, siteName } = useSession();
 const [params] = useSearchParams();
 const error = params.get('error');
 const detail = params.get('detail');

 if (viewer) return <Navigate to="/" replace />;

 const message = error
 ? (ERRORS[error] ?? 'Sign-in failed. Please try again.') +
 (error === 'discord' && detail ? ` (${detail})` : '')
 : null;

 return (
 <div className="login">
 <h1>{siteName}</h1>
 <p className="muted">Sign in with the Discord account you use in the clan server.</p>

 {message && <div className="alert">{message}</div>}

 Sign in with Discord

 </div>
);
}
```

**src/client/pages/Medals.tsx**

```

/**
 * Medal administration ? the catalog of medal definitions.
 *
 * A medal is a name, an optional image, and an optional tenure rule: set
 * "auto-award after N months" and the server hands it out automatically once a
 * member has been in the guild that long (see server/medals/tenure.ts). Handing
 * a medal to a specific member happens on that member's page, not here.
 */

import { useEffect, useState } from 'react';
import { api } from '../lib/api';
import { useAction, Alerts } from '../lib/action';

interface Medal {
 id: number;
 name: string;
 description: string | null;
 imageUrl: string | null;
 gameId: number | null;
 autoGrantMonths: number | null;
 sortOrder: number;
 awardCount: number;
}

// Common tenure milestones, offered as quick picks. Any positive number works.
const TENURE_PRESETS: [label: string, months: number][] = [
 ['6 months', 6],
 ['1 year', 12],
 ['2 years', 24],
 ['3 years', 36],
 ['5 years', 60],
];

export default function Medals() {
 const [medals, setMedals] = useState<Medal[]>([]);
 const [selectedId, setSelectedId] = useState<number | null>(null);
 const [newName, setNewName] = useState('');
 const [loading, setLoading] = useState(true);

 async function load() {
 const { medals } = await api.get<{ medals: Medal[] }>('/medals');
 setMedals(medals);
 }

 const { run, busy, error, notice, warning } = useAction(load);

 useEffect(() => {
 load().finally(() => setLoading(false));
 }, []);

 const addMedal = () =>
 run(async () => {
 if (!newName.trim()) return;
 const { medal } = await api.post<{ medal: Medal }>('/medals', { name: newName.trim() });

```

**src/client/pages/Medals.tsx**

```

 setNewName('');
 setSelectedId(medal.id);
 return `Created "${medal.name}".`;
 });

const selected = medals.find((m) => m.id === selectedId) ?? null;

if (loading) return <div className="loading">Loading medals...</div>;

return (
 <section className="panel medals-page">
 <header className="panel-head">
 <h2>Medals</h2>
 <p className="muted">
 {medals.length} medal{medals.length === 1 ? '' : 's'}. Give a member a medal from their
 profile; tenure medals are awarded automatically.
 </p>
 </header>

 <Alerts error={error} warning={warning} notice={notice} />

 <div className="add-row">
 <input
 value={newName}
 placeholder="New medal name"
 onChange={(e) => setNewName(e.target.value)}
 onKeyDown={(e) => e.key === 'Enter' && void addMedal()}
 disabled={busy}
 />
 <button onClick={() => void addMedal()} disabled={busy || !newName.trim()}>
 Add medal
 </button>
 </div>

 <div className="roles-layout">
 <ul className="medal-list">
 {medals.map((medal) => (
 <li key={medal.id}>
 <button
 className={medal.id === selectedId ? 'medal-chip active' : 'medal-chip'}
 onClick={() => setSelectedId(medal.id)}
 >

 {medal.imageUrl ? (

) : (
 ?
)}

 {medal.name}
 {medal.autoGrantMonths !== null && tenure}
 {medal.awardCount}
 </button>

)}

 </div>
 </section>
);

```



**src/client/pages/Medals.tsx**

```

))}

 {selected ? (
 <MedalEditor
 key={selected.id}
 medal={selected}
 busy={busy}
 onSave={(patch) =>
 run(async () => {
 await api.patch(`/medals/${selected.id}`, patch);
 return 'Saved.';
 })
 }
 onUpload={async (file) => {
 const { url } = await api.upload<{ url: string }>('/media/medals', file);
 return url;
 }}
 onDelete={() =>
 run(async () => {
 await api.del(`/medals/${selected.id}`);
 setSelectedId(null);
 return 'Medal deleted.';
 })
 }
 />
) : (
 <div className="role-editor empty-editor">Select a medal to edit it.</div>
)}
</div>
</section>
);
}

function MedalEditor({
 medal,
 busy,
 onSave,
 onUpload,
 onDelete,
}): {
 medal: Medal;
 busy: boolean;
 onSave: (patch: Partial<Medal>) => void;
 onUpload: (file: File) => Promise<string>;
 onDelete: () => void;
} {
 const [name, setName] = useState(medal.name);
 const [description, setDescription] = useState(medal.description ?? '');
 const [imageUrl, setImageUrl] = useState(medal.imageUrl);
 const [months, setMonths] = useState<string>(
 medal.autoGrantMonths != null ? String(medal.autoGrantMonths) : '',
);
 const [uploading, setUploading] = useState(false);

```

**src/client/pages/Medals.tsx**

```

const [uploadError, setUploadError] = useState<string | null>(null);

const parsedMonths = months.trim() === '' ? null : Number(months);
const monthsValid =
 parsedMonths === null || (Number.isInteger(parsedMonths) && parsedMonths > 0);

const dirty =
 name !== medal.name ||
 description !== (medal.description ?? '') ||
 imageUrl !== medal.imageUrl ||
 parsedMonths !== medal.autoGrantMonths;

async function pickFile(e: React.ChangeEvent<HTMLInputElement>) {
 const file = e.target.files?.[0];
 e.target.value = ''; // allow re-selecting the same file
 if (!file) return;
 setUploadError(null);
 setUploading(true);
 try {
 const url = await onUpload(file);
 setImageUrl(url);
 } catch (err) {
 setUploadError(err instanceof Error ? err.message : 'Upload failed.');
```

} finally {
 setUploading(false);
 }
 }
}

return (
 <div className="role-editor medal-editor">
 <div className="medal-editor-head">
 <span className="medal-thumb large">
 {imageUrl ? (
 <img src={imageUrl} alt="" width={64} height={64} />
 ) : (
 <span className="medal-thumb-empty">?</span>
 )}
 </span>
 <div className="medal-image-controls">
 <label className="upload-btn">
 {uploading ? 'Uploading...' : imageUrl ? 'Replace image' : 'Upload image'}
 <input
 type="file"
 accept="image/png,image/jpeg,image/gif,image/webp"
 onChange={(e) => void pickFile(e)}
 disabled={busy || uploading}
 hidden
 />
 </label>
 {imageUrl && (
 <button className="mini" disabled={busy || uploading} onClick={() =>
 setImageUrl(null)}>
 Remove image
 </button>
 )}
 </div>
 </div>
 </div>
)

**src/client/pages/Medals.tsx**

```

 })
 PNG, JPEG, GIF, or WebP ? up to 1 MB.
 {uploadError && {uploadError}}
 </div>
</div>

<label>
 Name
 <input value={name} onChange={(e) => setName(e.target.value)} disabled={busy} />
</label>

<label>
 Description
 <input
 value={description}
 onChange={(e) => setDescription(e.target.value)}
 placeholder="What this medal is for"
 disabled={busy}
 />
</label>

<fieldset className="tenure">
 <legend>Auto-award by time in guild</legend>
 <div className="tenure-row">
 <input
 type="number"
 min={1}
 step={1}
 value={months}
 placeholder="e.g. 12"
 onChange={(e) => setMonths(e.target.value)}
 disabled={busy}
 />

 months in the guild. Leave blank to award this medal by hand only.

 </div>
 <div className="tenure-presets">
 {TENURE_PRESETS.map(([label, m]) => (
 <button
 key={m}
 type="button"
 className={parsedMonths === m ? 'preset active' : 'preset'}
 disabled={busy}
 onClick={() => setMonths(String(m))}
 >
 {label}
 </button>
))}
 {parsedMonths !== null && (
 <button type="button" className="preset" disabled={busy} onClick={() =>
setMonths('')}>
 Clear
 </button>
)}
 </div>

```

**src/client/pages/Medals.tsx**

```

 })
 </div>
 {!monthsValid && <p className="small warn">Enter a positive whole number of months.</p>}
</fieldset>

<button
 className="primary"
 disabled={busy || uploading || !dirty || !monthsValid || !name.trim()}
 onClick={() =>
 onSave({
 name: name.trim(),
 description,
 imageUrl,
 autoGrantMonths: parsedMonths,
 })
 }
>
 Save
</button>

<div className="editor-footer">
 <button className="danger" disabled={busy} onClick={onDelete} title="Delete this medal">
 Delete medal
 </button>
 {medal.awardCount > 0 && (

 Held by {medal.awardCount} member{medal.awardCount === 1 ? '' : 's'}
 {medal.autoGrantMonths !== null && ' ? deleting removes it from all of them'}

)}
</div>
</div>
);
}

```

**src/client/pages/MemberDetail.tsx**

```

/**
 * A single member: identity, rank, self-editable bio, roles, and medals.
 *
 * Admin controls (promote/demote, status, role grants) are each gated on the
 * matching permission via the session's can(). That means holders of the
 * top-level Command role see them, and God is a fallback ? not a special case.
 */

import { useCallback, useEffect, useState, type ChangeEvent } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import { api } from '../lib/api';
import { useAction, Alerts } from '../lib/action';
import { useSession } from '../lib/session';
import { memberName } from '../../shared/names';
import { memberAvatar } from '../../shared/avatar';
import { useRecordRecent } from '../lib/recent';
import { useModules, useScConfig } from '../lib/modules';
import HangarView from '../components/HangarView';
import HangarImport from '../components/HangarImport';
import ScVerify from '../components/ScVerify';

interface Role {
 id: number;
 name: string;
 color: string | null;
 source?: 'manual' | 'rank';
}

interface Medal {
 awardId: number;
 id: number;
 name: string;
 imageUrl: string | null;
 citation: string | null;
 // NULL = awarded automatically by the tenure sweep, not by a person.
 awardedBy: number | null;
 awardedAt: number;
}

/** A medal definition, for the award picker. */
interface MedalDef {
 id: number;
 name: string;
 imageUrl: string | null;
 autoGrantMonths: number | null;
}

interface WarRecord {
 awardId: number;
 id: number;
 name: string;
 imageUrl: string | null;
 gameName: string | null;
 citation: string | null;
 awardedBy: number | null;
 awardedAt: number;
}

```

**src/client/pages/MemberDetail.tsx**

```

/** A war-record definition, for the award picker. */
interface WarRecordDef {
 id: number;
 name: string;
 imageUrl: string | null;
 gameName: string | null;
}
interface Member {
 id: number;
 discordId: string;
 username: string;
 globalName: string | null;
 displayName: string | null;
 avatar: string | null;
 profileImageUrl: string | null;
 status: string;
 joinedAt: number;
 rank: { id: number; name: string; sortOrder: number; imageUrl: string | null } | null;
 roles: Role[];
 medals: Medal[];
 warRecords: WarRecord[];
 bio: string | null;
}
interface Rank {
 id: number;
 name: string;
 sortOrder: number;
}

const STATUSES = ['active', 'inactive', 'loa', 'retired', 'banned'] as const;

export default function MemberDetail() {
 const { id } = useParams();
 const memberId = Number(id);
 const navigate = useNavigate();
 const { viewer, can } = useSession();
 const modules = useModules();
 const sc = useScConfig();
 // Bumped after a self-import so the hangar view below re-fetches.
 const [hangarKey, setHangarKey] = useState(0);

 const [member, setMember] = useState<Member | null>(null);
 const [ranks, setRanks] = useState<Rank[]>([]);
 const [assignable, setAssignable] = useState<Role[]>([]);
 const [medalCatalog, setMedalCatalog] = useState<MedalDef[]>([]);
 const [warRecordCatalog, setWarRecordCatalog] = useState<WarRecordDef[]>([]);
 const [loading, setLoading] = useState(true);

 const load = useCallback(async () => {
 const { member } = await api.get<{ member: Member }>(`/members/${memberId}`);
 setMember(member);
 }, [memberId]);

 const { run, busy, error, notice, warning, setError } = useAction(load);

```

**src/client/pages/MemberDetail.tsx**

```

// Negative actions (demote, remove/ban, revoke an award or role) require a
// written reason. Rather than a prompt per button, they funnel through one
// inline reason box: promptReason stashes the label + the call to make once a
// reason is entered, and the box renders near the top of the panel.
const [pending, setPending] = useState<{ label: string; onConfirm: (reason: string) => void } |
null>(
 null,
);
const [reasonText, setReasonText] = useState('');
const promptReason = (label: string, onConfirm: (reason: string) => void) => {
 setReasonText('');
 setPending({ label, onConfirm });
};
const confirmReason = () => {
 const reason = reasonText.trim();
 if (reason.length < 3 || !pending) return;
 pending.onConfirm(reason);
 setPending(null);
 setReasonText('');
};

useEffect(() => {
 setLoading(true);
 Promise.all([
 load(),
 api.get<{ ranks: Rank[] }>('/ranks').then(({ ranks }) => setRanks(ranks)),
 can('roles.assign')
 ? api.get<{ roles: Role[] }>('/roles/assignable').then(({ roles }) =>
setAssignable(roles))
 : Promise.resolve(),
 can('medals.award')
 ? api.get<{ medals: MedalDef[] }>('/medals').then(({ medals }) => setMedalCatalog(medals))
 : Promise.resolve(),
 can('warrecords.award')
 ? api
 .get<{ warRecords: WarRecordDef[] }>('/warrecords')
 .then(({ warRecords }) => setWarRecordCatalog(warRecords))
 : Promise.resolve(),
])
 .catch(() => setError('Failed to load member.'))
 .finally(() => setLoading(false));
}, [load, can, setError]);

useRecordRecent(member ? { group: 'people', label: memberName(member), to:
`/members/${member.id}` } : null);

if (loading) return <div className="loading">Loading member...</div>;
if (!member) return <div className="empty">Member not found.</div>;

const isSelf = viewer?.id === member.id;
const canEditProfile = isSelf || can('roster.edit');
const displayName = memberName(member);
// The underlying Discord identity, shown when a display name overrides it.

```

**src/client/pages/MemberDetail.tsx**

```

const discordHandle = member.globalName ?? member.username;
const showsDiscordHandle = displayName !== discordHandle;

// Adjacent ranks for one-step promote/demote, and whether the viewer outranks
// the target (God ignores the ladder).
const ordered = [...ranks].sort((a, b) => a.sortOrder - b.sortOrder);
const curOrder = member.rank?.sortOrder ?? -1;
const nextUp = ordered.find((r) => r.sortOrder > curOrder) ?? null;
const nextDown = [...ordered].reverse().find((r) => r.sortOrder < curOrder) ?? null;
const viewerOrder = viewer?.rank?.sortOrder ?? -1;
const outranksTarget = viewer?.isGod || (curOrder >= 0 && viewerOrder > curOrder) || curOrder <
0;
const canPromoteTo = (r: Rank | null) => !!r && (viewer?.isGod || viewerOrder > r.sortOrder);

const setRank = (rankId: number | null, label: string, reason?: string) =>
 run(async () => {
 const res = await api.put<{ rankRoleSync?: { added: string[]; removed: string[]; warnings:
string[] } }>(`members/${member.id}/rank`,
 { rankId, reason },
);
 const s = res.rankRoleSync;
 const changes = [
 s?.added.length ? `roles added: ${s.added.join(', ')}` : '',
 s?.removed.length ? `removed: ${s.removed.join(', ')}` : '',
]
 .filter(Boolean)
 .join('; ');
 const summary = `${label}.${changes ? ` (${changes})` : ''}`;
 return s?.warnings.length ? { warning: `${summary} ${s.warnings[0]}` } : summary;
 });

const setStatus = (status: string, reason?: string) =>
 run(async () => {
 await api.patch(`/members/${member.id}/status`, { status, reason });
 return `Status set to ${status}.`;
 });

const resyncDiscord = () =>
 run(async () => {
 const res = await api.post<{ ok: boolean; added?: number; removed?: number; warning?: string
}>(`members/${member.id}/resync`,
);
 if (res.warning) return { warning: res.warning };
 const n = (res.added ?? 0) + (res.removed ?? 0);
 return n === 0
 ? 'Discord already matches the website.'
 : `Discord synced ? ${res.added} added, ${res.removed} removed.`;
 });

const grantRole = (roleId: number) =>
 run(async () => {
 const res = await api.post<{ warning?: string }>(`members/${member.id}/roles`, { roleId });
 });

```



**src/client/pages/MemberDetail.tsx**

```
 return res.warning ? { warning: res.warning } : 'Role granted.';
 });

 const revokeRole = (roleId: number, reason: string) =>
 run(async () => {
 const res = await api.del<{ warning?: string }>(`/members/${member.id}/roles/${roleId}`, {
reason });
 return res.warning ? { warning: res.warning } : 'Role removed.';
 });

 const awardMedal = (medalId: number, citation: string) =>
 run(async () => {
 await api.post(`/members/${member.id}/medals`, {
 medalId,
 citation: citation.trim() || undefined,
 });
 return 'Medal awarded.';
 });

 const revokeMedal = (awardId: number, reason: string) =>
 run(async () => {
 await api.del(`/members/${member.id}/medals/${awardId}`, { reason });
 return 'Medal revoked.';
 });

 const awardWarRecord = (warRecordId: number, citation: string) =>
 run(async () => {
 await api.post(`/members/${member.id}/warrecords`, {
 warRecordId,
 citation: citation.trim() || undefined,
 });
 return 'War record awarded.';
 });

 const revokeWarRecord = (awardId: number, reason: string) =>
 run(async () => {
 await api.del(`/members/${member.id}/warrecords/${awardId}`, { reason });
 return 'War record revoked.';
 });

 const setProfileImage = (profileImageUrl: string | null) =>
 run(async () => {
 await api.patch(`/members/${member.id}/profile`, { profileImageUrl });
 return profileImageUrl
 ? 'Profile image updated.'
 : 'Profile image removed ? using the Discord avatar.';
 });

 const heldRoleIds = new Set(member.roles.map((r) => r.id));
 const grantable = assignable.filter((r) => !heldRoleIds.has(r.id));

 const heldMedalIds = new Set(member.medals.map((m) => m.id));
 const awardable = medalCatalog.filter((m) => !heldMedalIds.has(m.id));
```

**src/client/pages/MemberDetail.tsx**

```

const heldWarRecordIds = new Set(member.warRecords.map((w) => w.id));
const awardableWarRecords = warRecordCatalog.filter((w) => !heldWarRecordIds.has(w.id));

// Manual roles are the only ones managed here; rank-derived roles are set on
// the rank in the admin panel, so they aren't listed or revocable on a member.
const manualRoles = member.roles.filter((r) => r.source !== 'rank');
const showAdminBar =
 can('roster.promote') ||
 can('roster.edit') ||
 can('roster.remove') ||
 can('roles.assign') ||
 can('discord.sync');

return (
 <section className="panel member-detail">
 <button className="back" onClick={() => navigate('/roster')}>
 <- Roster
 </button>

 <Alerts error={error} warning={warning} notice={notice} />

 {pending && (
 <div className="reason-prompt">
 {pending.label} ? reason required:
 <input
 autoFocus
 value={reasonText}
 maxLength={500}
 placeholder="Why is this happening?"
 disabled={busy}
 onChange={(e) => setReasonText(e.target.value)}
 onKeyDown={(e) => {
 if (e.key === 'Enter') confirmReason();
 if (e.key === 'Escape') setPending(null);
 }}
 />
 <button
 className="danger mini"
 disabled={busy || reasonText.trim().length < 3}
 onClick={confirmReason}
 >
 Confirm
 </button>
 <button className="mini" disabled={busy} onClick={() => setPending(null)}>
 Cancel
 </button>
 </div>
)}

 <header className="member-head">
 <AvatarBlock
 member={member}
 canEdit={canEditProfile}
 hasOverride={!member.profileImageUrl}

```

**src/client/pages/MemberDetail.tsx**

```

 busy={busy}
 onSet={setProfileImage}
 />
<div>
 <h2>{displayName}</h2>
 {showsDiscordHandle && <div className="muted small">Discord: {discordHandle}</div>}
 <div className="member-sub muted">
 {member.rank ? member.rank.name : 'Unranked'}
 {member.status}
 joined {new Date(member.joinedAt * 1000).toLocaleDateString()}
 </div>
</div>
</header>

{showAdminBar && (
 <div className="member-admin-bar">
 {can('roster.promote') && (
 <div className="mab-group">

 {member.rank?.imageUrl ? (

) : (
 ?
)}

 {member.rank ? member.rank.name : 'Unranked'}
 {outranksTarget ? (

 <button
 disabled={busy || !canPromoteTo(nextUp)}
 title={nextUp ? `Promote to ${nextUp.name}` : 'Already at the top rank'}
 onClick={() => nextUp && void setRank(nextUp.id, `Promoted to
${nextUp.name}`)}
 >
 ? Promote
 </button>
 <button
 disabled={busy || !nextDown}
 title={nextDown ? `Demote to ${nextDown.name}` : 'Already at the lowest rank'}
 onClick={() =>
 nextDown &&
 promptReason(`Demote to ${nextDown.name}`, (reason) =>
 setRank(nextDown.id, `Demoted to ${nextDown.name}`, reason),
)
 }
 >
 ? Demote
 </button>

) : (
 Outranks you
)}
 </div>
)}
 </div>
)}

```

## src/client/pages/MemberDetail.tsx

```

 {(can('roster.edit') || can('roster.remove')) && (
 <div className="mab-group">
 Status
 <select
 value={member.status}
 disabled={busy || !outranksTarget}
 onChange={(e) => {
 const next = e.target.value;
 // Ban/retire are negative ? demand a reason. Softer states
 // (active/inactive/loa) apply straight away.
 if (next === 'banned' || next === 'retired') {
 promptReason(
 next === 'banned' ? 'Ban this member' : 'Retire this member',
 (reason) => setStatus(next, reason),
);
 } else {
 void setStatus(next);
 }
 }}
 >
 {STATUSES.map((st) => (
 <option key={st} value={st}>
 {st}
 </option>
))}
 </select>
 {can('roster.remove') && member.status !== 'retired' && (
 <button
 className="danger mini"
 disabled={busy || !outranksTarget}
 onClick={() =>
 promptReason('Retire (remove) this member', (reason) =>
 setStatus('retired', reason),
)
 >
 Remove
 </button>
)}
 </div>
)}

 {can('roles.assign') && (
 <div className="mab-group mab-roles">
 Roles
 {manualRoles.map((role) => (

 <span className="dot" style={{ background: role.color ?? 'var(--color-muted)' }}
 {role.name}
 <button
 className="mini danger"

```

**src/client/pages/MemberDetail.tsx**

```

 disabled={busy}
 title="Remove this role"
 onClick={() =>
 promptReason(`Remove role "${role.name}"`, (reason) =>
 revokeRole(role.id, reason),
)
 }
 >
 ?
 </button>

)}}
 {grantable.length > 0 && (
 <select
 disabled={busy}
 defaultValue=""
 onChange={(e) => {
 if (e.target.value) {
 void grantRole(Number(e.target.value));
 e.target.value = '';
 }
 }}
 >
 <option value="">Grant a role...</option>
 {grantable.map((r) => (
 <option key={r.id} value={r.id}>
 {r.name}
 </option>
))}
 </select>
)}
</div>
)}

{can('discord.sync') && (
 <div className="mab-group">
 <button
 disabled={busy}
 onClick={() => void resyncDiscord()}
 title="Force this member's Discord roles to match the website now"
 >
 Re-sync Discord
 </button>
 </div>
)}
</div>
)}

<div className="member-main">
 <section className="block">
 <h3>Profile</h3>
 {canEditProfile ? (
 <ProfileEditor
 displayName={member.displayName ?? ''}

```

**src/client/pages/MemberDetail.tsx**

```

 bio={member.bio ?? ''}
 busy={busy}
 isSelf={isSelf}
 onSave={({ displayName, bio }) =>
 run(async () => {
 const res = await api.patch<{
 discordSync?: { synced: boolean; warning?: string };
 }>(`/members/${member.id}/profile`, { displayName, bio });
 if (res.discordSync?.warning) return { warning: res.discordSync.warning };
 if (res.discordSync?.synced)
 return 'Saved ? the Discord nickname was updated to match.';
 return 'Profile saved.';
 })
 }
 />
) : member.bio ? (
 <p className="bio">{member.bio}</p>
) : (
 <p className="muted">No bio yet.</p>
)}
</section>

<section className="block">
 <h3>Medals</h3>
 {member.medals.length === 0 ? (
 <p className="muted">No medals yet.</p>
) : (
 <ul className="member-medals">
 {member.medals.map((m) => (
 <li key={m.awardId} title={m.citation ?? ''}>
 {m.imageUrl ? (

) : (
 ?
)}

 {m.name}
 {m.citation && {m.citation}}

 {m.awardedBy === null && auto}
 {can('medals.award') && (
 <button
 className="mini danger"
 disabled={busy}
 title="Revoke this medal"
 onClick={() =>
 promptReason(`Revoke medal "${m.name}"`, (reason) =>
 revokeMedal(m.awardId, reason),
)
 }
 >
 ?
 </button>
)}

)}

)}
</section>

```

## src/client/pages/MemberDetail.tsx

```


)}}

)}
 {can('medals.award') &&
 (awardable.length > 0 ? (
 <AwardMedalRow medals={awardable} busy={busy} onAward={awardMedal} />
) : (
 medalCatalog.length > 0 && (
 <p className="muted small">This member holds every medal in the catalog.</p>
)
)
)}}
</section>

<section className="block">
 <h3>War Records</h3>
 {member.warRecords.length === 0 ? (
 <p className="muted">No war records yet.</p>
) : (
 <ul className="member-medals">
 {member.warRecords.map((w) => (
 <li key={w.awardId} title={w.citation ?? ''}>
 {w.imageUrl ? (

) : (
 ?
)}

 {w.name}
 {w.gameName && {w.gameName}}
 {w.citation && {w.citation}}

 {can('warrecords.award') && (
 <button
 className="mini danger"
 disabled={busy}
 title="Revoke this war record"
 onClick={() =>
 promptReason(`Revoke war record "${w.name}"`, (reason) =>
 revokeWarRecord(w.awardId, reason),
)
 }
 >
 ?
 </button>
)}

)}}

)}
{can('warrecords.award') &&
 (awardableWarRecords.length > 0 ? (
 <AwardWarRecordRow records={awardableWarRecords} busy={busy}
onAward={awardWarRecord} />

```

**src/client/pages/MemberDetail.tsx**

```

) : (
 warRecordCatalog.length > 0 && (
 <p className="muted small">This member holds every war record in the
catalog.</p>
)
)}}
 </section>

 {modules.starcitizen && (sc.hangarEnabled || sc.verifyEnabled) && (isSelf ||
can('hangar.view')) && (
 <section className="block hangar-section">
 <h3>Star Citizen</h3>
 {sc.verifyEnabled && <ScVerify userId={member.id} isSelf={isSelf} />}
 {sc.hangarEnabled && isSelf && (
 <HangarImport userId={member.id} onImported={() => setHangarKey((k) => k + 1)} />
)}
 {sc.hangarEnabled && <HangarView userId={member.id} refreshKey={hangarKey} />}
 <p className="sc-disclaimer muted small">
 Star Citizen?, Squadron 42?, Roberts Space Industries?, and Cloud Imperium? are
 trademarks of Cloud Imperium Rights LLC. mustr is an unofficial community tool and
is
 not affiliated with, endorsed, sponsored, or approved by Cloud Imperium Games or
 Roberts Space Industries.
 </p>
 </section>
)}
</div>
</section>
);
}

/**
 * The member's avatar, with self-service upload when the viewer may edit this
 * profile. Uploads land in R2 under avatars/ and the returned URL is saved as
 * the profile-image override; "Use Discord" clears it back to the Discord
 * avatar. Defaults to the Discord avatar until an override is set.
 */
function AvatarBlock({
 member,
 canEdit,
 hasOverride,
 busy,
 onSet,
}): {
 member: { discordId: string; avatar: string | null; imageUrl: string | null };
 canEdit: boolean;
 hasOverride: boolean;
 busy: boolean;
 onSet: (url: string | null) => void;
} {
 const [uploading, setUploading] = useState(false);
 const [uploadError, setUploadError] = useState<string | null>(null);

 async function pickFile(e: ChangeEvent<HTMLInputElement>) {

```



**src/client/pages/MemberDetail.tsx**

```

 const file = e.target.files?.[0];
 e.target.value = ''; // allow re-selecting the same file
 if (!file) return;
 setUploadError(null);
 setUploading(true);
 try {
 const { url } = await api.upload<{ url: string }>('/media/avatars', file);
 onSet(url);
 } catch (err) {
 setUploadError(err instanceof Error ? err.message : 'Upload failed.');
```

```

 } finally {
 setUploading(false);
 }
 }
}

return (
 <div className="member-avatar">

 {canEdit && (
 <div className="avatar-controls">
 <label className="upload-btn mini">
 {uploading ? 'Uploading...' : hasOverride ? 'Change' : 'Upload'}
 <input
 type="file"
 accept="image/png,image/jpeg,image/gif,image/webp"
 onChange={(e) => void pickFile(e)}
 disabled={busy || uploading}
 hidden
 />
 </label>
 {hasOverride && (
 <button
 className="mini"
 disabled={busy || uploading}
 onClick={() => onSet(null)}
 title="Revert to the Discord avatar"
 >
 Use Discord
 </button>
)}
 {uploadError && {uploadError}}
 </div>
)}
 </div>
);
}

/**
 * Pick a medal and (optionally) write a citation, then award it. Kept as its
 * own component so the select + citation input have local state that resets
 * after each award.
 */
function AwardMedalRow({
 medals,
```

**src/client/pages/MemberDetail.tsx**

```

 busy,
 onAward,
 }: {
 medals: MedalDef[];
 busy: boolean;
 onAward: (medalId: number, citation: string) => void;
 }) {
 const [medalId, setMedalId] = useState<number | ''>('');
 const [citation, setCitation] = useState('');

 return (
 <div className="award-row">
 <select
 value={medalId}
 disabled={busy}
 onChange={(e) => setMedalId(e.target.value ? Number(e.target.value) : '')}
 >
 <option value="">Award a medal...</option>
 {medals.map((m) => (
 <option key={m.id} value={m.id}>
 {m.name}
 {m.autoGrantMonths !== null ? ' (tenure)' : ''}
 </option>
))}
 </select>
 <input
 value={citation}
 placeholder="Citation (optional)"
 maxLength={300}
 disabled={busy || medalId === ''}
 onChange={(e) => setCitation(e.target.value)}
 />
 <button
 disabled={busy || medalId === ''}
 onClick={() => {
 if (medalId !== '') {
 onAward(medalId, citation);
 setMedalId('');
 setCitation('');
 }
 }}
 >
 Award
 </button>
 </div>
);
 }

 /** Pick a war record and (optionally) a citation, then award it. */
 function AwardWarRecordRow({
 records,
 busy,
 onAward,
 }: {

```

**src/client/pages/MemberDetail.tsx**

```

 records: WarRecordDef[];
 busy: boolean;
 onAward: (warRecordId: number, citation: string) => void;
 }) {
 const [recordId, setRecordId] = useState<number | ''>('');
 const [citation, setCitation] = useState('');

 return (
 <div className="award-row">
 <select
 value={recordId}
 disabled={busy}
 onChange={(e) => setRecordId(e.target.value ? Number(e.target.value) : '')}
 >
 <option value="">Award a war record...</option>
 {records.map((r) => (
 <option key={r.id} value={r.id}>
 {r.name}
 {r.gameName ? ` (${r.gameName})` : ''}
 </option>
))}
 </select>
 <input
 value={citation}
 placeholder="Citation (optional)"
 maxLength={300}
 disabled={busy || recordId === ''}
 onChange={(e) => setCitation(e.target.value)}
 />
 <button
 disabled={busy || recordId === ''}
 onClick={() => {
 if (recordId !== '') {
 onAward(recordId, citation);
 setRecordId('');
 setCitation('');
 }
 }}
 >
 Award
 </button>
 </div>
);
 }

function ProfileEditor({
 displayName,
 bio,
 busy,
 isSelf,
 onSave,
}): {
 displayName: string;
 bio: string;

```

**src/client/pages/MemberDetail.tsx**

```

 busy: boolean;
 isSelf: boolean;
 onSave: (v: { displayName: string; bio: string }) => void;
 }) {
 const [name, setName] = useState(displayName);
 const [text, setText] = useState(bio);
 const dirty = name !== displayName || text !== bio;
 const nameValid = name.trim().length >= 2;

 return (
 <div className="bio-editor">
 <label className="field">
 Display name *
 <input
 value={name}
 maxLength={32}
 placeholder={isSelf ? 'Your in-game name' : 'Set a display name'}
 onChange={(e) => setName(e.target.value)}
 disabled={busy}
 aria-invalid={!nameValid}
 />
 </label>
 <p className="muted small">
 Required. Shown across the site instead of the Discord name, and applied as the member's
 Discord nickname.
 </p>

 <label className="field">
 Bio
 <textarea
 value={text}
 maxLength={2000}
 rows={4}
 placeholder={isSelf ? 'Tell the clan about yourself...' : 'Edit this member's bio...'}
 onChange={(e) => setText(e.target.value)}
 disabled={busy}
 />
 </label>

 <div className="bio-actions">
 <button
 className="primary"
 disabled={busy || !dirty || !nameValid}
 title={!nameValid ? 'A display name is required' : undefined}
 onClick={() => onSave({ displayName: name, bio: text })}
 >
 Save profile
 </button>
 {dirty && (
 <button
 disabled={busy}
 onClick={() => {
 setName(displayName);
 setText(bio);

```

**src/client/pages/MemberDetail.tsx**

```
 }}
 >
 Cancel
 </button>
)}
 </div>
</div>
);
}
```

**src/client/pages/ModulesAdmin.tsx**

```

/**
 * Settings -> Modules ? turn optional per-install features on or off.
 *
 * Star Citizen is the first: enabling it surfaces the hangar import + display on
 * member profiles. Everything defaults off so a fresh install stays lean.
 */

import { useEffect, useState } from 'react';
import { api } from '../lib/api';
import { useAction, Alerts } from '../lib/action';
import { clearModulesCache, clearScConfigCache, type ModuleFlags, type ScConfig } from
'../lib/modules';

interface ModuleDef {
 key: keyof ModuleFlags;
 label: string;
 description: string;
}

const MODULES: ModuleDef[] = [
 {
 key: 'starcitizen',
 label: 'Star Citizen',
 description:
 'Lets members import their RSI hangar (via a bookmarklet) and shows it, categorised and
searchable, at the bottom of their profile.',
 },
];

export default function ModulesAdmin() {
 const [flags, setFlags] = useState<ModuleFlags | null>(null);
 const [sc, setSc] = useState<ScConfig | null>(null);
 const [orgSidDraft, setOrgSidDraft] = useState('');
 const { run, busy, error, notice, warning } = useAction();

 useEffect(() => {
 api
 .get<{ modules: ModuleFlags }>('/settings/modules')
 .then(({ modules }) => setFlags(modules))
 .catch(() => setFlags({ starcitizen: false }));
 api
 .get<{ sc: ScConfig }>('/settings/sc')
 .then(({ sc }) => {
 setSc(sc);
 setOrgSidDraft(sc.orgSid);
 })
 .catch(() => {});
 }, []);

 // Every save sends the FULL SC config (the server replaces the blob), so
 // toggling a kill switch preserves the org SID and vice-versa.
 const saveSc = (next: ScConfig, okMsg = 'Saved.') =>
 run(async () => {
 const { sc: saved } = await api.put<{ sc: ScConfig }>('/settings/sc', { sc: next });

```

**src/client/pages/ModulesAdmin.tsx**

```

 setSc(saved);
 setOrgSidDraft(saved.orgSid);
 clearScConfigCache(); // profiles pick up a kill switch without a reload
 return okMsg;
 });

 const toggle = (key: keyof ModuleFlags, value: boolean) =>
 run(async () => {
 const next = { ...(flags ?? { starcitizen: false }), [key]: value };
 const { modules } = await api.put<{ modules: ModuleFlags }>('/settings/modules', { modules:
next });
 setFlags(modules);
 clearModulesCache(); // so profiles pick up the change without a reload
 return value ? 'Module enabled.' : 'Module disabled.';
 });

 if (!flags) return <div className="loading">Loading...</div>;

 return (
 <section className="panel modules-admin">
 <header className="panel-head">
 <h2>Modules</h2>
 <p className="muted">Optional features for this site. Turn on only what your group
needs.</p>
 </header>

 <Alerts error={error} warning={warning} notice={notice} />

 <ul className="modules-list">
 {MODULES.map((m) => (
 <li key={m.key} className="module-row">
 <div className="module-info">
 {m.label}
 {m.description}
 </div>
 <label className="module-toggle">
 <input
 type="checkbox"
 checked={!flags[m.key]}
 disabled={busy}
 onChange={(e) => void toggle(m.key, e.target.checked)}
 />
 {flags[m.key] ? 'On' : 'Off'}
 </label>

))}

 {flags.starcitizen && sc && (
 <div className="module-config">
 <h3>Star Citizen settings</h3>
 <label>
 Org SID
 <div className="module-config-row">

```

**src/client/pages/ModulesAdmin.tsx**

```

 <input
 type="text"
 value={orgSidDraft}
 placeholder="e.g. F919"
 maxLength={20}
 disabled={busy}
 onChange={(e) => setOrgSidDraft(e.target.value)}
 />
 <button
 type="button"
 className="primary small"
 disabled={busy || orgSidDraft.trim() === sc.orgSid}
 onClick={() => void saveSc({ ...sc, orgSid: orgSidDraft.trim() })}
 >
 Save
 </button>
 </div>

 Your org's RSI Spectrum Identification (the tag in your org URL{' '}
 <code>/orgs/<SID></code>). Used to confirm a member's verified RSI account
 actually lists your org.

</label>

<div className="module-killswitch">

 Feature kill switches ? turn either off instantly (e.g. if RSI / Cloud Imperium
 requests it). Turning the whole module off above disables both.

 <div className="module-row">
 <div className="module-info">
 Hangar import & display

 Members export their own hangar (client-side bookmarklet) and it shows on
 profiles.
 No server-side access to RSI.

 </div>
 <label className="module-toggle">
 <input
 type="checkbox"
 checked={sc.hangarEnabled}
 disabled={busy}
 onChange={(e) => void saveSc({ ...sc, hangarEnabled: e.target.checked })}
 />
 {sc.hangarEnabled ? 'On' : 'Off'}
 </label>
 </div>
 <div className="module-row">
 <div className="module-info">
 RSI account verification

 Reads a member's public RSI profile to confirm account ownership + org
 membership.

```



**src/client/pages/ModulesAdmin.tsx**

```
 This is the only feature that fetches RSI server-side.

 </div>
 <label className="module-toggle">
 <input
 type="checkbox"
 checked={sc.verifyEnabled}
 disabled={busy}
 onChange={(e) => void saveSc({ ...sc, verifyEnabled: e.target.checked })}
 />
 {sc.verifyEnabled ? 'On' : 'Off'}
 </label>
 </div>
</div>
</div>
)}
</section>
);
}
```

**src/client/pages/NavAdmin.tsx**

```

/**
 * Navigation builder ? arrange the top menu.
 *
 * The menu is an ordered list of entries; each is a link (to a built-in page, a
 * custom page, or a URL) or a category (a dropdown holding links). Reordering is
 * up/down plus "move into a category" / "move out", rather than nested drag-drop
 * ? deterministic and finger-friendly, and it sidesteps the drag-vs-select trap.
 * Every entry can be gated to a role. Saved to /api/nav; the top bar re-reads it
 * live via the `ct-nav-changed` event.
 */

import { useEffect, useMemo, useState } from 'react';
import { api } from '../lib/api';
import { useAction, Alerts } from '../lib/action';
import {
 BUILTIN_TARGETS,
 newNavId,
 navItemLabel,
 type NavItem,
 type NavConfig,
} from '../../shared/nav';

interface PageOpt {
 slug: string;
 title: string | null;
}

interface RoleOpt {
 id: number;
 name: string;
 color: string | null;
}

export default function NavAdmin() {
 const [loading, setLoading] = useState(true);
 const [items, setItems] = useState<NavItem[]>([]);
 const [saved, setSaved] = useState<string>('[]');
 const [pages, setPages] = useState<PageOpt[]>([]);
 const [roles, setRoles] = useState<RoleOpt[]>([]);

 const { run, busy, error, notice, warning } = useAction();

 useEffect(() => {
 api
 .get<{ nav: NavConfig; pages: PageOpt[]; roles: RoleOpt[] }>('/nav/meta')
 .then((d) => {
 setItems(d.nav.items);
 setSaved(JSON.stringify(d.nav.items));
 setPages(d.pages);
 setRoles(d.roles);
 })
 .finally(() => setLoading(false));
 }, []);

 const dirty = useMemo(() => JSON.stringify(items) !== saved, [items, saved]);

```

**src/client/pages/NavAdmin.tsx**

```

const groups = useMemo(() => items.filter((i) => i.type === 'group'), [items]);

/** Clone, mutate, set ? every edit goes through here. */
const update = (fn: (draft: NavItem[]) => void) =>
 setItems((prev) => {
 const next = structuredClone(prev);
 fn(next);
 return next;
 });

/* --- top-level ops --- */
const moveTop = (idx: number, dir: -1 | 1) =>
 update((it) => {
 const j = idx + dir;
 if (j < 0 || j >= it.length) return;
 [it[idx], it[j]] = [it[j]!, it[idx]!];
 });
const delTop = (idx: number) => update((it) => it.splice(idx, 1));
const setTop = (idx: number, patch: Partial<NavItem>) =>
 update((it) => {
 it[idx] = { ...it[idx]!, ...patch };
 });
const moveInto = (idx: number, groupId: string) =>
 update((it) => {
 const g = it.find((x) => x.id === groupId && x.type === 'group');
 if (!g) return;
 const [item] = it.splice(idx, 1);
 if (item) (g.children ??= []).push(item);
 });

/* --- child ops --- */
const moveChild = (gIdx: number, cIdx: number, dir: -1 | 1) =>
 update((it) => {
 const kids = it[gIdx]!.children!;
 const j = cIdx + dir;
 if (j < 0 || j >= kids.length) return;
 [kids[cIdx], kids[j]] = [kids[j]!, kids[cIdx]!];
 });
const delChild = (gIdx: number, cIdx: number) => update((it) => it[gIdx]!.children!.splice(cIdx, 1));
const setChild = (gIdx: number, cIdx: number, patch: Partial<NavItem>) =>
 update((it) => {
 const kids = it[gIdx]!.children!;
 kids[cIdx] = { ...kids[cIdx]!, ...patch };
 });
const moveOut = (gIdx: number, cIdx: number) =>
 update((it) => {
 const [child] = it[gIdx]!.children!.splice(cIdx, 1);
 if (child) it.splice(gIdx + 1, 0, child);
 });

/* --- adding --- */
const addLink = (kind: NavItem['kind'], target: string, label = '') =>
 update((it) => it.push({ id: newNavId(), type: 'link', label, kind, target }));

```

**src/client/pages/NavAdmin.tsx**

```

const addGroup = () =>
 update((it) => it.push({ id: newNavId(), type: 'group', label: 'New category', children: []
})));

const save = () =>
 run(async () => {
 const { nav } = await api.put<{ nav: NavConfig }>('/nav', {
 nav: { version: 1, items },
 });
 setItems(nav.items);
 setSaved(JSON.stringify(nav.items));
 window.dispatchEvent(new Event('ct-nav-changed'));
 return 'Menu saved. The top bar now reflects it.';
 });

const roleSelect = (value: number | undefined, onChange: (v: number | undefined) => void) => (
 <select
 className="nav-role"
 value={value != null ? String(value) : ''}
 onChange={(e) => onChange(e.target.value ? Number(e.target.value) : undefined)}
 title="Who can see this entry"
 >
 <option value="">Everyone</option>
 {roles.map((r) => (
 <option key={r.id} value={r.id}>
 {r.name}
 </option>
))}
 </select>
);

if (loading) return <div className="loading">Loading...</div>;

return (
 <section className="panel nav-admin">
 <header className="panel-head nav-admin-head">
 <div>
 <h2>Navigation</h2>
 <p className="muted">
 Arrange the top menu: reorder entries, nest them under categories, and choose who sees
 each. Categories become dropdowns.
 </p>
 </div>
 <button className="primary" disabled={busy || !dirty} onClick={() => void save()}>
 {busy ? 'Saving...' : dirty ? 'Save menu' : 'Saved'}
 </button>
 </header>

 <Alerts error={error} warning={warning} notice={notice} />

 <ol className="nav-tree">
 {items.map((item, idx) =>
 item.type === 'group' ? (
 <li key={item.id} className="nav-tree-group">

```

**src/client/pages/NavAdmin.tsx**

```

 <div className="nav-row nav-row-group">

 ? Category

 <input
 className="nav-row-label"
 value={item.label}
 placeholder="Category name"
 onChange={(e) => setTop(idx, { label: e.target.value })}
 />
 {roleSelect(item.visibleToRole, (v) => setTop(idx, { visibleToRole: v })})
 <div className="nav-row-tools">
 <button className="mini" title="Move up" disabled={idx === 0} onClick={() =>
moveTop(idx, -1)}></button>
 <button className="mini" title="Move down" disabled={idx === items.length - 1}
onClick={() => moveTop(idx, 1)}></button>
 <button className="mini danger" title="Delete category (its links move nowhere ?
they're removed)" onClick={() => delTop(idx)}></button>
 </div>
 </div>

 <ol className="nav-tree-children">
 {(item.children ?? []).length === 0 && (
 <li className="nav-child-empty muted small">
 Empty ? use "Move into" on a link below, or add one from the toolbar.

)}
 {(item.children ?? []).map((child, cIdx) => (
 <li key={child.id} className="nav-row nav-row-child">
 {describeKind(child)}
 <input
 className="nav-row-label"
 value={child.label}
 placeholder={navItemLabel(child)}
 onChange={(e) => setChild(idx, cIdx, { label: e.target.value })}
 />
 {roleSelect(child.visibleToRole, (v) => setChild(idx, cIdx, { visibleToRole: v
}}))}

 <div className="nav-row-tools">
 <button className="mini" title="Move up" disabled={cIdx === 0} onClick={()
=> moveChild(idx, cIdx, -1)}></button>
 <button className="mini" title="Move down" disabled={cIdx ===
(item.children?.length ?? 0) - 1} onClick={() => moveChild(idx, cIdx, 1)}></button>
 <button className="mini" title="Move out to top level" onClick={() =>
moveOut(idx, cIdx)}></button>
 <button className="mini danger" title="Remove" onClick={() => delChild(idx,
cIdx)}></button>
 </div>

)}

) : (
 <li key={item.id} className="nav-row nav-row-link">

```

**src/client/pages/NavAdmin.tsx**

```

 {describeKind(item)}
 <input
 className="nav-row-label"
 value={item.label}
 placeholder={navItemLabel(item)}
 onChange={(e) => setTop(idx, { label: e.target.value })}
 />
 {roleSelect(item.visibleToRole, (v) => setTop(idx, { visibleToRole: v })})}
 <div className="nav-row-tools">
 <button className="mini" title="Move up" disabled={idx === 0} onClick={() =>
moveTop(idx, -1)}></button>
 <button className="mini" title="Move down" disabled={idx === items.length - 1}
onClick={() => moveTop(idx, 1)}></button>
 {groups.length > 0 && (
 <select
 className="nav-into"
 value=""
 title="Move into a category"
 onChange={(e) => e.target.value && moveInto(idx, e.target.value)}
 >
 <option value="">Into ?</option>
 {groups.map((g) => (
 <option key={g.id} value={g.id}>
 {g.label || 'Category'}
 </option>
))}
 </select>
)}
 <button className="mini danger" title="Remove" onClick={() =>
delTop(idx)}></button>
 </div>

),
)}
{items.length === 0 && <li className="muted small">The menu is empty. Add entries
below.}

<AddToolbar pages={pages} onAddLink={addLink} onAddGroup={addGroup} />

<p className="muted small">
 Built-in destinations still respect their own access rules ? e.g. an
Events{' '}
 or Admin entry only appears for members who can already reach them,
whatever
 role you set here.
</p>
</section>
);
}

function describeKind(item: NavItem): string {
 if (item.kind === 'builtin') return `Built-in ? ${BUILTIN_TARGETS[item.target ?? '']?.label ??
item.target}`;

```

**src/client/pages/NavAdmin.tsx**

```

 if (item.kind === 'page') return `Page ? /p/${item.target}`;
 if (item.kind === 'url') return `Link ? ${item.target}`;
 return 'Link';
 }

 /** The "add an entry" strip: a built-in, a page, a custom URL, or a category. */
 function AddToolbar({
 pages,
 onAddLink,
 onAddGroup,
 }: {
 pages: PageOpt[];
 onAddLink: (kind: NavItem['kind'], target: string, label?: string) => void;
 onAddGroup: () => void;
 }) {
 const [urlLabel, setUrlLabel] = useState('');
 const [url, setUrl] = useState('');
 const [urlErr, setUrlErr] = useState<string | null>(null);

 const addUrl = () => {
 const u = url.trim();
 const ok = (u.startsWith('/') && !u.startsWith('//')) || /^https?:\/\//i.test(u);
 if (!ok) {
 setUrlErr('Enter a path like /roster or a full https:// link.');
```

return;

```

 }
 onAddLink('url', u, urlLabel.trim());
 setUrl('');
 setUrlLabel('');
 setUrlErr(null);
 };

 return (
 <div className="nav-add">
 <div className="nav-add-row">
 <label>
 Built-in page
 <select
 value=""
 onChange={(e) => {
 if (e.target.value) onAddLink('builtin', e.target.value);
 e.target.value = '';
 }}
 >
 <option value="">Add...</option>
 {Object.values(BUILTIN_TARGETS).map((b) => (
 <option key={b.key} value={b.key}>
 {b.label}
 </option>
))}
 </select>
 </label>

 <label>
```

**src/client/pages/NavAdmin.tsx**

```

 Custom page
 <select
 value=""
 disabled={pages.length === 0}
 onChange={(e) => {
 if (e.target.value) {
 const p = pages.find((x) => x.slug === e.target.value);
 onAddLink('page', e.target.value, (p?.title ?? '').slice(0, 40));
 }
 e.target.value = '';
 }}
 >
 <option value="">{pages.length ? 'Add...' : 'No pages yet'}</option>
 {pages.map((p) => (
 <option key={p.slug} value={p.slug}>
 {p.title ?? p.slug}
 </option>
))}
 </select>
 </label>

 <button type="button" className="ghost" onClick={onAddGroup}>
 + Category
 </button>
</div>

<div className="nav-add-row nav-add-url">
 <label>
 Custom link
 <input value={urlLabel} placeholder="Label" onChange={(e) =>
setUrlLabel(e.target.value)} />
 </label>
 <input
 className="nav-add-url-input"
 value={url}
 placeholder="/roster or https://discord.gg/..."
 onChange={(e) => setUrl(e.target.value)}
 />
 <button type="button" className="ghost" onClick={addUrl}>
 + Link
 </button>
 {urlErr && {urlErr}}
</div>
</div>
);
}

```



**src/client/pages/News.tsx**

```

/**
 * The news feed ? the site's front page. Published posts only, pinned first.
 */

import { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import { api } from '../lib/api';
import { useSession } from '../lib/session';
import { excerptFromHtml } from '../lib/richtext';

interface Post {
 id: number;
 slug: string;
 title: string;
 excerpt: string | null;
 body: string;
 pinned: boolean;
 publishedAt: number | null;
 author: string | null;
}

export default function News() {
 const { can } = useSession();
 const [posts, setPosts] = useState<Post[]>([]);
 const [loading, setLoading] = useState(true);

 useEffect(() => {
 api
 .get<{ posts: Post[] }>('/news')
 .then(({ posts }) => setPosts(posts))
 .finally(() => setLoading(false));
 }, []);

 if (loading) return <div className="loading">Loading news...</div>;

 return (
 <section className="panel">
 <header className="panel-head news-head">
 <div>
 <h2>News</h2>
 <p className="muted">{posts.length === 0 ? 'No posts yet.' : `${posts.length}
post${posts.length === 1 ? '' : 's'}`}</p>
 </div>
 {can('news.create') && (
 <Link className="btn-link" to="/admin/news">
 Manage posts
 </Link>
)}
 </header>

 {posts.length === 0 ? (
 <p className="muted">Nothing has been posted yet.</p>
) : (
 <ul className="news-feed">

```

**src/client/pages/News.tsx**

```
{posts.map((p) => (
 <li key={p.id} className="news-item">
 <Link to={` /news/${p.slug}`} className="news-item-link">
 <div className="news-item-head">
 {p.pinned && ? Pinned}
 <h3>{p.title}</h3>
 </div>
 <p className="news-excerpt">{p.excerpt || excerptFromHtml(p.body)}</p>
 <div className="muted small news-meta">
 {p.author ?? 'Unknown'}
 {p.publishedAt && <> ? {new Date(p.publishedAt * 1000).toLocaleDateString()}</>}
 </div>
 </Link>

))}

)
</section>
);
}
```

**src/client/pages/NewsAdmin.tsx**

```

/**
 * News administration ? the writer's side of the feed.
 *
 * Left: every post with its status. Right: the WYSIWYG editor for the selected
 * one. Writing needs news.create; the Publish/Unpublish/Archive controls need
 * news.publish; Delete needs news.delete ? so the buttons a given admin sees
 * match what they're allowed to do.
 */

import { lazy, Suspense, useCallback, useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import { api } from '../lib/api';
import { useAction, Alerts } from '../lib/action';
import { useSession } from '../lib/session';

// TipTap/ProseMirror is heavy; only pull it in when a post is actually open.
const RichTextEditor = lazy(() => import('../components/RichTextEditor'));

interface PostSummary {
 id: number;
 slug: string;
 title: string;
 status: 'draft' | 'published' | 'archived';
 pinned: boolean;
 updatedAt: number;
 author: string | null;
}

interface FullPost extends PostSummary {
 excerpt: string | null;
 body: string;
 publishedAt: number | null;
}

export default function NewsAdmin() {
 const [posts, setPosts] = useState<PostSummary[]>([]);
 const [selectedId, setSelectedId] = useState<number | null>(null);
 const [selected, setSelected] = useState<FullPost | null>(null);
 const [newTitle, setNewTitle] = useState('');
 const [loading, setLoading] = useState(true);

 const loadList = useCallback(async () => {
 const { posts } = await api.get<{ posts: PostSummary[] }>('/news/manage');
 setPosts(posts);
 }, []);

 const loadSelected = useCallback(async (id: number, slug: string) => {
 const { post } = await api.get<{ post: FullPost }>(`/news/${slug}`);
 setSelected(post);
 setSelectedId(id);
 }, []);

 const refresh = useCallback(async () => {
 await loadList();
 }, []);

```

**src/client/pages/NewsAdmin.tsx**

```

 if (selected) {
 const { post } = await api.get<{ post: FullPost }>(`/news/${selected.slug}`);
 setSelected(post);
 }
 }, [loadList, selected]);

 const { run, busy, error, notice, warning } = useAction(refresh);

 useEffect(() => {
 loadList().finally(() => setLoading(false));
 }, [loadList]);

 const create = () =>
 run(async () => {
 if (!newTitle.trim()) return;
 const { post } = await api.post<{ post: FullPost }>('/news', { title: newTitle.trim() });
 setNewTitle('');
 await loadSelected(post.id, post.slug);
 return `Created draft "${post.title}".`;
 });

 if (loading) return <div className="loading">Loading news...</div>;

 return (
 <section className="panel">
 <header className="panel-head">
 <h2>News</h2>
 <p className="muted">
 {posts.length} post{posts.length === 1 ? '' : 's'}. Drafts are only visible to writers.
 </p>
 </header>

 <Alerts error={error} warning={warning} notice={notice} />

 <div className="add-row">
 <input
 value={newTitle}
 placeholder="New post title"
 onChange={(e) => setNewTitle(e.target.value)}
 onKeyDown={(e) => e.key === 'Enter' && void create()}
 disabled={busy}
 />
 <button onClick={() => void create()} disabled={busy || !newTitle.trim()}>
 New draft
 </button>
 </div>

 <div className="roles-layout">
 <ul className="news-admin-list">
 {posts.map((p) => (
 <li key={p.id}>
 <button
 className={p.id === selectedId ? 'news-admin-item active' : 'news-admin-item'}
 onClick={() => void loadSelected(p.id, p.slug)}
 >

```

**src/client/pages/NewsAdmin.tsx**

```

 >

 {p.pinned && ?}
 {p.title}

 {p.status}
 </button>

)}
 {posts.length === 0 && <li className="muted small">No posts yet.}

 {selected ? (
 <PostEditor key={`_${selected.id}:${selected.updatedAt}`} post={selected} busy={busy}
run={run} onDelete={() => { setSelected(null); setSelectedId(null); }} />
) : (
 <div className="role-editor empty-editor">Select a post to edit, or start a new
draft.</div>
)}
</div>
</section>
);
}

function PostEditor({
 post,
 busy,
 run,
 onDelete,
}): {
 post: FullPost;
 busy: boolean;
 run: (fn: () => Promise<string | { warning: string } | void | null>) => void;
 onDelete: () => void;
}) {
 const { can } = useSession();
 const [title, setTitle] = useState(post.title);
 const [excerpt, setExcerpt] = useState(post.excerpt ?? '');
 const [pinned, setPinned] = useState(post.pinned);
 const [body, setBody] = useState(post.body);
 const [confirmDelete, setConfirmDelete] = useState(false);

 const dirty =
 title !== post.title ||
 excerpt !== (post.excerpt ?? '') ||
 pinned !== post.pinned ||
 body !== post.body;

 const save = () =>
 run(async () => {
 if (!title.trim()) return { warning: 'A title is required.' };
 await api.patch(`/news/${post.id}`, { title: title.trim(), excerpt, body, pinned });
 return 'Saved.';
 });

```

**src/client/pages/NewsAdmin.tsx**

```

const setStatus = (status: 'draft' | 'published' | 'archived', label: string) =>
 run(async () => {
 await api.post(`/news/${post.id}/status`, { status });
 return label;
 });

const remove = () =>
 run(async () => {
 await api.del(`/news/${post.id}`);
 onDelete();
 return 'Post deleted.';
 });

return (
 <div className="role-editor news-editor">
 <div className="news-editor-head">
 {post.status}
 <Link className="btn-link small" to={`/news/${post.slug}`} target="_blank" rel="noopener">
 View ?
 </Link>
 </div>

 <label>
 Title
 <input value={title} onChange={(e) => setTitle(e.target.value)} disabled={busy} />
 </label>

 <label>
 Excerpt (optional ? shown on the feed)
 <input
 value={excerpt}
 placeholder="A one-line summary"
 onChange={(e) => setExcerpt(e.target.value)}
 disabled={busy}
 />
 </label>

 <label className="check">
 <input type="checkbox" checked={pinned} onChange={(e) => setPinned(e.target.checked)}
disabled={busy} />
 Pin to the top of the feed
 </label>

 <label className="rte-label">Body</label>
 <Suspense fallback={<div className="loading">Loading editor...</div>}>
 <RichTextEditor value={body} onChange={setBody} disabled={busy} />
 </Suspense>

 <div className="news-editor-actions">
 <button className="primary" disabled={busy || !dirty} onClick={() => void save()}>
 Save
 </button>
 </div>
 </div>
)

```

**src/client/pages/NewsAdmin.tsx**

```

 {can('news.publish') && (
 <>
 {post.status !== 'published' ? (
 <button disabled={busy} onClick={() => void setStatus('published', 'Published.')}>
 Publish
 </button>
) : (
 <button disabled={busy} onClick={() => void setStatus('draft', 'Unpublished ? back
to draft.')}>
 Unpublish
 </button>
)}
 {post.status !== 'archived' && (
 <button disabled={busy} onClick={() => void setStatus('archived', 'Archived.')}>
 Archive
 </button>
)}
 </>
)}
 </div>

 {can('news.delete') && (
 <div className="editor-footer">
 {confirmDelete ? (

 Delete "{post.title}" permanently?
 <button className="danger" disabled={busy} onClick={() => void remove()}>
 Confirm delete
 </button>
 <button disabled={busy} onClick={() => setConfirmDelete(false)}>
 Cancel
 </button>

) : (
 <button className="danger" disabled={busy} onClick={() => setConfirmDelete(true)}>
 Delete post
 </button>
)}
 </div>
)}
</div>
);
}

```

**src/client/pages/NewsPost.tsx**

```

/**
 * A single news post. The body is HTML from the editor, re-sanitized here
 * before it's inserted ? stored markup is never trusted at render time.
 */

import { useEffect, useState } from 'react';
import { Link, useParams } from 'react-router-dom';
import { api } from '../lib/api';
import { ApiError } from '../lib/api';
import { sanitizeHtml } from '../lib/richtext';
import { useRecordRecent } from '../lib/recent';

interface Post {
 id: number;
 slug: string;
 title: string;
 body: string;
 status: string;
 pinned: boolean;
 publishedAt: number | null;
 updatedAt: number;
 author: string | null;
}

export default function NewsPost() {
 const { slug } = useParams();
 const [post, setPost] = useState<Post | null>(null);
 const [error, setError] = useState<string | null>(null);
 const [loading, setLoading] = useState(true);

 useEffect(() => {
 setLoading(true);
 api
 .get<{ post: Post }>(`/news/${slug}`)
 .then(({ post }) => setPost(post))
 .catch((e) => setError(e instanceof ApiError && e.status === 404 ? 'Post not found.' :
'Failed to load post.'))
 .finally(() => setLoading(false));
 }, [slug]);

 useRecordRecent(post ? { group: 'content', label: post.title, to: `/news/${post.slug}` } :
null);

 if (loading) return <div className="loading">Loading...</div>;
 if (error || !post) return <div className="empty">{error ?? 'Post not found.'}</div>;

 return (
 <section className="panel news-post">
 <Link className="back" to="/">
 <- News
 </Link>

 <header className="news-post-head">
 {post.status !== 'published' && {post.status}}

```



**src/client/pages/NewsPost.tsx**

```
 {post.pinned && ? Pinned}
 <h1>{post.title}</h1>
 <div className="muted small">
 {post.author ?? 'Unknown'}
 {post.publishedAt && <> ? {new Date(post.publishedAt * 1000).toLocaleDateString()}</>}
 </div>
 </header>

 <article
 className="news-body"
 dangerouslySetInnerHTML={{ __html: sanitizeHtml(post.body) }}
 />
</section>
);
}
```

**src/client/pages/OrgChartDesigner.tsx**

```

/**
 * The leadership-chart designer (admin, ranks.manage).
 *
 * Add members from a threshold-filtered picker, drag their boxes freely, and draw
 * report-to connectors by clicking a manager then a report in Connect mode. The
 * rank threshold controls who is eligible and who shows on the public tree. State
 * is the portable OrgChart JSON from shared/orgchart.ts, saved to /api/orgchart.
 */

import { useEffect, useMemo, useRef, useState, type CSSProperties, type PointerEvent as
RPointerEvent } from 'react';
import { api } from '../lib/api';
import { memberName } from '../../shared/names';
import { sanitizeOrgChart, type OrgChart } from '../../shared/orgchart';
import { BOX_W, BOX_H, chartSize, edgePath, type ChartMember } from '../components/OrgChartView';

interface RankOpt {
 id: number;
 name: string;
 sortOrder: number;
}

const MAX_COORD = 8000;
const clamp = (v: number) => Math.min(MAX_COORD, Math.max(0, Math.round(v)));

export default function OrgChartDesigner() {
 const [chart, setChart] = useState<OrgChart | null>(null);
 const [members, setMembers] = useState<ChartMember[]>([]);
 const [ranks, setRanks] = useState<RankOpt[]>([]);
 const [dirty, setDirty] = useState(false);
 const [saving, setSaving] = useState(false);
 const [message, setMessage] = useState<string | null>(null);

 const [connectMode, setConnectMode] = useState(false);
 const [connectFrom, setConnectFrom] = useState<number | null>(null);
 const canvasRef = useRef<HTMLDivElement>(null);

 useEffect(() => {
 api
 .get<{ chart: OrgChart; members: ChartMember[]; ranks: RankOpt[] }>('/orgchart/designer')
 .then((d) => {
 setChart(sanitizeOrgChart(d.chart));
 setMembers(d.members);
 setRanks(d.ranks);
 })
 .catch(() => setMessage('Could not load the chart.'));
 }, []);

 const byId = useMemo(() => new Map(members.map((m) => [m.id, m])), [members]);

 function edit(fn: (c: OrgChart) => OrgChart) {
 setChart((c) => (c ? fn(c) : c));
 setDirty(true);
 setMessage(null);
 }

```

**src/client/pages/OrgChartDesigner.tsx**

```

}

const meetsThreshold = (m: ChartMember | undefined, min: number) =>
 !!m && (min <= 0 || (m.rankSortOrder != null && m.rankSortOrder >= min));

/* --- drag --- */
function startDrag(e: RPointerEvent, memberId: number) {
 if (connectMode || !chart || !canvasRef.current) return;
 e.preventDefault();
 const rect = canvasRef.current.getBoundingClientRect();
 const node = chart.nodes.find((n) => n.memberId === memberId);
 if (!node) return;
 const offX = e.clientX - rect.left - node.x + canvasRef.current.scrollLeft;
 const offY = e.clientY - rect.top - node.y + canvasRef.current.scrollTop;
 const move = (ev: PointerEvent) => {
 const nx = clamp(ev.clientX - rect.left - offX + (canvasRef.current?.scrollLeft ?? 0));
 const ny = clamp(ev.clientY - rect.top - offY + (canvasRef.current?.scrollTop ?? 0));
 setChart((c) => (c ? { ...c, nodes: c.nodes.map((n) => (n.memberId === memberId ? { ...n, x:
nx, y: ny } : n)) } : c));
 };
 const up = () => {
 window.removeEventListener('pointermove', move);
 window.removeEventListener('pointerup', up);
 setDirty(true);
 };
 window.addEventListener('pointermove', move);
 window.addEventListener('pointerup', up);
}

/* --- connect --- */
function onBoxClick(memberId: number) {
 if (!connectMode) return;
 if (connectFrom == null) {
 setConnectFrom(memberId);
 return;
 }
 if (connectFrom === memberId) {
 setConnectFrom(null);
 return;
 }
 const from = connectFrom;
 edit((c) =>
 c.edges.some((e) => e.from === from && e.to === memberId)
 ? c
 : { ...c, edges: [...c.edges, { from, to: memberId }] },
);
 setConnectFrom(null);
}

/* --- add / remove --- */
function addMember(memberId: number) {
 edit((c) => {
 // Cascade new boxes so they don't stack exactly.
 const n = c.nodes.length;

```

**src/client/pages/OrgChartDesigner.tsx**

```

 return { ...c, nodes: [...c.nodes, { memberId, x: 40 + (n % 5) * 40, y: 40 + (n % 5) * 30 }]
 };
});
}
function removeNode(memberId: number) {
 edit((c) => ({
 ...c,
 nodes: c.nodes.filter((n) => n.memberId !== memberId),
 edges: c.edges.filter((e) => e.from !== memberId && e.to !== memberId),
 }));
}
function removeEdge(from: number, to: number) {
 edit((c) => ({ ...c, edges: c.edges.filter((e) => !(e.from === from && e.to === to)) }));
}

async function save() {
 if (!chart) return;
 setSaving(true);
 setMessage(null);
 try {
 await api.put('/orgchart', { chart });
 setDirty(false);
 setMessage('Saved. The leadership chart is live.');
```

} catch {  
 setMessage('Could not save. Please try again.');

} finally {  
 setSaving(false);  
}

}

if (!chart) return <div className="loading">Loading...</div>;

const placed = new Set(chart.nodes.map((n) => n.memberId));
const addable = members
 .filter((m) => !placed.has(m.id) && meetsThreshold(m, chart.minRankSortOrder))
 .sort((a, b) => (b.rankSortOrder ?? -1) - (a.rankSortOrder ?? -1) ||
memberName(a).localeCompare(memberName(b)));

const nodes = chart.nodes;
const nodeById = new Map(nodes.map((n) => [n.memberId, n]));
const { width, height } = chartSize(nodes);

return (
 <section className="panel org-designer">
 <header className="panel-head org-designer-head">
 <div>
 <h2>Org Chart</h2>
 <p className="muted">
 Add leaders, drag them into place, and connect a manager to their reports. This is the
 public leadership tree.
 </p>
 </div>
 <button type="button" className="primary" onClick={() => void save()} disabled={saving ||
!dirty}>

**src/client/pages/OrgChartDesigner.tsx**

```

 {saving ? 'Saving...' : dirty ? 'Save' : 'Saved'}
 </button>
</header>

{message && <div className="notice">{message}</div>}

<div className="org-toolbar">
 <label className="inline-field">
 Show ranks
 <select
 value={chart.minRankSortOrder}
 onChange={(e) => edit((c) => ({ ...c, minRankSortOrder: Number(e.target.value) })))}
 >
 <option value={0}>Everyone</option>
 {[...ranks].sort((a, b) => b.sortOrder - a.sortOrder).map((r) => (
 <option key={r.id} value={r.sortOrder}>
 {r.name} & above
 </option>
))}
 </select>
 </label>

 <label className="inline-field">
 Add member
 <select value="" onChange={(e) => e.target.value && addMember(Number(e.target.value))}>
 <option value="">{addable.length ? 'Choose...' : 'No eligible members'}</option>
 {addable.map((m) => (
 <option key={m.id} value={m.id}>
 {memberName(m)}
 {m.rankName ? ` ? ${m.rankName}` : ''}
 </option>
))}
 </select>
 </label>

 <button
 type="button"
 className={connectMode ? 'primary' : 'ghost'}
 onClick={() => {
 setConnectMode((v) => !v);
 setConnectFrom(null);
 }}
 >
 {connectMode ? 'Done connecting' : 'Connect reports'}
 </button>
 {connectMode && (

 {connectFrom == null
 ? 'Click a manager, then their report. Keep going to add more ? click "Done
connecting" when finished.'
 : 'Now click the report. (Click the same box again to cancel.)'}

)}
</div>

```

## src/client/pages/OrgChartDesigner.tsx

```

 <div className={connectMode ? 'org-scroll org-editing org-connect' : 'org-scroll
org-editing'}>
 <div className="org-canvas" style={{ width, height }} ref={canvasRef}>
 <svg className="org-edges" width={width} height={height}>
 {chart.edges.map((e) => {
 const a = nodeById.get(e.from);
 const b = nodeById.get(e.to);
 if (!a || !b) return null;
 return (
 <path
 key={`$${e.from}->${e.to}`}
 className="org-edge org-edge-editable"
 d={edgePath(a.x + BOX_W / 2, a.y + BOX_H, b.x + BOX_W / 2, b.y)}
 onClick={() => removeEdge(e.from, e.to)}
 >
 <title>Click to remove this connection</title>
 </path>
);
 })}
 </svg>

 {nodes.map((n) => {
 const m = byId.get(n.memberId);
 const hidden = !meetsThreshold(m, chart.minRankSortOrder);
 const cls = [
 'org-box org-box-editable',
 connectFrom === n.memberId ? 'connecting' : '',
 hidden ? 'org-box-hidden' : '',
].join(' ');
 const style: CSSProperties = { left: n.x, top: n.y, width: BOX_W, height: BOX_H };
 return (
 <div
 key={n.memberId}
 className={cls}
 style={style}
 onPointerDown={(e) => startDrag(e, n.memberId)}
 onClick={() => onBoxClick(n.memberId)}
 title={hidden ? 'Below the rank threshold ? hidden on the public chart' :
undefined}
 >
 {m ? (
 <>
 <div className="org-box-text">
 {memberName(m)}
 {m.rankName && {m.rankName}}
 </div>
 {!connectMode && (
 <button
 type="button"
 className="org-box-remove"
 title="Remove"
 onPointerDown={(e) => e.stopPropagation()}
 onClick={(e) => {

```

**src/client/pages/OrgChartDesigner.tsx**

```
 e.stopPropagation();
 removeNode(n.memberId);
 }}
 >
 ?
 </button>
)}
</>
) : (
 Unknown member
)}
</div>
);
}}
</div>
</div>
</section>
);
}
```

**src/client/pages/PagesAdmin.tsx**

```

/**
 * The page layout editor ? the WYSIWYG / drag-and-drop side of templating.
 *
 * An admin arranges a standard page (the home page today) as rows of columns,
 * dropping modules ? News, Roster, Events, a heading, a rich-text note ? into
 * each column and setting how wide each column is on desktop. Everything is the
 * portable JSON from shared/layout.ts, saved to /api/pages/:slug and rendered by
 * the very same PageRenderer members see, so the preview is truthful. Columns
 * stack on mobile automatically; there is no separate mobile layout to maintain.
 */

import { useEffect, useMemo, useRef, useState } from 'react';
import { api } from '../lib/api';
import PageRenderer from '../components/PageRenderer';
import RichTextEditor from '../components/RichTextEditor';
import {
 HOME_SLUG,
 MODULE_SPECS,
 moduleSpec,
 defaultLayout,
 slugify,
 GRID_UNITS,
 type PageLayout,
 type LayoutRow,
 type LayoutColumn,
 type LayoutModule,
 type ModuleType,
} from '../../shared/layout';
import { resolveEmbed } from '../../shared/embeds';

interface PageMeta {
 slug: string;
 title: string | null;
 showInNav: boolean;
 navOrder: number;
 isHome: boolean;
}

/** A role option for the "Visible to" audience picker. */
interface RoleOpt {
 id: number;
 name: string;
 color: string | null;
}

function newId(prefix: string): string {
 const rand =
 typeof crypto !== 'undefined' && 'randomUUID' in crypto
 ? crypto.randomUUID().slice(0, 8)
 : Math.random().toString(36).slice(2, 10);
 return `${prefix}-${rand}`;
}

type Drag = { rowId: string; colId: string; moduleId: string };

```



**src/client/pages/PagesAdmin.tsx**

```

export default function PagesAdmin() {
 const [pages, setPages] = useState<PageMeta[]>([]);
 const [roles, setRoles] = useState<RoleOpt[]>([]);
 // `null` = the page list (landing view); a slug = editing that page.
 const [slug, setSlug] = useState<string | null>(null);
 const [layout, setLayout] = useState<PageLayout | null>(null);
 const [loading, setLoading] = useState(false);
 const [dirty, setDirty] = useState(false);
 const [saving, setSaving] = useState(false);
 const [preview, setPreview] = useState(false);
 const [message, setMessage] = useState<string | null>(null);
 const dragRef = useRef<Drag | null>(null);

 const loadPages = () =>
 api
 .get<{ pages: PageMeta[] }>('/pages')
 .then(({ pages }) => setPages(pages))
 .catch(() => setPages([
 { slug: HOME_SLUG, title: 'Home page', showInNav: false, navOrder: 0,
isHome: true }
]));

 // After a create/rename/delete/nav-toggle, refresh our own list and tell the
 // rest of the app (the top nav loads page names once) that they changed.
 const refreshPages = async () => {
 await loadPages();
 window.dispatchEvent(new Event('ct-pages-changed'));
 };

 useEffect(() => {
 void loadPages();
 api
 .get<{ roles: RoleOpt[] }>('/pages/meta/roles')
 .then(({ roles }) => setRoles(roles))
 .catch(() => setRoles([]));
 }, []);

 useEffect(() => {
 if (!slug) {
 setLayout(null);
 return;
 }
 setLoading(true);
 setDirty(false);
 setPreview(false);
 api
 .get<{ layout: PageLayout }>(`/pages/${slug}`)
 .then(({ layout }) => setLayout(layout))
 .catch(() => setLayout(defaultLayout(slug)))
 .finally(() => setLoading(false));
 }, [slug]);

 const current = slug ? pages.find((p) => p.slug === slug) : undefined;

 /* --- page management --- */

```

**src/client/pages/PagesAdmin.tsx**

```

async function createPage() {
 const title = window.prompt('Name the new page (e.g. "About Us"):')?.trim();
 if (!title) return;
 const suggested = slugify(title);
 const chosen = window.prompt('Page address (/p/...):', suggested)?.trim().toLowerCase();
 if (!chosen) return;
 setSaving(true);
 setMessage(null);
 try {
 const res = await api.post<{ slug: string }>('/pages', { title, slug: chosen });
 await refreshPages();
 setSlug(res.slug);
 setMessage('Page created. Add some modules and Save.');
```

```
 } catch (err) {
 setMessage(err instanceof Error ? err.message : 'Could not create the page.');
```

```
 } finally {
 setSaving(false);
 }
}

async function renamePage() {
 if (!current) return;
 const title = window.prompt('Rename this page:', current.title ?? '')?.trim();
 if (!title) return;
 await api.patch(`/pages/${slug}`, { title }).catch(() => {});
 await refreshPages();
}

async function deletePage() {
 if (!current || current.isHome) return;
 if (!window.confirm(`Delete the page "${current.title ?? slug}"? This can't be undone.`))
return;
 setSaving(true);
 try {
 await api.del(`/pages/${slug}`);
 await refreshPages();
 setSlug(null);
 setMessage('Page deleted.');
```

```
 } catch {
 setMessage('Could not delete the page.');
```

```
 } finally {
 setSaving(false);
 }
}

/** Immutable edit: clone, mutate the rows, mark dirty. */
function edit(mutator: (rows: LayoutRow[]) => void) {
 setLayout((prev) => {
 if (!prev) return prev;
 const rows = structuredClone(prev.rows);
 mutator(rows);
 return { ...prev, rows };
 });
 setDirty(true);
}

```

**src/client/pages/PagesAdmin.tsx**

```

 setMessage(null);
 }

 const findCol = (rows: LayoutRow[], rowId: string, colId: string): LayoutColumn | undefined =>
 rows.find((r) => r.id === rowId)?.columns.find((c) => c.id === colId);

 /* --- row ops --- */
 const addRow = () =>
 edit((rows) =>
 rows.push({ id: newId('r'), columns: [{ id: newId('c'), span: GRID_UNITS, modules: [] }] }),
);
 const removeRow = (rowId: string) => edit((rows) => {
 const i = rows.findIndex((r) => r.id === rowId);
 if (i >= 0) rows.splice(i, 1);
 });
 const moveRow = (rowId: string, dir: -1 | 1) =>
 edit((rows) => {
 const i = rows.findIndex((r) => r.id === rowId);
 const j = i + dir;
 if (i < 0 || j < 0 || j >= rows.length) return;
 [rows[i], rows[j]] = [rows[j]!, rows[i]!];
 });

 /* --- column ops --- */
 const addColumn = (rowId: string) =>
 edit((rows) => {
 const row = rows.find((r) => r.id === rowId);
 if (row && row.columns.length < 6)
 row.columns.push({ id: newId('c'), span: GRID_UNITS, modules: [] });
 });
 const removeColumn = (rowId: string, colId: string) =>
 edit((rows) => {
 const row = rows.find((r) => r.id === rowId);
 if (!row) return;
 row.columns = row.columns.filter((c) => c.id !== colId);
 if (row.columns.length === 0) rows.splice(rows.indexOf(row), 1);
 });
 const setSpan = (rowId: string, colId: string, span: number) =>
 edit((rows) => {
 const col = findCol(rows, rowId, colId);
 if (col) col.span = span;
 });

 /* --- module ops --- */
 const addModule = (rowId: string, colId: string, type: ModuleType) =>
 edit((rows) => {
 const col = findCol(rows, rowId, colId);
 const spec = moduleSpec(type);
 if (col && spec)
 col.modules.push({ id: newId('m'), type, config: { ...spec.defaultConfig } });
 });
 const removeModule = (rowId: string, colId: string, moduleId: string) =>
 edit((rows) => {
 const col = findCol(rows, rowId, colId);

```

**src/client/pages/PagesAdmin.tsx**

```

 if (col) col.modules = col.modules.filter((m) => m.id !== moduleId);
 });
const moveModule = (rowId: string, colId: string, moduleId: string, dir: -1 | 1) =>
 edit((rows) => {
 const col = findCol(rows, rowId, colId);
 if (!col) return;
 const i = col.modules.findIndex((m) => m.id === moduleId);
 const j = i + dir;
 if (i < 0 || j < 0 || j >= col.modules.length) return;
 [col.modules[i], col.modules[j]] = [col.modules[j]!, col.modules[i]!];
 });
const patchConfig = (rowId: string, colId: string, moduleId: string, patch: Record<string,
unknown>) =>
 edit((rows) => {
 const mod = findCol(rows, rowId, colId)?.modules.find((m) => m.id === moduleId);
 if (mod) mod.config = { ...mod.config, ...patch };
 });
const setVisibility = (rowId: string, colId: string, moduleId: string, visibleToRole: number |
undefined) =>
 edit((rows) => {
 const mod = findCol(rows, rowId, colId)?.modules.find((m) => m.id === moduleId);
 if (!mod) return;
 if (visibleToRole !== null) mod.visibleToRole = visibleToRole;
 else delete mod.visibleToRole;
 });

/** Drop the dragged module into `toCol`, before `beforeId` (or at the end). */
const dropModule = (toRowId: string, toColId: string, beforeId: string | null) => {
 const from = dragRef.current;
 dragRef.current = null;
 if (!from) return;
 if (from.rowId === toRowId && from.colId === toColId && from.moduleId === beforeId) return;
 edit((rows) => {
 const src = findCol(rows, from.rowId, from.colId);
 const dst = findCol(rows, toRowId, toColId);
 if (!src || !dst) return;
 const idx = src.modules.findIndex((m) => m.id === from.moduleId);
 if (idx < 0) return;
 const [moved] = src.modules.splice(idx, 1);
 if (!moved) return;
 const before = beforeId ? dst.modules.findIndex((m) => m.id === beforeId) : -1;
 if (before < 0) dst.modules.push(moved);
 else dst.modules.splice(before, 0, moved);
 });
};

async function save() {
 if (!layout) return;
 setSaving(true);
 setMessage(null);
 try {
 await api.put(`/pages/${slug}`, { layout });
 setDirty(false);
 setMessage('Saved. The page is live.');
```

**src/client/pages/PagesAdmin.tsx**

```

 } catch {
 setMessage('Could not save. Please try again.');
```

} finally {
 setSaving(false);
 }
 }
}

async function resetDefault() {
 if (!window.confirm('Reset this page to the built-in default layout? Your customizations will be removed.')) return;
 setSaving(true);
 try {
 const { layout: def } = await api.del<{ layout: PageLayout }>(`/pages/\${slug}`);
 setLayout(def);
 setDirty(false);
 setMessage('Reset to the default layout.');

} catch {
 setMessage('Could not reset. Please try again.');

} finally {
 setSaving(false);
 }
}

const pageTitle = useMemo(() => current?.title ?? slug ?? '', [current, slug]);

// Landing view: pick a page to edit (or create one) instead of jumping
// straight into the home page.
if (!slug) {
 return (
 <section className="panel pages-admin">
 <header className="panel-head pages-admin-head">
 <div>
 <h2>Pages</h2>
 <p className="muted">Choose a page to edit, or build a new one. Columns stack on mobile.</p>
 </div>
 <div className="pages-admin-actions">
 <button type="button" className="primary" onClick={createPage} disabled={saving}>
 + New page
 </button>
 </div>
 </header>

 {message && <div className="notice">{message}</div>}

 <ul className="pages-list">
 {pages.map((p) => (
 <li key={p.slug}>
 <button type="button" className="pages-list-item" onClick={() => setSlug(p.slug)}>
 <span className="pages-list-name">{p.isHome ? 'Home page' : p.title ?? p.slug}</span>
 <span className="pages-list-meta muted small">
 {p.isHome ? '/' : `/p/\${p.slug}`}
 </span>
 </button>
 </li>
 ))}
 </ul>
 </section>
 );
}

**src/client/pages/PagesAdmin.tsx**

```

 </button>

)}}

</section>
);
}

if (loading || !layout) return <div className="loading">Loading...</div>;

return (
 <section className="panel pages-admin">
 <header className="panel-head pages-admin-head">
 <div>
 <h2>Pages</h2>
 <p className="muted">Arrange modules on the home page, or build custom pages. Columns
stack on mobile.</p>
 </div>
 <div className="pages-admin-actions">
 <button type="button" className="ghost" onClick={() => setSlug(null)} title="Back to all
pages">
 <- All pages
 </button>
 <select value={slug} onChange={(e) => setSlug(e.target.value)} title="Page to edit">
 {pages.map((p) => (
 <option key={p.slug} value={p.slug}>
 {p.isHome ? 'Home page' : p.title ?? p.slug}
 </option>
))}
 </select>
 <button type="button" className="ghost" onClick={createPage} disabled={saving}>
 + New page
 </button>
 <button type="button" className="ghost" onClick={() => setPreview((p) => !p)}>
 {preview ? 'Edit' : 'Preview'}
 </button>
 <button type="button" className="primary" onClick={save} disabled={saving || !dirty}>
 {saving ? 'Saving...' : dirty ? 'Save' : 'Saved'}
 </button>
 </div>
 </header>

 </* Per-page controls: nav placement + rename/delete for custom pages, reset for home. */>
 <div className="pages-admin-meta">
 {current?.isHome ? (
 <button type="button" className="ghost" onClick={resetDefault} disabled={saving}>
 Reset home to default
 </button>
) : (
 <>

 Add this page to the menu under Content -> Navigation.

 <button type="button" className="ghost" onClick={renamePage} disabled={saving}>

```

**src/client/pages/PagesAdmin.tsx**

```

 Rename
 </button>
 <button type="button" className="ghost danger" onClick={deletePage} disabled={saving}>
 Delete page
 </button>

 View ?

 </>
)}
</div>

{message && <div className="notice">{message}</div>}

{preview ? (
 <div className="pages-preview">
 <div className="muted small pages-preview-label">Preview ? {pageTitle}</div>
 <PageRenderer layout={layout} showHidden roles={roles} />
 </div>
) : (
 <div className="layout-editor">
 {layout.rows.map((row, ri) => (
 <RowEditor
 key={row.id}
 row={row}
 index={ri}
 total={layout.rows.length}
 onMoveRow={moveRow}
 onRemoveRow={removeRow}
 onAddColumn={addColumn}
 onRemoveColumn={removeColumn}
 onSetSpan={setSpan}
 onAddModule={addModule}
 onRemoveModule={removeModule}
 onMoveModule={moveModule}
 onPatchConfig={patchConfig}
 onSetVisibility={setVisibility}
 roles={roles}
 onDragStartModule={(d) => (dragRef.current = d)}
 onDropModule={dropModule}
 />
))}
 <button type="button" className="add-row" onClick={addRow}>
 + Add row
 </button>
 </div>
)}
</section>
);
}

/* ----- *
 * Row -> columns -> modules
 * ----- */

```

**src/client/pages/PagesAdmin.tsx**

```

function RowEditor(props: {
 row: LayoutRow;
 index: number;
 total: number;
 onMoveRow: (rowId: string, dir: -1 | 1) => void;
 onRemoveRow: (rowId: string) => void;
 onAddColumn: (rowId: string) => void;
 onRemoveColumn: (rowId: string, colId: string) => void;
 onSetSpan: (rowId: string, colId: string, span: number) => void;
 onAddModule: (rowId: string, colId: string, type: ModuleType) => void;
 onRemoveModule: (rowId: string, colId: string, moduleId: string) => void;
 onMoveModule: (rowId: string, colId: string, moduleId: string, dir: -1 | 1) => void;
 onPatchConfig: (rowId: string, colId: string, moduleId: string, patch: Record<string, unknown>)
=> void;
 onSetVisibility: (rowId: string, colId: string, moduleId: string, visibleToRole: number |
undefined) => void;
 roles: RoleOpt[];
 onDragStartModule: (d: Drag) => void;
 onDropModule: (rowId: string, colId: string, beforeId: string | null) => void;
}) {
 const { row, index, total } = props;
 return (
 <div className="row-editor">
 <div className="row-editor-bar">
 Row {index + 1}
 <div className="row-editor-tools">
 <button type="button" className="mini" title="Move up" disabled={index === 0}
onClick={() => props.onMoveRow(row.id, -1)}>?</button>
 <button type="button" className="mini" title="Move down" disabled={index === total - 1}
onClick={() => props.onMoveRow(row.id, 1)}>?</button>
 <button type="button" className="mini" title="Add column" onClick={() =>
props.onAddColumn(row.id)}>+ Col</button>
 <button type="button" className="mini danger" title="Delete row" onClick={() =>
props.onRemoveRow(row.id)}>?</button>
 </div>
 </div>
 <div className="row-editor-cols">
 {row.columns.map((col) => (
 <ColumnEditor key={col.id} rowId={row.id} col={col} {...props} />
))}
 </div>
 </div>
);
}

function ColumnEditor(props: {
 rowId: string;
 col: LayoutColumn;
 onRemoveColumn: (rowId: string, colId: string) => void;
 onSetSpan: (rowId: string, colId: string, span: number) => void;
 onAddModule: (rowId: string, colId: string, type: ModuleType) => void;
 onRemoveModule: (rowId: string, colId: string, moduleId: string) => void;
 onMoveModule: (rowId: string, colId: string, moduleId: string, dir: -1 | 1) => void;

```



**src/client/pages/PagesAdmin.tsx**

```

 onPatchConfig: (rowId: string, colId: string, moduleId: string, patch: Record<string, unknown>)
=> void;
 onSetVisibility: (rowId: string, colId: string, moduleId: string, visibleToRole: number |
undefined) => void;
 roles: RoleOpt[];
 onDragStartModule: (d: Drag) => void;
 onDropModule: (rowId: string, colId: string, beforeId: string | null) => void;
 }) {
 const { rowId, col } = props;
 const [over, setOver] = useState(false);
 return (
 <div
 className="col-editor"
 style={{ flexGrow: col.span, flexBasis: `${(col.span / GRID_UNITS) * 100}%` }}
 >
 <div className="col-editor-bar">
 <label className="col-span-label">
 Width
 <select value={col.span} onChange={(e) => props.onSetSpan(rowId, col.id,
Number(e.target.value))}>
 {Array.from({ length: GRID_UNITS }, (_, i) => i + 1).map((n) => (
 <option key={n} value={n}>
 {n}/{GRID_UNITS}
 </option>
))}
 </select>
 </label>
 <button type="button" className="mini danger" title="Delete column" onClick={() =>
props.onRemoveColumn(rowId, col.id)}>?</button>
 </div>

 <div
 className={over ? 'col-dropzone over' : 'col-dropzone'}
 onDragOver={(e) => {
 e.preventDefault();
 setOver(true);
 }}
 onDragLeave={() => setOver(false)}
 onDrop={(e) => {
 e.preventDefault();
 setOver(false);
 props.onDropModule(rowId, col.id, null);
 }}
 >
 {col.modules.length === 0 && <div className="col-empty">Drop or add a module</div>}
 {col.modules.map((m, mi) => (
 <ModuleEditor
 key={m.id}
 {...props}
 rowId={rowId}
 colId={col.id}
 module={m}
 index={mi}
 total={col.modules.length}

```

**src/client/pages/PagesAdmin.tsx**

```

 />
)}}
</div>

<select
 className="add-module"
 value=""
 onChange={(e) => {
 if (e.target.value) props.onAddModule(rowId, col.id, e.target.value as ModuleType);
 e.target.value = '';
 }}
>
 <option value="">+ Add module...</option>
 {MODULE_SPECS.map((spec) => (
 <option key={spec.type} value={spec.type}>
 {spec.label}
 </option>
))}
</select>
</div>
);
}

function ModuleEditor(props: {
 rowId: string;
 colId: string;
 module: LayoutModule;
 index: number;
 total: number;
 onRemoveModule: (rowId: string, colId: string, moduleId: string) => void;
 onMoveModule: (rowId: string, colId: string, moduleId: string, dir: -1 | 1) => void;
 onPatchConfig: (rowId: string, colId: string, moduleId: string, patch: Record<string, unknown>)
=> void;
 onSetVisibility: (rowId: string, colId: string, moduleId: string, visibleToRole: number |
undefined) => void;
 roles: RoleOpt[];
 onDragStartModule: (d: Drag) => void;
 onDropModule: (rowId: string, colId: string, beforeId: string | null) => void;
}) {
 const { rowId, colId, module: m, index, total } = props;
 const spec = moduleSpec(m.type);
 const cfg = m.config;
 // Only make the card draggable while the ?? handle is held. If the whole card
 // were always `draggable`, a click-drag inside any text field or the rich-text
 // editor would start an element drag instead of selecting text ? the reported
 // "can't select text after trying to move one" bug.
 const [grabbing, setGrabbing] = useState(false);

 return (
 <div
 className="module-editor"
 draggable={grabbing}
 onDragStart={(e) => {
 props.onDragStartModule({ rowId, colId, moduleId: m.id });

```

**src/client/pages/PagesAdmin.tsx**

```

 e.dataTransfer.effectAllowed = 'move';
 }}
 onDragEnd={() => setGrabbing(false)}
 onDragOver={(e) => e.preventDefault()}
 onDrop={(e) => {
 e.preventDefault();
 e.stopPropagation();
 setGrabbing(false);
 props.onDropModule(rowId, colId, m.id);
 }}
>
<div className="module-editor-bar">
 <span
 className="module-editor-drag"
 title="Drag to move"
 // Arm dragging only for a gesture that begins on the handle; disarm if
 // it was just a click (mouseup with no drag) so text stays selectable.
 onMouseDown={() => setGrabbing(true)}
 onMouseUp={() => setGrabbing(false)}
 >
 ??

 {spec?.label ?? m.type}
 <div className="module-editor-tools">
 <button type="button" className="mini" title="Move up" disabled={index === 0}
onClick={() => props.onMoveModule(rowId, colId, m.id, -1)}></button>
 <button type="button" className="mini" title="Move down" disabled={index === total - 1}
onClick={() => props.onMoveModule(rowId, colId, m.id, 1)}></button>
 <button type="button" className="mini danger" title="Remove" onClick={() =>
props.onRemoveModule(rowId, colId, m.id)}></button>
 </div>
</div>

<div className="module-editor-config">
 <label className="inline-field module-audience">
 Visible to
 <select
 value={m.visibleToRole !== null ? String(m.visibleToRole) : ''}
 onChange={(e) =>
 props.onSetVisibility(rowId, colId, m.id, e.target.value ? Number(e.target.value) :
undefined)
 }
 >
 <option value="">Everyone</option>
 {props.roles.map((r) => (
 <option key={r.id} value={r.id}>
 {r.name}
 </option>
))}
 </select>
 </label>

 {m.type === 'heading' && (
 <>

```

**src/client/pages/PagesAdmin.tsx**

```

 <input
 type="text"
 value={typeof cfg.text === 'string' ? cfg.text : ''}
 placeholder="Heading text"
 onChange={(e) => props.onPatchConfig(rowId, colId, m.id, { text: e.target.value })}
 />
 <label className="inline-field">
 Size
 <select
 value={typeof cfg.level === 'number' ? cfg.level : 2}
 onChange={(e) => props.onPatchConfig(rowId, colId, m.id, { level:
Number(e.target.value) })}
 >
 <option value={1}>Large</option>
 <option value={2}>Medium</option>
 <option value={3}>Small</option>
 </select>
 </label>
 </>
)}

{m.type === 'text' && (
 <RichTextEditor
 key={m.id}
 value={typeof cfg.html === 'string' ? cfg.html : ''}
 onChange={(html) => props.onPatchConfig(rowId, colId, m.id, { html })}
 />
)}

{m.type === 'html' && (
 <>
 <textarea
 className="html-config"
 rows={8}
 spellCheck={false}
 value={typeof cfg.html === 'string' ? cfg.html : ''}
 placeholder={'<h3>Title</h3>\n<p>Your HTML...</p>'}
 onChange={(e) => props.onPatchConfig(rowId, colId, m.id, { html: e.target.value })}
 />
 <p className="muted small">
 Headings, lists, tables, images, links and formatting are allowed. Scripts,
 inline styles, event handlers and iframes are removed when the page renders ?
 use a Video embed module for videos.
 </p>
 </>
)}

{m.type === 'hero' && (
 <HeroConfig config={cfg} onPatch={(patch) => props.onPatchConfig(rowId, colId, m.id,
patch)} />
)}

{m.type === 'embed' && (
 <EmbedConfig config={cfg} onPatch={(patch) => props.onPatchConfig(rowId, colId, m.id,

```

**src/client/pages/PagesAdmin.tsx**

```

patch)) />
 })

 {(m.type === 'news' ||
 m.type === 'roster' ||
 m.type === 'events' ||
 m.type === 'medals' ||
 m.type === 'warrecords' ||
 m.type === 'games') && (
 <>
 <input
 type="text"
 value={typeof cfg.title === 'string' ? cfg.title : ''}
 placeholder="Section title"
 onChange={(e) => props.onPatchConfig(rowId, colId, m.id, { title: e.target.value })}
 />
 <label className="inline-field">
 Show up to
 <input
 type="number"
 min={1}
 max={50}
 value={typeof cfg.limit === 'number' ? cfg.limit : 5}
 onChange={(e) => props.onPatchConfig(rowId, colId, m.id, { limit:
Number(e.target.value) })}
 />
 </label>
 </>
)}

 {m.type === 'image' && (
 <ImageConfig
 config={cfg}
 onPatch={(patch) => props.onPatchConfig(rowId, colId, m.id, patch)}
 />
)}

 {m.type === 'button' && (
 <>
 <input
 type="text"
 value={typeof cfg.label === 'string' ? cfg.label : ''}
 placeholder="Button label"
 onChange={(e) => props.onPatchConfig(rowId, colId, m.id, { label: e.target.value })}
 />
 <input
 type="text"
 value={typeof cfg.href === 'string' ? cfg.href : ''}
 placeholder="Link URL (/roster or https://...)"
 onChange={(e) => props.onPatchConfig(rowId, colId, m.id, { href: e.target.value })}
 />
 <label className="inline-field">
 Style
 <select

```

**src/client/pages/PagesAdmin.tsx**

```

 value={cfg.style === 'default' ? 'default' : 'primary'}
 onChange={(e) => props.onPatchConfig(rowId, colId, m.id, { style: e.target.value
 }}}

 >
 <option value="primary">Primary</option>
 <option value="default">Subtle</option>
 </select>
</label>
</>
)}

 {m.type === 'divider' && <p className="muted small">A horizontal divider ? no
options.</p>}
 </div>
</div>
);
}

/**
 * Video-embed config: paste any provider URL. We resolve it to a canonical,
 * origin-locked src on the spot (so the preview works immediately and the stored
 * value is already safe); the server re-derives it on save regardless.
 */
function EmbedConfig({
 config,
 onPatch,
}): {
 config: Record<string, unknown>;
 onPatch: (patch: Record<string, unknown>) => void;
}) {
 const url = typeof config.url === 'string' ? config.url : '';
 const resolved = resolveEmbed(url);
 return (
 <div className="module-embed-config">
 <input
 type="text"
 value={url}
 placeholder="Paste a YouTube, Twitch, Vimeo or Streamable link"
 onChange={(e) => {
 const next = e.target.value;
 const r = resolveEmbed(next);
 onPatch({ url: next, src: r?.src ?? '', provider: r?.provider ?? '' });
 }}
 />
 <input
 type="text"
 value={typeof config.title === 'string' ? config.title : ''}
 placeholder="Caption / accessible title (optional)"
 onChange={(e) => onPatch({ title: e.target.value })}
 />
 <label className="inline-field">
 Shape
 <select
 value={config.ratio === '4:3' ? '4:3' : '16:9'}

```

**src/client/pages/PagesAdmin.tsx**

```

 onChange={({e} => onPatch({ ratio: e.target.value })})
 >
 <option value="16:9">Widescreen 16:9</option>
 <option value="4:3">Classic 4:3</option>
 </select>
</label>
{url &&
 (resolved ? (
 <p className="muted small">? {resolved.provider} embed detected.</p>
) : (
 <p className="muted small module-image-err">
 Not a supported link ? use YouTube, Twitch, Vimeo or Streamable.
 </p>
))}
</div>
);
}

/** Image module config: an uploader (to the 'pages' media category) plus alt/link/caption. */
function ImageConfig({
 config,
 onPatch,
}: {
 config: Record<string, unknown>;
 onPatch: (patch: Record<string, unknown>) => void;
}) {
 const [uploading, setUploading] = useState(false);
 const [err, setErr] = useState<string | null>(null);
 const url = typeof config.url === 'string' ? config.url : '';

 return (
 <div className="module-image-config">
 {url && }
 <div className="avatar-controls">
 <label className="upload-btn mini">
 {uploading ? 'Uploading...' : url ? 'Change image' : 'Upload image'}
 <input
 type="file"
 accept="image/png,image/jpeg,image/gif,image/webp"
 hidden
 disabled={uploading}
 onChange={async (e) => {
 const file = e.target.files?.[0];
 e.target.value = '';
 if (!file) return;
 setErr(null);
 setUploading(true);
 try {
 const res = await api.upload<{ url: string }>('/media/pages', file);
 onPatch({ url: res.url });
 } catch (e2) {
 setErr(e2 instanceof Error ? e2.message : 'Upload failed.');
```

**src/client/pages/PagesAdmin.tsx**

```

 }
 }}
 />
 </label>
 {url && (
 <button type="button" className="mini" disabled={uploading} onClick={() => onPatch({
url: ' ' })}>
 Remove
 </button>
)}
</div>
{err && <p className="muted small module-image-err">{err}</p>}
<input
 type="text"
 value={typeof config.alt === 'string' ? config.alt : ''}
 placeholder="Alt text (accessibility)"
 onChange={(e) => onPatch({ alt: e.target.value })}
/>
<input
 type="text"
 value={typeof config.href === 'string' ? config.href : ''}
 placeholder="Link URL (optional)"
 onChange={(e) => onPatch({ href: e.target.value })}
/>
<input
 type="text"
 value={typeof config.caption === 'string' ? config.caption : ''}
 placeholder="Caption (optional)"
 onChange={(e) => onPatch({ caption: e.target.value })}
/>
</div>
);
}

interface HeroCardEdit {
 icon: string;
 title: string;
 tag: string;
 body: string;
}

/** Hero banner config: headline + CTAs + editable feature chips and value cards. */
function HeroConfig({
 config,
 onPatch,
}: {
 config: Record<string, unknown>;
 onPatch: (patch: Record<string, unknown>) => void;
}) {
 const s = (k: string): string => (typeof config[k] === 'string' ? (config[k] as string) : '');
 const chips: string[] = Array.isArray(config.chips)
 ? (config.chips as unknown[]).map((c) => (typeof c === 'string' ? c : ''))
 : [];
 const cards: HeroCardEdit[] = Array.isArray(config.cards)

```



**src/client/pages/PagesAdmin.tsx**

```

? (config.cards as unknown[]).map((c) => {
 const o = (c && typeof c === 'object' ? c : {}) as Record<string, unknown>;
 return {
 icon: typeof o.icon === 'string' ? o.icon : '',
 title: typeof o.title === 'string' ? o.title : '',
 tag: typeof o.tag === 'string' ? o.tag : '',
 body: typeof o.body === 'string' ? o.body : '',
 };
})
: [];

const patchCard = (i: number, patch: Partial<HeroCardEdit>) =>
 onPatch({ cards: cards.map((c, ci) => (ci === i ? { ...c, ...patch } : c)) });
const addCard = () => onPatch({ cards: [...cards, { icon: '', title: '', tag: '', body: '' }]
});
const removeCard = (i: number) => onPatch({ cards: cards.filter((_, ci) => ci !== i) });

return (
 <div className="hero-config">
 <input
 type="text"
 value={s('eyebrow')}
 placeholder="Eyebrow ? small label above the headline"
 onChange={(e) => onPatch({ eyebrow: e.target.value })}
 />
 <input
 type="text"
 value={s('headline')}
 placeholder="Headline"
 onChange={(e) => onPatch({ headline: e.target.value })}
 />
 <textarea
 rows={3}
 value={s('subhead')}
 placeholder="Sub-headline paragraph"
 onChange={(e) => onPatch({ subhead: e.target.value })}
 />
 <div className="hero-config-row">
 <input
 type="text"
 value={s('primaryLabel')}
 placeholder="Primary button label"
 onChange={(e) => onPatch({ primaryLabel: e.target.value })}
 />
 <input
 type="text"
 value={s('primaryHref')}
 placeholder="Primary link (/api/auth/login, /roster, https://...)"
 onChange={(e) => onPatch({ primaryHref: e.target.value })}
 />
 </div>
 <div className="hero-config-row">
 <input
 type="text"

```

**src/client/pages/PagesAdmin.tsx**

```

 value={s('secondaryLabel')}
 placeholder="Secondary button label"
 onChange={(e) => onPatch({ secondaryLabel: e.target.value })}
 />
 <input
 type="text"
 value={s('secondaryHref')}
 placeholder="Secondary link (https://discord.com/oauth2/...)"
 onChange={(e) => onPatch({ secondaryHref: e.target.value })}
 />
</div>
<label className="hero-config-label">Feature chips ? one per line</label>
<textarea
 rows={4}
 value={chips.join('\n')}
 placeholder={'? Real-Time Discord Role Sync\n?? Custom Medals & Service Records'}
 onChange={(e) => onPatch({ chips: e.target.value.split('\n') })}
/>
<label className="hero-config-label">Value cards</label>
{cards.map((c, i) => (
 <div className="hero-config-card" key={i}>
 <div className="hero-config-row">
 <input
 className="hero-config-icon"
 type="text"
 value={c.icon}
 placeholder="Icon"
 onChange={(e) => patchCard(i, { icon: e.target.value })}
 />
 <input
 type="text"
 value={c.title}
 placeholder="Card title"
 onChange={(e) => patchCard(i, { title: e.target.value })}
 />
 <button type="button" className="mini danger" title="Remove card" onClick={() =>
removeCard(i)}>
 ?
 </button>
 </div>
 <input
 type="text"
 value={c.tag}
 placeholder="Tagline"
 onChange={(e) => patchCard(i, { tag: e.target.value })}
 />
 <textarea
 rows={2}
 value={c.body}
 placeholder="Card description"
 onChange={(e) => patchCard(i, { body: e.target.value })}
 />
 </div>
))}

```

**src/client/pages/PagesAdmin.tsx**

```
 <button type="button" className="mini" onClick={addCard}>
 + Add card
 </button>
 </div>
);
}
```

**src/client/pages/Ranks.tsx**

```

/**
 * Rank administration ? add, rename, reorder, delete, and choose which roles
 * each rank grants.
 *
 * This screen is the reason for the rewrite: the 2003 version rendered 21
 * hardcoded rows of `

```

**src/client/pages/Ranks.tsx**

```

// A rank pending deletion that still has members ? the reassign dialog is open.
const [pendingDelete, setPendingDelete] = useState<Rank | null>(null);

async function load() {
 const { ranks } = await api.get<{ ranks: Rank[] }>('/ranks');
 setRanks([...ranks].sort((a, b) => b.sortOrder - a.sortOrder));
}

const { run, busy, error, notice, warning } = useAction(load);

useEffect(() => {
 void load();
 if (canManageRoles) {
 api
 .get<{ roles: Role[] }>('/roles')
 .then(({ roles }) => setRoles(roles))
 .catch(() => setRoles([]));
 }
}, [canManageRoles]);

const addRank = () =>
 run(async () => {
 if (!newName.trim()) return;
 await api.post('/ranks', { name: newName.trim() });
 setNewName('');
 });

const rename = (id: number, name: string) => run(() => api.patch(`/ranks/${id}`, { name }));
const setDefault = (id: number) => run(() => api.patch(`/ranks/${id}`, { isDefault: true }));
const setImage = (id: number, imageUrl: string | null) =>
 run(() => api.patch(`/ranks/${id}`, { imageUrl }));

// Member-less ranks delete with a plain confirm; ones with members open the
// reassign dialog so the admin says where those members go.
const remove = (rank: Rank) => {
 if (rank.memberCount > 0) {
 setPendingDelete(rank);
 return;
 }
 if (!window.confirm(`Delete the rank "${rank.name}"?`)) return;
 void run(() => api.del(`/ranks/${rank.id}`));
};

const confirmReassignDelete = (target: number | null, reason: string) => {
 const rank = pendingDelete;
 if (!rank) return;
 void run(async () => {
 await api.del(`/ranks/${rank.id}`, { reassignTo: target, reason });
 setPendingDelete(null);
 const where = target !== null ? `${ranks.find((r) => r.id === target)?.name ?? 'another rank'}` : 'no rank';
 return `Deleted "${rank.name}" and moved ${rank.memberCount} member${rank.memberCount === 1 ? '' : 's'} to ${where}.`;
 });
};

```

**src/client/pages/Ranks.tsx**

```

};

const move = (index: number, direction: -1 | 1) =>
 run(() => {
 const next = [...ranks];
 const target = index + direction;
 if (target < 0 || target >= next.length) return Promise.resolve();
 [next[index], next[target]] = [next[target]!, next[index]!];
 return api.put('/ranks/order', { order: next.map((r) => r.id).reverse() });
 });

const saveRankRoles = (rankId: number, roleIds: number[]) =>
 run(async () => {
 const res = await api.put<{
 applied: { members: number; warnings: string[] };
 }>(`/ranks/${rankId}/roles`, { roleIds });
 const { members, warnings } = res.applied;
 const base =
 members > 0
 ? `Saved. Applied to ${members} member${members === 1 ? '' : 's'} at this rank.`
 : 'Saved.';
 return warnings.length ? { warning: `${base} ${warnings[0]}` } : base;
 });

const selectedRank = ranks.find((r) => r.id === selectedRankId) ?? null;

return (
 <section className="panel">
 <header className="panel-head">
 <h2>Ranks</h2>
 <p className="muted">
 {ranks.length} rank{ranks.length === 1 ? '' : 's'}. Highest first.
 </p>
 </header>

 <Alerts error={error} warning={warning} notice={notice} />

 <div className="add-row">
 <input
 value={newName}
 placeholder="New rank name"
 onChange={(e) => setNewName(e.target.value)}
 onKeyDown={(e) => e.key === 'Enter' && void addRank()}
 disabled={busy}
 />
 <button onClick={() => void addRank()} disabled={busy || !newName.trim()}>
 Add rank
 </button>
 </div>

 <table className="grid">
 <thead>
 <tr>
 <th>Order</th>

```

**src/client/pages/Ranks.tsx**

```

 <th>Image</th>
 <th>Name</th>
 <th>Members</th>
 <th>Default</th>
 {canManageRoles && <th>Roles</th>}
 <th />
 </tr>
 </thead>
 <tbody>
 {ranks.map((rank, index) => (
 <tr key={rank.id}>
 <td className="reorder">
 <button onClick={() => void move(index, -1)} disabled={busy || index === 0}>
 ?
 </button>
 <button
 onClick={() => void move(index, 1)}
 disabled={busy || index === ranks.length - 1}
 >
 ?
 </button>
 </td>
 <td>
 <RankImageCell
 rank={rank}
 busy={busy}
 onSet={(url) => setImage(rank.id, url)}
 />
 </td>
 <td>
 <input
 defaultValue={rank.name}
 onBlur={(e) =>
 e.target.value !== rank.name && void rename(rank.id, e.target.value)
 }
 disabled={busy}
 />
 </td>
 <td>{rank.memberCount}</td>
 <td>
 <input
 type="radio"
 name="default-rank"
 checked={rank.isDefault}
 onChange={() => void setDefault(rank.id)}
 disabled={busy}
 title="Rank given to new recruits on first sign-in"
 />
 </td>
 {canManageRoles && (
 <td>
 <button
 className={rank.id === selectedRankId ? 'active' : ''}
 onClick={() => setSelectedRankId(rank.id === selectedRankId ? null : rank.id)}
 >

```

**src/client/pages/Ranks.tsx**

```

 >
 Roles ({rank.roleIds.length})
 </button>
 </td>
)}
</td>
 <button
 className="danger"
 onClick={() => remove(rank)}
 disabled={busy}
 title={
 rank.memberCount > 0
 ? 'Delete this rank and move its members elsewhere'
 : 'Delete this rank'
 }
 >
 Delete
 </button>
</td>
</tr>
))}
</tbody>
</table>

{pendingDelete && (
 <ReassignDialog
 title={`Delete rank "${pendingDelete.name}"`}
 count={pendingDelete.memberCount}
 options={ranks.filter((r) => r.id !== pendingDelete.id).map((r) => ({ value: r.id,
label: r.name })))}
 defaultValue={nextLowerRankId(ranks, pendingDelete)}
 noneLabel="No rank (unranked)"
 busy={busy}
 onConfirm={confirmReassignDelete}
 onCancel={() => setPendingDelete(null)}
 />
)}

{canManageRoles && selectedRank && (
 <RankRolesEditor
 key={selectedRank.id}
 rank={selectedRank}
 roles={roles}
 busy={busy}
 onSave={(roleIds) => saveRankRoles(selectedRank.id, roleIds)}
 onClose={() => setSelectedRankId(null)}
 />
)}
</section>
);
}

/**
 * A rank's insignia image, with upload/remove. Uploads land in R2 under ranks/

```



**src/client/pages/Ranks.tsx**

```

* and the returned URL is saved to the rank; the previous object is cleaned up
* server-side.
*/
function RankImageCell({
 rank,
 busy,
 onSet,
}: {
 rank: Rank;
 busy: boolean;
 onSet: (url: string | null) => void;
}) {
 const [uploading, setUploading] = useState(false);

 async function pickFile(e: ChangeEvent<HTMLInputElement>) {
 const file = e.target.files?.[0];
 e.target.value = '';
 if (!file) return;
 setUploading(true);
 try {
 const { url } = await api.upload<{ url: string }>('/media/ranks', file);
 onSet(url);
 } finally {
 setUploading(false);
 }
 }

 return (
 <div className="rank-image-cell">

 {rank.imageUrl ? (

) : (
 ?
)}

 <label className="upload-btn mini">
 {uploading ? '...' : rank.imageUrl ? 'Change' : 'Upload'}
 <input
 type="file"
 accept="image/png,image/jpeg,image/gif,image/webp"
 onChange={(e) => void pickFile(e)}
 disabled={busy || uploading}
 hidden
 />
 </label>
 {rank.imageUrl && (
 <button className="mini" disabled={busy || uploading} onClick={() => onSet(null)}
 title="Remove image">
 ?
 </button>
)}
 </div>
);
}

```

**src/client/pages/Ranks.tsx**

```

}

function RankRolesEditor({
 rank,
 roles,
 busy,
 onSave,
 onClose,
}): {
 rank: Rank;
 roles: Role[];
 busy: boolean;
 onSave: (roleIds: number[]) => void;
 onClose: () => void;
} {
 const [selected, setSelected] = useState<Set<number>>(new Set(rank.roleIds));

 const dirty =
 selected.size !== rank.roleIds.length || rank.roleIds.some((id) => !selected.has(id));

 const toggle = (id: number) =>
 setSelected((prev) => {
 const next = new Set(prev);
 next.has(id) ? next.delete(id) : next.add(id);
 return next;
 });

 return (
 <div className="rank-roles-editor">
 <div className="rre-head">
 <h3>Roles granted by "{rank.name}"</h3>
 <button onClick={onClose} title="Close">
 ?
 </button>
 </div>
 <p className="muted small">
 Members at this rank receive these roles automatically. Changing this updates everyone
 currently at the rank, and mapped Discord roles follow.
 </p>

 {roles.length === 0 ? (
 <p className="muted">No roles exist yet. Create some on the Roles page first.</p>
) : (
 <div className="rre-roles">
 {roles.map((role) => (
 <label key={role.id} className="rre-role">
 <input
 type="checkbox"
 checked={selected.has(role.id)}
 onChange={() => toggle(role.id)}
 disabled={busy}
 />

 {role.name}
 </label>
))}
 </div>
)}
 </div>
);
}

```

**src/client/pages/Ranks.tsx**

```
 </label>
)}
 </div>
)}

 <button className="primary" disabled={busy || !dirty} onClick={() => onSave([...selected])}>
 Save rank roles
 </button>
</div>
);
}
```

**src/client/pages/Roles.tsx**

```

/**
 * Role administration.
 *
 * Roles carry permissions and, when mapped to a Discord role, drive Discord
 * membership too. This is the browser side of the integration the original
 * ClanTek never had ? the 2003 version gated everything on a single rank
 * threshold with no concept of a capability grant.
 */

import { useEffect, useMemo, useState } from 'react';
import { api } from '../lib/api';
import { useAction, Alerts } from '../lib/action';
import { PERMISSIONS, type Permission } from '../../shared/permissions';
import ReassignDialog from '../components/ReassignDialog';

interface Role {
 id: number;
 name: string;
 description: string | null;
 color: string | null;
 discordRoleId: string | null;
 isSystem: boolean;
 memberCount: number;
 permissions: Permission[];
}

interface DiscordRole {
 id: string;
 name: string;
 color: string | null;
 position: number;
}

// Group "roster.view", "roster.edit", ... under a "roster" heading.
const PERMISSION_GROUPS = Object.entries(PERMISSIONS).reduce<Record<string, [Permission, string][]>>(
 (acc, [perm, label]) => {
 const area = perm.split('.')[0]!;
 (acc[area] ??= []).push([perm as Permission, label]);
 return acc;
 },
 {},
);

export default function Roles() {
 const [roles, setRoles] = useState<Role[]>([]);
 const [discordRoles, setDiscordRoles] = useState<DiscordRole[]>([]);
 const [discordWarning, setDiscordWarning] = useState<string | null>(null);
 const [selectedId, setSelectedId] = useState<number | null>(null);
 const [newName, setNewName] = useState('');
 const [loading, setLoading] = useState(true);
 // A role pending deletion that members still hold ? the reassign dialog is open.
 const [pendingDelete, setPendingDelete] = useState<Role | null>(null);

```

**src/client/pages/Roles.tsx**

```

 async function load() {
 const { roles } = await api.get<{ roles: Role[] }>('/roles');
 setRoles(roles);
 }

 const { run, busy, error, notice, warning } = useAction(load);

 useEffect(() => {
 Promise.all([
 load(),
 api
 .get<{ roles: DiscordRole[]; warning?: string }>('/roles/discord-roles')
 .then(({ roles, warning }) => {
 setDiscordRoles(roles);
 setDiscordWarning(warning ?? null);
 })
 .catch(() => setDiscordWarning('Could not reach Discord to load roles.')),
]).finally(() => setLoading(false));
 }, []);

 const addRole = () =>
 run(async () => {
 if (!newName.trim()) return;
 const { role } = await api.post<{ role: Role }>('/roles', { name: newName.trim() });
 setNewName('');
 setSelectedId(role.id);
 });

 const selected = roles.find((r) => r.id === selectedId) ?? null;

 if (loading) return <div className="loading">Loading roles...</div>;

 return (
 <section className="panel roles-page">
 <header className="panel-head">
 <h2>Roles</h2>
 <p className="muted">
 {roles.length} role{roles.length === 1 ? '' : 's'}. Roles grant permissions and can
 mirror
 a Discord role.
 </p>
 </header>

 <Alerts error={error} warning={warning} notice={notice} />

 <div className="add-row">
 <input
 value={newName}
 placeholder="New role name"
 onChange={(e) => setNewName(e.target.value)}
 onKeyDown={(e) => e.key === 'Enter' && void addRole()}
 disabled={busy}
 />
 <button onClick={() => void addRole()} disabled={busy || !newName.trim()}>

```

**src/client/pages/Roles.tsx**

```

 Add role
 </button>
</div>

<div className="roles-layout">
 <ul className="role-list">
 {roles.map((role) => (
 <li key={role.id}>
 <button
 className={role.id === selectedId ? 'role-chip active' : 'role-chip'}
 onClick={() => setSelectedId(role.id)}
 >

 {role.name}
 {role.isSystem && system}
 {role.memberCount}
 </button>

))}

 {selected ? (
 <RoleEditor
 key={selected.id}
 role={selected}
 discordRoles={discordRoles}
 discordWarning={discordWarning}
 busy={busy}
 onSave={(patch) =>
 run(async () => {
 const res = await api.patch<{
 discordSync?: { synced: boolean; warning?: string };
 }>(`/roles/${selected.id}`, patch);
 if (res.discordSync?.warning) return { warning: res.discordSync.warning };
 if (res.discordSync?.synced) return 'Saved ? the Discord role's name and color now
match.';
 return 'Saved.';
 })
 }
 onSavePermissions={(permissions) =>
 run(() => api.put(`/roles/${selected.id}/permissions`, { permissions }))
 }
 onDelete={() => {
 // With members, open the reassign dialog; empty roles delete outright.
 if (selected.memberCount > 0) {
 setPendingDelete(selected);
 return;
 }
 if (!window.confirm(`Delete the role "${selected.name}"?`)) return;
 void run(async () => {
 await api.del(`/roles/${selected.id}`);
 setSelectedId(null);
 });
 }
)}
)}

```

**src/client/pages/Roles.tsx**

```

 }}
 />
) : (
 <div className="role-editor empty-editor">Select a role to edit it.</div>
)}
</div>

{pendingDelete && (
 <ReassignDialog
 title={`Delete role "${pendingDelete.name}"`}
 count={pendingDelete.memberCount}
 options={roles.filter((r) => r.id !== pendingDelete.id).map((r) => ({ value: r.id,
label: r.name })))}
 defaultValue={null}
 noneLabel="Remove from everyone (no replacement)"
 busy={busy}
 onConfirm={({target, reason}) =>
 run(async () => {
 const name = pendingDelete.name;
 const memberCount = pendingDelete.memberCount;
 await api.del(`/roles/${pendingDelete.id}`, { reassignTo: target, reason });
 setSelectedId(null);
 setPendingDelete(null);
 const outcome =
 target !== null
 ? `moved ${memberCount} member${memberCount === 1 ? '' : 's'} to
"${roles.find((r) => r.id === target)?.name ?? 'another role'}"`
 : `removed it from ${memberCount} member${memberCount === 1 ? '' : 's'}`;
 return `Deleted "${name}" and ${outcome}.`;
 })
 }
 onCancel={() => setPendingDelete(null)}
 />
)}
</section>
);
}

function RoleEditor({
 role,
 discordRoles,
 discordWarning,
 busy,
 onSave,
 onSavePermissions,
 onDelete,
}: {
 role: Role;
 discordRoles: DiscordRole[];
 discordWarning: string | null;
 busy: boolean;
 onSave: (patch: Partial<Role>) => void;
 onSavePermissions: (permissions: Permission[]) => void;
 onDelete: () => void;

```

**src/client/pages/Roles.tsx**

```

})) {
 const [name, setName] = useState(role.name);
 const [description, setDescription] = useState(role.description ?? '');
 const [color, setColor] = useState(role.color ?? '#7f8c8d');
 const [discordRoleId, setDiscordRoleId] = useState(role.discordRoleId ?? '');
 const [perms, setPerms] = useState<Set<Permission>>(new Set(role.permissions));

 const permsDirty = useMemo(() => {
 const original = new Set(role.permissions);
 return original.size !== perms.size || [...perms].some((p) => !original.has(p));
 }, [perms, role.permissions]);

 const detailsDirty =
 name !== role.name ||
 description !== (role.description ?? '') ||
 color !== (role.color ?? '#7f8c8d') ||
 discordRoleId !== (role.discordRoleId ?? '');

 const toggle = (p: Permission) =>
 setPerms((prev) => {
 const next = new Set(prev);
 next.has(p) ? next.delete(p) : next.add(p);
 return next;
 });

 // A mapped Discord role that no longer exists in the guild would otherwise
 // vanish silently from the dropdown; keep it visible so it can be cleared.
 const mappedButMissing =
 role.discordRoleId && !discordRoles.some((d) => d.id === role.discordRoleId);

 return (
 <div className="role-editor">
 <div className="field-row">
 <label>
 Name
 <input value={name} onChange={(e) => setName(e.target.value)} disabled={busy} />
 </label>
 <label className="color-field">
 Color
 <input type="color" value={color} onChange={(e) => setColor(e.target.value)}
disabled={busy} />
 </label>
 </div>

 <label>
 Description
 <input
 value={description}
 onChange={(e) => setDescription(e.target.value)}
 placeholder="What this role is for"
 disabled={busy}
 />
 </label>
 </div>
);
}

```



**src/client/pages/Roles.tsx**

```

<label>
 Mirrored Discord role
 <select
 value={discordRoleId}
 onChange={(e) => setDiscordRoleId(e.target.value)}
 disabled={busy}
 >
 <option value="">? none ?</option>
 {mappedButMissing && (
 <option value={role.discordRoleId!}>
 (unknown role {role.discordRoleId})
 </option>
)}
 {discordRoles.map((d) => (
 <option key={d.id} value={d.id}>
 {d.name}
 </option>
))}
 </select>
</label>
{discordWarning && <p className="muted small warn">{discordWarning}</p>}
{discordRoleId && !discordWarning && (
 <p className="muted small">
 On save, the Discord role is renamed to "{name}" and recolored to match. Granting this
 role to a member also adds the Discord role; removing it takes the Discord role away.
 </p>
)}

<button
 className="primary"
 disabled={busy || !detailsDirty}
 onClick={() => onSave({ name, description, color, discordRoleId: discordRoleId || null })}
>
 Save details
</button>

<fieldset className="perms">
 <legend>Permissions</legend>
 {Object.entries(PERMISSION_GROUPS).map(([area, entries]) => (
 <div key={area} className="perm-group">
 <h4>{area}</h4>
 {entries.map(([perm, label]) => (
 <label key={perm} className="perm">
 <input
 type="checkbox"
 checked={perms.has(perm)}
 onChange={() => toggle(perm)}
 disabled={busy}
 />

 {label}
 <code>{perm}</code>

 </label>
))}
 </div>
))}

```

**src/client/pages/Roles.tsx**

```
)}}
 </div>
)}}
 <button
 className="primary"
 disabled={busy || !permsDirty}
 onClick={() => onSavePermissions([...perms])}
 >
 Save permissions
 </button>
</fieldset>

<div className="editor-footer">
 <button
 className="danger"
 disabled={busy || role.isSystem}
 onClick={onDelete}
 title={role.isSystem ? 'System roles cannot be deleted' : 'Delete this role'}
 >
 Delete role
 </button>
 {role.memberCount > 0 && (

 {role.memberCount} member{role.memberCount === 1 ? '' : 's'} hold this role

)}
</div>
</div>
);
}
```

**src/client/pages/Roster.tsx**

```

/**
 * The roster. Everyone sees the leadership tree (the org chart); members who can
 * view the full roster (roster.view) get a toggle to a sortable list of every
 * member, by name or by rank. This is the public face of the roster ? the tree
 * shows to members who can't see the full list.
 */

import { useCallback, useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import { api } from '../lib/api';
import { useSession } from '../lib/session';
import { memberName } from '../../shared/names';
import { memberAvatar } from '../../shared/avatar';
import OrgChartView, { type ChartMember } from '../components/OrgChartView';
import { sanitizeOrgChart, EMPTY_ORG_CHART, type OrgChart } from '../../shared/orgchart';

interface Member {
 id: number;
 discordId: string;
 username: string;
 globalName: string | null;
 displayName: string | null;
 avatar: string | null;
 profileImageUrl: string | null;
 status: string;
 joinedAt: number;
 rankName: string | null;
}

type SortField = 'rank' | 'name';
type SortDir = 'asc' | 'desc';
const PAGE_SIZE = 50;

export default function Roster() {
 const { can } = useSession();
 const canFullList = can('roster.view');
 const [view, setView] = useState<'tree' | 'list'>('tree');

 return (
 <section className="panel">
 <header className="panel-head roster-head">
 <h2>Roster</h2>
 {canFullList && (
 <div className="seg-control">
 <button
 type="button"
 className={view === 'tree' ? 'seg active' : 'seg'}
 onClick={() => setView('tree')}
 >
 Leadership
 </button>
 <button
 type="button"
 className={view === 'list' ? 'seg active' : 'seg'}

```

**src/client/pages/Roster.tsx**

```

 onClick={() => setView('list')}
 >
 All members
 </button>
 </div>
)}
</header>

 {view === 'tree' ? <LeadershipTree canOpenProfiles={canFullList} /> : <MemberList />}
</section>
);
}

function LeadershipTree({ canOpenProfiles }: { canOpenProfiles: boolean }) {
 const [data, setData] = useState<{ chart: OrgChart; members: ChartMember[] } | null>(null);

 useEffect(() => {
 api
 .get<{ chart: OrgChart; members: ChartMember[] }>('/orgchart')
 .then((d) => setData({ chart: sanitizeOrgChart(d.chart), members: d.members }))
 .catch(() => setData({ chart: EMPTY_ORG_CHART, members: [] }));
 }, []);

 if (!data) return <div className="loading">Loading...</div>;
 return <OrgChartView chart={data.chart} members={data.members} linkMembers={canOpenProfiles} />;
}

function MemberList() {
 const [members, setMembers] = useState<Member[]>([]);
 const [total, setTotal] = useState(0);
 const [sort, setSort] = useState<SortField>('rank');
 const [dir, setDir] = useState<SortDir>('desc');
 const [loading, setLoading] = useState(true);
 const [loadingMore, setLoadingMore] = useState(false);

 const loadPage = useCallback(
 async (offset: number, sortField: SortField, sortDir: SortDir) => {
 const { members: page, total } = await api.get<{ members: Member[]; total: number }>(
 `/members?limit=${PAGE_SIZE}&offset=${offset}&sort=${sortField}&dir=${sortDir}`,
);
 setTotal(total);
 setMembers((prev) => (offset === 0 ? page : [...prev, ...page]));
 },
 [],
);

 useEffect(() => {
 setLoading(true);
 loadPage(0, sort, dir).finally(() => setLoading(false));
 }, [sort, dir, loadPage]);

 function sortBy(field: SortField) {
 if (field === sort) {
 setDir((d) => (d === 'asc' ? 'desc' : 'asc'));
 }
 }
}

```

**src/client/pages/Roster.tsx**

```

 } else {
 setSort(field);
 setDir(field === 'rank' ? 'desc' : 'asc');
 }
 }

 const arrow = (field: SortField) => (sort === field ? (dir === 'asc' ? '↑' : '↓') : '');
 const hasMore = members.length < total;

 return (
 <>
 <div className="roster-sort">
 Sort:
 <button type="button" className={sort === 'name' ? 'seg active' : 'seg'} onClick={() =>
sortBy('name')}>
 Name{arrow('name')}
 </button>
 <button type="button" className={sort === 'rank' ? 'seg active' : 'seg'} onClick={() =>
sortBy('rank')}>
 Rank{arrow('rank')}
 </button>

 {hasMore ? `${members.length} of ${total}` : total} member{total === 1 ? '' : 's'}

 </div>

 {loading ? (
 <div className="loading">Loading roster...</div>
) : (
 <>
 <ul className="roster">
 {members.map((m) => (
 <li key={m.id}>
 <Link to={`/${members}/${m.id}`} className="roster-link">
 <img className="avatar" src={memberAvatar(m, 64)} alt="" width={40} height={40}
loading="lazy" />
 <div>
 <div className="name">{memberName(m)}</div>
 <div className="muted small">
 {m.rankName ?? 'Unranked'} ? joined {new Date(m.joinedAt *
1000).toLocaleDateString()}
 </div>
 </div>
 {m.status !== 'active' && {m.status}}
 </Link>

))}

 {hasMore && (
 <div className="roster-more">
 <button
 onClick={() => {
 setLoadingMore(true);

```

**src/client/pages/Roster.tsx**

```
 loadPage(members.length, sort, dir).finally(() => setLoadingMore(false));
 }}
 disabled={loadingMore}
 >
 {loadingMore ? 'Loading...' : `Load ${Math.min(PAGE_SIZE, total - members.length)}`}
 more`}
 </button>
</div>
)}
</>
)}
</>
);
}
```

**src/client/pages/SeoAdmin.tsx**

```

/**
 * SEO & Sharing admin ? the settings-driven meta tags the Worker injects into
 * the served HTML <head> (title/description/Open Graph/Twitter/robots/verification
 * /JSON-LD), so link previews (Discord, Google, X, Facebook) and search results
 * are correct even though the app is a client-side SPA.
 *
 * Structured fields only ? every value is escaped and URL-validated server-side,
 * so an operator can customise robustly without risking script in their own head.
 * The site name/URL live in Identity & Discord; changed there, shown here.
 */

import { useEffect, useState } from 'react';
import { api } from '../lib/api';
import { useAction, Alerts } from '../lib/action';

interface Seo {
 description?: string;
 keywords?: string;
 noindex?: boolean;
 themeColor?: string;
 ogImage?: string;
 ogTitle?: string;
 ogDescription?: string;
 twitterHandle?: string;
 googleVerification?: string;
 bingVerification?: string;
 social?: Record<string, string>;
}

const SOCIALS: { key: string; label: string }[] = [
 { key: 'website', label: 'Website' },
 { key: 'discord', label: 'Discord invite' },
 { key: 'youtube', label: 'YouTube' },
 { key: 'twitch', label: 'Twitch' },
 { key: 'twitter', label: 'X / Twitter' },
 { key: 'instagram', label: 'Instagram' },
 { key: 'facebook', label: 'Facebook' },
];

export default function SeoAdmin() {
 const [seo, setSeo] = useState<Seo>({});
 const [site, setSite] = useState<{ name: string; url: string }>({ name: '', url: '' });
 const [loading, setLoading] = useState(true);
 const [uploading, setUploading] = useState(false);
 const { run, busy, error, notice, warning } = useAction();

 useEffect(() => {
 api
 .get<{ seo: Seo; site: { name: string; url: string } }>('/settings/seo')
 .then(({ seo, site }) => {
 setSeo(seo ?? {});
 setSite(site);
 })
 .finally(() => setLoading(false));
 });

```

**src/client/pages/SeoAdmin.tsx**

```

 }, []));

 const set = (patch: Partial<Seo>) => setSeo((s) => ({ ...s, ...patch }));
 const setSocial = (key: string, val: string) =>
 setSeo((s) => ({ ...s, social: { ...(s.social ?? {}), [key]: val } }));

 const save = () =>
 run(async () => {
 const { seo: saved } = await api.put<{ seo: Seo }>('/settings/seo', { seo });
 setSeo(saved ?? {});
 return 'Saved. Link previews and search tags now use these settings.';
 });

 const uploadOg = async (file: File) => {
 setUploading(true);
 try {
 const res = await api.upload<{ url: string }>('/media/branding', file);
 set({ ogImage: res.url });
 } finally {
 setUploading(false);
 }
 };

 if (loading) return <div className="loading">Loading...</div>;

 return (
 <section className="panel seo-admin">
 <header className="panel-head">
 <div>
 <h2>SEO & Sharing</h2>
 <p className="muted">
 These control the browser-tab title, search-engine listing, and the link preview shown
 when your site is shared (Discord, Google, X, Facebook). The site name is{' '}
 {site.name || 'not set'} ? change it under Identity & Discord.
 </p>
 </div>
 <button className="primary" disabled={busy} onClick={() => void save()}>
 {busy ? 'Saving...' : 'Save'}
 </button>
 </header>

 <Alerts error={error} warning={warning} notice={notice} />

 <fieldset>
 <legend>Search</legend>
 <label>
 Description
 <input
 value={seo.description ?? ''}
 maxLength={300}
 placeholder="One or two sentences about your group ? shown under the title in search
results."
 onChange={(e) => set({ description: e.target.value })}
 disabled={busy}
 />
 </label>
 </fieldset>
 </section>
);

```



**src/client/pages/SeoAdmin.tsx**

```

 />
 </label>
 <label>
 Keywords (comma-separated, optional)
 <input value={seo.keywords ?? ''} maxLength={300} onChange={(e) => set({ keywords:
e.target.value })} disabled={busy} />
 </label>
 <label className="inline-field">
 <input
 type="checkbox"
 checked={!seo.noindex}
 onChange={(e) => set({ noindex: e.target.checked })}
 disabled={busy}
 />
 Hide from search engines (members-only site) ? sets <code>noindex</code>.
 </label>
</fieldset>

<fieldset>
 <legend>Link preview (Open Graph / Twitter)</legend>
 <p className="muted small">
 This is the card people see when your link is posted in Discord or on social media.
 Defaults to the site name and the description above when left blank.
 </p>
 <label>
 Preview title
 <input value={seo.ogTitle ?? ''} maxLength={120} placeholder={site.name} onChange={(e)
=> set({ ogTitle: e.target.value })} disabled={busy} />
 </label>
 <label>
 Preview description
 <input
 value={seo.ogDescription ?? ''}
 maxLength={300}
 placeholder="Falls back to the search description above."
 onChange={(e) => set({ ogDescription: e.target.value })}
 disabled={busy}
 />
 </label>
 <label>
 Preview image
 <div className="seo-og">
 {seo.ogImage && }
 <label className="upload-btn mini">
 {uploading ? 'Uploading...' : seo.ogImage ? 'Change image' : 'Upload image'}
 <input
 type="file"
 accept="image/png,image/jpeg,image/webp"
 hidden
 disabled={busy || uploading}
 onChange={(e) => {
 const f = e.target.files?.[0];
 e.target.value = '';
 if (f) void uploadOg(f);
 }}
 />
 </div>
 </label>

```

## src/client/pages/SeoAdmin.tsx

```

 }}
 />
</label>
{seo.ogImage && (
 <button type="button" className="mini" disabled={busy} onClick={() => set({ ogImage:
'' })}>
 Remove
 </button>
)}
</div>
1200?630 works best. Shown large in Discord/Twitter
previews.
</label>
<label>
 X / Twitter handle (optional)
 <input value={seo.twitterHandle ?? ''} maxLength={40} placeholder="@yourhandle"
onChange={(e) => set({ twitterHandle: e.target.value })} disabled={busy} />
</label>
</fieldset>

<fieldset>
 <legend>Appearance</legend>
 <label>
 Theme color (browser UI tint on mobile / PWA)
 <input
 type="color"
 value={/^#[0-9a-fA-F]{6}$/.test(seo.themeColor ?? '') ? seo.themeColor! : '#0f1115'}
 onChange={(e) => set({ themeColor: e.target.value })}
 disabled={busy}
 />
 </label>
</fieldset>

<fieldset>
 <legend>Verification</legend>
 <p className="muted small">Paste the codes to claim your site in each search console.</p>
 <label>
 Google Search Console
 <input value={seo.googleVerification ?? ''} maxLength={200}
placeholder="google-site-verification value" onChange={(e) => set({ googleVerification:
e.target.value })} disabled={busy} />
 </label>
 <label>
 Bing Webmaster
 <input value={seo.bingVerification ?? ''} maxLength={200} placeholder="msvalidate.01
value" onChange={(e) => set({ bingVerification: e.target.value })} disabled={busy} />
 </label>
</fieldset>

<fieldset>
 <legend>Social profiles (feed rich-result
data)</legend>
 {SOCIALS.map((sc) => (
 <label key={sc.key}>

```

**src/client/pages/SeoAdmin.tsx**

```
 {sc.label}
 <input
 value={seo.social?.[sc.key] ?? ''}
 maxLength={400}
 placeholder="https://..."
 onChange={(e) => setSocial(sc.key, e.target.value)}
 disabled={busy}
 />
 </label>
)}
 </fieldset>

 <div className="news-editor-actions">
 <button className="primary" disabled={busy} onClick={() => void save()}>
 {busy ? 'Saving...' : 'Save'}
 </button>
 </div>
</section>
);
}
```

**src/client/pages/Theme.tsx**

```

/**
 * Theme editor ? the admin surface for the theming pipeline that already runs
 * the whole app (see lib/theme.tsx). Every control edits a CSS custom property
 * and previews it live on :root, so the admin sees the change across the site
 * as they make it; Save persists it for everyone, Discard rolls back to the
 * last saved theme, and leaving the page without saving drops the preview.
 *
 * The token set here is exactly what styles.css consumes ? keep them in sync.
 */

import { useEffect, useRef, useState, type CSSProperties } from 'react';
import { useTheme, DEFAULT_THEME, type ThemeTokens } from '../lib/theme';
import { useAction, Alerts } from '../lib/action';

const TOKEN_KEYS = Object.keys(DEFAULT_THEME);

const COLOR_TOKENS: { key: string; label: string }[] = [
 { key: '--color-bg', label: 'Background' },
 { key: '--color-surface', label: 'Surface / panels' },
 { key: '--color-border', label: 'Borders & dividers' },
 { key: '--color-text', label: 'Text' },
 { key: '--color-muted', label: 'Muted text' },
 { key: '--color-accent', label: 'Accent' },
 { key: '--color-accent-text', label: 'Text on accent' },
];

const FONT_TOKENS: { key: string; label: string }[] = [
 { key: '--font-display', label: 'Headings' },
 { key: '--font-body', label: 'Body' },
];

const NAV_ALIGN: { label: string; value: string }[] = [
 { label: 'Left', value: 'flex-start' },
 { label: 'Center', value: 'center' },
 { label: 'Right', value: 'flex-end' },
];

const FONT_STACKS: { label: string; value: string }[] = [
 { label: 'System', value: 'system-ui, sans-serif' },
 { label: 'Humanist sans', value: '"Segoe UI", Roboto, Helvetica, Arial, sans-serif' },
 { label: 'Serif', value: 'Georgia, "Times New Roman", serif' },
 { label: 'Monospace', value: 'ui-monospace, "SFMono-Regular", Menlo, monospace' },
];

// Complete palettes as starting points. Each is a full token set so applying
// one leaves nothing half-changed. Dark palettes spread the default and swap a
// few tokens; light palettes are spelled out in full (different text/muted).
// A light base every light theme builds on.
const LIGHT_BASE: ThemeTokens = {
 '--color-bg': '#f6f7f9',
 '--color-surface': '#ffffff',
 '--color-border': '#d9dee6',
 '--color-text': '#1b1f27',
 '--color-muted': '#5c6672',

```

**src/client/pages/Theme.tsx**

```
'--color-accent': '#c0392b',
'--color-accent-text': '#ffffff',
'--font-body': 'system-ui, sans-serif',
'--font-display': 'system-ui, sans-serif',
'--radius': '8px',
};

const PRESETS: { name: string; tokens: ThemeTokens }[] = [
 // --- Dark ---
 { name: 'Midnight', tokens: DEFAULT_THEME },
 {
 name: 'Slate',
 tokens: { ...DEFAULT_THEME, '--color-bg': '#0d1117', '--color-surface': '#161b22',
'--color-border': '#30363d', '--color-accent': '#2f81f7' },
 },
 {
 name: 'Forest',
 tokens: { ...DEFAULT_THEME, '--color-bg': '#0e1512', '--color-surface': '#152019',
'--color-border': '#26362c', '--color-accent': '#2f9e5f' },
 },
 {
 name: 'Ember',
 tokens: { ...DEFAULT_THEME, '--color-bg': '#17110d', '--color-surface': '#211812',
'--color-border': '#3a2a1e', '--color-accent': '#e8833a' },
 },
 {
 name: 'Amethyst',
 tokens: { ...DEFAULT_THEME, '--color-bg': '#131019', '--color-surface': '#1c1726',
'--color-border': '#2f2740', '--color-accent': '#a56bf0' },
 },
 {
 name: 'Ocean',
 tokens: { ...DEFAULT_THEME, '--color-bg': '#0b1417', '--color-surface': '#111f24',
'--color-border': '#1f333b', '--color-accent': '#22b8cf', '--color-accent-text': '#04141a' },
 },
 {
 name: 'Rose',
 tokens: { ...DEFAULT_THEME, '--color-bg': '#190f14', '--color-surface': '#24151d',
'--color-border': '#3a2330', '--color-accent': '#ec4899' },
 },
 {
 name: 'Nord',
 tokens: { ...DEFAULT_THEME, '--color-bg': '#2e3440', '--color-surface': '#3b4252',
'--color-border': '#4c566a', '--color-text': '#ecef4', '--color-muted': '#c2cbdc',
'--color-accent': '#88c0d0', '--color-accent-text': '#2e3440' },
 },
 {
 name: 'Mono',
 tokens: { ...DEFAULT_THEME, '--color-bg': '#000000', '--color-surface': '#0d0d0d',
'--color-border': '#2a2a2a', '--color-text': '#ffffff', '--color-muted': '#a0a0a0',
'--color-accent': '#ffffff', '--color-accent-text': '#000000' },
 },
 // --- Light ---
 { name: 'Daylight', tokens: LIGHT_BASE },
```

**src/client/pages/Theme.tsx**

```

{
 name: 'Paper',
 tokens: { ...LIGHT_BASE, '--color-bg': '#f5f1e8', '--color-surface': '#fffdf8',
'--color-border': '#e2dccb', '--color-text': '#2b2620', '--color-muted': '#6b6355',
'--color-accent': '#b45309' },
},
{
 name: 'Sky',
 tokens: { ...LIGHT_BASE, '--color-bg': '#f4f8fc', '--color-surface': '#ffffff',
'--color-border': '#d5e2ef', '--color-text': '#14243a', '--color-muted': '#566a82',
'--color-accent': '#2563eb' },
},
];

function isHex(v: string): boolean {
 return /^#[0-9a-fA-F]{6}$/.test(v);
}

/**
 * A live example of a single token, shown beside its control. Each reads the CSS
 * variables straight off :root (which `preview` keeps in sync with the draft), so
 * every example updates the instant you change any setting.
 */
function TokenExample({ tokenKey }: { tokenKey: string }) {
 const surface: CSSProperties = {
 background: 'var(--color-surface)',
 color: 'var(--color-text)',
 border: '1px solid var(--color-border)',
 };
 switch (tokenKey) {
 case '--color-bg':
 return <div className="tex" style={{ background: 'var(--color-bg)', color:
'var(--color-text)', border: '1px solid var(--color-border)' }}>Page background</div>;
 case '--color-surface':
 return <div className="tex" style={surface}>Panel surface</div>;
 case '--color-border':
 return <div className="tex" style={{ background: 'var(--color-surface)', color:
'var(--color-muted)', border: '2px solid var(--color-border)' }}>Bordered</div>;
 case '--color-text':
 return <div className="tex" style={surface}>The quick brown fox</div>;
 case '--color-muted':
 return <div className="tex" style={{ ...surface, color: 'var(--color-muted)' }}>Secondary
text</div>;
 case '--color-accent':
 case '--color-accent-text':
 return (
 <div className="tex" style={{ background: 'var(--color-surface)', border: '1px solid
var(--color-border)' }}>
 <span className="tex-btn" style={{ background: 'var(--color-accent)', color:
'var(--color-accent-text)' }}>Button
 </div>
);
 case '--font-display':
 return <div className="tex" style={{ ...surface, fontFamily: 'var(--font-display)',

```

**src/client/pages/Theme.tsx**

```

fontSize: '1.15rem', fontWeight: 700 }}>Heading Aa</div>;
 case '--font-body':
 return <div className="tex" style={{ ...surface, fontFamily: 'var(--font-body)' }}>Body text
sample Aa</div>;
 case '--radius':
 return (
 <div className="tex" style={{ background: 'var(--color-surface)', border: '1px solid
var(--color-border)' }}>
 <span className="tex-radius" style={{ background: 'var(--color-accent)', borderRadius:
'var(--radius)' }} />
 </div>
);
 case '--nav-justify':
 return (
 <div className="tex tex-navbar">

 <div className="tex-nav-links" style={{ justifyContent: 'var(--nav-justify)' }}>

 </div>
 </div>
);
 default:
 return null;
}
}

export default function Theme() {
 const { tokens, preview, save } = useTheme();
 // Local working copy. Baseline is the last *saved* theme, held in a ref so
 // the unmount cleanup can revert to it without re-subscribing.
 const [draft, setDraft] = useState<ThemeTokens>(() => ({ ...DEFAULT_THEME, ...tokens }));
 const baselineRef = useRef<ThemeTokens>({ ...DEFAULT_THEME, ...tokens });
 const { run, busy, error, notice } = useAction();

 // Leaving the editor with unsaved edits shouldn't leak a preview into the
 // rest of the app ? restore the last saved theme on unmount.
 useEffect(() => () => preview(baselineRef.current), [preview]);

 const dirty = TOKEN_KEYS.some((k) => draft[k] !== baselineRef.current[k]);

 const update = (key: string, value: string) => {
 const next = { ...draft, [key]: value };
 setDraft(next);
 preview(next);
 };

 const applyTokens = (t: ThemeTokens) => {
 const next = { ...DEFAULT_THEME, ...t };
 setDraft(next);
 preview(next);
 };

 const onSave = () =>
 run(async () => {

```

**src/client/pages/Theme.tsx**

```

 await save(draft);
 baselineRef.current = { ...draft };
 return 'Theme saved ? everyone sees this now.';
 });

const radiusPx = parseInt(draft['--radius'] ?? '0', 10) || 0;

return (
 <section className="panel theme-page">
 <header className="panel-head">
 <h2>Theme</h2>
 <p className="muted">
 Restyle the whole site. Changes preview live as you edit; Save applies them for
everyone.
 </p>
 </header>

 <Alerts error={error} notice={notice} />

 <div className="theme-layout">
 <div className="theme-main">
 <fieldset className="theme-group">
 <legend>Colors</legend>
 {COLOR_TOKENS.map(({ key, label }) => (
 <div key={key} className="theme-row">
 <div className="theme-row-main">
 <label className="theme-row-label">
 {label}
 <code>{key}</code>
 </label>
 <div className="theme-row-controls">
 <input
 type="color"
 value={isHex(draft[key] ?? '') ? draft[key]! : '#000000'}
 disabled={busy}
 onChange={(e) => update(key, e.target.value)}
 aria-label={` ${label} color`}
 />
 <input
 type="text"
 className="theme-hex"
 value={draft[key] ?? ''}
 disabled={busy}
 spellCheck={false}
 onChange={(e) => update(key, e.target.value)}
 />
 </div>
 </div>
 <div className="theme-row-example">
 <TokenExample tokenKey={key} />
 </div>
 </div>
))}
 </fieldset>

```



## src/client/pages/Theme.tsx

```

<fieldset className="theme-group">
 <legend>Typography</legend>
 {FONT_TOKENS.map(({ key, label }) => {
 const known = FONT_STACKS.some((f) => f.value === draft[key]);
 return (
 <div key={key} className="theme-row">
 <div className="theme-row-main">
 <label className="theme-row-label">
 {label}
 <code>{key}</code>
 </label>
 <div className="theme-row-controls">
 <select
 value={known ? draft[key] : '__custom'}
 disabled={busy}
 onChange={(e) => e.target.value !== '__custom' && update(key,
e.target.value)}
 >
 {FONT_STACKS.map((f) => (
 <option key={f.value} value={f.value}>
 {f.label}
 </option>
))}
 <option value="__custom">Custom...</option>
 </select>
 <input
 type="text"
 className="theme-font"
 value={draft[key] ?? ''}
 disabled={busy}
 spellCheck={false}
 style={{ fontFamily: draft[key] }}
 onChange={(e) => update(key, e.target.value)}
 />
 </div>
 </div>
 <div className="theme-row-example">
 <TokenExample tokenKey={key} />
 </div>
 </div>
);
 })}
</fieldset>

<fieldset className="theme-group">
 <legend>Shape</legend>
 <div className="theme-row">
 <div className="theme-row-main">
 <label className="theme-row-label">
 Corner radius
 <code>--radius</code>
 </label>
 <div className="theme-row-controls">

```

**src/client/pages/Theme.tsx**

```

 <input
 type="range"
 min={0}
 max={24}
 value={radiusPx}
 disabled={busy}
 onChange={(e) => update('--radius', `${e.target.value}px`)}
 />
 {draft['--radius']}

```

**src/client/pages/Theme.tsx**

```

 <button disabled={busy || !dirty} onClick={() => applyTokens(baselineRef.current)}>
 Discard changes
 </button>
 <button
 className="danger"
 disabled={busy}
 onClick={() => applyTokens(DEFAULT_THEME)}
 title="Load the built-in defaults (then Save to keep them)"
 >
 Load defaults
 </button>
 </div>
</div>

{/* Presets as a vertical menu on the right. */}
<aside className="theme-rail" aria-label="Start from a preset">
 <div className="theme-rail-title">Start from</div>
 <div className="theme-presets-vertical">
 {PRESETS.map((p) => (
 <button
 key={p.name}
 className="preset"
 disabled={busy}
 // A palette shouldn't reset a layout choice, so keep the current menu alignment.
 onClick={() => applyTokens({ ...p.tokens, '--nav-justify': draft['--nav-justify']
?? 'flex-start' })}}
 >
 <span className="preset-swatch" style={{ background: p.tokens['--color-accent'] }}
 {p.name}
 </button>
)}}
 </div>
 </aside>
</div>
</section>
);
}

```

**src/client/pages/WarRecords.tsx**

```

/**
 * War-record administration ? the catalog of awardable "items of pride".
 *
 * A war record is a name, an optional image/description, and an optional game
 * it belongs to. Handing one to a specific member happens on that member's
 * page, not here. Structurally a sibling of the medals catalog, so it reuses
 * the medal styling.
 */

import { useEffect, useState, type ChangeEvent } from 'react';
import { api } from '../lib/api';
import { useAction, Alerts } from '../lib/action';

interface WarRecord {
 id: number;
 name: string;
 description: string | null;
 imageUrl: string | null;
 gameId: number | null;
 gameName: string | null;
 sortOrder: number;
 awardCount: number;
}

interface GameOption {
 id: number;
 name: string;
}

export default function WarRecords() {
 const [records, setRecords] = useState<WarRecord[]>([]);
 const [games, setGames] = useState<GameOption[]>([]);
 const [selectedId, setSelectedId] = useState<number | null>(null);
 const [newName, setNewName] = useState('');
 const [loading, setLoading] = useState(true);

 async function load() {
 const { warRecords } = await api.get<{ warRecords: WarRecord[] }>('/warrecords');
 setRecords(warRecords);
 }

 const { run, busy, error, notice, warning } = useAction(load);

 useEffect(() => {
 Promise.all([
 load(),
 api.get<{ games: GameOption[] }>('/games').then(({ games }) => setGames(games)).catch(() =>
setGames([])),
]).finally(() => setLoading(false));
 }, []);

 const addRecord = () =>
 run(async () => {
 if (!newName.trim()) return;

```

**src/client/pages/WarRecords.tsx**

```

 const { warRecord } = await api.post<{ warRecord: WarRecord }>('/warrecords', { name:
newName.trim() });
 setNewName('');
 setSelectedId(warRecord.id);
 return `Created "${warRecord.name}"`;
 });

 const selected = records.find((r) => r.id === selectedId) ?? null;

 if (loading) return <div className="loading">Loading war records...</div>;

 return (
 <section className="panel medals-page">
 <header className="panel-head">
 <h2>War Records</h2>
 <p className="muted">
 {records.length} record{records.length === 1 ? '' : 's'}. Award them to members from
their
 profile.
 </p>
 </header>

 <Alerts error={error} warning={warning} notice={notice} />

 <div className="add-row">
 <input
 value={newName}
 placeholder="New war record name"
 onChange={(e) => setNewName(e.target.value)}
 onKeyDown={(e) => e.key === 'Enter' && void addRecord()}
 disabled={busy}
 />
 <button onClick={() => void addRecord()} disabled={busy || !newName.trim()}>
 Add war record
 </button>
 </div>

 <div className="roles-layout">
 <ul className="medal-list">
 {records.map((rec) => (
 <li key={rec.id}>
 <button
 className={rec.id === selectedId ? 'medal-chip active' : 'medal-chip'}
 onClick={() => setSelectedId(rec.id)}
 >

 {rec.imageUrl ? (

) : (
 ?
)}

 {rec.name}
 {rec.gameName && {rec.gameName}}

)}
)}

 </div>
 </section>
);

```

**src/client/pages/WarRecords.tsx**

```

 {rec.awardCount}
 </button>

)))
 {records.length === 0 && <li className="muted small">No war records yet.}

{selected ? (
 <RecordEditor
 key={selected.id}
 record={selected}
 games={games}
 busy={busy}
 onSave={(patch) =>
 run(async () => {
 await api.patch(`/warrecords/${selected.id}`, patch);
 return 'Saved.';
 })
 }
 onUpload={async (file) => {
 const { url } = await api.upload<{ url: string }>('/media/warrecords', file);
 return url;
 }}
 onDelete={() =>
 run(async () => {
 await api.del(`/warrecords/${selected.id}`);
 setSelectedId(null);
 return 'War record deleted.';
 })
 }
 </>
) : (
 <div className="role-editor empty-editor">Select a war record to edit it.</div>
)}
</div>
</section>
);
}

```

```

function RecordEditor({
 record,
 games,
 busy,
 onSave,
 onUpload,
 onDelete,
}): {
 record: WarRecord;
 games: GameOption[];
 busy: boolean;
 onSave: (patch: { name: string; description: string; imageUrl: string | null; gameId: number |
null }) => void;
 onUpload: (file: File) => Promise<string>;
 onDelete: () => void;
}

```

**src/client/pages/WarRecords.tsx**

```

})) {
 const [name, setName] = useState(record.name);
 const [description, setDescription] = useState(record.description ?? '');
 const [imageUrl, setImageUrl] = useState(record.imageUrl);
 const [gameId, setGameId] = useState<number | null>(record.gameId);
 const [uploading, setUploading] = useState(false);
 const [uploadError, setUploadError] = useState<string | null>(null);

 const dirty =
 name !== record.name ||
 description !== (record.description ?? '') ||
 imageUrl !== record.imageUrl ||
 gameId !== record.gameId;

 async function pickFile(e: ChangeEvent<HTMLInputElement>) {
 const file = e.target.files?.[0];
 e.target.value = '';
 if (!file) return;
 setUploadError(null);
 setUploading(true);
 try {
 const url = await onUpload(file);
 setImageUrl(url);
 } catch (err) {
 setUploadError(err instanceof Error ? err.message : 'Upload failed.');
```

```
 } finally {
```

```
 setUploading(false);
```

```
 }
```

```
 }
```

```
 return (
```

```
 <div className="role-editor medal-editor">
```

```
 <div className="medal-editor-head">
```

```

```

```
 {imageUrl ? (
```

```

```

```
) : (
```

```
 ?
```

```
)}
```

```

```

```
 <div className="medal-image-controls">
```

```
 <label className="upload-btn">
```

```
 {uploading ? 'Uploading...' : imageUrl ? 'Replace image' : 'Upload image'}
```

```
 <input
```

```
 type="file"
```

```
 accept="image/png,image/jpeg,image/gif,image/webp"
```

```
 onChange={(e) => void pickFile(e)}
```

```
 disabled={busy || uploading}
```

```
 hidden
```

```
 />
```

```
 </label>
```

```
 {imageUrl && (
```

```
 <button className="mini" disabled={busy || uploading} onClick={() =>
```

```
setImageUrl(null)}>
```

**src/client/pages/WarRecords.tsx**

```

 Remove image
 </button>
)}
 PNG, JPEG, GIF, or WebP ? up to 1 MB.
 {uploadError && {uploadError}}
 </div>
</div>

<label>
 Name
 <input value={name} onChange={(e) => setName(e.target.value)} disabled={busy} />
</label>

<label>
 Description
 <input
 value={description}
 onChange={(e) => setDescription(e.target.value)}
 placeholder="What this war record is for"
 disabled={busy}
 />
</label>

<label>
 Game
 <select
 value={gameId ?? ''}
 onChange={(e) => setGameId(e.target.value ? Number(e.target.value) : null)}
 disabled={busy}
 >
 <option value="">? Clan-wide (no game) ?</option>
 {games.map((g) => (
 <option key={g.id} value={g.id}>
 {g.name}
 </option>
))}
 </select>
</label>

<button
 className="primary"
 disabled={busy || uploading || !dirty || !name.trim()}
 onClick={() => onSave({ name: name.trim(), description, imageUrl, gameId })}
>
 Save
</button>

<div className="editor-footer">
 <button className="danger" disabled={busy} onClick={onDelete} title="Delete this war
record">
 Delete war record
 </button>
 {record.awardCount > 0 && (


```



**src/client/pages/WarRecords.tsx**

```
 Held by {record.awardCount} member{record.awardCount === 1 ? '' : 's'} ? deleting
removes it
 from all of them

)}
</div>
</div>
);
}
```

**src/client/public/icon.svg**

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 512 512" role="img" aria-label="ClanTek">
 <rect width="512" height="512" rx="104" fill="#0f1115"/>
 <!-- Shield kept within the central safe zone so the icon survives maskable cropping. -->
 <path d="M256 120l112 42v98c0 78-50 124-112 142-62-18-112-64-112-142v-98z" fill="#c0392b"/>
 <text x="256" y="296" font-family="system-ui, -apple-system, Segoe UI, Roboto, sans-serif"
 font-size="132" font-weight="700" fill="ffffff" text-anchor="middle">CT</text>
</svg>
```

**src/client/public/sw.js**

```

/**
 * ClanTek service worker ? a deliberately conservative first pass.
 *
 * Goals: make the app installable and give it a usable offline shell without
 * ever risking a stale or broken UI.
 * - Navigations (HTML): network-first, so a deployed update always wins when
 * online; only when offline do we fall back to the cached shell.
 * - Static assets (hashed, immutable): cache-first with a background refresh.
 * - /api/* and /media/*: never touched ? always live from the network, so
 * auth, data, and uploads are never served from a stale cache.
 *
 * Bump CACHE to invalidate everything on the next activate.
 */

const CACHE = 'clantek-v1';
const SHELL = '/index.html';

self.addEventListener('install', (event) => {
 event.waitUntil(
 caches
 .open(CACHE)
 .then((c) => c.add(SHELL))
 .catch(() => {})
 .then(() => self.skipWaiting()),
);
});

self.addEventListener('activate', (event) => {
 event.waitUntil(
 caches
 .keys()
 .then((keys) => Promise.all(keys.filter((k) => k !== CACHE).map((k) => caches.delete(k))))
 .then(() => self.clients.claim()),
);
});

self.addEventListener('fetch', (event) => {
 const { request } = event;
 if (request.method !== 'GET') return;

 const url = new URL(request.url);
 if (url.origin !== self.location.origin) return;
 // Dynamic + authenticated surfaces are always live.
 if (url.pathname.startsWith('/api/') || url.pathname.startsWith('/media/')) return;

 // HTML navigations: network-first so updates land immediately when online.
 if (request.mode === 'navigate') {
 event.respondWith(
 fetch(request)
 .then((res) => {
 const copy = res.clone();
 caches.open(CACHE).then((c) => c.put(SHELL, copy)).catch(() => {});
 return res;
 })
);
 }
});

```

**src/client/public/sw.js**

```
 .catch(() => caches.match(SHELL)),
);
 return;
}

// Everything else (hashed JS/CSS/images): cache-first, refresh in background.
event.respondWith(
 caches.match(request).then((cached) => {
 const network = fetch(request)
 .then((res) => {
 if (res.ok) {
 const copy = res.clone();
 caches.open(CACHE).then((c) => c.put(request, copy)).catch(() => {});
 }
 return res;
 })
 .catch(() => cached);
 return cached || network;
 }),
);
});
```

**src/client/styles.css**

```
/* Every value here reads from a custom property set by ThemeProvider, so the
 admin theme editor restyles the whole app without touching this file. */

*,
*::before,
*::after {
 box-sizing: border-box;
}

body {
 margin: 0;
 background: var(--color-bg);
 color: var(--color-text);
 font-family: var(--font-body);
 line-height: 1.5;
}

button,
input {
 font: inherit;
 color: inherit;
}

.app {
 min-height: 100vh;
 display: flex;
 flex-direction: column;
}

/* Top bar ----- */

.topbar {
 display: flex;
 align-items: center;
 gap: 1.5rem;
 padding: 0.75rem 1.25rem;
 background: var(--color-surface);
 border-bottom: 1px solid var(--color-border);
 flex-wrap: wrap;
 /* Sticky so the logo can shrink into the bar as the page scrolls. */
 position: sticky;
 top: 0;
 z-index: 100;
}

.brand {
 display: flex;
 align-items: center;
 color: var(--color-text);
 text-decoration: none;
}

.brand-name {
 font-family: var(--font-display);
```

**src/client/styles.css**

```
font-weight: 700;
font-size: 1.1rem;
letter-spacing: 0.02em;
}

/* --- Header logo: oversized + overhanging at the top, docked once scrolled --- */
.topbar.has-logo .brand {
 position: relative;
 /* Reserve the logo's *full* (expanded) footprint so it never covers the menu ?
 the nav sits just past the widest the logo ever gets. The docked logo simply
 left-aligns within this reserved column. */
 width: var(--logo-fp, 120px);
 align-self: stretch;
}

.brand-logo {
 position: absolute;
 left: 0;
 top: 50%;
 transform: translateY(-50%);
 height: var(--logo-collapsed, 38px);
 width: auto;
 max-width: min(48vw, 460px);
 z-index: 120;
 transition: height 0.22s ease, top 0.22s ease, transform 0.22s ease, filter 0.22s ease;
}

/* At the top of the page: grow and overhang the bar, floating over the menu. */
.topbar.has-logo:not(.scrolled) .brand-logo {
 height: var(--logo-expanded, 88px);
 top: 0.4rem;
 transform: none;
 filter: drop-shadow(0 6px 16px rgba(0, 0, 0, 0.35));
}

/* On narrow screens the nav wraps and space is tight, so keep the logo docked
 and reserve only the docked footprint (the big overhang is a desktop touch). */
@media (max-width: 720px) {
 .topbar.has-logo .brand {
 width: var(--logo-fp-min, 44px);
 }
 .topbar.has-logo:not(.scrolled) .brand-logo {
 height: var(--logo-collapsed, 38px);
 top: 50%;
 transform: translateY(-50%);
 filter: none;
 }
}

.nav {
 display: flex;
 gap: 0.25rem;
 flex: 1;
 /* Menu alignment is a theme choice (left / center / right of the logo). */
}
```

**src/client/styles.css**

```
 justify-content: var(--nav-justify, flex-start);
 }

.nav a {
 color: var(--color-muted);
 text-decoration: none;
 padding: 0.4rem 0.75rem;
 border-radius: var(--radius);
}

.nav a:hover {
 color: var(--color-text);
 background: color-mix(in srgb, var(--color-border) 60%, transparent);
}

.nav a.active {
 color: var(--color-accent-text);
 background: var(--color-accent);
}

/* Admin gets an accent so authorized users can find it at a glance. */
.nav a.nav-admin {
 color: var(--color-accent);
 font-weight: 600;
}

.nav a.nav-admin.active {
 color: var(--color-accent-text);
}

/* Hamburger ? hidden on desktop, shown when the nav collapses on mobile. */
.nav-toggle {
 display: none;
 background: transparent;
 border: 1px solid var(--color-border);
 color: var(--color-text);
 font-size: 1.1rem;
 line-height: 1;
 padding: 0.35rem 0.6rem;
 border-radius: var(--radius);
}

@media (max-width: 720px) {
 .nav-toggle {
 display: inline-flex;
 align-items: center;
 }

 /* Full-width dropdown below the bar; hidden until the hamburger opens it. */
 .nav {
 order: 3;
 flex: 0 0 100%;
 flex-direction: column;
 gap: 0.15rem;
 }
}
```

**src/client/styles.css**

```
 display: none;
 margin-top: 0.5rem;
 /* Column dropdown: keep links top-aligned regardless of the desktop choice. */
 justify-content: flex-start;
 }

 .nav.open {
 display: flex;
 }

 .nav a {
 padding: 0.6rem 0.75rem;
 }
}

.account {
 margin-left: auto;
 display: flex;
 align-items: center;
 gap: 0.75rem;
}

.who {
 display: inline-flex;
 align-items: center;
 gap: 0.5rem;
 font-size: 0.9rem;
}

.rank-chip,
.status-chip {
 font-size: 0.72rem;
 text-transform: uppercase;
 letter-spacing: 0.05em;
 padding: 0.15rem 0.45rem;
 border-radius: 999px;
 background: var(--color-border);
 color: var(--color-muted);
}

.god-chip {
 color: var(--color-accent);
}

/* Content ----- */

.content {
 flex: 1;
 width: min(1280px, 100%);
 margin-inline: auto;
 padding: 1.5rem 1.5rem 3rem;
}

.panel {
```



**src/client/styles.css**

```
background: var(--color-surface);
border: 1px solid var(--color-border);
border-radius: var(--radius);
padding: 1.25rem;
}

.panel-head {
 margin-bottom: 1rem;
}

.panel-head h2 {
 margin: 0;
 font-family: var(--font-display);
}

.muted {
 color: var(--color-muted);
 margin: 0.25rem 0 0 0;
}

.small {
 font-size: 0.85rem;
}

.loading,
.empty {
 padding: 3rem 1rem;
 text-align: center;
 color: var(--color-muted);
}

.alert {
 background: color-mix(in srgb, var(--color-accent) 15%, transparent);
 border: 1px solid var(--color-accent);
 color: var(--color-text);
 padding: 0.6rem 0.9rem;
 border-radius: var(--radius);
 margin-bottom: 1rem;
}

.notice {
 background: color-mix(in srgb, var(--color-muted) 18%, transparent);
 border: 1px solid var(--color-border);
 color: var(--color-text);
 padding: 0.6rem 0.9rem;
 border-radius: var(--radius);
 margin-bottom: 1rem;
}

/* Saved locally, but a downstream push (usually Discord) was refused. */
.warn-alert {
 --amber: #e6a23c;
 background: color-mix(in srgb, var(--amber) 16%, transparent);
 border: 1px solid var(--amber);
```

**src/client/styles.css**

```
color: var(--color-text);
padding: 0.6rem 0.9rem;
border-radius: var(--radius);
margin-bottom: 1rem;
}

.warn-alert::before {
 content: '? ';
 color: var(--amber);
}

/* Controls ----- */

button {
 background: var(--color-border);
 border: 1px solid transparent;
 border-radius: var(--radius);
 padding: 0.4rem 0.75rem;
 cursor: pointer;
}

button:hover:not(:disabled) {
 border-color: var(--color-muted);
}

button:disabled {
 opacity: 0.45;
 cursor: not-allowed;
}

button.danger:hover:not(:disabled) {
 background: var(--color-accent);
 color: var(--color-accent-text);
}

input[type='text'],
input[type='password'],
input:not([type]) {
 background: var(--color-bg);
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 padding: 0.4rem 0.6rem;
 width: 100%;
}

/* Themed defaults for the remaining form controls, so any select or date/number
 field reads correctly on the dark surface even when it isn't nested in a
 .role-editor label. Specific components can still override. */
select,
input[type='date'],
input[type='number'],
input[type='datetime-local'] {
 background: var(--color-surface);
 border: 1px solid var(--color-border);
```

**src/client/styles.css**

```
border-radius: var(--radius);
padding: 0.4rem 0.6rem;
color: var(--color-text);
}
/* The native calendar/spinner glyphs are near-invisible on a dark field in some
 browsers; brighten them to match the text. */
input[type='date']::-webkit-calendar-picker-indicator {
 filter: invert(0.8);
}

input:focus-visible,
button:focus-visible,
a:focus-visible {
 outline: 2px solid var(--color-accent);
 outline-offset: 2px;
}

.add-row {
 display: flex;
 gap: 0.5rem;
 margin-bottom: 1rem;
}

.add-row input {
 max-width: 260px;
}

/* Tables and lists ----- */

.grid {
 width: 100%;
 border-collapse: collapse;
}

.grid th {
 text-align: left;
 font-size: 0.75rem;
 text-transform: uppercase;
 letter-spacing: 0.06em;
 color: var(--color-muted);
 padding: 0.5rem;
 border-bottom: 1px solid var(--color-border);
}

.grid td {
 padding: 0.4rem 0.5rem;
 border-bottom: 1px solid var(--color-border);
 vertical-align: middle;
}

.reorder {
 display: flex;
 gap: 0.2rem;
}
```

**src/client/styles.css**

```
.reorder button {
 padding: 0.15rem 0.4rem;
}

.roster {
 list-style: none;
 margin: 0;
 padding: 0;
 display: grid;
 gap: 0.5rem;
}

.roster li {
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
}

.roster-link {
 display: flex;
 align-items: center;
 gap: 0.75rem;
 padding: 0.5rem;
 text-decoration: none;
 color: inherit;
 border-radius: var(--radius);
}

.roster-link:hover {
 background: color-mix(in srgb, var(--color-border) 50%, transparent);
}

.roster img {
 border-radius: 50%;
}

.roster-more {
 display: flex;
 justify-content: center;
 margin-top: 1rem;
}

.roster .name {
 font-weight: 600;
}

.roster .status-chip {
 margin-left: auto;
}

/* Auth ----- */

.login {
 max-width: 380px;
```

**src/client/styles.css**

```
margin: 5rem auto;
text-align: center;
}

.login h1 {
 font-family: var(--font-display);
 margin-bottom: 0.25rem;
}

.discord-btn {
 display: inline-block;
 margin-top: 1.25rem;
 background: #5865f2;
 color: #fff;
 text-decoration: none;
 padding: 0.5rem 1rem;
 border-radius: var(--radius);
 font-weight: 600;
}

.discord-btn.large {
 padding: 0.7rem 1.5rem;
}

.discord-btn:hover {
 filter: brightness(1.1);
}

/* Roles admin ----- */

.roles-layout {
 display: grid;
 grid-template-columns: minmax(180px, 240px) 1fr;
 gap: 1.25rem;
 align-items: start;
}

@media (max-width: 640px) {
 .roles-layout {
 grid-template-columns: 1fr;
 }
}

.role-list {
 list-style: none;
 margin: 0;
 padding: 0;
 display: grid;
 gap: 0.3rem;
}

.role-chip {
 width: 100%;
 display: flex;
```

**src/client/styles.css**

```
 align-items: center;
 gap: 0.5rem;
 padding: 0.45rem 0.6rem;
 background: var(--color-bg);
 border: 1px solid var(--color-border);
 text-align: left;
}

.role-chip.active {
 border-color: var(--color-accent);
}

.role-chip .dot {
 width: 10px;
 height: 10px;
 border-radius: 50%;
 flex: none;
}

.role-chip .role-name {
 flex: 1;
 overflow: hidden;
 text-overflow: ellipsis;
 white-space: nowrap;
}

.role-chip .tag {
 font-size: 0.65rem;
 text-transform: uppercase;
 letter-spacing: 0.05em;
 color: var(--color-muted);
 border: 1px solid var(--color-border);
 border-radius: 999px;
 padding: 0 0.35rem;
}

.role-chip .count {
 font-size: 0.8rem;
 color: var(--color-muted);
 min-width: 1.4em;
 text-align: right;
}

.role-editor {
 display: flex;
 flex-direction: column;
 gap: 0.75rem;
 background: var(--color-bg);
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 padding: 1rem;
}

.empty-editor {
```

**src/client/styles.css**

```
 color: var(--color-muted);
 text-align: center;
 padding: 2.5rem 1rem;
}

.role-editor label {
 display: flex;
 flex-direction: column;
 gap: 0.25rem;
 font-size: 0.85rem;
 color: var(--color-muted);
}

.role-editor label input:not([type='color']):not([type='checkbox']),
.role-editor label select {
 background: var(--color-surface);
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 padding: 0.4rem 0.6rem;
 color: var(--color-text);
 width: 100%;
}

.field-row {
 display: flex;
 gap: 0.75rem;
}

.field-row > label:first-child {
 flex: 1;
}

.color-field input[type='color'] {
 width: 3rem;
 height: 2.2rem;
 padding: 0;
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 background: none;
}

button.primary {
 align-self: flex-start;
 background: var(--color-accent);
 color: var(--color-accent-text);
}

button.primary:disabled {
 background: var(--color-border);
 color: var(--color-muted);
}

.warn {
 color: var(--color-accent);
}
```

**src/client/styles.css**

```
}

.perms {
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 padding: 0.75rem;
 margin: 0;
 display: flex;
 flex-direction: column;
 gap: 0.75rem;
}

.perms legend {
 font-size: 0.8rem;
 text-transform: uppercase;
 letter-spacing: 0.06em;
 color: var(--color-muted);
 padding: 0 0.4rem;
}

.perm-group h4 {
 margin: 0 0 0.3rem;
 font-size: 0.8rem;
 text-transform: capitalize;
 color: var(--color-muted);
}

.perm {
 flex-direction: row !important;
 align-items: center;
 gap: 0.5rem !important;
 color: var(--color-text) !important;
 font-size: 0.9rem !important;
 padding: 0.15rem 0;
}

.perm code {
 margin-left: 0.5rem;
 font-size: 0.75rem;
 color: var(--color-muted);
}

.editor-footer {
 display: flex;
 align-items: center;
 gap: 0.75rem;
 justify-content: space-between;
 border-top: 1px solid var(--color-border);
 padding-top: 0.75rem;
}

/* Rank -> roles editor ----- */

.rank-roles-editor {
```



**src/client/styles.css**

```
margin-top: 1rem;
background: var(--color-bg);
border: 1px solid var(--color-border);
border-radius: var(--radius);
padding: 1rem;
display: flex;
flex-direction: column;
gap: 0.6rem;
}

.rre-head {
display: flex;
align-items: center;
justify-content: space-between;
}

.rre-head h3 {
margin: 0;
font-family: var(--font-display);
font-size: 1rem;
}

.rre-roles {
display: flex;
flex-wrap: wrap;
gap: 0.5rem 1.25rem;
}

.rre-role {
display: inline-flex;
align-items: center;
gap: 0.4rem;
font-size: 0.9rem;
}

.rre-role .dot {
width: 10px;
height: 10px;
border-radius: 50%;
}

button.active {
border-color: var(--color-accent);
color: var(--color-accent-text);
background: var(--color-accent);
}

/* Member detail ----- */

.back {
margin-bottom: 1rem;
background: none;
border: none;
color: var(--color-muted);
```

**src/client/styles.css**

```
padding: 0;
}

.back:hover {
 color: var(--color-text);
}

.member-head {
 display: flex;
 align-items: center;
 gap: 1rem;
 margin-bottom: 1.25rem;
}

.member-head img {
 border-radius: 50%;
}

.member-head h2 {
 margin: 0 0 0.3rem;
 font-family: var(--font-display);
}

.member-sub {
 display: flex;
 align-items: center;
 gap: 0.5rem;
 flex-wrap: wrap;
 font-size: 0.85rem;
}

.status-chip.status-retired,
.status-chip.status-banned {
 color: var(--color-accent);
 border-color: var(--color-accent);
}

.member-grid {
 display: grid;
 grid-template-columns: 1fr minmax(200px, 260px);
 gap: 1.25rem;
 align-items: start;
}

@media (max-width: 640px) {
 .member-grid {
 grid-template-columns: 1fr;
 }
}

.member-main {
 display: flex;
 flex-direction: column;
 gap: 1.25rem;
}
```

**src/client/styles.css**

```
}

.block h3,
.member-admin h3 {
 margin: 0 0 0.5rem;
 font-size: 0.8rem;
 text-transform: uppercase;
 letter-spacing: 0.06em;
 color: var(--color-muted);
}

.bio {
 white-space: pre-wrap;
 margin: 0;
}

.bio-editor {
 display: flex;
 flex-direction: column;
 gap: 0.4rem;
}

.bio-editor .field {
 display: flex;
 flex-direction: column;
 gap: 0.25rem;
 font-size: 0.85rem;
 color: var(--color-muted);
}

.bio-editor textarea,
.bio-editor .field input {
 width: 100%;
 background: var(--color-bg);
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 color: var(--color-text);
 padding: 0.5rem 0.6rem;
 resize: vertical;
}

.bio-actions {
 display: flex;
 gap: 0.5rem;
 margin-top: 0.5rem;
}

.member-roles {
 list-style: none;
 margin: 0;
 padding: 0;
 display: flex;
 flex-direction: column;
 gap: 0.35rem;
}
```

**src/client/styles.css**

```
}

.member-roles li {
 display: flex;
 align-items: center;
 gap: 0.5rem;
 font-size: 0.9rem;
}

.member-roles .dot {
 width: 10px;
 height: 10px;
 border-radius: 50%;
}

.member-roles .tag {
 font-size: 0.65rem;
 text-transform: uppercase;
 letter-spacing: 0.04em;
 color: var(--color-muted);
 border: 1px solid var(--color-border);
 border-radius: 999px;
 padding: 0 0.35rem;
}

.mini {
 padding: 0 0.4rem;
 font-size: 0.8rem;
 line-height: 1.4;
}

.grant-row {
 display: flex;
 align-items: center;
 gap: 0.6rem;
 margin-top: 0.6rem;
 flex-wrap: wrap;
}

.grant-row select,
.member-admin select {
 background: var(--color-bg);
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 color: var(--color-text);
 padding: 0.35rem 0.5rem;
}

.member-medals {
 list-style: none;
 margin: 0;
 padding: 0;
 display: flex;
 flex-wrap: wrap;
}
```

**src/client/styles.css**

```
 gap: 0.5rem;
}

.member-medals li {
 display: inline-flex;
 align-items: center;
 gap: 0.4rem;
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 padding: 0.25rem 0.5rem;
 font-size: 0.85rem;
}

.member-admin {
 background: var(--color-bg);
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 padding: 1rem;
 display: flex;
 flex-direction: column;
 gap: 1rem;
}

.admin-block {
 display: flex;
 flex-direction: column;
 gap: 0.4rem;
}

.admin-label {
 font-size: 0.8rem;
 color: var(--color-muted);
}

.admin-actions {
 display: flex;
 gap: 0.5rem;
 flex-wrap: wrap;
}

.current-rank {
 font-family: var(--font-display);
 font-size: 1.05rem;
 font-weight: 600;
}

.confirm-row {
 display: flex;
 flex-direction: column;
 gap: 0.5rem;
 background: color-mix(in srgb, var(--color-accent) 12%, transparent);
 border: 1px solid var(--color-accent);
 border-radius: var(--radius);
 padding: 0.6rem;
```

**src/client/styles.css**

```
}

/* ----- *
 * Medals admin + member-page medal controls
 * ----- */

.medal-list {
 list-style: none;
 margin: 0;
 padding: 0;
 display: grid;
 gap: 0.3rem;
}

.medal-chip {
 width: 100%;
 display: flex;
 align-items: center;
 gap: 0.5rem;
 padding: 0.4rem 0.6rem;
 background: var(--color-bg);
 border: 1px solid var(--color-border);
 text-align: left;
}

.medal-chip.active {
 border-color: var(--color-accent);
}

.medal-chip .medal-name {
 flex: 1;
 overflow: hidden;
 text-overflow: ellipsis;
 white-space: nowrap;
}

.medal-chip .tag,
.member-medals .tag {
 font-size: 0.65rem;
 text-transform: uppercase;
 letter-spacing: 0.05em;
 color: var(--color-muted);
 border: 1px solid var(--color-border);
 border-radius: 999px;
 padding: 0 0.35rem;
}

.medal-chip .count {
 font-size: 0.8rem;
 color: var(--color-muted);
 min-width: 1.4em;
 text-align: right;
}
```

**src/client/styles.css**

```
/* The little insignia thumbnail, on chips and in the editor. */
.medal-thumb {
 flex: none;
 display: inline-flex;
 align-items: center;
 justify-content: center;
 width: 28px;
 height: 28px;
}

.medal-thumb.large {
 width: 64px;
 height: 64px;
}

.medal-thumb img {
 object-fit: contain;
 max-width: 100%;
 max-height: 100%;
}

.medal-thumb-empty {
 display: inline-flex;
 align-items: center;
 justify-content: center;
 width: 100%;
 height: 100%;
 border: 1px dashed var(--color-border);
 border-radius: var(--radius);
 color: var(--color-muted);
}

.medal-thumb-empty.small {
 width: 28px;
 height: 28px;
}

.medal-editor-head {
 display: flex;
 gap: 1rem;
 align-items: center;
}

.medal-image-controls {
 display: flex;
 flex-direction: column;
 gap: 0.35rem;
 align-items: flex-start;
}

/* A file input styled as a button by hiding the input and dressing the label. */
.upload-btn {
 display: inline-block;
 cursor: pointer;
```

**src/client/styles.css**

```
padding: 0.4rem 0.75rem;
background: var(--color-bg);
border: 1px solid var(--color-border);
border-radius: var(--radius);
color: var(--color-text);
font-size: 0.9rem;
}

.upload-btn:hover {
border-color: var(--color-accent);
}

.tenure {
border: 1px solid var(--color-border);
border-radius: var(--radius);
padding: 0.75rem;
display: flex;
flex-direction: column;
gap: 0.5rem;
}

.announce-event {
align-items: flex-start;
}

/* Events ----- */

.event-list {
list-style: none;
margin: 0;
padding: 0;
display: flex;
flex-direction: column;
gap: 0.75rem;
}

.event-card {
padding: 1rem;
border: 1px solid var(--color-border);
border-radius: var(--radius);
background: var(--color-surface);
}

.event-card h3 {
margin: 0.15rem 0 0.35rem;
font-family: var(--font-display);
}

.event-when {
font-size: 0.8rem;
font-weight: 600;
color: var(--color-accent);
}

.event-meta {
margin: 0;
}
```



**src/client/styles.css**

```
.event-desc {
 margin: 0.6rem 0 0;
 white-space: pre-wrap;
}

.event-actions {
 display: flex;
 flex-wrap: wrap;
 align-items: center;
 gap: 0.5rem;
 margin-top: 0.75rem;
}

.event-form {
 margin-bottom: 1rem;
}

.event-form textarea {
 width: 100%;
 resize: vertical;
}

.tenure legend {
 font-size: 0.85rem;
 color: var(--color-muted);
 padding: 0 0.35rem;
}

.tenure-row {
 display: flex;
 align-items: center;
 gap: 0.5rem;
}

.tenure-row input {
 width: 6rem;
}

.tenure-presets {
 display: flex;
 flex-wrap: wrap;
 gap: 0.35rem;
}

.preset {
 font-size: 0.8rem;
 padding: 0.2rem 0.6rem;
 background: var(--color-bg);
 border: 1px solid var(--color-border);
 border-radius: 999px;
 color: var(--color-text);
 cursor: pointer;
}

.preset.active {
 border-color: var(--color-accent);
```

**src/client/styles.css**

```
 color: var(--color-accent);
}

/* The member-page award picker: medal select + citation + button. */
.award-row {
 display: flex;
 flex-wrap: wrap;
 gap: 0.5rem;
 margin-top: 0.6rem;
}

.award-row select {
 flex: none;
}

.award-row input {
 flex: 1;
 min-width: 8rem;
 background: var(--color-bg);
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 color: var(--color-text);
 padding: 0.35rem 0.5rem;
}

/* Give held medals room for a citation line under the name. */
.member-medals li {
 align-items: flex-start;
}

.medal-label {
 display: flex;
 flex-direction: column;
 line-height: 1.25;
}

.medal-citation {
 font-size: 0.72rem;
 color: var(--color-muted);
}

/* ----- *
 * Avatars ? profile images (member override) and rank insignia.
 * ----- */

/* Round profile avatars wherever they appear (roster img already rounds via
 its own rule; this covers the topbar and member header). */
.avatar {
 border-radius: 50%;
 object-fit: cover;
 flex: none;
}

.avatar.sm {
```

**src/client/styles.css**

```
width: 28px;
height: 28px;
}

.avatar.lg {
width: 72px;
height: 72px;
}

/* Member header avatar + its self-service upload controls. */
.member-avatar {
display: flex;
flex-direction: column;
align-items: center;
gap: 0.4rem;
}

.avatar-controls {
display: flex;
flex-wrap: wrap;
align-items: center;
justify-content: center;
gap: 0.35rem;
}

/* Compact variant of the file-picker button, used in the rank table and under
the member avatar. Higher specificity than .upload-btn so its padding wins. */
.upload-btn.mini {
padding: 0.1rem 0.45rem;
font-size: 0.78rem;
}

/* Rank insignia cell in the ranks admin table. */
.rank-image-cell {
display: flex;
align-items: center;
gap: 0.35rem;
}

.rank-thumb {
flex: none;
display: inline-flex;
align-items: center;
justify-content: center;
width: 28px;
height: 28px;
}

.rank-thumb img {
object-fit: contain;
max-width: 100%;
max-height: 100%;
}
```

**src/client/styles.css**

```
.rank-thumb-empty {
 display: inline-flex;
 align-items: center;
 justify-content: center;
 width: 28px;
 height: 28px;
 border: 1px dashed var(--color-border);
 border-radius: var(--radius);
 color: var(--color-muted);
}

/* ----- */
/* Account menu (top-right dropdown) */
/* ----- */

.account-menu {
 position: relative;
}

.account-trigger {
 display: inline-flex;
 align-items: center;
 gap: 0.5rem;
 padding: 0.25rem 0.5rem;
 background: transparent;
 border: 1px solid transparent;
 border-radius: var(--radius);
 color: inherit;
 cursor: pointer;
 font-size: 0.9rem;
}

.account-trigger:hover {
 border-color: var(--color-border);
 background: color-mix(in srgb, var(--color-border) 40%, transparent);
}

.account-name {
 font-weight: 600;
}

.account-trigger .caret {
 color: var(--color-muted);
 flex: none;
}

.account-dropdown {
 position: absolute;
 top: calc(100% + 0.35rem);
 right: 0;
 min-width: 180px;
 z-index: 50;
 display: flex;
 flex-direction: column;
```

**src/client/styles.css**

```
padding: 0.35rem;
background: var(--color-surface);
border: 1px solid var(--color-border);
border-radius: var(--radius);
box-shadow: 0 8px 24px rgba(0, 0, 0, 0.25);
}

.account-dropdown .menu-item {
display: block;
width: 100%;
text-align: left;
padding: 0.5rem 0.6rem;
background: transparent;
border: none;
border-radius: var(--radius);
color: var(--color-text);
text-decoration: none;
font-size: 0.9rem;
cursor: pointer;
}

.account-dropdown .menu-item:hover:not(:disabled) {
background: color-mix(in srgb, var(--color-border) 50%, transparent);
}

.account-dropdown .menu-item:disabled {
color: var(--color-muted);
cursor: default;
}

.account-dropdown .menu-sep {
height: 1px;
margin: 0.35rem 0.25rem;
background: var(--color-border);
}

/* ----- *
 * Admin panel shell (left sidebar + content)
 * ----- */

/* Admin header: section toggle + breadcrumb, so the panel reads as a place
 within the site rather than a separate app you dropped into. */
.admin-header {
display: flex;
align-items: center;
gap: 0.75rem;
margin-bottom: 1.25rem;
flex-wrap: wrap;
}

.admin-nav-toggle {
display: inline-flex;
align-items: center;
gap: 0.4rem;
```

**src/client/styles.css**

```
padding: 0.4rem 0.7rem;
background: var(--color-surface);
border: 1px solid var(--color-border);
border-radius: var(--radius);
color: var(--color-text);
cursor: pointer;
font: inherit;
font-size: 0.9rem;
}
.admin-nav-toggle:hover {
border-color: var(--color-accent);
}

.admin-crumbs {
display: flex;
align-items: center;
gap: 0.4rem;
flex-wrap: wrap;
font-size: 0.9rem;
color: var(--color-muted);
}
.admin-crumb {
color: var(--color-muted);
text-decoration: none;
}
a.admin-crumb:hover {
color: var(--color-text);
}
.admin-crumb-sep {
opacity: 0.6;
}
.admin-crumb.current {
color: var(--color-text);
font-weight: 600;
display: inline-flex;
align-items: center;
gap: 0.35rem;
}
.admin-crumb-icon {
color: var(--color-accent, currentColor);
}

.admin-shell {
display: grid;
grid-template-columns: 1fr;
gap: 1.25rem;
align-items: start;
}

/* The rail is only a column when open; otherwise the tool takes full width. */
.admin-shell.nav-open {
grid-template-columns: minmax(190px, 230px) 1fr;
}
```

**src/client/styles.css**

```
@media (max-width: 720px) {
 .admin-shell.nav-open {
 grid-template-columns: 1fr;
 }
}

.admin-nav {
 display: none;
 flex-direction: column;
 gap: 0.15rem;
 padding: 0.5rem;
 background: var(--color-surface);
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 /* Stay in view while a long tool scrolls; clear the sticky top bar. */
 position: sticky;
 top: 4.5rem;
 max-height: calc(100vh - 5.5rem);
 overflow: auto;
}
.admin-shell.nav-open .admin-nav {
 display: flex;
}
@media (max-width: 720px) {
 .admin-nav {
 position: static;
 max-height: none;
 }
}

/* Each group stacks its items vertically. */
.admin-nav-group {
 display: flex;
 flex-direction: column;
 gap: 0.1rem;
}
.admin-nav-group + .admin-nav-group {
 margin-top: 0.75rem;
 padding-top: 0.5rem;
 border-top: 1px solid var(--color-border);
}

.admin-nav-title {
 font-size: 0.75rem;
 text-transform: uppercase;
 letter-spacing: 0.06em;
 color: var(--color-muted);
 padding: 0.35rem 0.6rem;
}

/* Recently-viewed: a horizontal strip above the content, not in the sidebar. */
.admin-recentbar {
 display: flex;
 align-items: center;
```

**src/client/styles.css**

```
 flex-wrap: wrap;
 gap: 0.4rem;
 margin-bottom: 1rem;
 padding-bottom: 0.75rem;
 border-bottom: 1px solid var(--color-border);
}
.admin-recentbar-title {
 font-size: 0.68rem;
 text-transform: uppercase;
 letter-spacing: 0.05em;
 color: var(--color-muted);
 margin-right: 0.15rem;
}
.admin-recentbar-link {
 padding: 0.2rem 0.6rem;
 border: 1px solid var(--color-border);
 border-radius: 999px;
 color: var(--color-muted);
 text-decoration: none;
 font-size: 0.82rem;
 max-width: 220px;
 white-space: nowrap;
 overflow: hidden;
 text-overflow: ellipsis;
}
.admin-recentbar-link:hover {
 color: var(--color-text);
 border-color: var(--color-accent);
}

/* Tabs inside a multi-tool admin item (e.g. Ranks & Roles). */
.admin-tabs {
 display: flex;
 gap: 0.25rem;
 margin-bottom: 1rem;
 border-bottom: 1px solid var(--color-border);
 flex-wrap: wrap;
}
.admin-tab {
 padding: 0.5rem 0.9rem;
 color: var(--color-muted);
 text-decoration: none;
 border-bottom: 2px solid transparent;
 margin-bottom: -1px;
 font-size: 0.95rem;
}
.admin-tab:hover {
 color: var(--color-text);
}
.admin-tab.active {
 color: var(--color-accent);
 border-bottom-color: var(--color-accent);
 font-weight: 600;
}
```



**src/client/styles.css**

```
.admin-nav-link {
 display: flex;
 align-items: center;
 gap: 0.55rem;
 padding: 0.5rem 0.6rem;
 border-radius: var(--radius);
 color: var(--color-text);
 text-decoration: none;
 font-size: 0.95rem;
}

.admin-nav-icon {
 flex: 0 0 auto;
 color: var(--color-muted);
}

.admin-nav-link:hover {
 background: color-mix(in srgb, var(--color-border) 50%, transparent);
}

.admin-nav-link.active {
 background: var(--color-accent);
 color: #fff;
}

.admin-nav-link.active .admin-nav-icon {
 color: #fff;
}

.admin-content {
 min-width: 0;
}

/* ----- *
 * Member profile admin toolbar (condensed, was the right-side panel)
 * ----- */

.member-admin-bar {
 display: flex;
 flex-wrap: wrap;
 align-items: center;
 gap: 1rem 1.5rem;
 margin-bottom: 1.25rem;
 padding: 0.6rem 0.9rem;
 background: var(--color-bg);
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
}

.mab-group {
 display: flex;
 flex-wrap: wrap;
 align-items: center;
```

**src/client/styles.css**

```
 gap: 0.5rem;
}

.mab-group + .mab-group {
 padding-left: 1.5rem;
 border-left: 1px solid var(--color-border);
}

@media (max-width: 720px) {
 .mab-group + .mab-group {
 padding-left: 0;
 border-left: none;
 }
}

.mab-label {
 font-size: 0.75rem;
 text-transform: uppercase;
 letter-spacing: 0.06em;
 color: var(--color-muted);
}

.mab-rank-name {
 font-weight: 600;
}

.mab-buttons {
 display: inline-flex;
 gap: 0.35rem;
}

.member-admin-bar select {
 background: var(--color-surface);
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 color: var(--color-text);
 padding: 0.3rem 0.5rem;
}

/* Manual-role pills in the toolbar. */
.role-pill {
 display: inline-flex;
 align-items: center;
 gap: 0.35rem;
 padding: 0.2rem 0.5rem;
 background: var(--color-surface);
 border: 1px solid var(--color-border);
 border-radius: 999px;
 font-size: 0.85rem;
}

/* ----- */
/* Theme editor
/* ----- */
```

**src/client/styles.css**

```
.theme-presets {
 display: flex;
 flex-wrap: wrap;
 align-items: center;
 gap: 0.5rem;
 margin-bottom: 1.25rem;
}

.theme-presets .preset {
 display: inline-flex;
 align-items: center;
 gap: 0.4rem;
}

.preset-swatch {
 width: 14px;
 height: 14px;
 border-radius: 50%;
 border: 1px solid rgba(255, 255, 255, 0.25);
}

.theme-grid {
 display: grid;
 grid-template-columns: 1fr minmax(240px, 320px);
 gap: 1.5rem;
 align-items: start;
}

@media (max-width: 760px) {
 .theme-grid {
 grid-template-columns: 1fr;
 }
}

.theme-controls {
 display: flex;
 flex-direction: column;
 gap: 1rem;
 min-width: 0;
}

.theme-group {
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 padding: 0.75rem 1rem 1rem;
}

.theme-group legend {
 padding: 0 0.4rem;
 font-size: 0.75rem;
 text-transform: uppercase;
 letter-spacing: 0.06em;
 color: var(--color-muted);
}
```

**src/client/styles.css**

```
}

.theme-row {
 display: grid;
 grid-template-columns: 1fr minmax(130px, 210px);
 align-items: center;
 gap: 1rem;
 padding: 0.5rem 0;
}

/* Control side of a row: label on the left, its input(s) on the right. */
.theme-row-main {
 display: flex;
 align-items: center;
 justify-content: space-between;
 gap: 1rem;
 min-width: 0;
}

/* Live example of just this setting, shown to its right. */
.theme-row-example {
 min-width: 0;
}
.theme-row-example .tex {
 display: flex;
 align-items: center;
 justify-content: center;
 min-height: 40px;
 padding: 0.4rem 0.6rem;
 border-radius: var(--radius);
 font-size: 0.82rem;
 text-align: center;
 overflow: hidden;
 white-space: nowrap;
 text-overflow: ellipsis;
}
.theme-row-example .tex-btn {
 padding: 0.3rem 0.7rem;
 border-radius: var(--radius);
 font-size: 0.8rem;
 font-weight: 600;
}
.theme-row-example .tex-radius {
 display: block;
 width: 40px;
 height: 24px;
}

/* Mini header mock for the menu-alignment example. */
.theme-row-example .tex-navbar {
 display: flex;
 align-items: center;
 gap: 0.4rem;
 background: var(--color-surface);
 border: 1px solid var(--color-border);
}
```

**src/client/styles.css**

```
}
.theme-row-example .tex-nav-brand {
 width: 18px;
 height: 18px;
 border-radius: 4px;
 background: var(--color-accent);
 flex: 0 0 auto;
}
.theme-row-example .tex-nav-links {
 display: flex;
 gap: 4px;
 flex: 1;
}
.theme-row-example .tex-nav-links b {
 width: 14px;
 height: 5px;
 border-radius: 3px;
 background: var(--color-muted);
}

/* Two-column page: settings on the left, preset rail on the right. */
.theme-layout {
 display: grid;
 grid-template-columns: 1fr minmax(150px, 190px);
 gap: 1.5rem;
 align-items: start;
}
@media (max-width: 720px) {
 .theme-layout {
 grid-template-columns: 1fr;
 }
}
.theme-main {
 display: flex;
 flex-direction: column;
 gap: 1rem;
 min-width: 0;
}
.theme-rail {
 position: sticky;
 top: 1rem;
}
.theme-rail-title {
 font-size: 0.75rem;
 text-transform: uppercase;
 letter-spacing: 0.06em;
 color: var(--color-muted);
 margin-bottom: 0.5rem;
}
.theme-presets-vertical {
 display: flex;
 flex-direction: column;
 gap: 0.35rem;
}
```

**src/client/styles.css**

```
.theme-presets-vertical .preset {
 display: flex;
 align-items: center;
 gap: 0.5rem;
 width: 100%;
 border-radius: var(--radius);
 padding: 0.4rem 0.6rem;
 text-align: left;
}

.theme-row + .theme-row {
 border-top: 1px solid color-mix(in srgb, var(--color-border) 60%, transparent);
}

.theme-row-label {
 display: flex;
 flex-direction: column;
 gap: 0.15rem;
 font-size: 0.9rem;
}

.theme-row-label code {
 font-size: 0.72rem;
 color: var(--color-muted);
}

.theme-row-controls {
 display: flex;
 align-items: center;
 gap: 0.5rem;
}

.theme-row-controls input[type='color'] {
 width: 34px;
 height: 30px;
 padding: 0;
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 background: none;
 cursor: pointer;
}

.theme-hex {
 width: 92px;
 font-family: ui-monospace, Menlo, monospace;
 text-align: center;
}

.theme-font {
 width: 190px;
}

.theme-radius-value {
 min-width: 3ch;
```

**src/client/styles.css**

```
 color: var(--color-muted);
 font-size: 0.85rem;
}

/* Live preview card */
.theme-preview {
 position: sticky;
 top: 1rem;
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 padding: 1rem;
 background: var(--color-surface);
}

.theme-preview h3 {
 margin: 0 0 0.75rem;
 font-size: 0.75rem;
 text-transform: uppercase;
 letter-spacing: 0.06em;
 color: var(--color-muted);
}

.tp-card {
 background: var(--color-bg);
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 padding: 1rem;
}

.tp-title {
 font-family: var(--font-display);
 font-size: 1.15rem;
 margin-bottom: 0.35rem;
}

.tp-body {
 margin: 0 0 0.25rem;
}

.tp-muted {
 margin: 0 0 0.75rem;
 color: var(--color-muted);
 font-size: 0.9rem;
}

.tp-actions {
 display: flex;
 align-items: center;
 gap: 0.5rem;
 margin-bottom: 0.75rem;
}

.tp-input {
 width: 100%;
```

**src/client/styles.css**

```
background: var(--color-surface);
border: 1px solid var(--color-border);
border-radius: var(--radius);
padding: 0.4rem 0.6rem;
}

.theme-actions {
 display: flex;
 flex-wrap: wrap;
 gap: 0.75rem;
 margin-top: 1.5rem;
 padding-top: 1rem;
 border-top: 1px solid var(--color-border);
}

/* ----- */
/* News
/* ----- */

.news-head {
 display: flex;
 align-items: flex-start;
 justify-content: space-between;
 gap: 1rem;
}

.btn-link {
 color: var(--color-accent);
 text-decoration: none;
 font-size: 0.9rem;
 white-space: nowrap;
}

.btn-link:hover {
 text-decoration: underline;
}

/* Inline external help links (e.g. "where to get this" links in admin panels).
 Themed to the accent so they read as links even inside muted helper text. */
.ext-link {
 color: var(--color-accent);
 text-decoration: none;
 font-weight: 500;
}

.ext-link:hover {
 text-decoration: underline;
}

.tag.pin {
 background: color-mix(in srgb, var(--color-accent) 18%, transparent);
 color: var(--color-accent);
 border-color: color-mix(in srgb, var(--color-accent) 40%, transparent);
}

/* Feed */
```



**src/client/styles.css**

```
.news-feed {
 list-style: none;
 margin: 0;
 padding: 0;
 display: flex;
 flex-direction: column;
 gap: 0.75rem;
}

.news-item-link {
 display: block;
 padding: 1rem;
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 text-decoration: none;
 color: inherit;
 background: var(--color-surface);
}

.news-item-link:hover {
 border-color: var(--color-accent);
}

.news-item-head {
 display: flex;
 align-items: center;
 gap: 0.5rem;
}

.news-item-head h3 {
 margin: 0;
 font-family: var(--font-display);
}

.news-excerpt {
 margin: 0.4rem 0 0.5rem;
 color: var(--color-text);
}

.news-meta {
 margin: 0;
}

/* Single post */
.news-post-head {
 margin: 0.75rem 0 1.25rem;
}

.news-post-head h1 {
 margin: 0.35rem 0 0.4rem;
 font-family: var(--font-display);
}

/* Rendered post body ? style the sanitized HTML from the editor. */
.news-body {
 line-height: 1.65;
}
```

**src/client/styles.css**

```
.news-body h1,
.news-body h2,
.news-body h3 {
 font-family: var(--font-display);
 margin: 1.4rem 0 0.6rem;
 line-height: 1.25;
}
.news-body h1 { font-size: 1.5rem; }
.news-body h2 { font-size: 1.25rem; }
.news-body h3 { font-size: 1.1rem; }
.news-body p { margin: 0 0 0.9rem; }
.news-body ul,
.news-body ol { margin: 0 0 0.9rem 1.4rem; }
.news-body li { margin: 0.2rem 0; }
.news-body a {
 color: var(--color-accent);
 text-decoration: underline;
}
.news-body blockquote {
 margin: 0 0 0.9rem;
 padding: 0.3rem 0 0.3rem 1rem;
 border-left: 3px solid var(--color-border);
 color: var(--color-muted);
}
.news-body code {
 font-family: ui-monospace, Menlo, monospace;
 font-size: 0.9em;
 background: color-mix(in srgb, var(--color-muted) 18%, transparent);
 padding: 0.1rem 0.3rem;
 border-radius: 4px;
}
.news-body pre {
 background: var(--color-bg);
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 padding: 0.75rem 1rem;
 overflow-x: auto;
}
.news-body pre code {
 background: none;
 padding: 0;
}

/* Status chips for news */
.status-chip.status-draft {
 background: color-mix(in srgb, var(--color-muted) 20%, transparent);
 color: var(--color-muted);
}
.status-chip.status-published {
 background: color-mix(in srgb, #2f9e5f 22%, transparent);
 color: #7ee2a8;
}
.status-chip.status-archived {
 background: color-mix(in srgb, var(--color-muted) 14%, transparent);
```

**src/client/styles.css**

```
 color: var(--color-muted);
 text-decoration: line-through;
}

/* Admin list + editor */
.news-admin-list {
 list-style: none;
 margin: 0;
 padding: 0;
 display: flex;
 flex-direction: column;
 gap: 0.25rem;
 min-width: 220px;
}
.news-admin-item {
 display: flex;
 align-items: center;
 justify-content: space-between;
 gap: 0.5rem;
 width: 100%;
 text-align: left;
 background: transparent;
 border: 1px solid transparent;
}
.news-admin-item:hover {
 border-color: var(--color-border);
}
.news-admin-item.active {
 border-color: var(--color-accent);
}
.news-admin-title {
 display: inline-flex;
 align-items: center;
 gap: 0.35rem;
 overflow: hidden;
 text-overflow: ellipsis;
 white-space: nowrap;
}

.news-editor .news-editor-head {
 display: flex;
 align-items: center;
 justify-content: space-between;
 margin-bottom: 0.5rem;
}
.rte-label {
 display: block;
 margin: 0.5rem 0 0.25rem;
}
.news-editor-actions {
 display: flex;
 flex-wrap: wrap;
 gap: 0.5rem;
 margin-top: 0.9rem;
}
```

**src/client/styles.css**

```
}
label.check {
 display: flex;
 flex-direction: row;
 align-items: center;
 gap: 0.5rem;
}

/* ----- *
 * Rich-text editor (TipTap)
 * ----- */

.rte {
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 overflow: hidden;
}
.rte-toolbar {
 display: flex;
 flex-wrap: wrap;
 align-items: center;
 gap: 0.15rem;
 padding: 0.35rem;
 background: var(--color-bg);
 border-bottom: 1px solid var(--color-border);
}
.rte-tool {
 min-width: 30px;
 padding: 0.2rem 0.4rem;
 font-size: 0.85rem;
 background: transparent;
 border: 1px solid transparent;
}
.rte-tool:hover {
 border-color: var(--color-border);
}
.rte-tool.active {
 background: var(--color-accent);
 color: var(--color-accent-text);
}
.rte-sep {
 width: 1px;
 align-self: stretch;
 margin: 0.1rem 0.25rem;
 background: var(--color-border);
}
.rte-content .ProseMirror {
 min-height: 220px;
 padding: 0.75rem 1rem;
 outline: none;
 line-height: 1.6;
}
.rte-content .ProseMirror:focus {
 outline: none;
```

**src/client/styles.css**

```
}
.rte-content .ProseMirror > *:first-child {
 margin-top: 0;
}
.rte-content .ProseMirror p {
 margin: 0 0 0.7rem;
}
.rte-content .ProseMirror h1,
.rte-content .ProseMirror h2,
.rte-content .ProseMirror h3 {
 font-family: var(--font-display);
 margin: 1rem 0 0.5rem;
}
.rte-content .ProseMirror ul,
.rte-content .ProseMirror ol {
 margin: 0 0 0.7rem 1.4rem;
}
.rte-content .ProseMirror blockquote {
 border-left: 3px solid var(--color-border);
 padding-left: 1rem;
 color: var(--color-muted);
 margin: 0 0 0.7rem;
}
.rte-content .ProseMirror pre {
 background: var(--color-bg);
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 padding: 0.6rem 0.9rem;
 overflow-x: auto;
}
.rte-content .ProseMirror a {
 color: var(--color-accent);
 text-decoration: underline;
}

/* ----- *
 * Usage dashboard (top of the admin panel)
 * ----- */

.usage-bar {
 margin-bottom: 1.25rem;
 padding: 0.85rem 1rem;
 background: var(--color-surface);
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
}
.usage-bar.usage-loading {
 color: var(--color-muted);
 font-size: 0.85rem;
}

.usage-gauges {
 display: grid;
 grid-template-columns: repeat(auto-fit, minmax(170px, 1fr));
```

**src/client/styles.css**

```
 gap: 1rem 1.25rem;
 align-items: end;
}

.usage-gauge-head {
 display: flex;
 align-items: baseline;
 justify-content: space-between;
 gap: 0.5rem;
 margin-bottom: 0.35rem;
}

.usage-label {
 font-size: 0.8rem;
 font-weight: 600;
}

.usage-sub {
 font-weight: 400;
 color: var(--color-muted);
}

.usage-value {
 font-size: 0.78rem;
 white-space: nowrap;
}

.usage-track {
 height: 7px;
 border-radius: 999px;
 background: color-mix(in srgb, var(--color-border) 70%, transparent);
 overflow: hidden;
}

.usage-fill {
 height: 100%;
 border-radius: 999px;
 transition: width 0.4s ease, background 0.4s ease;
}

.usage-pct {
 margin-top: 0.2rem;
 font-size: 0.72rem;
 font-variant-numeric: tabular-nums;
}

.usage-connect {
 grid-column: 1 / -1;
 font-size: 0.82rem;
}

.usage-connect summary {
 display: flex;
 align-items: center;
 gap: 0.3rem;
 cursor: pointer;
 color: var(--color-accent);
 list-style: none;
}

.usage-connect summary::-webkit-details-marker {
```

**src/client/styles.css**

```
 display: none;
}
.disclosure-caret {
 flex: none;
}
.usage-connect ol {
 margin: 0.5rem 0 0 1.1rem;
 line-height: 1.5;
}
.usage-connect code {
 font-size: 0.78rem;
 background: color-mix(in srgb, var(--color-muted) 18%, transparent);
 padding: 0.05rem 0.3rem;
 border-radius: 4px;
}

.usage-error {
 margin-top: 0.6rem;
 color: #e6a23c;
}

/* Activity log ----- */

.audit-filters {
 display: flex;
 flex-wrap: wrap;
 gap: 0.4rem;
 margin-bottom: 1rem;
}
.chip {
 padding: 0.25rem 0.7rem;
 border-radius: 999px;
 border: 1px solid var(--color-border);
 background: transparent;
 color: var(--color-muted);
 cursor: pointer;
 font-size: 0.85rem;
}
.chip.active {
 background: var(--color-accent);
 color: var(--color-accent-text);
 border-color: var(--color-accent);
}

.audit-list {
 list-style: none;
 margin: 0;
 padding: 0;
 display: flex;
 flex-direction: column;
}
.audit-row {
 display: grid;
 grid-template-columns: minmax(9rem, 12rem) 1fr auto;
```

**src/client/styles.css**

```
gap: 0.75rem;
align-items: start;
padding: 0.6rem 0.25rem;
border-bottom: 1px solid var(--color-border);
}
.audit-actor {
 display: flex;
 align-items: center;
 gap: 0.45rem;
 min-width: 0;
}
.audit-actor a {
 color: var(--color-text);
 text-decoration: none;
 overflow: hidden;
 text-overflow: ellipsis;
 white-space: nowrap;
}
.audit-actor a:hover {
 text-decoration: underline;
}
.audit-body {
 min-width: 0;
}
.audit-action {
 font-weight: 600;
}
.audit-action.negative {
 color: var(--color-accent);
}
.audit-detail {
 margin-left: 0.5rem;
 color: var(--color-muted);
}
.audit-reason {
 margin-top: 0.2rem;
 font-style: italic;
 color: var(--color-muted);
 font-size: 0.9rem;
}
.audit-meta {
 display: flex;
 align-items: center;
 gap: 0.4rem;
 white-space: nowrap;
}
.audit-list .tag {
 font-size: 0.65rem;
 text-transform: uppercase;
 letter-spacing: 0.05em;
 color: var(--color-muted);
 border: 1px solid var(--color-border);
 border-radius: 999px;
 padding: 0 0.35rem;
```



**src/client/styles.css**

```
}

.center {
 display: flex;
 justify-content: center;
 margin-top: 1rem;
}

@media (max-width: 640px) {
 .audit-row {
 grid-template-columns: 1fr;
 gap: 0.3rem;
 }
}

/* Reason prompt (member admin negative actions) ----- */

.reason-prompt {
 display: flex;
 flex-wrap: wrap;
 align-items: center;
 gap: 0.4rem;
 padding: 0.5rem;
 margin: 0.4rem 0;
 border: 1px solid var(--color-accent);
 border-radius: var(--radius);
 background: color-mix(in srgb, var(--color-accent) 8%, transparent);
}
.reason-prompt input {
 flex: 1;
 min-width: 12rem;
}
.reason-prompt .reason-label {
 font-size: 0.85rem;
 font-weight: 600;
}

.req {
 color: var(--color-accent);
 font-weight: 700;
}
input[aria-invalid='true'] {
 border-color: var(--color-accent);
}

/* Events v2 ----- */

.event-banner {
 display: block;
 width: 100%;
 max-height: 220px;
 object-fit: cover;
 border-radius: var(--radius);
 margin-bottom: 0.6rem;
}
```

**src/client/styles.css**

```
}
.event-banner.preview {
 max-height: 140px;
}

.signup-controls {
 display: flex;
 flex-wrap: wrap;
 align-items: center;
 gap: 0.4rem;
 margin: 0.6rem 0;
}
.signup-label {
 font-size: 0.85rem;
 font-weight: 600;
 color: var(--color-muted);
}
.signup-count {
 margin-left: 0.35rem;
 font-size: 0.75rem;
 opacity: 0.8;
}
.chip.withdraw {
 border-color: var(--color-accent);
 color: var(--color-accent);
}

.attendees {
 margin: 0.4rem 0;
}
.link-btn {
 background: none;
 border: none;
 padding: 0.15rem 0;
 color: var(--color-muted);
 cursor: pointer;
}
.link-btn:hover {
 color: var(--color-text);
 border: none;
}
.attendee-groups {
 display: flex;
 flex-wrap: wrap;
 gap: 1rem;
 margin-top: 0.5rem;
}
.attendee-group {
 min-width: 8rem;
}
.attendee-role {
 font-weight: 600;
 font-size: 0.9rem;
 margin-bottom: 0.3rem;
```

**src/client/styles.css**

```
}
.attendee-members {
 list-style: none;
 margin: 0;
 padding: 0;
 display: flex;
 flex-direction: column;
 gap: 0.25rem;
}
.attendee-members li {
 display: flex;
 align-items: center;
 gap: 0.4rem;
 font-size: 0.9rem;
}

/* Event form: role editor + image field */
.field-label {
 font-weight: 600;
 font-size: 0.9rem;
 margin-bottom: 0.3rem;
}
.event-roles-editor {
 margin: 0.5rem 0;
}
.role-row {
 display: flex;
 gap: 0.4rem;
 align-items: center;
 margin-bottom: 0.4rem;
}
.role-row-head {
 margin-bottom: 0.2rem;
 padding: 0 0.1rem;
}
.role-col-icon {
 width: 4rem;
 text-align: center;
 flex: none;
}
.role-col-name {
 flex: 1;
 min-width: 0;
}
.role-col-cap {
 width: 8rem;
 flex: none;
}
.role-col-rm {
 width: 2rem;
 flex: none;
 text-align: center;
}
```

**src/client/styles.css**

```
/* Date + time picker */
.datetime-field {
 display: flex;
 flex-direction: column;
 gap: 0.25rem;
 flex: 1;
 min-width: 0;
}
.datetime-inputs {
 display: flex;
 gap: 0.4rem;
}
.datetime-inputs input[type='date'] {
 flex: 1;
 min-width: 7rem;
}
.datetime-inputs select {
 flex: none;
 min-width: 4.5rem;
}

.event-image-field {
 margin: 0.5rem 0;
}

/* ----- *
 * Page layout system ? responsive module grid (drag-and-drop pages)
 *
 * A page is a stack of rows; each row is a 12-unit grid of columns. A column's
 * width comes from its --span custom property. Below 860px every column takes
 * the full width, so the page becomes one readable stack on phones.
 * ----- */

.page-rows {
 display: flex;
 flex-direction: column;
 gap: 1.5rem;
}

.page-grid {
 display: grid;
 grid-template-columns: repeat(12, minmax(0, 1fr));
 gap: 1.5rem;
 align-items: start;
}

.page-col {
 grid-column: span var(--span, 12);
 min-width: 0;
 display: flex;
 flex-direction: column;
 gap: 1.5rem;
}
```

**src/client/styles.css**

```
@media (max-width: 860px) {
 .page-col {
 grid-column: 1 / -1;
 }
}

/* Module cards reuse .panel; the head becomes a title + action row. */
.module .panel-head {
 display: flex;
 align-items: baseline;
 justify-content: space-between;
 gap: 0.75rem;
}

.module-heading {
 margin: 0;
 font-family: var(--font-display);
}

/* Roster mini-module */
.roster-mini {
 list-style: none;
 margin: 0;
 padding: 0;
 display: flex;
 flex-direction: column;
 gap: 0.15rem;
}

.roster-mini-item {
 display: flex;
 align-items: center;
 gap: 0.6rem;
 padding: 0.35rem 0.5rem;
 border-radius: var(--radius);
 color: var(--color-text);
 text-decoration: none;
}

.roster-mini-item:hover {
 background: color-mix(in srgb, var(--color-border) 55%, transparent);
}

.avatar-sm {
 width: 32px;
 height: 32px;
 border-radius: 50%;
 object-fit: cover;
 flex: none;
 background: var(--color-border);
}

.roster-mini-name {
 font-weight: 600;
}
```

**src/client/styles.css**

```
}

.roster-mini-item .small {
 margin-left: auto;
}

/* Events mini-module */
.events-mini {
 list-style: none;
 margin: 0;
 padding: 0;
 display: flex;
 flex-direction: column;
 gap: 0.6rem;
}

.events-mini-item {
 display: flex;
 gap: 0.75rem;
 align-items: center;
}

.events-mini-date {
 display: flex;
 flex-direction: column;
 align-items: center;
 justify-content: center;
 width: 3rem;
 height: 3rem;
 flex: none;
 border-radius: var(--radius);
 background: color-mix(in srgb, var(--color-accent) 16%, transparent);
 border: 1px solid color-mix(in srgb, var(--color-accent) 38%, transparent);
}

.events-mini-mon {
 font-size: 0.65rem;
 text-transform: uppercase;
 letter-spacing: 0.05em;
 color: var(--color-muted);
}

.events-mini-day {
 font-size: 1.1rem;
 font-weight: 700;
 line-height: 1;
}

.events-mini-title {
 font-weight: 600;
}

/* ----- */
* Page layout editor (admin -> Pages)
```

**src/client/styles.css**

```
* ----- */

.pages-admin-head {
 display: flex;
 align-items: flex-start;
 justify-content: space-between;
 gap: 1rem;
 flex-wrap: wrap;
}

.pages-admin-actions {
 display: flex;
 align-items: center;
 gap: 0.5rem;
 flex-wrap: wrap;
}

/* Landing view: the list of pages to pick from. */
.pages-list {
 list-style: none;
 margin: 1rem 0 0;
 padding: 0;
 display: flex;
 flex-direction: column;
 gap: 0.5rem;
}

.pages-list-item {
 width: 100%;
 display: flex;
 align-items: baseline;
 justify-content: space-between;
 gap: 1rem;
 text-align: left;
 padding: 0.85rem 1rem;
 background: var(--color-surface);
 border: 1px solid var(--color-border);
 border-radius: 8px;
 cursor: pointer;
 transition: border-color 0.12s ease, background 0.12s ease;
}

.pages-list-item:hover {
 border-color: var(--color-accent);
 background: var(--color-surface-2, var(--color-surface));
}

.pages-list-name {
 font-weight: 600;
 color: var(--color-text);
}

.pages-admin-meta {
 display: flex;
```

**src/client/styles.css**

```
 align-items: center;
 gap: 0.9rem;
 flex-wrap: wrap;
 margin: 0.25rem 0 1rem;
 padding-bottom: 0.85rem;
 border-bottom: 1px solid var(--color-border);
}

.pages-admin-meta .inline-field {
 color: var(--color-text);
}

.page-title {
 font-family: var(--font-display);
 margin: 0 0 1rem;
}

button.ghost {
 background: transparent;
 border: 1px solid var(--color-border);
 color: var(--color-text);
}

.add-row {
 width: 100%;
 border: 1px dashed var(--color-border);
 background: transparent;
 color: var(--color-muted);
 padding: 0.6rem;
}

.add-row:hover:not(:disabled) {
 color: var(--color-text);
 border-color: var(--color-accent);
}

.layout-editor {
 display: flex;
 flex-direction: column;
 gap: 1rem;
}

.row-editor {
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 background: color-mix(in srgb, var(--color-bg) 40%, transparent);
 padding: 0.75rem;
}

.row-editor-bar {
 display: flex;
 align-items: center;
 justify-content: space-between;
 margin-bottom: 0.6rem;
```



**src/client/styles.css**

```
}

.row-editor-label {
 font-size: 0.8rem;
 text-transform: uppercase;
 letter-spacing: 0.05em;
 color: var(--color-muted);
}

.row-editor-tools,
.module-editor-tools {
 display: flex;
 gap: 0.3rem;
}

.row-editor-cols {
 display: flex;
 gap: 0.75rem;
 align-items: flex-start;
 flex-wrap: wrap;
}

.col-editor {
 min-width: 12rem;
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 background: var(--color-surface);
 padding: 0.6rem;
 display: flex;
 flex-direction: column;
 gap: 0.5rem;
}

.col-editor-bar {
 display: flex;
 align-items: center;
 justify-content: space-between;
 gap: 0.5rem;
}

.col-span-label {
 display: inline-flex;
 align-items: center;
 gap: 0.35rem;
 font-size: 0.8rem;
 color: var(--color-muted);
}

.col-span-label select {
 padding: 0.2rem 0.35rem;
}

.col-dropzone {
 display: flex;
```

**src/client/styles.css**

```
flex-direction: column;
gap: 0.5rem;
min-height: 3rem;
border-radius: var(--radius);
padding: 0.25rem;
transition: background 0.12s, outline-color 0.12s;
outline: 2px dashed transparent;
}

.col-dropzone.over {
 outline-color: var(--color-accent);
 background: color-mix(in srgb, var(--color-accent) 10%, transparent);
}

.col-empty {
 text-align: center;
 color: var(--color-muted);
 font-size: 0.82rem;
 padding: 0.75rem;
}

.add-module {
 width: 100%;
}

.module-editor {
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 background: color-mix(in srgb, var(--color-bg) 55%, transparent);
 padding: 0.5rem;
}

.module-editor-bar {
 display: flex;
 align-items: center;
 gap: 0.4rem;
 margin-bottom: 0.5rem;
}

.module-editor-drag {
 cursor: grab;
 color: var(--color-muted);
 user-select: none;
}

.module-editor-type {
 font-weight: 600;
 font-size: 0.85rem;
 margin-right: auto;
}

.module-editor-config {
 display: flex;
 flex-direction: column;
```

**src/client/styles.css**

```
 gap: 0.5rem;
}

.module-editor-config input[type='text'],
.module-editor-config input[type='number'] {
 width: 100%;
}

.inline-field {
 display: inline-flex;
 align-items: center;
 gap: 0.4rem;
 font-size: 0.82rem;
 color: var(--color-muted);
}

.inline-field input[type='number'] {
 width: 5rem;
}

button.mini {
 padding: 0.2rem 0.45rem;
 font-size: 0.8rem;
 line-height: 1;
}

.pages-preview {
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 padding: 1rem;
 background: color-mix(in srgb, var(--color-bg) 40%, transparent);
}

.pages-preview-label {
 margin-bottom: 0.75rem;
}

/* ----- *
 * Extra content modules (image, button, divider, galleries)
 * ----- */

.module-image {
 margin: 0;
}

.module-image-img {
 display: block;
 width: 100%;
 height: auto;
 border-radius: var(--radius);
}

.module-image-caption {
 margin-top: 0.4rem;
```

**src/client/styles.css**

```
 text-align: center;
}

.module-button {
 display: flex;
 justify-content: center;
}

.btn-cta {
 display: inline-block;
 padding: 0.6rem 1.4rem;
 border-radius: var(--radius);
 border: 1px solid var(--color-border);
 color: var(--color-text);
 text-decoration: none;
 font-weight: 600;
 transition: filter 0.12s;
}

.btn-cta:hover {
 filter: brightness(1.08);
}

.btn-cta.primary {
 background: var(--color-accent);
 color: var(--color-accent-text);
 border-color: transparent;
}

.module-divider {
 border: none;
 border-top: 1px solid var(--color-border);
 margin: 0.5rem 0;
}

/* Hero banner module. The panel is intentionally dark regardless of theme so it
 reads as a landing hero; the accent pulls from the active theme. */
.module-hero {
 --hero-accent: var(--color-accent, #5b8cff);
 display: flex;
 flex-direction: column;
 gap: 1rem;
}

.hero-panel {
 position: relative;
 overflow: hidden;
 border-radius: 20px;
 padding: clamp(2.5rem, 6vw, 4.5rem) clamp(1.5rem, 5vw, 4rem);
 text-align: center;
 color: #f5f7fb;
 background:
 radial-gradient(1200px 500px at 80% -10%, color-mix(in srgb, var(--hero-accent) 28%,
transparent), transparent 60%),
 radial-gradient(900px 400px at 0% 110%, color-mix(in srgb, var(--hero-accent) 16%,
```

**src/client/styles.css**

```
transparent), transparent 55%),
 linear-gradient(160deg, #0c1020 0%, #0a0d18 60%, #080a12 100%);
}
.hero-eyebrow {
 display: inline-block;
 letter-spacing: 0.14em;
 text-transform: uppercase;
 font-size: 0.72rem;
 font-weight: 700;
 color: var(--hero-accent);
 margin-bottom: 1rem;
}
.hero-headline {
 margin: 0 auto 0.9rem;
 max-width: 16ch;
 font-weight: 800;
 line-height: 1.05;
 font-size: clamp(2.1rem, 5.5vw, 3.6rem);
 color: #ffffff;
}
.hero-subhead {
 margin: 0 auto;
 max-width: 60ch;
 color: #aeb7c8;
 font-size: clamp(1rem, 2.2vw, 1.18rem);
 line-height: 1.6;
}
.hero-cta {
 display: flex;
 flex-wrap: wrap;
 gap: 0.9rem;
 justify-content: center;
 margin: 2rem 0 1.4rem;
}
.hero-btn {
 display: inline-flex;
 align-items: center;
 gap: 0.55rem;
 padding: 0.85rem 1.6rem;
 border-radius: 12px;
 font-size: 1.02rem;
 font-weight: 700;
 text-decoration: none;
 transition: transform 0.15s ease, box-shadow 0.15s ease, border-color 0.15s ease;
}
.hero-btn:hover {
 transform: translateY(-2px);
}
.hero-btn-primary {
 background: var(--hero-accent);
 color: #0a0d18;
 box-shadow: 0 10px 30px color-mix(in srgb, var(--hero-accent) 45%, transparent);
}
.hero-btn-primary:hover {
```

**src/client/styles.css**

```
 box-shadow: 0 14px 38px color-mix(in srgb, var(--hero-accent) 60%, transparent);
}
.hero-btn-secondary {
 background: transparent;
 color: #f5f7fb;
 border: 1.5px solid color-mix(in srgb, #fff 28%, transparent);
}
.hero-btn-secondary:hover {
 border-color: var(--hero-accent);
 color: #fff;
}
.hero-chips {
 display: flex;
 flex-wrap: wrap;
 gap: 0.55rem;
 justify-content: center;
}
.hero-chip {
 display: inline-flex;
 align-items: center;
 gap: 0.4rem;
 padding: 0.4rem 0.85rem;
 border-radius: 999px;
 font-size: 0.85rem;
 font-weight: 600;
 color: #dfe6f5;
 background: color-mix(in srgb, #fff 6%, transparent);
 border: 1px solid color-mix(in srgb, #fff 12%, transparent);
}
.hero-grid {
 display: grid;
 gap: 1rem;
 grid-template-columns: repeat(auto-fit, minmax(230px, 1fr));
}
.hero-card {
 text-align: left;
 padding: 1.4rem 1.5rem;
 border-radius: 16px;
 background: linear-gradient(180deg, color-mix(in srgb, #fff 5%, transparent), color-mix(in srgb, #fff 2%, transparent));
 border: 1px solid color-mix(in srgb, #fff 10%, transparent);
 transition: transform 0.15s ease, border-color 0.15s ease;
}
.hero-card:hover {
 transform: translateY(-3px);
 border-color: color-mix(in srgb, var(--hero-accent) 55%, transparent);
}
.hero-card-icon {
 font-size: 1.6rem;
 line-height: 1;
 display: block;
 margin-bottom: 0.7rem;
}
.hero-card-title {
```

**src/client/styles.css**

```
margin: 0 0 0.15rem;
font-size: 1.08rem;
font-weight: 700;
color: #f5f7fb;
}
.hero-card-tag {
margin: 0 0 0.55rem;
font-size: 0.86rem;
font-weight: 600;
color: var(--hero-accent);
}
.hero-card-body {
margin: 0;
font-size: 0.92rem;
line-height: 1.55;
color: #c2cadb;
}
@media (prefers-reduced-motion: reduce) {
.hero-btn:hover,
.hero-card:hover {
transform: none;
}
}

/* Hero module editor form */
.hero-config {
display: flex;
flex-direction: column;
gap: 0.5rem;
}
.hero-config-row {
display: flex;
gap: 0.5rem;
align-items: center;
}
.hero-config-row > input {
flex: 1;
min-width: 0;
}
.hero-config-icon {
flex: 0 0 4rem;
}
.hero-config-label {
font-size: 0.8rem;
font-weight: 600;
color: var(--color-text-muted, #888);
margin-top: 0.4rem;
}
.hero-config-card {
display: flex;
flex-direction: column;
gap: 0.4rem;
padding: 0.6rem;
border: 1px solid var(--color-border);
```

**src/client/styles.css**

```
border-radius: var(--radius);
}

/* Medals / war-records / games galleries */
.module-gallery {
 list-style: none;
 margin: 0;
 padding: 0;
 display: grid;
 grid-template-columns: repeat(auto-fill, minmax(84px, 1fr));
 gap: 0.75rem;
}

.module-gallery-item {
 display: flex;
 flex-direction: column;
 align-items: center;
 gap: 0.35rem;
 text-align: center;
}

.module-gallery-item img {
 width: 48px;
 height: 48px;
 object-fit: contain;
}

.module-gallery-icon {
 font-size: 2rem;
 line-height: 48px;
}

.module-gallery-name {
 font-size: 0.78rem;
 color: var(--color-muted);
 line-height: 1.2;
}

/* Account settings ? API tokens */
.account-subhead {
 margin: 1.25rem 0 0.25rem;
}

.token-create {
 display: flex;
 gap: 0.5rem;
 margin: 0.75rem 0 1rem;
 flex-wrap: wrap;
}

.token-create input {
 flex: 1;
 min-width: 200px;
}

.token-reveal {
 border: 1px solid var(--color-accent);
```



**src/client/styles.css**

```
border-radius: var(--radius);
padding: 0.9rem 1rem;
margin: 0.75rem 0;
background: color-mix(in srgb, var(--color-accent) 10%, transparent);
display: flex;
flex-direction: column;
gap: 0.6rem;
}
.token-reveal-row {
display: flex;
gap: 0.5rem;
align-items: center;
}
.token-value {
flex: 1;
overflow-x: auto;
white-space: nowrap;
padding: 0.5rem 0.6rem;
background: var(--color-bg);
border: 1px solid var(--color-border);
border-radius: var(--radius);
font-family: ui-monospace, Menlo, monospace;
font-size: 0.85rem;
user-select: all;
}
.token-reveal .ghost {
align-self: flex-start;
}
.token-table {
width: 100%;
border-collapse: collapse;
}
.token-table th,
.token-table td {
text-align: left;
padding: 0.5rem 0.6rem;
border-bottom: 1px solid var(--color-border);
font-size: 0.9rem;
}
.token-table th {
color: var(--color-muted);
font-weight: 600;
}
.notice.error {
border-color: var(--color-accent);
}

/* Identity & Discord admin: the copy-these-into-Discord URL rows */
.identity-url {
margin: 0.5rem 0;
}
.identity-url-label {
display: block;
margin-bottom: 0.2rem;
}
```

**src/client/styles.css**

```
}
.identity-url-row {
 display: flex;
 align-items: center;
 gap: 0.5rem;
}
.identity-url-row code {
 flex: 1;
 min-width: 0;
 overflow-x: auto;
 white-space: nowrap;
 padding: 0.4rem 0.6rem;
 border: 1px solid var(--color-border);
 border-radius: 6px;
 background: var(--color-surface, rgba(127, 127, 127, 0.08));
 font-size: 0.85rem;
}
.identity-url-row .ghost.small {
 flex: none;
 padding: 0.3rem 0.7rem;
 font-size: 0.8rem;
}

/* Roster: header toggle + sort bar */
.roster-head {
 display: flex;
 align-items: center;
 justify-content: space-between;
 gap: 1rem;
 flex-wrap: wrap;
}
.roster-sort {
 display: flex;
 align-items: center;
 gap: 0.4rem;
 margin: 0.25rem 0 1rem;
 flex-wrap: wrap;
}
.roster-sort .seg {
 border: 1px solid var(--color-border);
 border-radius: 999px;
 background: transparent;
 color: var(--color-muted);
 padding: 0.25rem 0.7rem;
 cursor: pointer;
 font-size: 0.85rem;
}
.roster-sort .seg.active {
 color: var(--color-accent-text);
 background: var(--color-accent);
 border-color: var(--color-accent);
}
.roster-count {
 margin-left: auto;
```

**src/client/styles.css**

```
}

/* Org chart (leadership tree) */
.org-scroll {
 overflow: auto;
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 background: var(--color-bg);
 max-height: 72vh;
}
.org-canvas {
 position: relative;
}
.org-edges {
 position: absolute;
 inset: 0;
 pointer-events: none;
}
.org-edge {
 fill: none;
 stroke: var(--color-muted);
 stroke-width: 2;
 opacity: 0.7;
}
.org-edge-editable {
 pointer-events: stroke;
 cursor: pointer;
 stroke-width: 3;
}
.org-edge-editable:hover {
 stroke: var(--color-accent);
 opacity: 1;
}
.org-box {
 position: absolute;
 display: flex;
 align-items: center;
 gap: 0.5rem;
 padding: 0.4rem 0.6rem;
 background: var(--color-surface);
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 box-shadow: 0 2px 6px rgba(0, 0, 0, 0.18);
 text-decoration: none;
 color: var(--color-text);
 overflow: hidden;
 z-index: 1;
}
a.org-box:hover {
 border-color: var(--color-accent);
}
.org-box-avatar {
 width: 36px;
 height: 36px;
```

**src/client/styles.css**

```
border-radius: 50%;
flex: 0 0 auto;
object-fit: cover;
}
.org-box-text {
display: flex;
flex-direction: column;
min-width: 0;
}
.org-box-name {
font-weight: 600;
font-size: 0.9rem;
white-space: nowrap;
overflow: hidden;
text-overflow: ellipsis;
}
.org-box-rank {
font-size: 0.75rem;
color: var(--color-muted);
}

/* Designer interactions */
.org-editing .org-box-editable {
cursor: grab;
touch-action: none;
user-select: none;
}
.org-connect .org-box-editable {
cursor: pointer;
}
.org-box-editable.connecting {
outline: 2px solid var(--color-accent);
outline-offset: 1px;
}
.org-box-hidden {
opacity: 0.5;
border-style: dashed;
}
.org-box-remove {
position: absolute;
top: 2px;
right: 2px;
width: 18px;
height: 18px;
line-height: 1;
padding: 0;
border: 0;
border-radius: 50%;
background: color-mix(in srgb, var(--color-accent) 85%, transparent);
color: #fff;
font-size: 0.7rem;
cursor: pointer;
opacity: 0;
}
```

**src/client/styles.css**

```
.org-box-editable:hover .org-box-remove {
 opacity: 1;
}
.org-toolbar {
 display: flex;
 align-items: flex-end;
 gap: 1rem;
 flex-wrap: wrap;
 margin: 0.5rem 0 1rem;
}
.org-designer-head {
 display: flex;
 align-items: flex-start;
 justify-content: space-between;
 gap: 1rem;
}

/* Segmented control (e.g. theme menu-alignment) */
.seg-control {
 display: inline-flex;
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 overflow: hidden;
}
.seg-control .seg {
 border: 0;
 background: transparent;
 color: var(--color-muted);
 padding: 0.4rem 0.9rem;
 cursor: pointer;
 border-right: 1px solid var(--color-border);
}
.seg-control .seg:last-child {
 border-right: 0;
}
.seg-control .seg.active {
 background: var(--color-accent);
 color: var(--color-accent-text);
}
.seg-control .seg.disabled {
 cursor: default;
 opacity: 0.6;
}

/* Branding admin */
.branding-grid {
 display: grid;
 grid-template-columns: minmax(0, 2fr) minmax(0, 1fr);
 gap: 1.5rem;
 align-items: start;
}
@media (max-width: 720px) {
 .branding-grid {
 grid-template-columns: 1fr;
 }
}
```

**src/client/styles.css**

```
}
}
.branding-controls {
 display: flex;
 flex-direction: column;
 gap: 1rem;
}
.branding-preview {
 display: flex;
 align-items: center;
 justify-content: center;
 padding: 1.25rem;
 border: 1px dashed var(--color-border);
 border-radius: var(--radius);
 background: var(--color-bg);
 /* A checkerboard so transparent logos read clearly. */
 background-image:
 linear-gradient(45deg, rgba(255, 255, 255, 0.04) 25%, transparent 25%),
 linear-gradient(-45deg, rgba(255, 255, 255, 0.04) 25%, transparent 25%),
 linear-gradient(45deg, transparent 75%, rgba(255, 255, 255, 0.04) 75%),
 linear-gradient(-45deg, transparent 75%, rgba(255, 255, 255, 0.04) 75%);
 background-size: 16px 16px;
 background-position: 0 0, 0 8px, 8px -8px, -8px 0;
}
.branding-preview img {
 height: var(--h, 88px);
 width: auto;
 max-width: 100%;
 object-fit: contain;
}
.branding-size {
 display: flex;
 flex-direction: column;
 gap: 0.35rem;
 font-size: 0.9rem;
}
.branding-size input[type='range'] {
 width: 100%;
}
.branding-err {
 color: var(--color-accent);
}
.branding-hint ul {
 margin: 0.4rem 0 0 1.1rem;
 padding: 0;
}
.branding-hint li {
 margin: 0.3rem 0;
}
.branding-favicon {
 margin-top: 1rem;
 padding-top: 1rem;
 border-top: 1px solid var(--color-border);
 display: flex;
```

**src/client/styles.css**

```
 flex-direction: column;
 gap: 0.6rem;
}
.branding-favicon-head {
 display: flex;
 align-items: flex-start;
 gap: 0.75rem;
}
.branding-favicon-preview {
 width: 40px;
 height: 40px;
 flex: 0 0 auto;
 border-radius: 8px;
 object-fit: contain;
 background: var(--color-surface-2, rgba(127, 127, 127, 0.12));
 border: 1px solid var(--color-border);
}
.branding-favicon-preview.placeholder {
 display: inline-block;
}
}
.branding-favicon-head p {
 margin: 0.15rem 0 0 0;
}
}

/* HTML block module */
.module-html .page-html img {
 max-width: 100%;
 height: auto;
 border-radius: var(--radius);
}
.module-html .page-html table {
 width: 100%;
 border-collapse: collapse;
 margin: 0 0 0.9rem;
}
.module-html .page-html th,
.module-html .page-html td {
 border: 1px solid var(--color-border);
 padding: 0.4rem 0.6rem;
 text-align: left;
}
.module-html .page-html th {
 background: color-mix(in srgb, var(--color-muted) 12%, transparent);
}
.module-html .page-html figure {
 margin: 0 0 0.9rem;
}
.module-html .page-html figcaption {
 font-size: 0.82rem;
 color: var(--color-muted);
 text-align: center;
 margin-top: 0.3rem;
}
}
```

**src/client/styles.css**

```
/* Video embed module ? responsive fixed-ratio frame */
.module-embed {
 position: relative;
 width: 100%;
 aspect-ratio: 16 / 9;
 border-radius: var(--radius);
 overflow: hidden;
 background: #000;
}
.module-embed[data-ratio='4:3'] {
 aspect-ratio: 4 / 3;
}
.module-embed iframe {
 position: absolute;
 inset: 0;
 width: 100%;
 height: 100%;
 border: 0;
}
.module-embed-empty {
 text-align: center;
 border-style: dashed;
}

/* HTML + embed config (editor) */
.html-config {
 width: 100%;
 resize: vertical;
 font-family: ui-monospace, Menlo, monospace;
 font-size: 0.82rem;
 line-height: 1.5;
}
.module-embed-config {
 display: flex;
 flex-direction: column;
 gap: 0.5rem;
}

/* Image module config (editor) */
.module-image-config {
 display: flex;
 flex-direction: column;
 gap: 0.5rem;
}

.module-image-preview {
 width: 100%;
 max-height: 140px;
 object-fit: cover;
 border-radius: var(--radius);
 border: 1px solid var(--color-border);
}

.module-image-err {
```



**src/client/styles.css**

```
 color: var(--color-accent);
}

/* Visibility-gated module (shown only in the editor preview) */
.module-gated {
 opacity: 0.9;
}

.module-gated-badge {
 display: inline-block;
 margin-bottom: 0.4rem;
 font-size: 0.72rem;
 padding: 0.15rem 0.55rem;
 border-radius: 999px;
 background: color-mix(in srgb, var(--color-accent) 16%, transparent);
 border: 1px solid color-mix(in srgb, var(--color-accent) 38%, transparent);
 color: var(--color-text);
}

.module-audience {
 border-bottom: 1px dashed var(--color-border);
 padding-bottom: 0.5rem;
 margin-bottom: 0.25rem;
}

.module-audience select {
 max-width: 14rem;
}

/* ----- *
 * Top-nav categories (dropdowns) ? see components/SiteNav.tsx
 * ----- */

.nav-cat {
 position: relative;
 display: inline-flex;
}

.nav-cat-trigger {
 display: inline-flex;
 align-items: center;
 gap: 0.25rem;
 color: var(--color-muted);
 background: transparent;
 border: none;
 cursor: pointer;
 font: inherit;
 padding: 0.4rem 0.75rem;
 border-radius: var(--radius);
}

.nav-cat-trigger:hover,
.nav-cat.open .nav-cat-trigger {
 color: var(--color-text);
}
```

**src/client/styles.css**

```
background: color-mix(in srgb, var(--color-border) 60%, transparent);
}

.nav-cat-caret {
 font-size: 0.7em;
 transition: transform 0.15s ease;
}

.nav-cat.open .nav-cat-caret {
 transform: rotate(180deg);
}

.nav-cat-menu {
 position: absolute;
 top: calc(100% + 0.35rem);
 left: 0;
 min-width: 180px;
 z-index: 60;
 display: none;
 flex-direction: column;
 padding: 0.35rem;
 background: var(--color-surface);
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 box-shadow: 0 8px 24px rgba(0, 0, 0, 0.25);
}

.nav-cat.open .nav-cat-menu {
 display: flex;
}

.nav-cat-menu a {
 padding: 0.5rem 0.6rem;
 border-radius: var(--radius);
 color: var(--color-text);
 white-space: nowrap;
}

@media (max-width: 720px) {
 /* Inside the hamburger column the dropdown becomes an inline, full-width
 sub-list rather than a floating popover. */
 .nav-cat {
 display: block;
 width: 100%;
 }
 .nav-cat-menu {
 position: static;
 box-shadow: none;
 border: none;
 padding: 0 0 0 0.75rem;
 min-width: 0;
 }
}
```

**src/client/styles.css**

```
/* ----- */
* Navigation builder ? see pages/NavAdmin.tsx
* ----- */

.nav-admin-head {
 display: flex;
 align-items: flex-start;
 justify-content: space-between;
 gap: 1rem;
}

.nav-tree,
.nav-tree-children {
 list-style: none;
 margin: 1rem 0 0;
 padding: 0;
 display: flex;
 flex-direction: column;
 gap: 0.4rem;
}

.nav-tree-children {
 margin: 0.4rem 0 0 1.25rem;
 padding-left: 0.75rem;
 border-left: 2px solid var(--color-border);
}

.nav-row {
 display: flex;
 align-items: center;
 gap: 0.5rem;
 padding: 0.4rem 0.5rem;
 background: color-mix(in srgb, var(--color-bg) 55%, transparent);
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
}

.nav-row-group {
 background: color-mix(in srgb, var(--color-accent) 10%, transparent);
}

.nav-row-kind {
 flex: 0 0 auto;
 font-size: 0.72rem;
 color: var(--color-muted);
 min-width: 6.5rem;
 overflow: hidden;
 text-overflow: ellipsis;
 white-space: nowrap;
}

.nav-row-label {
 flex: 1 1 8rem;
 min-width: 5rem;
```

**src/client/styles.css**

```
}

.nav-role {
 flex: 0 0 auto;
 max-width: 9rem;
}

.nav-row-tools {
 flex: 0 0 auto;
 display: flex;
 align-items: center;
 gap: 0.25rem;
}

.nav-into {
 max-width: 6.5rem;
}

.nav-child-empty {
 padding: 0.3rem 0.1rem;
}

.nav-add {
 margin-top: 1rem;
 padding-top: 0.9rem;
 border-top: 1px solid var(--color-border);
 display: flex;
 flex-direction: column;
 gap: 0.6rem;
}

.nav-add-row {
 display: flex;
 align-items: flex-end;
 gap: 0.75rem;
 flex-wrap: wrap;
}

.nav-add-row label {
 display: flex;
 flex-direction: column;
 gap: 0.2rem;
 font-size: 0.8rem;
 color: var(--color-muted);
}

.nav-add-url-input {
 flex: 1 1 16rem;
 min-width: 12rem;
}

@media (max-width: 640px) {
 .nav-row {
 flex-wrap: wrap;
 }
}
```

**src/client/styles.css**

```
}
.nav-row-label {
 flex-basis: 100%;
 order: 3;
}
}

/* ----- *
 * Reassign-on-delete dialog ? see components/ReassignDialog.tsx
 * ----- */

.reassign-dialog {
 margin-top: 1rem;
 border-color: var(--color-accent);
}
.reassign-dialog h3 {
 margin: 0 0 0.25rem;
}
.reassign-field {
 display: flex;
 flex-direction: column;
 gap: 0.25rem;
 margin: 0.6rem 0;
 max-width: 26rem;
}
.reassign-actions {
 display: flex;
 gap: 0.5rem;
 margin-top: 0.75rem;
}

/* ----- *
 * Modules admin ? see pages/ModulesAdmin.tsx
 * ----- */

.modules-list {
 list-style: none;
 margin: 1rem 0 0;
 padding: 0;
 display: flex;
 flex-direction: column;
 gap: 0.6rem;
}
.module-row {
 display: flex;
 align-items: center;
 justify-content: space-between;
 gap: 1rem;
 padding: 0.75rem 0.9rem;
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 background: color-mix(in srgb, var(--color-bg) 55%, transparent);
}
.module-info {
```

**src/client/styles.css**

```

 display: flex;
 flex-direction: column;
 gap: 0.15rem;
}
.module-name {
 font-weight: 600;
}
.module-toggle {
 display: inline-flex;
 align-items: center;
 gap: 0.4rem;
 flex: 0 0 auto;
 white-space: nowrap;
}

/* ----- *
 * Star Citizen hangar ? see components/HangarView.tsx + HangarImport.tsx
 * ----- */

.hangar-section .hangar-import {
 margin-bottom: 1rem;
}
.hangar-setup {
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 padding: 0.5rem 0.75rem;
 margin-bottom: 0.75rem;
 background: color-mix(in srgb, var(--color-bg) 55%, transparent);
}
.hangar-setup summary {
 cursor: pointer;
 font-weight: 600;
 font-size: 0.9rem;
}
.hangar-setup-steps {
 margin: 0.6rem 0 0;
 padding-left: 1.1rem;
 display: flex;
 flex-direction: column;
 gap: 0.5rem;
 font-size: 0.9rem;
}
.hangar-bookmarklet,
.hangar-json {
 width: 100%;
 background: var(--color-bg);
 color: var(--color-text);
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 padding: 0.5rem;
 font-family: monospace;
 font-size: 0.78rem;
 margin: 0.35rem 0;
 resize: vertical;

```

**src/client/styles.css**

```
}
.hangar-import-row {
 display: flex;
 gap: 0.6rem;
 align-items: flex-start;
}
.hangar-import-row .hangar-json {
 flex: 1;
 margin: 0;
}
.hangar-import-actions {
 display: flex;
 flex-direction: column;
 gap: 0.35rem;
 flex: 0 0 auto;
}
.hangar-share {
 display: flex;
 align-items: flex-start;
 gap: 0.55rem;
 margin-top: 0.75rem;
 padding: 0.6rem 0.75rem;
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 cursor: pointer;
}
.hangar-share input {
 margin-top: 0.15rem;
 flex: 0 0 auto;
}
.hangar-share > span {
 display: flex;
 flex-direction: column;
 gap: 0.1rem;
}

.hangar-toolbar {
 display: flex;
 flex-wrap: wrap;
 gap: 0.6rem;
 align-items: center;
 margin-bottom: 0.5rem;
}
.hangar-filters {
 display: flex;
 flex-wrap: wrap;
 gap: 0.35rem;
}
.hangar-chip {
 background: transparent;
 border: 1px solid var(--color-border);
 color: var(--color-muted);
 border-radius: 999px;
 padding: 0.3rem 0.65rem;
```

**src/client/styles.css**

```
 cursor: pointer;
 font: inherit;
 font-size: 0.82rem;
}
.hangar-chip.active {
 background: var(--color-accent);
 border-color: var(--color-accent);
 color: var(--color-accent-text);
}
.hangar-chip .n {
 opacity: 0.7;
 margin-left: 0.3rem;
 font-size: 0.85em;
}
.hangar-search {
 flex: 1;
 min-width: 160px;
 background: var(--color-bg);
 color: var(--color-text);
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 padding: 0.4rem 0.6rem;
 font: inherit;
}
.hangar-summary {
 display: flex;
 flex-wrap: wrap;
 gap: 0.5rem 1.4rem;
 margin: 0.2rem 0 0.35rem;
}
.hangar-stat {
 font-size: 0.9rem;
 color: var(--color-text-dim, var(--color-muted, inherit));
}
.hangar-stat strong {
 font-size: 1.15rem;
 color: var(--color-text, inherit);
 margin-right: 0.15rem;
}
.hangar-count {
 margin-bottom: 0.4rem;
}
.hangar-selfreport {
 margin: 0 0 0.6rem;
 opacity: 0.75;
}

/* Star Citizen account verification */
.sc-verify {
 margin: 0 0 0.9rem;
 display: flex;
 flex-direction: column;
 gap: 0.5rem;
}
```



**src/client/styles.css**

```
.sc-verify-badge {
 display: inline-flex;
 align-items: center;
 gap: 0.35rem;
 flex-wrap: wrap;
 font-size: 0.9rem;
 padding: 0.3rem 0.6rem;
 border-radius: 999px;
 width: fit-content;
 border: 1px solid var(--color-border);
}
.sc-verify-badge.verified {
 color: var(--color-text);
 border-color: color-mix(in srgb, var(--color-accent) 45%, transparent);
 background: color-mix(in srgb, var(--color-accent) 12%, transparent);
}
.sc-verify-badge.unverified {
 color: var(--color-text-dim, var(--color-muted, inherit));
}
.sc-verify-check {
 color: var(--color-accent);
 font-weight: 800;
}
.sc-verify-org {
 font-size: 0.85rem;
}
.sc-verify-actions {
 display: flex;
 gap: 0.5rem;
 flex-wrap: wrap;
}
.sc-verify-flow {
 display: flex;
 flex-direction: column;
 gap: 0.5rem;
 padding: 0.6rem 0.75rem;
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
}
.sc-verify-code {
 display: inline-block;
 width: fit-content;
 padding: 0.35rem 0.6rem;
 border-radius: 6px;
 font-size: 1rem;
 letter-spacing: 0.03em;
 background: color-mix(in srgb, var(--color-accent) 16%, transparent);
 border: 1px solid color-mix(in srgb, var(--color-accent) 35%, transparent);
 cursor: pointer;
 user-select: all;
}
.sc-verify-start {
 display: flex;
 gap: 0.5rem;
```

**src/client/styles.css**

```
 flex-wrap: wrap;
}
.sc-verify-start input {
 flex: 1;
 min-width: 12rem;
}

/* Site-wide footer */
.site-footer {
 margin-top: auto;
 padding: 1.5rem 1.25rem 2rem;
 border-top: 1px solid var(--color-border);
 display: flex;
 flex-direction: column;
 align-items: center;
 gap: 0.6rem;
 text-align: center;
 color: var(--color-text-dim, var(--color-muted, inherit));
 font-size: 0.82rem;
 line-height: 1.5;
}
.site-footer-text {
 max-width: 70ch;
}
.site-footer-text a {
 color: var(--color-accent);
 text-decoration: none;
}
.site-footer-text a:hover {
 text-decoration: underline;
}
.site-footer-links {
 display: flex;
 flex-wrap: wrap;
 justify-content: center;
 gap: 1rem;
}
.site-footer-links a {
 color: var(--color-accent);
 text-decoration: none;
 font-weight: 500;
}
.site-footer-links a:hover {
 text-decoration: underline;
}
.site-footer-copy {
 opacity: 0.8;
}

/* Footer admin ? link rows */
.footer-link-row {
 display: flex;
 gap: 0.5rem;
 align-items: center;
```

**src/client/styles.css**

```
 margin-bottom: 0.5rem;
}
.footer-link-row input:first-of-type {
 flex: 0 0 10rem;
}
.footer-link-row input:last-of-type {
 flex: 1;
 min-width: 0;
}

/* Non-affiliation disclaimer under the Star Citizen section */
.sc-disclaimer {
 margin: 1rem 0 0;
 padding-top: 0.6rem;
 border-top: 1px solid var(--color-border);
 opacity: 0.7;
 font-size: 0.78rem;
 line-height: 1.5;
}

/* Modules admin ? per-feature kill switches */
.module-killswitch {
 margin-top: 1rem;
 padding-top: 0.75rem;
 border-top: 1px solid var(--color-border);
 display: flex;
 flex-direction: column;
 gap: 0.5rem;
}
.module-killswitch-title {
 display: block;
 margin-bottom: 0.25rem;
}

/* Modules admin ? per-module config block (e.g. SC org SID) */
.module-config {
 margin-top: 1.25rem;
 padding-top: 1rem;
 border-top: 1px solid var(--color-border);
}
.module-config h3 {
 margin: 0 0 0.6rem;
 font-size: 1rem;
}
.module-config-row {
 display: flex;
 gap: 0.5rem;
 align-items: center;
 margin: 0.25rem 0;
}
.module-config-row input {
 max-width: 12rem;
}
.hangar-table-wrap {
```

**src/client/styles.css**

```
 overflow-x: auto;
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
}
.hangar-table {
 width: 100%;
 border-collapse: collapse;
 font-size: 0.9rem;
}
.hangar-table th,
.hangar-table td {
 text-align: left;
 padding: 0.4rem 0.55rem;
 border-bottom: 1px solid var(--color-border);
 vertical-align: middle;
 white-space: nowrap;
}
.hangar-table th {
 position: sticky;
 top: 0;
 background: var(--color-surface);
 cursor: pointer;
 user-select: none;
 font-size: 0.72rem;
 text-transform: uppercase;
 letter-spacing: 0.04em;
 color: var(--color-muted);
}
.hangar-table tr:hover td {
 background: color-mix(in srgb, var(--color-accent) 6%, transparent);
}
.hangar-name {
 white-space: normal;
 font-weight: 600;
 min-width: 12rem;
}
.hangar-thumb {
 width: 56px;
 height: 34px;
 object-fit: cover;
 border-radius: 4px;
 background: #000;
 border: 1px solid var(--color-border);
}
.hangar-type {
 font-size: 0.72rem;
 text-transform: uppercase;
 letter-spacing: 0.04em;
 color: var(--color-accent);
}
.hangar-empty {
 padding: 0.75rem;
}
```

**src/client/styles.css**

```
/* SEO & Sharing ? og image preview (see pages/SeoAdmin.tsx) */
.seo-og {
 display: flex;
 align-items: center;
 gap: 0.6rem;
 flex-wrap: wrap;
 margin: 0.25rem 0;
}
.seo-og-preview {
 width: 160px;
 max-width: 100%;
 aspect-ratio: 1200 / 630;
 object-fit: cover;
 border: 1px solid var(--color-border);
 border-radius: var(--radius);
 background: #000;
}
```

**src/db/schema.ts**

```

import { sqliteTable, text, integer, index, uniqueIndex, primaryKey } from
'drizzle-orm/sqlite-core';
import { sql } from 'drizzle-orm';
import type { HangarItem } from '../shared/hangar';

const now = sql`(unixepoch())`;

/* ----- *
 * Identity
 *
 * Discord snowflakes exceed Number.MAX_SAFE_INTEGER, so every *_id that
 * comes from Discord is TEXT. Never parse these into a JS number.
 * ----- */

export const users = sqliteTable(
 'users',
 {
 id: integer('id').primaryKey({ autoIncrement: true }),
 discordId: text('discord_id').notNull().unique(),
 username: text('username').notNull(),
 globalName: text('global_name'),
 // Clan/game name a member sets for themselves. When present it is shown in
 // place of the Discord name everywhere; see shared/names.ts for the order.
 displayName: text('display_name'),
 avatar: text('avatar'), // Discord avatar hash, not a URL
 // A member-chosen profile image (an R2 /media/avatars/... URL). When set it
 // overrides the Discord avatar everywhere the member is shown; NULL falls
 // back to the Discord avatar. See shared/avatar.ts for the resolution order.
 profileImageUrl: text('profile_image_url'),
 email: text('email'),

 rankId: integer('rank_id').references(() => ranks.id, { onDelete: 'set null' }),

 // Bypasses every permission check. Seeded for the founder; see seed.sql.
 isGod: integer('is_god', { mode: 'boolean' }).notNull().default(false),

 // When Discord says this member joined the guild ? the authoritative basis
 // for tenure medals. Captured at login and backfilled by the reconcile
 // sweep; falls back to joinedAt (site-join) when Discord hasn't been read
 // for them yet.
 guildJoinedAt: integer('guild_joined_at'),

 status: text('status', { enum: ['active', 'inactive', 'loa', 'retired', 'banned'] })
 .notNull()
 .default('active'),

 recruiterId: integer('recruiter_id'),

 joinedAt: integer('joined_at').notNull().default(now),
 lastSeenAt: integer('last_seen_at'),
 // When the Discord reconcile sweep last checked this member. The cron
 // processes members least-recently-reconciled first, rotating through the
 // roster a bounded batch at a time (see discord/sync.ts reconcileBatch).
 lastReconciledAt: integer('last_reconciled_at'),
 }
);

```

**src/db/schema.ts**

```

 promotedAt: integer('promoted_at'),
 createdAt: integer('created_at').notNull().default(now),
 updatedAt: integer('updated_at').notNull().default(now),
 },
 (t) => [
 index('users_rank_idx').on(t.rankId),
 index('users_status_idx').on(t.status),
],
);

export const profiles = sqliteTable('profiles', {
 userId: integer('user_id')
 .primaryKey()
 .references(() => users.id, { onDelete: 'cascade' }),
 bio: text('bio'),
 location: text('location'),
 timezone: text('timezone'),
 // { "steam": "...", "xbox": "...", "psn": "..."}
 gamertags: text('gamertags', { mode: 'json' }).$type<Record<string, string>>(),
 updatedAt: integer('updated_at').notNull().default(now),
});

/**
 * A member's imported Star Citizen hangar (the SC module) ? one JSON blob per
 * member, the normalised item list from the scraper. Replaced wholesale on each
 * re-import; cascades away with the member. Only used when the Star Citizen
 * module is enabled in settings.
 */
export const scHangars = sqliteTable('sc_hangars', {
 userId: integer('user_id')
 .primaryKey()
 .references(() => users.id, { onDelete: 'cascade' }),
 items: text('items', { mode: 'json' }).$type<HangarItem[]>().notNull(),
 itemCount: integer('item_count').notNull().default(0),
 // Whether the owner has chosen to share this hangar with members who hold the
 // hangar.view permission. Off by default ? sharing is opt-in per member.
 isPublic: integer('is_public', { mode: 'boolean' }).notNull().default(false),
 importedAt: integer('imported_at').notNull().default(now),
});

/**
 * RSI account verification (the SC module's anti-poser track). A member proves
 * control of an RSI account by placing a one-time code in their public citizen
 * bio; the Worker fetches the profile, confirms the code, and records the parsed
 * org membership. `rsiHandle` is unique so two members can't claim one account
 * (multiple NULLs are allowed for the unverified). The pending_* columns hold an
 * in-flight challenge before it's confirmed. Cascades away with the member.
 */
export const scVerifications = sqliteTable('sc_verifications', {
 userId: integer('user_id')
 .primaryKey()
 .references(() => users.id, { onDelete: 'cascade' }),
 // In-flight challenge: the code the member must place in their RSI bio and the
 // handle they claimed. Cleared once verified (or replaced on a new attempt).

```

**src/db/schema.ts**

```

 pendingCode: text('pending_code'),
 pendingHandle: text('pending_handle'),
 pendingAt: integer('pending_at'),
 // Last time we fetched RSI for this member ? a light rate-limit so the confirm
 // button can't hammer RSI (politeness, set before each fetch).
 lastCheckAt: integer('last_check_at'),
 // The verified result ? NULL until a confirm succeeds.
 rsiHandle: text('rsi_handle').unique(),
 verifiedAt: integer('verified_at'),
 orgSid: text('org_sid'),
 orgRank: text('org_rank'),
 // Whether the profile's org block was public (vs redacted), and whether the
 // parsed org SID matched this install's configured org.
 orgVisible: integer('org_visible', { mode: 'boolean' }),
 inOrg: integer('in_org', { mode: 'boolean' }),
 });

/* ----- *
 * Hierarchy
 *
 * The 2003 version stored ranks as 21 fixed columns (name_0..name_20) in
 * a single row. Here they are rows: add, delete, and reorder freely.
 * ----- */

export const ranks = sqliteTable(
 'ranks',
 {
 id: integer('id').primaryKey({ autoIncrement: true }),
 name: text('name').notNull(),
 abbreviation: text('abbreviation'),
 imageUrl: text('image_url'),

 // 0 = lowest. Gaps are fine; reorder by rewriting these.
 sortOrder: integer('sort_order').notNull(),

 // Carried over from the original's auto-promotion criteria. 0 = ignored.
 reqDays: integer('req_days').notNull().default(0),
 reqWins: integer('req_wins').notNull().default(0),

 // Rank handed to new recruits on first Discord login. Exactly one should be true.
 isDefault: integer('is_default', { mode: 'boolean' }).notNull().default(false),

 createdAt: integer('created_at').notNull().default(now),
 updatedAt: integer('updated_at').notNull().default(now),
 },
 (t) => [uniqueIndex('ranks_sort_order_idx').on(t.sortOrder)],
);

/* ----- *
 * Permissions
 *
 * A role is a bundle of permission strings AND (optionally) the mirror of
 * a Discord role. Setting discordRoleId makes membership changes here
 * push to Discord, and lets a Discord-side change pull back in.

```



**src/db/schema.ts**

```

* ----- */

export const roles = sqliteTable('roles', {
 id: integer('id').primaryKey({ autoIncrement: true }),
 name: text('name').notNull().unique(),
 description: text('description'),
 color: text('color'), // hex, for admin UI chips

 discordRoleId: text('discord_role_id').unique(),

 sortOrder: integer('sort_order').notNull().default(0),

 // System roles cannot be deleted through the admin UI (they can be renamed).
 isSystem: integer('is_system', { mode: 'boolean' }).notNull().default(false),

 createdAt: integer('created_at').notNull().default(now),
 updatedAt: integer('updated_at').notNull().default(now),
});

export const rolePermissions = sqliteTable(
 'role_permissions',
 {
 roleId: integer('role_id')
 .notNull()
 .references(() => roles.id, { onDelete: 'cascade' }),
 // A Permission literal from src/shared/permissions.ts
 permission: text('permission').notNull(),
 },
 (t) => [primaryKey({ columns: [t.roleId, t.permission] })],
);

export const userRoles = sqliteTable(
 'user_roles',
 {
 userId: integer('user_id')
 .notNull()
 .references(() => users.id, { onDelete: 'cascade' }),
 roleId: integer('role_id')
 .notNull()
 .references(() => roles.id, { onDelete: 'cascade' }),
 // How the member came to hold this role. 'manual' grants are permanent
 // until revoked by hand; 'rank' grants are reconciled when the member's
 // rank changes. A manual grant always wins, so promoting/demoting never
 // strips a role an admin gave on purpose.
 source: text('source', { enum: ['manual', 'rank'] })
 .notNull()
 .default('manual'),
 grantedBy: integer('granted_by').references(() => users.id, { onDelete: 'set null' }),
 grantedAt: integer('granted_at').notNull().default(now),
 },
 (t) => [primaryKey({ columns: [t.userId, t.roleId] })],
 index('user_roles_role_idx').on(t.roleId),
);

```

**src/db/schema.ts**

```

/**
 * Which roles a rank confers. Assigning a member to a rank grants these roles
 * (source='rank'), and a rank change reconciles them. Many-to-many: a rank can
 * grant several roles, and a role can be granted by several ranks.
 */
export const rankRoles = sqliteTable(
 'rank_roles',
 {
 rankId: integer('rank_id')
 .notNull()
 .references(() => ranks.id, { onDelete: 'cascade' }),
 roleId: integer('role_id')
 .notNull()
 .references(() => roles.id, { onDelete: 'cascade' }),
 },
 (t) => [primaryKey({ columns: [t.rankId, t.roleId] })],
 index('rank_roles_role_idx').on(t.roleId)],
);

/* ----- *
 * Sessions ? opaque random id in an httpOnly cookie, state kept here.
 * ----- */

export const sessions = sqliteTable(
 'sessions',
 {
 id: text('id').primaryKey(), // 256-bit random, base64url
 userId: integer('user_id')
 .notNull()
 .references(() => users.id, { onDelete: 'cascade' }),
 expiresAt: integer('expires_at').notNull(),
 userAgent: text('user_agent'),
 ip: text('ip'),
 createdAt: integer('created_at').notNull().default(now),
 },
 (t) => [index('sessions_user_idx').on(t.userId), index('sessions_expires_idx').on(t.expiresAt)],
);

/* ----- *
 * API tokens ? long-lived personal access tokens for the mobile/native app and
 * scripts. Like sessions, only the SHA-256 of the token is stored, so a database
 * dump can't be replayed. Unlike sessions they carry a user-chosen label and a
 * short display prefix, are managed from account settings, and resolve to the
 * same Viewer (rank + union of role permissions) as a web session.
 * ----- */

export const apiTokens = sqliteTable(
 'api_tokens',
 {
 id: integer('id').primaryKey({ autoIncrement: true }),
 userId: integer('user_id')
 .notNull()
 .references(() => users.id, { onDelete: 'cascade' }),
 tokenHash: text('token_hash').notNull().unique(), // SHA-256 (base64url) of the raw token
 }
);

```

**src/db/schema.ts**

```

 label: text('label').notNull(),
 prefix: text('prefix').notNull(), // e.g. "clt_ab12cd" ? shown so a token is identifiable
 expiresAt: integer('expires_at'), // null = no expiry
 lastUsedAt: integer('last_used_at'),
 createdAt: integer('created_at').notNull().default(now),
 },
 (t) => [index('api_tokens_user_idx').on(t.userId)],
);

/* ----- *
 * Content
 * ----- */

export const news = sqliteTable(
 'news',
 {
 id: integer('id').primaryKey({ autoIncrement: true }),
 title: text('title').notNull(),
 slug: text('slug').notNull().unique(),
 excerpt: text('excerpt'),
 // HTML from the WYSIWYG editor (TipTap). Sanitized on save and again with
 // DOMPurify on render, so stored markup is never trusted at display time.
 body: text('body').notNull(),
 authorId: integer('author_id').references(() => users.id, { onDelete: 'set null' }),
 status: text('status', { enum: ['draft', 'published', 'archived'] })
 .notNull()
 .default('draft'),
 pinned: integer('pinned', { mode: 'boolean' }).notNull().default(false),
 publishedAt: integer('published_at'),
 createdAt: integer('created_at').notNull().default(now),
 updatedAt: integer('updated_at').notNull().default(now),
 },
 (t) => [index('news_status_published_idx').on(t.status, t.publishedAt)],
);

/**
 * Replaces the original site_info + site_prefs + templates + header tables.
 * Theme tokens live under the 'theme' key as CSS custom properties, e.g.
 * { "--color-accent": "#c0392b", "--font-body": "Inter" }
 */
export const settings = sqliteTable('settings', {
 key: text('key').primaryKey(),
 value: text('value', { mode: 'json' }).notNull(),
 updatedBy: integer('updated_by').references(() => users.id, { onDelete: 'set null' }),
 updatedAt: integer('updated_at').notNull().default(now),
});

/**
 * Drag-and-drop page layouts. Each standard page (home, etc.) is one row keyed
 * by a stable `slug`; `layout` is the JSON module tree (rows -> columns ->
 * modules) defined in src/shared/layout.ts. Any client ? the web app today, a
 * native app later ? renders the same data, so the layout is deliberately kept
 * as portable JSON rather than baked into components. A missing row falls back
 * to the built-in DEFAULT_LAYOUTS for that slug.

```

**src/db/schema.ts**

```

*/
export const pageLayouts = sqliteTable('page_layouts', {
 slug: text('slug').primaryKey(),
 // Human title, shown in the admin page list and used as the nav label.
 title: text('title'),
 layout: text('layout', { mode: 'json' }).$type<unknown>().notNull(),
 // Custom pages can be surfaced in the top nav. 'home' is navigated to at / and
 // is never listed here. nav_order sorts the extra links.
 showInNav: integer('show_in_nav', { mode: 'boolean' }).notNull().default(false),
 navOrder: integer('nav_order').notNull().default(0),
 updatedBy: integer('updated_by').references(() => users.id, { onDelete: 'set null' }),
 updatedAt: integer('updated_at').notNull().default(now),
});

/* ----- *
 * Competition
 * ----- */

export const games = sqliteTable('games', {
 id: integer('id').primaryKey({ autoIncrement: true }),
 name: text('name').notNull(),
 slug: text('slug').notNull().unique(),
 iconUrl: text('icon_url'),
 sortOrder: integer('sort_order').notNull().default(0),
 active: integer('active', { mode: 'boolean' }).notNull().default(true),
 createdAt: integer('created_at').notNull().default(now),
});

export const medals = sqliteTable(
 'medals',
 {
 id: integer('id').primaryKey({ autoIncrement: true }),
 name: text('name').notNull(),
 description: text('description'),
 imageUrl: text('image_url'),
 // NULL = clan-wide medal. Set = specific to one game (was the game_medals table).
 gameId: integer('game_id').references(() => games.id, { onDelete: 'cascade' }),
 // Tenure medals: when set, this medal is granted automatically once a member
 // reaches this many months in the guild (6, 12, 24, ...). NULL = awarded by
 // hand only. The auto-grant sweep lives in server/medals/tenure.ts.
 autoGrantMonths: integer('auto_grant_months'),
 sortOrder: integer('sort_order').notNull().default(0),
 createdAt: integer('created_at').notNull().default(now),
 },
 (t) => [index('medals_game_idx').on(t.gameId)],
);

export const memberMedals = sqliteTable(
 'member_medals',
 {
 id: integer('id').primaryKey({ autoIncrement: true }),
 userId: integer('user_id')
 .notNull()
 .references(() => users.id, { onDelete: 'cascade' }),
 },

```

**src/db/schema.ts**

```

 medalId: integer('medal_id')
 .notNull()
 .references(() => medals.id, { onDelete: 'cascade' }),
 citation: text('citation'), // why it was awarded
 awardedBy: integer('awarded_by').references(() => users.id, { onDelete: 'set null' }),
 awardedAt: integer('awarded_at').notNull().default(now),
 },
 (t) => [
 index('member_medals_user_idx').on(t.userId),
 index('member_medals_medal_idx').on(t.medalId),
],
);

/* -----
 * War records ? awardable "items of pride", handed to members like medals
 * but themed as clan-war honours. A definition (optionally tied to a game)
 * plus a join table of who holds it. Distinct from `matches` below, which is
 * the (currently unused) detailed battle-log design.
 * ----- */

export const warRecords = sqliteTable(
 'war_records',
 {
 id: integer('id').primaryKey({ autoIncrement: true }),
 name: text('name').notNull(),
 description: text('description'),
 imageUrl: text('image_url'),
 // NULL = clan-wide. Set = specific to one game.
 gameId: integer('game_id').references(() => games.id, { onDelete: 'set null' }),
 sortOrder: integer('sort_order').notNull().default(0),
 createdAt: integer('created_at').notNull().default(now),
 },
 (t) => [index('war_records_game_idx').on(t.gameId)],
);

export const memberWarRecords = sqliteTable(
 'member_war_records',
 {
 id: integer('id').primaryKey({ autoIncrement: true }),
 userId: integer('user_id')
 .notNull()
 .references(() => users.id, { onDelete: 'cascade' }),
 warRecordId: integer('war_record_id')
 .notNull()
 .references(() => warRecords.id, { onDelete: 'cascade' }),
 citation: text('citation'),
 awardedBy: integer('awarded_by').references(() => users.id, { onDelete: 'set null' }),
 awardedAt: integer('awarded_at').notNull().default(now),
 },
 (t) => [
 index('member_war_records_user_idx').on(t.userId),
 index('member_war_records_record_idx').on(t.warRecordId),
],
);

```

**src/db/schema.ts**

```

/** Replaces the original records + stat_records tables. */
export const matches = sqliteTable(
 'matches',
 {
 id: integer('id').primaryKey({ autoIncrement: true }),
 gameId: integer('game_id')
 .notNull()
 .references(() => games.id, { onDelete: 'cascade' }),
 opponent: text('opponent').notNull(),
 opponentTag: text('opponent_tag'),
 result: text('result', { enum: ['win', 'loss', 'draw', 'forfeit'] }).notNull(),
 scoreUs: integer('score_us'),
 scoreThem: integer('score_them'),
 map: text('map'),
 notes: text('notes'),
 playedAt: integer('played_at').notNull(),
 recordedBy: integer('recorded_by').references(() => users.id, { onDelete: 'set null' }),
 createdAt: integer('created_at').notNull().default(now),
 },
 (t) => [index('matches_game_played_idx').on(t.gameId, t.playedAt)],
);

export const matchParticipants = sqliteTable(
 'match_participants',
 {
 matchId: integer('match_id')
 .notNull()
 .references(() => matches.id, { onDelete: 'cascade' }),
 userId: integer('user_id')
 .notNull()
 .references(() => users.id, { onDelete: 'cascade' }),
 // Per-game shape varies, so keep it open: { "kills": 12, "deaths": 4 }
 stats: text('stats', { mode: 'json' }).$type<Record<string, number>>(),
 },
 (t) => [primaryKey({ columns: [t.matchId, t.userId] }), index('mp_user_idx').on(t.userId)],
);

/* ----- *
 * Events ? clan happenings (war nights, tournaments, movie nights).
 * Authorised members create them; the bot mirrors each to a native Discord
 * scheduled event AND an announcement message, tracked by the id columns so
 * edits and cancellations stay in sync. Times are unix seconds (UTC).
 * ----- */

export const events = sqliteTable(
 'events',
 {
 id: integer('id').primaryKey({ autoIncrement: true }),
 title: text('title').notNull(),
 description: text('description'),
 // An optional banner image (an R2 /media/events/... URL), shown on the events
 // page and as the Discord announcement embed image.
 imageUrl: text('image_url'),
 }

```

**src/db/schema.ts**

```

 startsAt: integer('starts_at').notNull(),
 // Discord's external scheduled events require an end time and a location.
 endsAt: integer('ends_at').notNull(),
 location: text('location').notNull(),
 gameId: integer('game_id').references(() => games.id, { onDelete: 'set null' }),
 createdBy: integer('created_by').references(() => users.id, { onDelete: 'set null' }),
 // The native Discord scheduled event and the channel announcement message,
 // so an edit/cancel here can update or remove them.
 discordEventId: text('discord_event_id'),
 discordMessageId: text('discord_message_id'),
 status: text('status', { enum: ['scheduled', 'cancelled'] })
 .notNull()
 .default('scheduled'),
 // How often this event repeats. The event itself is the first occurrence;
 // when it ends the cron spawns the next one and clears this back to 'none'
 // on the finished row (the recurrence "baton" moves to the new occurrence).
 // So at most one future occurrence per series carries a non-'none' value.
 recurrence: text('recurrence', { enum: ['none', 'daily', 'weekly', 'biweekly', 'monthly'] })
 .notNull()
 .default('none'),
 createdAt: integer('created_at').notNull().default(now),
 updatedAt: integer('updated_at').notNull().default(now),
 },
 (t) => [index('events_starts_idx').on(t.startsAt)],
);

/**
 * The sign-up "roles" for an event ? e.g. Tank / Healer / DPS. Defined per
 * event when it's created. A member picks at most one; an optional capacity
 * caps how many can hold it. Deleting a role frees its holders back to
 * "attending, no role" (the signup's eventRoleId is set null).
 */
export const eventRoles = sqliteTable(
 'event_roles',
 {
 id: integer('id').primaryKey({ autoIncrement: true }),
 eventId: integer('event_id')
 .notNull()
 .references(() => events.id, { onDelete: 'cascade' }),
 name: text('name').notNull(),
 // Optional emoji shown on the Discord sign-up button (a unicode char).
 emoji: text('emoji'),
 // NULL = unlimited. Otherwise the max number of members who can hold it.
 capacity: integer('capacity'),
 sortOrder: integer('sort_order').notNull().default(0),
 },
 (t) => [index('event_roles_event_idx').on(t.eventId)],
);

/**
 * A member's sign-up for an event. One row per member per event (unique), so
 * choosing a different role updates the same row and withdrawing deletes it.
 * eventRoleId NULL means "attending, no specific role". Written from both the
 * website and Discord button clicks ? the two stay in sync because they share

```

**src/db/schema.ts**

```

* this table.
*/
export const eventSignups = sqliteTable(
 'event_signups',
 {
 id: integer('id').primaryKey({ autoIncrement: true }),
 eventId: integer('event_id')
 .notNull()
 .references(() => events.id, { onDelete: 'cascade' }),
 userId: integer('user_id')
 .notNull()
 .references(() => users.id, { onDelete: 'cascade' }),
 eventRoleId: integer('event_role_id').references(() => eventRoles.id, { onDelete: 'set null'
 }),
 createdAt: integer('created_at').notNull().default(now),
 updatedAt: integer('updated_at').notNull().default(now),
},
(t) => [
 uniqueIndex('event_signups_event_user_idx').on(t.eventId, t.userId),
 index('event_signups_event_idx').on(t.eventId),
],
);

/* ----- *
 * Audit ? replaces the original log + security tables.
 * ----- */

export const auditLog = sqliteTable(
 'audit_log',
 {
 id: integer('id').primaryKey({ autoIncrement: true }),
 actorId: integer('actor_id').references(() => users.id, { onDelete: 'set null' }),
 action: text('action').notNull(), // e.g. 'member.promote', 'role.grant'
 targetType: text('target_type'),
 targetId: text('target_id'),
 // A human-written justification. Required for negative actions (demote,
 // remove/ban, revoke a medal/war record/role); optional otherwise. Kept as
 // its own column ? not buried in meta ? so the log can show and filter on it.
 reason: text('reason'),
 meta: text('meta', { mode: 'json' }).$type<Record<string, unknown>>(),
 // 'web' for admin portal, 'discord' for slash commands
 source: text('source', { enum: ['web', 'discord', 'system'] })
 .notNull()
 .default('web'),
 ip: text('ip'),
 createdAt: integer('created_at').notNull().default(now),
 },
 (t) => [
 index('audit_actor_idx').on(t.actorId),
 index('audit_created_idx').on(t.createdAt),
 index('audit_action_idx').on(t.action),
],
);

```



**src/db/seed.sql**

```

-- ClanTek initial data.
-- Safe to re-run: every statement is INSERT OR IGNORE / idempotent.
--
-- npm run db:seed:local (local dev)
-- npm run db:seed:remote (production)

/* ----- *
 * Ranks ? ten to start. Add, rename, reorder, or delete any of these
 * from the admin portal; nothing in the code depends on these names.
 * sort_order 0 is the lowest rung.
 * ----- */

INSERT OR IGNORE INTO ranks (id, name, abbreviation, sort_order, req_days, req_wins, is_default)
VALUES
 (1, 'Recruit', 'RCT', 0, 0, 0, 1),
 (2, 'Private', 'PVT', 1, 14, 0, 0),
 (3, 'Corporal', 'CPL', 2, 30, 5, 0),
 (4, 'Sergeant', 'SGT', 3, 60, 15, 0),
 (5, 'Staff Sergeant', 'SSG', 4, 90, 30, 0),
 (6, 'Lieutenant', 'LT', 5, 120, 50, 0),
 (7, 'Captain', 'CPT', 6, 180, 75, 0),
 (8, 'Major', 'MAJ', 7, 240, 100, 0),
 (9, 'Colonel', 'COL', 8, 300, 125, 0),
 (10, 'General', 'GEN', 9, 365, 150, 0);

/* ----- *
 * Roles ? capability bundles, independent of the rank ladder.
 * Map each to a Discord role id from the admin portal to have membership
 * changes here push into Discord.
 * ----- */

INSERT OR IGNORE INTO roles (id, name, description, color, sort_order, is_system) VALUES
 (1, 'Command', 'Full administrative control of the site', '#c0392b', 100, 1),
 (2, 'Officer', 'Manage the roster, awards, and match records', '#e67e22', 80, 1),
 (3, 'Editor', 'Write and publish news', '#2980b9', 60, 0),
 (4, 'Recorder', 'Record match results', '#27ae60', 40, 0),
 (5, 'Member', 'Verified member of the clan', '#7f8c8d', 0, 1);

INSERT OR IGNORE INTO role_permissions (role_id, permission) VALUES
 -- Command
 (1, 'roster.view'), (1, 'roster.edit'), (1, 'roster.promote'), (1, 'roster.remove'),
 (1, 'roles.assign'), (1, 'roles.manage'), (1, 'ranks.manage'),
 (1, 'news.create'), (1, 'news.publish'), (1, 'news.delete'),
 (1, 'medals.award'), (1, 'medals.manage'),
 (1, 'games.manage'), (1, 'matches.record'), (1, 'matches.manage'),
 (1, 'warrecords.manage'), (1, 'warrecords.award'),
 (1, 'events.view'), (1, 'events.manage'), (1, 'events.attendees'),
 (1, 'theme.manage'), (1, 'pages.manage'), (1, 'settings.manage'),
 (1, 'audit.view'), (1, 'discord.sync'),
 -- Officer
 (2, 'roster.view'), (2, 'roster.edit'), (2, 'roster.promote'),
 (2, 'roles.assign'),
 (2, 'news.create'), (2, 'news.publish'),
 (2, 'medals.award'), (2, 'warrecords.award'),

```

**src/db/seed.sql**

```

(2, 'games.manage'), (2, 'matches.record'), (2, 'matches.manage'),
(2, 'events.view'), (2, 'events.manage'), (2, 'events.attendees'),
(2, 'audit.view'),
-- Editor
(3, 'roster.view'), (3, 'news.create'), (3, 'news.publish'), (3, 'events.view'),
-- Recorder
(4, 'roster.view'), (4, 'matches.record'), (4, 'events.view'),
-- Member
(5, 'roster.view'), (5, 'events.view');

/* ----- *
 * Founder account.
 *
 * Seeded ahead of first login so there is never a bootstrap window where
 * nobody can administer the site. The username and avatar below are
 * placeholders ? they are overwritten with real Discord data the first
 * time this account signs in.
 *
 * is_god = 1 bypasses every permission check and is not assignable from
 * the UI. To hand it to someone else, change it here and re-run.
 * ----- */

INSERT OR IGNORE INTO users (discord_id, username, global_name, rank_id, is_god, status)
VALUES ('161833822307090432', 'founder', 'Founder', 10, 1, 'active');

INSERT OR IGNORE INTO user_roles (user_id, role_id)
SELECT id, 1 FROM users WHERE discord_id = '161833822307090432';

/* ----- *
 * Site settings. Theme values are CSS custom properties applied at the
 * document root ? the modern replacement for the old templates/header
 * tables full of attributes and IE scrollbar colors.
 * ----- */

INSERT OR IGNORE INTO settings (key, value) VALUES
('site', json({'name':"ClanTek","tagline":"","description":"","copyright":""})),
('theme', json({'
 "--color-bg": "#0f1115",
 "--color-surface": "#171a21",
 "--color-border": "#262b36",
 "--color-text": "#e6e8ec",
 "--color-muted": "#9aa3b2",
 "--color-accent": "#c0392b",
 "--color-accent-text": "#ffffff",
 "--font-body": "system-ui, sans-serif",
 "--font-display": "system-ui, sans-serif",
 "--radius": "8px"
}')));

```

**src/server/auth/apiToken.ts**

```

/**
 * Personal access tokens ? the credential the mobile/native app (or a script)
 * uses. A member mints a labelled token in account settings; the raw value is
 * shown once, and only its SHA-256 is stored. It's presented on every API call
 * as `Authorization: Bearer clt_...` and resolves, via buildViewer, to the exact
 * same Viewer and permission set as that member's web session.
 */

import { and, eq, gt, isNull, or } from 'drizzle-orm';
import type { Drizzled1Database } from 'drizzle-orm/d1';
import * as s from '../db/schema';
import type { Viewer } from '../shared/permissions';
import { buildViewer } from './session';
import { randomToken, sha256Base64url } from './crypto';

type DB = Drizzled1Database<typeof s>;

/** Human-recognizable, namespaced prefix so a leaked token is easy to spot. */
const TOKEN_PREFIX = 'clt_';
const DEFAULT_TTL_SECONDS = 60 * 60 * 24 * 365; // 1 year
/** Only rewrite last_used_at at most this often, to avoid a write per request. */
const LAST_USED_THROTTLE_SECONDS = 60 * 60;

export interface CreatedToken {
 id: number;
 token: string; // the raw value ? shown to the caller exactly once
 label: string;
 prefix: string;
 expiresAt: number | null;
}

export async function createApiToken(
 db: DB,
 userId: number,
 label: string,
): Promise<CreatedToken> {
 const token = `${TOKEN_PREFIX}${randomToken()}`;
 const prefix = token.slice(0, TOKEN_PREFIX.length + 6); // e.g. clt_ab12cd
 const expiresAt = Math.floor(Date.now() / 1000) + DEFAULT_TTL_SECONDS;

 const inserted = await db
 .insert(s.apiTokens)
 .values({ userId, tokenHash: await sha256Base64url(token), label, prefix, expiresAt })
 .returning({ id: s.apiTokens.id });

 return { id: inserted[0]!.id, token, label, prefix, expiresAt };
}

/**
 * Resolve a bearer token to a Viewer, or null if it's unknown, expired, or the
 * user is gone/banned. Touches last_used_at at most hourly (best effort).
 */
export async function resolveApiToken(db: DB, raw: string | undefined): Promise<Viewer | null> {
 if (!raw || !raw.startsWith(TOKEN_PREFIX)) return null;

```

**src/server/auth/apiToken.ts**

```
const now = Math.floor(Date.now() / 1000);
const hash = await sha256Base64url(raw);
const row = await db.query.apiTokens.findFirst({
 where: and(
 eq(s.apiTokens.tokenHash, hash),
 // null expiry = never expires; otherwise must be in the future.
 or(isNull(s.apiTokens.expiresAt), gt(s.apiTokens.expiresAt, now)),
),
});
if (!row) return null;

const viewer = await buildViewer(db, row.userId);
if (!viewer) return null;

if (!row.lastUsedAt || now - row.lastUsedAt > LAST_USED_THROTTLE_SECONDS) {
 await db
 .update(s.apiTokens)
 .set({ lastUsedAt: now })
 .where(eq(s.apiTokens.id, row.id))
 .catch(() => {});
}

return viewer;
}
```

**src/server/auth/crypto.ts**

```
/**
 * Shared token crypto for sessions and API tokens. Raw tokens are random bytes
 * shown to the client once; only their SHA-256 is ever stored, so a database
 * dump can't be replayed as a live credential.
 */

export function base64url(bytes: Uint8Array): string {
 return btoa(String.fromCharCode(...bytes))
 .replace(/\+/g, '-')
 .replace(/\//g, '_')
 .replace(/=+$/, '');
}

export async function sha256Base64url(input: string): Promise<string> {
 const digest = await crypto.subtle.digest('SHA-256', new TextEncoder().encode(input));
 return base64url(new Uint8Array(digest));
}

/** A fresh random token as a base64url string (default 256 bits). */
export function randomToken(bytes = 32): string {
 const raw = new Uint8Array(bytes);
 crypto.getRandomValues(raw);
 return base64url(raw);
}
```

**src/server/auth/discord.ts**

```
/**
 * Discord OAuth2 authorization-code flow.
 *
 * There are no passwords in this system at all. Discord is the only identity
 * provider, which removes password storage, reset flows, and credential
 * stuffing from the threat model entirely.
 */

const DISCORD_API = 'https://discord.com/api/v10';

export const OAUTH_SCOPES = ['identify', 'email', 'guilds.members.read'] as const;

export const OAUTH_STATE_COOKIE = 'ct_oauth_state';

export interface DiscordUser {
 id: string;
 username: string;
 global_name: string | null;
 avatar: string | null;
 email?: string | null;
}

export interface DiscordGuildMember {
 roles: string[];
 nick: string | null;
 joined_at: string;
}

export function authorizeUrl(clientId: string, redirectUri: string, state: string): string {
 // No `prompt` param: Discord shows the consent screen on first authorization
 // and skips it on later logins. `prompt=none` would demand a pre-existing
 // grant and error out with consent_required for anyone who hasn't authorized
 // the app before ? i.e. every first-time login.
 const params = new URLSearchParams({
 client_id: clientId,
 redirect_uri: redirectUri,
 response_type: 'code',
 scope: OAUTH_SCOPES.join(' '),
 state,
 });
 return `${DISCORD_API}/oauth2/authorize?${params}`;
}

export function newState(): string {
 const bytes = new Uint8Array(16);
 crypto.getRandomValues(bytes);
 return [...bytes].map((b) => b.toString(16).padStart(2, '0')).join('');
}

export function stateCookie(state: string): string {
 // Short-lived; only has to survive the round trip to Discord and back.
 return `${OAUTH_STATE_COOKIE}=${state}; Path=/; HttpOnly; Secure; SameSite=Lax; Max-Age=600`;
}
```

**src/server/auth/discord.ts**

```

export const clearedStateCookie = `${OAUTH_STATE_COOKIE}=; Path=/; HttpOnly; Secure; SameSite=Lax;
Max-Age=0`;

export async function exchangeCode(
 clientId: string,
 clientSecret: string,
 code: string,
 redirectUri: string,
): Promise<{ access_token: string; token_type: string; expires_in: number }> {
 const res = await fetch(`${DISCORD_API}/oauth2/token`, {
 method: 'POST',
 headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
 body: new URLSearchParams({
 client_id: clientId,
 client_secret: clientSecret,
 grant_type: 'authorization_code',
 code,
 redirect_uri: redirectUri,
 }),
 });

 if (!res.ok) {
 throw new Error(`Discord token exchange failed (${res.status}): ${await res.text()}`);
 }
 return res.json();
}

export async function fetchUser(accessToken: string): Promise<DiscordUser> {
 const res = await fetch(`${DISCORD_API}/users/@me`, {
 headers: { Authorization: `Bearer ${accessToken}` },
 });
 if (!res.ok) throw new Error(`Discord /users/@me failed (${res.status})`);
 return res.json();
}

/**
 * The caller's membership in *your* guild. Returns null when they are not in
 * the server ? which is how "you must be in the Discord to log in" is enforced.
 */
export async function fetchGuildMember(
 accessToken: string,
 guildId: string,
): Promise<DiscordGuildMember | null> {
 const res = await fetch(`${DISCORD_API}/users/@me/guilds/${guildId}/member`, {
 headers: { Authorization: `Bearer ${accessToken}` },
 });
 if (res.status === 404) return null;
 if (!res.ok) throw new Error(`Discord guild member lookup failed (${res.status})`);
 return res.json();
}

export function avatarUrl(discordId: string, hash: string | null, size = 128): string {
 if (!hash) {
 const index = (BigInt(discordId) >> 22n) % 6n;

```

**src/server/auth/discord.ts**

```
 return `https://cdn.discordapp.com/embed/avatars/${index}.png`;
 }
 const ext = hash.startsWith('a_') ? 'gif' : 'png';
 return `https://cdn.discordapp.com/avatars/${discordId}/${hash}.${ext}?size=${size}`;
}
```



**src/server/auth/session.ts**

```

import { eq, lt } from 'drizzle-orm';
import type { DrizzleD1Database } from 'drizzle-orm/d1';
import * as s from '../db/schema';
import type { Permission, Viewer } from '../shared/permissions';
import { randomToken, sha256Base64url } from './crypto';

export const SESSION_COOKIE = 'ct_session';
const SESSION_TTL_SECONDS = 60 * 60 * 24 * 30; // 30 days

type DB = DrizzleD1Database<typeof s>;

export async function createSession(
 db: DB,
 userId: number,
 meta: { userAgent?: string | null; ip?: string | null } = {},
): Promise<{ token: string; expiresAt: number }> {
 const token = randomToken();
 const expiresAt = Math.floor(Date.now() / 1000) + SESSION_TTL_SECONDS;

 await db.insert(s.sessions).values({
 id: await sha256Base64url(token),
 userId,
 expiresAt,
 userAgent: meta.userAgent ?? null,
 ip: meta.ip ?? null,
 });

 return { token, expiresAt };
}

/**
 * Build the Viewer for a user id ? their rank plus the union of every permission
 * across all their roles. Shared by session and API-token auth so both credential
 * kinds resolve to exactly the same identity and permission set. Returns null for
 * a missing or banned user.
 */
export async function buildViewer(db: DB, userId: number): Promise<Viewer | null> {
 const user = await db.query.users.findFirst({ where: eq(s.users.id, userId) });
 if (!user || user.status === 'banned') return null;

 const rank = user.rankId
 ? ((await db.query.ranks.findFirst({ where: eq(s.ranks.id, user.rankId) })) ?? null)
 : null;

 const grants = await db
 .select({
 roleId: s.roles.id,
 roleName: s.roles.name,
 roleColor: s.roles.color,
 permission: s.rolePermissions.permission,
 })
 .from(s.userRoles)
 .innerJoin(s.roles, eq(s.userRoles.roleId, s.roles.id))
 .leftJoin(s.rolePermissions, eq(s.rolePermissions.roleId, s.roles.id))

```

**src/server/auth/session.ts**

```

 .where(eq(s.userRoles.userId, user.id));

 const roles = new Map<number, { id: number; name: string; color: string | null }>();
 const permissions = new Set<Permission>();
 for (const g of grants) {
 roles.set(g.roleId, { id: g.roleId, name: g.roleName, color: g.roleColor });
 if (g.permission) permissions.add(g.permission as Permission);
 }

 return {
 id: user.id,
 discordId: user.discordId,
 username: user.username,
 globalName: user.globalName,
 displayName: user.displayName,
 avatar: user.avatar,
 profileImageUrl: user.profileImageUrl,
 isGod: user.isGod,
 rank: rank ? { id: rank.id, name: rank.name, sortOrder: rank.sortOrder } : null,
 roles: [...roles.values()],
 permissions: [...permissions],
 };
}

/**
 * Resolves a session token (from the cookie) to a Viewer, expiring the row if
 * it's past its TTL.
 */
export async function resolveViewer(db: DB, token: string | undefined): Promise<Viewer | null> {
 if (!token) return null;

 const id = await sha256Base64url(token);
 const session = await db.query.sessions.findFirst({ where: eq(s.sessions.id, id) });
 if (!session) return null;

 if (session.expiresAt < Math.floor(Date.now() / 1000)) {
 await db.delete(s.sessions).where(eq(s.sessions.id, id));
 return null;
 }

 return buildViewer(db, session.userId);
}

export async function invalidateSession(db: DB, token: string): Promise<void> {
 await db.delete(s.sessions).where(eq(s.sessions.id, await sha256Base64url(token)));
}

/** Called opportunistically via ctx.waitUntil ? no need to block a request on it. */
export async function purgeExpiredSessions(db: DB): Promise<void> {
 await db.delete(s.sessions).where(lt(s.sessions.expiresAt, Math.floor(Date.now() / 1000)));
}

export function sessionCookie(token: string, maxAge: number): string {
 return [

```

**src/server/auth/session.ts**

```
 `${SESSION_COOKIE}=${token}`,
 'Path=/',
 'HttpOnly',
 'Secure',
 'SameSite=Lax',
 `Max-Age=${maxAge}`,
].join('; ');
}
```

```
export const clearedSessionCookie = `${SESSION_COOKIE}=; Path=;/; HttpOnly; Secure; SameSite=Lax;
Max-Age=0`;
```

**src/server/config.ts**

```

/**
 * Runtime configuration ? the layer that lets an operator reconfigure Discord
 * and site identity from the admin panel (or a first-run setup wizard) instead
 * of editing wrangler.jsonc and redeploying.
 *
 * Resolution is DB-over-env: a value saved in settings['identity'] wins; when it
 * is blank or unset, the wrangler var/secret in `env` is used. So a fresh,
 * env-only deploy keeps working, and the moment someone saves config in the UI
 * it takes effect without a redeploy. This is the seam that makes the product
 * deployable by other groups: they set their own Discord app + domain here.
 *
 * SECURITY: the stored blob can hold secrets (Discord client secret, bot token).
 * They live in D1, so they are readable by anything with DB access and by god
 * admins ? unlike wrangler secrets, which are encrypted and unreadable. That is
 * an accepted trade for self-serve configuration; the admin API never echoes a
 * secret value back to a client (presence only). See settings.get('/identity').
 */

import { drizzle } from 'drizzle-orm/d1';
import { eq } from 'drizzle-orm';
import * as schema from '../db/schema';
import * as s from '../db/schema';
import type { Env } from './env';
import { DiscordRest } from './discord/rest';

/** settings key under which the identity/Discord blob is stored. */
export const IDENTITY_KEY = 'identity';

export interface DiscordConfig {
 clientId: string;
 guildId: string;
 publicKey: string;
 clientSecret: string;
 botToken: string;
}

export interface AppConfig {
 siteName: string;
 siteUrl: string;
 discord: DiscordConfig;
}

/** The shape persisted in settings['identity']. Every field optional ? a blank
 * or missing value falls through to the matching env var in loadConfig(). */
export interface StoredIdentity {
 siteName?: string;
 siteUrl?: string;
 discord?: Partial<DiscordConfig>;
}

type DB = ReturnType<typeof drizzle<typeof schema>>;

/** First non-empty (after trim) of the candidates, else ''. */
function firstNonEmpty(...vals: (string | undefined | null)[]): string {

```

**src/server/config.ts**

```

 for (const v of vals) {
 if (typeof v === 'string' && v.trim() !== '') return v.trim();
 }
 return '';
}

/**
 * Resolve the effective configuration: DB overrides (settings['identity']) first,
 * then the env fallbacks. Never throws ? a missing row or table (first boot)
 * degrades cleanly to "env only".
 */
export async function loadConfig(env: Env, database?: DB): Promise<AppConfig> {
 const dbi = database ?? drizzle(env.DB, { schema });
 let stored: StoredIdentity = {};
 try {
 const row = await dbi.query.settings.findFirst({ where: eq(s.settings.key, IDENTITY_KEY) });
 if (row?.value && typeof row.value === 'object') stored = row.value as StoredIdentity;
 } catch {
 // No settings row/table yet (fresh install) -> fall back to env entirely.
 }
 const d = stored.discord ?? {};
 return {
 siteName: firstNonEmpty(stored.siteName, env.SITE_NAME, 'ClanTek'),
 siteUrl: firstNonEmpty(stored.siteUrl, env.SITE_URL),
 discord: {
 clientId: firstNonEmpty(d.clientId, env.DISCORD_CLIENT_ID),
 guildId: firstNonEmpty(d.guildId, env.DISCORD_GUILD_ID),
 publicKey: firstNonEmpty(d.publicKey, env.DISCORD_PUBLIC_KEY),
 clientSecret: firstNonEmpty(d.clientSecret, env.DISCORD_CLIENT_SECRET),
 botToken: firstNonEmpty(d.botToken, env.DISCORD_BOT_TOKEN),
 },
 };
}

/** Build a bot REST client from resolved config, or null when the bot isn't set up. */
export function discordRestFromConfig(cfg: AppConfig): DiscordRest | null {
 if (!cfg.discord.botToken || !cfg.discord.guildId) return null;
 return new DiscordRest(cfg.discord.botToken, cfg.discord.guildId);
}

/**
 * Resolve config and build the bot client in one step. Replaces the ~nine
 * duplicated `env.DISCORD_BOT_TOKEN && env.DISCORD_GUILD_ID ? new DiscordRest(...)`
 * blocks, so every surface (web routes, cron, background) reads the same config.
 */
export async function discordClient(env: Env, database?: DB): Promise<DiscordRest | null> {
 return discordRestFromConfig(await loadConfig(env, database));
}

/** Thrown by mergeStoredIdentity on malformed input; the route maps it to 400. */
export class ConfigValidationError extends Error {}

/** Raw client payload for a config save ? every field untrusted. */
export interface StoredIdentityInput {

```

**src/server/config.ts**

```

siteName?: unknown;
siteUrl?: unknown;
discord?: {
 clientId?: unknown;
 guildId?: unknown;
 publicKey?: unknown;
 clientSecret?: unknown;
 botToken?: unknown;
};
}

function trimmed(v: unknown, max: number): string {
 return typeof v === 'string' ? v.trim().slice(0, max) : '';
}

/**
 * Validate a config update and merge it onto what's stored. Semantics:
 * - Non-secret fields are stored as given; a blank clears the override (so the
 * env value shows through again). Blanks are harmless ? loadConfig() skips
 * empty strings.
 * - Secret fields (clientSecret, botToken) are write-only from the UI: the form
 * only ever knows whether one is set, never its value. A non-empty value
 * replaces the stored secret; a blank LEAVES THE EXISTING SECRET INTACT.
 * Malformed non-empty ids/keys throw ConfigValidationError rather than silently
 * corrupting the config.
 */
export function mergeStoredIdentity(
 existing: StoredIdentity,
 incoming: StoredIdentityInput,
): StoredIdentity {
 const inD = incoming.discord ?? {};
 const exD = existing.discord ?? {};

 const siteName = trimmed(incoming.siteName, 80);
 const siteUrl = trimmed(incoming.siteUrl, 200);
 if (siteUrl && !/^https?:\\\/\\\/i.test(siteUrl)) {
 throw new ConfigValidationError('Site URL must start with http:// or https://.');
 }

 const clientId = trimmed(inD.clientId, 40);
 const guildId = trimmed(inD.guildId, 40);
 const publicKey = trimmed(inD.publicKey, 100);

 const snowflakeOk = (v: string) => v === '' || /^\\d{5,25}$/.test(v);
 if (!snowflakeOk(clientId)) throw new ConfigValidationError('Discord Client ID must be numeric.');
```

numeric.');

```

 if (!snowflakeOk(guildId)) throw new ConfigValidationError('Discord Server (Guild) ID must be
numeric.');
```

numeric.');

```

 if (!(publicKey === '' || /^[0-9a-fA-F]{64}$/.test(publicKey))) {
 throw new ConfigValidationError('Discord Public Key must be 64 hexadecimal characters.');
```

}

```

// Secrets: only overwrite when a fresh non-empty value arrives.
const clientSecretIn = trimmed(inD.clientSecret, 200);
```

**src/server/config.ts**

```

const botTokenIn = trimmed(inD.botToken, 200);

return {
 siteName,
 siteUrl,
 discord: {
 clientId,
 guildId,
 publicKey,
 clientSecret: clientSecretIn || exD.clientSecret || '',
 botToken: botTokenIn || exD.botToken || '',
 },
};
}

/* ----- *
 * Analytics ? the Cloudflare account/token/script/db the usage dashboard
 * queries for live request & query rates. Same DB-over-env pattern as
 * identity, so an operator can wire up (or fix) their own analytics from the
 * admin panel instead of `wrangler secret put` + redeploy.
 * ----- */

/** settings key under which the analytics blob is stored. */
export const ANALYTICS_KEY = 'analytics';

// Fallback defaults for this install ? the worker name and D1 database id from
// wrangler.jsonc. Another group's deploy overrides these in the Analytics panel.
const DEFAULT_SCRIPT_NAME = 'clantek';
const DEFAULT_D1_DATABASE_ID = '19d7ebe6-e379-411b-8597-17b630cb8394';

export interface AnalyticsConfig {
 /** Cloudflare account id (not secret ? appears in dashboard URLs). */
 accountId: string;
 /** Read-only "Account Analytics: Read" API token (secret). */
 apiToken: string;
 /** Worker script name whose invocations are counted. */
 scriptName: string;
 /** D1 database id whose rows-read/written are counted. */
 d1DatabaseId: string;
}

/** Persisted shape in settings['analytics']. Blank/missing -> env or default. */
export interface StoredAnalytics {
 accountId?: string;
 apiToken?: string;
 scriptName?: string;
 d1DatabaseId?: string;
}

/** Resolve analytics config: DB overrides first, then env vars, then defaults. */
export async function loadAnalytics(env: Env, database?: DB): Promise<AnalyticsConfig> {
 const dbi = database ?? drizzle(env.DB, { schema });
 let stored: StoredAnalytics = {};
 try {

```

**src/server/config.ts**

```

 const row = await dbi.query.settings.findFirst({ where: eq(s.settings.key, ANALYTICS_KEY) });
 if (row?.value && typeof row.value === 'object') stored = row.value as StoredAnalytics;
 } catch {
 // No settings row/table yet -> env/defaults only.
 }
 return {
 accountId: firstNonEmpty(stored.accountId, env.CLOUDFLARE_ACCOUNT_ID),
 apiToken: firstNonEmpty(stored.apiToken, env.CLOUDFLARE_API_TOKEN),
 scriptName: firstNonEmpty(stored.scriptName, DEFAULT_SCRIPT_NAME),
 d1DatabaseId: firstNonEmpty(stored.d1DatabaseId, DEFAULT_D1_DATABASE_ID),
 };
}

/** Raw client payload for an analytics save ? every field untrusted. */
export interface StoredAnalyticsInput {
 accountId?: unknown;
 apiToken?: unknown;
 scriptName?: unknown;
 d1DatabaseId?: unknown;
}

/**
 * Validate and merge an analytics update. Non-secret fields store as given
 * (blank clears the override); the apiToken is write-only (blank keeps existing).
 */
export function mergeStoredAnalytics(
 existing: StoredAnalytics,
 incoming: StoredAnalyticsInput,
): StoredAnalytics {
 const accountId = trimmed(incoming.accountId, 40);
 const scriptName = trimmed(incoming.scriptName, 64);
 const d1DatabaseId = trimmed(incoming.d1DatabaseId, 64);
 const apiTokenIn = trimmed(incoming.apiToken, 200);

 if (accountId && !/^[0-9a-fA-F]{32}$/.test(accountId)) {
 throw new ConfigValidationError('Cloudflare Account ID must be 32 hexadecimal characters.');
```



**src/server/discord/announce.ts**

```

/**
 * Discord announcements.
 *
 * When something worth celebrating happens on the site ? a medal or war record
 * awarded, a promotion ? the bot posts an embed to a configured channel and
 * pings the member. Everything here is best-effort and fire-and-forget: an
 * announcement failure (misconfigured channel, missing permission, rate limit)
 * must never affect the action that triggered it, so callers invoke this inside
 * ctx.waitUntil and it swallows its own errors.
 *
 * Which events fire, and where, is set in the admin panel (settings key
 * 'announcements'); nothing posts until a channel is chosen and the event is
 * enabled.
 */

import { drizzle } from 'drizzle-orm/d1';
import { eq } from 'drizzle-orm';
import * as s from '../db/schema';
import type { Env } from '../env';
import { type Embed } from './rest';
import { discordClient } from '../config';

export type AnnouncementEventKey = 'medalAward' | 'warRecordAward' | 'promotion';

export interface AnnouncementConfig {
 channelId: string | null;
 events: Record<AnnouncementEventKey, boolean>;
}

export const DEFAULT_ANNOUNCEMENTS: AnnouncementConfig = {
 channelId: null,
 events: { medalAward: false, warRecordAward: false, promotion: false },
};

type DB = ReturnType<typeof drizzle<typeof s>>;

export async function loadAnnouncementConfig(db: DB): Promise<AnnouncementConfig> {
 const row = await db.query.settings.findFirst({ where: eq(s.settings.key, 'announcements') });
 const stored = (row?.value as Partial<AnnouncementConfig> | undefined) ?? {};
 return {
 channelId: stored.channelId ?? null,
 events: { ...DEFAULT_ANNOUNCEMENTS.events, ...(stored.events ?? {}) },
 };
}

interface MemberRef {
 memberName: string;
 memberDiscordId: string;
 memberAvatarUrl: string;
}

export type AnnounceEvent =
 | (MemberRef & { type: 'medalAward'; medalName: string; medalImageUrl: string | null; citation:
string | null })

```

**src/server/discord/announce.ts**

```

 | (MemberRef & {
 type: 'warRecordAward';
 recordName: string;
 recordImageUrl: string | null;
 gameName: string | null;
 citation: string | null;
 })
 | (MemberRef & { type: 'promotion'; rankName: string; rankImageUrl: string | null; byName:
string | null });

const COLORS = { medalAward: 0xc0392b, warRecordAward: 0xd4af37, promotion: 0x2f9e5f };

/** Absolute-ise a stored /media/ URL so Discord can fetch it; pass through http(s) URLs. */
function absolute(url: string | null, baseUrl?: string): string | undefined {
 if (!url) return undefined;
 if (/^https?:\/\//i.test(url)) return url;
 return baseUrl ? `${baseUrl}${url}` : undefined;
}

function buildEmbed(event: AnnounceEvent, baseUrl?: string): Embed {
 const author = { name: event.memberName, icon_url: event.memberAvatarUrl };
 switch (event.type) {
 case 'medalAward':
 return {
 author,
 color: COLORS.medalAward,
 title: '?? Medal awarded',
 description: `**${event.memberName}** was awarded the **${event.medalName}** medal.${{
 event.citation ? `\n"${event.citation}"` : ''
 }}`,
 thumbnail: absolute(event.medalImageUrl, baseUrl)
 ? { url: absolute(event.medalImageUrl, baseUrl)! }
 : undefined,
 };
 case 'warRecordAward':
 return {
 author,
 color: COLORS.warRecordAward,
 title: '? War record awarded',
 description: `**${event.memberName}** earned the **${event.recordName}** war record${{
 event.gameName ? ` (${event.gameName})` : ''
 }}.${{
 event.citation ? `\n"${event.citation}"` : ''
 }}`,
 thumbnail: absolute(event.recordImageUrl, baseUrl)
 ? { url: absolute(event.recordImageUrl, baseUrl)! }
 : undefined,
 };
 case 'promotion':
 return {
 author,
 color: COLORS.promotion,
 title: '?? Promotion',
 description: `**${event.memberName}** was promoted to **${event.rankName}**.${{
 event.byName ? `\n_by ${event.byName}_` : ''
 }}`,
 };
 }
}

```

**src/server/discord/announce.ts**

```
 thumbnail: absolute(event.rankImageUrl, baseUrl)
 ? { url: absolute(event.rankImageUrl, baseUrl)! }
 : undefined,
 };
}
}

/**
 * Post an announcement if the event is enabled and a channel is configured.
 * Safe to call unconditionally inside ctx.waitUntil ? it no-ops without a bot,
 * channel, or when the event is switched off, and never throws.
 */
export async function announce(env: Env, event: AnnounceEvent, baseUrl?: string): Promise<void> {
 try {
 const db = drizzle(env.DB, { schema: s });
 const rest = await discordClient(env, db);
 if (!rest) return;
 const config = await loadAnnouncementConfig(db);
 if (!config.channelId || !config.events[event.type]) return;

 await rest.createMessage(config.channelId, {
 content: `<@${event.memberDiscordId}>`,
 embeds: [buildEmbed(event, baseUrl)],
 allowed_mentions: { users: [event.memberDiscordId] },
 });
 } catch (err) {
 console.error('Announcement failed', err);
 }
}
```

**src/server/discord/commands.ts**

```

/**
 * Slash command handlers.
 *
 * Every command resolves the invoking Discord user to a ClanTek member and
 * runs the same permission checks the web portal uses. There is no separate
 * "Discord admin" concept ? one identity, one permission model, both surfaces.
 */

import { asc, desc, eq, sql } from 'drizzle-orm';
import { drizzle } from 'drizzle-orm/d1';
import * as schema from '../../db/schema';
import * as s from '../../db/schema';
import { can, outranks, type Viewer, type Permission } from '../../shared/permissions';
import type { Env } from '../../env';
import { discordClient } from '../../config';
import { syncMemberRankRoles } from './sync';
import { announce } from './announce';
import { memberName } from '../../shared/names';
import { discordAvatar } from '../../shared/avatar';
import { ephemeral, invoker, optionValue, reply, type Interaction } from './interactions';

type DB = ReturnType<typeof drizzle<typeof schema>>;

/** Same Viewer shape the web app uses, resolved from a Discord snowflake. */
async function viewerFromDiscordId(db: DB, discordId: string): Promise<Viewer | null> {
 const user = await db.query.users.findFirst({ where: eq(s.users.discordId, discordId) });
 if (!user || user.status === 'banned') return null;

 const rank = user.rankId
 ? ((await db.query.ranks.findFirst({ where: eq(s.ranks.id, user.rankId) })) ?? null)
 : null;

 const grants = await db
 .select({ roleId: s.roles.id, name: s.roles.name, permission: s.rolePermissions.permission })
 .from(s.userRoles)
 .innerJoin(s.roles, eq(s.userRoles.roleId, s.roles.id))
 .leftJoin(s.rolePermissions, eq(s.rolePermissions.roleId, s.roles.id))
 .where(eq(s.userRoles.userId, user.id));

 const roles = new Map<number, { id: number; name: string; color: null }>();
 const permissions = new Set<Permission>();
 for (const g of grants) {
 roles.set(g.roleId, { id: g.roleId, name: g.name, color: null });
 if (g.permission) permissions.add(g.permission as Permission);
 }

 return {
 id: user.id,
 discordId: user.discordId,
 username: user.username,
 globalName: user.globalName,
 displayName: user.displayName,
 avatar: user.avatar,
 profileImageUrl: user.profileImageUrl,
 };
}

```

**src/server/discord/commands.ts**

```

 isGod: user.isGod,
 rank: rank ? { id: rank.id, name: rank.name, sortOrder: rank.sortOrder } : null,
 roles: [...roles.values()],
 permissions: [...permissions],
 };
}

export async function handleCommand(env: Env, i: Interaction, baseUrl?: string) {
 const db = drizzle(env.DB, { schema });
 const caller = invoker(i);
 if (!caller) return ephemeral('Could not identify you.');
```

const viewer = await viewerFromDiscordId(db, caller.id);
 if (!viewer) {
 return ephemeral('You do not have a ClanTek account yet ? sign in on the website once first.');

switch (i.data?.name) {
 case 'whois':
 return whois(db, i);
 case 'roster':
 return roster(db);
 case 'promote':
 return promote(env, db, viewer, i, baseUrl);
 default:
 return ephemeral(`Unknown command: \${i.data?.name}`);
 }
 }

 async function whois(db: DB, i: Interaction) {
 const targetId = optionValue<string>(i, 'member');
 if (!targetId) return ephemeral('Specify a member.');

const user = await db.query.users.findFirst({ where: eq(s.users.discordId, targetId) });
 if (!user) return ephemeral('That person has no ClanTek account.');

const rank = user.rankId
 ? await db.query.ranks.findFirst({ where: eq(s.ranks.id, user.rankId) })
 : null;

const roles = await db
 .select({ name: s.roles.name })
 .from(s.userRoles)
 .innerJoin(s.roles, eq(s.userRoles.roleId, s.roles.id))
 .where(eq(s.userRoles.userId, user.id));

const medalCount = await db
 .select({ n: sql<number>`count(\*)` })
 .from(s.memberMedals)
 .where(eq(s.memberMedals.userId, user.id));

return reply(
 [

**src/server/discord/commands.ts**

```

 `**${memberName(user)}**`,
 `Rank: ${rank?.name ?? '?'}`,
 `Roles: ${roles.length ? roles.map((r) => r.name).join(', ') : '?'}`,
 `Medals: ${medalCount[0]?.n ?? 0}`,
 `Status: ${user.status}`,
 `Joined: <t:${user.joinedAt}:D>`,
].join('\n'),
);
}

async function roster(db: DB) {
 const rows = await db
 .select({ rank: s.ranks.name, order: s.ranks.sortOrder, n: sql<number>`count(${s.users.id})`
 })
 .from(s.ranks)
 .leftJoin(s.users, eq(s.users.rankId, s.ranks.id))
 .groupBy(s.ranks.id)
 .orderBy(desc(s.ranks.sortOrder));

 const total = rows.reduce((sum, r) => sum + Number(r.n), 0);
 const lines = rows.map((r) => `${r.rank}: ${r.n}`);
 return reply(`**Roster ? ${total} members**\n${lines.join('\n')}`);
}

async function promote(env: Env, db: DB, viewer: Viewer, i: Interaction, baseUrl?: string) {
 if (!can(viewer, 'roster.promote')) {
 return ephemeral('You do not have permission to promote members.');
```

**src/server/discord/commands.ts**

```

const nowSec = Math.floor(Date.now() / 1000);
await db
 .update(s.users)
 .set({ rankId: nextRank.id, promotedAt: nowSec, updatedAt: nowSec })
 .where(eq(s.users.id, target.id));

await db.insert(s.auditLog).values({
 actorId: viewer.id,
 action: 'member.promote',
 targetType: 'user',
 targetId: String(target.id),
 meta: { from: currentRank?.name ?? null, to: nextRank.name },
 source: 'discord',
});

// Apply the new rank's roles and reflect them into Discord.
const client = await discordClient(env, db);
const rankRoleSync = await syncMemberRankRoles(db, client, {
 userId: target.id,
 rankId: nextRank.id,
 actorId: viewer.id,
});

const roleNote = rankRoleSync.added.length
 ? `
 \nRoles added: ${rankRoleSync.added.join(', ')}
 `
 : '';

await announce(
 env,
 {
 type: 'promotion',
 memberName: memberName(target),
 memberDiscordId: target.discordId,
 memberAvatarUrl: discordAvatar(target.discordId, target.avatar, 128),
 rankName: nextRank.name,
 rankImageUrl: nextRank.imageUrl,
 byName: memberName(viewer),
 },
 baseUrl,
);

return reply(
 `**${memberName(target)}** promoted to **${nextRank.name}**` +
 (currentRank ? ` (from ${currentRank.name})` : '') +
 roleNote,
);
}

```

**src/server/discord/eventInteractions.ts**

```

/**
 * Handles clicks on the event sign-up buttons (MESSAGE_COMPONENT interactions).
 *
 * custom_id forms (built in discord/events.ts):
 * evt:role:<eventId>:<roleId> -> attend as that role
 * evt:role:<eventId>:none -> attend with no specific role
 * evt:withdraw:<eventId> -> withdraw
 *
 * Runs after a deferred-update ack (index.ts): it writes the same event_signups
 * rows the website uses, refreshes the source message with new counts, and
 * sends the clicker a private confirmation. Never throws ? a failure just means
 * the click had no visible effect, which the member can retry.
 */

import { drizzle } from 'drizzle-orm/d1';
import { eq } from 'drizzle-orm';
import * as schema from '../db/schema';
import * as s from '../db/schema';
import type { Env } from '../env';
import { loadConfig } from '../config';
import { setSignup, removeSignup, loadEventState } from '../events/signups';
import { buildEventMessage } from '../events';
import { editOriginalMessage, followUpEphemeral, type Interaction } from '../interactions';

/** True if this component interaction is one of our event buttons. */
export function isEventComponent(i: Interaction): boolean {
 return !!i.data?.custom_id?.startsWith('evt:');
}

export async function handleEventComponent(env: Env, i: Interaction): Promise<void> {
 const customId = i.data?.custom_id;
 if (!customId?.startsWith('evt:')) return;

 const [, kind, eventIdStr, arg] = customId.split(':');
 const eventId = Number(eventIdStr);
 if (!Number.isFinite(eventId)) return;

 const db = drizzle(env.DB, { schema });
 const discordId = (i.member?.user ?? i.user)?.id;
 if (!discordId) return;

 const user = await db.query.users.findFirst({ where: eq(s.users.discordId, discordId) });

 let note: string;
 if (!user) {
 note = 'You need a ClanTek account first ? sign in on the website once, then try again.';
 } else if (user.status === 'banned') {
 note = 'Your account is not active.';
 } else if (kind === 'withdraw') {
 await removeSignup(db, eventId, user.id);
 note = 'You've withdrawn from this event.';
 } else if (kind === 'role') {
 const roleId = arg === 'none' ? null : Number(arg);
 const res = await setSignup(db, eventId, user.id, roleId);
 }
}

```



**src/server/discord/eventInteractions.ts**

```
 note = res.ok ? '? You're signed up.' : `?? ${res.error}`;
 } else {
 note = 'Unknown action.';
 }

 // Refresh the sign-up message with new counts (edits the button's own message).
 try {
 const state = await loadEventState(db, eventId);
 if (state) {
 const { siteUrl } = await loadConfig(env, db);
 await editOriginalMessage(i.application_id, i.token, buildEventMessage(state, siteUrl));
 }
 } catch (err) {
 console.error('Event component message refresh failed', err);
 }

 // Private confirmation to the clicker.
 try {
 await followUpEphemeral(i.application_id, i.token, note);
 } catch (err) {
 console.error('Event component follow-up failed', err);
 }
}
```

**src/server/discord/events.ts**

```

/**
 * Mirrors a ClanTek event into Discord: a native guild scheduled event (so
 * members get reminders) AND an announcement message that doubles as the
 * sign-up sheet ? an embed with live per-role counts plus a row of buttons
 * (one per role, "Attending", "Withdraw"). Clicking a button is a
 * MESSAGE_COMPONENT interaction handled in discord/eventInteractions.ts, which
 * writes the same event_signups rows the website uses, so the two stay in sync.
 *
 * Everything here is best-effort ? a Discord failure is logged and swallowed,
 * never blocking the site action. The announcement channel is the one the
 * medal/promotion announcements use (settings key 'announcements'); the native
 * scheduled event needs the bot's MANAGE_EVENTS permission.
 */

import { drizzle } from 'drizzle-orm/d1';
import * as s from '../db/schema';
import type { Env } from '../env';
import { type ActionRow, type Embed } from './rest';
import { loadConfig, discordRestFromConfig } from '../config';
import { loadAnnouncementConfig } from './announce';
import { loadEventState, type EventState } from '../events/signups';

type EventRow = typeof s.events.$inferSelect;

const iso = (unixSec: number) => new Date(unixSec * 1000).toISOString();

/** base64-encode raw bytes (chunked so a ~1 MB image doesn't blow the call stack). */
function bytesToBase64(buf: ArrayBuffer): string {
 const bytes = new Uint8Array(buf);
 let binary = '';
 const chunk = 0x8000;
 for (let i = 0; i < bytes.length; i += chunk) {
 binary += String.fromCharCode(...bytes.subarray(i, i + chunk));
 }
 return btoa(binary);
}

/** Turn a stored relative media URL into an absolute one Discord can fetch. */
function absoluteMedia(siteUrl: string | undefined, url: string | null): string | null {
 if (!url) return null;
 if (/^https?:\/\//.test(url)) return url;
 if (!siteUrl) return null;
 return `${siteUrl.replace(/\/$/, ' ')}${url.startsWith('/') ? '' : '/'}${url}`;
}

/**
 * The event banner as a data URI, for the scheduled event's cover image ?
 * Discord stores the bytes rather than fetching a URL, so we read the object
 * out of R2 (a binding call, not a subrequest) and base64 it. Best-effort:
 * any problem just means no cover image. Falls back to fetching the public URL
 * if the media binding isn't available.
 */
async function eventCoverDataUri(
 env: Env,

```

**src/server/discord/events.ts**

```

 imageUrl: string | null,
 siteUrl?: string,
): Promise<string | undefined> {
 if (!imageUrl) return undefined;
 try {
 let bytes: ArrayBuffer;
 let contentType = 'image/png';
 if (env.MEDIA && imageUrl.startsWith('/media/')) {
 const key = imageUrl.replace(/^\/media\/\//, '');
 const obj = await env.MEDIA.get(key);
 if (!obj) return undefined;
 bytes = await obj.arrayBuffer();
 contentType = obj.httpMetadata?.contentType || contentType;
 } else {
 const abs = absoluteMedia(siteUrl, imageUrl);
 if (!abs) return undefined;
 const res = await fetch(abs);
 if (!res.ok) return undefined;
 bytes = await res.arrayBuffer();
 contentType = res.headers.get('content-type') || contentType;
 }
 return `data:${contentType};base64,${bytesToBase64(bytes)}`;
 } catch (err) {
 console.error('Event cover image load failed', err);
 return undefined;
 }
 }
}

const RECURRENCE_LABEL: Record<string, string> = {
 daily: 'Daily',
 weekly: 'Weekly',
 biweekly: 'Every 2 weeks',
 monthly: 'Monthly',
};

function eventEmbed(state: EventState, siteUrl?: string): Embed {
 const { event, gameName } = state;
 const lines = [
 event.description || undefined,
 `**When:** <t:${event.startsAt}:F> (<t:${event.startsAt}:R>)\``,
 `**Where:** ${event.location}\``,
 gameName ? `**Game:** ${gameName}\` : undefined,
 event.recurrence && event.recurrence !== 'none'
 ? `**Repeats:** ${RECURRENCE_LABEL[event.recurrence] ?? event.recurrence}\`
 : undefined,
].filter(Boolean) as string[];

 const fields: NonNullable<Embed['fields']> = [];
 for (const r of state.roles) {
 const names = state.signups.filter((su) => su.roleId === r.id).map((su) => su.name);
 fields.push({
 name: `${r.emoji ? r.emoji + ' ' : ''}${r.name} ? ${r.count}${r.capacity != null ?
`/${r.capacity}` : ''}.slice(0, 256),
 value: (names.length ? names.join(', ') : '?').slice(0, 1024),
 });
 }
}

```

**src/server/discord/events.ts**

```

 inline: true,
 });
}
const noRole = state.signups.filter((su) => su.roleId === null).map((su) => su.name);
if (noRole.length) {
 fields.push({ name: `Attending ? ${noRole.length}`, value: noRole.join(', ').slice(0, 1024),
inline: true });
}

const embed: Embed = {
 color: 0x5865f2,
 title: `? ${event.title}`,
 description: lines.join('\n'),
 footer: { text: `${state.total} signed up` },
};
if (fields.length) embed.fields = fields;
const img = absoluteMedia(siteUrl, event.imageUrl);
if (img) embed.image = { url: img };
return embed;
}

/**
 * The sign-up buttons: one per role (secondary, showing its count), then a
 * final row with "Attending" (join with no specific role) and "Withdraw".
 * Discord caps a message at 5 action rows of 5 buttons; role buttons take up to
 * four rows (20 roles) and the controls take the fifth.
 */
function eventComponents(state: EventState): ActionRow[] {
 const eid = state.event.id;
 const rows: ActionRow[] = [];

 const roleButtons = state.roles.slice(0, 20).map((r) => {
 const count = r.capacity != null ? `${r.count}/${r.capacity}` : r.count ? ` (${r.count})` :
'';
 const button: ActionRow['components'][number] = {
 type: 2,
 style: 2,
 label: `${r.name}${count}.slice(0, 80),
 custom_id: `evt:role:${eid}:${r.id}`,
 };
 if (r.emoji) button.emoji = { name: r.emoji };
 return button;
 });
 for (let i = 0; i < roleButtons.length; i += 5) {
 rows.push({ type: 1, components: roleButtons.slice(i, i + 5) });
 }

 rows.push({
 type: 1,
 components: [
 { type: 2, style: 1, label: 'Attending', custom_id: `evt:role:${eid}:none` },
 { type: 2, style: 4, label: 'Withdraw', custom_id: `evt:withdraw:${eid}` },
],
 });
}

```

**src/server/discord/events.ts**

```

 return rows;
}

/** The full announcement message payload for an event's current state. */
export function buildEventMessage(
 state: EventState,
 siteUrl?: string,
): { embeds: Embed[]; components: ActionRow[] } {
 return { embeds: [eventEmbed(state, siteUrl)], components: eventComponents(state) };
}

export interface DiscordEventIds {
 discordEventId: string | null;
 discordMessageId: string | null;
}

/**
 * Create or update the Discord scheduled event and announcement message for an
 * event. Reuses the ids already on the row when present (an edit). Returns the
 * ids to persist. Never throws.
 */
export async function syncEventToDiscord(env: Env, eventId: number): Promise<DiscordEventIds> {
 const db = drizzle(env.DB, { schema: s });
 const state = await loadEventState(db, eventId);
 if (!state) return { discordEventId: null, discordMessageId: null };

 const ids: DiscordEventIds = {
 discordEventId: state.event.discordEventId,
 discordMessageId: state.event.discordMessageId,
 };
 const cfg = await loadConfig(env, db);
 const rest = discordRestFromConfig(cfg);
 if (!rest) return ids;

 const event = state.event;

 const scheduled = {
 name: event.title.slice(0, 100),
 description: event.description?.slice(0, 1000) || undefined,
 scheduled_start_time: iso(event.startsAt),
 scheduled_end_time: iso(event.endsAt),
 privacy_level: 2 as const,
 entity_type: 3 as const,
 entity_metadata: { location: event.location.slice(0, 100) },
 image: await eventCoverDataUri(env, event.imageUrl, cfg.siteUrl),
 };

 try {
 if (ids.discordEventId) {
 await rest.modifyScheduledEvent(ids.discordEventId, scheduled);
 } else {
 ids.discordEventId = await rest.createScheduledEvent(scheduled);
 }
 } catch (err) {

```

**src/server/discord/events.ts**

```

 console.error('Scheduled-event sync failed', err);
 }

 try {
 const channelId = (await loadAnnouncementConfig(db)).channelId;
 if (channelId) {
 const message = buildEventMessage(state, cfg.siteUrl);
 if (ids.discordMessageId) {
 await rest.editMessage(channelId, ids.discordMessageId, message);
 } else {
 ids.discordMessageId = await rest.createMessage(channelId, message);
 }
 }
 } catch (err) {
 console.error('Event announcement failed', err);
 }

 return ids;
}

/**
 * Re-render just the announcement message (embed counts + button labels) after
 * a sign-up changes on the website. No scheduled-event touch. Never throws.
 */
export async function refreshEventMessage(env: Env, eventId: number): Promise<void> {
 const db = drizzle(env.DB, { schema: s });
 const cfg = await loadConfig(env, db);
 const rest = discordRestFromConfig(cfg);
 if (!rest) return;
 const state = await loadEventState(db, eventId);
 if (!state || !state.event.discordMessageId) return;

 try {
 const channelId = (await loadAnnouncementConfig(db)).channelId;
 if (channelId) {
 await rest.editMessage(channelId, state.event.discordMessageId, buildEventMessage(state,
cfg.siteUrl));
 }
 } catch (err) {
 console.error('Event message refresh failed', err);
 }
}

/** Remove an event's Discord scheduled event and announcement message. Never throws. */
export async function removeEventFromDiscord(env: Env, event: EventRow): Promise<void> {
 const rest = discordRestFromConfig(await loadConfig(env));
 if (!rest) return;

 if (event.discordEventId) {
 try {
 await rest.deleteScheduledEvent(event.discordEventId);
 } catch (err) {
 console.error('Scheduled-event delete failed', err);
 }
 }
}

```

**src/server/discord/events.ts**

```
}
if (event.discordMessageId) {
 try {
 const db = drizzle(env.DB, { schema: s });
 const channelId = (await loadAnnouncementConfig(db)).channelId;
 if (channelId) await rest.deleteMessage(channelId, event.discordMessageId);
 } catch (err) {
 console.error('Event message delete failed', err);
 }
}
```

**src/server/discord/interactions.ts**

```

/**
 * Discord slash commands, served over HTTP Interactions.
 *
 * Discord POSTs to a single "Interactions Endpoint URL" you register on the
 * application ? no gateway WebSocket, which is exactly what makes this work on
 * Workers. The tradeoff worth knowing up front:
 *
 * Slash commands and component clicks -> work over HTTP, supported here.
 * Passive gateway events (member joins, message sent, role changed in
 * Discord's own UI) -> require a persistent WebSocket
 * connection, which Workers cannot
 * hold. Those need either a small
 * always-on bot elsewhere, or a
 * periodic reconciliation pull.
 *
 * The reconciliation approach is in sync.ts and covers most of the gap without
 * a second host.
 */

export const InteractionType = {
 PING: 1,
 APPLICATION_COMMAND: 2,
 MESSAGE_COMPONENT: 3,
 AUTOCOMPLETE: 4,
 MODAL_SUBMIT: 5,
} as const;

export const InteractionResponseType = {
 PONG: 1,
 CHANNEL_MESSAGE_WITH_SOURCE: 4,
 DEFERRED_CHANNEL_MESSAGE_WITH_SOURCE: 5,
 // For component (button) clicks: acknowledge without a visible loading state,
 // then edit the source message later via editOriginalMessage().
 DEFERRED_UPDATE_MESSAGE: 6,
 UPDATE_MESSAGE: 7,
} as const;

export const MessageFlags = { EPOCH: 1 << 6 } as const;

// The explicit <ArrayBuffer> matters: TypeScript 5.7 made Uint8Array generic
// over its backing buffer, and the bare form widens to ArrayBufferLike, which
// crypto.subtle will not accept.
function hexToBytes(hex: string): Uint8Array<ArrayBuffer> {
 const bytes = new Uint8Array(hex.length / 2);
 for (let i = 0; i < bytes.length; i++) {
 bytes[i] = parseInt(hex.slice(i * 2, i * 2 + 2), 16);
 }
 return bytes;
}

/**
 * Discord signs every interaction request with Ed25519 and *will* send
 * deliberately invalid signatures when you register the endpoint ? if this
 * returns true for a bad signature, registration fails by design.

```



**src/server/discord/interactions.ts**

```
*
* Workers' Web Crypto supports Ed25519 natively, so no library is needed.
*/
export async function verifySignature(
 body: string,
 signature: string | null,
 timestamp: string | null,
 publicKey: string,
): Promise<boolean> {
 if (!signature || !timestamp) return false;

 try {
 const key = await crypto.subtle.importKey(
 'raw',
 hexToBytes(publicKey),
 { name: 'Ed25519' },
 false,
 ['verify'],
);
 return await crypto.subtle.verify(
 { name: 'Ed25519' },
 key,
 hexToBytes(signature),
 new TextEncoder().encode(timestamp + body),
);
 } catch {
 return false;
 }
}

export interface InteractionOption {
 name: string;
 type: number;
 value?: string | number | boolean;
 options?: InteractionOption[];
}

export interface Interaction {
 id: string;
 application_id: string;
 type: number;
 token: string;
 guild_id?: string;
 // Slash commands carry name/options; component (button) clicks carry
 // custom_id/component_type.
 data?: {
 id?: string;
 name?: string;
 options?: InteractionOption[];
 custom_id?: string;
 component_type?: number;
 };
 // Present on component interactions ? the message the button lives on.
 message?: { id: string };
```

**src/server/discord/interactions.ts**

```

 member?: {
 user: { id: string; username: string; global_name: string | null };
 roles: string[];
 };
 user?: { id: string; username: string; global_name: string | null };
 }

 /** The invoking Discord user, whether the command came from a guild or a DM. */
 export function invoker(i: Interaction): { id: string; username: string } | null {
 const u = i.member?.user ?? i.user;
 return u ? { id: u.id, username: u.username } : null;
 }

 export function optionValue<T = string>(i: Interaction, name: string): T | undefined {
 return i.data?.options?.find((o) => o.name === name)?.value as T | undefined;
 }

 export function ephemeral(content: string) {
 return {
 type: InteractionResponseType.CHANNEL_MESSAGE_WITH_SOURCE,
 data: { content, flags: MessageFlags.EPHEMERAL },
 };
 }

 export function reply(content: string) {
 return {
 type: InteractionResponseType.CHANNEL_MESSAGE_WITH_SOURCE,
 data: { content },
 };
 }

 /**
 * Discord hangs up if you take longer than 3 seconds. Defer immediately, then
 * finish the work in ctx.waitUntil() and fill in the message with
 * editOriginalResponse().
 */
 export function defer(isEphemeral = true) {
 return {
 type: InteractionResponseType.DEFERRED_CHANNEL_MESSAGE_WITH_SOURCE,
 data: isEphemeral ? { flags: MessageFlags.EPHEMERAL } : {},
 };
 }

 /**
 * Replaces the "thinking..." placeholder from a deferred response with the real
 * content. Authenticated by the interaction token itself ? no bot token needed.
 * The ephemeral flag is fixed at defer time and cannot be changed here.
 */
 export async function editOriginalResponse(
 applicationId: string,
 interactionToken: string,
 content: string,
): Promise<void> {
 const res = await fetch(

```

**src/server/discord/interactions.ts**

```

`https://discord.com/api/v10/webhooks/${applicationId}/${interactionToken}/messages/@original`,
 {
 method: 'PATCH',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify({ content }),
 },
);
if (!res.ok) {
 console.error(`Failed to edit interaction response (${res.status}):`, await res.text());
}
}

/** A deferred acknowledgement for a component (button) click. */
export function deferUpdate() {
 return { type: InteractionResponseType.DEFERRED_UPDATE_MESSAGE };
}

/**
 * Edit the message a component interaction belongs to (its "@original"). For a
 * button click that's the announcement/sign-up message ? this refreshes its
 * embed counts and button labels. No bot token needed; the interaction token
 * authenticates it.
 */
export async function editOriginalMessage(
 applicationId: string,
 interactionToken: string,
 payload: { content?: string; embeds?: unknown[]; components?: unknown[] },
): Promise<void> {
 const res = await fetch(
 `https://discord.com/api/v10/webhooks/${applicationId}/${interactionToken}/messages/@original`,
 {
 method: 'PATCH',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify(payload),
 },
);
 if (!res.ok) {
 console.error(`Failed to edit component message (${res.status}):`, await res.text());
 }
}

/** Send a private (ephemeral) follow-up to whoever triggered the interaction. */
export async function followUpEphemeral(
 applicationId: string,
 interactionToken: string,
 content: string,
): Promise<void> {
 const res = await
 fetch(`https://discord.com/api/v10/webhooks/${applicationId}/${interactionToken}`, {
 method: 'POST',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify({ content, flags: MessageFlags.EPHEMERAL }),
 });

```

**src/server/discord/interactions.ts**

```
});
if (!res.ok) {
 console.error(`Failed to send follow-up (${res.status}):`, await res.text());
}
}
```

**src/server/discord/rest.ts**

```
/**
 * Bot-token calls against the Discord REST API.
 *
 * Two gotchas that will otherwise cost you an afternoon:
 *
 * 1. The bot needs the MANAGE_ROLES permission, and
 * 2. the bot's own role must sit ABOVE every role it manages in the guild's
 * role list. Discord refuses (50013 Missing Permissions) otherwise, even
 * for an administrator bot. Drag the bot's role to the top in
 * Server Settings -> Roles.
 */

const DISCORD_API = 'https://discord.com/api/v10';

export interface DiscordRole {
 id: string;
 name: string;
 color: number;
 position: number;
 managed: boolean;
}

export interface GuildMember {
 roles: string[];
 /** When they joined the guild, unix seconds ? or null if Discord omitted it. */
 joinedAt: number | null;
}

export interface TextChannel {
 id: string;
 name: string;
 position: number;
}

/** An EXTERNAL guild scheduled event (entity_type 3). */
export interface ScheduledEventInput {
 name: string;
 description?: string;
 scheduled_start_time: string; // ISO 8601
 scheduled_end_time: string; // ISO 8601 ? required for EXTERNAL
 privacy_level: 2; // GUILD_ONLY
 entity_type: 3; // EXTERNAL
 entity_metadata: { location: string };
 // Cover image as a data URI (data:image/png;base64,...). Discord decodes and
 // stores it; it can't fetch a URL here, which is why we inline the bytes.
 image?: string;
}

/** A minimal Discord embed ? the shape createMessage accepts. */
export interface Embed {
 title?: string;
 description?: string;
 color?: number;
 thumbnail?: { url: string };
}
```

**src/server/discord/rest.ts**

```

 image?: { url: string };
 author?: { name: string; icon_url?: string };
 footer?: { text: string };
 fields?: { name: string; value: string; inline?: boolean }[];
}

/**
 * A message component action row (type 1) holding buttons (type 2). Kept loose
 * ? the event sign-up buttons are the only components we build, in
 * discord/events.ts. Button styles: 1 primary, 2 secondary, 3 success, 4 danger.
 */
export type ActionRow = {
 type: 1;
 components: {
 type: 2;
 style: 1 | 2 | 3 | 4;
 label: string;
 custom_id: string;
 emoji?: { name: string };
 disabled?: boolean;
 }[];
};

export class DiscordRest {
 constructor(
 private readonly botToken: string,
 private readonly guildId: string,
) {}

 private async call(path: string, init: RequestInit = {}, reason?: string): Promise<Response> {
 const headers: Record<string, string> = {
 Authorization: `Bot ${this.botToken}`,
 'Content-Type': 'application/json',
 ...(init.headers as Record<string, string> | undefined),
 };
 // Shows up in the Discord server's own audit log ? worth always sending.
 if (reason) headers['X-Audit-Log-Reason'] = reason.slice(0, 512);

 const res = await fetch(`${DISCORD_API}${path}`, { ...init, headers });

 if (res.status === 429) {
 const retryAfter = Number(res.headers.get('retry-after') ?? '1');
 throw new RateLimited(retryAfter);
 }
 return res;
 }

 async listRoles(): Promise<DiscordRole[]> {
 const res = await this.call(`/guilds/${this.guildId}/roles`);
 if (!res.ok) throw new DiscordError('listRoles', res.status, await res.text());
 return res.json();
 }
}

/**
```

**src/server/discord/rest.ts**

```

* The full guild member: their roles and guild-join date. Returns null when
* they're no longer in the server (404).
*/
async getMember(discordUserId: string): Promise<GuildMember | null> {
 const res = await this.call(`/guilds/${this.guildId}/members/${discordUserId}`);
 if (res.status === 404) return null;
 if (!res.ok) throw new DiscordError('getMember', res.status, await res.text());
 const body = (await res.json()) as { roles: string[]; joined_at?: string };
 const parsed = body.joined_at ? Math.floor(Date.parse(body.joined_at) / 1000) : NaN;
 return { roles: body.roles, joinedAt: Number.isNaN(parsed) ? null : parsed };
}

async getMemberRoles(discordUserId: string): Promise<string[] | null> {
 const member = await this.getMember(discordUserId);
 return member ? member.roles : null;
}

async addRole(discordUserId: string, roleId: string, reason?: string): Promise<void> {
 const res = await this.call(
 `/guilds/${this.guildId}/members/${discordUserId}/roles/${roleId}`,
 { method: 'PUT' },
 reason,
);
 if (!res.ok && res.status !== 204) {
 throw new DiscordError('addRole', res.status, await res.text());
 }
}

async removeRole(discordUserId: string, roleId: string, reason?: string): Promise<void> {
 const res = await this.call(
 `/guilds/${this.guildId}/members/${discordUserId}/roles/${roleId}`,
 { method: 'DELETE' },
 reason,
);
 if (!res.ok && res.status !== 204) {
 throw new DiscordError('removeRole', res.status, await res.text());
 }
}

/**
* Updates a guild role's name and/or colour in a single request. Colour is a
* 24-bit integer (0 = no colour). Discord caps names at 100 characters. Same
* hierarchy rule as membership changes: the bot's role must sit above the
* target, or this returns 50013.
*/
async updateRole(
 roleId: string,
 fields: { name?: string; color?: number },
 reason?: string,
): Promise<void> {
 const body: Record<string, unknown> = {};
 if (fields.name !== undefined) body.name = fields.name.slice(0, 100);
 if (fields.color !== undefined) body.color = fields.color;
}

```

**src/server/discord/rest.ts**

```

 const res = await this.call(
 `/guilds/${this.guildId}/roles/${roleId}`,
 { method: 'PATCH', body: JSON.stringify(body) },
 reason,
);
 if (!res.ok) {
 throw new DiscordError('updateRole', res.status, await res.text());
 }
 }

 /**
 * Sets a guild member's nickname ? their per-server display name. Pass an
 * empty string to clear it (reverting to their Discord name). Needs the bot's
 * MANAGE_NICKNAMES permission and its role above the member's. Discord will
 * not let anyone change the guild owner's nickname, permissions aside.
 */
 async setNickname(discordUserId: string, nick: string, reason?: string): Promise<void> {
 const res = await this.call(
 `/guilds/${this.guildId}/members/${discordUserId}`,
 { method: 'PATCH', body: JSON.stringify({ nick: nick.slice(0, 32) || null }) },
 reason,
);
 if (!res.ok) {
 throw new DiscordError('setNickname', res.status, await res.text());
 }
 }

 /** Text and announcement channels in the guild, for the announcement-channel picker. */
 async listTextChannels(): Promise<TextChannel[]> {
 const res = await this.call(`/guilds/${this.guildId}/channels`);
 if (!res.ok) throw new DiscordError('listChannels', res.status, await res.text());
 const channels = (await res.json()) as { id: string; name: string; type: number; position:
number }[];
 // 0 = GUILD_TEXT, 5 = GUILD_ANNOUNCEMENT ? the ones the bot can post plain messages to.
 return channels
 .filter((c) => c.type === 0 || c.type === 5)
 .map((c) => ({ id: c.id, name: c.name, position: c.position }))
 .sort((a, b) => a.position - b.position);
 }

 /**
 * Post a message (optionally with embeds) to a channel and return its id.
 * Needs the bot's VIEW_CHANNEL + SEND_MESSAGES permission in that channel.
 */
 async createMessage(
 channelId: string,
 payload: {
 content?: string;
 embeds?: Embed[];
 components?: ActionRow[];
 allowed_mentions?: { users?: string[]; parse?: string[] };
 },
): Promise<string> {
 const res = await this.call(`/channels/${channelId}/messages`, {

```



**src/server/discord/rest.ts**

```

 method: 'POST',
 body: JSON.stringify(payload),
 });
 if (!res.ok) throw new DiscordError('createMessage', res.status, await res.text());
 const body = (await res.json()) as { id: string };
 return body.id;
}

async editMessage(
 channelId: string,
 messageId: string,
 payload: { content?: string; embeds?: Embed[]; components?: ActionRow[] },
): Promise<void> {
 const res = await this.call(`/channels/${channelId}/messages/${messageId}`, {
 method: 'PATCH',
 body: JSON.stringify(payload),
 });
 if (!res.ok) throw new DiscordError('editMessage', res.status, await res.text());
}

async deleteMessage(channelId: string, messageId: string): Promise<void> {
 const res = await this.call(`/channels/${channelId}/messages/${messageId}`, { method: 'DELETE' });
 if (!res.ok && res.status !== 404) throw new DiscordError('deleteMessage', res.status, await res.text());
}

/**
 * Native guild scheduled event (EXTERNAL entity). Needs the bot's
 * MANAGE_EVENTS permission. Times are ISO 8601. Returns the new event's id.
 */
async createScheduledEvent(payload: ScheduledEventInput): Promise<string> {
 const res = await this.call(`/guilds/${this.guildId}/scheduled-events`, {
 method: 'POST',
 body: JSON.stringify(payload),
 });
 if (!res.ok) throw new DiscordError('createScheduledEvent', res.status, await res.text());
 const body = (await res.json()) as { id: string };
 return body.id;
}

async modifyScheduledEvent(eventId: string, payload: Partial<ScheduledEventInput> & { status?: number }): Promise<void> {
 const res = await this.call(`/guilds/${this.guildId}/scheduled-events/${eventId}`, {
 method: 'PATCH',
 body: JSON.stringify(payload),
 });
 if (!res.ok) throw new DiscordError('modifyScheduledEvent', res.status, await res.text());
}

async deleteScheduledEvent(eventId: string): Promise<void> {
 const res = await this.call(`/guilds/${this.guildId}/scheduled-events/${eventId}`, { method: 'DELETE' });
 if (!res.ok && res.status !== 404) throw new DiscordError('deleteScheduledEvent', res.status,

```

**src/server/discord/rest.ts**

```
await res.text());
}

/** Used by slash commands to reply after an initial deferred response. */
async followUp(applicationId: string, interactionToken: string, content: unknown): Promise<void>
{
 await fetch(`${DISCORD_API}/webhooks/${applicationId}/${interactionToken}`, {
 method: 'POST',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify(content),
 });
}

export class DiscordError extends Error {
 constructor(
 readonly operation: string,
 readonly status: number,
 readonly body: string,
) {
 super(`Discord ${operation} failed (${status}): ${body}`);
 this.name = 'DiscordError';
 }
}

export class RateLimited extends Error {
 constructor(readonly retryAfterSeconds: number) {
 super(`Rate limited by Discord; retry after ${retryAfterSeconds}s`);
 this.name = 'RateLimited';
 }
}
```

**src/server/discord/sync.ts**

```

/**
 * Keeps website roles and Discord roles in agreement.
 *
 * The website is the source of truth. Granting a role here pushes to Discord;
 * reconcile() pulls Discord back into line for anything changed directly in
 * Discord's UI (which Workers cannot observe live ? see interactions.ts).
 */

import { and, eq, inArray, ne, sql } from 'drizzle-orm';
import type { Drizzled1Database } from 'drizzle-orm/d1';
import * as s from '../db/schema';
import { DiscordRest, DiscordError } from './rest';

type DB = Drizzled1Database<typeof s>;

export interface SyncResult {
 discordSynced: boolean;
 warning?: string;
}

export async function grantRole(
 db: DB,
 rest: DiscordRest | null,
 opts: { userId: number; roleId: number; actorId: number; reason?: string },
): Promise<SyncResult> {
 const [user, role] = await Promise.all([
 db.query.users.findFirst({ where: eq(s.users.id, opts.userId) }),
 db.query.roles.findFirst({ where: eq(s.roles.id, opts.roleId) }),
]);
 if (!user) throw new Error(`No such member: ${opts.userId}`);
 if (!role) throw new Error(`No such role: ${opts.roleId}`);

 // A manual grant wins: if the role was previously held via rank, upgrade the
 // source so a later rank change can't strip it.
 await db
 .insert(s.userRoles)
 .values({ userId: user.id, roleId: role.id, grantedBy: opts.actorId, source: 'manual' })
 .onConflictDoUpdate({
 target: [s.userRoles.userId, s.userRoles.roleId],
 set: { source: 'manual', grantedBy: opts.actorId },
 });

 await db.insert(s.auditLog).values({
 actorId: opts.actorId,
 action: 'role.grant',
 targetType: 'user',
 targetId: String(user.id),
 meta: { roleId: role.id, roleName: role.name, reason: opts.reason },
 });

 return pushToDiscord(rest, 'add', user.discordId, role.discordRoleId, opts.reason);
}

export async function revokeRole(

```

**src/server/discord/sync.ts**

```

db: DB,
rest: DiscordRest | null,
opts: { userId: number; roleId: number; actorId: number; reason?: string },
): Promise<SyncResult> {
 const [user, role] = await Promise.all([
 db.query.users.findFirst({ where: eq(s.users.id, opts.userId) }),
 db.query.roles.findFirst({ where: eq(s.roles.id, opts.roleId) }),
]);
 if (!user) throw new Error(`No such member: ${opts.userId}`);
 if (!role) throw new Error(`No such role: ${opts.roleId}`);

 await db
 .delete(s.userRoles)
 .where(and(eq(s.userRoles.userId, user.id), eq(s.userRoles.roleId, role.id)));

 await db.insert(s.auditLog).values({
 actorId: opts.actorId,
 action: 'role.revoke',
 targetType: 'user',
 targetId: String(user.id),
 reason: opts.reason ?? null,
 meta: { roleId: role.id, roleName: role.name },
 });

 return pushToDiscord(rest, 'remove', user.discordId, role.discordRoleId, opts.reason);
}

/**
 * A Discord failure must not roll back the website grant ? the site is
 * authoritative, and reconcile() will catch up later. Report it, don't throw.
 */
async function pushToDiscord(
 rest: DiscordRest | null,
 op: 'add' | 'remove',
 discordUserId: string,
 discordRoleId: string | null,
 reason?: string,
): Promise<SyncResult> {
 if (!rest || !discordRoleId) return { discordSynced: false };

 const auditReason = reason ?? 'ClanTek admin portal';
 try {
 if (op === 'add') await rest.addRole(discordUserId, discordRoleId, auditReason);
 else await rest.removeRole(discordUserId, discordRoleId, auditReason);
 return { discordSynced: true };
 } catch (err) {
 const warning =
 err instanceof DiscordError && err.status === 403
 ? 'Saved here, but Discord refused: the bot needs MANAGE_ROLES and its role must sit above'
 : `Saved here, but the Discord update failed: ${err as Error}.message`;
 return { discordSynced: false, warning };
 }
}

```

**src/server/discord/sync.ts**

```

/**
 * Pushes every mapped role for one member so Discord matches the website.
 * Run after a manual change in Discord, or on a cron.
 */
export async function reconcileMember(
 db: DB,
 rest: DiscordRest,
 userId: number,
 allRoles?: (typeof s.roles.$inferSelect)[],
): Promise<{ added: string[]; removed: string[] }> {
 const user = await db.query.users.findFirst({ where: eq(s.users.id, userId) });
 if (!user) throw new Error(`No such member: ${userId}`);

 // Callers that reconcile many members in a row (the cron batch) pass the role
 // set in so it isn't re-queried per member.
 const mapped = allRoles ?? (await db.select().from(s.roles));
 const byDiscordId = new Map(
 mapped.filter((r) => r.discordRoleId).map((r) => [r.discordRoleId!, r]),
);

 const held = await db
 .select({ discordRoleId: s.roles.discordRoleId })
 .from(s.userRoles)
 .innerJoin(s.roles, eq(s.userRoles.roleId, s.roles.id))
 .where(eq(s.userRoles.userId, user.id));

 const shouldHave = new Set(held.map((r) => r.discordRoleId).filter((id): id is string => !!id));
 const member = await rest.getMember(user.discordId);
 if (member === null) return { added: [], removed: [] }; // left the server
 const currentlyHas = member.roles;

 // Backfill the authoritative guild-join date (for tenure medals) the first
 // time we read a member who hasn't logged in since it was added.
 if (user.guildJoinedAt == null && member.joinedAt != null) {
 await db.update(s.users).set({ guildJoinedAt: member.joinedAt }).where(eq(s.users.id,
user.id));
 }

 const added: string[] = [];
 const removed: string[] = [];

 for (const roleId of shouldHave) {
 if (!currentlyHas.includes(roleId)) {
 await rest.addRole(user.discordId, roleId, 'ClanTek reconciliation');
 added.push(roleId);
 }
 }

 // Only touch roles ClanTek manages; leave unmapped Discord roles alone.
 for (const roleId of currentlyHas) {
 if (byDiscordId.has(roleId) && !shouldHave.has(roleId)) {
 await rest.removeRole(user.discordId, roleId, 'ClanTek reconciliation');
 removed.push(roleId);
 }
 }
}

```

**src/server/discord/sync.ts**

```

 }

 return { added, removed };
}

/* -----
 * Rank-driven roles
 *
 * A rank confers a set of roles. Assigning a member to a rank grants those
 * roles (marked source='rank'); changing rank reconciles them. Manual grants
 * (source='manual') are never touched here, so an admin's explicit grant
 * survives promotions and demotions.
 * ----- */

export interface RankRoleSyncResult {
 added: string[];
 removed: string[];
 warnings: string[];
}

/**
 * Make a member's rank-derived roles exactly match what `rankId` confers,
 * pushing each change to Discord. Roles the member holds manually are left as
 * they are ? including ones the rank also happens to grant.
 */
export async function syncMemberRankRoles(
 db: DB,
 rest: DiscordRest | null,
 opts: { userId: number; rankId: number | null; actorId: number | null },
): Promise<RankRoleSyncResult> {
 const user = await db.query.users.findFirst({ where: eq(s.users.id, opts.userId) });
 if (!user) throw new Error(`No such member: ${opts.userId}`);

 const desired = opts.rankId
 ? (
 await db
 .select({ roleId: s.rankRoles.roleId })
 .from(s.rankRoles)
 .where(eq(s.rankRoles.rankId, opts.rankId))
).map((r) => r.roleId)
 : [];
 const desiredSet = new Set(desired);

 const current = await db
 .select({ roleId: s.userRoles.roleId, source: s.userRoles.source })
 .from(s.userRoles)
 .where(eq(s.userRoles.userId, opts.userId));
 const heldIds = new Set(current.map((c) => c.roleId));
 const rankHeldIds = current.filter((c) => c.source === 'rank').map((c) => c.roleId);

 // Add anything the rank grants that the member doesn't already hold (in any
 // form); drop rank-held roles the new rank no longer grants.
 const toAdd = desired.filter((id) => !heldIds.has(id));
 const toRemove = rankHeldIds.filter((id) => !desiredSet.has(id));

```

**src/server/discord/sync.ts**

```

const result: RankRoleSyncResult = { added: [], removed: [], warnings: [] };
if (!toAdd.length && !toRemove.length) return result;

const involved = [...new Set([...toAdd, ...toRemove])];
const roleRows = involved.length
 ? await db.select().from(s.roles).where(inArray(s.roles.id, involved))
 : [];
const roleById = new Map(roleRows.map((r) => [r.id, r]));

for (const roleId of toAdd) {
 const role = roleById.get(roleId);
 if (!role) continue;
 await db
 .insert(s.userRoles)
 .values({ userId: opts.userId, roleId, source: 'rank', grantedBy: opts.actorId })
 .onConflictDoNothing();
 result.added.push(role.name);
 const push = await pushToDiscord(rest, 'add', user.discordId, role.discordRoleId, 'ClanTek
rank role');
 if (push.warning) result.warnings.push(push.warning);
}

for (const roleId of toRemove) {
 const role = roleById.get(roleId);
 if (!role) continue;
 await db
 .delete(s.userRoles)
 .where(and(eq(s.userRoles.userId, opts.userId), eq(s.userRoles.roleId, roleId)));
 result.removed.push(role.name);
 const push = await pushToDiscord(rest, 'remove', user.discordId, role.discordRoleId, 'ClanTek
rank role');
 if (push.warning) result.warnings.push(push.warning);
}

await db.insert(s.auditLog).values({
 actorId: opts.actorId,
 action: 'rank.roles_sync',
 targetType: 'user',
 targetId: String(opts.userId),
 meta: { rankId: opts.rankId, added: result.added, removed: result.removed },
});

return result;
}

/**
 * The drift-correction sweep, paced so it scales to any roster size.
 *
 * Each member costs one Discord read plus one call per role that needs fixing.
 * The free plan caps a Worker invocation at 50 subrequests and Discord rate-
 * limits us globally, so instead of touching everyone every tick we process the
 * least-recently-reconciled members up to a call budget, stamping
 * `lastReconciledAt` so the next tick picks up where this one left off. Over

```

**src/server/discord/sync.ts**

```

* many ticks the whole roster is covered, and the per-tick cost is bounded no
* matter how large the clan grows. Banned members are skipped by design.
*
* This only paces *drift correction* ? a website change (grant, rank, etc.)
* still pushes to Discord immediately when it happens.
*/
const RECONCILE_CALL_BUDGET = 40; // Discord calls per tick ? under the 50-subrequest ceiling
const RECONCILE_POOL = 60; // candidate rows to consider per tick

export async function reconcileBatch(
 db: DB,
 rest: DiscordRest,
 opts: { callBudget?: number; poolSize?: number } = {},
): Promise<{ processed: number; changed: number; failed: number; calls: number }> {
 const budget = opts.callBudget ?? RECONCILE_CALL_BUDGET;
 const poolSize = opts.poolSize ?? RECONCILE_POOL;

 // Load the role set once for the whole batch rather than per member.
 const allRoles = await db.select().from(s.roles);

 const candidates = await db
 .select({ id: s.users.id })
 .from(s.users)
 .where(ne(s.users.status, 'banned'))
 // NULL (never reconciled) sorts first via the coalesce; then oldest first.
 .orderBy(sql`coalesce(${s.users.lastReconciledAt}, 0) asc`)
 .limit(poolSize);

 const nowSec = Math.floor(Date.now() / 1000);
 let processed = 0;
 let changed = 0;
 let failed = 0;
 let calls = 0;

 for (const m of candidates) {
 if (calls >= budget) break;
 try {
 const r = await reconcileMember(db, rest, m.id, allRoles);
 // getMember (1) + one call per role add/remove.
 calls += 1 + r.added.length + r.removed.length;
 processed++;
 if (r.added.length || r.removed.length) changed++;
 } catch {
 calls += 1; // the getMember attempt still counted against the budget
 failed++;
 }
 // Stamp regardless of outcome so a persistently-failing member rotates to
 // the back rather than blocking everyone behind it.
 await db.update(s.users).set({ lastReconciledAt: nowSec }).where(eq(s.users.id, m.id));
 }

 return { processed, changed, failed, calls };
}

```



**src/server/discord/sync.ts**

```
/** Re-apply a rank's role set to everyone currently holding that rank. */
export async function syncRankHolders(
 db: DB,
 rest: DiscordRest | null,
 rankId: number,
): Promise<{ members: number; warnings: string[] }> {
 const holders = await db
 .select({ id: s.users.id })
 .from(s.users)
 .where(eq(s.users.rankId, rankId));

 const warnings = new Set<string>();
 for (const h of holders) {
 const r = await syncMemberRankRoles(db, rest, { userId: h.id, rankId, actorId: null });
 r.warnings.forEach((w) => warnings.add(w));
 }
 return { members: holders.length, warnings: [...warnings] };
}
```

**src/server/env.ts**

```
import type { Viewer } from '../shared/permissions';

export interface Env {
 DB: D1Database;
 ASSETS: Fetcher;

 // Bound only once R2 is enabled on the account ? see wrangler.jsonc.
 MEDIA?: R2Bucket;

 // Public config, committed in wrangler.jsonc
 SITE_NAME: string;
 // Canonical public origin (e.g. https://mustr.gg), for building
 // absolute media URLs in Discord embeds. Optional ? absent -> images omitted.
 SITE_URL?: string;
 DISCORD_CLIENT_ID: string;
 DISCORD_GUILD_ID: string;
 DISCORD_PUBLIC_KEY: string;

 // Secrets: .dev.vars locally, `wrangler secret put` in production
 DISCORD_CLIENT_SECRET: string;
 DISCORD_BOT_TOKEN: string;
 SESSION_SECRET: string;

 // Optional. Enables the live usage dashboard's request/query-rate gauges by
 // reading Cloudflare's GraphQL Analytics API. The account id is not secret
 // (it's a var); the token is a read-only "Account Analytics: Read" secret set
 // via `wrangler secret put CLOUDFLARE_API_TOKEN`. Absent -> storage gauges only.
 CLOUDFLARE_ACCOUNT_ID?: string;
 CLOUDFLARE_API_TOKEN?: string;
}

export interface Variables {
 viewer: Viewer | null;
 /** How the viewer authenticated: a browser session cookie, or a bearer API token. */
 authKind: 'web' | 'token' | null;
}

export type AppContext = { Bindings: Env; Variables: Variables };
```

**src/server/events/recurrence.ts**

```

/**
 * Recurring events ? the "baton" model.
 *
 * An event with a non-'none' recurrence is the current occurrence of a series.
 * Once it has ended, the hourly cron spawns the next occurrence (same title,
 * roles, image, location and duration; a fresh sign-up sheet and its own
 * Discord scheduled event + message) and clears the finished row's recurrence
 * back to 'none'. The baton therefore moves forward: at most one future
 * occurrence per series ever carries a recurrence, so nothing double-spawns and
 * stopping the series is just setting the current occurrence back to 'none'.
 */

import { drizzle } from 'drizzle-orm/d1';
import { and, eq, lt, ne } from 'drizzle-orm';
import * as schema from '../db/schema';
import * as s from '../db/schema';
import type { Env } from '../env';
import { syncEventToDiscord } from '../discord/events';

type Recurrence = 'none' | 'daily' | 'weekly' | 'biweekly' | 'monthly';

/** Cap per run so a long-idle cron can't fan out into a huge Discord burst. */
const MAX_SPAWN_PER_RUN = 10;

/** Advance a unix-second moment by one recurrence interval. */
function addInterval(unixSec: number, rec: Recurrence): number {
 switch (rec) {
 case 'daily':
 return unixSec + 86_400;
 case 'weekly':
 return unixSec + 604_800;
 case 'biweekly':
 return unixSec + 1_209_600;
 case 'monthly': {
 // Calendar month, preserving the UTC time-of-day (setUTCMonth handles the
 // year rollover; a day that doesn't exist next month rolls forward, which
 // is fine for clan events).
 const d = new Date(unixSec * 1000);
 d.setUTCMonth(d.getUTCMonth() + 1);
 return Math.floor(d.getTime() / 1000);
 }
 default:
 return unixSec;
 }
}

/** The next occurrence strictly in the future, skipping any missed intervals. */
function nextFutureStart(startsAt: number, rec: Recurrence, nowSec: number): number {
 let next = addInterval(startsAt, rec);
 // Guard the loop (monthly ~ up to a few iterations; others bounded too).
 let guard = 0;
 while (next <= nowSec && guard < 1000) {
 next = addInterval(next, rec);
 guard += 1;
 }
}

```

**src/server/events/recurrence.ts**

```

 }
 return next;
}

export interface RecurrenceResult {
 spawned: number;
}

/**
 * Find ended recurring events and roll each forward one occurrence. Run from
 * the hourly cron. DB-only when the bot isn't configured (the Discord sync
 * simply no-ops), so it's always safe to call.
 */
export async function materializeRecurringEvents(env: Env): Promise<RecurrenceResult> {
 const db = drizzle(env.DB, { schema });
 const nowSec = Math.floor(Date.now() / 1000);

 const due = await db
 .select()
 .from(s.events)
 .where(and(ne(s.events.recurrence, 'none'), lt(s.events.endsAt, nowSec), eq(s.events.status,
'scheduled'))))
 .limit(MAX_SPAWN_PER_RUN);

 let spawned = 0;
 for (const src of due) {
 const rec = src.recurrence as Recurrence;
 const duration = src.endsAt - src.startsAt;
 const nextStart = nextFutureStart(src.startsAt, rec, nowSec);
 const nextEnd = nextStart + duration;

 // Clear the finished row's recurrence FIRST so a retry (or overlapping run)
 // can't spawn it twice; the new occurrence carries the baton.
 await db.update(s.events).set({ recurrence: 'none' }).where(eq(s.events.id, src.id));

 const created = (
 await db
 .insert(s.events)
 .values({
 title: src.title,
 description: src.description,
 imageUrl: src.imageUrl,
 startsAt: nextStart,
 endsAt: nextEnd,
 location: src.location,
 gameId: src.gameId,
 createdBy: src.createdBy,
 recurrence: rec,
 status: 'scheduled',
 })
 .returning()
)[0]!;

 // Copy the sign-up roles (not the sign-ups ? each occurrence starts empty).

```

**src/server/events/recurrence.ts**

```
const roles = await db.select().from(s.eventRoles).where(eq(s.eventRoles.eventId, src.id));
if (roles.length) {
 await db.insert(s.eventRoles).values(
 roles.map((r) => ({
 eventId: created.id,
 name: r.name,
 emoji: r.emoji,
 capacity: r.capacity,
 sortOrder: r.sortOrder,
 })),
);
}

await db.insert(s.auditLog).values({
 action: 'event.recurrence_spawn',
 targetType: 'event',
 targetId: String(created.id),
 meta: { fromEventId: src.id, recurrence: rec, title: created.title },
 source: 'system',
});

// Mirror the new occurrence to Discord and persist the returned ids.
try {
 const ids = await syncEventToDiscord(env, created.id);
 if (ids.discordEventId || ids.discordMessageId) {
 await db.update(s.events).set(ids).where(eq(s.events.id, created.id));
 }
} catch (err) {
 console.error('Recurring event Discord sync failed', err);
}

spawned += 1;
}

return { spawned };
}
```

**src/server/events/signups.ts**

```

/**
 * Event sign-ups ? the shared source of truth used by BOTH the website routes
 * and the Discord button handler, so a sign-up made in either place is the same
 * row and the two surfaces stay in agreement.
 *
 * One row per member per event (unique index). Picking a different role updates
 * the row; withdrawing deletes it. eventRoleId NULL = "attending, no role".
 */

import { and, asc, eq, ne, sql } from 'drizzle-orm';
import type { DrizzleD1Database } from 'drizzle-orm/d1';
import * as s from '../db/schema';
import { memberName } from '../shared/names';
import { memberAvatar } from '../shared/avatar';

type DB = DrizzleD1Database<typeof s>;

export interface RoleState {
 id: number;
 name: string;
 emoji: string | null;
 capacity: number | null;
 sortOrder: number;
 count: number;
}

export interface SignupEntry {
 userId: number;
 name: string;
 avatarUrl: string;
 roleId: number | null;
 roleName: string | null;
}

export interface EventState {
 event: typeof s.events.$inferSelect;
 gameName: string | null;
 roles: RoleState[];
 signups: SignupEntry[];
 total: number;
}

export interface SignupResult {
 ok: boolean;
 error?: string;
}

/**
 * Everything the website and Discord both need to render an event: the row, its
 * game name, its roles with live counts, and the full sign-up list (names +
 * avatars). One place so counts never disagree between surfaces.
 */

export async function loadEventState(db: DB, eventId: number): Promise<EventState | null> {
 const event = await db.query.events.findFirst({ where: eq(s.events.id, eventId) });
 if (!event) return null;

```

**src/server/events/signups.ts**

```

const gameName = event.gameId
 ? ((await db.query.games.findFirst({ where: eq(s.games.id, event.gameId) }))??.name ?? null)
 : null;

const roles = await db
 .select()
 .from(s.eventRoles)
 .where(eq(s.eventRoles.eventId, eventId))
 .orderBy(asc(s.eventRoles.sortOrder), asc(s.eventRoles.id));

const signupRows = await db
 .select({
 userId: s.eventSignups.userId,
 roleId: s.eventSignups.eventRoleId,
 username: s.users.username,
 globalName: s.users.globalName,
 displayName: s.users.displayName,
 discordId: s.users.discordId,
 avatar: s.users.avatar,
 profileImageUrl: s.users.profileImageUrl,
 })
 .from(s.eventSignups)
 .innerJoin(s.users, eq(s.eventSignups.userId, s.users.id))
 .where(eq(s.eventSignups.eventId, eventId))
 .orderBy(asc(s.eventSignups.createdAt));

const roleNameById = new Map(roles.map((r) => [r.id, r.name]));
const signups: SignupEntry[] = signupRows.map((r) => ({
 userId: r.userId,
 name: memberName({ displayName: r.displayName, globalName: r.globalName, username: r.username
}),
 avatarUrl: memberAvatar({ discordId: r.discordId, avatar: r.avatar, profileImageUrl:
r.profileImageUrl }, 64),
 roleId: r.roleId,
 roleName: r.roleId != null ? (roleNameById.get(r.roleId) ?? null) : null,
})));

const counts = new Map<number, number>();
for (const su of signups) {
 if (su.roleId != null) counts.set(su.roleId, (counts.get(su.roleId) ?? 0) + 1);
}
const roleStates: RoleState[] = roles.map((r) => ({
 id: r.id,
 name: r.name,
 emoji: r.emoji,
 capacity: r.capacity,
 sortOrder: r.sortOrder,
 count: counts.get(r.id) ?? 0,
})));

return { event, gameName, roles: roleStates, signups, total: signups.length };
}

/**

```

**src/server/events/signups.ts**

```

* Sign a member up (or move them to a different role). roleId null = attending
* with no specific role. Enforces the event being open and the role's capacity.
* Idempotent via the unique (event,user) index.
*/
export async function setSignup(
 db: DB,
 eventId: number,
 userId: number,
 roleId: number | null,
): Promise<SignupResult> {
 const event = await db.query.events.findFirst({ where: eq(s.events.id, eventId) });
 if (!event) return { ok: false, error: 'No such event.' };
 if (event.status !== 'scheduled') return { ok: false, error: 'This event is not open for
sign-ups.' };

 if (roleId !== null) {
 const role = await db.query.eventRoles.findFirst({
 where: and(eq(s.eventRoles.id, roleId), eq(s.eventRoles.eventId, eventId)),
 });
 if (!role) return { ok: false, error: 'That role is not part of this event.' };
 if (role.capacity !== null) {
 // Count current holders other than this member ? moving within your own
 // seat mustn't count against the cap.
 const held = await db
 .select({ n: sql<number>`count(*)` })
 .from(s.eventSignups)
 .where(
 and(
 eq(s.eventSignups.eventId, eventId),
 eq(s.eventSignups.eventRoleId, roleId),
 ne(s.eventSignups.userId, userId),
),
);
 if (Number(held[0]?.n ?? 0) >= role.capacity) {
 return { ok: false, error: ` "${role.name}" is full.` };
 }
 }
 }

 const nowSec = Math.floor(Date.now() / 1000);
 await db
 .insert(s.eventSignups)
 .values({ eventId, userId, eventRoleId: roleId, createdAt: nowSec, updatedAt: nowSec })
 .onConflictDoUpdate({
 target: [s.eventSignups.eventId, s.eventSignups.userId],
 set: { eventRoleId: roleId, updatedAt: nowSec },
 });

 return { ok: true };
}

/** Withdraw a member's sign-up. No-op if they weren't signed up. */
export async function removeSignup(db: DB, eventId: number, userId: number): Promise<void> {
 await db

```



**src/server/events/signups.ts**

```
 .delete(s.eventSignups)
 .where(and(eq(s.eventSignups.eventId, eventId), eq(s.eventSignups.userId, userId)));
}
```

**src/server/footer.ts**

```
/**
 * Server-side loader for the site footer config (settings key 'footer').
 * Returns the stored, re-sanitised footer, or the shipped default when a fresh
 * install has never edited it.
 */

import { drizzle } from 'drizzle-orm/d1';
import { eq } from 'drizzle-orm';
import * as schema from '../db/schema';
import * as s from '../db/schema';
import type { Env } from '../env';
import { DEFAULT_FOOTER, cleanFooter, type FooterConfig } from '../shared/footer';

export const FOOTER_KEY = 'footer';

type DB = ReturnType<typeof drizzle<typeof schema>>;

export async function loadFooter(env: Env, database?: DB): Promise<FooterConfig> {
 const dbi = database ?? drizzle(env.DB, { schema });
 try {
 const row = await dbi.query.settings.findFirst({ where: eq(s.settings.key, FOOTER_KEY) });
 if (row?.value && typeof row.value === 'object') return cleanFooter(row.value);
 } catch {
 // No settings row/table yet -> ship the default footer.
 }
 return DEFAULT_FOOTER;
}
```

**src/server/index.ts**

```
import { Hono } from 'hono';
import { getCookie } from 'hono/cookie';
import { eq } from 'drizzle-orm';
import * as s from '../db/schema';
import type { AppContext, Env } from './env';
import { db, withViewer } from './middleware/auth';
import {
 authorizeUrl,
 newState,
 stateCookie,
 clearedStateCookie,
 exchangeCode,
 fetchUser,
 fetchGuildMember,
 OAUTH_STATE_COOKIE,
} from './auth/discord';
import {
 SESSION_COOKIE,
 createSession,
 invalidateSession,
 purgeExpiredSessions,
 sessionCookie,
 clearedSessionCookie,
} from './auth/session';
import {
 InteractionType,
 verifySignature,
 ephemeral,
 defer,
 deferUpdate,
 editOriginalResponse,
 InteractionResponseType,
 type Interaction,
} from './discord/interactions';
import { handleCommand } from './discord/commands';
import { handleEventComponent, isEventComponent } from './discord/eventInteractions';
import { loadConfig, discordClient } from './config';
import { syncMemberRankRoles, reconcileBatch } from './discord/sync';
import { awardTenureMedals } from './medals/tenure';
import { materializeRecurringEvents } from './events/recurrence';
import ranks from './routes/ranks';
import members from './routes/members';
import settings from './routes/settings';
import rolesRoutes from './routes/roles';
import medalsRoutes from './routes/medals';
import mediaRoutes, { serveMediaObject } from './routes/media';
import newsRoutes from './routes/news';
import usageRoutes from './routes/usage';
import gamesRoutes from './routes/games';
import warRecordsRoutes from './routes/warrecords';
import eventsRoutes from './routes/events';
import auditRoutes from './routes/audit';
import pagesRoutes from './routes/pages';
import navRoutes from './routes/nav';
```

**src/server/index.ts**

```

import hangarRoutes from './routes/hangar';
import scVerifyRoutes from './routes/scVerify';
import { loadSeo, renderHead } from './seo';
import tokensRoutes from './routes/tokens';
import orgChartRoutes from './routes/orgchart';

const app = new Hono<AppContext>();

/* ----- *
 * Canonical host ? redirect `www.` to the bare apex, preserving path and
 * query, so a link to www.<domain> (or a browser that auto-prepends www)
 * lands on the one host where cookies and OAuth live instead of dead-ending.
 * Registered first so it runs ahead of every route and auth check. Host-based
 * (not domain-hardcoded) so it works for any install's own domain.
 * ----- */

app.use('*', async (c, next) => {
 const url = new URL(c.req.url);
 if (url.hostname.startsWith('www.')) {
 url.hostname = url.hostname.slice(4);
 return c.redirect(url.toString(), 301);
 }
 return next();
});

/* ----- *
 * Discord interactions ? must be registered BEFORE withViewer, because
 * it authenticates by Ed25519 signature, not by session cookie, and the
 * raw body has to stay unparsed until the signature is checked.
 * ----- */

app.post('/api/discord/interactions', async (c) => {
 const raw = await c.req.text();

 const { discord } = await loadConfig(c.env);
 const valid = await verifySignature(
 raw,
 c.req.header('x-signature-ed25519') ?? null,
 c.req.header('x-signature-timestamp') ?? null,
 discord.publicKey,
);
 // Discord probes with bad signatures on purpose when you save the endpoint
 // URL; 401 here is what proves the endpoint is genuine.
 if (!valid) return c.text('Invalid request signature', 401);

 const interaction = JSON.parse(raw) as Interaction;

 if (interaction.type === InteractionType.PING) {
 return c.json({ type: InteractionResponseType.PONG });
 }

 if (interaction.type === InteractionType.APPLICATION_COMMAND) {
 // Acknowledge inside Discord's 3-second window, then run the command in the
 // background and edit the placeholder with the result. A cold start plus a

```

**src/server/index.ts**

```

// handful of D1 round-trips can otherwise miss the deadline, which Discord
// reports to the user as "didn't respond in time".
const baseUrl = new URL(c.req.url).origin;
c.executionCtx.waitUntil(
 (async () => {
 let content: string;
 try {
 const result = await handleCommand(c.env, interaction, baseUrl);
 content = result.data.content;
 } catch (err) {
 console.error('Slash command failed', err);
 content = `Something broke: ${err as Error}.message`;
 }
 await editOriginalResponse(interaction.application_id, interaction.token, content);
 })(),
);

return c.json(defer(true));
}

if (interaction.type === InteractionType.MESSAGE_COMPONENT) {
 // Event sign-up buttons. Acknowledge with a deferred update (no visible
 // spinner), then apply the change and refresh the message in the background.
 if (isEventComponent(interaction)) {
 c.executionCtx.waitUntil(
 handleEventComponent(c.env, interaction).catch((err) =>
 console.error('Event component handler failed', err),
),
);
 return c.json(deferUpdate());
 }
 return c.json(ephemeral('This control is no longer active.'));
}

return c.json(ephemeral('Unsupported interaction type.'));
});

/* ----- */
/* Session */
/* ----- */

// API responses must never be cached by the browser or an intermediary.
// A cached /api/auth/login redirect (or a stale 404 from an earlier deploy)
// otherwise means the sign-in click is answered from cache and never reaches
// the Worker at all ? which looks exactly like a broken login.
app.use('/api/*', async (c, next) => {
 await next();
 // Re-wrap so the header sticks regardless of who built the response: a
 // mounted sub-router (e.g. /api/ranks) hands back a response with immutable
 // headers, so a direct .set() there silently no-ops. A fresh Response copies
 // the status, body, and existing headers into a mutable set.
 c.res = new Response(c.res.body, c.res);
 c.res.headers.set('Cache-Control', 'no-store, no-cache, must-revalidate');
});

```

**src/server/index.ts**

```
app.use('/api/*', withViewer);

app.get('/api/auth/login', async (c) => {
 const { discord } = await loadConfig(c.env);
 const state = newState();
 const redirectUri = new URL('/api/auth/callback', c.req.url).toString();
 c.header('Set-Cookie', stateCookie(state));
 return c.redirect(authorizeUrl(discord.clientId, redirectUri, state));
});

app.get('/api/auth/callback', async (c) => {
 // Discord signals its own failures (consent_required, access_denied, an
 // unregistered redirect_uri, ...) with an `error` query param and no code.
 // Surface it verbatim instead of mislabelling everything as a state error.
 const discordError = c.req.query('error');
 if (discordError) {
 console.error('Discord OAuth error:', discordError, c.req.query('error_description'));
 return c.redirect(`/login?error=discord&detail=${encodeURIComponent(discordError)}`);
 }

 const code = c.req.query('code');
 const state = c.req.query('state');
 const expected = getCookie(c, OAUTH_STATE_COOKIE);

 if (!code || !state || !expected || state !== expected) {
 return c.redirect(`/login?error=state`);
 }

 const { discord } = await loadConfig(c.env);
 const redirectUri = new URL('/api/auth/callback', c.req.url).toString();
 const token = await exchangeCode(discord.clientId, discord.clientSecret, code, redirectUri);

 const discordUser = await fetchUser(token.access_token);

 // Membership in the clan's Discord server is the gate. No invite, no account.
 const guildMember = await fetchGuildMember(token.access_token, discord.guildId);
 if (!guildMember) {
 c.header('Set-Cookie', clearedStateCookie);
 return c.redirect(`/login?error=not_in_guild`);
 }

 // Discord's guild-join date is the authoritative basis for tenure medals.
 const guildJoinedAt = guildMember.joined_at
 ? Math.floor(Date.parse(guildMember.joined_at) / 1000)
 : null;
 const validGuildJoinedAt =
 guildJoinedAt != null && !Number.isNaN(guildJoinedAt) ? guildJoinedAt : null;

 const database = db(c.env);
 const existing = await database.query.users.findFirst({
 where: eq(s.users.discordId, discordUser.id),
 });
});
```

**src/server/index.ts**

```

if (existing?.status === 'banned') {
 c.header('Set-Cookie', clearedStateCookie);
 return c.redirect('/login?error=banned');
}

let userId: number;
if (existing) {
 await database
 .update(s.users)
 .set({
 username: discordUser.username,
 globalName: discordUser.global_name,
 avatar: discordUser.avatar,
 email: discordUser.email ?? existing.email,
 // Only set once ? the guild-join date doesn't change, and a re-join
 // shouldn't reset accrued tenure.
 guildJoinedAt: existing.guildJoinedAt ?? validGuildJoinedAt,
 lastSeenAt: Math.floor(Date.now() / 1000),
 updatedAt: Math.floor(Date.now() / 1000),
 })
 .where(eq(s.users.id, existing.id));
 userId = existing.id;
} else {
 const defaultRank = await database.query.ranks.findFirst({
 where: eq(s.ranks.isDefault, true),
 });
 const inserted = await database
 .insert(s.users)
 .values({
 discordId: discordUser.id,
 username: discordUser.username,
 globalName: discordUser.global_name,
 avatar: discordUser.avatar,
 email: discordUser.email ?? null,
 rankId: defaultRank?.id ?? null,
 guildJoinedAt: validGuildJoinedAt,
 lastSeenAt: Math.floor(Date.now() / 1000),
 })
 .returning({ id: s.users.id });
 const created = inserted[0];
 if (!created) throw new Error('Failed to create member record');
 userId = created.id;

 await database.insert(s.auditLog).values({
 action: 'member.join',
 targetType: 'user',
 targetId: String(userId),
 meta: { discordId: discordUser.id, username: discordUser.username },
 source: 'system',
 });

 // Apply the default rank's roles to the new member. Done in the background
 // so the Discord round-trips don't hold up the login redirect.
 if (defaultRank) {

```

**src/server/index.ts**

```

 const client = await discordClient(c.env, database);
 c.executionCtx.waitUntil(
 syncMemberRankRoles(database, client, {
 userId,
 rankId: defaultRank.id,
 actorId: null,
 }),
);
 }
}

const { token: sessionToken, expiresAt } = await createSession(database, userId, {
 userAgent: c.req.header('user-agent'),
 ip: c.req.header('cf-connecting-ip'),
});

c.executionCtx.waitUntil(purgeExpiredSessions(database));

// Hand out any tenure medals this member has earned, so they show up the
// moment they log in rather than on the next cron sweep. DB-only, so it's
// safe to fire and forget.
c.executionCtx.waitUntil(
 awardTenureMedals(database, { userId }).catch((err) =>
 console.error('Tenure award on login failed', err),
),
);

c.header('Set-Cookie', clearedStateCookie);
c.header('Set-Cookie', sessionCookie(sessionToken, expiresAt - Math.floor(Date.now() / 1000)), {
 append: true,
});
return c.redirect('/');
});

app.post('/api/auth/logout', async (c) => {
 const token = getCookie(c, SESSION_COOKIE);
 if (token) await invalidateSession(db(c.env), token);
 c.header('Set-Cookie', clearedSessionCookie);
 return c.json({ ok: true });
});

/** What the React app calls on boot to learn who it is talking to. */
app.get('/api/me', async (c) => {
 const { siteName } = await loadConfig(c.env);
 return c.json({ viewer: c.get('viewer'), siteName, authKind: c.get('authKind') });
});

/**
 * API contract version, for the mobile/native client to check compatibility.
 * Public and unauthenticated. `apiVersion` is the integer contract version;
 * bump it only on a breaking change to the documented JSON shapes.
 */
app.get('/api/version', async (c) => {
 const { siteName } = await loadConfig(c.env);

```



**src/server/index.ts**

```

 return c.json({ apiVersion: 1, service: 'clantek', siteName });
 });

/* ----- *
 * API
 * ----- */

app.route('/api/ranks', ranks);
app.route('/api/members', members);
app.route('/api/settings', settings);
app.route('/api/roles', rolesRoutes);
app.route('/api/medals', medalsRoutes);
app.route('/api/media', mediaRoutes);
app.route('/api/news', newsRoutes);
app.route('/api/usage', usageRoutes);
app.route('/api/games', gamesRoutes);
app.route('/api/warrecords', warRecordsRoutes);
app.route('/api/events', eventsRoutes);
app.route('/api/audit', auditRoutes);
app.route('/api/pages', pagesRoutes);
app.route('/api/nav', navRoutes);
app.route('/api/hangar', hangarRoutes);
app.route('/api/sc', scVerifyRoutes);
app.route('/api/orgchart', orgChartRoutes);
app.route('/api/auth/tokens', tokensRoutes);

app.get('/api/health', (c) => c.json({ ok: true, service: 'clantek' }));

app.onError((err, c) => {
 console.error(err);
 return c.json({ error: err.message || 'Internal error' }, 500);
});

/* ----- *
 * Fallthrough
 *
 * Requests reach the Worker before the asset handler, so Hono's default
 * 404 would answer client-side routes like /admin/ranks and the SPA would
 * never load on refresh or deep link. Unmatched non-API paths are handed
 * to ASSETS, which wrangler.jsonc configures with
 * not_found_handling: "single-page-application".
 * ----- */

// Keep unmatched API routes answering JSON rather than serving index.html.
app.all('/api/*', (c) => c.json({ error: 'Not found' }, 404));

// Uploaded images (medal art, ...) live in R2 and are served here ? deliberately
// outside /api so the no-store middleware doesn't apply and they cache hard.
// wrangler.jsonc puts /media/* in run_worker_first so these reach the Worker.
app.get('/media/*', async (c) => {
 const res = await serveMediaObject(c.env, c.req.raw, c.executionCtx);
 return res ?? c.json({ error: 'Not found' }, 404);
});

```

**src/server/index.ts**

```

// Everything else is a static asset or an SPA navigation, served by ASSETS.
// For HTML (the SPA shell) we rewrite <head> from the site's SEO settings so the
// title and Open Graph tags are correct for the browser tab AND for crawlers /
// social scrapers (Discord, Google) that never run the React app. Non-HTML
// assets pass straight through untouched.
// Public, server-rendered version of the 'legal' custom page. The rest of the
// SPA is auth-gated, so this gives the footer's Legal link (and rights holders
// like RSI) a URL they can read WITHOUT an account ? and crawlers can index it.
// Reads the same page_layouts row members see; renders heading + text modules.
app.get('/legal', async (c) => {
 const esc = (v: string) =>
 v.replace(/&/g, '&').replace(/</g, '<').replace(/>/g, '>').replace(/"/g, '"');
 // Admin-authored page HTML is already script-free, but strip anything
 // executable as defense in depth before serving it publicly.
 const clean = (html: string) =>
 html
 .replace(/<\s*(script|style|iframe|object|embed)[\s\S]*?<\/\s*\1\s*>/gi, '')
 .replace(/\/son\w+\s*=\s*"([^"]*"|'[^']*')"/gi, '')
 .replace(/javascript:/gi, '');

 let title = 'Legal';
 let body = '';
 try {
 const row = await db(c.env).query.pageLayouts.findFirst({
 where: eq(s.pageLayouts.slug, 'legal'),
 });
 if (!row) return c.notFound();
 title = row.title || 'Legal';
 const layout = (row.layout ?? {}) as {
 rows?: { columns?: { modules?: { type?: string; config?: Record<string, unknown> }[] }[]
 }[];
 } catch {
 return c.notFound();
 }

 const { siteName } = await loadConfig(c.env);
 const site = siteName || 'mustr';
 const page =
 `<!doctype html><html lang="en"><head><meta charset="utf-8">` +
 `<meta name="viewport" content="width=device-width, initial-scale=1">` +

```

**src/server/index.ts**

```

 `<title>${esc(title)} ? ${esc(site)}</title><meta name="robots"
content="index,follow"><style>` +
 `:root{color-scheme:light dark}` +
 `body{margin:0;font:16px/1.6 system-ui,-apple-system,"Segoe
UI",Roboto,sans-serif;background:#0b0d12;color:#e7ebf3}` +
 `.wrap{max-width:760px;margin:0 auto;padding:3rem 1.25rem 4rem}` +
 `a{color:#5b8cff}h1{font-size:1.9rem;margin:0 0 1rem}h2{font-size:1.25rem;margin:2rem 0
.5rem}` +
 `p,li{color:#c2cadb}hr{border:none;border-top:1px solid #232a39;margin:2rem 0}` +
 `.home{display:inline-block;margin-top:1rem;font-size:.9rem}` +

`@media(prefers-color-scheme:light){body{background:#fff;color:#1a1f2b}p,li{color:#3a4353}hr{border-top-co
+
 `</style></head><body><div class="wrap">${body}<hr><-
${esc(site)}</div></body></html>`;

 return c.html(page, 200, { 'Cache-Control': 'public, max-age=300' });
 });

app.all('*', async (c) => {
 const res = await c.env.ASSETS.fetch(c.req.raw);
 const contentType = res.headers.get('content-type') ?? '';
 if (!contentType.includes('text/html')) return res;

 let head;
 try {
 head = renderHead(await loadSeo(c.env));
 } catch {
 return res; // never let a settings hiccup take the whole site down
 }

 const rewritten = new HTMLRewriter()
 .on('title', {
 element(el) {
 el.setInnerContent(head.title, { html: false });
 },
 })
 .on('meta[name="theme-color"]', {
 element(el) {
 if (head.themeColor) el.setAttribute('content', head.themeColor);
 },
 })
 .on('meta[name="apple-mobile-web-app-title"]', {
 element(el) {
 el.setAttribute('content', head.title);
 },
 })
 // Swap the default /icon.svg for the operator's uploaded favicon. We rewrite
 // the existing tags (rather than appending) so no competing icon lingers;
 // drop the stale `type` so the browser sniffs the uploaded format.
 .on('link[rel="icon"]', {
 element(el) {
 if (head.favicon) {
 el.setAttribute('href', head.favicon);

```

**src/server/index.ts**

```

 el.removeAttribute('type');
 }
},
})
.on('link[rel="apple-touch-icon"]', {
 element(el) {
 if (head.favicon) el.setAttribute('href', head.favicon);
 },
})
.on('head', {
 element(el) {
 el.append(head.tags, { html: true });
 },
})
})
.transform(res);

// Never edge-cache the shell: it's tiny, references hashed (cached) assets, and
// caching it would serve a stale injected <head> (old title/OG). no-store keeps
// the injected tags always fresh; a settings change shows on the next load.
const out = new Response(rewritten.body, rewritten);
out.headers.set('Cache-Control', 'no-store, must-revalidate');
return out;
});

/* ----- *
 * Scheduled work ? two cadences, set in wrangler.jsonc (triggers.crons):
 *
 * - every 5 min -> a bounded batch of the Discord reconcile sweep, so its
 * cost stays flat as the roster grows (see reconcileBatch).
 * - hourly -> the tenure-medal sweep. Tenure only changes on a daily
 * boundary, so a per-hour scan is plenty; new members still
 * get their tenure medals instantly at login.
 * ----- */

const TENURE_CRON = '7 * * * *';

export default {
 fetch: app.fetch,
 async scheduled(controller: ScheduledController, env: Env, ctx: ExecutionContext) {
 const database = db(env);

 if (controller.cron === TENURE_CRON) {
 // DB-only ? runs regardless of whether the bot is configured.
 ctx.waitUntil(
 awardTenureMedals(database)
 .then((r) => console.log('Hourly tenure award:', r.awarded.length, 'granted'))
 .catch((err) => console.error('Tenure award failed', err)),
);
 // Roll ended recurring events forward to their next occurrence. Safe with
 // or without the bot (the Discord sync no-ops when it's absent).
 ctx.waitUntil(
 materializeRecurringEvents(env)
 .then((r) => r.spawned && console.log('Recurring events spawned:', r.spawned))
 .catch((err) => console.error('Recurrence sweep failed', err)),
);
 }
 }
};

```

**src/server/index.ts**

```
);
 return;
 }

 // The 5-minute reconcile tick needs the bot.
 const rest = await discordClient(env, database);
 if (!rest) return;
 ctx.waitUntil(
 reconcileBatch(database, rest)
 .then((r) => console.log('Reconcile batch:', JSON.stringify(r)))
 .catch((err) => console.error('Reconcile batch failed', err)),
);
},
} satisfies ExportedHandler<Env>;
```

**src/server/medals/tenure.ts**

```

/**
 * Tenure medals: handed out automatically once a member has been in the guild
 * long enough. A medal opts in by setting `autoGrantMonths` (6, 12, 24, ...).
 *
 * This is pure database work ? no Discord calls ? so it's cheap to run for the
 * whole roster on the cron and for a single member at login. Basis for "time in
 * guild" is the Discord guild-join date when known (users.guildJoinedAt),
 * falling back to the site-join date so members always accrue *something*.
 */

import { and, eq, inArray, isNotNull, ne } from 'drizzle-orm';
import type { Drizzled1Database } from 'drizzle-orm/d1';
import * as s from '../db/schema';

type DB = Drizzled1Database<typeof s>;

// Average month. Tenure thresholds are coarse (6/12/24 months), so a day of
// drift at the boundary doesn't matter ? the next sweep catches it regardless.
const SECONDS_PER_MONTH = 2_629_746;

/** Whole months between `sinceSec` (unix seconds) and now. */
function monthsSince(sinceSec: number): number {
 const now = Math.floor(Date.now() / 1000);
 return Math.floor(Math.max(0, now - sinceSec) / SECONDS_PER_MONTH);
}

/** "6 months", "1 year", "2 years" ? for the auto-award citation. */
function tenureLabel(months: number): string {
 if (months % 12 === 0) {
 const years = months / 12;
 return `${years} year${years === 1 ? '' : 's'}`;
 }
 return `${months} months`;
}

export interface TenureResult {
 awarded: { userId: number; medalName: string }[];
}

/**
 * Grant every tenure medal a member has earned but doesn't yet hold. Pass a
 * userId to evaluate one member (login); omit it to sweep all non-banned
 * members (cron). Idempotent: a medal already held is skipped.
 */
export async function awardTenureMedals(
 db: DB,
 opts: { userId?: number } = {},
): Promise<TenureResult> {
 const tenureMedals = await db
 .select({ id: s.medals.id, name: s.medals.name, months: s.medals.autoGrantMonths })
 .from(s.medals)
 .where(isNotNull(s.medals.autoGrantMonths));
 if (!tenureMedals.length) return { awarded: [] };

```

**src/server/medals/tenure.ts**

```

const tenureMedalIds = tenureMedals.map((m) => m.id);

const members = await db
 .select({
 id: s.users.id,
 guildJoinedAt: s.users.guildJoinedAt,
 joinedAt: s.users.joinedAt,
 })
 .from(s.users)
 .where(opts.userId ? eq(s.users.id, opts.userId) : ne(s.users.status, 'banned'));
if (!members.length) return { awarded: [] };

// Which tenure medals each of these members already holds.
const memberIds = members.map((m) => m.id);
const existing = await db
 .select({ userId: s.memberMedals.userId, medalId: s.memberMedals.medalId })
 .from(s.memberMedals)
 .where(
 and(inArray(s.memberMedals.medalId, tenureMedalIds), inArray(s.memberMedals.userId,
memberIds)),
);
const held = new Set(existing.map((e) => `${e.userId}:${e.medalId}`));

const awarded: TenureResult['awarded'] = [];

for (const member of members) {
 const months = monthsSince(member.guildJoinedAt ?? member.joinedAt);
 for (const medal of tenureMedals) {
 if (months < medal.months!) continue;
 if (held.has(`${member.id}:${medal.id}`)) continue;

 await db.insert(s.memberMedals).values({
 userId: member.id,
 medalId: medal.id,
 citation: `${tenureLabel(medal.months!)} of service`,
 awardedBy: null,
 });
 await db.insert(s.auditLog).values({
 action: 'medal.auto_award',
 targetType: 'user',
 targetId: String(member.id),
 meta: { medalId: medal.id, medalName: medal.name, months: medal.months },
 source: 'system',
 });
 awarded.push({ userId: member.id, medalName: medal.name });
 }
}

return { awarded };
}

```

**src/server/middleware/auth.ts**

```

import { createMiddleware } from 'hono/factory';
import { getCookie } from 'hono/cookie';
import { drizzle } from 'drizzle-orm/d1';
import * as schema from '../db/schema';
import { SESSION_COOKIE, resolveViewer } from '../auth/session';
import { resolveApiToken } from '../auth/apiToken';
import { can, type Permission } from '../shared/permissions';
import type { AppContext } from '../env';

export function db(env: { DB: D1Database }) {
 return drizzle(env.DB, { schema });
}

/**
 * Attaches the current Viewer (or null) to every request. Never rejects.
 * An `Authorization: Bearer <token>` header (the mobile/native app) is tried
 * first; otherwise the browser session cookie. `authKind` records which won, so
 * routes can, e.g., require a real web session for sensitive account actions.
 */
export const withViewer = createMiddleware<AppContext>(async (c, next) => {
 const auth = c.req.header('Authorization');
 if (auth && /^Bearer\s+/i.test(auth)) {
 const raw = auth.replace(/^Bearer\s+/i, '').trim();
 const viewer = await resolveApiToken(db(c.env), raw);
 c.set('viewer', viewer);
 c.set('authKind', viewer ? 'token' : null);
 } else {
 const viewer = await resolveViewer(db(c.env), getCookie(c, SESSION_COOKIE));
 c.set('viewer', viewer);
 c.set('authKind', viewer ? 'web' : null);
 }
 await next();
});

/**
 * Requires that the caller authenticated with a browser session, not an API
 * token ? used to contain a leaked token: it can read/act per its permissions
 * but cannot manage credentials (mint or revoke tokens, etc.).
 */
export const requireWebSession = createMiddleware<AppContext>(async (c, next) => {
 if (!c.get('viewer')) return c.json({ error: 'Authentication required' }, 401);
 if (c.get('authKind') !== 'web') {
 return c.json({ error: 'This action requires signing in through the website.' }, 403);
 }
 await next();
});

/** Requires a signed-in member. */
export const requireAuth = createMiddleware<AppContext>(async (c, next) => {
 if (!c.get('viewer')) {
 return c.json({ error: 'Authentication required' }, 401);
 }
 await next();
});

```



**src/server/middleware/auth.ts**

```
/**
 * Requires a specific permission. God bypasses this ? see can() for why that
 * escape hatch exists.
 */
export function requirePermission(permission: Permission) {
 return createMiddleware<AppContext>(async (c, next) => {
 const viewer = c.get('viewer');
 if (!viewer) return c.json({ error: 'Authentication required' }, 401);
 if (!can(viewer, permission)) {
 return c.json({ error: 'Forbidden', missing: permission }, 403);
 }
 await next();
 });
}
```

**src/server/modules.ts**

```

/**
 * Game modules ? optional, per-install features an operator turns on in Settings.
 *
 * Star Citizen is the first. Stored as one flags blob in settings['modules'];
 * everything defaults OFF so a fresh install ships lean. Resolved DB-over-nothing
 * (there's no env fallback ? modules are purely an admin choice). Same shape as
 * the other settings-backed config so it reads consistently.
 */

import { drizzle } from 'drizzle-orm/d1';
import { eq } from 'drizzle-orm';
import * as schema from '../db/schema';
import * as s from '../db/schema';
import type { Env } from '../env';

export const MODULES_KEY = 'modules';

export interface ModuleFlags {
 starcitizen: boolean;
}

type DB = ReturnType<typeof drizzle<typeof schema>>;

/** The enabled-module flags, all false unless an admin turned one on. */
export async function loadModules(env: Env, database?: DB): Promise<ModuleFlags> {
 const dbi = database ?? drizzle(env.DB, { schema });
 let stored: Record<string, unknown> = {};
 try {
 const row = await dbi.query.settings.findFirst({ where: eq(s.settings.key, MODULES_KEY) });
 if (row?.value && typeof row.value === 'object') stored = row.value as Record<string,
unknown>;
 } catch {
 // No settings row/table yet -> everything off.
 }
 return { starcitizen: stored.starcitizen === true };
}

/** Sanitise an incoming flags payload to exactly the known booleans. */
export function cleanModuleFlags(raw: unknown): ModuleFlags {
 const o = (raw ?? {}) as Record<string, unknown>;
 return { starcitizen: o.starcitizen === true };
}

/* ----- */
/* Star Citizen module config ? settings that only matter when the SC module is
 * on. Kept in its own settings key so the module flags stay pure booleans. The
 * org SID lets account verification confirm a member's RSI profile lists THIS
 * install's org (each buyer sets their own).
 * ----- */

export const SC_KEY = 'sc';

export interface ScConfig {
 /** This org's RSI Spectrum Identification (SID), e.g. "F919". Case-insensitive. */

```

**src/server/modules.ts**

```
orgSid: string;
/**
 * Per-feature kill switches for the two things that touch RSI. Both default
 * ON; an admin can turn either off instantly (e.g. if CIG/RSI ever asks) ?
 * `hangarEnabled` gates the member's own hangar import + display, and
 * `verifyEnabled` gates the account-verification profile fetch. The whole SC
 * module toggle remains the master switch above these.
 */
hangarEnabled: boolean;
verifyEnabled: boolean;
}

export async function loadScConfig(env: Env, database?: DB): Promise<ScConfig> {
 const dbi = database ?? drizzle(env.DB, { schema });
 let stored: Record<string, unknown> = {};
 try {
 const row = await dbi.query.settings.findFirst({ where: eq(s.settings.key, SC_KEY) });
 if (row?.value && typeof row.value === 'object') stored = row.value as Record<string,
unknown>;
 } catch {
 // No settings row/table yet -> no org, features default on.
 }
 return {
 orgSid: typeof stored.orgSid === 'string' ? stored.orgSid : '',
 // Absent (older config) or true ? on; only an explicit false disables.
 hangarEnabled: stored.hangarEnabled !== false,
 verifyEnabled: stored.verifyEnabled !== false,
 };
}

export function cleanScConfig(raw: unknown): ScConfig {
 const o = (raw ?? {}) as Record<string, unknown>;
 return {
 orgSid: (typeof o.orgSid === 'string' ? o.orgSid : '').trim().slice(0, 20),
 hangarEnabled: o.hangarEnabled !== false,
 verifyEnabled: o.verifyEnabled !== false,
 };
}
```

**src/server/routes/audit.ts**

```

/**
 * The activity log ? who did what. Every mutation across the site already
 * writes an audit_log row (promotions, demotions, medal and war-record awards
 * and revocations, role changes, status changes, ...); this route is the read
 * side, gated on audit.view so only trusted members can see it.
 *
 * News and event posts are deliberately absent from the useful view: those
 * already carry a visible author, so they don't need to surface here.
 */

import { Hono } from 'hono';
import { and, desc, eq, like, sql } from 'drizzle-orm';
import { alias } from 'drizzle-orm/sqlite-core';
import * as s from '../db/schema';
import type { AppContext } from '../env';
import { db, requirePermission } from '../middleware/auth';
import { memberName } from '../shared/names';

const audit = new Hono<AppContext>();

/**
 * A page of log entries, most recent first. Filters (all optional):
 * action ? prefix match, so "medal" catches medal.award and medal.revoke,
 * while "medal.revoke" is exact.
 * actorId ? only what this member did.
 * targetId ? only entries about this member (targetType=user).
 * Paginated with limit/offset; the client loads more until a short page.
 */
audit.get('/', requirePermission('audit.view'), async (c) => {
 const database = db(c.env);
 const limit = Math.min(Math.max(Number(c.req.query('limit')) || 50, 1), 100);
 const offset = Math.max(Number(c.req.query('offset')) || 0, 0);
 const action = c.req.query('action')?.trim();
 const actorId = c.req.query('actorId') ? Number(c.req.query('actorId')) : null;
 const targetId = c.req.query('targetId')?.trim();

 const actor = alias(s.users, 'actor');
 const target = alias(s.users, 'target');

 const filters = [
 action ? like(s.auditLog.action, `${action}%`) : undefined,
 actorId ? eq(s.auditLog.actorId, actorId) : undefined,
 targetId ? eq(s.auditLog.targetId, targetId) : undefined,
].filter(Boolean);

 const rows = await database
 .select({
 id: s.auditLog.id,
 action: s.auditLog.action,
 reason: s.auditLog.reason,
 meta: s.auditLog.meta,
 source: s.auditLog.source,
 createdAt: s.auditLog.createdAt,
 targetType: s.auditLog.targetType,

```

**src/server/routes/audit.ts**

```

 targetId: s.auditLog.targetId,
 actorId: actor.id,
 actorUsername: actor.username,
 actorGlobalName: actor.globalName,
 actorDisplayName: actor.displayName,
 actorDiscordId: actor.discordId,
 actorAvatar: actor.avatar,
 actorProfileImageUrl: actor.profileImageUrl,
 targetUserId: target.id,
 targetUsername: target.username,
 targetGlobalName: target.globalName,
 targetDisplayName: target.displayName,
 })
 .from(s.auditLog)
 .leftJoin(actor, eq(actor.id, s.auditLog.actorId))
 // Resolve the target's name only when it is a member; other target types
 // (news, ranks, ...) reuse the numeric id space, so gate the join on user.
 .leftJoin(
 target,
 and(eq(s.auditLog.targetType, 'user'), eq(target.id, sql`cast(${s.auditLog.targetId} as integer)`)),
)
 .where(filters.length ? and(...filters) : undefined)
 .orderBy(desc(s.auditLog.createdAt), desc(s.auditLog.id))
 .limit(limit)
 .offset(offset);

const entries = rows.map((r) => ({
 id: r.id,
 action: r.action,
 reason: r.reason,
 meta: r.meta,
 source: r.source,
 createdAt: r.createdAt,
 targetType: r.targetType,
 targetId: r.targetId,
 // System actions (the tenure sweep, member.join) have no actor.
 actor: r.actorId
 ? {
 id: r.actorId,
 name: memberName({
 displayName: r.actorDisplayName,
 globalName: r.actorGlobalName,
 username: r.actorUsername ?? 'unknown',
 }),
 discordId: r.actorDiscordId,
 avatar: r.actorAvatar,
 profileImageUrl: r.actorProfileImageUrl,
 }
 : null,
 target: r.targetUserId
 ? {
 id: r.targetUserId,
 name: memberName({

```

**src/server/routes/audit.ts**

```
 displayName: r.targetDisplayName,
 globalName: r.targetGlobalName,
 username: r.targetUsername ?? 'unknown',
 })),
 }
 : null,
 }));

 return c.json({ entries, limit, offset, hasMore: rows.length === limit });
});

export default audit;
```

**src/server/routes/events.ts**

```

/**
 * Events ? clan happenings that mirror to Discord (a native scheduled event +
 * an announcement message that doubles as a sign-up sheet). Viewing needs
 * events.view; creating/editing/cancelling needs events.manage; seeing the full
 * attendee list needs events.attendees. Members with events.view can sign
 * themselves up (and pick a role). Times are unix seconds (UTC).
 */

import { Hono } from 'hono';
import { and, asc, eq, gte, inArray, sql } from 'drizzle-orm';
import * as s from '../db/schema';
import type { AppContext } from '../env';
import { db, requirePermission } from '../middleware/auth';
import { syncEventToDiscord, removeEventFromDiscord, refreshEventMessage } from
'../discord/events';
import { loadEventState, setSignup, removeSignup } from '../events/signups';

const events = new Hono<AppContext>();

interface RoleInput {
 id?: number;
 name: string;
 emoji?: string | null;
 capacity?: number | null;
}

type Recurrence = 'none' | 'daily' | 'weekly' | 'biweekly' | 'monthly';
const RECURRENCES: Recurrence[] = ['none', 'daily', 'weekly', 'biweekly', 'monthly'];

interface EventBody {
 title?: string;
 description?: string | null;
 imageUrl?: string | null;
 startsAt?: number;
 endsAt?: number;
 location?: string;
 gameId?: number | null;
 roles?: RoleInput[];
 recurrence?: Recurrence;
}

/** Validate the time/text fields shared by create and edit. Returns an error string or null. */
function validate(b: EventBody, requireAll: boolean): string | null {
 if (requireAll || b.title !== undefined) {
 if (!b.title?.trim()) return 'A title is required.';
 }
 if (requireAll || b.location !== undefined) {
 if (!b.location?.trim()) return 'A location is required.';
 }
 const bothTimes = b.startsAt !== undefined && b.endsAt !== undefined;
 if (requireAll || bothTimes) {
 if (!Number.isFinite(b.startsAt) || !Number.isFinite(b.endsAt)) return 'Start and end times
are required.';
 if ((b.endsAt as number) <= (b.startsAt as number)) return 'The end time must be after the
start time.';
 }
}

```

**src/server/routes/events.ts**

```

 } else if (b.startsAt !== undefined || b.endsAt !== undefined) {
 return 'Change the start and end times together.';
 }
 if (b.imageUrl !== null && b.imageUrl !== '' && !b.imageUrl.startsWith('/media/events/')) {
 return 'Invalid event image.';
 }
 if (b.recurrence !== undefined && !RECURRENCES.includes(b.recurrence)) {
 return 'Invalid recurrence.';
 }
 return null;
 }
}

/**
 * Persist an event's roles. When `roles` is given it's the full desired set:
 * rows carrying an `id` are updated, new rows inserted, and any existing role
 * not present is deleted (freeing its holders to "attending, no role"). Called
 * on create (existing empty) and on edit (when roles are supplied).
 */
async function saveRoles(database: ReturnType<typeof db>, eventId: number, roles: RoleInput[]):
Promise<void> {
 const clean = roles
 .map((r, idx) => ({
 id: r.id,
 name: (r.name ?? '').trim().slice(0, 40),
 emoji: r.emoji?.trim()?.slice(0, 8) || null,
 capacity: r.capacity !== null && r.capacity > 0 ? Math.min(Math.floor(r.capacity), 999) :
null,
 sortOrder: idx,
 })))
 .filter((r) => r.name)
 .slice(0, 20);

 const existing = await database
 .select({ id: s.eventRoles.id })
 .from(s.eventRoles)
 .where(eq(s.eventRoles.eventId, eventId));
 const existingIds = new Set(existing.map((e) => e.id));
 const keepIds = new Set(clean.filter((r) => r.id && existingIds.has(r.id)).map((r) => r.id!));

 const toDelete = existing.filter((e) => !keepIds.has(e.id)).map((e) => e.id);
 if (toDelete.length) {
 await database.delete(s.eventRoles).where(inArray(s.eventRoles.id, toDelete));
 }

 for (const r of clean) {
 if (r.id && existingIds.has(r.id)) {
 await database
 .update(s.eventRoles)
 .set({ name: r.name, emoji: r.emoji, capacity: r.capacity, sortOrder: r.sortOrder })
 .where(and(eq(s.eventRoles.id, r.id), eq(s.eventRoles.eventId, eventId)));
 } else {
 await database
 .insert(s.eventRoles)
 .values({ eventId, name: r.name, emoji: r.emoji, capacity: r.capacity, sortOrder:

```



**src/server/routes/events.ts**

```

r.sortOrder });
 }
 }
}

/** Upcoming (and in-progress) events with their roles, counts, and the viewer's own sign-up. */
events.get('/', requirePermission('events.view'), async (c) => {
 const viewer = c.get('viewer')!;
 const nowSec = Math.floor(Date.now() / 1000);
 const database = db(c.env);

 const rows = await database
 .select({
 id: s.events.id,
 title: s.events.title,
 description: s.events.description,
 imageUrl: s.events.imageUrl,
 startsAt: s.events.startsAt,
 endsAt: s.events.endsAt,
 location: s.events.location,
 gameId: s.events.gameId,
 gameName: s.games.name,
 createdBy: s.events.createdBy,
 recurrence: s.events.recurrence,
 })
 .from(s.events)
 .leftJoin(s.games, eq(s.events.gameId, s.games.id))
 .where(and(eq(s.events.status, 'scheduled'), gte(s.events.endsAt, nowSec)))
 .orderBy(asc(s.events.startsAt));

 const ids = rows.map((r) => r.id);
 if (ids.length === 0) return c.json({ events: [] });

 // Three bounded queries regardless of how many events are listed.
 const [roleRows, countRows, mine] = await Promise.all([
 database
 .select()
 .from(s.eventRoles)
 .where(inArray(s.eventRoles.eventId, ids))
 .orderBy(asc(s.eventRoles.sortOrder), asc(s.eventRoles.id)),
 database
 .select({ eventId: s.eventSignups.eventId, roleId: s.eventSignups.eventRoleId, n:
sql<number>`count(*)` })
 .from(s.eventSignups)
 .where(inArray(s.eventSignups.eventId, ids))
 .groupBy(s.eventSignups.eventId, s.eventSignups.eventRoleId),
 database
 .select({ eventId: s.eventSignups.eventId, roleId: s.eventSignups.eventRoleId })
 .from(s.eventSignups)
 .where(and(inArray(s.eventSignups.eventId, ids), eq(s.eventSignups.userId, viewer.id))),
]);

 const totalByEvent = new Map<number, number>();
 const roleCount = new Map<string, number>();

```

**src/server/routes/events.ts**

```

for (const cr of countRows) {
 const n = Number(cr.n);
 totalByEvent.set(cr.eventId, (totalByEvent.get(cr.eventId) ?? 0) + n);
 if (cr.roleId != null) roleCount.set(`${cr.eventId}:${cr.roleId}`, n);
}

const rolesByEvent = new Map<number, unknown[]>();
for (const r of roleRows) {
 const list = rolesByEvent.get(r.eventId) ?? [];
 list.push({
 id: r.id,
 name: r.name,
 emoji: r.emoji,
 capacity: r.capacity,
 count: roleCount.get(`${r.eventId}:${r.id}`) ?? 0,
 });
 rolesByEvent.set(r.eventId, list);
}

const mineByEvent = new Map<number, number | null>();
for (const m of mine) mineByEvent.set(m.eventId, m.roleId);

const enriched = rows.map((r) => ({
 ...r,
 roles: rolesByEvent.get(r.id) ?? [],
 signupCount: totalByEvent.get(r.id) ?? 0,
 mySignup: mineByEvent.has(r.id) ? { roleId: mineByEvent.get(r.id) ?? null } : null,
}));

return c.json({ events: enriched });
});

/** The full sign-up roster for one event ? who's coming, and as what. */
events.get('/:id/signups', requirePermission('events.attendees'), async (c) => {
 const id = Number(c.req.param('id'));
 const state = await loadEventState(db(c.env), id);
 if (!state) return c.json({ error: 'No such event' }, 404);
 return c.json({ signups: state.signups, roles: state.roles, total: state.total });
});

events.post('/', requirePermission('events.manage'), async (c) => {
 const body = await c.req.json<EventBody>();
 const err = validate(body, true);
 if (err) return c.json({ error: err }, 400);

 const database = db(c.env);
 const viewer = c.get('viewer')!;
 const created = (
 await database
 .insert(s.events)
 .values({
 title: body.title!.trim().slice(0, 120),
 description: body.description?.trim().slice(0, 1500) || null,
 imageUrl: body.imageUrl?.trim() || null,

```

**src/server/routes/events.ts**

```

 startsAt: body.startsAt!,
 endsAt: body.endsAt!,
 location: body.location!.trim().slice(0, 100),
 gameId: body.gameId ?? null,
 createdBy: viewer.id,
 recurrence: body.recurrence ?? 'none',
 })
 .returning()
)[0]!;

if (body.roles) await saveRoles(database, created.id, body.roles);

await database.insert(s.auditLog).values({
 actorId: viewer.id,
 action: 'event.create',
 targetType: 'event',
 targetId: String(created.id),
 meta: { title: created.title },
});

// Mirror to Discord in the background, then store the returned ids.
c.executionCtx.waitUntil(
 (async () => {
 const ids = await syncEventToDiscord(c.env, created.id);
 if (ids.discordEventId !== created.discordEventId || ids.discordMessageId !==
created.discordMessageId) {
 await database.update(s.events).set(ids).where(eq(s.events.id, created.id));
 }
 })(),
);

return c.json({ event: created }, 201);
});

events.patch('/:id', requirePermission('events.manage'), async (c) => {
 const id = Number(c.req.param('id'));
 const body = await c.req.json<EventBody>();
 const err = validate(body, false);
 if (err) return c.json({ error: err }, 400);

 const database = db(c.env);
 const existing = await database.query.events.findFirst({ where: eq(s.events.id, id) });
 if (!existing) return c.json({ error: 'No such event' }, 404);

 const patch: Partial<typeof s.events.$inferInsert> = { updatedAt: Math.floor(Date.now() / 1000)
};
 if (body.title !== undefined) patch.title = body.title.trim().slice(0, 120);
 if (body.description !== undefined) patch.description = body.description?.trim().slice(0, 1500)
|| null;
 if (body.imageUrl !== undefined) patch.imageUrl = body.imageUrl?.trim() || null;
 if (body.location !== undefined) patch.location = body.location.trim().slice(0, 100);
 if (body.startsAt !== undefined) patch.startsAt = body.startsAt;
 if (body.endsAt !== undefined) patch.endsAt = body.endsAt;
 if (body.gameId !== undefined) patch.gameId = body.gameId ?? null;

```

**src/server/routes/events.ts**

```

 if (body.recurrence !== undefined) patch.recurrence = body.recurrence;

 const updated = (await database.update(s.events).set(patch).where(eq(s.events.id,
id)).returning())[0]!;
 if (body.roles !== undefined) await saveRoles(database, id, body.roles);

 c.executionCtx.waitUntil(
 (async () => {
 const ids = await syncEventToDiscord(c.env, id);
 if (ids.discordEventId !== updated.discordEventId || ids.discordMessageId !==
updated.discordMessageId) {
 await database.update(s.events).set(ids).where(eq(s.events.id, id));
 }
 })(),
);

 return c.json({ event: updated });
 });

events.delete('/:id', requirePermission('events.manage'), async (c) => {
 const id = Number(c.req.param('id'));
 const database = db(c.env);
 const event = await database.query.events.findFirst({ where: eq(s.events.id, id) });
 if (!event) return c.json({ error: 'No such event' }, 404);

 await database.delete(s.events).where(eq(s.events.id, id));
 c.executionCtx.waitUntil(removeEventFromDiscord(c.env, event));

 await database.insert(s.auditLog).values({
 actorId: c.get('viewer')!.id,
 action: 'event.delete',
 targetType: 'event',
 targetId: String(id),
 meta: { title: event.title },
 });

 return c.json({ ok: true });
});

/** Sign the viewer up (or move them to a different role). roleId null = no specific role. */
events.post('/:id/signup', requirePermission('events.view'), async (c) => {
 const id = Number(c.req.param('id'));
 const { roleId } = await c.req.json<{ roleId?: number | null }>();
 const viewer = c.get('viewer')!;

 const res = await setSignup(db(c.env), id, viewer.id, roleId ?? null);
 if (!res.ok) return c.json({ error: res.error }, 400);

 c.executionCtx.waitUntil(refreshEventMessage(c.env, id));
 return c.json({ ok: true });
});

/** Withdraw the viewer's own sign-up. */
events.delete('/:id/signup', requirePermission('events.view'), async (c) => {

```

**src/server/routes/events.ts**

```
const id = Number(c.req.param('id'));
const viewer = c.get('viewer')!;

await removeSignup(db(c.env), id, viewer.id);
c.executionCtx.waitUntil(refreshEventMessage(c.env, id));
return c.json({ ok: true });
});

export default events;
```

**src/server/routes/games.ts**

```

/**
 * Games catalog ? the titles the clan plays. A war record (and a medal) can be
 * tied to one, or left clan-wide. Editable end to end from the admin panel.
 */

import { Hono } from 'hono';
import { and, asc, eq, ne, sql } from 'drizzle-orm';
import * as s from '../../db/schema';
import type { AppContext } from '../../env';
import { db, requireAuth, requirePermission } from '../../middleware/auth';
import { deleteMediaByUrl } from '../../media';

const games = new Hono<AppContext>();

function slugify(name: string): string {
 return (
 name
 .toLowerCase()
 .trim()
 .replace(/[^\a-z0-9]+/g, '-')
 .replace(/^-+|-+$/g, '')
 .slice(0, 60) || 'game'
);
}

async function uniqueSlug(database: ReturnType<typeof db>, base: string, ignoreId?: number):
Promise<string> {
 let slug = base;
 let n = 1;
 // eslint-disable-next-line no-constant-condition
 while (true) {
 const clash = await database.query.games.findFirst({
 where: ignoreId ? and(eq(s.games.slug, slug), ne(s.games.id, ignoreId)) : eq(s.games.slug,
slug),
 });
 if (!clash) return slug;
 n += 1;
 slug = `${base}-${n}`;
 }
}

/** Every game, with how many war records reference it. Visible to any member. */
games.get('/', requireAuth, async (c) => {
 const rows = await db(c.env)
 .select({
 id: s.games.id,
 name: s.games.name,
 slug: s.games.slug,
 iconUrl: s.games.iconUrl,
 active: s.games.active,
 sortOrder: s.games.sortOrder,
 recordCount: sql<number>`(select count(*) from war_records where war_records.game_id =
games.id)`,
 })
});

```

**src/server/routes/games.ts**

```

 .from(s.games)
 .orderBy(asc(s.games.sortOrder), asc(s.games.name));

 return c.json({ games: rows });
});

games.post('/', requirePermission('games.manage'), async (c) => {
 const body = await c.req.json<{ name?: string; imageUrl?: string; active?: boolean }>();
 if (!body.name?.trim()) return c.json({ error: 'A name is required' }, 400);

 const database = db(c.env);
 const slug = await uniqueSlug(database, slugify(body.name));
 const highest = await database.select({ max: sql<number | null>`max(${s.games.sortOrder})`
}).from(s.games);

 const created = (
 await database
 .insert(s.games)
 .values({
 name: body.name.trim().slice(0, 100),
 slug,
 imageUrl: body.imageUrl?.trim() || null,
 active: body.active ?? true,
 sortOrder: (highest[0]?.max ?? -1) + 1,
 })
 .returning()
)[0]!;

 await database.insert(s.auditLog).values({
 actorId: c.get('viewer')!.id,
 action: 'game.create',
 targetType: 'game',
 targetId: String(created.id),
 meta: { name: created.name },
 });

 return c.json({ game: { ...created, recordCount: 0 } }, 201);
});

games.patch('/:id', requirePermission('games.manage'), async (c) => {
 const id = Number(c.req.param('id'));
 const body = await c.req.json<{ name?: string; imageUrl?: string | null; active?: boolean;
sortOrder?: number }>();

 const database = db(c.env);
 const game = await database.query.games.findFirst({ where: eq(s.games.id, id) });
 if (!game) return c.json({ error: 'No such game' }, 404);

 const patch: Partial<typeof s.games.$inferInsert> = {};
 if (body.name !== undefined) {
 if (!body.name.trim()) return c.json({ error: 'Name cannot be empty' }, 400);
 patch.name = body.name.trim().slice(0, 100);
 }
 if (body.imageUrl !== undefined) {

```

**src/server/routes/games.ts**

```
 patch.iconUrl = body.iconUrl?.trim() || null;
 if (patch.iconUrl !== game.iconUrl) c.executionCtx.waitUntil(deleteMediaByUrl(c.env,
game.iconUrl));
 }
 if (body.active !== undefined) patch.active = body.active;
 if (body.sortOrder !== undefined) patch.sortOrder = body.sortOrder;

 const updated = (await database.update(s.games).set(patch).where(eq(s.games.id,
id)).returning())[0]!;
 return c.json({ game: updated });
});

games.delete('/:id', requirePermission('games.manage'), async (c) => {
 const id = Number(c.req.param('id'));
 const database = db(c.env);
 const game = await database.query.games.findFirst({ where: eq(s.games.id, id) });
 if (!game) return c.json({ error: 'No such game' }, 404);

 await database.delete(s.games).where(eq(s.games.id, id));
 c.executionCtx.waitUntil(deleteMediaByUrl(c.env, game.iconUrl));

 await database.insert(s.auditLog).values({
 actorId: c.get('viewer')!.id,
 action: 'game.delete',
 targetType: 'game',
 targetId: String(id),
 meta: { name: game.name },
 });

 return c.json({ ok: true });
});

export default games;
```



**src/server/routes/hangar.ts**

```

/**
 * Star Citizen hangar ? import + read, gated on the SC module being enabled.
 *
 * A member imports their OWN hangar (PUT, from a bookmarklet paste). Viewing
 * SOMEONE ELSE'S hangar takes two things: the viewer must hold `hangar.view`,
 * AND that member must have opted their hangar public (self-service, any
 * rank/role). Owners always see their own. When the module is off, every route
 * 404s so nothing leaks.
 */

import { Hono } from 'hono';
import { eq } from 'drizzle-orm';
import * as s from '../db/schema';
import type { AppContext } from '../env';
import { db, requireAuth } from '../middleware/auth';
import { loadModules, loadScConfig } from '../modules';
import { sanitizeHangar } from '../shared/hangar';
import { can } from '../shared/permissions';

const hangar = new Hono<AppContext>();

/**
 * Hangar available for this install? The SC module must be on AND the hangar
 * feature not killed (its own kill switch, independent of verification).
 */
async function scEnabled(env: AppContext['Bindings']): Promise<boolean> {
 const [mods, cfg] = await Promise.all([loadModules(env, db(env)), loadScConfig(env, db(env))]);
 return mods.starcitizen && cfg.hangarEnabled;
}

/**
 * A member's stored hangar ? subject to visibility. Owners always get theirs
 * (with the public flag so their toggle reflects state); others get it only
 * with `hangar.view` AND when the owner has shared it. `canView`/`isPublic` let
 * the client explain an empty result ("private") without leaking the contents.
 */
hangar.get('/:userId', requireAuth, async (c) => {
 if (!(await scEnabled(c.env))) return c.json({ error: 'Module not enabled.' }, 404);
 const viewer = c.get('viewer')!;
 const userId = Number(c.req.param('userId'));
 const self = viewer.id === userId;

 // A member with no view permission never sees another member's hangar.
 if (!self && !can(viewer, 'hangar.view')) {
 return c.json({ hangar: null, self: false, canView: false, isPublic: false });
 }

 const row = await db(c.env).query.scHangars.findFirst({ where: eq(s.scHangars.userId, userId) });
 const isPublic = !!row?.isPublic;
 const allowed = self || isPublic; // permission already confirmed above

 // Monetary value is the main incentive to inflate a self-reported hangar, so it
 // is withheld from the response unless the viewer owns the hangar or holds

```

**src/server/routes/hangar.ts**

```

// `hangar.value` ? the client can't read it off the network even though the
// roster also hides it visually.
const canValue = self || can(viewer, 'hangar.value');
const items =
 allowed && row
 ? canValue
 ? row.items
 : row.items.map((i) => ({ ...i, value: '' }))
 : null;

return c.json({
 hangar: items ? { items, itemCount: row!.itemCount, importedAt: row!.importedAt } : null,
 self,
 canView: allowed,
 isPublic,
});
});

/** The signed-in member's own hangar sharing state (for their toggle). */
hangar.get('/me/visibility', requireAuth, async (c) => {
 if (!(await scEnabled(c.env))) return c.json({ error: 'Module not enabled.' }, 404);
 const viewer = c.get('viewer')!;
 const row = await db(c.env).query.scHangars.findFirst({ where: eq(s.scHangars.userId, viewer.id) });
});
return c.json({ hasHangar: !!row, isPublic: !!row?.isPublic });
});

/** Set whether the signed-in member's own hangar is shared (self-service). */
hangar.patch('/visibility', requireAuth, async (c) => {
 if (!(await scEnabled(c.env))) return c.json({ error: 'Module not enabled.' }, 404);
 const viewer = c.get('viewer')!;
 const body = await c.req.json<{ public?: unknown }>().catch(() => ({})) as { public?: unknown };
 const isPublic = body.public === true;
 await db(c.env).update(s.scHangars).set({ isPublic }).where(eq(s.scHangars.userId, viewer.id));
 return c.json({ ok: true, isPublic });
});

/** Import (replace) the signed-in member's hangar. */
hangar.put('/', requireAuth, async (c) => {
 if (!(await scEnabled(c.env))) return c.json({ error: 'Module not enabled.' }, 404);
 const viewer = c.get('viewer')!;
 const body = await c.req.json<{ items?: unknown }>().catch(() => ({})) as { items?: unknown };
 const items = sanitizeHangar(body.items);
 if (!items.length) {
 return c.json({ error: 'No hangar items found in that paste ? re-copy from your RSI hangar.' }, 400);
 }

 const now = Math.floor(Date.now() / 1000);
 await db(c.env)
 .insert(s.scHangars)
 .values({ userId: viewer.id, items, itemCount: items.length, importedAt: now })
 .onConflictDoUpdate({
 target: s.scHangars.userId,

```

**src/server/routes/hangar.ts**

```
 set: { items, itemCount: items.length, importedAt: now },
 });

 await db(c.env).insert(s.auditLog).values({
 actorId: viewer.id,
 action: 'hangar.import',
 targetType: 'user',
 targetId: String(viewer.id),
 meta: { itemCount: items.length },
 });

 return c.json({ ok: true, itemCount: items.length });
});

/** Clear the signed-in member's own hangar. */
hangar.delete('/', requireAuth, async (c) => {
 const viewer = c.get('viewer')!;
 await db(c.env).delete(s.scHangars).where(eq(s.scHangars.userId, viewer.id));
 return c.json({ ok: true });
});

export default hangar;
```

**src/server/routes/medals.ts**

```

/**
 * The medal catalog ? the definitions, not the awards.
 *
 * Awarding and revoking a medal on a specific member lives with the other
 * member-scoped actions in routes/members.ts (mirroring how role grants do).
 * A medal with autoGrantMonths set is handed out automatically by the tenure
 * sweep (server/medals/tenure.ts); it can still be awarded by hand as well.
 */

import { Hono } from 'hono';
import { asc, eq, sql } from 'drizzle-orm';
import * as s from '../db/schema';
import type { AppContext } from '../env';
import { db, requireAuth, requirePermission } from '../middleware/auth';
import { deleteMediaByUrl } from '../media';

const medals = new Hono<AppContext>();

/** NULL, or a positive whole number of months. Anything else is a 400. */
function parseAutoGrantMonths(v: unknown): number | null | undefined {
 if (v === undefined) return undefined; // field omitted ? leave unchanged
 if (v === null || v === '') return null;
 const n = Number(v);
 if (!Number.isInteger(n) || n <= 0) throw new Error('Auto-grant months must be a positive whole number.');
```

```

 return n;
}

/** Every medal, with how many members hold it. Visible to any signed-in member. */
medals.get('/', requireAuth, async (c) => {
 const rows = await db(c.env)
 .select({
 id: s.medals.id,
 name: s.medals.name,
 description: s.medals.description,
 imageUrl: s.medals.imageUrl,
 gameId: s.medals.gameId,
 autoGrantMonths: s.medals.autoGrantMonths,
 sortOrder: s.medals.sortOrder,
 awardCount: sql<number>`(select count(*) from member_medals where member_medals.medal_id =
medals.id)`,
 })
 .from(s.medals)
 .orderBy(asc(s.medals.sortOrder), asc(s.medals.name));

 return c.json({ medals: rows });
});

medals.post('/', requirePermission('medals.manage'), async (c) => {
 const body = await c.req.json<{
 name: string;
 description?: string;
 imageUrl?: string;
 autoGrantMonths?: number | null;
 >();

```

**src/server/routes/medals.ts**

```

 }>();

 if (!body.name?.trim()) return c.json({ error: 'Name is required' }, 400);

 let autoGrantMonths: number | null;
 try {
 autoGrantMonths = parseAutoGrantMonths(body.autoGrantMonths) ?? null;
 } catch (err) {
 return c.json({ error: (err as Error).message }, 400);
 }

 const database = db(c.env);
 const highest = await database
 .select({ max: sql<number | null>`max(${s.medals.sortOrder})` })
 .from(s.medals);

 const created = (
 await database
 .insert(s.medals)
 .values({
 name: body.name.trim(),
 description: body.description?.trim() || null,
 imageUrl: body.imageUrl?.trim() || null,
 autoGrantMonths,
 sortOrder: (highest[0]?.max ?? -1) + 1,
 })
 .returning()
)[0]!;

 await database.insert(s.auditLog).values({
 actorId: c.get('viewer')!.id,
 action: 'medal.create',
 targetType: 'medal',
 targetId: String(created.id),
 meta: { name: created.name, autoGrantMonths },
 });

 return c.json({ medal: { ...created, awardCount: 0 } }, 201);
 });

medals.patch('/:id', requirePermission('medals.manage'), async (c) => {
 const id = Number(c.req.param('id'));
 const body = await c.req.json<{
 name?: string;
 description?: string | null;
 imageUrl?: string | null;
 autoGrantMonths?: number | null;
 }>();

 const database = db(c.env);
 const medal = await database.query.medals.findFirst({ where: eq(s.medals.id, id) });
 if (!medal) return c.json({ error: 'No such medal' }, 404);

 const patch: Partial<typeof s.medals.$inferInsert> = {};

```

**src/server/routes/medals.ts**

```

 if (body.name !== null) {
 if (!body.name.trim()) return c.json({ error: 'Name cannot be empty' }, 400);
 patch.name = body.name.trim();
 }
 if (body.description !== undefined) patch.description = body.description?.trim() || null;
 if (body.imageUrl !== undefined) {
 patch.imageUrl = body.imageUrl?.trim() || null;
 // Free the object the image is replacing, once the update has committed.
 if (patch.imageUrl !== medal.imageUrl) {
 c.executionCtx.waitUntil(deleteMediaByUrl(c.env, medal.imageUrl));
 }
 }
 if (body.autoGrantMonths !== undefined) {
 try {
 patch.autoGrantMonths = parseAutoGrantMonths(body.autoGrantMonths) ?? null;
 } catch (err) {
 return c.json({ error: (err as Error).message }, 400);
 }
 }
 }

 const updated = (await database.update(s.medals).set(patch).where(eq(s.medals.id, id)).returning())[0]!;

 await database.insert(s.auditLog).values({
 actorId: c.get('viewer')!.id,
 action: 'medal.update',
 targetType: 'medal',
 targetId: String(id),
 meta: patch,
 });

 const awardCount = (
 await database
 .select({ n: sql<number>`count(*)` })
 .from(s.memberMedals)
 .where(eq(s.memberMedals.medalId, id))
)[0]!.n;

 return c.json({ medal: { ...updated, awardCount } });
});

/** Delete a medal definition. Every award of it is cascaded away (see schema). */
medals.delete('/:id', requirePermission('medals.manage'), async (c) => {
 const id = Number(c.req.param('id'));
 const database = db(c.env);

 const medal = await database.query.medals.findFirst({ where: eq(s.medals.id, id) });
 if (!medal) return c.json({ error: 'No such medal' }, 404);

 await database.delete(s.medals).where(eq(s.medals.id, id));
 c.executionCtx.waitUntil(deleteMediaByUrl(c.env, medal.imageUrl));

 await database.insert(s.auditLog).values({
 actorId: c.get('viewer')!.id,

```

**src/server/routes/medals.ts**

```
 action: 'medal.delete',
 targetType: 'medal',
 targetId: String(id),
 meta: { name: medal.name },
 });

 return c.json({ ok: true });
});

export default medals;
```

**src/server/routes/media.ts**

```

/**
 * Image uploads, backed by R2.
 *
 * Uploads come in through POST /api/media/<category> (each category is
 * permission-gated) and are served back out at /media/<key> ? a path
 * deliberately outside /api so the no-store middleware doesn't apply and the
 * browser/edge can cache them hard. See the run_worker_first note in
 * wrangler.jsonc for why /media/* must reach the Worker rather than the assets
 * layer.
 */

import { Hono } from 'hono';
import type { AppContext, Env } from '../env';
import { requireAuth } from '../middleware/auth';
import { can, type Permission } from '../../shared/permissions';

// Raster only. An uploaded SVG served from our own origin can execute script
// when opened directly (it's an XSS vector); PNG/JPEG/GIF/WebP shown in an
// cannot. The map also fixes the file extension we store under.
const ALLOWED: Record<string, string> = {
 'image/png': 'png',
 'image/jpeg': 'jpg',
 'image/gif': 'gif',
 'image/webp': 'webp',
};

const MAX_BYTES = 1_000_000; // 1 MB ? these are small insignia/avatars, not photos.

// What can be uploaded, where it's stored, and who may upload it. An avatar is
// self-service (any signed-in member can set their own), so it has no extra
// permission beyond being authenticated; medal and rank art are admin-only.
const CATEGORIES: Record<string, { prefix: string; permission?: Permission }> = {
 medals: { prefix: 'medals', permission: 'medals.manage' },
 ranks: { prefix: 'ranks', permission: 'ranks.manage' },
 avatars: { prefix: 'avatars' },
 games: { prefix: 'games', permission: 'games.manage' },
 warrecords: { prefix: 'warrecords', permission: 'warrecords.manage' },
 events: { prefix: 'events', permission: 'events.manage' },
 pages: { prefix: 'pages', permission: 'pages.manage' },
 branding: { prefix: 'branding', permission: 'settings.manage' },
};

const media = new Hono<AppContext>();

/**
 * Store an uploaded image and return its stable URL. Multipart form with a
 * single `file` field; the `category` path segment selects the storage prefix
 * and the permission required to write it.
 */
media.post('/:category', requireAuth, async (c) => {
 const conf = CATEGORIES[c.req.param('category')];
 if (!conf) return c.json({ error: 'Unknown upload category.' }, 404);

 const viewer = c.get('viewer')!;

```



**src/server/routes/media.ts**

```

 if (conf.permission && !can(viewer, conf.permission)) {
 return c.json({ error: 'Forbidden', missing: conf.permission }, 403);
 }

 if (!c.env.MEDIA) {
 return c.json({ error: 'Image storage (R2) is not configured on this deployment.' }, 503);
 }

 const form = await c.req.formData();
 const file = form.get('file');
 if (!(file instanceof File)) {
 return c.json({ error: 'No file provided.' }, 400);
 }

 const ext = ALLOWED[file.type];
 if (!ext) {
 return c.json({ error: 'Unsupported image type. Use PNG, JPEG, GIF, or WebP.' }, 415);
 }
 if (file.size > MAX_BYTES) {
 return c.json({ error: 'Image is too large (max 1 MB).' }, 413);
 }

 const key = `${conf.prefix}/${crypto.randomUUID()}.${ext}`;
 await c.env.MEDIA.put(key, await file.arrayBuffer(), {
 httpMetadata: { contentType: file.type, cacheControl: 'public, max-age=31536000, immutable' },
 });

 return c.json({ url: `/media/${key}`, key }, 201);
 });
}

export default media;

/**
 * Delete the R2 object a /media/... URL points at. Used to clean up the previous
 * image when one is replaced or its owner is deleted, so orphaned objects don't
 * accumulate against the storage quota. A no-op for anything that isn't one of
 * our own /media/ URLs (e.g. a null value, or a Discord avatar URL). Best
 * effort ? a failed delete is logged, never fatal to the request.
 */
export async function deleteMediaByUrl(env: Env, url: string | null | undefined): Promise<void> {
 if (!env.MEDIA || !url) return;
 const prefix = '/media/';
 if (!url.startsWith(prefix)) return;
 const key = url.slice(prefix.length);
 if (!key) return;
 try {
 await env.MEDIA.delete(key);
 } catch (err) {
 console.error('Failed to delete media object', key, err);
 }
}

/**
 * Serve one R2 object for the /media/* route (registered in index.ts, outside

```

**src/server/routes/media.ts**

```

* the /api middleware). Keys are content-addressed and never reused, so the
* response is immutable and cached for a year.
*
* The Cache API is checked first and populated on a miss, so a hot image is
* answered from the colo's edge cache without a round-trip to R2 ? which is
* what keeps R2 Class B (read) operations flat as traffic grows. Returns null
* when the object is absent so the caller can 404.
*/
export async function serveMediaObject(
 env: Env,
 request: Request,
 ctx: { waitUntil(promise: Promise<unknown>): void },
): Promise<Response | null> {
 if (!env.MEDIA) return null;

 const key = new URL(request.url).pathname.slice('/media/'.length);
 if (!key) return null;

 // caches.default is a Workers global; the DOM lib's CacheStorage type (pulled
 // in for React) doesn't declare it, so reach it through a narrow cast.
 const cache = (caches as unknown as { default: Cache }).default;
 const hit = await cache.match(request);
 if (hit) return hit;

 const object = await env.MEDIA.get(key);
 if (!object) return null;

 const headers = new Headers();
 object.writeHttpMetadata(headers);
 headers.set('etag', object.httpEtag);
 // writeHttpMetadata carries the stored cache-control, but set a floor in case
 // an older object was stored without one.
 if (!headers.has('cache-control')) {
 headers.set('cache-control', 'public, max-age=31536000, immutable');
 }
 headers.set('x-content-type-options', 'nosniff');

 const response = new Response(object.body, { headers });
 // Populate the edge cache without holding up the response.
 ctx.waitUntil(cache.put(request, response.clone()));
 return response;
}

```

**src/server/routes/members.ts**

```

import { Hono } from 'hono';
import { and, asc, desc, eq, sql } from 'drizzle-orm';
import * as s from '../db/schema';
import type { AppContext } from '../env';
import { db, requireAuth, requirePermission } from '../middleware/auth';
import { can, outranks } from '../shared/permissions';
import { memberName } from '../shared/names';
import { discordAvatar } from '../shared/avatar';
import { DiscordError } from '../discord/rest';
import { discordClient } from '../config';
import { grantRole, revokeRole, syncMemberRankRoles, reconcileMember } from '../discord/sync';
import { announce } from '../discord/announce';
import { deleteMediaByUrl } from '../media';

const members = new Hono<AppContext>();

/** Bot client from resolved (DB-over-env) config, or null when unconfigured. */
const rest = (env: AppContext['Bindings']) => discordClient(env);

const MIN_REASON = 3;
const MAX_REASON = 500;
/**
 * Negative actions (demote, remove/ban, revoke an award or role) must carry a
 * written justification. Returns the trimmed reason, or null when it is missing
 * or too short ? the caller turns null into a 400.
 */
function cleanReason(raw: unknown): string | null {
 if (typeof raw !== 'string') return null;
 const r = raw.trim();
 if (r.length < MIN_REASON) return null;
 return r.slice(0, MAX_REASON);
}

/** DELETE requests may arrive with no body; parse defensively. */
async function jsonBody<T>(c: { req: { json: () => Promise<unknown> } }): Promise<Partial<T>> {
 return (await c.req.json().catch(() => ({}))) as Partial<T>;
}

members.get('/', requirePermission('roster.view'), async (c) => {
 // Paginated so a page load reads one page, not the whole roster ? the read
 // cost stays flat as the clan grows. The client appends pages ("load more").
 const limit = Math.min(Math.max(Number(c.req.query('limit')) || 50, 1), 100);
 const offset = Math.max(Number(c.req.query('offset')) || 0, 0);
 const database = db(c.env);

 // Sortable by name or rank. Name sorts on the resolved display name
 // (display -> global -> username), matching what the UI shows.
 const sort = c.req.query('sort') === 'name' ? 'name' : 'rank';
 const dir = c.req.query('dir') === 'asc' ? 'asc' : c.req.query('dir') === 'desc' ? 'desc' : sort
 === 'rank' ? 'desc' : 'asc';
 const nameExpr = sql`coalesce(${s.users.displayName}, ${s.users.globalName},
 ${s.users.username})`;
 const nameOrder = dir === 'asc' ? asc(nameExpr) : desc(nameExpr);
 const orderBy =

```

**src/server/routes/members.ts**

```

 sort === 'name'
 ? [nameOrder]
 : [dir === 'asc' ? asc(s.ranks.sortOrder) : desc(s.ranks.sortOrder), asc(nameExpr)];

const rows = await database
 .select({
 id: s.users.id,
 discordId: s.users.discordId,
 username: s.users.username,
 globalName: s.users.globalName,
 displayName: s.users.displayName,
 avatar: s.users.avatar,
 profileImageUrl: s.users.profileImageUrl,
 status: s.users.status,
 joinedAt: s.users.joinedAt,
 rankId: s.ranks.id,
 rankName: s.ranks.name,
 rankOrder: s.ranks.sortOrder,
 })
 .from(s.users)
 .leftJoin(s.ranks, eq(s.users.rankId, s.ranks.id))
 .orderBy(...orderBy)
 .limit(limit)
 .offset(offset);

const totalRow = await database.select({ n: sql<number>`count(*)` }).from(s.users);
const total = Number(totalRow[0]?.n ?? 0);

return c.json({ members: rows, total, limit, offset });
});

members.get('/:id', requireAuth, async (c) => {
 const id = Number(c.req.param('id'));
 const viewer = c.get('viewer')!;
 // The roster can be restricted to certain roles (roster.view). A member can
 // always see their own profile; viewing anyone else needs roster.view.
 if (id !== viewer.id && !can(viewer, 'roster.view')) {
 return c.json({ error: 'Forbidden', missing: 'roster.view' }, 403);
 }
 const database = db(c.env);

 const user = await database.query.users.findFirst({ where: eq(s.users.id, id) });
 if (!user) return c.json({ error: 'No such member' }, 404);

 const [rank, roles, medals, warRecords, profile] = await Promise.all([
 user.rankId ? database.query.ranks.findFirst({ where: eq(s.ranks.id, user.rankId) }) : null,
 database
 .select({
 id: s.roles.id,
 name: s.roles.name,
 color: s.roles.color,
 source: s.userRoles.source,
 })
 .from(s.userRoles)

```

**src/server/routes/members.ts**

```

 .innerJoin(s.roles, eq(s.userRoles.roleId, s.roles.id))
 .where(eq(s.userRoles.userId, id)),
 database
 .select({
 awardId: s.memberMedals.id,
 id: s.medals.id,
 name: s.medals.name,
 imageUrl: s.medals.imageUrl,
 citation: s.memberMedals.citation,
 // NULL awardedBy = handed out by the tenure sweep, not a person.
 awardedBy: s.memberMedals.awardedBy,
 awardedAt: s.memberMedals.awardedAt,
 })
 .from(s.memberMedals)
 .innerJoin(s.medals, eq(s.memberMedals.medalId, s.medals.id))
 .where(eq(s.memberMedals.userId, id))
 .orderBy(desc(s.memberMedals.awardedAt)),
 database
 .select({
 awardId: s.memberWarRecords.id,
 id: s.warRecords.id,
 name: s.warRecords.name,
 imageUrl: s.warRecords.imageUrl,
 gameName: s.games.name,
 citation: s.memberWarRecords.citation,
 awardedBy: s.memberWarRecords.awardedBy,
 awardedAt: s.memberWarRecords.awardedAt,
 })
 .from(s.memberWarRecords)
 .innerJoin(s.warRecords, eq(s.memberWarRecords.warRecordId, s.warRecords.id))
 .leftJoin(s.games, eq(s.warRecords.gameId, s.games.id))
 .where(eq(s.memberWarRecords.userId, id))
 .orderBy(desc(s.memberWarRecords.awardedAt)),
 database.query.profiles.findFirst({ where: eq(s.profiles.userId, id) })),
]);

return c.json({ member: { ...user, rank, roles, medals, warRecords, bio: profile?.bio ?? null }
});
});

/**
 * Edit a member's profile ? display name, bio, and profile image. A member may
 * always edit their own; editing anyone else's needs roster.edit.
 *
 * Changing the display name also sets the member's Discord nickname (their
 * per-server name). Website stays the source of truth; the push is best-effort
 * and reported back, never blocking the save.
 *
 * profileImageUrl is a /media/avatars/... URL from a prior upload, or null to
 * clear it and revert to the Discord avatar. Replacing or clearing it deletes
 * the previous R2 object so orphans don't pile up.
 */
members.patch('/:id/profile', requireAuth, async (c) => {
 const id = Number(c.req.param('id'));

```

**src/server/routes/members.ts**

```

const viewer = c.get('viewer')!;
const isSelf = viewer.id === id;
if (!isSelf && !can(viewer, 'roster.edit')) {
 return c.json({ error: 'You can only edit your own profile.' }, 403);
}

const body = await c.req.json<{
 bio?: string;
 displayName?: string;
 profileImageUrl?: string | null;
}>();
const nowSec = Math.floor(Date.now() / 1000);
const database = db(c.env);

const target = await database.query.users.findFirst({ where: eq(s.users.id, id) });
if (!target) return c.json({ error: 'No such member' }, 404);

let profileImageUrl = target.profileImageUrl;
if (body.profileImageUrl !== undefined) {
 const next = body.profileImageUrl?.trim() || null;
 // Only accept a cleared value or one of our own uploaded avatar URLs ? never
 // an arbitrary off-site URL, which would let this field point tags at
 // anything and bypass the raster-only upload guard.
 if (next !== null && !next.startsWith('/media/avatars/')) {
 return c.json({ error: 'Invalid profile image.' }, 400);
 }
 if (next !== target.profileImageUrl) {
 await database.update(s.users).set({ profileImageUrl: next, updatedAt: nowSec
 }).where(eq(s.users.id, id));
 // Drop the image it replaced (best effort, off the request path).
 c.executionCtx.waitUntil(deleteMediaByUrl(c.env, target.profileImageUrl));
 profileImageUrl = next;
 }
}

let bio: string | undefined;
if (body.bio !== undefined) {
 bio = (body.bio ?? '').slice(0, 2000);
 await database
 .insert(s.profiles)
 .values({ userId: id, bio, updatedAt: nowSec })
 .onConflictDoUpdate({ target: s.profiles.userId, set: { bio, updatedAt: nowSec } });
}

let displayName = target.displayName;
let discordSync: { synced: boolean; warning?: string } | undefined;

if (body.displayName !== undefined) {
 // Display name is a required profile field: it can't be blanked out. (It
 // still defaults to the Discord name for members who've never opened the
 // editor ? this only guards an explicit save.)
 const trimmed = body.displayName.trim();
 if (trimmed.length < 2) {
 return c.json({ error: 'A display name is required (at least 2 characters).' }, 400);
 }
}

```

**src/server/routes/members.ts**

```

 }
 displayName = trimmed.slice(0, 32);
 const changed = displayName !== target.displayName;
 await database
 .update(s.users)
 .set({ displayName, updatedAt: nowSec })
 .where(eq(s.users.id, id));

 if (changed) {
 // Push the new display name as the member's Discord nickname. An empty
 // value clears the nickname, reverting them to their Discord name.
 const client = await rest(c.env);
 if (client) {
 try {
 await client.setNickname(
 target.discordId,
 displayName ?? '',
 `ClanTek: display name set by ${viewer.username}`,
);
 discordSync = { synced: true };
 } catch (err) {
 discordSync = {
 synced: false,
 warning:
 err instanceof DiscordError && err.status === 403
 ? 'Saved here, but Discord refused the nickname change: the bot needs Manage
Nicknames and a role above this member. (Discord also blocks changing the server owner's
nickname.)'
 : `Saved here, but the Discord nickname change failed: ${err as Error}.message`,
 };
 }
 }
 }

 await database.insert(s.auditLog).values({
 actorId: viewer.id,
 action: 'member.display_name',
 targetType: 'user',
 targetId: String(id),
 meta: { displayName, discordSynced: discordSync?.synced ?? false },
 });
 }
}

return c.json({ ok: true, bio, displayName, profileImageUrl, discordSync });
});

/**
 * Change a member's status (active/inactive/loa/retired/banned). "Remove" from
 * the roster is a status change to retired, which keeps history. Banning needs
 * the stronger roster.remove; softer states need roster.edit. You cannot change
 * the status of someone who outranks you.
 */
members.patch('/:id/status', requireAuth, async (c) => {
 const id = Number(c.req.param('id'));

```

**src/server/routes/members.ts**

```

const viewer = c.get('viewer')!;
const { status, reason } = await c.req.json<{ status: string; reason?: string }>();

const allowed = ['active', 'inactive', 'loa', 'retired', 'banned'] as const;
if (!allowed.includes(status as (typeof allowed)[number])) {
 return c.json({ error: 'Invalid status' }, 400);
}

const needsRemove = status === 'banned' || status === 'retired';
if (!can(viewer, needsRemove ? 'roster.remove' : 'roster.edit')) {
 return c.json({ error: 'Forbidden' }, 403);
}

// Removing, retiring, or banning a member is a negative action ? it demands
// a written reason, which is recorded on the audit entry.
const cleanedReason = cleanReason(reason);
if (needsRemove && !cleanedReason) {
 return c.json({ error: 'A reason is required to retire or ban a member.' }, 400);
}

const database = db(c.env);
const target = await database.query.users.findFirst({ where: eq(s.users.id, id) });
if (!target) return c.json({ error: 'No such member' }, 404);

const targetRank = target.rankId
 ? await database.query.ranks.findFirst({ where: eq(s.ranks.id, target.rankId) })
 : null;
if (!outranks(viewer, targetRank?.sortOrder ?? null)) {
 return c.json({ error: 'You cannot change the status of someone at or above your rank.' },
403);
}

const nowSec = Math.floor(Date.now() / 1000);
await database
 .update(s.users)
 .set({ status: status as (typeof allowed)[number], updatedAt: nowSec })
 .where(eq(s.users.id, id));

await database.insert(s.auditLog).values({
 actorId: viewer.id,
 action: 'member.status',
 targetType: 'user',
 targetId: String(id),
 reason: cleanedReason,
 meta: { from: target.status, to: status },
 ip: c.req.header('cf-connecting-ip'),
});

return c.json({ ok: true, status });
});

/** Set a member's rank outright (the ladder-step version lives in Discord's /promote). */
members.put('/:id/rank', requirePermission('roster.promote'), async (c) => {
 const id = Number(c.req.param('id'));

```



**src/server/routes/members.ts**

```

const { rankId, reason } = await c.req.json<{ rankId: number | null; reason?: string }>();
const viewer = c.get('viewer')!;
const database = db(c.env);

const target = await database.query.users.findFirst({ where: eq(s.users.id, id) });
if (!target) return c.json({ error: 'No such member' }, 404);

const currentRank = target.rankId
 ? await database.query.ranks.findFirst({ where: eq(s.ranks.id, target.rankId) })
 : null;
if (!outRanks(viewer, currentRank?.sortOrder ?? null)) {
 return c.json({ error: 'You can only change ranks below your own' }, 403);
}

const newRank = rankId
 ? await database.query.ranks.findFirst({ where: eq(s.ranks.id, rankId) })
 : null;
if (rankId && !newRank) return c.json({ error: 'No such rank' }, 404);
if (newRank && !outRanks(viewer, newRank.sortOrder)) {
 return c.json({ error: 'You cannot assign a rank at or above your own' }, 403);
}

// A demotion (or un-ranking) is negative and needs a written reason; a
// promotion does not. Treat "no new rank while they had one" as a demotion.
const isDemotion = !!currentRank && (!newRank || newRank.sortOrder < currentRank.sortOrder);
const cleanedReason = cleanReason(reason);
if (isDemotion && !cleanedReason) {
 return c.json({ error: 'A reason is required to demote or un-rank a member.' }, 400);
}

const nowSec = Math.floor(Date.now() / 1000);
await database
 .update(s.users)
 .set({ rankId: newRank?.id ?? null, promotedAt: nowSec, updatedAt: nowSec })
 .where(eq(s.users.id, id));

await database.insert(s.auditLog).values({
 actorId: viewer.id,
 action: 'member.rank_change',
 targetType: 'user',
 targetId: String(id),
 reason: cleanedReason,
 meta: { from: currentRank?.name ?? null, to: newRank?.name ?? null, demotion: isDemotion },
 ip: c.req.header('cf-connecting-ip'),
});

// Apply the new rank's roles (and drop the old rank's), cascading to Discord.
const rankRoleSync = await syncMemberRankRoles(database, await rest(c.env), {
 userId: id,
 rankId: newRank?.id ?? null,
 actorId: viewer.id,
});

// Announce only a move UP the ladder; demotions and un-ranking stay quiet.

```

**src/server/routes/members.ts**

```

 if (newRank && newRank.sortOrder > (currentRank?.sortOrder ?? -1)) {
 c.executionCtx.waitUntil(
 announce(
 c.env,
 {
 type: 'promotion',
 memberName: memberName(target),
 memberDiscordId: target.discordId,
 memberAvatarUrl: discordAvatar(target.discordId, target.avatar, 128),
 rankName: newRank.name,
 rankImageUrl: newRank.imageUrl,
 byName: memberName(viewer),
 },
 new URL(c.req.url).origin,
),
);
 }
 }

 return c.json({ ok: true, rank: newRank ?? null, rankRoleSync });
});

/**
 * Grant a role. If the role is mapped to a Discord role, this also updates
 * Discord ? which is what makes website roles gate Discord channels.
 */
members.post('/:id/roles', requirePermission('roles.assign'), async (c) => {
 const userId = Number(c.req.param('id'));
 const { roleId, reason } = await c.req.json<{ roleId: number; reason?: string }>();
 const viewer = c.get('viewer')!;

 const result = await grantRole(db(c.env), await rest(c.env), {
 userId,
 roleId,
 actorId: viewer.id,
 reason: reason ?? `Granted by ${viewer.username} via ClanTek`,
 });

 return c.json({ ok: true, ...result });
});

/**
 * Force this member's Discord roles back into line with the website now, rather
 * than waiting for the scheduled sweep. Adds mapped roles they should have,
 * removes mapped ones they shouldn't; unmanaged Discord roles are left alone.
 */
members.post('/:id/resync', requirePermission('discord.sync'), async (c) => {
 const id = Number(c.req.param('id'));
 const client = await rest(c.env);
 if (!client) {
 return c.json({ ok: false, warning: 'Discord bot is not configured.' });
 }
 try {
 const r = await reconcileMember(db(c.env), client, id);
 return c.json({ ok: true, added: r.added.length, removed: r.removed.length });
 }

```

**src/server/routes/members.ts**

```

 } catch (err) {
 return c.json({ ok: false, warning: `Discord re-sync failed: ${err as Error}.message` });
 }
 });

/**
 * Award a medal to a member, with an optional citation for why. A member can
 * hold any given medal only once, so re-awarding the same one is a 409 rather
 * than a silent duplicate.
 */
members.post('/:id/medals', requirePermission('medals.award'), async (c) => {
 const userId = Number(c.req.param('id'));
 const { medalId, citation } = await c.req.json<{ medalId: number; citation?: string }>();
 const viewer = c.get('viewer')!;
 const database = db(c.env);

 const [target, medal] = await Promise.all([
 database.query.users.findFirst({ where: eq(s.users.id, userId) }),
 database.query.medals.findFirst({ where: eq(s.medals.id, medalId) }),
]);
 if (!target) return c.json({ error: 'No such member' }, 404);
 if (!medal) return c.json({ error: 'No such medal' }, 404);

 const already = await database
 .select({ id: s.memberMedals.id })
 .from(s.memberMedals)
 .where(and(eq(s.memberMedals.userId, userId), eq(s.memberMedals.medalId, medalId)));
 if (already.length) {
 return c.json({ error: `${target.username} already has the "${medal.name}" medal.` }, 409);
 }

 const award = (
 await database
 .insert(s.memberMedals)
 .values({
 userId,
 medalId,
 citation: citation?.trim() || null,
 awardedBy: viewer.id,
 })
 .returning()
)[0]!;

 await database.insert(s.auditLog).values({
 actorId: viewer.id,
 action: 'medal.award',
 targetType: 'user',
 targetId: String(userId),
 meta: { medalId, medalName: medal.name, citation: citation?.trim() || null },
 ip: c.req.header('cf-connecting-ip'),
 });

 c.executionCtx.waitUntil(
 announce(

```

**src/server/routes/members.ts**

```

 c.env,
 {
 type: 'medalAward',
 memberName: memberName(target),
 memberDiscordId: target.discordId,
 memberAvatarUrl: discordAvatar(target.discordId, target.avatar, 128),
 medalName: medal.name,
 medalImageUrl: medal.imageUrl,
 citation: citation?.trim() || null,
 },
 new URL(c.req.url).origin,
),
);

return c.json({ ok: true, awardId: award.id }, 201);
});

/** Revoke a specific award (by member_medals id, not medal id). */
members.delete('/:id/medals/:awardId', requirePermission('medals.award'), async (c) => {
 const userId = Number(c.req.param('id'));
 const awardId = Number(c.req.param('awardId'));
 const viewer = c.get('viewer')!;
 const database = db(c.env);

 const { reason } = await jsonBody<{ reason: string }>(c);
 const cleanedReason = cleanReason(reason);
 if (!cleanedReason) {
 return c.json({ error: 'A reason is required to revoke a medal.' }, 400);
 }

 const award = await database.query.memberMedals.findFirst({
 where: and(eq(s.memberMedals.id, awardId), eq(s.memberMedals.userId, userId)),
 });
 if (!award) return c.json({ error: 'No such award on this member' }, 404);

 await database.delete(s.memberMedals).where(eq(s.memberMedals.id, awardId));

 await database.insert(s.auditLog).values({
 actorId: viewer.id,
 action: 'medal.revoke',
 targetType: 'user',
 targetId: String(userId),
 reason: cleanedReason,
 meta: { medalId: award.medalId, awardId },
 ip: c.req.header('cf-connecting-ip'),
 });

 return c.json({ ok: true });
});

/**
 * Award a war record to a member, with an optional citation. Like medals, a
 * member holds any given war record only once (re-awarding is a 409).
 */

```

**src/server/routes/members.ts**

```

members.post('/:id/warrecords', requirePermission('warrecords.award'), async (c) => {
 const userId = Number(c.req.param('id'));
 const { warRecordId, citation } = await c.req.json<{ warRecordId: number; citation?: string }>();
 const viewer = c.get('viewer')!;
 const database = db(c.env);

 const [target, record] = await Promise.all([
 database.query.users.findFirst({ where: eq(s.users.id, userId) }),
 database.query.warRecords.findFirst({ where: eq(s.warRecords.id, warRecordId) }),
]);
 if (!target) return c.json({ error: 'No such member' }, 404);
 if (!record) return c.json({ error: 'No such war record' }, 404);

 const already = await database
 .select({ id: s.memberWarRecords.id })
 .from(s.memberWarRecords)
 .where(and(eq(s.memberWarRecords.userId, userId), eq(s.memberWarRecords.warRecordId, warRecordId)));
 if (already.length) {
 return c.json({ error: `${target.username} already holds the "${record.name}" war record.` }, 409);
 }

 const award = (
 await database
 .insert(s.memberWarRecords)
 .values({ userId, warRecordId, citation: citation?.trim() || null, awardedBy: viewer.id })
 .returning()
)[0]!;

 await database.insert(s.auditLog).values({
 actorId: viewer.id,
 action: 'warrecord.award',
 targetType: 'user',
 targetId: String(userId),
 meta: { warRecordId, name: record.name, citation: citation?.trim() || null },
 ip: c.req.header('cf-connecting-ip'),
 });

 const game = record.gameId
 ? await database.query.games.findFirst({ where: eq(s.games.id, record.gameId) })
 : null;
 c.executionCtx.waitUntil(
 announce(
 c.env,
 {
 type: 'warRecordAward',
 memberName: memberName(target),
 memberDiscordId: target.discordId,
 memberAvatarUrl: discordAvatar(target.discordId, target.avatar, 128),
 recordName: record.name,
 recordImageUrl: record.imageUrl,
 gameName: game?.name ?? null,
 }
)
);
}

```

**src/server/routes/members.ts**

```
 citation: citation?.trim() || null,
 },
 new URL(c.req.url).origin,
),
);

 return c.json({ ok: true, awardId: award.id }, 201);
});

/** Revoke a specific war-record award (by member_war_records id). */
members.delete('/:id/warrecords/:awardId', requirePermission('warrecords.award'), async (c) => {
 const userId = Number(c.req.param('id'));
 const awardId = Number(c.req.param('awardId'));
 const viewer = c.get('viewer')!;
 const database = db(c.env);

 const { reason } = await jsonBody<{ reason: string }>(c);
 const cleanedReason = cleanReason(reason);
 if (!cleanedReason) {
 return c.json({ error: 'A reason is required to revoke a war record.' }, 400);
 }

 const award = await database.query.memberWarRecords.findFirst({
 where: and(eq(s.memberWarRecords.id, awardId), eq(s.memberWarRecords.userId, userId)),
 });
 if (!award) return c.json({ error: 'No such award on this member' }, 404);

 await database.delete(s.memberWarRecords).where(eq(s.memberWarRecords.id, awardId));

 await database.insert(s.auditLog).values({
 actorId: viewer.id,
 action: 'warrecord.revoke',
 targetType: 'user',
 targetId: String(userId),
 reason: cleanedReason,
 meta: { warRecordId: award.warRecordId, awardId },
 ip: c.req.header('cf-connecting-ip'),
 });

 return c.json({ ok: true });
});

members.delete('/:id/roles/:roleId', requirePermission('roles.assign'), async (c) => {
 const userId = Number(c.req.param('id'));
 const roleId = Number(c.req.param('roleId'));
 const viewer = c.get('viewer')!;

 const { reason } = await jsonBody<{ reason: string }>(c);
 const cleanedReason = cleanReason(reason);
 if (!cleanedReason) {
 return c.json({ error: 'A reason is required to revoke a role.' }, 400);
 }

 const result = await revokeRole(db(c.env), await rest(c.env), {
```

**src/server/routes/members.ts**

```
 userId,
 roleId,
 actorId: viewer.id,
 reason: cleanedReason,
 });

 return c.json({ ok: true, ...result });
});

export default members;
```

**src/server/routes/nav.ts**

```

/**
 * Site navigation ? the composable top menu (see shared/nav.ts for the model).
 *
 * Stored as one JSON blob in settings['nav']. GET is available to any signed-in
 * member (the bar renders for them); the returned tree is always sanitised, and
 * page links whose page has since been deleted are pruned. When nothing is
 * stored yet, the classic menu is synthesised from the built-ins + the pages
 * that had the old "show in nav" flag, so upgrading loses nothing. PUT is gated
 * on `pages.manage` ? whoever arranges pages arranges the menu that reaches them.
 */

import { Hono } from 'hono';
import { eq, asc } from 'drizzle-orm';
import * as s from '../db/schema';
import type { AppContext } from '../env';
import { db, requireAuth, requirePermission } from '../middleware/auth';
import { sanitizeNav, defaultNavConfig, BUILTIN_TARGETS, type NavConfig } from '../shared/nav';
import { HOME_SLUG } from '../shared/layout';

const nav = new Hono<AppContext>();

const NAV_KEY = 'nav';

/** Every custom page's slug ? for pruning page links to pages that still exist. */
async function pageSlugs(database: ReturnType<typeof db>): Promise<Set<string>> {
 const rows = await database.select({ slug: s.pageLayouts.slug }).from(s.pageLayouts);
 return new Set(rows.map((r) => r.slug).filter((slug) => slug !== HOME_SLUG));
}

/** Resolve the effective nav: the stored+sanitised tree, or the classic default. */
async function loadNav(database: ReturnType<typeof db>): Promise<NavConfig> {
 const slugs = await pageSlugs(database);
 const row = await database.query.settings.findFirst({ where: eq(s.settings.key, NAV_KEY) });
 if (row?.value && typeof row.value === 'object') {
 return sanitizeNav(row.value, { validSlugs: slugs });
 }
 // Nothing stored -> reproduce the old menu from the pages that opted in.
 const navPages = await database
 .select({ slug: s.pageLayouts.slug, title: s.pageLayouts.title })
 .from(s.pageLayouts)
 .where(eq(s.pageLayouts.showInNav, true))
 .orderBy(asc(s.pageLayouts.navOrder), asc(s.pageLayouts.slug));
 return defaultNavConfig(navPages.filter((p) => p.slug !== HOME_SLUG));
}

/** The rendered menu, for the top bar. */
nav.get('/', requireAuth, async (c) => {
 return c.json({ nav: await loadNav(db(c.env)) });
});

/** Everything the builder needs: the current tree + the pages, roles and built-ins to compose from. */
nav.get('/meta', requirePermission('pages.manage'), async (c) => {
 const database = db(c.env);

```



**src/server/routes/nav.ts**

```
const [navConfig, pages, roles] = await Promise.all([
 loadNav(database),
 database
 .select({ slug: s.pageLayouts.slug, title: s.pageLayouts.title })
 .from(s.pageLayouts)
 .orderBy(asc(s.pageLayouts.title)),
 database
 .select({ id: s.roles.id, name: s.roles.name, color: s.roles.color })
 .from(s.roles)
 .orderBy(asc(s.roles.sortOrder), asc(s.roles.name)),
]);

return c.json({
 nav: navConfig,
 pages: pages.filter((p) => p.slug !== HOME_SLUG),
 roles,
 builtins: Object.values(BUILTIN_TARGETS).map((b) => ({ key: b.key, label: b.label })),
});

/** Save the arranged menu. */
nav.put('/', requirePermission('pages.manage'), async (c) => {
 const body = await c.req.json<{ nav?: unknown }>();
 const clean = sanitizeNav(body.nav, { validSlugs: await pageSlugs(db(c.env)) });

 const viewer = c.get('viewer')!;
 await db(c.env)
 .insert(s.settings)
 .values({ key: NAV_KEY, value: clean, updatedBy: viewer.id })
 .onConflictDoUpdate({
 target: s.settings.key,
 set: { value: clean, updatedBy: viewer.id, updatedAt: Math.floor(Date.now() / 1000) },
 });

 await db(c.env).insert(s.auditLog).values({
 actorId: viewer.id,
 action: 'nav.update',
 targetType: 'settings',
 targetId: NAV_KEY,
 meta: { items: clean.items.length },
 });

 return c.json({ ok: true, nav: clean });
});

export default nav;
```

**src/server/routes/news.ts**

```

/**
 * News / announcements.
 *
 * Posts move draft -> published -> archived. Writing a post needs news.create,
 * flipping its published state needs news.publish, and removing it needs
 * news.delete ? so a trusted writer can draft without being able to publish.
 *
 * The body is HTML from the WYSIWYG editor; it's sanitized client-side on save
 * and again on render (see client/lib/richtext.ts), so nothing here trusts it.
 */

import { Hono } from 'hono';
import { and, desc, eq, ne } from 'drizzle-orm';
import * as s from '../../db/schema';
import type { AppContext } from '../../env';
import { db, requireAuth, requirePermission } from '../../middleware/auth';
import { can } from '../../shared/permissions';
import { memberName } from '../../shared/names';

const news = new Hono<AppContext>();

const now = () => Math.floor(Date.now() / 1000);

function slugify(title: string): string {
 return (
 title
 .toLowerCase()
 .trim()
 .replace(/[^\a-z0-9]+/g, '-')
 .replace(/^-+|-+$/g, '')
 .slice(0, 80) || 'post'
);
}

/** A slug not already taken; disambiguates with -2, -3, ... Optionally ignores one id (self on
rename). */
async function uniqueSlug(
 database: ReturnType<typeof db>,
 base: string,
 ignoreId?: number,
): Promise<string> {
 let slug = base;
 let n = 1;
 // eslint-disable-next-line no-constant-condition
 while (true) {
 const clash = await database.query.news.findFirst({
 where: ignoreId ? and(eq(s.news.slug, slug), ne(s.news.id, ignoreId)) : eq(s.news.slug,
slug),
 });
 if (!clash) return slug;
 n += 1;
 slug = `${base}-${n}`;
 }
}

```

**src/server/routes/news.ts**

```
const authorFields = {
 authorId: s.users.id,
 authorName: s.users.displayName,
 authorGlobalName: s.users.globalName,
 authorUsername: s.users.username,
};

function withAuthor<T extends { authorName: string | null; authorGlobalName: string | null;
authorUsername: string | null }>(
 row: T,
) {
 const { authorName, authorGlobalName, authorUsername, ...rest } = row;
 return {
 ...rest,
 author: authorUsername
 ? memberName({ displayName: authorName, globalName: authorGlobalName, username:
authorUsername })
 : null,
 };
}

/** The public feed: published posts only, pinned first, newest next. */
news.get('/', requireAuth, async (c) => {
 const rows = await db(c.env)
 .select({
 id: s.news.id,
 slug: s.news.slug,
 title: s.news.title,
 excerpt: s.news.excerpt,
 body: s.news.body,
 pinned: s.news.pinned,
 publishedAt: s.news.publishedAt,
 ...authorFields,
 })
 .from(s.news)
 .leftJoin(s.users, eq(s.news.authorId, s.users.id))
 .where(eq(s.news.status, 'published'))
 .orderBy(desc(s.news.pinned), desc(s.news.publishedAt));

 return c.json({ posts: rows.map(withAuthor) });
});

/** Every post regardless of status, for the admin list. */
news.get('/manage', requirePermission('news.create'), async (c) => {
 const rows = await db(c.env)
 .select({
 id: s.news.id,
 slug: s.news.slug,
 title: s.news.title,
 status: s.news.status,
 pinned: s.news.pinned,
 publishedAt: s.news.publishedAt,
 updatedAt: s.news.updatedAt,
```

**src/server/routes/news.ts**

```

 ...authorFields,
 })
 .from(s.news)
 .leftJoin(s.users, eq(s.news.authorId, s.users.id))
 .orderBy(desc(s.news.pinned), desc(s.news.updatedAt));

 return c.json({ posts: rows.map(withAuthor) });
});

/** A single post by slug. Non-published posts are only visible to writers. */
news.get('/:slug', requireAuth, async (c) => {
 const slug = c.req.param('slug');
 const rows = await db(c.env)
 .select({
 id: s.news.id,
 slug: s.news.slug,
 title: s.news.title,
 excerpt: s.news.excerpt,
 body: s.news.body,
 status: s.news.status,
 pinned: s.news.pinned,
 publishedAt: s.news.publishedAt,
 createdAt: s.news.createdAt,
 updatedAt: s.news.updatedAt,
 ...authorFields,
 })
 .from(s.news)
 .leftJoin(s.users, eq(s.news.authorId, s.users.id))
 .where(eq(s.news.slug, slug))
 .limit(1);

 const post = rows[0];
 if (!post) return c.json({ error: 'No such post' }, 404);
 if (post.status !== 'published' && !can(c.get('viewer'), 'news.create')) {
 return c.json({ error: 'No such post' }, 404);
 }

 return c.json({ post: withAuthor(post) });
});

news.post('/', requirePermission('news.create'), async (c) => {
 const body = await c.req.json<{ title?: string; excerpt?: string; body?: string; pinned?:
boolean }>();
 if (!body.title?.trim()) return c.json({ error: 'A title is required' }, 400);

 const database = db(c.env);
 const viewer = c.get('viewer')!;
 const nowSec = now();
 const slug = await uniqueSlug(database, slugify(body.title));

 const created = (
 await database
 .insert(s.news)
 .values({

```

**src/server/routes/news.ts**

```

 title: body.title.trim().slice(0, 200),
 slug,
 excerpt: body.excerpt?.trim().slice(0, 500) || null,
 body: body.body ?? '',
 authorId: viewer.id,
 status: 'draft',
 pinned: body.pinned ?? false,
 createdAt: nowSec,
 updatedAt: nowSec,
 })
 .returning()
)[0]!;

await database.insert(s.auditLog).values({
 actorId: viewer.id,
 action: 'news.create',
 targetType: 'news',
 targetId: String(created.id),
 meta: { title: created.title },
});

return c.json({ post: created }, 201);
});

/** Edit content. Publishing state is changed via /:id/status, not here. */
news.patch('/:id', requirePermission('news.create'), async (c) => {
 const id = Number(c.req.param('id'));
 const body = await c.req.json<{
 title?: string;
 excerpt?: string | null;
 body?: string;
 pinned?: boolean;
 }>();

 const database = db(c.env);
 const post = await database.query.news.findFirst({ where: eq(s.news.id, id) });
 if (!post) return c.json({ error: 'No such post' }, 404);

 const patch: Partial<typeof s.news.$inferInsert> = { updatedAt: now() };
 if (body.title !== undefined) {
 if (!body.title.trim()) return c.json({ error: 'Title cannot be empty' }, 400);
 patch.title = body.title.trim().slice(0, 200);
 }
 if (body.excerpt !== undefined) patch.excerpt = body.excerpt?.trim().slice(0, 500) || null;
 if (body.body !== undefined) patch.body = body.body;
 if (body.pinned !== undefined) patch.pinned = body.pinned;

 const updated = (await database.update(s.news).set(patch).where(eq(s.news.id,
id)).returning())[0]!;
 return c.json({ post: updated });
});

/** Move a post between draft / published / archived. First publish stamps publishedAt. */
news.post('/:id/status', requirePermission('news.publish'), async (c) => {

```

**src/server/routes/news.ts**

```
const id = Number(c.req.param('id'));
const { status } = await c.req.json<{ status: string }>();
const allowed = ['draft', 'published', 'archived'] as const;
if (!allowed.includes(status as (typeof allowed)[number])) {
 return c.json({ error: 'Invalid status' }, 400);
}

const database = db(c.env);
const post = await database.query.news.findFirst({ where: eq(s.news.id, id) });
if (!post) return c.json({ error: 'No such post' }, 404);

const patch: Partial<typeof s.news.$inferInsert> = {
 status: status as (typeof allowed)[number],
 updatedAt: now(),
};
// Stamp the publish date once, on first publish, so re-publishing an archived
// post doesn't reset its original date.
if (status === 'published' && post.publishedAt == null) patch.publishedAt = now();

const updated = (await database.update(s.news).set(patch).where(eq(s.news.id,
id)).returning())[0]!;

await database.insert(s.auditLog).values({
 actorId: c.get('viewer')!.id,
 action: 'news.status',
 targetType: 'news',
 targetId: String(id),
 meta: { from: post.status, to: status },
});

return c.json({ post: updated });
});

news.delete('/:id', requirePermission('news.delete'), async (c) => {
 const id = Number(c.req.param('id'));
 const database = db(c.env);
 const post = await database.query.news.findFirst({ where: eq(s.news.id, id) });
 if (!post) return c.json({ error: 'No such post' }, 404);

 await database.delete(s.news).where(eq(s.news.id, id));
 await database.insert(s.auditLog).values({
 actorId: c.get('viewer')!.id,
 action: 'news.delete',
 targetType: 'news',
 targetId: String(id),
 meta: { title: post.title },
 });

 return c.json({ ok: true });
});

export default news;
```

**src/server/routes/orgchart.ts**

```

/**
 * The leadership org chart ? a drag-and-drop tree of who reports to whom, which
 * (Phase 2) becomes the public face of the roster.
 *
 * GET '/' is for the public tree: any signed-in member may see it, and it returns
 * ONLY the placed members who meet the rank threshold ? never the full directory,
 * so it can safely show to members who lack roster.view. GET '/designer' and PUT
 * are gated on ranks.manage (managing the rank structure). The chart itself is
 * portable JSON stored under the 'org_chart' settings key ? no migration.
 */

import { Hono } from 'hono';
import { asc, eq } from 'drizzle-orm';
import * as s from '../db/schema';
import type { AppContext } from '../env';
import { db, requireAuth, requirePermission } from '../middleware/auth';
import { sanitizeOrgChart, EMPTY_ORG_CHART, type OrgChart } from '../shared/orgchart';

const orgchart = new Hono<AppContext>();
const ORG_KEY = 'org_chart';

type DB = ReturnType<typeof db>;

async function loadChart(database: DB): Promise<OrgChart> {
 const row = await database.query.settings.findFirst({ where: eq(s.settings.key, ORG_KEY) });
 return sanitizeOrgChart(row?.value ?? EMPTY_ORG_CHART);
}

/** Non-banned members with their rank name + sort-order, for chart boxes. */
async function membersWithRank(database: DB) {
 const rows = await database
 .select({
 id: s.users.id,
 discordId: s.users.discordId,
 username: s.users.username,
 globalName: s.users.globalName,
 displayName: s.users.displayName,
 avatar: s.users.avatar,
 profileImageUrl: s.users.profileImageUrl,
 status: s.users.status,
 rankName: s.ranks.name,
 rankSortOrder: s.ranks.sortOrder,
 })
 .from(s.users)
 .leftJoin(s.ranks, eq(s.users.rankId, s.ranks.id));
 return rows.filter((r) => r.status !== 'banned');
}

function meetsThreshold(rankSortOrder: number | null, min: number): boolean {
 if (min <= 0) return true;
 return rankSortOrder !== null && rankSortOrder >= min;
}

/** Public tree: the placed members who currently meet the threshold, plus edges. */

```

**src/server/routes/orgchart.ts**

```

orgchart.get('/', requireAuth, async (c) => {
 const database = db(c.env);
 const chart = await loadChart(database);
 const placed = new Set(chart.nodes.map((n) => n.memberId));
 const members = (await membersWithRank(database)).filter(
 (m) => placed.has(m.id) && meetsThreshold(m.rankSortOrder, chart.minRankSortOrder),
);
 return c.json({ chart, members });
});

/** Designer data: the full directory to place from, plus ranks for the threshold picker. */
orgchart.get('/designer', requirePermission('ranks.manage'), async (c) => {
 const database = db(c.env);
 const [chart, members, ranks] = await Promise.all([
 loadChart(database),
 membersWithRank(database),
 database
 .select({ id: s.ranks.id, name: s.ranks.name, sortOrder: s.ranks.sortOrder })
 .from(s.ranks)
 .orderBy(asc(s.ranks.sortOrder)),
]);
 return c.json({ chart, members, ranks });
});

/** Save the chart. */
orgchart.put('/', requirePermission('ranks.manage'), async (c) => {
 const body = await c.req.json<{ chart?: unknown }>();
 const chart = sanitizeOrgChart(body.chart);
 const viewer = c.get('viewer')!;
 const now = Math.floor(Date.now() / 1000);

 await db(c.env)
 .insert(s.settings)
 .values({ key: ORG_KEY, value: chart, updatedBy: viewer.id, updatedAt: now })
 .onConflictDoUpdate({
 target: s.settings.key,
 set: { value: chart, updatedBy: viewer.id, updatedAt: now },
 });

 await db(c.env).insert(s.auditLog).values({
 actorId: viewer.id,
 action: 'orgchart.save',
 targetType: 'orgchart',
 targetId: ORG_KEY,
 meta: { nodes: chart.nodes.length, edges: chart.edges.length, minRankSortOrder:
chart.minRankSortOrder },
 });

 return c.json({ ok: true, chart });
});

export default orgchart;

```



**src/server/routes/pages.ts**

```

/**
 * Page layouts ? the drag-and-drop module arrangement for the home page and any
 * number of admin-created custom content pages (served at /p/:slug).
 *
 * 'home' is the one built-in page: it always exists (falling back to the
 * DEFAULT_LAYOUTS default), is navigated to at /, is never listed in the custom
 * nav, and can't be deleted ? only reset. Every other page is a row in
 * page_layouts an admin created.
 *
 * GET is public (pages render for everyone); mutations are gated on
 * `pages.manage`. Stored JSON is always structurally sanitized before it can
 * reach the renderer; rich-text html inside a module is sanitized client-side on
 * save and again on render, exactly like news bodies.
 */

import { Hono } from 'hono';
import { eq, asc } from 'drizzle-orm';
import * as s from '../db/schema';
import type { AppContext } from '../env';
import { db, requirePermission } from '../middleware/auth';
import {
 defaultLayout,
 sanitizeLayout,
 isValidPageSlug,
 slugify,
 HOME_SLUG,
} from '../shared/layout';

const pages = new Hono<AppContext>();

/** Admin list: the built-in home first, then every custom page. */
pages.get('/', requirePermission('pages.manage'), async (c) => {
 const rows = await db(c.env)
 .select({
 slug: s.pageLayouts.slug,
 title: s.pageLayouts.title,
 showInNav: s.pageLayouts.showInNav,
 navOrder: s.pageLayouts.navOrder,
 updatedAt: s.pageLayouts.updatedAt,
 })
 .from(s.pageLayouts)
 .orderBy(asc(s.pageLayouts.navOrder), asc(s.pageLayouts.slug));

 const home = rows.find((r) => r.slug === HOME_SLUG);
 const custom = rows.filter((r) => r.slug !== HOME_SLUG);
 const list = [
 { slug: HOME_SLUG, title: home?.title ?? 'Home page', showInNav: false, navOrder: 0, isHome:
true },
 ...custom.map((r) => ({ ...r, showInNav: !!r.showInNav, isHome: false })),
];
 return c.json({ pages: list });
});

/** Public: the custom pages that opt into the top nav, in order. */

```

**src/server/routes/pages.ts**

```

pages.get('/nav', async (c) => {
 const rows = await db(c.env)
 .select({ slug: s.pageLayouts.slug, title: s.pageLayouts.title, navOrder:
s.pageLayouts.navOrder })
 .from(s.pageLayouts)
 .where(eq(s.pageLayouts.showInNav, true))
 .orderBy(asc(s.pageLayouts.navOrder), asc(s.pageLayouts.slug));
 return c.json({ pages: rows.filter((r) => r.slug !== HOME_SLUG) });
});

/**
 * The role list for the editor's "Visible to" audience picker. Gated on
 * pages.manage (a page editor may hold neither roles.manage nor roles.assign,
 * so /roles and /roles/assignable aren't reachable to them). Non-sensitive ?
 * role names and colours already show on the public roster. Lives under a
 * two-segment path so it can never collide with a custom page's `/:slug`.
 */
pages.get('/meta/roles', requirePermission('pages.manage'), async (c) => {
 const list = await db(c.env)
 .select({ id: s.roles.id, name: s.roles.name, color: s.roles.color })
 .from(s.roles)
 .orderBy(asc(s.roles.sortOrder), asc(s.roles.name));
 return c.json({ roles: list });
});

/** Public: a page's layout. `exists` distinguishes a real page from a 404 target. */
pages.get('/:slug', async (c) => {
 const slug = c.req.param('slug');
 const row = await db(c.env).query.pageLayouts.findFirst({ where: eq(s.pageLayouts.slug, slug)
});
 const exists = !!row || slug === HOME_SLUG;
 const layout = row ? sanitizeLayout(row.layout) : defaultLayout(slug);
 return c.json({ slug, title: row?.title ?? (slug === HOME_SLUG ? 'Home' : slug), layout, exists,
customized: !!row });
});

/** Create a custom page. */
pages.post('/', requirePermission('pages.manage'), async (c) => {
 const body = await c.req.json<{ title?: string; slug?: string }>();
 const title = (body.title ?? '').trim().slice(0, 80);
 if (!title) return c.json({ error: 'A title is required.' }, 400);

 const slug = (body.slug?.trim() || slugify(title)).toLowerCase();
 if (!isValidPageSlug(slug)) {
 return c.json({ error: 'That name can't be used ? pick another (letters, numbers, dashes; not
a reserved word).' }, 400);
 }

 const existing = await db(c.env).query.pageLayouts.findFirst({ where: eq(s.pageLayouts.slug,
slug) });
 if (existing) return c.json({ error: 'A page with that address already exists.' }, 409);

 const viewer = c.get('viewer')!;
 const now = Math.floor(Date.now() / 1000);

```

**src/server/routes/pages.ts**

```
 await db(c.env)
 .insert(s.pageLayouts)
 .values({ slug, title, layout: { version: 1, rows: [] }, updatedBy: viewer.id, updatedAt: now
 });

 await db(c.env).insert(s.auditLog).values({
 actorId: viewer.id,
 action: 'page.create',
 targetType: 'page',
 targetId: slug,
 meta: { title },
 });

 return c.json({ ok: true, slug, title }, 201);
 });

/** Save a page's layout. */
pages.put('/:slug', requirePermission('pages.manage'), async (c) => {
 const slug = c.req.param('slug');
 const isHome = slug === HOME_SLUG;
 const existing = await db(c.env).query.pageLayouts.findFirst({ where: eq(s.pageLayouts.slug,
slug) });
 if (!isHome && !existing) return c.json({ error: 'No such page.' }, 404);

 const body = await c.req.json<{ layout?: unknown }>();
 const layout = sanitizeLayout(body.layout);
 const viewer = c.get('viewer')!;
 const now = Math.floor(Date.now() / 1000);
 const title = existing?.title ?? (isHome ? 'Home page' : slug);

 await db(c.env)
 .insert(s.pageLayouts)
 .values({ slug, title, layout, updatedBy: viewer.id, updatedAt: now })
 .onConflictDoUpdate({
 target: s.pageLayouts.slug,
 set: { layout, updatedBy: viewer.id, updatedAt: now },
 });

 await db(c.env).insert(s.auditLog).values({
 actorId: viewer.id,
 action: 'page.layout',
 targetType: 'page',
 targetId: slug,
 meta: { rows: layout.rows.length },
 });

 return c.json({ ok: true, slug, layout });
});

/** Update a page's metadata (title + nav placement). Home's nav flag is ignored. */
pages.patch('/:slug', requirePermission('pages.manage'), async (c) => {
 const slug = c.req.param('slug');
 const existing = await db(c.env).query.pageLayouts.findFirst({ where: eq(s.pageLayouts.slug,
slug) });
});
```

**src/server/routes/pages.ts**

```
if (!existing) return c.json({ error: 'No such page.' }, 404);

const body = await c.req.json<{ title?: string; showInNav?: boolean; navOrder?: number }>();
const set: Record<string, unknown> = { updatedAt: Math.floor(Date.now() / 1000) };
if (typeof body.title === 'string' && body.title.trim()) set.title = body.title.trim().slice(0,
80);
if (slug !== HOME_SLUG && typeof body.showInNav === 'boolean') set.showInNav = body.showInNav;
if (typeof body.navOrder === 'number' && Number.isFinite(body.navOrder)) set.navOrder =
Math.round(body.navOrder);

await db(c.env).update(s.pageLayouts).set(set).where(eq(s.pageLayouts.slug, slug));
return c.json({ ok: true, slug });
});

/**
 * Delete a custom page, or ? for home ? reset it to the built-in default by
 * removing its stored override.
 */
pages.delete('/:slug', requirePermission('pages.manage'), async (c) => {
 const slug = c.req.param('slug');
 const viewer = c.get('viewer')!;
 await db(c.env).delete(s.pageLayouts).where(eq(s.pageLayouts.slug, slug));
 await db(c.env).insert(s.auditLog).values({
 actorId: viewer.id,
 action: slug === HOME_SLUG ? 'page.reset' : 'page.delete',
 targetType: 'page',
 targetId: slug,
 });
 return c.json({ ok: true, slug, layout: defaultLayout(slug) });
});

export default pages;
```

**src/server/routes/ranks.ts**

```

/**
 * Rank management.
 *
 * The whole point of this rewrite: ranks are rows, created and destroyed at
 * will. The 2003 version had exactly 21, baked into column names.
 */

import { Hono } from 'hono';
import { asc, eq, ne, sql } from 'drizzle-orm';
import * as s from '../db/schema';
import type { AppContext } from '../env';
import { db, requirePermission } from '../middleware/auth';
import { discordClient } from '../config';
import { syncRankHolders } from '../discord/sync';
import { deleteMediaByUrl } from '../media';

const ranks = new Hono<AppContext>();

/** Bot client from resolved (DB-over-env) config, or null when unconfigured. */
const rest = (env: AppContext['Bindings']) => discordClient(env);

/** Public: the ladder is visible to anyone. Includes the roles each rank grants. */
ranks.get('/', async (c) => {
 const database = db(c.env);
 const rows = await database
 .select({
 id: s.ranks.id,
 name: s.ranks.name,
 abbreviation: s.ranks.abbreviation,
 imageUrl: s.ranks.imageUrl,
 sortOrder: s.ranks.sortOrder,
 reqDays: s.ranks.reqDays,
 reqWins: s.ranks.reqWins,
 isDefault: s.ranks.isDefault,
 memberCount: sql<number>`(select count(*) from users where users.rank_id = ranks.id)`,
 })
 .from(s.ranks)
 .orderBy(asc(s.ranks.sortOrder));

 const rankRoles = await database.select().from(s.rankRoles);
 const roleIdsByRank = new Map<number, number[]>();
 for (const rr of rankRoles) {
 const list = roleIdsByRank.get(rr.rankId) ?? [];
 list.push(rr.roleId);
 roleIdsByRank.set(rr.rankId, list);
 }

 return c.json({
 ranks: rows.map((r) => ({ ...r, roleIds: roleIdsByRank.get(r.id) ?? [] })),
 });
});

/**
 * Set the roles a rank grants, then re-apply to everyone at that rank so the

```

**src/server/routes/ranks.ts**

```

* change reaches current members (and Discord) immediately.
*/
// Deciding which roles a rank hands out is a role-granting policy, so it needs
// roles.manage rather than merely ranks.manage.
ranks.put('/:id/roles', requirePermission('roles.manage'), async (c) => {
 const id = Number(c.req.param('id'));
 const { roleIds } = await c.req.json<{ roleIds: number[] }>();
 if (!Array.isArray(roleIds)) {
 return c.json({ error: 'Expected a roleIds array' }, 400);
 }

 const database = db(c.env);
 const rank = await database.query.ranks.findFirst({ where: eq(s.ranks.id, id) });
 if (!rank) return c.json({ error: 'No such rank' }, 404);

 const unique = [...new Set(roleIds)];
 await database.batch([
 database.delete(s.rankRoles).where(eq(s.rankRoles.rankId, id)),
 ...(unique.length
 ? [database.insert(s.rankRoles).values(unique.map((roleId) => ({ rankId: id, roleId })))]
 : []),
] as never);

 await database.insert(s.auditLog).values({
 actorId: c.get('viewer')!.id,
 action: 'rank.set_roles',
 targetType: 'rank',
 targetId: String(id),
 meta: { roleIds: unique },
 });

 // Reconcile current holders so the mapping change takes effect now.
 const applied = await syncRankHolders(database, await rest(c.env), id);

 return c.json({ ok: true, roleIds: unique, applied });
});

ranks.post('/', requirePermission('ranks.manage'), async (c) => {
 const body = await c.req.json<{
 name: string;
 abbreviation?: string;
 imageUrl?: string;
 reqDays?: number;
 reqWins?: number;
 }>();

 if (!body.name?.trim()) return c.json({ error: 'Name is required' }, 400);

 const database = db(c.env);
 const highest = await database
 .select({ max: sql<number | null>`max(${s.ranks.sortOrder})` })
 .from(s.ranks);

 const inserted = await database

```

**src/server/routes/ranks.ts**

```

 .insert(s.ranks)
 .values({
 name: body.name.trim(),
 abbreviation: body.abbreviation?.trim() || null,
 imageUrl: body.imageUrl?.trim() || null,
 sortOrder: (highest[0]?.max ?? -1) + 1,
 reqDays: body.reqDays ?? 0,
 reqWins: body.reqWins ?? 0,
 })
 .returning();

const created = inserted[0];
if (!created) return c.json({ error: 'Failed to create rank' }, 500);

await database.insert(s.auditLog).values({
 actorId: c.get('viewer')!.id,
 action: 'rank.create',
 targetType: 'rank',
 targetId: String(created.id),
 meta: { name: created.name },
});

return c.json({ rank: created }, 201);
});

ranks.patch('/:id', requirePermission('ranks.manage'), async (c) => {
 const id = Number(c.req.param('id'));
 const body = await c.req.json<Partial<typeof s.ranks.$inferInsert>>>();
 const database = db(c.env);

 // The image being replaced, so we can free it once the update commits.
 const previousImageUrl =
 body.imageUrl !== undefined
 ? ((await database.query.ranks.findFirst({ where: eq(s.ranks.id, id) }))?.imageUrl ?? null)
 : null;

 // Exactly one default rank, or new recruits land nowhere.
 if (body.isDefault) {
 await database.update(s.ranks).set({ isDefault: false }).where(ne(s.ranks.id, id));
 }

 const updated = await database
 .update(s.ranks)
 .set({ ...body, id: undefined, updatedAt: Math.floor(Date.now() / 1000) })
 .where(eq(s.ranks.id, id))
 .returning();

 if (!updated.length) return c.json({ error: 'No such rank' }, 404);

 if (body.imageUrl !== undefined && previousImageUrl !== updated[0]!.imageUrl) {
 c.executionCtx.waitUntil(deleteMediaByUrl(c.env, previousImageUrl));
 }

 return c.json({ rank: updated[0] });
});

```

**src/server/routes/ranks.ts**

```

});

/**
 * Reordering rewrites every sortOrder in one shot, which avoids the unique
 * constraint tripping over an intermediate state.
 */
ranks.put('/order', requirePermission('ranks.manage'), async (c) => {
 const { order } = await c.req.json<{ order: number[] }>(); // ids, lowest rank first
 if (!Array.isArray(order) || !order.length) {
 return c.json({ error: 'Expected a non-empty array of rank ids' }, 400);
 }

 const database = db(c.env);
 // Park them out of the way first so no two rows collide mid-update.
 await database.batch([
 database.update(s.ranks).set({ sortOrder: sql`${s.ranks.sortOrder} + 10000` }),
 ...order.map((id, index) =>
 database.update(s.ranks).set({ sortOrder: index }).where(eq(s.ranks.id, id)),
),
] as never);

 return c.json({ ok: true });
});

/**
 * Delete a rank. When members still hold it, the caller must say where they go
 * via `reassignTo` (another rank id, or null for "unranked") plus a `reason`.
 *
 * The move is set-based and keyed on the rank being deleted ? no per-member
 * loop or inline Discord call ? so it scales to any roster size. Rank-derived
 * roles are reconciled to the target (manual grants are left alone), affected
 * members are pushed to the front of the reconcile sweep so Discord catches up
 * shortly after, and the default-rank flag is handed off if this was it.
 */
ranks.delete('/:id', requirePermission('ranks.manage'), async (c) => {
 const id = Number(c.req.param('id'));
 const database = db(c.env);
 const actorId = c.get('viewer')!.id;

 const rank = await database.query.ranks.findFirst({ where: eq(s.ranks.id, id) });
 if (!rank) return c.json({ error: 'No such rank' }, 404);

 const body = await c.req
 .json<{ reassignTo?: number | null; reason?: string }>()
 .catch(() => ({})) as { reassignTo?: number | null; reason?: string };

 const holders = await database
 .select({ n: sql<number>`count(*)` })
 .from(s.users)
 .where(eq(s.users.rankId, id));
 const memberCount = Number(holders[0]?.n ?? 0);

 let target: typeof s.ranks.$inferSelect | null = null;
 if (memberCount > 0) {

```



**src/server/routes/ranks.ts**

```

 if (!Object.prototype.hasOwnProperty.call(body, 'reassignTo')) {
 return c.json(
 { error: `${memberCount} member(s) hold this rank. Choose where to move them.` },
 memberCount },
 409,
);
 }
 if (!body.reason?.trim()) {
 return c.json({ error: 'A reason is required to delete a rank that has members.' }, 400);
 }
 if (body.reassignTo !== null) {
 if (Number(body.reassignTo) === id) {
 return c.json({ error: 'Cannot reassign members to the rank being deleted.' }, 400);
 }
 target =
 (await database.query.ranks.findFirst({ where: eq(s.ranks.id, Number(body.reassignTo)) }))
 ??
 null;
 if (!target) return c.json({ error: 'The rank to move members to does not exist.' }, 400);
 }

 // Reconcile rank-derived roles while the members still carry the old rank,
 // so the same code path serves "move to a rank" and "make unranked".
 const oldRoleIds = (
 await database.select({ roleId: s.rankRoles.roleId
 }).from(s.rankRoles).where(eq(s.rankRoles.rankId, id))
).map((r) => r.roleId);
 const newRoleIds = target
 ? (
 await database
 .select({ roleId: s.rankRoles.roleId })
 .from(s.rankRoles)
 .where(eq(s.rankRoles.rankId, target.id))
).map((r) => r.roleId)
 : [];
 const staleRoleIds = oldRoleIds.filter((r) => !newRoleIds.includes(r));

 // Front-of-queue the affected members so the reconcile sweep pushes their
 // Discord role changes soon.
 await database.update(s.users).set({ lastReconciledAt: 0 }).where(eq(s.users.rankId, id));

 // Drop old rank-granted roles the new rank doesn't grant (source='manual'
 // grants are never matched, so an admin's explicit grant survives).
 if (staleRoleIds.length) {
 await database.run(
 sql`delete from user_roles where source = 'rank'
 and role_id in (${sql.join(
 staleRoleIds.map((r) => sql`${r}`),
 sql`, `,
)})
 and user_id in (select id from users where rank_id = ${id})`,
);
 }
 // Add the target rank's roles (skips any the member already holds).

```

**src/server/routes/ranks.ts**

```

 for (const roleId of newRoleIds) {
 await database.run(
 sql`insert into user_roles (user_id, role_id, source, granted_by)
 select id, ${roleId}, 'rank', ${actorId} from users where rank_id = ${id}
 on conflict(user_id, role_id) do nothing`,
);
 }

 // Finally move the members off the rank we're about to delete.
 await database.update(s.users).set({ rankId: target ? target.id : null
 }).where(eq(s.users.rankId, id));
 }

 // Never leave the roster without a default rank for new recruits.
 if (rank.isDefault) {
 const fallback =
 target?.id ??
 (
 await database
 .select({ id: s.ranks.id })
 .from(s.ranks)
 .where(ne(s.ranks.id, id))
 .orderBy(asc(s.ranks.sortOrder))
 .limit(1)
)[0]?.id;
 if (fallback) await database.update(s.ranks).set({ isDefault: true }).where(eq(s.ranks.id,
 fallback));
 }

 await database.delete(s.ranks).where(eq(s.ranks.id, id));
 c.executionCtx.waitUntil(deleteMediaByUrl(c.env, rank.imageUrl));

 await database.insert(s.auditLog).values([
 actorId,
 action: 'rank.delete',
 targetType: 'rank',
 targetId: String(id),
 reason: body.reason?.trim() || null,
 meta: {
 name: rank.name,
 memberCount,
 reassignedTo: target?.id ?? null,
 unranked: memberCount > 0 && !target,
 },
],
 });

 return c.json({ ok: true, memberCount, reassignedTo: target?.id ?? null });
});

export default ranks;

```

**src/server/routes/roles.ts**

```

/**
 * Role management ? the capability side of the permission model.
 *
 * A role bundles permission strings and, optionally, mirrors a Discord role.
 * Setting discordRoleId is what lets granting a role here also grant it in
 * Discord (see discord/sync.ts and the member role routes).
 */

import { Hono } from 'hono';
import { asc, eq, sql } from 'drizzle-orm';
import * as s from '../db/schema';
import type { AppContext } from '../env';
import { db, requireAuth, requirePermission } from '../middleware/auth';
import { can, isPermission, type Permission } from '../shared/permissions';
import { DiscordError } from '../discord/rest';
import { discordClient } from '../config';

const roles = new Hono<AppContext>();

/** Bot client from resolved (DB-over-env) config, or null when unconfigured. */
const rest = (env: AppContext['Bindings']) => discordClient(env);

/** "#c0392b" -> 12597931. Null/blank/invalid -> 0, which Discord shows as no colour. */
function hexToInt(hex: string | null): number {
 if (!hex) return 0;
 const n = parseInt(hex.replace(/^#/ , ''), 16);
 return Number.isNaN(n) ? 0 : n;
}

/** Every role with its permission list and how many members hold it. */
roles.get('/', requirePermission('roles.manage'), async (c) => {
 const database = db(c.env);

 const rows = await database
 .select({
 id: s.roles.id,
 name: s.roles.name,
 description: s.roles.description,
 color: s.roles.color,
 discordRoleId: s.roles.discordRoleId,
 sortOrder: s.roles.sortOrder,
 isSystem: s.roles.isSystem,
 memberCount: sql<number>`(select count(*) from user_roles where user_roles.role_id =
roles.id)`,
 })
 .from(s.roles)
 .orderBy(asc(s.roles.sortOrder), asc(s.roles.name));

 const perms = await database.select().from(s.rolePermissions);
 const byRole = new Map<number, Permission[]>();
 for (const p of perms) {
 const list = byRole.get(p.roleId) ?? [];
 list.push(p.permission as Permission);
 byRole.set(p.roleId, list);
 }

```

**src/server/routes/roles.ts**

```

 }

 return c.json({
 roles: rows.map((r) => ({ ...r, permissions: byRole.get(r.id) ?? [] })),
 });
 });

/**
 * A minimal role list for people who can assign roles but not necessarily
 * manage them (an Officer has roles.assign but not roles.manage). Used by the
 * member page's grant control.
 */
roles.get('/assignable', requireAuth, async (c) => {
 const viewer = c.get('viewer');
 if (!can(viewer, 'roles.assign') && !can(viewer, 'roles.manage')) {
 return c.json({ error: 'Forbidden' }, 403);
 }
 const list = await db(c.env)
 .select({ id: s.roles.id, name: s.roles.name, color: s.roles.color })
 .from(s.roles)
 .orderBy(asc(s.roles.sortOrder), asc(s.roles.name));
 return c.json({ roles: list });
});

/**
 * The guild's assignable Discord roles, for the mapping dropdown.
 *
 * Excludes @everyone and managed roles (bot/integration roles Discord won't
 * let anyone assign by hand). Returns [] with a reason when the bot isn't
 * configured or lacks access, so the UI can explain rather than just break.
 */
roles.get('/discord-roles', requirePermission('roles.manage'), async (c) => {
 const client = await rest(c.env);
 if (!client) {
 return c.json({ roles: [], warning: 'Discord bot token or guild ID is not configured.' });
 }

 try {
 const all = await client.listRoles();
 const assignable = all
 .filter((r) => r.name !== '@everyone' && !r.managed)
 .sort((a, b) => b.position - a.position)
 .map((r) => ({
 id: r.id,
 name: r.name,
 // Discord stores role colour as a 24-bit int; 0 means "no colour".
 color: r.color ? `#${r.color.toString(16).padStart(6, '0')}` : null,
 position: r.position,
 }));
 return c.json({ roles: assignable });
 } catch (err) {
 const warning =
 err instanceof DiscordError && err.status === 403
 ? 'The bot cannot read this server's roles ? check it has been invited and has the View

```

**src/server/routes/roles.ts**

```

Server permission.'
 : `Could not load Discord roles: ${err as Error}.message`;
 return c.json({ roles: [], warning });
 }
});

roles.post('/', requirePermission('roles.manage'), async (c) => {
 const body = await c.req.json<{
 name: string;
 description?: string;
 color?: string;
 discordRoleId?: string;
 }>();

 if (!body.name?.trim()) return c.json({ error: 'Name is required' }, 400);

 const database = db(c.env);
 const highest = await database
 .select({ max: sql<number | null>`max(${s.roles.sortOrder})` })
 .from(s.roles);

 try {
 const inserted = await database
 .insert(s.roles)
 .values({
 name: body.name.trim(),
 description: body.description?.trim() || null,
 color: body.color?.trim() || null,
 discordRoleId: body.discordRoleId?.trim() || null,
 sortOrder: (highest[0]?.max ?? -1) + 1,
 })
 .returning();

 const created = inserted[0];
 if (!created) return c.json({ error: 'Failed to create role' }, 500);

 await database.insert(s.auditLog).values({
 actorId: c.get('viewer')!.id,
 action: 'role.create',
 targetType: 'role',
 targetId: String(created.id),
 meta: { name: created.name },
 });

 return c.json({ role: { ...created, permissions: [] } }, 201);
 } catch (err) {
 // roles.name is unique; surface the collision instead of a 500.
 if (String(err).includes('UNIQUE')) {
 return c.json({ error: `A role named "${body.name.trim()}" already exists.` }, 409);
 }
 throw err;
 }
});

```

**src/server/routes/roles.ts**

```

roles.patch('/:id', requirePermission('roles.manage'), async (c) => {
 const id = Number(c.req.param('id'));
 const body = await c.req.json<{
 name?: string;
 description?: string | null;
 color?: string | null;
 discordRoleId?: string | null;
 }>();

 const database = db(c.env);
 const role = await database.query.roles.findFirst({ where: eq(s.roles.id, id) });
 if (!role) return c.json({ error: 'No such role' }, 404);

 // Renaming any role is safe: nothing in the code looks a role up by name
 // (permissions and memberships key off the role id), so names are cosmetic.
 // The isSystem flag only guards against deletion, below.

 const patch: Partial<typeof s.roles.$inferInsert> = { updatedAt: Math.floor(Date.now() / 1000)
};
 if (body.name !== null) patch.name = body.name.trim();
 if (body.description !== undefined) patch.description = body.description?.trim() || null;
 if (body.color !== undefined) patch.color = body.color?.trim() || null;
 if (body.discordRoleId !== undefined) patch.discordRoleId = body.discordRoleId?.trim() || null;

 const updated = (await database.update(s.roles).set(patch).where(eq(s.roles.id,
id)).returning())[0]!;

 // Website is the source of truth for name and colour. Push both to Discord
 // when the role is (or has just become) mapped and any of name/colour/mapping
 // changed, so "set it here -> set there" holds. A newly-set mapping also
 // adopts the current website name and colour (the "associate -> sync"
 // behaviour). Name and colour go in one PATCH, so a change to either keeps
 // the whole Discord role in step.
 const nameChanged = patch.name !== null && patch.name !== role.name;
 const colorChanged = body.color !== undefined && updated.color !== role.color;
 const mappingChanged =
 body.discordRoleId !== undefined && updated.discordRoleId !== role.discordRoleId;
 let discordSync: { synced: boolean; warning?: string } | undefined;

 if (updated.discordRoleId && (nameChanged || colorChanged || mappingChanged)) {
 const client = await rest(c.env);
 if (!client) {
 discordSync = {
 synced: false,
 warning: 'Saved. Discord bot is not configured, so the Discord role was not updated.',
 };
 } else {
 try {
 await client.updateRole(
 updated.discordRoleId,
 { name: updated.name, color: hexToInt(updated.color) },
 `ClanTek: synced by ${c.get('viewer')!.username}`,
);
 discordSync = { synced: true };
 }
 }
 }
}

```

**src/server/routes/roles.ts**

```

 } catch (err) {
 discordSync =
 err instanceof DiscordError && err.status === 403
 ? {
 synced: false,
 warning:
 'Saved here, but Discord refused: the bot needs its role dragged above this one
in Server Settings -> Roles.',
 }
 : { synced: false, warning: `Saved here, but the Discord update failed: ${err as
Error}.message}` };
 }
 }

 await database.insert(s.auditLog).values({
 actorId: c.get('viewer')!.id,
 action: 'role.discord_sync',
 targetType: 'role',
 targetId: String(id),
 meta: {
 discordRoleId: updated.discordRoleId,
 name: updated.name,
 color: updated.color,
 synced: discordSync.synced,
 },
 });
}

return c.json({ role: updated, discordSync });
});

/** Replace a role's permission set wholesale. */
roles.put('/:id/permissions', requirePermission('roles.manage'), async (c) => {
 const id = Number(c.req.param('id'));
 const { permissions } = await c.req.json<{ permissions: string[] }>();

 if (!Array.isArray(permissions)) {
 return c.json({ error: 'Expected a permissions array' }, 400);
 }
 const invalid = permissions.filter((p) => !isPermission(p));
 if (invalid.length) {
 return c.json({ error: `Unknown permission(s): ${invalid.join(', ')} ` }, 400);
 }

 const database = db(c.env);
 const role = await database.query.roles.findFirst({ where: eq(s.roles.id, id) });
 if (!role) return c.json({ error: 'No such role' }, 404);

 const unique = [...new Set(permissions)];
 await database.batch([
 database.delete(s.rolePermissions).where(eq(s.rolePermissions.roleId, id)),
 ...(unique.length
 ? [database.insert(s.rolePermissions).values(unique.map((permission) => ({ roleId: id,
permission }))]
]
)

```

**src/server/routes/roles.ts**

```

 : []),
] as never);

 await database.insert(s.auditLog).values({
 actorId: c.get('viewer')!.id,
 action: 'role.permissions',
 targetType: 'role',
 targetId: String(id),
 meta: { permissions: unique },
 });

 return c.json({ ok: true, permissions: unique });
});

/**
 * Delete a role. When members hold it, the caller passes a `reason` and, if they
 * want the members to keep an equivalent capability, a `reassignTo` role id ?
 * holders are granted that role (as an explicit/manual grant) before this one
 * cascades away. Omitting `reassignTo` (or null) just removes the role from
 * everyone, the old behaviour. The move is set-based so it scales, and holders
 * are pushed to the front of the reconcile sweep so Discord follows.
 */
roles.delete('/:id', requirePermission('roles.manage'), async (c) => {
 const id = Number(c.req.param('id'));
 const database = db(c.env);
 const actorId = c.get('viewer')!.id;

 const role = await database.query.roles.findFirst({ where: eq(s.roles.id, id) });
 if (!role) return c.json({ error: 'No such role' }, 404);
 if (role.isSystem) {
 return c.json({ error: `"$${role.name}" is a system role and cannot be deleted.` }, 403);
 }

 const body = await c.req
 .json<{ reassignTo?: number | null; reason?: string }>()
 .catch(() => ({})) as { reassignTo?: number | null; reason?: string };

 const holders = await database
 .select({ n: sql<number>`count(*)` })
 .from(s.userRoles)
 .where(eq(s.userRoles.roleId, id));
 const memberCount = Number(holders[0]?.n ?? 0);

 let target: typeof s.roles.$inferSelect | null = null;
 if (memberCount > 0) {
 if (!body.reason?.trim()) {
 return c.json({ error: 'A reason is required to delete a role that has members.' }, 400);
 }
 if (body.reassignTo !== null) {
 if (Number(body.reassignTo) === id) {
 return c.json({ error: 'Cannot reassign members to the role being deleted.' }, 400);
 }
 target =
 (await database.query.roles.findFirst({ where: eq(s.roles.id, Number(body.reassignTo)) }));
 }
 }

```



**src/server/routes/roles.ts**

```

??
 null;
 if (!target) return c.json({ error: 'The role to move members to does not exist.' }, 400);
 }

 // Front-of-queue holders so the reconcile sweep updates Discord soon.
 await database.run(
 sql`update users set last_reconciled_at = 0 where id in (select user_id from user_roles
where role_id = ${id})`,
);

 // Move holders onto the replacement (explicit grant) before the old role ?
 // and its memberships ? cascade away on delete.
 if (target) {
 await database.run(
 sql`insert into user_roles (user_id, role_id, source, granted_by)
 select user_id, ${target.id}, 'manual', ${actorId} from user_roles where role_id =
${id}
 on conflict(user_id, role_id) do nothing`,
);
 }

 // user_roles and role_permissions cascade on delete (see schema). The Discord
 // side is left to the reconcile sweep ? it removes the old mapped role and
 // adds the replacement's, from the memberships we just wrote.
 await database.delete(s.roles).where(eq(s.roles.id, id));

 await database.insert(s.auditLog).values({
 actorId,
 action: 'role.delete',
 targetType: 'role',
 targetId: String(id),
 reason: body.reason?.trim() || null,
 meta: { name: role.name, memberCount, reassignedTo: target?.id ?? null },
 });

 return c.json({ ok: true, memberCount, reassignedTo: target?.id ?? null });
});

export default roles;

```

**src/server/routes/scVerify.ts**

```

/**
 * Star Citizen account verification ? the anti-poser track for the SC module.
 *
 * Flow: a member claims an RSI handle (`verify/start`) and gets a one-time code;
 * they paste it into their public RSI Short Bio; `verify/confirm` fetches the
 * profile, checks the code is present (proving control of the account), parses
 * the org block, and records whether it matches this install's configured org.
 * Verification is informational (a badge) ? it gates nothing. All routes 404
 * when the module is off. Handle uniqueness stops two members claiming one
 * account. We never store RSI credentials or log in on the member's behalf.
 */

import { Hono } from 'hono';
import { eq, and, ne } from 'drizzle-orm';
import * as s from '../db/schema';
import type { AppContext } from '../env';
import { db, requireAuth } from '../middleware/auth';
import { loadModules, loadScConfig } from '../modules';
import { fetchCitizen, parseCitizen, isValidHandle } from '../sc/rsi';

const scVerify = new Hono<AppContext>();

/**
 * Verification available? The SC module must be on AND the verification feature
 * not killed (its own kill switch ? this is the RSI-fetching path, so it can be
 * turned off independently of the client-side hangar import).
 */
async function scEnabled(env: AppContext['Bindings']): Promise<boolean> {
 const [mods, cfg] = await Promise.all([loadModules(env, db(env)), loadScConfig(env, db(env))]);
 return mods.starcitizen && cfg.verifyEnabled;
}

/** A one-time bio challenge code, e.g. muststr-verify-9f3a2c1b. */
function genCode(): string {
 const hex = Array.from(crypto.getRandomValues(new Uint8Array(4)))
 .map((b) => b.toString(16).padStart(2, '0'))
 .join('');
 return `muststr-verify-${hex}`;
}

/**
 * Verification status for a member, for the profile badge. Everyone (auth'd)
 * can read the verified result; only the owner gets their in-flight challenge.
 */
scVerify.get('/verify/:userId', requireAuth, async (c) => {
 if (!(await scEnabled(c.env))) return c.json({ error: 'Module not enabled.' }, 404);
 const viewer = c.get('viewer')!;
 const userId = Number(c.req.param('userId'));
 const self = viewer.id === userId;
 const row = await db(c.env).query.scVerifications.findFirst({
 where: eq(s.scVerifications.userId, userId),
 });
 return c.json({
 verified: !!row?.verifiedAt,

```

**src/server/routes/scVerify.ts**

```

 rsiHandle: row?.rsiHandle ?? null,
 orgSid: row?.orgSid ?? null,
 orgRank: row?.orgRank ?? null,
 inOrg: !!row?.inOrg,
 orgVisible: !!row?.orgVisible,
 verifiedAt: row?.verifiedAt ?? null,
 // The pending code/handle is private to the owner (mid-verification).
 pending: self && row?.pendingCode ? { code: row.pendingCode, handle: row.pendingHandle } :
null,
 });
});

/** Claim a handle and receive a code to paste into the RSI bio. */
scVerify.post('/verify/start', requireAuth, async (c) => {
 if (!(await scEnabled(c.env))) return c.json({ error: 'Module not enabled.' }, 404);
 const viewer = c.get('viewer')!;
 const body = await c.req.json<{ handle?: unknown }>().catch(() => ({})) as { handle?: unknown };
 const handle = typeof body.handle === 'string' ? body.handle.trim() : '';
 if (!isValidHandle(handle)) {
 return c.json({ error: 'Enter a valid RSI handle (letters, numbers, "_" or "-").' }, 400);
 }
 const code = genCode();
 const now = Math.floor(Date.now() / 1000);
 await db(c.env)
 .insert(s.scVerifications)
 .values({ userId: viewer.id, pendingCode: code, pendingHandle: handle, pendingAt: now })
 .onConflictDoUpdate({
 target: s.scVerifications.userId,
 set: { pendingCode: code, pendingHandle: handle, pendingAt: now },
 });
 return c.json({ code, handle });
});

/** Fetch the RSI profile, confirm the code + org, and record the verification. */
scVerify.post('/verify/confirm', requireAuth, async (c) => {
 if (!(await scEnabled(c.env))) return c.json({ error: 'Module not enabled.' }, 404);
 const viewer = c.get('viewer')!;
 const row = await db(c.env).query.scVerifications.findFirst({
 where: eq(s.scVerifications.userId, viewer.id),
 });
 if (!row?.pendingCode || !row.pendingHandle) {
 return c.json({ ok: false, error: 'Start a verification first.' }, 400);
 }

 // Light rate-limit: at most one RSI fetch per member per cooldown, to be a
 // polite citizen (the check button can't hammer RSI). Stamp it BEFORE fetching
 // so rapid retries are throttled even while a fetch is in flight.
 const COOLDOWN_SECONDS = 15;
 const nowSec = Math.floor(Date.now() / 1000);
 if (row.lastCheckAt && nowSec - row.lastCheckAt < COOLDOWN_SECONDS) {
 const wait = COOLDOWN_SECONDS - (nowSec - row.lastCheckAt);
 return c.json({ ok: false, error: `Please wait ${wait}s before checking again.` });
 }
 await db(c.env)

```

**src/server/routes/scVerify.ts**

```

 .update(s.scVerifications)
 .set({ lastCheckAt: nowSec })
 .where(eq(s.scVerifications.userId, viewer.id));

 let html: string;
 try {
 const r = await fetchCitizen(row.pendingHandle);
 if (!r.ok) {
 return c.json({ ok: false, error: `Could not load that RSI profile (status ${r.status}).
Check the handle.` });
 }
 html = r.html;
 } catch {
 return c.json({ ok: false, error: 'Could not reach RSI right now ? try again in a moment.' });
 }

 const profile = parseCitizen(html);
 if (!profile.bio.includes(row.pendingCode)) {
 return c.json({
 ok: false,
 error: 'The code wasn't found in your RSI Short Bio yet. Add it, Save on RSI, then try
again.',
 });
 }

 // No other member may already own this handle (rsiHandle is also UNIQUE).
 const taken = await db(c.env).query.scVerifications.findFirst({
 where: and(eq(s.scVerifications.rsiHandle, row.pendingHandle), ne(s.scVerifications.userId,
viewer.id)),
 });
 if (taken) {
 return c.json({ ok: false, error: 'That RSI handle is already verified by another member.' });
 }

 const cfg = await loadScConfig(c.env, db(c.env));
 const inOrg = !!(
 profile.orgVisible &&
 profile.orgSid &&
 cfg.orgSid &&
 profile.orgSid.toLowerCase() === cfg.orgSid.toLowerCase()
);
 const now = Math.floor(Date.now() / 1000);
 await db(c.env)
 .update(s.scVerifications)
 .set({
 rsiHandle: row.pendingHandle,
 verifiedAt: now,
 orgSid: profile.orgSid,
 orgRank: profile.orgRank,
 orgVisible: profile.orgVisible,
 inOrg,
 pendingCode: null,
 pendingHandle: null,
 pendingAt: null,
 });

```

**src/server/routes/scVerify.ts**

```
 })
 .where(eq(s.scVerifications.userId, viewer.id));

 await db(c.env).insert(s.auditLog).values({
 actorId: viewer.id,
 action: 'sc.verify',
 targetType: 'user',
 targetId: String(viewer.id),
 meta: { rsiHandle: row.pendingHandle, orgSid: profile.orgSid, inOrg },
 });

 return c.json({
 ok: true,
 verified: true,
 rsiHandle: row.pendingHandle,
 orgSid: profile.orgSid,
 orgName: profile.orgName,
 orgRank: profile.orgRank,
 orgVisible: profile.orgVisible,
 inOrg,
 });
 });
});

/** Abandon an in-flight challenge without touching an existing verification. */
scVerify.post('/verify/cancel', requireAuth, async (c) => {
 const viewer = c.get('viewer')!;
 await db(c.env)
 .update(s.scVerifications)
 .set({ pendingCode: null, pendingHandle: null, pendingAt: null })
 .where(eq(s.scVerifications.userId, viewer.id));
 return c.json({ ok: true });
});

/** Remove the member's own verification (and any pending challenge). */
scVerify.delete('/verify', requireAuth, async (c) => {
 const viewer = c.get('viewer')!;
 await db(c.env).delete(s.scVerifications).where(eq(s.scVerifications.userId, viewer.id));
 return c.json({ ok: true });
});

export default scVerify;
```

**src/server/routes/settings.ts**

```

import { Hono } from 'hono';
import { eq } from 'drizzle-orm';
import * as s from '../db/schema';
import type { AppContext, Env } from '../env';
import { db, requireAuth, requirePermission } from '../middleware/auth';
import { loadModules, cleanModuleFlags, MODULES_KEY, loadScConfig, cleanScConfig, SC_KEY } from
'../modules';
import { loadFooter, FOOTER_KEY } from '../footer';
import { cleanFooter } from '../shared/footer';
import { mergeSeo, SEO_KEY, type StoredSeo } from '../seo';
import {
 loadConfig,
 loadAnalytics,
 discordClient,
 mergeStoredIdentity,
 mergeStoredAnalytics,
 ConfigValidationError,
 IDENTITY_KEY,
 ANALYTICS_KEY,
 type StoredIdentity,
 type StoredIdentityInput,
 type StoredAnalytics,
 type StoredAnalyticsInput,
} from '../config';
import {
 loadAnnouncementConfig,
 DEFAULT_ANNOUNCEMENTS,
 type AnnouncementConfig,
 type AnnouncementEventKey,
} from '../discord/announce';
import { fetchRates } from './usage';

const settings = new Hono<AppContext>();

/** Bot client from resolved (DB-over-env) config, or null when unconfigured. */
const rest = (env: Env) => discordClient(env);

/** Effective identity for the admin UI ? secret values become presence booleans. */
function identityView(cfg: Awaited<ReturnType<typeof loadConfig>>) {
 return {
 siteName: cfg.siteName,
 siteUrl: cfg.siteUrl,
 discord: {
 clientId: cfg.discord.clientId,
 guildId: cfg.discord.guildId,
 publicKey: cfg.discord.publicKey,
 clientSecretSet: cfg.discord.clientSecret !== '',
 botTokenSet: cfg.discord.botToken !== '',
 },
 };
}

/* -----
* Game modules ? which optional features (Star Citizen, ...) are enabled.

```

**src/server/routes/settings.ts**

```

* Any signed-in member may read the flags (to show/hide module UI); only
* settings.manage may change them.
* ----- */

settings.get('/modules', requireAuth, async (c) => {
 return c.json({ modules: await loadModules(c.env, db(c.env)) });
});

settings.put('/modules', requirePermission('settings.manage'), async (c) => {
 const body = await c.req.json<{ modules?: unknown }>();
 const clean = cleanModuleFlags(body.modules);
 const viewer = c.get('viewer')!;
 await db(c.env)
 .insert(s.settings)
 .values({ key: MODULES_KEY, value: clean, updatedBy: viewer.id })
 .onConflictDoUpdate({
 target: s.settings.key,
 set: { value: clean, updatedBy: viewer.id, updatedAt: Math.floor(Date.now() / 1000) },
 });
 return c.json({ ok: true, modules: clean });
});

/* ----- *
* Star Citizen module config (org SID for account verification). Readable by
* any signed-in member; only settings.manage may change it.
* ----- */

settings.get('/sc', requireAuth, async (c) => {
 return c.json({ sc: await loadScConfig(c.env, db(c.env)) });
});

settings.put('/sc', requirePermission('settings.manage'), async (c) => {
 const body = await c.req.json<{ sc?: unknown }>();
 const clean = cleanScConfig(body.sc);
 const viewer = c.get('viewer')!;
 await db(c.env)
 .insert(s.settings)
 .values({ key: SC_KEY, value: clean, updatedBy: viewer.id })
 .onConflictDoUpdate({
 target: s.settings.key,
 set: { value: clean, updatedBy: viewer.id, updatedAt: Math.floor(Date.now() / 1000) },
 });
 return c.json({ ok: true, sc: clean });
});

/* ----- *
* Site footer ? shown on every page. GET is public (the footer renders on the
* logged-out login page too); only settings.manage may change it.
* ----- */

settings.get('/footer', async (c) => {
 return c.json({ footer: await loadFooter(c.env, db(c.env)) });
});

```

**src/server/routes/settings.ts**

```

settings.put('/footer', requirePermission('settings.manage'), async (c) => {
 const body = await c.req.json<{ footer?: unknown }>();
 const clean = cleanFooter(body.footer);
 const viewer = c.get('viewer')!;
 await db(c.env)
 .insert(s.settings)
 .values({ key: FOOTER_KEY, value: clean, updatedBy: viewer.id })
 .onConflictDoUpdate({
 target: s.settings.key,
 set: { value: clean, updatedBy: viewer.id, updatedAt: Math.floor(Date.now() / 1000) },
 });
 return c.json({ ok: true, footer: clean });
});

/* ----- *
 * SEO & sharing ? the settings-driven <head> the Worker injects (title,
 * description, Open Graph, Twitter, robots, verification, JSON-LD). The site
 * name/URL themselves live in Identity & Discord; shown here for reference.
 * ----- */

settings.get('/seo', requirePermission('settings.manage'), async (c) => {
 const row = await db(c.env).query.settings.findFirst({ where: eq(s.settings.key, SEO_KEY) });
 const cfg = await loadConfig(c.env, db(c.env));
 return c.json({
 seo: (row?.value as StoredSeo | undefined) ?? {},
 site: { name: cfg.siteName, url: cfg.siteUrl },
 });
});

settings.put('/seo', requirePermission('settings.manage'), async (c) => {
 const body = await c.req.json<{ seo?: unknown }>();
 const clean = mergeSeo(body.seo);
 const viewer = c.get('viewer')!;
 await db(c.env)
 .insert(s.settings)
 .values({ key: SEO_KEY, value: clean, updatedBy: viewer.id })
 .onConflictDoUpdate({
 target: s.settings.key,
 set: { value: clean, updatedBy: viewer.id, updatedAt: Math.floor(Date.now() / 1000) },
 });
 return c.json({ ok: true, seo: clean });
});

/** Public ? the theme has to load before anyone signs in. */
settings.get('/theme', async (c) => {
 const row = await db(c.env).query.settings.findFirst({ where: eq(s.settings.key, 'theme') });
 return c.json({ theme: (row?.value as Record<string, string>) ?? {} });
});

settings.put('/theme', requirePermission('theme.manage'), async (c) => {
 const { theme } = await c.req.json<{ theme: Record<string, string> }>();

 // Only CSS custom properties are storable, so a rogue key cannot become a
 // selector or a declaration when the client applies these.

```



**src/server/routes/settings.ts**

```

const clean = Object.fromEntries(
 Object.entries(theme ?? {}).filter(
 ([k, v]) => k.startsWith('--') && typeof v === 'string' && !v.includes('}'),
),
);

const viewer = c.get('viewer')!;
await db(c.env)
 .insert(s.settings)
 .values({ key: 'theme', value: clean, updatedBy: viewer.id })
 .onConflictDoUpdate({
 target: s.settings.key,
 set: { value: clean, updatedBy: viewer.id, updatedAt: Math.floor(Date.now() / 1000) },
 });

return c.json({ ok: true, theme: clean });
});

settings.get('/site', async (c) => {
 const row = await db(c.env).query.settings.findFirst({ where: eq(s.settings.key, 'site') });
 return c.json({ site: row?.value ?? {} });
});

/**
 * Accept only a same-origin ("/media/...") or absolute http(s) logo URL; anything
 * else ? javascript:, data:, protocol-relative ? becomes empty, so a stored logo
 * URL can never smuggle a script into the header .
 */
function cleanLogoUrl(v: unknown): string {
 if (typeof v !== 'string') return '';
 const t = v.trim();
 if (!t) return '';
 if (t.startsWith('/') && !t.startsWith('//')) return t.slice(0, 500);
 if (/^https?:\/\/\//i.test(t)) return t.slice(0, 500);
 return '';
}

settings.put('/site', requirePermission('settings.manage'), async (c) => {
 const { site } = await c.req.json<{ site: Record<string, unknown> }>();
 const viewer = c.get('viewer')!;

 // Store only the known branding fields, validated ? never the raw payload.
 const size = Math.round(Number((site ?? {}).logoSize));
 const clean = {
 logoUrl: cleanLogoUrl((site ?? {}).logoUrl),
 logoSize: Number.isFinite(size) ? Math.min(200, Math.max(40, size)) : 88,
 faviconUrl: cleanLogoUrl((site ?? {}).faviconUrl),
 };

 await db(c.env)
 .insert(s.settings)
 .values({ key: 'site', value: clean, updatedBy: viewer.id })
 .onConflictDoUpdate({
 target: s.settings.key,

```

**src/server/routes/settings.ts**

```

 set: { value: clean, updatedBy: viewer.id, updatedAt: Math.floor(Date.now() / 1000) },
 });

 return c.json({ ok: true, site: clean });
});

/* -----
 * Discord announcements
 * ----- */

/** The text/announcement channels the bot can see, for the picker. */
settings.get('/discord-channels', requirePermission('settings.manage'), async (c) => {
 const client = await rest(c.env);
 if (!client) return c.json({ channels: [], warning: 'The Discord bot is not configured.' });
 try {
 return c.json({ channels: await client.listTextChannels() });
 } catch (err) {
 return c.json({ channels: [], warning: `Could not load channels: ${err as Error}.message` });
 }
});

settings.get('/announcements', requirePermission('settings.manage'), async (c) => {
 return c.json({ announcements: await loadAnnouncementConfig(db(c.env)) });
});

settings.put('/announcements', requirePermission('settings.manage'), async (c) => {
 const body = await c.req.json<Partial<AnnouncementConfig>>();

 // Only keep the known event flags as booleans, and a plain channel id string.
 const events = {} as AnnouncementConfig['events'];
 for (const key of Object.keys(DEFAULT_ANNOUNCEMENTS.events) as AnnouncementEventKey[]) {
 events[key] = Boolean(body.events?.[key]);
 }
 const channelId =
 typeof body.channelId === 'string' && /^\\d+$/.test(body.channelId) ? body.channelId : null;
 const clean: AnnouncementConfig = { channelId, events };

 const viewer = c.get('viewer')!;
 await db(c.env)
 .insert(s.settings)
 .values({ key: 'announcements', value: clean, updatedBy: viewer.id })
 .onConflictDoUpdate({
 target: s.settings.key,
 set: { value: clean, updatedBy: viewer.id, updatedAt: Math.floor(Date.now() / 1000) },
 });

 return c.json({ ok: true, announcements: clean });
});

/** Post a test message to a channel, to confirm the bot can actually reach it. */
settings.post('/announcements/test', requirePermission('settings.manage'), async (c) => {
 const { channelId } = await c.req.json<{ channelId?: string }>();
 if (!channelId || !/^\\d+$/.test(channelId)) return c.json({ error: 'Choose a channel first.' },

```

**src/server/routes/settings.ts**

```

400);

const client = await rest(c.env);
if (!client) return c.json({ error: 'The Discord bot is not configured.' }, 503);

try {
 await client.createMessage(channelId, {
 embeds: [
 {
 color: 0x5865f2,
 title: '? ClanTek announcements are working',
 description: 'This is a test message. Award a medal or promote someone to see the real
thing.',
 },
],
 });
 return c.json({ ok: true });
} catch (err) {
 return c.json({ error: `Discord refused the message: ${err as Error}.message` }, 502);
}
});

/* ----- *
* Identity & Discord ? the config layer that lets an operator point this
* install at their own Discord app + domain from the UI, instead of editing
* wrangler.jsonc. DB-over-env: what's saved here wins; blanks fall back to the
* wrangler vars/secrets. Secret VALUES are never returned ? only presence.
* ----- */

settings.get('/identity', requirePermission('settings.manage'), async (c) => {
 const cfg = await loadConfig(c.env, db(c.env));
 return c.json({
 identity: identityView(cfg),
 // The exact URLs to paste into the Discord Developer Portal for THIS origin,
 // so the operator never has to hand-assemble them.
 urls: {
 redirectUri: new URL('/api/auth/callback', c.req.url).toString(),
 interactionsUrl: new URL('/api/discord/interactions', c.req.url).toString(),
 },
 });
});

settings.put('/identity', requirePermission('settings.manage'), async (c) => {
 const body = await c.req.json<{ identity?: StoredIdentityInput }>();

 // Merge onto the stored blob so blank secret fields keep the existing secret.
 const existingRow = await db(c.env).query.settings.findFirst({
 where: eq(s.settings.key, IDENTITY_KEY),
 });
 const existing = (existingRow?.value as StoredIdentity | undefined) ?? {};

 let clean: StoredIdentity;
 try {
 clean = mergeStoredIdentity(existing, body.identity ?? {});
 }

```

**src/server/routes/settings.ts**

```

 } catch (err) {
 if (err instanceof ConfigValidationError) return c.json({ error: err.message }, 400);
 throw err;
 }

 const viewer = c.get('viewer')!;
 await db(c.env)
 .insert(s.settings)
 .values({ key: IDENTITY_KEY, value: clean, updatedBy: viewer.id })
 .onConflictDoUpdate({
 target: s.settings.key,
 set: { value: clean, updatedBy: viewer.id, updatedAt: Math.floor(Date.now() / 1000) },
 });

 return c.json({ ok: true, identity: identityView(await loadConfig(c.env, db(c.env))) });
 });

 /** Prove the currently-configured bot token + Server ID actually work. */
 settings.post('/identity/test', requirePermission('settings.manage'), async (c) => {
 const client = await discordClient(c.env, db(c.env));
 if (!client) return c.json({ ok: false, error: 'Bot token or Server (Guild) ID is not set.' }, 400);
 try {
 const channels = await client.listTextChannels();
 return c.json({ ok: true, channelCount: channels.length });
 } catch (err) {
 return c.json({ ok: false, error: `Discord rejected the bot: ${err as Error}.message` }, 502);
 }
 });

 /** ----- *
 * Analytics ? the Cloudflare account/token/script/db the usage dashboard
 * queries for live request & query rates. DB-over-env like identity; the API
 * token is write-only (returned as a presence boolean, never its value).
 * ----- */

 /** Effective analytics config for the admin UI ? the token becomes presence only. */
 function analyticsView(cfg: Awaited<ReturnType<typeof loadAnalytics>>) {
 return {
 accountId: cfg.accountId,
 scriptName: cfg.scriptName,
 d1DatabaseId: cfg.d1DatabaseId,
 apiTokenSet: cfg.apiToken !== '',
 };
 }

 settings.get('/analytics', requirePermission('settings.manage'), async (c) => {
 return c.json({ analytics: analyticsView(await loadAnalytics(c.env, db(c.env))) });
 });

 settings.put('/analytics', requirePermission('settings.manage'), async (c) => {
 const body = await c.req.json<{ analytics?: StoredAnalyticsInput }>();

```

**src/server/routes/settings.ts**

```
const existingRow = await db(c.env).query.settings.findFirst({
 where: eq(s.settings.key, ANALYTICS_KEY),
});
const existing = (existingRow?.value as StoredAnalytics | undefined) ?? {};

let clean: StoredAnalytics;
try {
 clean = mergeStoredAnalytics(existing, body.analytics ?? {});
} catch (err) {
 if (err instanceof ConfigValidationError) return c.json({ error: err.message }, 400);
 throw err;
}

const viewer = c.get('viewer')!;
await db(c.env)
 .insert(s.settings)
 .values({ key: ANALYTICS_KEY, value: clean, updatedBy: viewer.id })
 .onConflictDoUpdate({
 target: s.settings.key,
 set: { value: clean, updatedBy: viewer.id, updatedAt: Math.floor(Date.now() / 1000) },
 });

return c.json({ ok: true, analytics: analyticsView(await loadAnalytics(c.env, db(c.env))) });
});

/** Actually query the Analytics API so the operator sees whether their token works. */
settings.post('/analytics/test', requirePermission('settings.manage'), async (c) => {
 const cfg = await loadAnalytics(c.env, db(c.env));
 if (!cfg.apiToken || !cfg.accountId) {
 return c.json({ ok: false, error: 'Set the Account ID and API token first.' }, 400);
 }
 const { rates, error } = await fetchRates(cfg);
 if (error) return c.json({ ok: false, error }, 502);
 return c.json({ ok: true, requests: rates?.workersRequests.used ?? 0 });
});

export default settings;
```

**src/server/routes/tokens.ts**

```

/**
 * Personal access token management, under /api/auth/tokens.
 *
 * Listing is available to any signed-in member (over web *or* an existing
 * token). Minting and revoking require a real web session (requireWebSession),
 * so a leaked token can neither spawn siblings nor revoke the ones that would
 * lock the attacker out ? credential management stays anchored to the browser.
 */

import { Hono } from 'hono';
import { and, desc, eq } from 'drizzle-orm';
import * as s from '../db/schema';
import type { AppContext } from '../env';
import { db, requireAuth, requireWebSession } from '../middleware/auth';
import { createApiToken } from '../auth/apiToken';

const tokens = new Hono<AppContext>();

/** The caller's own tokens ? never the hash or the raw value. */
tokens.get('/', requireAuth, async (c) => {
 const viewer = c.get('viewer')!;
 const rows = await db(c.env)
 .select({
 id: s.apiTokens.id,
 label: s.apiTokens.label,
 prefix: s.apiTokens.prefix,
 createdAt: s.apiTokens.createdAt,
 lastUsedAt: s.apiTokens.lastUsedAt,
 expiresAt: s.apiTokens.expiresAt,
 })
 .from(s.apiTokens)
 .where(eq(s.apiTokens.userId, viewer.id))
 .orderBy(desc(s.apiTokens.createdAt));
 return c.json({ tokens: rows });
});

/** Mint a token. The raw value is returned exactly once ? never retrievable again. */
tokens.post('/', requireWebSession, async (c) => {
 const viewer = c.get('viewer')!;
 const body = await c.req.json<{ label?: string }>().catch(() => ({})) as { label?: string };
 const label = (body.label ?? '').trim().slice(0, 60) || 'Mobile app';

 const created = await createApiToken(db(c.env), viewer.id, label);

 await db(c.env).insert(s.auditLog).values({
 actorId: viewer.id,
 action: 'token.create',
 targetType: 'api_token',
 targetId: String(created.id),
 meta: { label: created.label, prefix: created.prefix },
 });

 // `token` is the secret; the client must copy it now.
 return c.json(created, 201);
});

```

**src/server/routes/tokens.ts**

```
});

/** Revoke one of the caller's own tokens. */
tokens.delete('/:id', requireWebSession, async (c) => {
 const viewer = c.get('viewer')!;
 const id = Number(c.req.param('id'));
 if (!Number.isInteger(id)) return c.json({ error: 'Bad token id.' }, 400);

 const existing = await db(c.env).query.apiTokens.findFirst({
 where: and(eq(s.apiTokens.id, id), eq(s.apiTokens.userId, viewer.id)),
 });
 if (!existing) return c.json({ error: 'No such token.' }, 404);

 await db(c.env).delete(s.apiTokens).where(eq(s.apiTokens.id, id));

 await db(c.env).insert(s.auditLog).values({
 actorId: viewer.id,
 action: 'token.revoke',
 targetType: 'api_token',
 targetId: String(id),
 meta: { label: existing.label, prefix: existing.prefix },
 });

 return c.json({ ok: true });
});

export default tokens;
```

**src/server/routes/usage.ts**

```

/**
 * Free-tier usage for the admin dashboard.
 *
 * Two tiers of data:
 * - Storage (R2 bytes/objects, D1 size) is measured from inside the Worker,
 * so it always works.
 * - Daily rate counters (Workers requests, D1 rows read/written) come from
 * Cloudflare's GraphQL Analytics API and only appear when a read-only
 * "Account Analytics: Read" token is configured (CLOUDFLARE_API_TOKEN +
 * CLOUDFLARE_ACCOUNT_ID). Absent or failing -> rates are null with a reason.
 *
 * Limits below are the current free-tier allowances; they're sent to the client
 * so the gauges stay correct if Cloudflare changes them (edit here).
 */

import { Hono } from 'hono';
import type { AppContext } from '../env';
import { db, requireAuth } from '../middleware/auth';
import { loadAnalytics, type AnalyticsConfig } from '../config';

const usage = new Hono<AppContext>();

const GB = 1024 * 1024 * 1024;
const R2_STORAGE_LIMIT = 10 * GB;
const D1_STORAGE_LIMIT = 5 * GB;
const WORKERS_REQUESTS_PER_DAY = 100_000;
const D1_ROWS_READ_PER_DAY = 5_000_000;
const D1_ROWS_WRITTEN_PER_DAY = 100_000;

/** Sum every object's size in the media bucket. Clan-scale buckets are small; cap the paging to
 stay cheap. */
async function measureR2(bucket: R2Bucket): Promise<{ bytes: number; objects: number; truncated:
 boolean }> {
 let bytes = 0;
 let objects = 0;
 let cursor: string | undefined;
 let pages = 0;
 do {
 const page = await bucket.list({ limit: 1000, cursor });
 for (const o of page.objects) {
 bytes += o.size;
 objects += 1;
 }
 cursor = page.truncated ? page.cursor : undefined;
 pages += 1;
 } while (cursor && pages < 25); // 25k objects ceiling ? beyond that report truncated
 return { bytes, objects, truncated: Boolean(cursor) };
}

/** D1 reports the database's total size in the `meta.size_after` of any query. */
async function measureD1(db: D1Database): Promise<number> {
 const res = await db.prepare('SELECT 1').all();
 return (res.meta as { size_after?: number } | undefined)?.size_after ?? 0;
}

```



**src/server/routes/usage.ts**

```

interface Rates {
 windowHours: number;
 workersRequests: { used: number; limit: number };
 d1RowsRead: { used: number; limit: number };
 d1RowsWritten: { used: number; limit: number };
}

/** Query the GraphQL Analytics API for the last 24h. Returns null + a reason on any problem. */
export async function fetchRates(cfg: AnalyticsConfig): Promise<{ rates: Rates | null; error:
string | null }> {
 if (!cfg.apiToken || !cfg.accountId) {
 return { rates: null, error: null }; // not configured ? not an error
 }

 const until = new Date();
 const since = new Date(until.getTime() - 24 * 60 * 60 * 1000);

 const query = `
 query Usage($account: String!, $script: String!, $db: String!, $since: Time!, $until: Time!) {
 viewer {
 accounts(filter: { accountTag: $account }) {
 workersInvocationsAdaptive(limit: 10000, filter: { scriptName: $script, datetime_geq:
$since, datetime_leq: $until }) {
 sum { requests }
 }
 d1AnalyticsAdaptiveGroups(limit: 10000, filter: { databaseId: $db, datetimeHour_geq:
$since, datetimeHour_leq: $until }) {
 sum { rowsRead rowsWritten }
 }
 }
 }
 }
 `;

 try {
 const resp = await fetch('https://api.cloudflare.com/client/v4/graphql', {
 method: 'POST',
 headers: {
 'content-type': 'application/json',
 authorization: `Bearer ${cfg.apiToken}`,
 },
 body: JSON.stringify({
 query,
 variables: {
 account: cfg.accountId,
 script: cfg.scriptName,
 db: cfg.d1DatabaseId,
 since: since.toISOString(),
 until: until.toISOString(),
 },
 })),
 });

 const json = (await resp.json()) as {

```

**src/server/routes/usage.ts**

```

 data?: { viewer?: { accounts?: Array<Record<string, Array<{ sum?: Record<string, number>
}>>> } } };
 errors?: Array<{ message: string }>;
 };

 if (json.errors?.length) {
 return { rates: null, error: json.errors.map((e) => e.message).join('; ') };
 }

 const acct = json.data?.viewer?.accounts?.[0];
 if (!acct) return { rates: null, error: 'Analytics returned no data for this account.' };

 const sumField = (groups: Array<{ sum?: Record<string, number> }> | undefined, field: string)
=>
 (groups ?? []).reduce((n, g) => n + (g.sum?.[field] ?? 0), 0);

 return {
 rates: {
 windowHours: 24,
 workersRequests: { used: sumField(acct.workersInvocationsAdaptive, 'requests'), limit:
WORKERS_REQUESTS_PER_DAY },
 d1RowsRead: { used: sumField(acct.d1AnalyticsAdaptiveGroups, 'rowsRead'), limit:
D1_ROWS_READ_PER_DAY },
 d1RowsWritten: { used: sumField(acct.d1AnalyticsAdaptiveGroups, 'rowsWritten'), limit:
D1_ROWS_WRITTEN_PER_DAY },
 },
 error: null,
 };
} catch (err) {
 return { rates: null, error: `Could not reach the Analytics API: ${err as Error}.message` };
}
}

usage.get('/', requireAuth, async (c) => {
 const analytics = await loadAnalytics(c.env, db(c.env));
 const [r2, d1Bytes, rateResult] = await Promise.all([
 c.env.MEDIA ? measureR2(c.env.MEDIA) : Promise.resolve(null),
 measureD1(c.env.DB),
 fetchRates(analytics),
]);

 return c.json({
 generatedAt: Math.floor(Date.now() / 1000),
 storage: {
 r2: r2 ? { ...r2, limitBytes: R2_STORAGE_LIMIT } : null,
 d1: { bytes: d1Bytes, limitBytes: D1_STORAGE_LIMIT },
 },
 rates: rateResult.rates,
 ratesConfigured: Boolean(analytics.apiToken && analytics.accountId),
 ratesError: rateResult.error,
 });
});

export default usage;

```

**src/server/routes/usage.ts**

**src/server/routes/warrecords.ts**

```

/**
 * War records ? the catalog of awardable "items of pride".
 *
 * A definition is a name, optional description/image, and an optional game it
 * belongs to. Awarding one to a member lives with the other member-scoped
 * actions in routes/members.ts, mirroring how medals work.
 */

import { Hono } from 'hono';
import { asc, eq, sql } from 'drizzle-orm';
import * as s from '../db/schema';
import type { AppContext } from '../env';
import { db, requireAuth, requirePermission } from '../middleware/auth';
import { deleteMediaByUrl } from '../media';

const warrecords = new Hono<AppContext>();

/** Every war record, with its game (if any) and how many members hold it. */
warrecords.get('/', requireAuth, async (c) => {
 const rows = await db(c.env)
 .select({
 id: s.warRecords.id,
 name: s.warRecords.name,
 description: s.warRecords.description,
 imageUrl: s.warRecords.imageUrl,
 gameId: s.warRecords.gameId,
 gameName: s.games.name,
 sortOrder: s.warRecords.sortOrder,
 awardCount: sql<number>`(select count(*) from member_war_records where
member_war_records.war_record_id = war_records.id)`,
 })
 .from(s.warRecords)
 .leftJoin(s.games, eq(s.warRecords.gameId, s.games.id))
 .orderBy(asc(s.warRecords.sortOrder), asc(s.warRecords.name));

 return c.json({ warRecords: rows });
});

warrecords.post('/', requirePermission('warrecords.manage'), async (c) => {
 const body = await c.req.json<{
 name?: string;
 description?: string;
 imageUrl?: string;
 gameId?: number | null;
 }>();
 if (!body.name?.trim()) return c.json({ error: 'A name is required' }, 400);

 const database = db(c.env);
 const highest = await database
 .select({ max: sql<number | null>`max(${s.warRecords.sortOrder})` })
 .from(s.warRecords);

 const created = (
 await database

```

**src/server/routes/warrecords.ts**

```

 .insert(s.warRecords)
 .values({
 name: body.name.trim().slice(0, 120),
 description: body.description?.trim() || null,
 imageUrl: body.imageUrl?.trim() || null,
 gameId: body.gameId ?? null,
 sortOrder: (highest[0]?.max ?? -1) + 1,
 })
 .returning()
)[0]!;

 await database.insert(s.auditLog).values({
 actorId: c.get('viewer')!.id,
 action: 'warrecord.create',
 targetType: 'war_record',
 targetId: String(created.id),
 meta: { name: created.name },
 });

 return c.json({ warRecord: { ...created, awardCount: 0 } }, 201);
});

warrecords.patch('/:id', requirePermission('warrecords.manage'), async (c) => {
 const id = Number(c.req.param('id'));
 const body = await c.req.json<{
 name?: string;
 description?: string | null;
 imageUrl?: string | null;
 gameId?: number | null;
 }>();

 const database = db(c.env);
 const record = await database.query.warRecords.findFirst({ where: eq(s.warRecords.id, id) });
 if (!record) return c.json({ error: 'No such war record' }, 404);

 const patch: Partial<typeof s.warRecords.$inferInsert> = {};
 if (body.name !== null) {
 if (!body.name.trim()) return c.json({ error: 'Name cannot be empty' }, 400);
 patch.name = body.name.trim().slice(0, 120);
 }
 if (body.description !== undefined) patch.description = body.description?.trim() || null;
 if (body.imageUrl !== undefined) {
 patch.imageUrl = body.imageUrl?.trim() || null;
 if (patch.imageUrl !== record.imageUrl) c.executionCtx.waitUntil(deleteMediaByUrl(c.env, record.imageUrl));
 }
 if (body.gameId !== undefined) patch.gameId = body.gameId ?? null;

 const updated = (await database.update(s.warRecords).set(patch).where(eq(s.warRecords.id, id)).returning())[0]!;

 const awardCount = (
 await database
 .select({ n: sql<number>`count(*)` })

```

**src/server/routes/warrecords.ts**

```
 .from(s.memberWarRecords)
 .where(eq(s.memberWarRecords.warRecordId, id))
)[0]!.n;

 return c.json({ warRecord: { ...updated, awardCount } });
});

warrecords.delete('/:id', requirePermission('warrecords.manage'), async (c) => {
 const id = Number(c.req.param('id'));
 const database = db(c.env);
 const record = await database.query.warRecords.findFirst({ where: eq(s.warRecords.id, id) });
 if (!record) return c.json({ error: 'No such war record' }, 404);

 await database.delete(s.warRecords).where(eq(s.warRecords.id, id));
 c.executionCtx.waitUntil(deleteMediaByUrl(c.env, record.imageUrl));

 await database.insert(s.auditLog).values({
 actorId: c.get('viewer')!.id,
 action: 'warrecord.delete',
 targetType: 'war_record',
 targetId: String(id),
 meta: { name: record.name },
 });

 return c.json({ ok: true });
});

export default warrecords;
```

**src/server/sc/rsi.ts**

```

/**
 * RSI citizen-profile fetch + parse, for Star Citizen account verification.
 *
 * Star Citizen has no public API, but each account has a PUBLIC citizen profile
 * at robertsspaceindustries.com/citizens/<handle> that a Cloudflare Worker can
 * fetch server-side (confirmed: HTTP 200, no bot challenge). We read two things:
 * - the Short Bio text (where a member pastes a one-time verification code)
 * - the "Main organization" block (SID, name, rank, and whether it's redacted)
 *
 * There is no DOM parser in the Worker, so we regex the stable markers found on
 * the real page. Selectors (verified against a live profile):
 * bio : <div class="entry bio"> ... <div class="value"> ...text... </div>
 * org : <div class="main-org ... visibility-V"> (V = public, else redacted)
 * org name = the /orgs/<SID> link text; SID = the link href / SID field;
 * rank = the "Organization rank" entry's .
 * We never store RSI credentials and never log in on the member's behalf.
 */

const RSI_ORIGIN = 'https://robertsspaceindustries.com';
const HANDLE_RE = /^[a-z0-9_-]{1,50}$/i;

export interface CitizenProfile {
 /** Short Bio text, tags stripped and whitespace collapsed ('' if none). */
 bio: string;
 /** Main-org block present AND public (not redacted). */
 orgVisible: boolean;
 /** Parsed org fields ? only meaningful when orgVisible. */
 orgSid: string | null;
 orgName: string | null;
 orgRank: string | null;
}

/** RSI handles are letters/numbers/underscore/hyphen ? reject anything else. */
export function isValidHandle(h: string): boolean {
 return HANDLE_RE.test(h);
}

/**
 * An honest, self-identifying User-Agent (the standard polite-bot convention:
 * a Mozilla token for compatibility, our name + version, and a contact URL).
 * We do NOT masquerade as a browser ? our verification traffic says who it is,
 * so RSI can identify (or block) it, which is the good-faith posture we want.
 */
export const SC_USER_AGENT = 'Mozilla/5.0 (compatible; muststr.gg-verification/1.0;
+https://muststr.gg)';

/** Fetch a public citizen profile. `ok` is a clean 200; `html` is the body. */
export async function fetchCitizen(
 handle: string,
): Promise<{ ok: boolean; status: number; html: string }> {
 const res = await fetch(`${RSI_ORIGIN}/citizens/${handle}`, {
 headers: {
 'User-Agent': SC_USER_AGENT,
 Accept: 'text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8',
 },
 });

```

**src/server/sc/rsi.ts**

```

 'Accept-Language': 'en-US,en;q=0.9',
 },
 redirect: 'follow',
});
const html = await res.text();
return { ok: res.status === 200, status: res.status, html };
}

const stripTags = (s: string): string =>
 s
 .replace(/<[>]*>/g, ' ')
 .replace(/\s+/g, ' ')
 .trim();

/** Parse the bio + main-org block out of a citizen-profile HTML page. */
export function parseCitizen(html: string): CitizenProfile {
 // Bio only renders when non-empty; absent -> ''.
 const bioM = html.match(/class="entry bio"[\s\S]*?<div class="value">([\s\S]*?)<\div>/i);
 const bio = bioM ? stripTags(bioM[1]) : '';

 // main-org carries a visibility-<X> suffix: V = public, anything else redacted.
 const visM = html.match(/class="main-org[^\"]*visibility-([A-Za-z])/i);
 const orgVisible = !!visM && visM[1].toUpperCase() === 'V';

 let orgSid: string | null = null;
 let orgName: string | null = null;
 let orgRank: string | null = null;
 if (orgVisible) {
 const sidM = html.match(/href="\s\/orgs\/([A-Za-z0-9]+)"/i);
 orgSid = sidM ? sidM[1] : null;
 const nameM = html.match(/href="\s\/orgs\/[A-Za-z0-9]+"[^>]*>([<]+)<\a>/i);
 orgName = nameM ? nameM[1].trim() : null;
 const rankM = html.match(/Organization rank<\span>\s*<strong[>]*>([<]+)<\strong>/i);
 orgRank = rankM ? rankM[1].trim() : null;
 }

 return { bio, orgVisible, orgSid, orgName, orgRank };
}

```



**src/server/seo.ts**

```

/**
 * SEO / social meta ? the settings-driven <head> the Worker injects into the
 * served index.html so it is correct for crawlers AND social scrapers (Discord,
 * Google, Facebook, X), which read the raw HTML and never run the React app.
 *
 * Everything an operator sets here is untrusted, so every value is HTML-escaped
 * and URLs are validated before they reach a tag ? robust per-tenant
 * customisation that can never inject script into their own <head>. The real
 * security headers (CSP, etc.) stay in the response headers, not here.
 */

import { loadConfig } from './config';
import type { Env } from './env';
import { drizzle } from 'drizzle-orm/d1';
import { eq } from 'drizzle-orm';
import * as schema from '../db/schema';
import * as s from '../db/schema';

export const SEO_KEY = 'seo';

type DB = ReturnType<typeof drizzle<typeof schema>>;

/** The persisted, all-optional SEO blob (settings['seo']). */
export interface StoredSeo {
 description?: string;
 keywords?: string;
 noindex?: boolean;
 themeColor?: string;
 ogImage?: string;
 ogTitle?: string;
 ogDescription?: string;
 twitterHandle?: string;
 googleVerification?: string;
 bingVerification?: string;
 social?: {
 discord?: string;
 youtube?: string;
 twitch?: string;
 twitter?: string;
 instagram?: string;
 facebook?: string;
 website?: string;
 };
};

export interface ResolvedSeo extends StoredSeo {
 siteName: string;
 siteUrl: string;
 /** Uploaded favicon (browser-tab icon), from the branding `site` blob. */
 favicon: string;
}

const SOCIAL_KEYS = ['discord', 'youtube', 'twitch', 'twitter', 'instagram', 'facebook',
'website'] as const;

```

**src/server/se0.ts**

```

/** siteName + siteUrl (from identity) plus the stored SEO blob, sanitised. */
export async function loadSeo(env: Env, database?: DB): Promise<ResolvedSeo> {
 const dbi = database ?? drizzle(env.DB, { schema });
 const cfg = await loadConfig(env, dbi);
 let stored: StoredSeo = {};
 let favicon = '';
 try {
 const row = await dbi.query.settings.findFirst({ where: eq(s.settings.key, SEO_KEY) });
 if (row?.value && typeof row.value === 'object') stored = row.value as StoredSeo;
 } catch {
 // no settings row yet
 }
 try {
 const site = await dbi.query.settings.findFirst({ where: eq(s.settings.key, 'site') });
 const raw = (site?.value as { faviconUrl?: unknown } | undefined)?.faviconUrl;
 favicon = safeUrl(typeof raw === 'string' ? raw : '');
 } catch {
 // no branding row yet
 }
 return { ...stored, siteName: cfg.siteName || 'ClanTek', siteUrl: cfg.siteUrl || '', favicon };
}

/* ----- validation ----- */

function str(v: unknown, max: number): string {
 return typeof v === 'string' ? v.trim().slice(0, max) : '';
}
function safeUrl(v: string): string {
 const t = v.trim();
 if (/^https?:\/\//i.test(t)) return t.slice(0, 400);
 if (t.startsWith('/') && !t.startsWith('//')) return t.slice(0, 400);
 return '';
}
function hexColor(v: string): string {
 const t = v.trim();
 return /^#[0-9a-fA-F]{3,8}$/.test(t) ? t : '';
}

/** Validate + merge an SEO update (all fields optional; blanks clear). */
export function mergeSeo(incoming: unknown): StoredSeo {
 const o = (incoming ?? {}) as Record<string, unknown>;
 const inSocial = (o.social ?? {}) as Record<string, unknown>;
 const social: StoredSeo['social'] = {};
 for (const k of SOCIAL_KEYS) {
 const u = safeUrl(str(inSocial[k], 400));
 if (u) social[k] = u;
 }
 const rawHandle = str(o.twitterHandle, 40).replace(/^[A-Za-z0-9_@]/g, '');
 return {
 description: str(o.description, 300),
 keywords: str(o.keywords, 300),
 noindex: o.noindex === true,
 themeColor: hexColor(str(o.themeColor, 9)),
 };
}

```

**src/server/seo.ts**

```

 ogImage: safeUrl(str(o.ogImage, 400)),
 ogTitle: str(o.ogTitle, 120),
 ogDescription: str(o.ogDescription, 300),
 // Store as a single-@ handle, or '' when blank.
 twitterHandle: rawHandle ? '@' + rawHandle.replace(/^@+/, '') : '',
 googleVerification: str(o.googleVerification, 200),
 bingVerification: str(o.bingVerification, 200),
 social,
 };
}

/* ----- rendering ----- */

function esc(v: string): string {
 return v
 .replace(/&/g, '&')
 .replace(/</g, '<')
 .replace(/>/g, '>')
 .replace(/"/g, '"');
}

/** Absolute URL for og:image etc. ? same-origin "/media/..." gets the site origin. */
function absolute(siteUrl: string, u: string): string {
 if (!u) return '';
 if (/^https?:\/\/\//i.test(u)) return u;
 if (u.startsWith('/') && siteUrl) return siteUrl.replace(/\/$/, '') + u;
 return '';
}

export interface HeadInjection {
 /** New <title> text (already escaped). */
 title: string;
 /** New content for the existing theme-color meta, or '' to leave it. */
 themeColor: string;
 /** HTML string of extra tags to append into <head> (all escaped/validated). */
 tags: string;
 /** Href for the tab icon (escaped, absolute), or '' to keep the default. */
 favicon: string;
}

/** Build the (escaped) head pieces the Worker injects for a given config. */
export function renderHead(seo: ResolvedSeo): HeadInjection {
 const name = seo.siteName;
 const title = seo.ogTitle || name;
 const desc = seo.description || '';
 const ogTitle = seo.ogTitle || name;
 const ogDesc = seo.ogDescription || desc;
 const ogImage = absolute(seo.siteUrl, seo.ogImage || '');
 const lines: string[] = [];

 const meta = (attr: 'name' | 'property', key: string, content: string) => {
 if (content) lines.push(`<meta ${attr}="${esc(key)}" content="${esc(content)}">`);
 };
};

```

**src/server/seo.ts**

```

 if (desc) meta('name', 'description', desc);
 if (seo.keywords) meta('name', 'keywords', seo.keywords);
 meta('name', 'robots', seo.noindex ? 'noindex, nofollow' : 'index, follow');
 if (seo.googleVerification) meta('name', 'google-site-verification', seo.googleVerification);
 if (seo.bingVerification) meta('name', 'msvalidate.01', seo.bingVerification);

 // Open Graph ? powers Discord/Facebook/etc. link previews.
 meta('property', 'og:site_name', name);
 meta('property', 'og:type', 'website');
 meta('property', 'og:title', ogTitle);
 if (ogDesc) meta('property', 'og:description', ogDesc);
 if (seo.siteUrl) meta('property', 'og:url', seo.siteUrl);
 if (ogImage) meta('property', 'og:image', ogImage);

 // Twitter/X card.
 meta('name', 'twitter:card', ogImage ? 'summary_large_image' : 'summary');
 meta('name', 'twitter:title', ogTitle);
 if (ogDesc) meta('name', 'twitter:description', ogDesc);
 if (ogImage) meta('name', 'twitter:image', ogImage);
 if (seo.twitterHandle && seo.twitterHandle !== '@') meta('name', 'twitter:site',
seo.twitterHandle);

 // JSON-LD Organization ? additive, helps search + rich results.
 const sameAs = Object.values(seo.social ?? {}).filter(Boolean);
 const org: Record<string, unknown> = { '@context': 'https://schema.org', '@type':
'Organization', name };
 if (seo.siteUrl) org.url = seo.siteUrl;
 if (ogImage) org.logo = ogImage;
 if (sameAs.length) org.sameAs = sameAs;
 // Escape "</" so the JSON can't close the <script> early.
 const jsonLd = JSON.stringify(org).replace(/</g, '\\u003c');
 lines.push(`<script type="application/ld+json">${jsonLd}</script>`);

 // Favicon (tab icon) ? absolute so it resolves the same for crawlers.
 const favicon = esc(absolute(seo.siteUrl, seo.favicon || '') || seo.favicon || '');

 return { title: esc(title), themeColor: seo.themeColor || '', tags: lines.join(''), favicon };
}

```

**src/shared/avatar.ts**

```
/**
 * The one place that decides which image to show for a member.
 *
 * Order: the member's self-chosen profile image (an uploaded /media/avatars/...
 * URL), then their Discord avatar, then Discord's default embed avatar. Used on
 * both the server and the client so an avatar never renders two ways. Mirrors
 * the name-resolution rule in shared/names.ts.
 */

const DISCORD_CDN = 'https://cdn.discordapp.com';

/**
 * Discord's built-in default avatar for accounts with no custom one. For the
 * modern username system it's derived from the account id, not the (legacy)
 * discriminator.
 */
function defaultDiscordAvatar(discordId: string): string {
 const index = (BigInt(discordId) >> 22n) % 6n;
 return `${DISCORD_CDN}/embed/avatars/${index}.png`;
}

/** The member's Discord avatar (or the default when they have none). */
export function discordAvatar(discordId: string, hash: string | null, size = 64): string {
 if (!hash) return defaultDiscordAvatar(discordId);
 const ext = hash.startsWith('a_') ? 'gif' : 'png';
 return `${DISCORD_CDN}/avatars/${discordId}/${hash}.${ext}?size=${size}`;
}

/**
 * The image to show for a member. A self-chosen profile image always wins;
 * otherwise fall back to their Discord avatar. `size` only affects the Discord
 * URL ? an uploaded image is served at its stored size.
 */
export function memberAvatar(
 u: { discordId: string; avatar: string | null; profileImageUrl?: string | null },
 size = 64,
): string {
 return u.profileImageUrl?.trim() || discordAvatar(u.discordId, u.avatar, size);
}
```

**src/shared/embeds.ts**

```

/**
 * Video-embed allowlist ? the security core of the "Video embed" module.
 *
 * The rule that keeps this safe: an author pastes *any* URL, but we NEVER echo
 * that string into an <iframe src>. Instead we match the URL against a small set
 * of known providers, extract only an id/slug/channel with a strict character
 * pattern, and rebuild a canonical embed URL from a fixed template on a hardcoded
 * origin. So the only things that can ever reach an iframe are:
 * - one of the fixed origins in EMBED_FRAME_ORIGINS, and
 * - an id that already passed a `^[w-]+$`-style test.
 * A `javascript:` URL, a data: URI, or a link to some other origin simply fails
 * to match and resolveEmbed() returns null ? the module then shows a placeholder
 * instead of framing anything. This is why the sanitized-HTML module forbids
 * <iframe> entirely: embeds only ever happen through this validated path.
 */

/**
 * The page hosts Twitch will accept as the embedding parent. Twitch refuses to
 * load its player unless the embedder's hostname is declared here, and it allows
 * several. Production + localhost (for dev) cover our cases.
 */
export const EMBED_PARENTS = ['mustr.gg', 'localhost'];

/**
 * Every origin an embed iframe can load from. This list is the single source of
 * truth: it's mirrored in the CSP `frame-src` allowlist (public/_headers) and
 * re-checked at render time before an iframe is emitted. Adding a provider means
 * adding its origin here (and a matcher below) ? one place.
 */
export const EMBED_FRAME_ORIGINS = [
 'https://www.youtube-nocookie.com',
 'https://player.twitch.tv',
 'https://clips.twitch.tv',
 'https://player.vimeo.com',
 'https://streamable.com',
] as const;

export type EmbedProvider = 'youtube' | 'twitch' | 'vimeo' | 'streamable';

export interface ResolvedEmbed {
 provider: EmbedProvider;
 /** A canonical https embed URL on one of EMBED_FRAME_ORIGINS. */
 src: string;
}

/** Parse loosely (accept a bare "youtu.be/..." without a scheme) but only http(s). */
function parseUrl(raw: string): URL | null {
 const t = raw.trim();
 if (!t) return null;
 // Normalize to an absolute https URL: keep an existing http(s) scheme, treat a
 // protocol-relative "//host/..." as https, and prepend https:// to a bare host.
 const withProto = /^https?:\/\//i.test(t)
 ? t
 : t.startsWith('//')

```

**src/shared/embeds.ts**

```

 ? `https:${t}`
 : /^[a-z][a-z0-9+.-]*:/i.test(t)
 ? t // some other scheme (javascript:, data:, mailto:) ? let the guard below reject it
 : `https://${t}`;
 try {
 const u = new URL(withProto);
 if (u.protocol !== 'https:' && u.protocol !== 'http:') return null;
 return u;
 } catch {
 return null;
 }
}

/** Host without a leading www., lowercased. */
function host(u: URL): string {
 return u.hostname.replace(/^www\./i, '').toLowerCase();
}

function twitchParents(): string {
 return EMBED_PARENTS.map((p) => `parent=${encodeURIComponent(p)}`).join('&');
}

function youtube(u: URL): string | null {
 const h = host(u);
 const id = (() => {
 if (h === 'youtu.be') return u.pathname.slice(1).split('/')[0] ?? '';
 if (h === 'youtube.com' || h === 'm.youtube.com' || h === 'music.youtube.com' || h ===
'youtube-nocookie.com') {
 if (u.pathname === '/watch') return u.searchParams.get('v') ?? '';
 const m = u.pathname.match(/^\/(?:embed|shorts|live|v)\\/([^\?#]+)/);
 return m ? m[1]! : '';
 }
 })();
 // YouTube ids are exactly 11 URL-safe base64 chars.
 if (!/^[A-Za-z0-9_-]{11}$/.test(id)) return null;
 return `https://www.youtube-nocookie.com/embed/${id}`;
}

function twitch(u: URL): string | null {
 const h = host(u);
 const parents = twitchParents();

 if (h === 'clips.twitch.tv') {
 const slug = u.pathname.slice(1).split('/')[0] ?? '';
 return /^[A-Za-z0-9_-]{1,120}$/.test(slug)
 ? `https://clips.twitch.tv/embed?clip=${encodeURIComponent(slug)}&${parents}`
 : null;
 }

 if (h === 'twitch.tv' || h === 'm.twitch.tv' || h === 'player.twitch.tv') {
 const vid = u.pathname.match(/^\/videos\/(\d{1,20})/);
 if (vid) return `https://player.twitch.tv/?video=${vid[1]}&${parents}`;
 }
}

```

**src/shared/embeds.ts**

```

 const clip = u.pathname.match(/^\/([A-Za-z0-9_]{1,40})\/clip\/([A-Za-z0-9_-]{1,120})/);
 if (clip) return
`https://clips.twitch.tv/embed?clip=${encodeURIComponent(clip[1])}&${parents}`;

 const qVideo = u.searchParams.get('video');
 if (qVideo && /^d{1,20}$/.test(qVideo)) return
`https://player.twitch.tv/?video=${qVideo}&${parents}`;

 const qChannel = u.searchParams.get('channel');
 if (qChannel && /^[A-Za-z0-9_]{1,40}$/.test(qChannel))
 return `https://player.twitch.tv/?channel=${encodeURIComponent(qChannel)}&${parents}`;

 // A bare /<channel> path ? but not Twitch's own non-channel sections.
 const ch = u.pathname.match(/^\/([A-Za-z0-9_]{1,40})\/?$/);
 const reserved = ['videos', 'directory', 'settings', 'downloads', 'subscriptions', 'p',
'jobs', 'turbo'];
 if (ch && !reserved.includes(ch[1]!.toLowerCase()))
 return `https://player.twitch.tv/?channel=${ch[1]}&${parents}`;

 return null;
}

return null;
}

function vimeo(u: URL): string | null {
 if (host(u) !== 'vimeo.com' && host(u) !== 'player.vimeo.com') return null;
 const m = u.pathname.match(/^\/(\d{6,15})(?:\/|$/));
 return m ? `https://player.vimeo.com/video/${m[1]}` : null;
}

function streamable(u: URL): string | null {
 if (host(u) !== 'streamable.com') return null;
 const m = u.pathname.match(/^\/(?:e\/)?([A-Za-z0-9]{3,20})/);
 return m ? `https://streamable.com/e/${m[1]}` : null;
}

const MATCHERS: { provider: EmbedProvider; fn: (u: URL) => string | null }[] = [
 { provider: 'youtube', fn: youtube },
 { provider: 'twitch', fn: twitch },
 { provider: 'vimeo', fn: vimeo },
 { provider: 'streamable', fn: streamable },
];

/**
 * Turn a user-pasted URL into a safe, canonical embed, or null if it isn't a
 * supported provider. The returned `src` is always on an EMBED_FRAME_ORIGINS host.
 */
export function resolveEmbed(raw: string): ResolvedEmbed | null {
 if (typeof raw !== 'string') return null;
 const u = parseUrl(raw);
 if (!u) return null;
 for (const { provider, fn } of MATCHERS) {
 const src = fn(u);

```



**src/shared/embeds.ts**

```
 if (src) return { provider, src };
 }
 return null;
}

/** Belt-and-suspenders: is this string an embed URL on a known origin? */
export function isAllowedEmbedSrc(src: unknown): src is string {
 return typeof src === 'string' && EMBED_FRAME_ORIGINS.some((o) => src.startsWith(`${o}/`));
}
```

**src/shared/footer.ts**

```

/**
 * Site footer ? a small, per-install editable strip shown on every page.
 *
 * Deliberately lean: a short blurb (rendered as sanitised HTML), a handful of
 * links, and a copyright line. Stored as one settings blob so each deployment
 * edits its own footer; a fresh install ships the default below (which carries
 * the non-affiliation / trademark notice, so every instance starts compliant).
 */

export interface FooterLink {
 label: string;
 href: string;
}

export interface FooterConfig {
 /** Short blurb, rendered as sanitised HTML (e.g. a disclaimer line). */
 text: string;
 links: FooterLink[];
 /** Free-text copyright; blank ? the client shows "? {year} {siteName}". */
 copyright: string;
}

/**
 * What a brand-new install shows before anyone edits it: the trademark /
 * non-affiliation notice plus a link to the legal page. Kept generic (no org
 * name) so it's sane for any deployment of the product.
 */
export const DEFAULT_FOOTER: FooterConfig = {
 text:
 'Star Citizen?, Roberts Space Industries?, and Cloud Imperium? are trademarks of Cloud

 Imperium Rights LLC. ' +
 'This site is an unofficial community tool and is not affiliated with, endorsed, or sponsored

 by Cloud Imperium Games or Roberts Space Industries.',
 links: [{ label: 'Legal', href: '/legal' }],
 copyright: '',
};

function asObject(v: unknown): Record<string, unknown> {
 return v && typeof v === 'object' && !Array.isArray(v) ? (v as Record<string, unknown>) : {};
}

/**
 * Accept only same-origin paths ("/...") and absolute http(s) URLs ? everything
 * else (javascript:, data:, protocol-relative) becomes empty, so a stored href
 * can never smuggle a script into an <a href>.
 */
function cleanUrl(v: unknown): string {
 if (typeof v !== 'string') return '';
 const t = v.trim();
 if (!t) return '';
 if (t.startsWith('/') && !t.startsWith('//')) return t.slice(0, 300);
 if (/^https?:\/\//i.test(t)) return t.slice(0, 300);
 return '';
}

```

**src/shared/footer.ts**

```
/** Coerce arbitrary JSON into a valid, bounded FooterConfig. */
export function cleanFooter(raw: unknown): FooterConfig {
 const o = asObject(raw);
 const text = typeof o.text === 'string' ? o.text.slice(0, 2000) : '';
 const links = (Array.isArray(o.links) ? o.links : [])
 .slice(0, 12)
 .map((l) => {
 const lo = asObject(l);
 return {
 label: (typeof lo.label === 'string' ? lo.label : '').slice(0, 60),
 href: cleanUrl(lo.href),
 };
 })
 .filter((l) => l.label && l.href);
 const copyright = typeof o.copyright === 'string' ? o.copyright.slice(0, 200) : '';
 return { text, links, copyright };
}
```

**src/shared/hangar.ts**

```

/**
 * Star Citizen hangar ? the shared model for the SC module.
 *
 * A member imports their hangar from the RSI website (a private, login-only
 * page with no official API) via a bookmarklet that scrapes the fully-loaded
 * item list and copies it to the clipboard; they paste it into their profile.
 * This file is the one authority on the item shape, the category filters, and
 * the sanitiser applied on the way in ? so a hand-edited or malformed paste can
 * never reach storage or the renderer unchecked.
 */

export interface HangarItem {
 id: string;
 name: string;
 /** RSI's own item type slug: ship, skin, equipment, loot, pledge, coupon, component,
 decoration. */
 type: string;
 value: string;
 currency: string;
 availability: string;
 created: string;
 /** RSI media URL ? absolute https, or a same-origin "/media/..." path. */
 image: string;
}

export interface HangarCategory {
 key: string;
 label: string;
 test: (i: HangarItem) => boolean;
}

/**
 * The filter buttons, mirroring RSI's own hangar categories. `type` is reliable
 * for the broad buckets (ships/paints/equipment). RSI does NOT expose the finer
 * Standalone/Game/Combo split per item ? it's a server-side filter there ? so
 * those key off the pledge-name prefix RSI usually applies (see the SC memory).
 */
export const HANGAR_CATEGORIES: HangarCategory[] = [
 { key: 'all', label: 'All items', test: () => true },
 { key: 'allships', label: 'All ships', test: (i) => i.type === 'ship' },
 { key: 'standalone', label: 'Standalone ships', test: (i) => i.type === 'ship' && /standalone
ship/i.test(i.name) },
 { key: 'game', label: 'Game packages', test: (i) => i.type === 'ship' &&
/\b(package|starter|game package)\b/i.test(i.name) },
 { key: 'combo', label: 'Combo packs', test: (i) => i.type === 'ship' && /combo/i.test(i.name) },
 { key: 'paints', label: 'Paints', test: (i) => i.type === 'skin' },
];

/**
 * Item types we never surface. RSI's hangar is mostly noise for a showcase ?
 * store credits, ship equipment, loot, hull components, decorations and coupons
 * clutter the view ? so we keep only the interesting pledges (ships, paints,
 * packages, and anything not on this list). Kept here so the filter is one
 * authority the view can trust.

```

**src/shared/hangar.ts**

```

*/
export const HANGAR_HIDDEN_TYPES = new Set(['coupon', 'decoration', 'equipment', 'loot',
'component']);

/** Whether an item is one we display (not a hidden RSI clutter type). */
export function isDisplayableHangarItem(i: HangarItem): boolean {
 return !HANGAR_HIDDEN_TYPES.has((i.type || '').toLowerCase());
}

/** A hangar reduced to the items worth showing (drops the hidden types). */
export function visibleHangarItems(items: HangarItem[]): HangarItem[] {
 return items.filter(isDisplayableHangarItem);
}

const RSI_ORIGIN = 'https://robertsspaceindustries.com';

/** Absolute, safe image URL for an item, or '' if it isn't http(s)/same-origin. */
export function hangarImageUrl(i: HangarItem): string {
 const s = i.image || '';
 if (s.startsWith('/') && !s.startsWith('//')) return RSI_ORIGIN + s;
 return /^https?:\/\//i.test(s) ? s : '';
}

/** Numeric value for sorting ? strips $, thousands commas and " USD". */
export function hangarValueNum(i: HangarItem): number {
 const n = parseFloat(String(i.value || '').replace(/^[^0-9.]/g, ''));
 return Number.isFinite(n) ? n : -1;
}

const MAX_ITEMS = 3000;
function str(v: unknown, max: number): string {
 return typeof v === 'string' ? v.slice(0, max) : '';
}

/**
 * Structurally sanitise an untrusted hangar paste into storable items. Every
 * field is coerced to a bounded string; images are kept only when http(s) or a
 * same-origin "/media/..." path (so a stored image URL can't smuggle a
 * javascript:/data: payload into an). The noisy `nameableShips` field
 * from the scrape is intentionally dropped.
 */
export function sanitizeHangar(raw: unknown): HangarItem[] {
 const arr = Array.isArray(raw) ? raw : [];
 const items: HangarItem[] = [];
 for (const r of arr.slice(0, MAX_ITEMS)) {
 const o = (r ?? {}) as Record<string, unknown>;
 const image = str(o.image, 400);
 const safeImage = (image.startsWith('/') && !image.startsWith('//')) ||
/^https?:\/\//i.test(image) ? image : '';
 items.push({
 id: str(o.id, 40),
 name: str(o.name, 200),
 type: str(o.type, 40),
 value: str(o.value, 40),

```

**src/shared/hangar.ts**

```

 currency: str(o.currency, 40),
 availability: str(o.availability, 60),
 created: str(o.created, 40),
 image: safeImage,
 });
}
return items;
}

/**
 * The bookmarklet a member runs on their RSI hangar. It scrapes the whole item
 * list and copies it to the clipboard. Kept here (String.raw so the regex
 * backslashes survive) so the profile import UI can show it verbatim.
 */
export const SC_HANGAR_BOOKMARKLET = String.raw`javascript:(()=>{const
q=(e,s)=>e.querySelector(s),v=(e,s)=>{const n=q(e,s);return n?n.value:null},pj=(e,s)=>{const
n=q(e,s);if(!n)return null;try{return JSON.parse((n.textContent||'').trim())}catch(_){return
null}},bg=e=>{const
n=q(e,'.image'),m=n&&((n.getAttribute('style')||'').match(/url\(['"]?([^\"]+)+/));return
m?m[1]:null},cr=e=>{const m=(e.innerText||'').match(/Created:\s*([A-Za-z0-9,]+)/);return
m?m[1].trim():null};const items=[...document.querySelectorAll('ul.list-items.js-inventory >
li')].map(li=>{const t=(q(li,'h3')?.textContent||'').replace(/\s+/g,'
').trim();return{id:v(li,'.js-pledge-id'),name:v(li,'.js-pledge-name'),type:t.includes(' -
')?t.split(' -
')[0].trim():null,value:v(li,'.js-pledge-value'),currency:v(li,'.js-pledge-currency'),availability:(q(li,'
hangar items found. Open your RSI hangar (robertsspaceindustries.com/en/account/pledges), let it
load, then click
again.')});return;}navigator.clipboard.writeText(JSON.stringify(items)).then(()=>alert('Copied
'+items.length+' hangar items. Paste them into your profile on the site.'),()=>{const
ta=document.createElement('textarea');ta.value=JSON.stringify(items);document.body.appendChild(ta);ta.sele
was blocked ? select-all + copy the box that appeared, then paste into the site.'}));})();`;
```

**src/shared/layout.ts**

```

/**
 * The page-layout contract shared by the server (which stores + sanitizes it)
 * and the client (which renders + edits it).
 *
 * A page is a stack of ROWS. Each row is a set of COLUMNS laid out on a 12-unit
 * responsive grid ? a column's `span` is how many of the 12 units it occupies on
 * desktop; on a narrow screen every column collapses to full width and the whole
 * page becomes a single readable stack. Each column holds an ordered list of
 * MODULES ? self-contained blocks (a news feed, the roster, a rich-text note)
 * that fetch their own data at render time. Because the layout is just data,
 * the exact same JSON drives the web app now and can drive a native app later.
 */

import { resolveEmbed } from './embeds';

export type ModuleType =
 | 'heading'
 | 'text'
 | 'html'
 | 'hero'
 | 'image'
 | 'button'
 | 'embed'
 | 'divider'
 | 'news'
 | 'roster'
 | 'events'
 | 'medals'
 | 'warrecords'
 | 'games';

export interface LayoutModule {
 /** Stable id for React keys and drag-and-drop; not meaningful to the server. */
 id: string;
 type: ModuleType;
 /** Per-module options (title, item limit, rich-text html, ...). */
 config: Record<string, unknown>;
 /**
 * Optional audience gate: when set to a role id, the module renders only for
 * viewers who hold that role (e.g. a leadership-only callout). God bypasses it.
 * Absent = everyone. Referenced by id so renaming the role never breaks a page.
 */
 visibleToRole?: number;
}

export interface LayoutColumn {
 id: string;
 /** 1?12 grid units on desktop. Columns in a row need not sum to 12. */
 span: number;
 modules: LayoutModule[];
}

export interface LayoutRow {
 id: string;

```

**src/shared/layout.ts**

```

 columns: LayoutColumn[];
}

export interface PageLayout {
 version: 1;
 rows: LayoutRow[];
}

/** The grid width. Kept here so client CSS and server clamping agree. */
export const GRID_UNITS = 12;

/* ----- *
 * Module catalog ? the palette shown in the editor and the whitelist the
 * server validates against. Adding a module type means adding it here AND
 * giving it a renderer in the client module registry.
 * ----- */

export interface ModuleSpec {
 type: ModuleType;
 label: string;
 description: string;
 /** Config seeded when this module is dropped onto a page. */
 defaultConfig: Record<string, unknown>;
}

export const MODULE_SPECS: readonly ModuleSpec[] = [
 {
 type: 'heading',
 label: 'Heading',
 description: 'A section title.',
 defaultConfig: { text: 'Heading', level: 2 },
 },
 {
 type: 'text',
 label: 'Rich text',
 description: 'A free-form block of formatted text.',
 defaultConfig: { html: '<p>Write something...</p>' },
 },
 {
 type: 'html',
 label: 'HTML block',
 description: 'Hand-written HTML ? tags are sanitized when the page renders.',
 defaultConfig: { html: '' },
 },
 {
 type: 'hero',
 label: 'Hero banner',
 description: 'A bold landing headline with call-to-action buttons, feature chips, and a value grid.',
 defaultConfig: {
 eyebrow: 'Muster your community',
 headline: 'Gather Your Fleet. Build Your Legacy.',
 subhead:
 'Stop managing your gaming organization with static spreadsheets and disconnected bots.'
 }
 }
]

```



**src/shared/layout.ts**

must unify your community with custom rank structures, real-time Discord role synchronization, automated event signups, service medal tracking, and custom web portals ? all in one place.',

```

 primaryLabel: '? Claim Your Org Space',
 primaryHref: '/api/auth/login',
 secondaryLabel: '? Add Discord Bot',
 secondaryHref: '',
 chips: [
 '? Real-Time Discord Role Sync',
 '?? Custom Medals & Service Records',
 '? Native Discord Event RSVPs',
 '? Fully Customizable Web Hubs',
],
 cards: [
 {
 icon: '?',
 title: 'Bi-Directional Discord Sync',
 tag: 'Never hand-assign a Discord role again.',
 body: 'Promote a member on your web portal, and their Discord roles, nicknames, and
channel permissions update instantly in real time.',
 },
 {
 icon: '??',
 title: 'Custom Ranks & Service Medals',
 tag: 'Reward loyalty and recognize achievements.',
 body: 'Build bespoke rank chains, award military-style service medals, and preserve
every member's historical record and service history.',
 },
 {
 icon: '?',
 title: 'Seamless Event Operations',
 tag: 'Schedule once, deploy everywhere.',
 body: 'Create raids, fleet ops, or meetings on the web. Members RSVP directly from
Discord or your web hub with zero friction.',
 },
 {
 icon: '?',
 title: 'Bespoke Community Web Hubs',
 tag: 'A real online home for your organization.',
 body: 'Design custom pages, embed videos, tailor your theme, and showcase a public
recruitment portal that stands out.',
 },
],
 },
},
{
 type: 'embed',
 label: 'Video embed',
 description: 'Embed a YouTube, Twitch, Vimeo, or Streamable video.',
 defaultConfig: { url: '', title: '', ratio: '16:9' },
},
{
 type: 'image',
 label: 'Image / banner',
 description: 'A standalone image, optionally linked.',

```

**src/shared/layout.ts**

```

 defaultConfig: { url: '', alt: '', href: '', caption: '' },
 },
 {
 type: 'button',
 label: 'Button / link',
 description: 'A call-to-action link.',
 defaultConfig: { label: 'Learn more', href: '/', style: 'primary' },
 },
 {
 type: 'divider',
 label: 'Divider',
 description: 'A horizontal rule to separate content.',
 defaultConfig: {},
 },
 {
 type: 'news',
 label: 'News feed',
 description: 'The latest news posts.',
 defaultConfig: { title: 'Latest News', limit: 5 },
 },
 {
 type: 'roster',
 label: 'Roster',
 description: 'A summary of the member roster.',
 defaultConfig: { title: 'Roster', limit: 12 },
 },
 {
 type: 'events',
 label: 'Events',
 description: 'Upcoming clan events.',
 defaultConfig: { title: 'Upcoming Events', limit: 5 },
 },
 {
 type: 'medals',
 label: 'Medals',
 description: 'The medals the clan awards.',
 defaultConfig: { title: 'Medals', limit: 12 },
 },
 {
 type: 'warrecords',
 label: 'War records',
 description: 'The clan's war-record honours.',
 defaultConfig: { title: 'War Records', limit: 12 },
 },
 {
 type: 'games',
 label: 'Games',
 description: 'The games the clan plays.',
 defaultConfig: { title: 'Games We Play', limit: 12 },
 },
];

export const MODULE_TYPES = MODULE_SPECS.map((m) => m.type);

```

**src/shared/layout.ts**

```

export function moduleSpec(type: ModuleType): ModuleSpec | undefined {
 return MODULE_SPECS.find((m) => m.type === type);
}

/* ----- *
 * Defaults ? what a page looks like before anyone customizes it. The home
 * page matches the example brief: news on the right, roster + events stacked
 * on the left.
 * ----- */

export const DEFAULT_LAYOUTS: Record<string, PageLayout> = {
 home: {
 version: 1,
 rows: [
 {
 id: 'r-home',
 columns: [
 {
 id: 'c-left',
 span: 5,
 modules: [
 { id: 'm-roster', type: 'roster', config: { title: 'Roster', limit: 12 } },
 { id: 'm-events', type: 'events', config: { title: 'Upcoming Events', limit: 5 } },
],
 },
 {
 id: 'c-right',
 span: 7,
 modules: [{ id: 'm-news', type: 'news', config: { title: 'Latest News', limit: 6 } }],
 },
],
 },
],
 },
};

/** The always-present built-in page, navigated to at `/.`. */
export const HOME_SLUG = 'home';

/**
 * Slugs a custom page may not claim: the built-in home plus every top-level app
 * route, so a custom page can never shadow /roster, /admin, the API, etc.
 */
export const RESERVED_PAGE_SLUGS = [
 'home',
 'news',
 'roster',
 'events',
 'members',
 'admin',
 'login',
 'api',
 'media',
 'p',

```

**src/shared/layout.ts**

```

];

/** A valid custom-page slug: url-safe, reasonable length, not reserved. */
export function isValidPageSlug(slug: string): boolean {
 return /^[a-z0-9][a-z0-9-]{1,40}$/.test(slug) && !RESERVED_PAGE_SLUGS.includes(slug);
}

/** Turn free-text (a page title) into a candidate slug. */
export function slugify(input: string): string {
 return input
 .toLowerCase()
 .trim()
 .replace(/^[a-z0-9]+/g, '-')
 .replace(/^-+|-+$/g, '')
 .slice(0, 40);
}

export function defaultLayout(slug: string): PageLayout {
 return DEFAULT_LAYOUTS[slug] ?? { version: 1, rows: [] };
}

/* ----- *
 * Sanitizer ? the server never trusts a stored/submitted layout. This coerces
 * arbitrary JSON into a valid PageLayout, dropping anything unknown, clamping
 * spans, and enforcing sane bounds so a hostile or corrupt payload can't blow
 * up the renderer. Rich-text html is left as-is here and sanitized separately
 * (it needs a DOM sanitizer); everything else is structural.
 * ----- */

const MAX_ROWS = 20;
const MAX_COLS = 6;
const MAX_MODULES = 20;

function asObject(v: unknown): Record<string, unknown> {
 return v && typeof v === 'object' && !Array.isArray(v) ? (v as Record<string, unknown>) : {};
}

function clampSpan(v: unknown): number {
 const n = Math.round(Number(v));
 if (!Number.isFinite(n)) return GRID_UNITS;
 return Math.min(GRID_UNITS, Math.max(1, n));
}

let idCounter = 0;
function fallbackId(prefix: string): string {
 idCounter += 1;
 return `${prefix}-${Date.now().toString(36)}-${idCounter}`;
}

function cleanId(v: unknown, prefix: string): string {
 return typeof v === 'string' && v.length > 0 && v.length <= 64 ? v : fallbackId(prefix);
}

/**

```

**src/shared/layout.ts**

```

* Accept only same-origin paths ("/...") and absolute http(s) URLs. Everything
* else ? javascript:, data:, mailto:, protocol-relative ? becomes empty, so a
* stored url/href can never smuggle a script into an or <a href>.
*/
function cleanUrl(v: unknown): string {
 if (typeof v !== 'string') return '';
 const t = v.trim();
 if (!t) return '';
 if (t.startsWith('/') && !t.startsWith('//')) return t.slice(0, 500);
 if (/^https?:\/\//i.test(t)) return t.slice(0, 500);
 return '';
}

/**
* Sanitize a module's config to a small, known set of primitive keys so nothing
* unexpected is persisted. `sanitizeText` (a DOM-aware html cleaner) is injected
* because it isn't available in the worker without a dependency; when omitted,
* html is stored verbatim (callers that can't sanitize should reject text edits
* or run their own pass).
*/
function cleanConfig(
 type: ModuleType,
 raw: unknown,
 sanitizeText?: (html: string) => string,
): Record<string, unknown> {
 const src = asObject(raw);
 const out: Record<string, unknown> = {};

 if (typeof src.title === 'string') out.title = src.title.slice(0, 120);
 if (src.limit !== null) {
 const n = Math.round(Number(src.limit));
 if (Number.isFinite(n)) out.limit = Math.min(50, Math.max(1, n));
 }

 if (type === 'heading') {
 out.text = (typeof src.text === 'string' ? src.text : 'Heading').slice(0, 200);
 const lvl = Math.round(Number(src.level));
 out.level = [1, 2, 3].includes(lvl) ? lvl : 2;
 }

 if (type === 'text') {
 const html = typeof src.html === 'string' ? src.html.slice(0, 20000) : '';
 out.html = sanitizeText ? sanitizeText(html) : html;
 }

 if (type === 'html') {
 // Stored verbatim (bounded); the html block is sanitized at render time with
 // its own wider allowlist (sanitizePageHtml) ? the DOM cleaner isn't available
 // here in the worker, and the renderer is the authoritative, always-run pass.
 out.html = typeof src.html === 'string' ? src.html.slice(0, 20000) : '';
 }

 if (type === 'hero') {
 const s = (v: unknown, max: number): string => (typeof v === 'string' ? v.slice(0, max) : '');

```

**src/shared/layout.ts**

```
 out.eyebrow = s(src.eyebrow, 120);
 out.headline = s(src.headline, 200);
 out.subhead = s(src.subhead, 600);
 out.primaryLabel = s(src.primaryLabel, 80);
 out.primaryHref = cleanUrl(src.primaryHref);
 out.secondaryLabel = s(src.secondaryLabel, 80);
 out.secondaryHref = cleanUrl(src.secondaryHref);
 out.chips = (Array.isArray(src.chips) ? src.chips : [])
 .slice(0, 8)
 .map((c) => s(c, 80))
 .filter(Boolean);
 out.cards = (Array.isArray(src.cards) ? src.cards : [])
 .slice(0, 8)
 .map((c) => {
 const card = asObject(c);
 return {
 icon: s(card.icon, 8),
 title: s(card.title, 80),
 tag: s(card.tag, 140),
 body: s(card.body, 400),
 };
 })
 .filter((c) => c.title || c.body);
 }

 if (type === 'embed') {
 // Never trust a stored/submitted iframe. Keep the pasted url for the editor,
 // but (re)derive the canonical, origin-locked embed src from it every time ?
 // resolveEmbed only ever emits an allowlisted origin + a strict id, so `src`
 // is safe to drop straight into an <iframe> at render.
 const url = typeof src.url === 'string' ? src.url.slice(0, 500) : '';
 const resolved = resolveEmbed(url);
 out.url = url;
 out.provider = resolved?.provider ?? '';
 out.src = resolved?.src ?? '';
 out.ratio = src.ratio === '4:3' ? '4:3' : '16:9';
 }

 if (type === 'image') {
 out.url = cleanUrl(src.url);
 out.href = cleanUrl(src.href);
 out.alt = (typeof src.alt === 'string' ? src.alt : '').slice(0, 200);
 out.caption = (typeof src.caption === 'string' ? src.caption : '').slice(0, 200);
 }

 if (type === 'button') {
 out.label = (typeof src.label === 'string' ? src.label : 'Learn more').slice(0, 80);
 out.href = cleanUrl(src.href);
 out.style = src.style === 'default' ? 'default' : 'primary';
 }

 return out;
}
```

**src/shared/layout.ts**

```

export function sanitizeLayout(raw: unknown, sanitizeText?: (html: string) => string): PageLayout
{
 const root = asObject(raw);
 const rowsIn = Array.isArray(root.rows) ? root.rows : [];
 const rows: LayoutRow[] = [];

 for (const rowRaw of rowsIn.slice(0, MAX_ROWS)) {
 const row = asObject(rowRaw);
 const colsIn = Array.isArray(row.columns) ? row.columns : [];
 const columns: LayoutColumn[] = [];

 for (const colRaw of colsIn.slice(0, MAX_COLS)) {
 const col = asObject(colRaw);
 const modsIn = Array.isArray(col.modules) ? col.modules : [];
 const modules: LayoutModule[] = [];

 for (const modRaw of modsIn.slice(0, MAX_MODULES)) {
 const mod = asObject(modRaw);
 const type = mod.type as ModuleType;
 if (!MODULE_TYPES.includes(type)) continue;
 const cleaned: LayoutModule = {
 id: cleanId(mod.id, 'm'),
 type,
 config: cleanConfig(type, mod.config, sanitizeText),
 };
 // Only a positive integer role id becomes an audience gate; anything else
 // is dropped (so a stale/invalid gate fails open to "everyone", not hidden).
 const roleId = Number(mod.visibleToRole);
 if (Number.isInteger(roleId) && roleId > 0) {
 cleaned.visibleToRole = roleId;
 }
 modules.push(cleaned);
 }

 columns.push({ id: cleanId(col.id, 'c'), span: clampSpan(col.span), modules });
 }

 if (columns.length) rows.push({ id: cleanId(row.id, 'r'), columns });
 }

 return { version: 1, rows };
}

```

**src/shared/names.ts**

```
/**
 * The one place that decides what name to show for a member.
 *
 * Order: the member's self-set display (game/clan) name, then their Discord
 * global name, then their Discord username as a last resort. Used on both the
 * server (slash commands) and the client so a name never renders two ways.
 */
export function memberName(u: {
 displayName?: string | null;
 globalName?: string | null;
 username: string;
}): string {
 return u.displayName?.trim() || u.globalName?.trim() || u.username;
}
```



**src/shared/nav.ts**

```

/**
 * The site navigation model ? the composable top menu.
 *
 * The menu used to be hardcoded (Home/News/Roster/Events + custom pages + Admin).
 * Here it is a small ordered tree an admin arranges: each entry is a link (to
 * a built-in destination, a custom page, or any URL) or a category (a
 * dropdown holding links). One level of nesting ? categories hold links, not
 * other categories ? which is all a top bar needs and keeps the editor sane.
 *
 * Two independent gates decide whether a viewer sees an entry:
 * - a built-in destination carries a permission (Events needs `events.view`,
 * Admin needs any admin tool). This is NON-bypassable ? a nav entry can never
 * grant access it shouldn't, it only shows a door the viewer may open.
 * - an optional `visibleToRole` the admin sets, mirroring page-module visibility.
 * Both are enforced where the nav renders; the stored tree holds every entry.
 *
 * Stored in settings['nav']; sanitised on every read and write (this module is
 * the one authority on the shape) so a hand-edited or stale blob can never reach
 * the renderer malformed. When unset, `defaultNavConfig` reproduces the classic
 * menu, so an install that never touches the builder keeps its old nav.
 */

import type { Permission } from './permissions';

export interface BuiltinTarget {
 key: string;
 path: string;
 label: string;
 /** Required permission to see the entry, if any. */
 permission?: Permission;
 /** True -> gated on "can reach any admin tool" rather than a single permission. */
 admin?: boolean;
}

/** The fixed destinations the app always ships. `key` is what a link stores. */
export const BUILTIN_TARGETS: Record<string, BuiltinTarget> = {
 home: { key: 'home', path: '/', label: 'Home' },
 news: { key: 'news', path: '/news', label: 'News' },
 roster: { key: 'roster', path: '/roster', label: 'Roster' },
 events: { key: 'events', path: '/events', label: 'Events', permission: 'events.view' },
 admin: { key: 'admin', path: '/admin', label: 'Admin', admin: true },
};

export const BUILTIN_KEYS = Object.keys(BUILTIN_TARGETS);

export type NavKind = 'builtin' | 'page' | 'url';

export interface NavItem {
 id: string;
 type: 'link' | 'group';
 /** Display text. Falls back to the target's own label when blank. */
 label: string;
 /** Link only: what kind of destination `target` names. */
 kind?: NavKind;
}

```

**src/shared/nav.ts**

```

 /** Link only: a builtin key, a page slug, or a URL. */
 target?: string;
 /** Optional role gate ? the entry hides for viewers who lack this role (god bypasses). */
 visibleToRole?: number;
 /** Category (group) only: its child links, in order. */
 children?: NavItem[];
}

export interface NavConfig {
 version: 1;
 items: NavItem[];
}

const MAX_TOP = 40;
const MAX_CHILDREN = 30;
const ID_RE = /^[a-zA-Z0-9_-]{1,40}$/;
const SLUG_RE = /^[a-z0-9]+(?:-[a-z0-9]+)*$/;

let idSeq = 0;
/** A stable-ish id for a fresh item. Collisions are resolved during sanitise. */
export function newNavId(prefix = 'n'): string {
 if (typeof crypto !== 'undefined' && 'randomUUID' in crypto) {
 return `${prefix}-${crypto.randomUUID().slice(0, 8)}`;
 }
 idSeq += 1;
 return `${prefix}-${Date.now().toString(36)}${idSeq}`;
}

/** The href a link resolves to, or null for a category (which has no destination). */
export function navItemHref(item: NavItem): string | null {
 if (item.type !== 'link' || !item.kind || !item.target) return null;
 if (item.kind === 'builtin') return BUILTIN_TARGETS[item.target]?.path ?? null;
 if (item.kind === 'page') return `/p/${item.target}`;
 return item.target; // url ? already validated to be same-origin or http(s)
}

/** The effective label for an entry, falling back to its target's name. */
export function navItemLabel(item: NavItem): string {
 if (item.label.trim()) return item.label.trim();
 if (item.kind === 'builtin' && item.target) return BUILTIN_TARGETS[item.target]?.label ??
'Link';
 if (item.kind === 'page' && item.target) return item.target;
 return item.type === 'group' ? 'Category' : 'Link';
}

/** True for a URL we allow: same-origin ("/...", not "///...") or absolute http(s). */
function safeUrl(v: string): boolean {
 if (v.startsWith('/') && !v.startsWith('///')) return true;
 return /^https?:\/\//i.test(v);
}

function str(v: unknown): string {
 return typeof v === 'string' ? v : '';
}

```

**src/shared/nav.ts**

```

/**
 * Structurally sanitise an untrusted nav blob into a valid config. Invalid
 * entries are dropped rather than throwing, so a partially-bad blob still yields
 * a usable menu. `validSlugs`, when given, prunes page links whose page no
 * longer exists (e.g. the page was deleted after being added to the menu).
 */
export function sanitizeNav(raw: unknown, opts: { validSlugs?: Set<string> } = {}): NavConfig {
 const seen = new Set<string>();
 const rawItems = Array.isArray((raw as { items?: unknown })?.items)
 ? ((raw as { items: unknown[] }).items)
 : [];

 const takeId = (v: unknown): string => {
 let id = str(v);
 if (!ID_RE.test(id) || seen.has(id)) id = newNavId();
 seen.add(id);
 return id;
 };

 const sanitizeLink = (rawItem: unknown): NavItem | null => {
 const o = (rawItem ?? {}) as Record<string, unknown>;
 const kind = o.kind === 'builtin' || o.kind === 'page' || o.kind === 'url' ? o.kind : null;
 const target = str(o.target).trim();
 if (!kind || !target) return null;

 if (kind === 'builtin' && !BUILTIN_KEYS.includes(target)) return null;
 if (kind === 'page') {
 if (!SLUG_RE.test(target)) return null;
 if (opts.validSlugs && !opts.validSlugs.has(target)) return null;
 }
 if (kind === 'url' && !safeUrl(target)) return null;

 const item: NavItem = {
 id: takeId(o.id),
 type: 'link',
 label: str(o.label).trim().slice(0, 40),
 kind,
 target: target.slice(0, 300),
 };
 const role = Number(o.visibleToRole);
 if (Number.isInteger(role) && role > 0) item.visibleToRole = role;
 return item;
 };

 const items: NavItem[] = [];
 for (const rawItem of rawItems.slice(0, MAX_TOP)) {
 const o = (rawItem ?? {}) as Record<string, unknown>;
 if (o.type === 'group') {
 const kids: NavItem[] = [];
 const rawKids = Array.isArray(o.children) ? o.children.slice(0, MAX_CHILDREN) : [];
 for (const k of rawKids) {
 const link = sanitizeLink(k);
 if (link) kids.push(link);
 }
 }
 }

```

**src/shared/nav.ts**

```

 }
 const group: NavItem = {
 id: takeId(o.id),
 type: 'group',
 label: str(o.label).trim().slice(0, 40) || 'Category',
 children: kids,
 };
 const role = Number(o.visibleToRole);
 if (Number.isInteger(role) && role > 0) group.visibleToRole = role;
 items.push(group);
 } else {
 const link = sanitizeLink(rawItem);
 if (link) items.push(link);
 }
}

return { version: 1, items };
}

/**
 * The classic menu, reproduced from the built-ins plus the custom pages that
 * opted into the nav ? the fallback when settings['nav'] is unset, so upgrading
 * to the builder preserves whatever an install already had.
 */
export function defaultNavConfig(navPages: { slug: string; title: string | null }[]): NavConfig {
 const items: NavItem[] = [
 { id: newNavId(), type: 'link', label: '', kind: 'builtin', target: 'home' },
 { id: newNavId(), type: 'link', label: '', kind: 'builtin', target: 'news' },
 { id: newNavId(), type: 'link', label: '', kind: 'builtin', target: 'roster' },
 { id: newNavId(), type: 'link', label: '', kind: 'builtin', target: 'events' },
 ...navPages.map((p) => ({
 id: newNavId(),
 type: 'link' as const,
 label: (p.title ?? p.slug).slice(0, 40),
 kind: 'page' as const,
 target: p.slug,
 })),
 { id: newNavId(), type: 'link', label: '', kind: 'builtin', target: 'admin' },
];
 return { version: 1, items };
}

```

**src/shared/orgchart.ts**

```

/**
 * The leadership org-chart contract, shared by the server (stores + sanitizes)
 * and the client (renders + edits).
 *
 * The chart is portable JSON: a rank threshold, a set of NODES (a member id at an
 * x/y position) and EDGES (who reports to whom, member id -> member id). Members
 * are referenced by id only ? their display name, rank and avatar are resolved
 * live at render time, so a rename or promotion updates the chart automatically,
 * and a member who drops below the threshold (or leaves) simply stops appearing.
 * The same JSON drives the web view now and a native app later.
 */

export interface OrgNode {
 memberId: number;
 x: number;
 y: number;
}

export interface OrgEdge {
 /** Manager. */
 from: number;
 /** Direct report. */
 to: number;
}

export interface OrgChart {
 version: 1;
 /**
 * Only members whose rank sort-order is >= this appear on the public tree.
 * 0 = everyone eligible. (Higher sort-order = higher rank; see outranks().)
 */
 minRankSortOrder: number;
 nodes: OrgNode[];
 edges: OrgEdge[];
}

export const EMPTY_ORG_CHART: OrgChart = { version: 1, minRankSortOrder: 0, nodes: [], edges: [] };

const MAX_NODES = 400;
const MAX_EDGES = 800;
const MAX_COORD = 8000;

function asObject(v: unknown): Record<string, unknown> {
 return v && typeof v === 'object' && !Array.isArray(v) ? (v as Record<string, unknown>) : {};
}

function clampCoord(v: unknown): number {
 const n = Math.round(Number(v));
 if (!Number.isFinite(n)) return 0;
 return Math.min(MAX_COORD, Math.max(0, n));
}

/**

```

**src/shared/orgchart.ts**

```
* Coerce arbitrary JSON into a valid OrgChart: integer member ids, clamped
* coordinates, no duplicate nodes, and edges only between nodes that exist (so a
* stale or hostile payload can't dangle an edge or blow up the renderer).
*/
export function sanitizeOrgChart(raw: unknown): OrgChart {
 const root = asObject(raw);

 const minRank = Math.round(Number(root.minRankSortOrder));
 const nodesIn = Array.isArray(root.nodes) ? root.nodes : [];
 const edgesIn = Array.isArray(root.edges) ? root.edges : [];

 const nodes: OrgNode[] = [];
 const ids = new Set<number>();
 for (const n of nodesIn.slice(0, MAX_NODES)) {
 const o = asObject(n);
 const memberId = Math.round(Number(o.memberId));
 if (!Number.isInteger(memberId) || memberId <= 0 || ids.has(memberId)) continue;
 ids.add(memberId);
 nodes.push({ memberId, x: clampCoord(o.x), y: clampCoord(o.y) });
 }

 const edges: OrgEdge[] = [];
 const seenEdge = new Set<string>();
 for (const e of edgesIn.slice(0, MAX_EDGES)) {
 const o = asObject(e);
 const from = Math.round(Number(o.from));
 const to = Math.round(Number(o.to));
 if (from === to || !ids.has(from) || !ids.has(to)) continue;
 const key = `${from}->${to}`;
 if (seenEdge.has(key)) continue;
 seenEdge.add(key);
 edges.push({ from, to });
 }

 return {
 version: 1,
 minRankSortOrder: Number.isFinite(minRank) ? Math.max(0, minRank) : 0,
 nodes,
 edges,
 };
}
```

**src/shared/permissions.ts**

```
/**
 * The permission vocabulary.
 *
 * The 2003 version gated actions on `member.rank >= auth.<action>` ? a single
 * linear ladder. That made "trusted member who isn't an officer" impossible to
 * express. Here, rank is prestige and roles carry capability; they move
 * independently.
 */

export const PERMISSIONS = {
 'roster.view': 'See the member list (roster page + home page)',
 'roster.edit': 'Edit member details and profiles',
 'roster.promote': 'Change a member's rank',
 'roster.remove': 'Remove or retire a member',

 'roles.assign': 'Grant and revoke roles on members',
 'roles.manage': 'Create, edit, and delete roles and their permissions',
 'ranks.manage': 'Create, edit, reorder, and delete ranks',

 'news.create': 'Write news posts',
 'news.publish': 'Publish and unpublish news posts',
 'news.delete': 'Delete news posts',

 'medals.award': 'Award and revoke medals',
 'medals.manage': 'Create, edit, and delete medal definitions',

 'games.manage': 'Create, edit, and delete games',
 'matches.record': 'Record match results',
 'matches.manage': 'Edit and delete any match record',

 'warrecords.manage': 'Create, edit, and delete war records',
 'warrecords.award': 'Award and revoke war records',

 'events.view': 'See the events page',
 'events.manage': 'Create, edit, and cancel events',
 'events.attendees': 'See who has signed up for an event',

 'theme.manage': 'Edit site theme and appearance',
 'pages.manage': 'Arrange page layouts and modules',
 'settings.manage': 'Edit site settings',

 'hangar.view': 'View other members' Star Citizen hangars (when they've shared them)',
 'hangar.value': 'See the monetary value of members' hangars (hidden from others)',

 'audit.view': 'View the audit log',
 'discord.sync': 'Trigger Discord role synchronization',
} as const;

export type Permission = keyof typeof PERMISSIONS;

export const ALL_PERMISSIONS = Object.keys(PERMISSIONS) as Permission[];

export function isPermission(value: string): value is Permission {
 return value in PERMISSIONS;
}
```

**src/shared/permissions.ts**

```
}

/** What the session endpoint hands to the React app. */
export interface Viewer {
 id: number;
 discordId: string;
 username: string;
 globalName: string | null;
 displayName: string | null;
 avatar: string | null;
 profileImageUrl: string | null;
 isGod: boolean;
 rank: { id: number; name: string; sortOrder: number } | null;
 roles: { id: number; name: string; color: string | null }[];
 permissions: Permission[];
}

/**
 * The single authority on "can this person do this".
 *
 * God status bypasses everything by design ? it is the recovery hatch that
 * prevents anyone, including the founder, from locking themselves out of
 * their own site. Grant it sparingly; it is not assignable through the UI.
 */
export function can(viewer: Viewer | null, permission: Permission): boolean {
 if (!viewer) return false;
 if (viewer.isGod) return true;
 return viewer.permissions.includes(permission);
}

export function canAll(viewer: Viewer | null, ...permissions: Permission[]): boolean {
 return permissions.every((p) => can(viewer, p));
}

export function canAny(viewer: Viewer | null, ...permissions: Permission[]): boolean {
 return permissions.some((p) => can(viewer, p));
}

/**
 * Rank-ladder guard, kept separate from permissions on purpose.
 *
 * Even with `roster.promote`, you should not be able to promote someone above
 * yourself or demote a peer. God ignores this.
 */
export function outranks(
 actor: Pick<Viewer, 'isGod' | 'rank'>,
 targetRankOrder: number | null,
): boolean {
 if (actor.isGod) return true;
 if (actor.rank == null) return false;
 if (targetRankOrder == null) return true;
 return actor.rank.sortOrder > targetRankOrder;
}
```



**tsconfig.json**

```
{
 "compilerOptions": {
 "target": "ES2022",
 "lib": ["ES2022", "DOM", "DOM.Iterable"],
 "module": "ESNext",
 "moduleResolution": "bundler",
 "types": ["@cloudflare/workers-types", "vite/client"],

 "jsx": "react-jsx",

 "strict": true,
 "noUnusedLocals": true,
 "noUnusedParameters": true,
 "noFallthroughCasesInSwitch": true,
 "noUncheckedIndexedAccess": true,

 "esModuleInterop": true,
 "skipLibCheck": true,
 "resolveJsonModule": true,
 "isolatedModules": true,
 "verbatimModuleSyntax": true,
 "noEmit": true
 },
 // scripts/ is excluded on purpose: it runs under Node via tsx, and pulling
 // @types/node into the same program as @cloudflare/workers-types produces
 // conflicting globals.
 "include": ["src", "vite.config.ts", "drizzle.config.ts"]
}
```

**vite.config.ts**

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
 plugins: [react()],
 root: 'src/client',
 build: {
 // wrangler.jsonc serves this directory as the Worker's static assets.
 outDir: '../../../dist/client',
 emptyOutDir: true,
 sourcemap: true,
 },
 server: {
 port: 5173,
 proxy: {
 // `npm run dev` runs Vite and `wrangler dev` side by side; the API lives
 // on the Worker at 8787 and the React app calls it through this proxy.
 '/api': {
 target: 'http://127.0.0.1:8787',
 changeOrigin: true,
 },
 },
 },
});
```